
IFM 2.1: Programmieren 2 (Sommer 2026)

Carsten Gips (HSBI)

9. Juli 2026

Inhaltsverzeichnis

I. Syllabus	1
1. Syllabus	2
1.1. Kursbeschreibung	2
1.2. Überblick Modulinhalte	2
1.3. Team	3
1.4. Kursformat	3
1.5. Sprechstunde	3
1.6. Fahrplan	3
1.7. Prüfungsform, Note und Credits	4
1.8. Materialien	5
1.8.1. Literatur	5
1.8.2. Tools	5
1.9. Förderungen und Kooperationen	5
1.9.1. Förderung durch DH.NRW (Digi Fellowships)	5
1.9.2. Kooperation mit dem DigikoS-Projekt	5
1.10. LICENSE	6
II. Vorlesungsunterlagen	7
2. Versionierung mit Git	8
2.1. Git1: Basics der Versionsverwaltung mit Git	8
2.1.1. Prinzip Versionsverwaltung	9
2.1.2. Varianten: Zentrale Versionsverwaltung (Beispiel SVN)	11
2.1.3. Varianten: Verteilte Versionsverwaltung (Beispiel Git)	11
2.1.4. Versionsverwaltung mit Git: Typische Arbeitsschritte	12
2.1.5. (Globale) Konfiguration	12
2.1.6. Neues Repo anlegen	13
2.1.7. Dateien unter Versionskontrolle stellen	13
2.1.8. Änderungen einpflegen	14
2.1.9. Letzten Commit ergänzen	14
2.1.10. Weitere Datei-Operationen: hinzufügen, umbenennen, löschen	15
2.1.11. Commits suchen und betrachten	15
2.1.12. Änderungen und Logs betrachten	16
2.1.13. Dateien ignorieren: <i>.gitignore</i>	17

2.1.14. Zeilenenden: Windows vs. der Rest der Welt	17
2.1.15. Zeitmaschine	18
2.1.16. Wann und wie committen?	18
2.1.17. Schreiben von Commit-Messages: WARUM?!	19
2.1.18. Ausflug “Conventional Commits”	21
2.1.19. Wrap-Up	23
2.2. Git2: Branches und Remotes	24
2.2.1. Neues Feature entwickeln/ausprobieren	25
2.2.2. Problem: Fehler im ausgelieferten Produkt	26
2.2.3. Fix ist stabil: Integration in <i>master</i>	27
2.2.4. Feature weiter entwickeln und Integration in <i>master</i>	27
2.2.5. Konflikte beim Mergen	28
2.2.6. Merge-Konflikte auflösen	29
2.2.7. Rebasen: Verschieben von Branches	29
2.2.8. Don’t lose your HEAD	30
2.2.9. Erinnerung: Clonen kann sich lohnen => Remote-Branches	30
2.2.10. Eigener und entfernter <i>master</i> entwickeln sich weiter	31
2.2.11. Änderungen vom Remote holen und Branches lokal zusammenführen	32
2.2.12. Branches und Remotes	33
2.2.13. Wrap-Up	34
2.3. Git4: Worktree	36
2.3.1. Git Worktree - Mehrere Branches parallel auschecken	37
2.3.2. How to use Git Worktree	38
2.3.3. Worktree anlegen	38
2.3.4. Worktree wechseln	39
2.3.5. Worktree löschen	39
2.3.6. Wrap-Up	39
2.4. Branching-Strategien & Git-Workflows	40
2.4.1. Nutzung von Git in Projekten: Verteiltes Git (und Workflows)	41
2.4.2. Umgang mit Branches: Themen-Branches	41
2.4.3. Umgang mit Branches: Langlaufende Branches	42
2.4.4. Komplexe Branching-Strategie: Git-Flow	43
2.4.5. Vereinfachte Branching-Strategie: GitHub Flow	45
2.4.6. Diskussion: Git-Flow vs. GitHub Flow	45
2.4.7. Zusammenarbeit: Zentraler Workflow mit Git (analog zu SVN)	46
2.4.8. Zusammenarbeit: Einfacher verteilter Workflow mit Git	47
2.4.9. Feature-Branches aktualisieren: Mergen mit <i>master</i> vs. Rebase auf <i>master</i>	48
2.4.10. Kommunikation: Merge- bzw. Pull-Requests	50
2.4.11. Best Practices bei Merge-/Pull-Requests	51
2.4.12. Wrap-Up	52
3. Bauen von Programmen, Automatisierung, Continuous Integration	54
3.1. Build-Systeme: Gradle	54
3.1.1. Automatisieren von Arbeitsabläufen	54

3.1.2.	Gradle: Eine DSL in Groovy	55
3.1.3.	Gradle-DSL	56
3.1.4.	Konfigurationsdateien	56
3.1.5.	Neues Gradle-Projekt mit Gradle Init anlegen	56
3.1.6.	Gradle und IntelliJ	57
3.1.7.	Ordner in einem Gradle-Projekt	60
3.1.8.	Ablauf eines Gradle-Builds	62
3.1.9.	Plugin-Architektur	62
3.1.10.	Auflösen von Abhängigkeiten	62
3.1.11.	Beispiel mit weiteren Konfigurationen (u.a. Checkstyle und Javadoc)	63
3.1.12.	Gradle und Ant (und Maven)	64
3.1.13.	Gradle-Wrapper	64
3.1.14.	Ausblick: Maven	65
3.1.15.	Wrap-Up	68
3.2.	Java: Strukturieren mit Packages	69
3.2.1.	Typisches Java-Projekt	69
3.2.2.	Listen aus dem JDK	69
3.2.3.	Was ist ein Package in Java	70
3.2.4.	Aufgaben von Packages in Java	70
3.2.5.	Konventionen	71
3.2.6.	Packages strukturieren	71
3.2.7.	Praktischer Einsatz von Packages	72
3.2.8.	Einbahnstraße Default-Package	73
3.2.9.	Wrap-Up	74
3.3.	Continuous Integration (CI)	75
3.3.1.	Motivation: Zusammenarbeit in Teams	75
3.3.2.	Continuous Integration (CI)	76
3.3.3.	GitLab CI/CD	77
3.3.4.	GitHub Actions	80
3.3.5.	Wrap-Up	86
3.4.	Einführung in ANTLR	87
3.4.1.	Motivation & Einordnung	88
3.4.2.	ANTLR als Blackbox-Pipeline	89
3.4.3.	Technische Einbindung (Gradle, Projektstruktur)	89
3.4.4.	Beispielgrammatik	91
3.4.5.	Minimaler Java-Code: Zerlegen der Eingabe in Token	92
3.4.6.	Minimaler Java-Code: Text -> Baum	93
3.4.7.	Der Parse-Tree: Struktur	94
3.4.8.	Der Parse-Tree: Klassenhierarchie	95
3.4.9.	Traversierung mit Visitor-Pattern	97
3.4.10.	Ausblick: Pattern Matching auf Bäumen (neuere Java-Versionen)	101
3.4.11.	Syntaxhighlighting: Vergleich Regex-Ansatz vs. ANTLR-Ansatz	101
3.4.12.	Ausblick auf das 3. Semester (Compilerbau)	101

3.4.13. Wrap-Up	102
3.5. Debugging	103
3.5.1. Software hat (immer) Fehler	104
3.5.2. Warum Debuggen?	104
3.5.3. Die wichtigsten Debugger-Werkzeuge	105
3.5.4. Standard-Vorgehen beim Debuggen	106
3.5.5. Demo 1: Exception beim Median (Crash)	106
3.5.6. Demo 2: Logikfehler bei der Summe (falsches Ergebnis)	108
3.5.7. Tipps zum selbstständigen Üben	109
3.5.8. Wrap-Up	109
3.6. Logging	110
3.6.1. Wie prüfen Sie die Werte von Variablen/Objekten?	111
3.6.2. Java Logging API - Überblick	111
3.6.3. Erzeugen neuer Logger	112
3.6.4. Ausgabe von Logmeldungen	112
3.6.5. Wichtigkeit von Logmeldungen: Stufen	113
3.6.6. Jemand muss die Arbeit machen ...	113
3.6.7. Ich ... bin ... Dein ... Vater ...	113
3.6.8. Ausgabe von Logmeldungen - revisited	114
3.6.9. Best Practices	115
3.6.10. Wrap-Up	115
3.7. Einführung in Docker	116
3.7.1. Motivation CI/CD: WFM (<i>Works For Me</i>)	118
3.7.2. Virtualisierung: Container vs. VM	119
3.7.3. Getting started	120
3.7.4. Images selbst definieren	122
3.7.5. CI-Pipeline (GitLab)	123
3.7.6. CI-Pipeline (GitHub)	123
3.7.7. Bonus/Ausblick: VSCode und DevContainers (früher "Remote - Containers")	124
3.7.8. Wrap-Up	126
4. Testen mit JUnit und Mockito	128
4.1. Testen mit JUnit (JUnit-Basics)	128
4.1.1. Software-Fehler und ihre Folgen	129
4.1.2. Zentrale Begriffe: Was wann testen?	130
4.1.3. JUnit: Test-Framework für Java	131
4.1.4. Einbinden von JUnit (Gradle)	132
4.1.5. Anlegen und Organisation der Tests mit JUnit	132
4.1.6. Definition von Testmethoden	134
4.1.7. Ergebnis prüfen	135
4.1.8. Mögliche Testausgänge bei JUnit: rot/grün/ignoriert	135
4.1.9. Anmerkungen zu den Testmethoden	136
4.1.10. BDD: "Given - When - Then"-Mantra	138
4.1.11. Test-Fixtures: Testübergreifende Konfiguration	139

4.1.12. Test von Exceptions	141
4.1.13. Parametrisierte Tests	142
4.1.14. Best Practices	143
4.1.15. Wrap-Up	144
4.2. JUnit2 Testfallermittlung: Wie viel und was muss man testen?	148
4.2.1. Hands-On (10 Minuten): Wieviel und was muss man testen?	148
4.2.2. Äquivalenzklassenbildung	149
4.2.3. ÄK: Erstellung der Testfälle	151
4.2.4. ÄK: Beispiel: Eingabewert x soll zw. 10 und 100 liegen	151
4.2.5. Grenzwertanalyse	152
4.2.6. GW: Beispiel: Eingabewert x soll zw. 10 und 100 liegen	152
4.2.7. Anmerkung: Analyse abhängiger Parameter	153
4.2.8. Wrap-Up	153
4.3. JUnit3: Mocking mit Mockito	156
4.3.1. Motivation: Entwicklung einer Studi-/Prüfungsverwaltung	156
4.3.2. Manuell Stubs implementieren	160
4.3.3. Mockito: Mocking von ganzen Klassen	160
4.3.4. Mockito: Spy = Wrapper um ein Objekt	161
4.3.5. Wurde eine Methode aufgerufen?	162
4.3.6. Fangen von Argumenten	163
4.3.7. Ausblick: PowerMock	164
4.3.8. Ausführlicheres Beispiel: WuppiWarenlager	164
4.3.9. Mockito und Annotationen	166
4.3.10. Wrap-Up	168
4.4. JUnit4: Property based Testing & Approval Testing	170
4.4.1. Motivation: Mehr als nur Beispieltests	170
4.4.2. Approval Testing: Idee und Einsatzgebiete	170
4.4.3. Beispiel: Order-Report mit manuellen Approval-Tests	171
4.4.4. Beispiel: Order-Report mit Bibliothek "ApprovalTests"	173
4.4.5. Approval Testing & Git	174
4.4.6. Property-Based Testing: Grundidee	174
4.4.7. Anknüpfung: Äquivalenzklassenanalyse	175
4.4.8. Durchgängiges Beispiel: Steuerfunktion - Spezifikation	175
4.4.9. Steuerfunktion: Äquivalenzklassen & Grenzwerte	176
4.4.10. Klassische JUnit-Tests: Beispiele aus den Klassen	176
4.4.11. jqwik: Einfache Property - Steuer nie negativ	177
4.4.12. jqwik: Monotonie-Eigenschaft für die Steuer	178
4.4.13. jqwik: Verbindung zu Äquivalenzklassen (Bracket-Property)	178
4.4.14. Wrap-Up	179
5. Modern Java: Funktionaler Stil und Stream-API	181
5.1. Lambda-Ausdrücke und funktionale Interfaces	181
5.1.1. Problem: Sortieren einer Studi-Liste	182
5.1.2. Erinnerung: Verschachtelte Klassen ("Nested Classes")	182

5.1.3.	Lösung: Comparator als anonyme innere Klasse	183
5.1.4.	Vereinfachung mit Lambda-Ausdruck	184
5.1.5.	Syntax für Lambdas	184
5.1.6.	Quiz: Welches sind keine gültigen Lambda-Ausdrücke?	185
5.1.7.	Definition “Funktionales Interface” (“ <i>functional interfaces</i> ”)	185
5.1.8.	Quiz: Welches ist kein funktionales Interface?	186
5.1.9.	Lambdas und funktionale Interfaces: Typprüfung	186
5.1.10.	Wrap-Up	187
5.2.	Methoden-Referenzen	189
5.2.1.	Beispiel: Sortierung einer Liste	189
5.2.2.	Überblick: Arten von Methoden-Referenzen	190
5.2.3.	Methoden-Referenz 1: Referenz auf statische Methode	191
5.2.4.	Methoden-Referenz 2: Referenz auf Instanz-Methode (Objekt)	191
5.2.5.	Methoden-Referenz 3: Referenz auf Instanz-Methode (Typ)	192
5.2.6.	Ausblick: Threads	192
5.2.7.	Ausblick: Datenstrukturen als Streams	193
5.2.8.	Wrap-Up	194
5.3.	Interfaces: Default-Methoden	195
5.3.1.	Problem: Etablierte API (Interfaces) erweitern	195
5.3.2.	Default-Methoden: Interfaces mit Implementierung	196
5.3.3.	Problem: Mehrfachvererbung	196
5.3.4.	Auflösung Mehrfachvererbung: 1. Klassen gewinnen	196
5.3.5.	Auflösung Mehrfachvererbung: 2. Sub-Interfaces gewinnen	197
5.3.6.	Auflösung Mehrfachvererbung: 3. Methode explizit auswählen	197
5.3.7.	Quiz: Was kommt hier raus?	198
5.3.8.	Statische Methoden in Interfaces	198
5.3.9.	Interfaces vs. Abstrakte Klassen	199
5.3.10.	Wrap-Up	199
5.4.	Record-Klassen	200
5.4.1.	Motivation; Klasse Studi	201
5.4.2.	Klasse Studi als Record	201
5.4.3.	Eigenschaften und Einschränkungen von Record-Klassen	202
5.4.4.	Records: Prüfungen im Konstruktor	202
5.4.5.	Getter und Methoden	203
5.4.6.	Weitere Eigenschaften von Records	203
5.4.7.	Beispiel aus den Challenges	204
5.4.8.	Wrap-Up	205
5.5.	Sealed-Typen & Pattern Matching	207
5.5.1.	Was nervt Sie an diesem Code?	207
5.5.2.	Pattern Matching für <code>instanceof</code>	208
5.5.3.	Switch Expressions	208
5.5.4.	Type Patterns im <code>switch</code>	210
5.5.5.	Guarded Patterns im <code>switch</code>	211

5.5.6.	Record-Patterns / Dekonstruktion	212
5.5.7.	Verbindung zu “funktionalem Stil”	213
5.5.8.	Wrap-Up	214
5.6.	Stream-API	215
5.6.1.	Motivation	215
5.6.2.	Innere Schleife mit Streams umgeschrieben	216
5.6.3.	Erzeugen von Streams	217
5.6.4.	Intermediäre Operationen auf Streams	218
5.6.5.	Was tun, wenn eine Methode Streams zurückliefert	219
5.6.6.	Streams abschließen: Terminale Operationen	220
5.6.7.	Spielregeln	221
5.6.8.	Wrap-Up	221
5.7.	Fehlerbehandlung mit Optional und Result	223
5.7.1.	<code>Optional<T></code> - “Vielleicht gibt es einen Wert”	224
5.7.2.	Erzeugen von <i>Optional</i> -Objekten	228
5.7.3.	LSF liefert jetzt <i>Optional</i> zurück	229
5.7.4.	Zugriff auf <i>Optional</i> -Objekte	229
5.7.5.	Einsatz mit Stream-API	230
5.7.6.	Weitere <i>Optionals</i>	231
5.7.7.	Regeln für <i>Optional</i>	231
5.7.8.	Was ist das Problem mit Exceptions?	232
5.7.9.	<code>Result<E, R></code> : Fehler als Datentyp modellieren	235
5.7.10.	Umbau unseres Beispiels auf <code>Result<E, R></code>	236
5.7.11.	Handling auf Aufrufer-Seite	236
5.7.12.	Diskussion: Exceptions vs. Result/Optional	242
5.7.13.	Wrap-Up	244
6.	Entwurfsmuster	247
6.1.	Observer-Pattern	247
6.1.1.	Gedankenexperiment: Verteilung der Prüfungsergebnisse	248
6.1.2.	Elegantere Lösung: Observer-Entwurfsmuster	249
6.1.3.	Observer-Pattern verallgemeinert	249
6.1.4.	Wrap-Up	252
6.2.	Template-Method-Pattern	253
6.2.1.	Motivation: Syntax-Highlighting im Tokenizer	254
6.2.2.	Token-Klassen mit formatiertem Inhalt	254
6.2.3.	Don’t call us, we’ll call you	255
6.2.4.	Template-Method-Pattern	256
6.2.5.	Wrap-Up	257
6.3.	Visitor-Pattern	258
6.3.1.	Motivation: Parsen von “5*4+3”	259
6.3.2.	Erinnerung: Baumstrukturen	260
6.3.3.	Erinnerung: Varianten der Datenhaltung	261
6.3.4.	Erinnerung: Implementierungsvarianten	261

6.3.5.	Erinnerung: Algorithmische Varianten bei Bäumen	262
6.3.6.	Strukturen für den Parse-Tree	263
6.3.7.	Ergänzung I: Ausrechnen des Ausdrucks	264
6.3.8.	Ergänzung II: Pretty-Print des Ausdrucks	265
6.3.9.	Visitor-Pattern (Besucher-Entwurfsmuster)	266
6.3.10.	Ausrechnen des Ausdrucks mit einem Visitor	275
6.3.11.	Diskussion	275
6.3.12.	Ausblick: Pattern Matching auf Bäumen (neuere Java-Versionen)	276
6.3.13.	Wrap-Up	276
6.4.	Dependency Injection	277
6.4.1.	Motivation aus Tests: Ich kann nicht mocken	278
6.4.2.	Was ist eine "Dependency"?	279
6.4.3.	Warum ist Kopplung schlecht? (speziell für Tests)	279
6.4.4.	Idee von Dependency Injection	280
6.4.5.	Schritt 1: Abstraktion der Abhängigkeit	280
6.4.6.	Schritt 2: Student mit Dependency Injection	281
6.4.7.	Schritt 3: Verdrahtung im "Composition Root"	282
6.4.8.	DI und Testbarkeit (JUnit)	283
6.4.9.	Ausblick: Verbindung zu Frameworks	284
6.4.10.	Wrap-Up	284
6.5.	Command-Pattern	286
6.5.1.	Motivation	287
6.5.2.	Auflösen der starren Zuordnung über Zwischenobjekte	287
6.5.3.	Command: Objektorientierte Antwort auf Callback-Funktionen	288
6.5.4.	Undo	289
6.5.5.	Abgrenzung zum Strategy-Pattern	290
6.5.6.	Wrap-Up	290
7.	Graphische Oberflächen mit Swing und Java2D	292
7.1.	Swing 4: Events	292
7.1.1.	Reaktion auf Events: Anwendung Observer-Pattern	292
7.1.2.	Arten von Events	293
7.1.3.	Details zu Listeners	293
7.1.4.	Wie komme ich an die Daten eines Events?	294
7.1.5.	Listener vs. Adapter	294
7.1.6.	Exkurs: Nützliches zu Swing	295
7.1.7.	Wrap-Up	295
7.2.	Swing 1: Basics	295
7.2.1.	Wiederholung GUI in Java	296
7.2.2.	Graphische Komponenten einer GUI	296
7.2.3.	Ein einfaches Fenster	297
7.2.4.	Wrap-Up	298
7.3.	Swing 2: Nützliche Widgets	299
7.3.1.	Radiobuttons: <i>JRadioButton</i>	299

7.3.2.	Dateien oder Verzeichnisse auswählen: <i>JFileChooser</i>	300
7.3.3.	TabbedPane und Scroll-Bars	301
7.3.4.	Dialoge mit <i>JOptionPane</i>	301
7.3.5.	Menüs mit <i>JMenuBar</i> , <i>JMenu</i> und <i>JMenuItem</i>	302
7.3.6.	Kontextmenü mit <i>JPopupMenu</i>	302
7.3.7.	Wrap-Up	303
7.4.	Swing 3: Layout-Manager	303
7.4.1.	Überblick	304
7.4.2.	<i>BorderLayout</i>	304
7.4.3.	<i>FlowLayout</i>	305
7.4.4.	<i>GridLayout</i>	306
7.4.5.	Komplexer Layout-Manager: <i>GridBagLayout</i>	306
7.4.6.	Wrap-Up	307
7.5.	Swing 6: Einführung in Graphics und Java 2D	309
7.5.1.	GUIs mit Java	309
7.5.2.	Einführung in die Java 2D API	310
7.5.3.	Java2D Koordinatensystem	310
7.5.4.	Einfache Objekte zeichnen	311
7.5.5.	Fonts und Strings	311
7.5.6.	Einfache Polygone definieren	311
7.5.7.	Ausblick I: Umgang mit Bildern	312
7.5.8.	Ausblick II: <i>Graphics2D</i> kann noch mehr ...	312
7.5.9.	Spiele mit Bewegung	312
7.5.10.	Erinnerung: Observer Pattern	313
7.5.11.	Spielobjekte als Observer (Listener)	313
7.5.12.	Oberfläche zusammenbauen	314
7.5.13.	Wrap-Up	315
8.	Fortgeschrittene Java-Themen und Umgang mit JVM	316
8.1.	Reguläre Ausdrücke	316
8.1.1.	Suchen in Strings	316
8.1.2.	Definition Regulärer Ausdruck	317
8.1.3.	Einfachste reguläre Ausdrücke	317
8.1.4.	Zeichenklassen	318
8.1.5.	Vordefinierte Ausdrücke	318
8.1.6.	Nutzung in Java	319
8.1.7.	Unterschied zw. Finden und Matchen	320
8.1.8.	Quantifizierung	320
8.1.9.	Interessante Effekte	321
8.1.10.	Nicht gierige Quantifizierung mit “?”	321
8.1.11.	(Fangende) Gruppierungen	322
8.1.12.	Gruppen und Backreferences	323
8.1.13.	Beispiel Gruppen und Backreferences	323
8.1.14.	Umlaute und reguläre Ausdrücke	323

8.1.15. Wrap-Up	324
8.2. Generics1: Generische Klassen & Methoden	325
8.2.1. Generische Strukturen	325
8.2.2. Generische Klassen/Interfaces definieren	326
8.2.3. Generische Klassen instantiieren	327
8.2.4. Beispiel I: Einfache generische Klassen	327
8.2.5. Beispiel II: Vererbung mit Typparametern	327
8.2.6. Beispiel III: Überschreiben/Überladen von Methoden	328
8.2.7. Vorsicht: So geht es nicht!	328
8.2.8. Generische Methoden definieren	328
8.2.9. Aufruf generischer Methoden	329
8.2.10. Wrap-Up	330
8.3. Generics2: Bounds & Wildcards	331
8.3.1. Bounds: Einschränken der generischen Typen	331
8.3.2. Wildcards: Dieser Typ ist mir nicht so wichtig	332
8.3.3. Hands-On: Ausgabe für generische Listen	333
8.3.4. Wrap-Up	334
8.4. Generics3: Generics und Polymorphie	336
8.4.1. Motivation: Wie verhalten sich generische Typen zueinander	336
8.4.2. Invarianz generischer Klassen in Java	337
8.4.3. Kovarianz mit ? extends (Producer: nur lesen)	338
8.4.4. Kontravarianz mit ? super (Consumer: nur schreiben)	339
8.4.5. PECS: Producer Extends, Consumer Super	340
8.4.6. Bezug zu funktionalen Interfaces	341
8.4.7. Typ-Löschung (<i>Type Erasure</i>)	342
8.4.8. Type-Erasure bei Nutzung von Bounds	343
8.4.9. Folgen der Typ-Löschung: <i>new</i>	343
8.4.10. Folgen der Typ-Löschung: <i>static</i>	343
8.4.11. Folgen der Typ-Löschung: <i>instanceof</i>	344
8.4.12. Folgen der Typ-Löschung: <i>.class</i>	344
8.4.13. Raw-Types: Ich mag meine Generics "well done" :-)	345
8.4.14. Abgrenzung: Arrays vs. Generics: Reifizierung	345
8.4.15. Generics + Arrays: Warum passt das nicht gut?	346
8.4.16. Diskussion Vererbung vs. Generics	346
8.4.17. Wrap-Up	347
8.5. Exception-Handling	348
8.5.1. Was kann schiefgehen?	349
8.5.2. Begriffe	350
8.5.3. Fangen und Behandeln von Exceptions mit <i>Try-Catch</i> oder <i>Throws</i>	352
8.5.4. <i>Try</i> und mehrstufiges <i>Catch</i>	352
8.5.5. <i>Finally</i> und Aufräumen	353
8.5.6. Freigeben von Ressourcen: <i>Try-with-Resources</i>	354
8.5.7. Werfen von Exceptions mit <i>Throws</i>	354

8.5.8. Anti-Beispiele und Best Practices	355
8.5.9. Stilfrage A: Wann checked, wann unchecked	355
8.5.10. Vertiefung: Stilfrage B: Wie viel Code im <i>Try</i> ?	356
8.5.11. Vertiefung: Stilfrage C: Wo fange ich die Exception?	358
8.5.12. Vertiefung: Eigene Exceptions definieren	358
8.5.13. Wrap-Up	359
9. Clean Code - Smells, Rules, Refactoring	362
9.1. Code Smells	362
9.1.1. Code Smells: Ist das Code oder kann das weg?	362
9.1.2. Was ist guter ("sauberer") Code ("Clean Code")?	363
9.1.3. Warum ist guter ("sauberer") Code so wichtig?	363
9.1.4. Code Smells: Nichtbeachtung von Coding Conventions	365
9.1.5. Code Smells: Schlechte Kommentare I	365
9.1.6. Code Smells: Schlechte Kommentare II	365
9.1.7. Code Smells: Schlechte Namen und fehlende Kapselung	366
9.1.8. Code Smells: Duplizierter Code	366
9.1.9. Code Smells: Langer Code	367
9.1.10. Code Smells: Fehlender Umgang mit Fehlerfällen	369
9.1.11. Code Smells: Feature Neid (engl. Feature Envy)	369
9.1.12. Wrap-Up	369
9.2. Coding Conventions und Metriken	370
9.2.1. Coding Conventions: Richtlinien für einheitliches Aussehen von Code	371
9.2.2. Beispiel nach Google Java Style/AOSP formatiert	372
9.2.3. Formatieren Sie Ihren Code (mit der IDE)	373
9.2.4. Metriken: Kennzahlen für verschiedene Aspekte zum Code	374
9.2.5. Stil & Metriken: Checkstyle	376
9.2.6. Checkstyle: Konfiguration	376
9.2.7. SpotBugs: Finde Anti-Pattern und potentielle Bugs (Linter)	377
9.2.8. Konfiguration für das PR2-Praktikum (Format, Metriken, Checkstyle, SpotBugs)	378
9.2.9. Coding Conventions: Einige zentrale Grundsätze	380
9.2.10. Wrap-Up	380
9.3. Refactoring	381
9.3.1. Was ist Refactoring?	381
9.3.2. Anzeichen, dass Refactoring jetzt eine gute Idee wäre	383
9.3.3. Bevor Sie loslegen	383
9.3.4. Vorgehen beim Refactoring	384
9.3.5. Refactoring-Methode: Rename Method/Class/Field	384
9.3.6. Refactoring-Methode: Encapsulate Field	385
9.3.7. Refactoring-Methode: Extract Method/Class	386
9.3.8. Refactoring-Methode: Move Method	387
9.3.9. Refactoring-Methode: Pull Up, Push Down (Field, Method)	388
9.3.10. Wrap-Up	389

9.4. Javadoc	390
9.4.1. Dokumentation mit Javadoc	391
9.4.2. Standard-Aufbau	391
9.4.3. Veraltete Elemente	392
9.4.4. Autoren, Versionen,	392
9.4.5. Was muss kommentiert werden?	393
9.4.6. Wrap-Up	393
 III. Praktikum	 395
10. Blatt 01: Git Basics, Gradle	397
10.1. Zusammenfassung	397
10.2. Aufgaben	397
10.2.1. Git	397
10.2.2. Installation der Tools für Prog2	398
10.2.3. Aufgabe 3: Anlegen eines Java-Projektes als Basisprojekt für das Semester	400
10.3. Bearbeitung und Abgabe	401
 11. Blatt 02: Git Branches, JUnit Basics; CI-Pipeline	 402
11.1. Zusammenfassung	402
11.2. Aufgaben	402
11.2.1. Aufgabe 1: Git-Spiel	402
11.2.2. Katzen-Café	402
11.2.3. Aufgabe 3: Remotes und CI-Pipeline	403
11.3. Bearbeitung und Abgabe	403
 12. Blatt 03: Methodenrefs, Lambdas, Observer	 404
12.1. Zusammenfassung	404
12.2. Aufgaben	404
12.2.1. Aufgabe 1: Calculator: Anonyme Klassen und Lambda-Ausdrücke	404
12.2.2. LockSnake: Lambda-Ausdrücke, Methodenreferenzen und Observer-Pattern	404
12.3. Bearbeitung und Abgabe	407
 13. Blatt 04: RegEx, Template-Method; JUnit; PR	 408
13.1. Zusammenfassung	408
13.1.1. Aufgabenüberblick	409
13.1.2. Orientierung (Vorgaben)	409
13.1.3. Pflicht- und Bonusaufgaben	410
13.1.4. Begriffe	410
13.2. Aufgaben	410
13.2.1. 1. Reguläre Ausdrücke für das Syntaxhighlighting (MiniJavaTokens)	410
13.2.2. 2. Syntaxhighlighting mit dem RegexHighlighter	412
13.2.3. 3. Syntaxhighlighting mit dem ScanningHighlighter	413
13.2.4. 4. Git: Pull-Requests und CI	415

13.3. Bearbeitung und Abgabe	416
14. Blatt 05: ANTLR, Visitor; Äquivalenzklassen & Grenzwerte, Mocking	417
14.1. Zusammenfassung	417
14.2. Aufgaben	417
14.2.1. 1. ANTLR	417
14.2.2. 2. Cycle Chronicles	420
14.3. Bearbeitung und Abgabe	422
15. Blatt 06: Visitor vs. Pattern Matching, Records	423
15.1. Zusammenfassung	423
15.1.1. Basismodellierung	423
15.1.2. Filter-DSL	424
15.1.3. AST	425
15.1.4. Pretty-Printing, Evaluierung und GUI	427
15.1.5. Wie wird aus Text ein Filter-Ausdruck?	428
15.2. Aufgaben	428
15.3. Bearbeitung und Abgabe	430
16. Blatt 07: Generics, Sealed Types, Stream-API, Logging	431
16.1. Zusammenfassung	431
16.2. Aufgaben	431
16.2.1. Aufgabe 1: Zoo	431
16.2.2. Aufgabe 2: Logging	434
16.2.3. Aufgabe 3: Reflektion	434
16.3. Bearbeitung und Abgabe	434
17. Blatt 08: Command, Optional, Fehlermodell	436
17.1. Zusammenfassung	436
17.2. Aufgaben	436
17.2.1. Aufgabe 1: Einführen von <code>Optional<T></code>	436
17.2.2. 2. Command-Pattern für Gehege-Aktionen	436
17.2.3. Aufgabe 3: <code>Result<E, R></code>	439
17.3. Bearbeitung und Abgabe	440
IV. Organisatorisches	441
18. Prüfungsvorbereitung	442
18.1. Elektronische Klausur: Termin, Materialien	442
18.1.1. Termin	442
18.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig)	443
18.1.3. Einsicht	443
18.2. Technische Vorbereitungen	443
18.3. Bearbeitung des E-Assessment	443

18.4. Fragetypen-Demo	444
18.5. Hinweise zu den Inhalten	444
18.6. Beispiele für mögliche Fragen	444
18.6.1. Vererbung und Polymorphie	444
18.6.2. Reguläre Ausdrücke	445
18.6.3. Versionieren mit Git	445
18.6.4. Generics	445
18.6.5. Logging	446
18.6.6. Methodenreferenzen	446

Teil I.

Syllabus

1. Syllabus

... And, lastly, there's the explosive growth in demand, which has led to many people doing it who aren't any good at it. Code is merely a means to an end. **Programming is an art and code is merely its medium.** Pointing a camera at a subject does not make one a proper photographer. There are a lot of self-described coders out there who couldn't program their way out of a paper bag.

– John Gruber auf daringfireball.net

1.1. Kursbeschreibung

Sie haben letztes Semester in **Prog1** die *wichtigsten* Elemente und Konzepte der Programmiersprache Java kennen gelernt.

In diesem Modul geht es darum, diese Kenntnisse sowohl auf der Java- als auch auf der Methoden-Seite so zu erweitern, dass Sie gemeinsam größere Anwendungen erstellen und pflegen können. Sie werden fortgeschrittene Konzepte in Java kennenlernen und sich mit etablierten Methoden in der Softwareentwicklung wie Versionierung von Code, Einhaltung von Coding Conventions, Grundlagen des Softwaretests, Anwendung von Refactoring, Einsatz von Build-Tools und Logging auseinander setzen. Wenn uns dabei ein Entwurfsmuster “über den Weg läuft”, werden wir die Gelegenheit nutzen und uns dieses genauer anschauen.

1.2. Überblick Modulinhalte

1. Fortgeschrittene Konzepte in Java (“Classic Java”)

- Reguläre Ausdrücke, Generics, Dependency Injection
- Fremde APIs nutzen: ANTLR
- Graphische Oberflächen mit Swing
- Fehlerbehandlungskonzepte

2. Fortgeschrittene Konzepte in Java (“FP”)

- Default-Methoden, Funktionsinterfaces, Methodenreferenzen, Lambdas, Optional, Stream-API
- Records, Sealed Classes, Pattern Matching

3. Versionierung mit Git

4. Softwarequalität

- Testen mit JUnit und Mockito

- Coding Conventions & Smells, Refactoring
5. Entwurfsmuster
- Observer, Visitor, Template-Method, Command
6. Tooling und Bauen von Software
- Packages
 - Logging, Debugging
 - Gradle, Docker, Continuous Integration (GitHub Workflows)

1.3. Team

- [Carsten Gips](#) (Sprechstunde nach Vereinbarung)
- Alesia Herbertz (Tutorin)

1.4. Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mo, 08:00 - 09:30 Uhr (<i>Flipped Classroom</i>)	G1: Mo, 11:30 - 13:00 Uhr G2: Mo, 14:00 - 15:30 Uhr G3: Mo, 09:45 - 11:15 Uhr G4: Mi, 08:00 - 09:30 Uhr

Alle Sitzungen online per Zoom (**Zugangsdaten siehe [ILIAS](#)**).

1.5. Sprechstunde

- Dienstag, 10:00 - 11:00 Uhr
- Freitag, 10:00 - 11:00 Uhr

1.6. Fahrplan

Abgabe der Post Mortems jeweils **Montag bis 08:00 Uhr** im [ILIAS](#). Vorstellung der Lösung im jeweiligen Praktikum in der Abgabewoche.

Monat	Tag	Vorlesung (Mo)	Praktikum (Mo/Mi)
April	20.	Orga	
	27.	Git1: Basics; Gradle, Packages	
Mai	04.	Git2: Branches, Git4: Worktree; JUnit1: Basics; CI	B01
	11.	Lambdas, Methodenrefs; Observer, Swing Events	B02
	18.	Git3: Workflows; RegExp; Template-Method	B03
	25.	Feiertag	Feiertag
Juni	01.	Visitor; ANTLR; Debugging	B04
	08.	JUnit2: Testfälle, JUnit3: Mocking; Dependency Injection, Defaultmethoden	Kein Praktikum (Parcoursprüfung BC)
	15.	Records, Pattern Matching, Stream-API; JUnit4: Property Testing	B05
	22.	Generics1: Klassen/Methoden, Generics2: Bounds & Wildcards; Command; Logging	B06
	29.	Generics3: Type Erasure & Polymorphie, Exceptions, Optional & Result	B07
Juli	06.	Bad Smells, Coding Rules, Refactoring; Docker	B08
	13.	Rückblick, Prüfungsvorbereitung	

1.7. Prüfungsform, Note und Credits

(Digitale) Klausur plus Studienleistung (Portfolio), 5 ECTS

- **Studienleistung:** "Portfolio":
Mindestens 6 der Übungsblätter B01..B08 erfolgreich bearbeitet (inkl. Post Mortem).
- **Gesamtnote:** (Digitale) Klausur im B40 (90 Minuten)

Hinweise

- Die Bearbeitung der Übungsblätter erfolgt individuell.
- Ein Team umfasst 1 Person.
- Die Post Mortems sind individuell zu erstellen und fristgerecht abzugeben.
- "Aktive Beteiligung" umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.
- "Erfolgreiche Bearbeitung" eines Blattes umfasst die individuelle Bearbeitung aller Aufgaben des Blattes sowie die fristgerechte Abgabe des ausreichenden Post Mortems im ILIAS. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.

- **Post Mortem:** Jede Person beschreibt individuell(!) die Bearbeitung des jeweiligen Übungsblattes zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast; Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:
 1. Zusammenfassung: Was wurde gemacht?
 2. Details: Kurze Beschreibung besonders interessanter Aspekte.
 3. Reflexion: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
 4. Reflexion: Was haben Sie gelernt oder (besser) verstanden?
 5. Link zu Ihrem Repo/Branch/PR mit den relevanten Artefakten.

Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im [ILIAS](#) ab.

Siehe auch <https://github.com/Programmierungsmethoden-CampusMinden/Prog2-Lecture-S26/discussions/2>.

1.8. Materialien

1.8.1. Literatur

1. [“Learn Java”](#). Oracle Corporation, 2026.
2. [“Pro Git \(Second Edition\)”](#). Chacon, S. und Straub, B., Apress, 2014. ISBN [978-1-4842-0077-3](#).
3. [“Java ist auch eine Insel”](#). Ullnboom, C., Rheinwerk-Verlag, 2021. ISBN [978-3-8362-8745-6](#).
4. [“The Java Tutorials”](#). Oracle Corporation, 2024.

1.8.2. Tools

- JDK: **Java SE 25 (LTS)** ([Oracle](#) oder [Alternativen](#), bitte 64-bit Version nutzen)
- IDE: [Eclipse IDE for Java Developers](#) oder [IntelliJ IDEA \(Community Edition\)](#)
- [Git](#)

1.9. Förderungen und Kooperationen

1.9.1. Förderung durch DH.NRW (Digi Fellowships)

Die Überarbeitung dieser Lehrveranstaltung wurde vom Ministerium für Kultur und Wissenschaft (MKW) in NRW im Einvernehmen mit der Digitalen Hochschule NRW (DH.NRW) gefördert: [“Fellowships für Innovationen in der digitalen Hochschulbildung” \(Digi Fellowships\)](#).

1.9.2. Kooperation mit dem DigikoS-Projekt

Diese Vorlesung wurde vom Projekt [“Digitalbaukasten für kompetenzorientiertes Selbststudium” \(DigikoS\)](#) unterstützt. Ein vom DigikoS-Projekt ausgebildeter Digital Learning Scout hat insbesondere die Koordination

der digitalen Gruppenarbeiten, des Peer-Feedbacks und der Postersessions in ILIAS technisch und inhaltlich begleitet. DigikoS wird als Verbundprojekt von der Stiftung Innovation in der Hochschullehre gefördert.

1.10. LICENSE



Unless otherwise noted, [this work](#) by [Carsten Gips](#) and [contributors](#) is licensed under [CC BY-SA 4.0](#). See the [credits](#) for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

2. Versionierung mit Git

2.1. Git1: Basics der Versionsverwaltung mit Git

TL;DR

In der Softwareentwicklung wird häufig ein Versionsmanagementsystem (VCS) eingesetzt, welches die Verwaltung von Versionsständen und Änderungen ermöglicht. Ein Repository sammelt dabei die verschiedenen Änderungen (quasi wie eine Datenbank der Software-Versionenstände). Die Software *Git* ist verbreiteter Vertreter und arbeitet mit dezentralen Repositories.

Ein neues lokales Repository kann man mit `git init` anlegen. Der Befehl legt im aktuellen Projektordner einen versteckten Unterordner `.git/` an (FINGER WEG!). anderen Unterordner im aktuellen Ordner können nun der Versionskontrolle hinzugefügt werden. Der Projektordner selbst (mit Ihren Quelltexten) ist die *Workingcopy*.

Ein bereits existierendes Repo kann mit `git clone <url>` geklont werden.

Änderungen an Dateien (in der Workingcopy) werden mit `git add` zum “Staging” (Index) hinzugefügt. Dies ist eine Art Sammelbereich für Änderungen, die mit dem nächsten Commit in das Repository überführt werden. Neue (bisher nicht versionierte Dateien) müssen ebenfalls zunächst mit `git add` zum Staging hinzugefügt werden.

Änderungen kann man mit `git log` betrachten, dabei erhält man u.a. eine Liste der Commits und der jeweiligen Commit-Messages.

Mit `git diff` kann man gezielt Änderungen zwischen Commits oder Branches betrachten.

Mit `git tag` kann man bestimmte Commits mit einem “Stempel” (zusätzlicher Name) versehen, um diese leichter finden zu können.

Wichtig sind die Commit-Messages: Diese sollten eine kurze Zusammenfassung haben, die **aktiv** formuliert wird (was ändert dieser Commit: “Formatiere den Java-Code entsprechend Style”; nicht aber “Java-Code nach Style formatiert”). Falls der Kommentar länger sein soll, folgt eine Leerzeile auf die erste Zeile (Zusammenfassung) und danach ein Block mit der längeren Erklärung.

Videos

Vorlesung [YT], [HSBI]

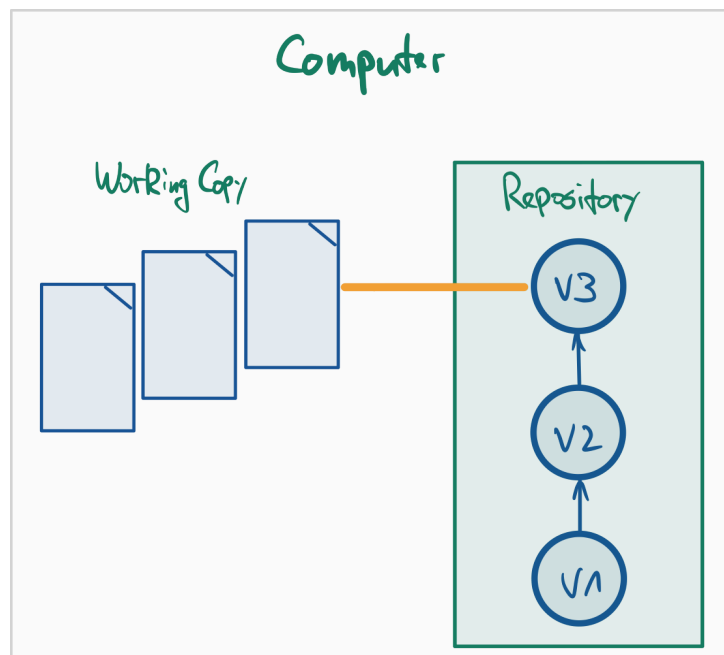
Demos:

- Konfiguration [YT], [HSBI]
- Erstellung Repo [YT], [HSBI]
- Dateien hinzufügen [YT], [HSBI]
- Arbeitsablauf: Datei ändern - stagen - committen [YT], [HSBI]

- Hinzufügen zu Commits (Amend) [YT], [HSBI]
- Anschauen von Änderungen (Log) [YT], [HSBI]
- Anschauen von Unterschieden (Diff) [YT], [HSBI]
- Umgang mit Tags [YT], [HSBI]

Introduction to Git with Scott Chacon of GitHub (erster Teil, bis ca. Minute 45)

2.1.1. Prinzip Versionsverwaltung



- **Repository:** (interne) **Datenbank** mit verschiedenen Versionsständen, Kommentaren, Tags etc.
 - Technisch ist das das Verzeichnis `.git` im Projektordner
 - Unterscheidung:
 - * Lokales Repository: Das `.git`-Verzeichnis auf dem lokalen Rechner
 - * Remote-Repository: Kopie der Historie auf einem Server (z.B. GitHub, GitLab, Codeberg), Zugriff über Web-GUI oder Netzwerk

Enthält u.a. alle Commits (Schnappschüsse), alle Blobs (Dateiinhalte), Trees (Verzeichnisstrukturen), Referenzen wie Branches, Tags, HEAD, (lokale) Konfigurationen (z.B. `.git/config`).

Das Repository selbst enthält keine "normalen" Dateien, die Sie mit einem Editor öffnen (sollten); es ist ein interner Speicher, den Git verwaltet.

`git clone <url>` erstellt aus einem Remote-Repository eine lokale Kopie (Workingcopy + lokales Repository).

- **Workingcopy / Arbeitskopie:** Projektordner mit bestimmtem Versionsstand

Das ist der Projektordner, das Sie in der IDE oder im Editor öffnen, z.B. `~/projekte/shop-system/`.

Der Projektordner, in dem Sie tatsächlich arbeiten (editieren, kompilieren, testen). Nach einem `git clone` oder `git checkout` entspricht der Inhalt einem bestimmten Commit. Danach können Sie beliebig neue Dateien anlegen und bestehende Dateien ändern – diese Änderungen existieren zunächst nur in der Workingcopy, nicht im Repository.

Die Dateien in der Working Copy können sein:

- unverändert (entsprechen genau dem letzten Commit),
- modifiziert (geändert seit dem letzten Commit),
- neu/untracked (Git kennt sie noch nicht).

In der Workingcopy gibt es den `.git`-Ordner (Repository), welcher i.d.R. die gesamte Historie beinhaltet.

- **Staging Area / Index:** Zwischenablage für Dateistände, Vormerkung für Commit

Dies ist eine Art Zwischenstufe zw. Arbeitsverzeichnis und Repository, in der Änderungen festgehalten werden, die beim nächsten Commit gemeinsam ins Repository geschrieben werden.

Änderungen landen hier durch das Hinzufügen mit `git add`. Erst nach einem Commit (`git commit`) landen diese Änderungen tatsächlich im Repository.

Eine Datei kann gleichzeitig in drei Ständen existieren:

- Im letzten Commit (Repository): “offiziell gespeicherter” Stand
- Im Index (Staging Area): Stand, der für den nächsten Commit vorgemerkt ist
- In der Workingcopy: aktueller Arbeitsstand im Editor (kann weiter verändert sein)

- **Tracked / Untracked Files:** versionierte vs. nicht-versionierte Dateien im Arbeitsverzeichnis

- “Tracked”: Dateien, die Git (im Repository) bereits kennt und versioniert
- “Untracked”: neue Dateien im Arbeitsverzeichnis, von Git (noch) ignoriert
- `git status` zeigt diese Unterscheidung sehr gut

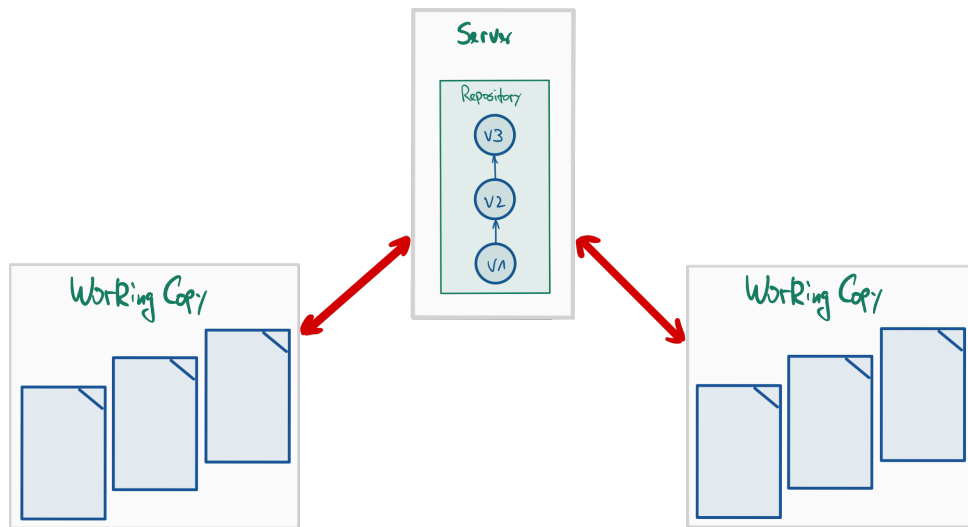
- **Commit:** “Schnappschuss” von Änderungen zu einem bestimmten Zeitpunkt

Enthält Änderungen (Delta) im Vergleich zum Vorgänger-Commit sowie Metadaten (Autor, Datum, Commit-Message, Hash-ID).

Jeder Commit erhält eine eindeutige **Commit-ID** (SHA-1-Hash, z.B. `a1b2c3d . . .`). Mit dieser ID kann man sich gezielt bestimmte Commits anschauen.

Bildet einen Knoten im Versionsgraphen - Folgen von Commits nennt man auch “Branch” (das schauen wir uns in der Sitzung Git Branches genauer an).

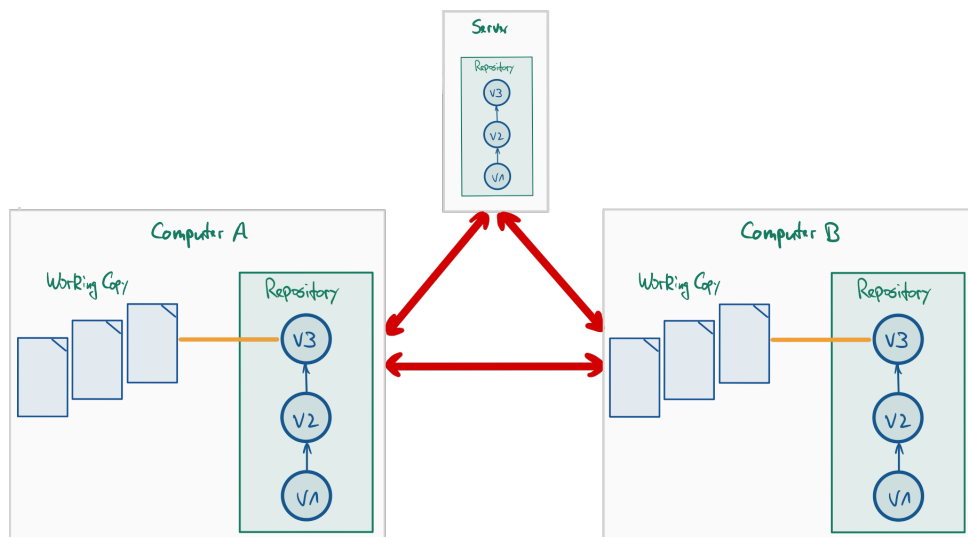
2.1.2. Varianten: Zentrale Versionsverwaltung (Beispiel SVN)



Es gibt ein zentrales Repository (typischerweise auf einem Server), von dem die Developer einen bestimmten Versionsstand “auschecken” (sich lokal kopieren) und in welches sie Änderungen wieder zurück “pushen”.

Zur Abfrage der Historie und zum Veröffentlichen von Änderungen benötigt man entsprechend immer eine Verbindung zum Server.

2.1.3. Varianten: Verteilte Versionsverwaltung (Beispiel Git)



In diesem Szenario hat jeder Developer nicht nur die Workingcopy, sondern auch noch eine Kopie des Repositories. Zusätzlich kann es einen oder mehrere Server geben, auf denen dann nur das Repository vorgehalten wird, d.h. dort gibt es normalerweise keine Workingcopy. Damit kann unabhängig voneinander gearbeitet werden.

Allerdings besteht nun die Herausforderung, die geänderten Repositories miteinander abzugleichen. Das kann zwischen dem lokalen Rechner und dem Server passieren, aber auch zwischen zwei “normalen” Rechnern (also zwischen den Developern).

Hinweis: *GitHub ain't no Git!* Git ist eine Technologie zur Versionsverwaltung. Es gibt verschiedene Implementierungen und Plugins für IDEs und Editoren. [GitHub](#) ist dagegen *ein* Dienstleister, wo man Git-Repositories ablegen kann und auf diese mit Git (von der Konsole oder aus der IDE) zugreifen kann. Darüber hinaus bietet der Service aber zusätzliche Features an, beispielsweise ein Issue-Management oder sogenannte *Pull-Requests*. Dies hat aber zunächst mit Git nichts zu tun. Weitere populäre Anbieter sind beispielsweise [Bitbucket](#) oder [Gitlab](#) oder [Gitea](#), wobei einige auch selbst gehostet werden können.

2.1.4. Versionsverwaltung mit Git: Typische Arbeitsschritte

1. Repository anlegen (oder clonen)
2. Dateien neu erstellen (und löschen, umbenennen, verschieben)
3. Änderungen einpflegen (“committen”)
4. Änderungen und Logs betrachten
5. Änderungen rückgängig machen
6. Projektstand markieren (“taggen”)
7. Entwicklungszweige anlegen (“branchen”)
8. Entwicklungszweige zusammenführen (“mergen”)
9. Änderungen verteilen (verteiltes Arbeiten, Workflows)

2.1.5. (Globale) Konfiguration

Minimum:

- `git config --global user.name <name>`
- `git config --global user.email <email>`

Diese Konfiguration muss man nur einmal machen.

Wenn man den Schalter `--global` weglässt, gelten die Einstellungen nur für das aktuelle Projekt/Repo.

Zumindest Namen und EMail-Adresse **muss** man setzen, da Git diese Information beim Anlegen der Commits speichert (== benötigt!).

Aliase:

- `git config --global alias.ci commit`

- `git config --global alias.co checkout`
- `git config --global alias.br branch`
- `git config --global alias.st status`
- `git config --global alias.ll 'log --all --graph --decorate --oneline'`

Zusätzlich kann man weitere Einstellungen vornehmen, etwa auf bunte Ausgabe umschalten: `git config --global color.ui auto` oder Abkürzungen (Aliase) für Befehle definieren: `git config --global alias.ll 'log --all --oneline --graph --decorate' ...`

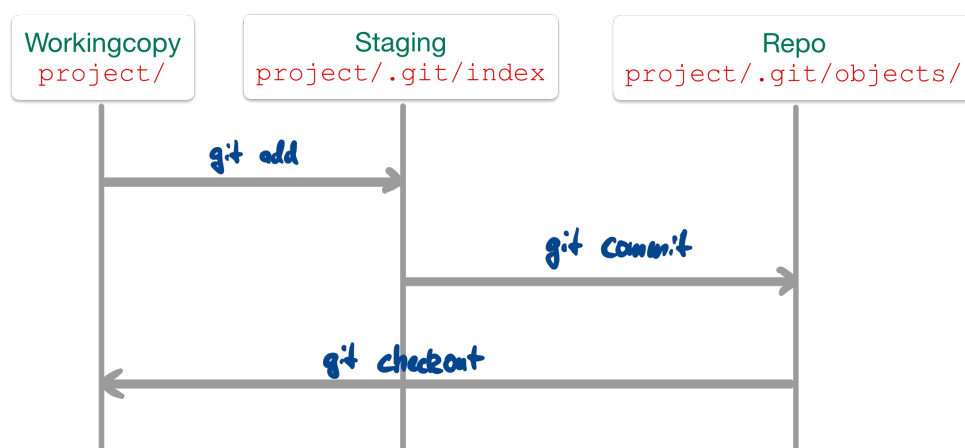
Git (und auch GitHub) hat kürzlich den Namen des Default-Branches von `master` auf `main` geändert. Dies kann man in Git ebenfalls selbst einstellen: `git config --global init.defaultBranch <name>`.

Anschauen kann man sich die Einstellungen in der Textdatei `~/.gitconfig` oder per Befehl `git config --global -l`.

2.1.6. Neues Repo anlegen

- `git init`
=> Erzeugt neues Repository im akt. Verzeichnis
- `git clone <url>`
=> Erzeugt (verlinkte) Kopie des Repos unter `<url>`

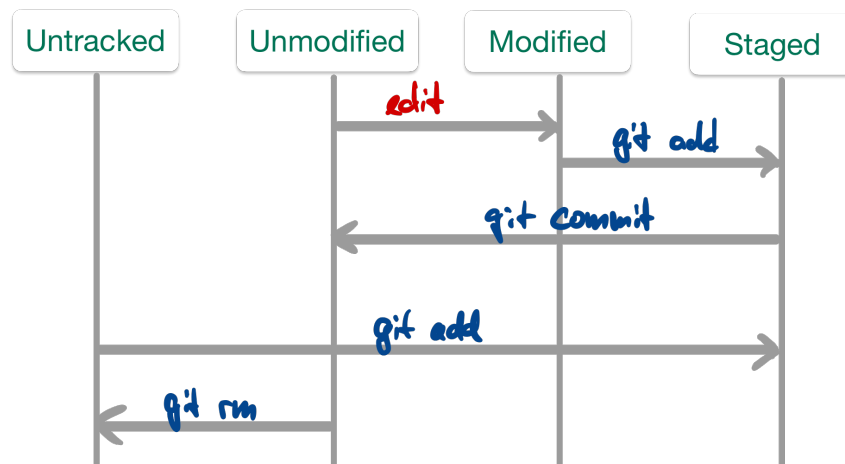
2.1.7. Dateien unter Versionskontrolle stellen



1. `git add .` (oder `git add <file>`)
=> Stellt alle Dateien (bzw. die Datei `<file>`) im aktuellen Verzeichnis unter Versionskontrolle
2. `git commit`
=> Fügt die Dateien dem Repository hinzu

Abfrage mit `git status`

2.1.8. Änderungen einpflegen



- Abfrage mit: `git status`
- “Staging” von modifizierten Dateien: `git add <file>`
- Committen der Änderungen im Stage: `git commit`

Anmerkung: Alternativ auch mit `git commit -m "Kommentar"`, um das Öffnen des Editors zu vermeiden ... geht einfach schneller ;)

Das “staging area” stellt eine Art Zwischenebene zwischen Working Copy und Repository dar: Die Änderungen sind temporär “gesichert”, aber noch nicht endgültig im Repository eingepflegt (“committed”).

Man kann den Stage dazu nutzen, um Änderungen an einzelnen Dateien zu sammeln und diese dann (in einem Commit) gemeinsam einzuchecken.

Man kann den Stage in der Wirkung umgehen, indem man alle in der Working Copy vorliegenden Änderungen per `git commit -a -m "Kommentar"` eincheckt. Der Schalter “-a” nimmt alle vorliegenden Änderungen an **bereits versionierten** Dateien, fügt diese dem Stage hinzu und führt dann den Commit durch. Das ist das von SVN bekannte Verhalten. Achtung: Nicht versionierte Dateien bleiben dabei außen vor!

2.1.9. Letzten Commit ergänzen

- `git commit --amend -m "Eigentlich wollte ich das so sagen"`

Wenn keine Änderungen im Stage sind, wird so die letzte Commit-Message geändert.

- `git add <file>; git commit --amend`

Damit können vergessene Änderungen an der Datei `<file>` zusätzlich im letzten Commit aufgezeichnet werden.

In beiden Fällen ändert sich die Commit-ID!

2.1.10. Weitere Datei-Operationen: hinzufügen, umbenennen, löschen

- Neue (unversionierte) Dateien und Änderungen an versionierten Dateien zum Staging hinzufügen: `git add <file>`
- Löschen von Dateien (Repo+Workingcopy): `git rm <file>`
- Löschen von Dateien (nur Repo): `git rm --cached <file>`
- Verschieben/Umbenennen: `git mv <fileAlt> <fileNeu>`

Aus Sicht von Git sind zunächst alle Dateien “untracked”, d.h. stehen nicht unter Versionskontrolle.

Mit `git add <file>` (und `git commit`) werden Dateien in den Index (den Staging-Bereich, d.h. nach dem Commit letztlich in das Repository) aufgenommen. Danach stehen sie unter “Beobachtung” (Versionskontrolle). So lange, wie eine Datei identisch zur Version im Repository ist, gilt sie als unverändert (“unmodified”). Eine Änderung führt entsprechend zum Zustand “modified”, und ein `git add <file>` speichert die Änderungen im Stage. Ein Commit überführt die im Stage vorgemerkte Änderung in das Repo, d.h. die Datei gilt wieder als “unmodified”.

Wenn eine Datei nicht weiter versioniert werden soll, kann sie aus dem Repo entfernt werden. Dies kann mit `git rm <file>` geschehen, wobei die Datei auch aus der Workingcopy gelöscht wird. Wenn die Datei erhalten bleiben soll, aber nicht versioniert werden soll (also als “untracked” markiert werden soll), dann muss sie mit `git rm --cached <file>` aus der Versionskontrolle gelöscht werden. Achtung: Die Datei ist dann nur ab dem aktuellen Commit gelöscht, d.h. frühere Revisionen enthalten die Datei noch!

Wenn eine Datei umbenannt werden soll, geht das mit `git mv <fileAlt> <fileNeu>`. Letztlich ist dies nur eine Abkürzung für die Folge `git rm --cached <fileAlt>`, manuelles Umbenennen der Datei in der Workingcopy und `git add <fileNeu>`.

2.1.11. Commits suchen und betrachten

- Liste aller Commits: `git log`
 - `git log -<n>` oder `git log --since="3 days ago"` Meldungen eingrenzen ...
 - `git log --stat` Statistik ...
 - `git log --author="pattern"` Commits eines Autors
 - `git log <file>` Änderungen einer Datei
- Inhalt eines Commits: `git show`

Anmerkung: Ausgewählte interessante Optionen für `git log`:

Für `git log` gibt es eine schöne Option `-p`, die einen “Patch” ausgibt: Gelöschte Zeilen werden mit einem “-” und hinzugefügte Zeilen werden mit einem “+” angezeigt. Zusätzlich werden jeweils noch ein paar ungeänderte Zeilen vor und nach der Änderung angezeigt.

Mit der Option `-S '<SUCHSTRING>'` zeigt `git log` alle Commits an, in deren Diff sich die Anzahl der Vorkommen des Suchstrings ändert. Beispielsweise wenn in einem Commit der Suchstring neu eingefügt wurde, ändert sich die Anzahl von 0 auf 1. Beispiel: Wir suchen nach allen Commits, in denen der Begriff “foobar” neu hinzukommt oder verschwindet oder sich verdoppelt: `git log -p -S'foobar'`.

Oft hilft `-S` nicht richtig, weil man Commits haben möchte, in deren Diff sich etwas in der selben Zeile ändert, in der der Suchstring vorkommt, aber nicht der Suchbegriff selbst. Da sich hier die Anzahl des Suchstrings im Diff nicht unbedingt ändert, würden diese Commits nicht angezeigt. Beispiel: Wir suchen nach allen Commits mit Änderungen des Zahlenwerts in der Zeile `foobar 10`. Hier hilft die Option `-G '<REGEX>'`: Damit werden alle Commits gefunden, in deren Diff sich in den Zeilen mit dem Suchausdruck etwas ändert: `git log -p -G'foobar'` findet alle Zeilen im Diff, wo “foobar” vorkommt, d.h. wenn sich beispielsweise die Zeile von `foobar 3` auf `foobar 10` ändert. Vorsicht: Im Unterschied zu `-S` ist hier der Suchstring ein regulärer Ausdruck (Regex). Git verwendet dabei eine etwas spezielle Regex-Syntax (siehe Git-Dokumentation).

Wenn man nicht nach Änderungen im Diff, sondern stattdessen nach Begriffen in den Commit-Messages suchen möchte, kann man die Option `--grep= '<SUCHSTRING>'` nutzen. Auch hier ist `<SUCHSTRING>` ein regulärer Ausdruck.

Wenn man die gefundenen Stellen auf einen bestimmten Pfad (Datei) begrenzen möchte, kann man `-- pfad /zur/datei.md` hinten an den Log-Befehl anhängen. Beispielsweise würde `git log -p -G'foobar' -- foo/bar.txt` alle Commits finden, in deren Diff “foobar” vorkommt und die die Datei `foo/bar.txt` betreffen.

Normalerweise arbeitet sich `git log` vom jüngsten zum ältesten Commit durch, geht also in der Geschichte zurück. Wenn man die Reihenfolge umkehren möchte, kann man noch `--reverse` als zusätzliche Option nutzen. Man kann auch einen “Commit-Range” angeben, etwa `5c73aca..cb73f92`.

Die Option `--all` zeigt alle Branches an, also nicht nur die Änderungen auf dem aktuell ausgecheckten Branch. Mit der zusätzlichen Option `--graph` bekommt man in der Konsole eine hübsche kleine baumartige Struktur angezeigt.

Mit der Option `--oneline` wird der ausgegebene Log abgekürzt und pro Commit nur die wichtigsten Dinge (SHA-ID und abgekürzte Commit-Message) ausgegeben.

Ich persönlich nutze häufig `git log --all --graph --oneline` und habe mir dazu einen Alias (s.o.) angelegt.

2.1.12. Änderungen und Logs betrachten

- `git diff [<file>]`

Änderungen zwischen Workingcopy und letztem Commit (ohne Stage)

Das “staging area” wird beim Diff von Git behandelt, als wären die dort hinzugefügten Änderungen bereits eingchecked (genauer: als letzter Commit im aktuellen Branch im Repo vorhanden). D.h. wenn Änderungen in einer Datei mittels `git add <datei>` dem Stage hinzugefügt wurden, zeigt `git diff <datei>` keine Änderungen an!

- `git diff commitA commitB`

Änderungen zwischen Commits

Git arbeitet zeilenbasiert. Änderungen an einem Wort tauchen also im Diff immer als ganze geänderte Zeile auf. (Das ist auch ein guter Grund, die Zeilenlänge zu begrenzen!)

Gelöschte Zeilen werden mit einem “-” und hinzugefügte Zeilen werden mit einem “+” angezeigt. Zusätzlich werden jeweils noch ein paar ungeänderte Zeilen vor und nach der Änderung angezeigt.

- Blame: `git blame <file>`

Wer hat was wann gemacht?

2.1.13. Dateien ignorieren: `.gitignore`

- Nicht alle Dateien gehören ins Repo:
 - generierte Dateien: `.class`
 - temporäre Dateien
- Datei `.gitignore` anlegen und committen
 - Wirkt auch für Unterordner
 - Inhalt: Reguläre Ausdrücke für zu ignorierende Dateien und Ordner

```
1      # Compiled source #
2      *.class
3      *.o
4      *.so
5
6      # Packages #
7      *.zip
8
9      # All directories and files in a directory #
10     bin/**/*
```

[man 5 gitignore](#)

2.1.14. Zeilenenden: Windows vs. der Rest der Welt

Windows nutzt CRLF (`\r\n`) als Zeilenende, der Rest der Welt arbeitet mit LF (`\n`). Git versucht das zu erkennen und beim Commit bzw. Checkout automatisch zu konvertieren (z.B. alles im Repository als LF zu speichern und lokal auf Windows mit CRLF auszuchecken). Da es dabei oft Schwierigkeiten gibt und Warnungen wie “*LF will be*

replaced by CRLF the next time Git touches it“ auftauchen, kann man im Projekt eine Datei `.gitattributes` anlegen.

Hier ein Beispiel mit einer minimalen Konfiguration, die ich häufig verwende:

```
1 * text=auto eol=lf
2
3 *.cmd text eol=crlf
4 *.bat text eol=crlf
```

Das sagt Git, dass es Text-Dateien automatisch erkennen soll und sie im Repository mit `LF (\n)` als Zeilentrenner speichern soll. In der Workingcopy werden diese Dateien dann ebenfalls mit `LF (\n)` ausgecheckt - auch unter Windows. Für Dateien, die für Windows wichtig sind (beispielsweise die `gradlew.bat`), geben die letzten beiden Zeilen explizit an, dass Git sie in der Workingcopy mit Windows-üblichen Zeilenenden `CRLF (\r\n)` auschecken soll. Im Repository liegen sie trotzdem normalisiert mit `LF (\n)`. So bleiben die Skripte unter Windows funktionsfähig, ohne dass jeder Entwickler:in die eigene Git-Konfiguration anpassen muss.

Siehe auch [man 5 gitattributes](#).

2.1.15. Zeitmaschine

- Änderungen in Workingcopy rückgängig machen
 - Änderungen nicht in Stage: `git checkout <file>` oder `git restore <file>`
 - Änderungen in Stage: `git reset HEAD <file>` oder `git restore --staged <file>`

=> Hinweise von `git status` beachten!

- Datei aus altem Stand holen:
 - `git checkout <commit> <file>`, oder
 - `git restore --source <commit> <file>`
- Commit verwerfen, Geschichte neu: `git revert <commit>`

Hinweis: In den neueren Versionen von Git ist der Befehl `git restore` hinzugekommen, mit dem Änderungen rückgängig gemacht werden können. Der bisherige Befehl `git checkout` steht immer noch zur Verfügung und bietet über `git restore` hinaus weitere Anwendungsmöglichkeiten.

- Stempel (Tag) vergeben: `git tag <tagname> <commit>`
- Tags anzeigen: `git tag` und `git show <tagname>`

2.1.16. Wann und wie committen?

Jeder Commit stellt einen Rücksetzpunkt dar!

Typische Regeln:

- Kleinere “Häppchen” einchecken: ein Feature oder Task (das nennt man auch *atomic commit*: das kleinste Set an Änderungen, die gemeinsam Sinn machen und die ggf. gemeinsam zurückgesetzt werden können)
- Logisch zusammenhängende Änderungen gemeinsam einchecken
- Projekt muss nach Commit compilierbar sein
- Projekt sollte nach Commit lauffähig sein

Ein Commit sollte in sich geschlossen sein, d.h. die kleinste Menge an Änderungen enthalten, die gemeinsam einen Sinn ergeben und die (bei Bedarf) gemeinsam zurückgesetzt oder verschoben werden können. Das nennt man auch **atomic commit**.

Wenn Sie versuchen, die Änderungen in Ihrem Commit zu beschreiben (siehe nächste Folie “Commit-Messages”), dann werden Sie einen *atomic commit* mit einem kurzen Satz (natürlich im Imperativ!) beschreiben können. Wenn Sie mehr Text brauchen, haben Sie wahrscheinlich keinen *atomic commit* mehr vor sich.

Lesen Sie dazu auch [How atomic Git commits dramatically increased my productivity - and will increase yours too](#).

2.1.17. Schreiben von Commit-Messages: WARUM?!

Schauen Sie sich einmal einen Screenshot eines `git log --oneline 61e48f0..e2c8076` im [Dungeon-CampusMinden/Dungeon](#) an:

```
e2c8076 [Toolchain] Re-rename groupID (#374)
a991cf3 Repository umbenennen (#370)
8a3f213 Fix Typo in graphics Template (#360)
4cd9463 Fix für Issue 357 Pathfinding Probleme mit ID 0 (#359)
a21cd66 Merge frame methods (#353)
719836d [GITHUB] New Issue-Templates (#336)
20caad9 Update: github_ci.yml, cache gradle wrapper (#350)
0ed33af Add dungeon-starter to core (#348)
8c73ab2 [GENERATOR] Perlin Noise #332 (#334)
9c2c611 [FEATURE] Refactor Level (#329)
144d7b4 Update gson version and add dependencies checker (#344)
a75d101 [ENHANCEMENT] Add List for AbstractController (#337)
8170ade [BUG] add Javadoc for beginFrame/endFrame/setup (#339)
4348a13 Remove Xms and Xmx options (#330)
0b18d69 rename Tile.texturePath to Tile.texture (#322)
46530b6 Layer system 5 (#300)
d5c20f5 rename getTexture in getTexturePath (#312)
cb24ce5 Levelgenerierung: Remove Wildcards and Replacement (#310)
763c02e Remove version number (master, 5.x) (#301)
d73aa0c Door bug (#298)
637acbe Pausefunktion für Gameloop (#299)
1a9326c Update workflows and buildscripts (develop) (#293)
2ccc6ed konstante Laufzeit der getTileAt(coordinate) Methode in Level (#266)
0b565d4 Better start and exit (#258)
```

Nun stellen Sie sich vor, Sie sind auf der Suche nach Informationen, suchen einen bestimmten Commit oder wollen eine bestimmte Änderung finden ...

Wenn man das genauer analysiert, dann stören bestimmte Dinge:

- Mischung aus Deutsch und Englisch
- “Vor-sich-hin-Murmeln”: “Layer system 5”
- Teilweise werden Tags genutzt wie [BUG], aber nicht durchgängig
- Mischung zwischen verschiedenen Formen: “Repo umbenennen”, “Benenne Repo um”, “Repo umbenannt”
- Unterschiedliche Groß- und Kleinschreibung
- Sehr unterschiedlich lange Zeilen/Kommentare

Das Beachten einheitlicher Regeln ist enorm wichtig!

Leider sagt sich das so leicht - in der Praxis macht man es dann doch schnell wieder unsauber. Dennoch, auch im Dungeon-Repo gibt es einen positiven Trend (`git log --online 8039d6c..7f49e89`):

```
7f49e89 [GAME] add better level size generation options (#493)
c64f7c1 [GAME] add fireball textures (#477)
f645815 [GAME] remove old logo files, use new logo (#478)
3f1f2c9 new logo with the red cat in the dimensions 35x35, 64x64 and 128x128 (#476)
c6f0fc1 [Game] Schatztruhe (#463)
803e2d0 [GAME] Change Exit/StartTile (#459)
8483714 [Game] Fix Exception in Configuration on Game Startup (#475)
cb8017f [GAME] Erweiterung des Collision checks um onEnter und onLeave (#452)
382e541 [GAME] Add fight skill (#469)
9c429af [GAME] SkillSystem (#395)
6d76e79 [GAME] add Interaktionssystem & Interaktionscomponent (#389)
a90dfe6 [GAME] add Skill Cool Down (#394)
e08c3b3 [GAME] add Basis Logging (#455)
7290999 [GAME] fix getRandomTile mögliche Endlosschleife (#458)
13f460d [GAME] Items (Active & Passive) + Stats (#450)
5f0459f [GAME] onDeath drop Loot (#396)
03e1ff0 [GAME] Funktionales Handling von Optionals (#400)
4bb8b9b [GAME] Configuration File (#443)
2e0caaf [GAME] Inventory Component (#388)
195c686 [GAME] add XPComponent & -System
3d342bb [GAME] Merge `pm-dungeon` and `game` Projects and create `api`Project (#401)
64a0fce [GAME] Build w/ local code, i.e. do NOT use Maven Central, and remove duplicated assets (#399)
5c11d7d [GAME] IIIdle: PatrouilleWalk (#352)
d0da90f [GAME] add ECS Dokumentation (#211)
```

Typische Regeln und Konventionen tauchen überall auf, beispielsweise in Chacon und Straub (2014) oder bei Tim Pope (siehe nächstes Beispiel) oder bei “[How to Write a Git Commit Message](#)”.

```
1 Short (50 chars or less) summary of changes
2
3 More detailed explanatory text, if necessary. Wrap it to about
4 72 characters or so. In some contexts, the first line is treated
5 as the subject of an email and the rest of the text as the body.
6 The blank line separating the summary from the body is critical
7 (unless you omit the body entirely); tools like rebase can get
8 confused if you run the two together.
9
10 Further paragraphs come after blank lines.
11
12 - Bullet points are okay, too
13 - Typically a hyphen or asterisk is used for the bullet, preceded
14   by a single space, with blank lines in between, but conventions
15   vary here
```

Quelle: “A Note About Git Commit Messages” by Tim Pope on [tbagery.com](#)

Denken Sie sich die Commit-Message als E-Mail an einen zukünftigen Entwickler, der das in fünf Jahren liest!

Vom Aufbau her hat eine E-Mail auch eine Summary und dann den eigentlichen Inhalt ... Erklären Sie das **“WARUM”** der Änderung! (Das “WER”, “WAS”, “WANN” wird bereits automatisch von Git aufgezeichnet ...)

Lesen (und beachten) Sie unbedingt auch [“How to Write a Git Commit Message”](#)!

2.1.18. Ausflug “Conventional Commits”

Die Commit-Messages dienen vor allem der Dokumentation und werden von Entwicklern gelesen.

Wenn man die Messages ein wenig stärker formalisieren würde, dann könnte man diese aber auch mit Tools verarbeiten und beispielsweise automatisiert Changelogs oder Release-Texte verfassen!

Betrachten Sie einmal das Projekt [ConventionalCommits.org](#). Dies ist ein solcher Versuch, die Commit-Messages (a) einheitlicher und lesbarer zu gestalten und (b) auch eine Tool-gestützte Auswertung zu erlauben.

Das Projekt schlägt als Erweiterung der üblichen Regeln zum Formatieren von Commit-Messages vor, dass in der ersten Zeile der *Summary* noch eine Abkürzung für die in diesem Commit erfolgte Änderung (Bug-Fix, neues Feature, ...) vorangestellt wird. Dieser Abkürzung kann in Klammern noch der Scope der Änderung hinzugefügt werden, beispielsweise den Bereich im Projekt, der von diesem Commit berührt wird. Wenn es eine *breaking change* ist, also alter Code nach dieser Änderung sich anders verhält oder vielleicht sogar nicht mehr kompiliert, wird noch ein “!” hinter dem Typ der Änderung ergänzt.

Beispiel: Stellen Sie sich vor, im Dungeon-Projekt wurde ein neues Verhalten hinzugefügt.

1. Normalerweise hätten Sie vielleicht diese Message geschrieben (angepasste Version aus [Dungeon-CampusMinden/Dungeon/pull/469](#)):

```
1  add fight skill
2
3  -  `DamageProjectileSkill` creates a new entity which causes `HealthDamage`
    -  when hitting another entity
4  -  `FireballSkill` is a more concrete implementation of this
5  -  Melee skills can be created with `DamageProjectileSkill` using a
    -  customised range
6      -  Example: the `FireballSkill` has a range of 10, a melee would have
    -  a considerably smaller range
7
8  fixes #24
9  fixes #126
10 fixes #224
```

2. Mit [ConventionalCommits.org](#) könnte das dann so aussehen:

```
1  feat: add fight skill
2
3  -  `DamageProjectileSkill` creates a new entity which causes `HealthDamage`
    -  when hitting another entity
4  -  `FireballSkill` is a more concrete implementation of this
5  -  Melee skills can be created with `DamageProjectileSkill` using a
    -  customised range
```



```

6      -   Example: the `FireballSkill` has a range of 10, a melee would have
          a considerably smaller range
7
8  fixes #24
9  fixes #126
10 fixes #224

```

Da es sich um ein neues Feature handelt, wurde der Summary in der ersten Zeile ein **feat**: vorangestellt.

Die zu verwendenden Typen/Abkürzungen sind im Prinzip frei definierbar. Das Projekt [ConventionalCommits.org](https://conventionalcommits.org) schlägt eine Reihe von Abkürzungen vor. Auf diese Weise sollen in möglichst allen Projekten, die Conventional Commits nutzen, die selben Abkürzungen/Typen eingesetzt werden und so eine Tool-gestützte Auswertung möglich werden.

3. Oder zusätzlich mit dem Scope der Änderung:

```

1 feat(game): add fight skill
2
3 - `DamageProjectileSkill` creates a new entity which causes `HealthDamage`
  when hitting another entity
4 - `FireballSkill` is a more concrete implementation of this
5 - Melee skills can be created with `DamageProjectileSkill` using a
  customised range
6   - Example: the `FireballSkill` has a range of 10, a melee would have
     a considerably smaller range
7
8  fixes #24
9  fixes #126
10 fixes #224

```

Der Typ **feat** wurde hier noch ergänzt um einen frei definierbaren Identifier für den Projektbereich. Dieser wird in Klammern direkt hinter den Typ notiert (hier **feat(game)** :).

Im Beispiel habe ich als Bereich “game” genommen, weil die Änderung sich auf den Game-Aspekt des Projekts bezieht. Im konkreten Projekt wären andere Bereiche eventuell “dsl” (für die im Projekt entwickelte Programmiersprache plus Interpreter) und “blockly” (für die Integration von Google Blockly zur Programmierung des Dungeons mit LowCode-Ansätzen). Das ist aber letztlich vom Projekt abhängig und weitestgehend Geschmackssache.

4. Oder zusätzlich noch als Auszeichnung “breaking change” (hier mit **scope**, geht aber auch ohne **scope**):

```

1 feat(game)!: add fight skill
2
3 - `DamageProjectileSkill` creates a new entity which causes `HealthDamage`
  when hitting another entity
4 - `FireballSkill` is a more concrete implementation of this
5 - Melee skills can be created with `DamageProjectileSkill` using a
  customised range
6   - Example: the `FireballSkill` has a range of 10, a melee would have
     a considerably smaller range
7
8  fixes #24
9  fixes #126
10 fixes #224

```

Angenommen, das neue Feature muss in der API etwas ändern, so dass existierender Code nun nicht mehr funktionieren würde. Dies wird mit dem extra Ausrufezeichen hinter dem Typ/Scope kenntlich gemacht (hier `feat(game) !` :).

Zusätzlich kann man einen “Footer” in die Message einbauen, also eine extra Zeile am Ende, die mit dem String “BREAKING CHANGE:” eingeleitet wird. (vgl. [Conventional Commits > Examples](#))

Es gibt noch viele weitere Initiativen, Commit-Messages lesbarer zu gestalten und zu vereinheitlichen. Schauen Sie sich beispielsweise einmal [gitmoji.dev](#) an. (Mit einem Einsatz in einem professionellen Umfeld wäre ich hier aber sehr ... vorsichtig.)

2.1.19. Wrap-Up

- Anlegen eines lokalen Repos mit `git init`
- Clonen eines existierenden Repos mit `git clone <url>`
- Änderungen einpflegen zweistufig (`git add`, `git commit`)
- Status der Workingcopy mit `git status` ansehen
- Logmeldungen mit `git log` ansehen
- Änderungen auf einem File mit `git diff` bzw. `git blame` ansehen
- Projektstand markieren mit `git tag`
- Ignorieren von Dateien/Ordern: Datei `.gitignore`

Zum Nachlesen

Sie finden den Inhalt dieser Sitzung im Chacon und Straub (2014, Kap. 1 und 2).

Zusätzlich gibt es viele hilfreiche Tutorials wie beispielsweise die [Atlassian Git Tutorials](#). Auf [GitHub Training Kit](#) finden Sie ein gutes Cheat-Sheet.

Lernziele

- k1: Ich kenne verschiedene Varianten der Versionierung
- k1: Ich kann die Begriffe ‘Workingcopy’ und ‘Repository’ und ‘Index’ erklären
- k2: Ich kann zwischen ‘Github’ und ‘Git’ unterscheiden
- k2: Ich kann auf meinem Rechner lokale Git-Repositories anlegen
- k3: Ich kann mit den Git-Befehlen zum Anlegen von lokalen Repos auf der Konsole umgehen
- k3: Ich kann Dateien zur Versionskontrolle hinzufügen bzw. aus der Versionierung löschen
- k3: Ich kann Änderungen (geänderte Dateien) zum Staging hinzufügen und committen
- k3: Ich kann Unterschiede zwischen Commits herausfinden und mir die Historie des Repos anschauen
- k3: Ich kann gezielt Dateien und Ordner von der Versionierung ausnehmen (ignorieren)

Challenges

Versionierung 101

1. Legen Sie ein Repository an.
2. Fügen Sie Dateien dem Verzeichnis hinzu und stellen Sie *einige* davon unter Versionskontrolle.

3. Ändern Sie eine Datei und versionieren Sie die Änderung.
4. Was ist der Unterschied zwischen `git add .; git commit` und `git commit -a`?
5. Wie finden Sie heraus, welche Dateien geändert wurden?
6. Entfernen Sie eine Datei aus der Versionskontrolle, aber nicht aus dem Verzeichnis!
7. Entfernen Sie eine Datei komplett (Versionskontrolle und Verzeichnis).
8. Ändern Sie eine Datei und betrachten die Unterschiede zum letzten Commit.
9. Fügen Sie eine geänderte Datei zum Index hinzu. Was erhalten Sie bei `git diff <datei>`?
10. Wie können Sie einen früheren Stand einer Datei wiederherstellen? Wie finden Sie überhaupt den Stand?
11. Legen Sie sich ein Java-Projekt in Ihrer IDE an an. Stellen Sie dieses Projekt unter Git-Versionskontrolle. Führen Sie die vorigen Schritte mit Ihrer IDE durch.

2.2. Git2: Branches und Remotes

TL;DR

Die Commits in Git bauen aufeinander auf und bilden dabei eine verkettete "Liste". Diese "Liste" nennt man auch *Branch* (Entwicklungszweig). Beim Initialisieren eines Repositories wird automatisch ein Default-Branch angelegt, auf dem die Commits dann eingefügt werden.

Weitere Branches kann man mit `git branch` anlegen, und die Workingcopy kann mit `git switch` oder `git checkout` auf einen anderen Branch umgeschaltet werden. Auf diese Weise kann man an mehreren Features parallel arbeiten, ohne dass die Arbeiten sich gegenseitig stören.

Zum Mergen (Vereinigen) von Branches gibt es `git merge`. Dabei werden die Änderungen im angegebenen Branch in den aktuell in der Workingcopy ausgecheckten Branch integriert und hier ggf. ein neuer Merge-Commit erzeugt. Falls es in beiden Branches inkompatible Änderungen an der selben Stelle gab, entsteht beim Mergen ein Merge-Konflikt. Dabei zeigt Git in den betroffenen Dateien jeweils an, welche Änderung aus welchem Branch stammt und man muss diesen Konflikt durch Editieren der Stellen manuell beheben.

Mit `git rebase` kann die Wurzel eines Branches an eine andere Stelle verschoben werden. Dies wird später bei Workflows eine Rolle spielen.

Beim Klonen eines Repositories mit `git clone <url>` wird das fremde Repo mit dem Namen `origin` im lokalen Repo festgehalten. Dieser Name wird auch als Präfix für die Branches in diesem Repo genutzt, d.h. die Branches im Remote-Repo tauchen als `origin/<branch>` im lokalen Repo auf. Diese Remote-Branches kann man nicht direkt bearbeiten, sondern man muss diese Remote-Branches in einem lokalen Branch auschecken und dann darin weiterarbeiten. Es können beliebig viele weitere Remotes dem eigenen Repository hinzugefügt werden.

Änderungen aus einem Remote-Repo können mit `git fetch <remote>` in das lokale Repo geholt werden. Dies aktualisiert **nur** die Remote-Branches `<remote>/<branch>`! Die Änderungen können anschließend mit `git merge <remote>/<branch>` in den aktuell in der Workingcopy ausgecheckten Branch gemergt werden. (*Anmerkung:* Wenn mehrere Personen an einem Branch arbeiten, will man die eigenen Arbeiten in dem Branch vermutlich eher auf den aktuellen Stand des Remote **rebasen** statt mergen!) Eigene Änderungen können mit `git push <remote> <branch>` in das Remote-Repo geschoben

werden.

Videos

Vorlesung:

- Teil 1: Branches [YT], [HSBI]
- Teil 2: Remotes [YT], [HSBI]

Demos:

- Anlegen und Mergen von Branches [YT], [HSBI]
- Auflösen von Merge-Konflikten [YT], [HSBI]
- HEAD [YT], [HSBI]
- Fetch, Pull und Push [YT], [HSBI]
- Verknüpfen weiterer Remotes [YT], [HSBI]

[Introduction to Git with Scott Chacon of GitHub \(zweiter Teil, ab ca. Minute 45\)](#)

2.2.1. Neues Feature entwickeln/ausprobieren

```
1 A---B---C master
```

- Bisher nur lineare Entwicklung: Commits bauen aufeinander auf (lineare Folge von Commits)
- `master` ist der (Default-) Hauptentwicklungsweig
 - Pointer auf letzten Commit
 - Default-Name: “`master`” (muss aber nicht so sein bzw. kann geändert werden)

Anmerkung: Git und auch Github haben den Namen für den Default-Branch von `master` auf `main` geändert. Der Name an sich ist aber für Git bedeutungslos und kann mittels `git config --global init.defaultBranch <name>` geändert werden. In Github hat der Default-Branch eine gewisse Bedeutung, beispielsweise ist der Default-Branch das automatische Ziel beim Anlegen von Pull-Requests. In Github kann man den Default-Namen global in den User-Einstellungen (Abschnitt “Repositories”) und für jedes einzelne Repository in den Repo-Einstellungen (Abschnitt “Branches”) ändern.

Entwicklung des neuen Features soll stabilen `master`-Branch nicht beeinflussen => Eigenen Entwicklungsweig für die Entwicklung des Features anlegen:

1. Neuen Branch erstellen: `git branch wuppie`
2. Neuen Branch auschecken: `git checkout wuppie` oder `git switch wuppie`

Alternativ: `git checkout -b wuppie` oder `git switch -c wuppie` (neuer Branch und auschecken in einem Schritt)

```
1 A---B---C master, wuppie
```

Startpunkt: prinzipiell beliebig (jeder Commit in der Historie möglich).

Die gezeigten Beispiel zweigen den neuen Branch direkt vom aktuell ausgecheckten Commit/Branch ab. Also aufpassen, was gerade in der Workingcopy los ist!

Alternativ nutzen Sie die Langform: `git branch wuppie master` (mit `master` als Startpunkt; hier kann jeder beliebige Branch, Tag oder Commit genutzt werden).

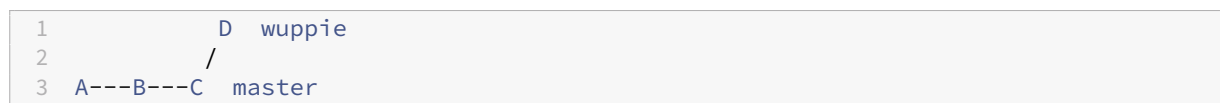
Nach Anlegen des neuen Branches zeigen beide Pointer auf den selben Commit.

`git switch <branchname>` bzw. `git checkout <branchname>` holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. Man kann also jederzeit in der Workingcopy die Branches wechseln und entsprechend weiterarbeiten.

Anmerkung: In neueren Git-Versionen wurde der Befehl “**switch**” eingeführt, mit dem Sie in der Workingcopy auf einen anderen Branch wechseln können. Der bisherige Befehl “**checkout**” funktioniert aber weiterhin. Während der neue `git switch`-Befehl allerdings nur Branches umschalten kann, funktioniert `git checkout` sowohl mit Branchnamen und Dateinamen - damit kann man also auch eine andere Version einer Datei in der Workingcopy “auschecken”. Falls gleiche Branch- und Dateinamen existieren, muss man für das Auschecken einer Datei noch “--” nutzen: `git checkout -- <dateiname>`.

...

Commit(s) auf `wuppie` ...



- Entwicklung des neuen Features erfolgt im eigenen Branch: beeinflusst den stabilen `master`-Branch nicht
- Wenn in der Workingcopy der Feature-Branch ausgecheckt ist, gehen die Commits in den Feature-Branch; der `master` bleibt auf dem alten Stand
- Wenn der `master` ausgecheckt wäre, würden die Änderungen in den `master` gehen, d.h. der `master` würde sich ab Commit `C` parallel zu `wuppie` entwickeln

2.2.2. Problem: Fehler im ausgelieferten Produkt

Fix für `master` nötig:

1. `git checkout master`
2. `git checkout -b fix`
3. Änderungen in `fix` vornehmen ...

Das führt zu dieser Situation:



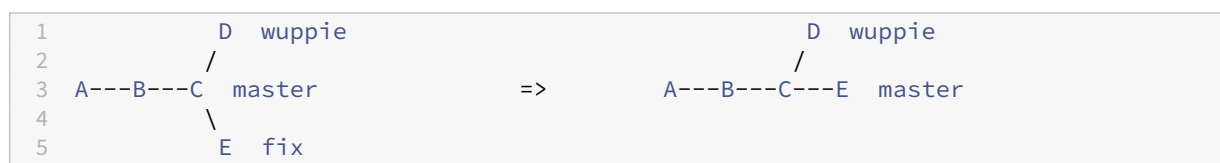
`git checkout <branchname>` holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. (Das geht in neueren Git-Versionen auch mit `git switch <branchname>`.)

Man kann weitere Branches anlegen, d.h. hier im Beispiel ein neuer Feature-Branch `fix`, der auf dem `master` basiert. Analog könnte man auch Branches auf der Basis von `wuppie` anlegen ...

2.2.3. Fix ist stabil: Integration in *master*

1. `git checkout master`
2. `git merge fix` => **fast forward** von `master`
3. `git branch -d fix`

Der letzte Schritt entfernt den Branch `fix`.



- Allgemein: `git merge <branchname>` führt die Änderungen im angegebenen Branch `<branchname>` in den aktuell in der Workingcopy ausgecheckten Branch ein. Daraus resultiert für den aktuell ausgecheckten Branch ein neuer Commit, der Branch `<branchname>` bleibt dagegen auf seinem bisherigen Stand.

Beispiel:

- Die Workingcopy ist auf `A`
- `git merge B` führt `A` und `B` zusammen: `B` wird **in A** gemergt
- Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in `A` statt!

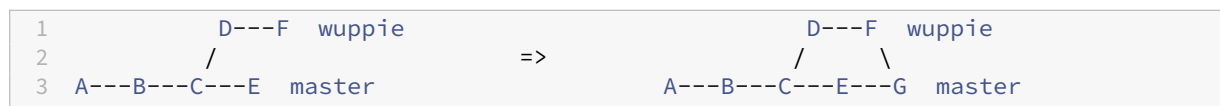
In der Abbildung ist `A` der `master` und `B` der `fix`.

- Nach dem Merge existieren beide Branches weiter (sofern sie nicht explizit gelöscht werden)
- Hier im Beispiel findet ein sogenannter “Fast forward” statt.

“Fast forward” ist ein günstiger Spezialfall beim Merge: Beide Branches liegen in einer direkten Kette, d.h. der Zielbranch kann einfach “weitergeschaltet” werden. Ein Merge-Commit ist in diesem Fall nicht notwendig und wird auch nicht angelegt.

2.2.4. Feature weiter entwickeln und Integration in *master*

1. `git checkout master`
2. `git merge wuppie`
=> Kein *fast forward* möglich: Git sucht nach gemeinsamen Vorgänger + neuer Commit



Hier im Beispiel ist der Standardfall beim Mergen dargestellt: Die beiden Branches liegen nicht in einer direkten Kette von Commits, d.h. hier wurde parallel weitergearbeitet.

Git sucht in diesem Fall nach dem gemeinsamen Vorgänger beider Branches und führt die jeweiligen Änderungen (Differenzen) seit diesem Vorgänger in einem Merge-Commit zusammen.

Im `master` entsteht ein neuer Commit, da kein *fast forward* beim Zusammenführen der Branches möglich!

Anmerkung: `git checkout wuppie; git merge master` würde den `master` in den `wuppie` mergen, d.h. der Merge-Commit wäre dann in `wuppie`.

Beachten Sie dabei die "Merge-Richtung":

- Die Workingcopy ist auf `A`
- `git merge B` führt `A` und `B` zusammen: `B` wird **in** `A` gemergt
- Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in `A` statt!

In der Abbildung ist `A` der `master` und `B` der `wuppie`.

Achtung: Richtung beachten! `git checkout A; git merge B` führt beide Branches zusammen, genauer: führt die Änderungen von `B` in `A` ein, d.h. der entsprechende Merge-Commit ist in `A`!

2.2.5. Konflikte beim Mergen

Merksatz: (Parallele) Änderungen an selber Stelle => Merge-Konflikte

```

1 $ git merge wuppie
2 Auto-merging hero.java
3 CONFLICT (content): Merge conflict in hero.java
4 Automatic merge failed; fix conflicts and then commit the result.

```

Git fügt Konflikt-Marker in die Datei ein:

```

1 <<<<<< HEAD:hero.java
2 public void getActiveAnimation() {
3     return null;
4     =====
5 public Animation getActiveAnimation() {
6     return this.idleAnimation;
7 >>>>>> wuppie:hero.java

```

- Der Teil mit `HEAD` ist aus dem aktuellen Branch in der Workingcopy
- Der Teil aus dem zu mergenden Branch ist unter `wuppie` notiert
- Das `=====` trennt beide Bereiche

2.2.6. Merge-Konflikte auflösen

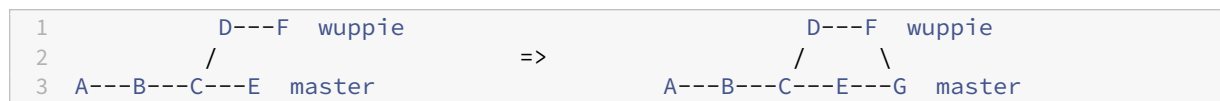
Manuelles Editieren nötig (Auflösung des Konflikts):

- 1) Entfernen der Marker
- 2) Hinzufügen der Datei zum Index
- 3) Analog für restliche Dateien mit Konflikt
- 4) Commit zum Abschließen des Merge-Vorgangs

Alternativ: Nutzung graphischer Oberflächen mittels `git mergetool`

[Konsole: Branchen und Mergen](#)

2.2.7. Rebases: Verschieben von Branches

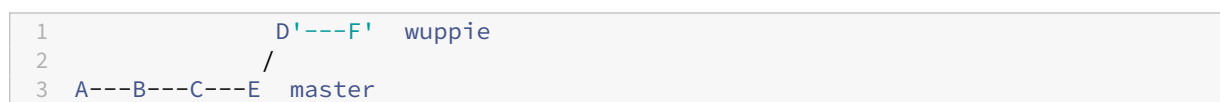


Bisher haben wir Branches durch Mergen zusammengeführt. Dabei entsteht in der Regel ein extra Merge-Commit (im Beispiel G), außer es handelt sich um ein *fast forward*. Außerdem erkennt man in der Historie sehr gut, dass hier in einem separaten Branch gearbeitet wurde, der irgendwann in den `master` gemergt wurde.

Leider wird dieses Vorgehen in großen Projekten recht schnell sehr unübersichtlich. Außerdem werden Merges in der Regeln nur von besonders berechtigten Personen (Manager) durchgeführt, die im Falle von Merge-Konflikten diese dann selbst auflösen müssten (ohne aber die fachliche Befähigung zu haben). Hier greift man dann häufig zur Alternative *Rebase*. Dabei wird der Ursprung eines Branches auf einen bestimmten Commit verschoben. Im Anschluss ist dann ein Merge mit *fast forward*, also ohne die typischen rautenförmigen Ketten in der Historie und ohne extra Merge-Commit möglich. Dies kann aber auch als Nachteil gesehen werden, da man in der Historie den früheren Branch nicht mehr erkennt! Ein weiterer schwerwiegender Nachteil ist, dass alle Commits im verschobenen Branch umgeschrieben werden und damit neue Commit-IDs bekommen. Das verursacht bei der Zusammenarbeit in Projekten massive Probleme! Als Vorteil gilt, dass man mögliche Merge-Konflikte bereits beim Rebasen auflösen muss, d.h. hier muss derjenige, der den Merge “beantragt”, durch einen vorherigen Rebase den konfliktfreien Merge sicherstellen. Mehr dazu in “Branching-Strategien & Git-Workflows”.

```
1  git rebase master wuppie
```

führt zu



Nach dem Rebase von `wuppie` auf `master` sieht es so aus, als ob der Branch `wuppie` eben erst vom `master` abgezweigt wurde. Damit ist dann ein *fast forward* Merge von `wuppie` in den `master` möglich, d.h. es gibt keine Raute und auch keinen extra Merge-Commit (hier nicht gezeigt).

Man beachte aber die Änderung der Commit-IDs von `wuppie`: Aus `D` wird `D'`! (Datum, Ersteller und Message bleiben aber erhalten.)

2.2.8. Don't lose your HEAD

- Branches sind wie Zeiger auf letzten Stand (Commit) eines Zweiges
- `HEAD`: Spezieller Pointer
 - Zeigt auf den aktuellen Branch der Workingcopy

- Früheren Commit auschecken (ohne Branch): “headless state”

- Workingcopy ist auf früherem Commit
- Kein Branch => Änderungen gehen verloren!

Eventuelle Änderungen würden ganz normal als Commits auf dem `HEAD`-Branch aufgezeichnet. Sobald man aber einen anderen Branch auscheckt, wird der `HEAD` auf diesen anderen Branch gesetzt, so dass die eben gemachten Commits “in der Luft hängen”. Sofern man die SHA's kennt, kommt man noch auf die Commits zurück. Allerdings laufen von Zeit zu Zeit interne Aufräum-Aktionen, so dass die Chance gut steht, dass die “kopflosen” Commits irgendwann tatsächlich verschwinden.

2.2.9. Erinnerung: Clonen kann sich lohnen => Remote-Branches

```
1 https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture
2
3 ---C---D---E master
```

=> `git clone https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture`

```
1 ./Prog2-Lecture/ (lokaler Rechner)
2
3 ---C---D---E master
4          ^origin/master
```

Erinnerung: Git-Repository mit der URL `<URL-Repo>` in lokalen Ordner `<directory>` auschecken: `git clone <URL-Repo> [<directory>]`.

Für die URL sind verschiedene Protokolle möglich, beispielsweise:

- “`file://`” für über das Dateisystem erreichbare Repositories (ohne Server)
- “`https://`” für Repo auf einem Server: Authentifikation mit Username und Passwort (!)

- “`git@`” für Repo auf einem Server: Authentifikation mit **SSH-Key** (diese Variante wird im Praktikum im Zusammenspiel mit dem Gitlab-Server im SW-Labor verwendet)

Neue Beobachtung: Die Workingcopy ist automatisch über den Namen `origin` mit dem Remote-Repo (gern auch “Remote” oder “Upstream” genannt) auf dem Server verbunden. Der lokale Branch `master` ist automatisch mit dem Remote-Branch `origin/master` verbunden, der den Stand des `master`-Branches auf dem Server spiegelt.

Mit `git remote add <name> <url>` kann man beliebig viele weitere Remotes hinzufügen. Das Arbeiten mit den weiteren Remotes unterscheidet sich nicht von dem hier gezeigten Vorgehen mit dem Default-Remote `origin`.

Hinweis zum SSH-Protokoll (“`git@`”): Häufig kann man über das “`https://`”-Protokoll zwar Repos klonen und lokal bearbeiten, aber die Änderungen nicht mehr auf den Server zurück pushen, weil die Authentifikation mit Username und Passwort als unsicher betrachtet wird und von den Anbietern deaktiviert wurde/wird. Das SSH-Protokoll (also die “`git@`-URLs”) arbeitet dagegen mit **SSH-Keys**, die man sich beispielsweise gezielt nur für die Authentifikation bei GitHub anlegen kann und den öffentlichen Schlüssel (*public key*) in den User-Einstellungen von GitHub registrieren kann: “[Generating a new SSH key and adding it to the ssh-agent](#)” und “[Adding a new SSH key to your GitHub account](#)” (bitte darauf achten, dass das richtige Betriebssystem ausgewählt ist).

2.2.10. Eigener und entfernter `master` entwickeln sich weiter ...

```
1 https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture
2
3 ---C---D---E---F---G master
```

```
1 ./Prog2-Lecture/ (lokaler Rechner)
2
3 ---C---D---E---H master
4           ^origin/master
```

Nach dem Auschecken liegen (in diesem Beispiel) drei `master`-Branches vor:

1. Der `master` auf dem Server,
2. der lokale `master`, und
3. die lokale Referenz auf den `master`-Branch auf dem Server: `origin/master`.

Der lokale `master` ist ein normaler Branch und kann durch Commits verändert werden.

Der `master` auf dem Server kann sich ebenfalls ändern, beispielsweise weil jemand anderes seine lokalen Änderungen mit dem Server abgeglichen hat (`git push`, s.u.).

Der Branch `origin/master` lässt sich nicht direkt verändern! Das ist lediglich eine lokale Referenz auf den `master`-Branch auf dem Server und zeigt an, welchen Stand man bei der letzten Synchronisierung hatte. D.h. erst mit dem nächsten Abgleich wird sich dieser Branch ändern (sofern sich der entsprechende Branch auf dem Server verändert hat).

Anmerkung: Dies gilt analog für alle anderen Branches. Allerdings wird nur der `origin/master` beim Clonen automatisch als lokaler Branch ausgecheckt.

Zur Abbildung: Während man lokal arbeitet (Commit **H** auf dem lokalen **master**), kann es passieren, dass sich auch das remote Repo ändert. Im Beispiel wurden dort die beiden Commits **F** und **G** angelegt (durch **git push**, s.u.).

Wichtig: Da in der Zwischenzeit das lokale Repo nicht mit dem Server abgeglichen wurde, zeigt der remote Branch **origin/master** immer noch auf den Commit **E**!

2.2.11. Änderungen vom Remote holen und Branches lokal zusammenführen

```
1 https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture
2
3 ---C---D---E---F---G master
```

=> **git fetch origin**

```
1 ./Prog2-Lecture/ (lokaler Rechner)
2
3 ---C---D---E---H master
4                \
5                F---G origin/master
```

2.2.11.1. Änderungen auf dem Server mit dem eigenen Repo abgleichen

Mit **git fetch origin** alle Änderungen holen

- Alle remote Branches werden aktualisiert und entsprechen den jeweiligen Branches auf dem Server: Im Beispiel zeigt jetzt **origin/master** ebenso wie der **master** auf dem Server auf den Commit **G**.
- Neue Branches auf dem Server werden ebenfalls “geholt”, d.h. sie liegen nach dem Fetch als entsprechende remote Branches vor
- Auf dem Server gelöschte Branches werden nicht automatisch lokal gelöscht; dies kann man mit **git fetch --prune origin** automatisch erreichen

Wichtig: Es werden nur die remote Branches aktualisiert, nicht die lokalen Branches!

2.2.11.2. master-Branch nach “git fetch origin” zusammenführen

```
1 https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture
2
3 ---C---D---E---F---G master
4
5
6
7 ./Prog2-Lecture/ (lokaler Rechner)
8
9 ---C---D---E---H---I master
10                \   /
11                F---G origin/master
```

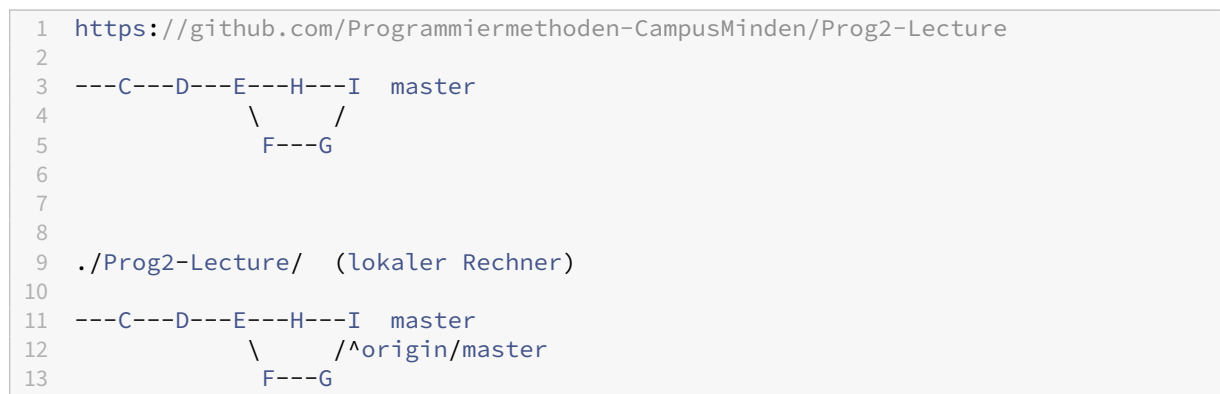
1. Mit `git checkout master` Workingcopy auf eigenen `master` umstellen
2. Mit `git merge origin/master` Änderungen am `origin/master` in eigenen `master` mergen

Anmerkung: Schritt (2) kann man auch direkt per `git pull origin master` erledigen ... Ein `pull` fasst `fetch` und `merge` zusammen.

Anmerkung Statt dem `merge` in Schritt (2) kann man auch den lokalen `master` auf den aktualisierten `origin/master` rebasen und vermeidet damit die "Raute". Der `pull` kann mit der Option "`--rebase`" auf "rebase" umgestellt werden (per Default wird bei `pull` ein "merge" ausgeführt).

2.2.11.3. master-Branch ins Remote pushen

Mit `git push origin master` eigene Änderungen ins Remote-Repo pushen



2.2.11.4. Achtung: Auf dem Server ist nur ein fast forward merge möglich

Sie können Ihre Änderungen in Ihrem lokalen `master` auch direkt in das remote Repo pushen, solange auf dem Server ein **fast forward merge** möglich ist.

Wenn aber (wie in der Abbildung) der lokale und der remote `master` divergieren, müssen Sie den Merge wie beschrieben lokal durchführen (`fetch/merge` oder `pull`) und das Ergebnis wieder in das remote Repo pushen (dann ist ja wieder ein *fast forward merge* möglich, es sei denn, jemand hat den remote `master` in der Zwischenzeit weiter geschoben - dann muss die Aktualisierung erneut durchgeführt werden).

Beispiel für Zusammenführen (merge und push), Anmerkung zu fast forward merge

2.2.12. Branches und Remotes

Das Arbeiten mit Remote-Branche ist nicht nur bereits beim Klonen vorhandene Branches beschränkt.

1. Neue lokale Branches pushen mit `git push <remote> <branch>`.
2. Neue remote Branches fetchen:
 - `git fetch <remote>` holt (auch) alle neuen (Remote-) Branches

- Lokale Änderungen an Remote-Branched nicht möglich!
=> **Remote-Branch in lokalen Branch auschecken**

2.2.12.1. Anmerkung: *push* geht nur, wenn

1. Ziel ein “bare”-Repository ist, **und**
2. keine Konflikte entstehen

=> im remote Repo **nur** “fast forward”-Merge möglich

=> bei Konflikten erst *fetch* und *merge*, danach *push*

Anmerkung: Ein “bare”-Repository enthält keine Workingcopy, sondern nur das Repo an sich. Die ist bei Repos, die Sie auf einem Server wie Gitlab oder Github anlegen, automatisch der Fall. Sie können aber auch lokal ein solches “bare”-Repo anlegen, indem Sie beim Initialisieren den Schalter `--bare` mitgeben: `git init --bare ...`

2.2.12.2. Beispiel

```
1 git fetch origin          # alle Änderungen vom Server holen
2 git checkout master       # auf lokalen Master umschalten
3 git merge origin/master   # lokalen Master aktualisieren
4
5 ... # Herumspielen am lokalen Master
6
7 git push origin master    # lokalen Master auf Server schicken
```

2.2.13. Wrap-Up

- Anlegen von Branches mit `git branch`
- Umschalten der Workingcopy auf anderen Branch: `git checkout` oder `git switch`
- Mergen von Branches und Auflösen von Konflikten: `git merge`
- Verschieben von Branches mit `git rebase`
- Änderungen vom Server holen: `git fetch <remote>`
- Lokalen Branch auffrischen: `git merge <remote>/<branch>`
- Eigene Änderungen hochladen: `git push <remote> <branch>`

Zum Nachlesen

Sie finden den Inhalt dieser Sitzung im Chacon und Straub (2014, Kap. 3).

Zusätzlich gibt es viele hilfreiche Tutorials wie beispielsweise die [Atlassian Git Tutorials](#).

Lernziele

- k3: Ich kann neue Branches anlegen
- k3: Ich kann Branches mergen und mögliche Konflikte auflösen

- k3: Ich kann Branches rebasen
- k3: Ich kann Änderungen vom fremden Repo holen
- k2: Ich kann den Unterschied zwischen lokalen Branches und entfernten Branches
- k3: Ich kann meine lokale Branches aktualisieren
- k3: Ich kann lokale Änderungen ins fremde Repo pushen erklären

Challenges

Branches und Merges

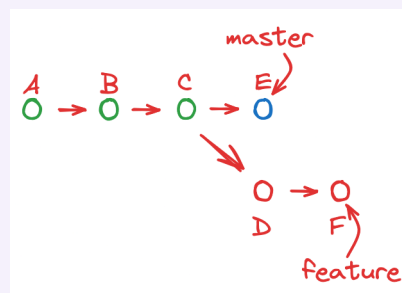
1. Legen Sie in Ihrem Projekt einen Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und mergen Sie die Änderungen. Was beobachten Sie?
2. Legen Sie einen weiteren Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und ändern Sie dort ebenfalls:
 - Ändern Sie eine Datei an einer Stelle, die nicht bereits im Branch modifiziert wurde.
 - Ändern Sie eine Datei an einer Stelle, die bereits im Branch manipuliert wurde.

Commiten Sie die Änderungen.

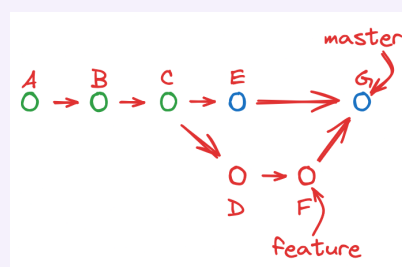
Mergen Sie den Branch jetzt in den Master-Branch. Was beobachten Sie? Wie lösen Sie Konflikte auf?

Mergen am Beispiel

Sie verwalten Ihr Projekt mit Git. Es existieren zwei Branches: **master** (zeigt auf Commit *C*) und **feature** (zeigt auf Version *F*). In Ihrer Workingcopy haben Sie den Branch **feature** ausgecheckt:



- (1) Mit welcher Befehlsfolge können Sie den Branch **feature** in den Branch **master** mergen, so dass nach dem Merge die im folgenden Bild dargestellte Situation entsteht?

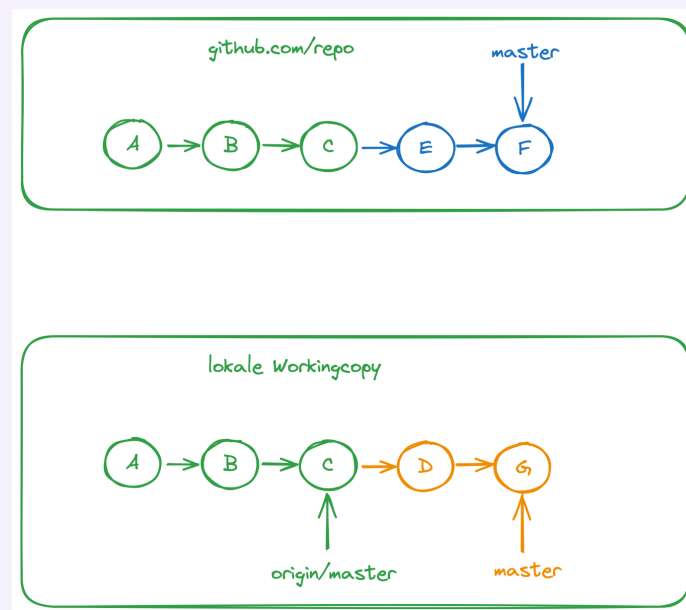


(Der Merge läuft ohne Konflikte ab. Es ist irrelevant, welcher Branch am Ende in der Workingcopy ausgecheckt ist.)

- (2) Wie können Sie erreichen, dass es keinen Merge-Commit gibt, sondern dass die Änderungen in *D* und *F* im *master* als eine lineare Folge von Commits erscheinen?

Synchronisierung mit Remote-Repos

Sie haben ein Repo von github.com geklont. Beide Repos, das Original auf dem Server als auch Ihre lokale Kopie, haben sich danach unabhängig voneinander weiter entwickelt (siehe Skizze).



Wie können Sie Ihre Änderung im lokalen Repo auf den Server pushen? Analysieren Sie die Situation und erklären Sie zwei verschiedene Lösungsansätze und geben Sie jeweils die entsprechenden Git-Befehle an.

Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- [Learn Git Branching](#)
- [Oh My Git!](#)
- [Git Time](#)
- [Tutorial: A Visual Git Reference](#)

2.3. Git4: Worktree

TL;DR

Git Worktree erlaubt es, Branches in separaten Ordnern auszuchecken. Diese Ordner sind mit der Workingcopy verknüpft, d.h. alle Änderungen über Git-Befehle werden automatisch mit der Workingcopy "synchronisiert". Im Unterschied zum erneuten Clonen hat man in den verknüpften Ordnern aber nicht die gesamte Historie noch einmal neu als `.git`-Ordner, sondern nur den Link auf die Workingcopy, wodurch

viel Platz gespart wird. Damit bilden Git Worktrees eine elegante Möglichkeit, parallel an verschiedenen Branches zu arbeiten.

Videos

Vorlesung Git Worktree [YT], [HSBI]

Demo Git Worktree [YT], [HSBI]

2.3.1. Git Worktree - Mehrere Branches parallel auschecken

2.3.1.1. Szenario

- Sie arbeiten an einem Projekt
- Großes Repo mit vielen Versionen und Branches
- Ungesicherte Änderungen im Featurebranch
- Wichtige Bugfixes an alter Version nötig

2.3.1.2. Lösungsansätze

- `git stash` nutzen und Branch wechseln
- Repo erneut in anderem Ordner auschecken

2.3.1.3. Probleme

1. `git stash` und `git switch`

Funktioniert für die meisten Fälle relativ gut und ist daher die “Lösung to go”.

Aber Sie müssen später aufpassen, dass Sie auch wirklich wieder im richtigen Branch sind, wenn Sie die Änderungen im Stash anwenden (`git stash pop`)! Und wenn Sie mehrere Einträge in der Stash-Liste haben, kann es recht schnell recht unübersichtlich werden - zu welchem Branch gehören welche Einträge in der Stash-Liste?

Außerdem kann es gerade in größeren Projekten passieren, dass sich die Konfiguration zwischenzeitlich ändert. Wenn Sie jetzt in der IDE einfach auf einen alten Stand mit einer anderen Konfiguration wechseln, kann es schnell passieren, dass sich die IDE “verschluckt” und Sie dadurch viel Arbeit haben.

2. Nochmal woanders auschecken

Im Prinzip ist das eine Möglichkeit. Sie können dann den anderen Ordner in Ihrer IDE als neues Projekt öffnen und sofort starten.

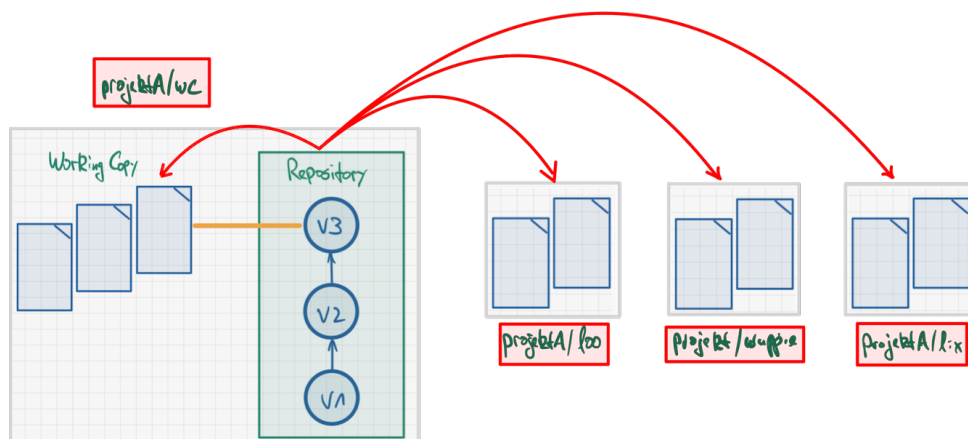
Aber: Sie benötigen noch einmal den Platz auf der Festplatte/SSD/... wie für die ursprüngliche Workingcopy! Das kann bei alten/großen Projekten schnell recht groß werden und Probleme verursachen.

Außerdem ist die Synchronisierung zwischen den beiden Workingcopies (der ursprünglichen und der neuen) nicht vorhanden bzw. das müssen Sie manuell per `git push` und `git pull` (in jeder Kopie des Repos!) erledigen!

2.3.1.4. Git Worktree kann helfen!

=> Mehrere Branches gleichzeitig auschecken (als neue Ordner im Dateisystem)

2.3.2. How to use Git Worktree



2.3.3. Worktree anlegen

```
git worktree add <path> <branch>
```

Legt neuen Ordner `<path>` an und checkt darin `<branch>` als “linked worktree” aus.

Mit `git worktree add ../wuppie foo` würden Sie also parallel zum aktuellen Ordner (wo Ihre Workingcopy enthalten ist) einen neuen Ordner `wuppie/` anlegen und darin den Branch `foo` auschecken.

Wenn Sie in den Ordner `wuppie` wechseln, finden Sie auch eine Datei `.git`. Darin ist lediglich der Pfad zur Workingcopy vermerkt, damit Git Änderungen auch in die eigentliche Workingcopy spiegeln kann. Dies ist der sogenannte “linked worktree”.

Im Vergleich dazu finden Sie in der eigentlichen Workingcopy einen Ordner `.git`, der üblicherweise die gesamte Historie etc. enthält und entsprechend groß werden kann.

Den Befehl `git worktree add` gibt es in verschiedenen Versionen. In der Kurzform `git worktree add <path>` würde ein neuer Branch angelegt und ausgecheckt, der der letzten Komponente von `<path>` entspricht ...

Warnung: Nicht in selben Ordner oder in Unterordner auschecken!

Die neuen Worktrees sollten immer **außerhalb** der Workingcopy liegen! Sie können Git sehr schnell sehr gründlich durcheinanderbringen, wenn Sie einen Worktree im selben Ordner oder in einem Unterordner anlegen.

`git worktree` sollte nach Möglichkeit nicht zusammen mit Git Submodules eingesetzt werden (unstabiles Verhalten)!

2.3.4. Worktree wechseln

- Worktrees anzeigen: `git worktree list`
- Worktree wechseln: Ordner wechseln (IDE: neues Projekt)

Die Worktrees sind aus Sicht des Dateisystems einfach Ordner. Die `.git`-Datei verlinkt für Git den Ordner mit der ursprünglichen Workingcopy.

Um also mit einem Worktree arbeiten zu können, wechseln Sie einfach das Verzeichnis. In einer IDE würden Sie entsprechend ein neues Projekt anlegen. So können Sie gleichzeitig in verschiedenen Branches arbeiten.

Änderungen in einem Worktree werden automatisch in die ursprüngliche Workingcopy gespiegelt. Analog können Sie in einem Worktree auf die aktuelle Historie aus der ursprünglichen Workingcopy zugreifen.

Hinweis: Sie können in den Ordnern zwar Branches wechseln, aber nicht auf einen Branch, der bereits in einem anderen Ordner (Worktree) ausgecheckt ist. Es ist gute Praxis, dass die Ordernamen dem ausgecheckten Branch (linked Worktree) entsprechen, um Verwirrungen zu vermeiden.

2.3.5. Worktree löschen

`git worktree remove <worktree>`

Sofern der Worktree "clean" ist, es also keine nicht comitteten Änderungen gibt, können Sie mit `git worktree remove <worktree>` einen Worktree `<worktree>` wieder löschen.

Dabei bleibt der Ordner erhalten - Sie können ihn selbst löschen oder später wiederverwenden.

2.3.6. Wrap-Up

Git Worktree: Auschecken von Branches in separate Ordner

- Anlegen: `git worktree add <path> <branch>`
- Anschauen: `git worktree list`
- Löschen: `git worktree remove <worktree>`

Zum Nachlesen

Eine gute Quelle zum Nachlesen bietet die [Git-Dokumentation](#).

Lernziele

- k2: Vorteile von Git Worktree
- k2: Prinzipielle Arbeitsweise von Git Worktree
- k3: Anlegen von Worktrees
- k3: Anzeigen von Worktrees
- k3: Löschen von Worktrees

2.4. Branching-Strategien & Git-Workflows

TL;DR

Das Erstellen und Mergen von Branches ist in Git besonders einfach. Dies kann man sich in der Entwicklung zunutze machen und die einzelnen Features unabhängig voneinander in eigenen Hilfs-Banches ausarbeiten.

Es haben sich zwei grundlegende Modelle etabliert: “Git-Flow” und “GitHub Flow”.

1. In **Git-Flow** gibt es ein umfangreiches Konzept mit verschiedenen Branches für feste Aufgaben, welches sich besonders gut für Entwicklungsmodelle mit festen Releases eignet. Es gibt zwei langlaufende Branches: `master` enthält den stabilen veröffentlichten Stand, in `develop` werden die Ergebnisse der Entwicklung gesammelt. Features werden in kleinen Feature-Banches entwickelt, die von `develop` abzweigen und dort wieder hineinmünden. Für Releases wird von `develop` ein eigener Release-Branch angelegt und nach Finalisierung in den `master` und in `develop` gemergt. Fixes werden vom `master` abgezweigt, und wieder in den `master` und auch nach `develop` integriert. Dadurch stehen auf dem `master` immer die stabilen Release-Stände zur Verfügung, und im `develop` sammeln sich die Entwicklungsergebnisse.
2. Der **GitHub Flow** basiert auf einem deutlich schlankeren Konzept und passt gut für die kontinuierliche Entwicklung ohne echte Releases. Hier hat man auch wieder einen `master` als langlaufenden Branch, der die stabilen Release-Stände enthält. Vom `master` zweigen direkt die kleinen Feature-Banches ab und werden auch wieder direkt in den `master` integriert.

Git erlaubt unterschiedliche Formen der Zusammenarbeit.

1. Bei kleinen Teams kann man einen einfachen zentralen Ansatz einsetzen. Dabei gibt es ein zentrales Repo auf dem Server, und alle Team-Mitglieder dürfen direkt in dieses Repo pushen. Hier muss man sich gut absprechen und ein vernünftiges Branching-Schema ist besonders wichtig.
2. In größeren Projekten gibt es oft ein zentrales öffentliches Repo, wo aber nur wenige Personen Schreibrechte haben. Hier forkt man sich dieses Repo, erstellt also eine öffentliche Kopie auf dem Server. Diese Kopie klonet man lokal und arbeitet hier und pusht die Änderungen in den eigenen öffentlich sichtbaren Fork. Um die Änderungen ins Projekt-Repo integrieren zu lassen, wird auf dem Server ein

sogenannter Merge-Request (Gitlab) bzw. Pull-Request (GitHub) erstellt. Dies erlaubt zusätzlich ein Review und eine Diskussion direkt am Code. Damit man die Änderungen im Hauptprojekt in den eigenen Fork bekommt, trägt man das Hauptprojekt als weiteres Remote in die Workingcopy ein und aktualisiert regelmäßig die Hauptbranches, von denen dann auch die eigenen Feature-Banches ausgehen sollten.

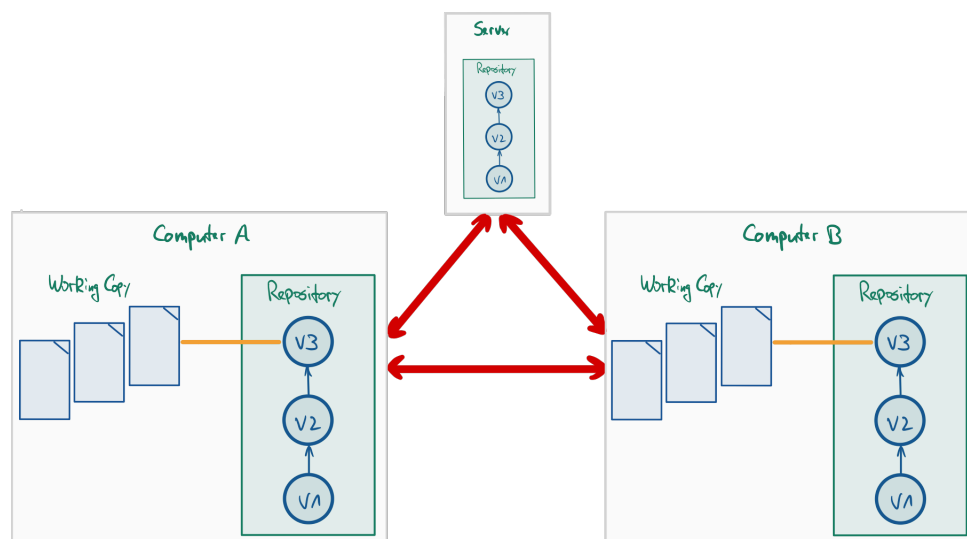
Videos

Vorlesung [YT], [HSBI]

Demos:

- Anlegen eines Forks, Erstellen eines Pull-Requests (PR) [YT], [HSBI]
- Arbeiten mit einem PR, Review eines PR [YT], [HSBI]
- `pull merge` vs. `pull rebase` [YT], [HSBI]

2.4.1. Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Git ermöglicht ein einfaches und schnelles Branching. Dies kann man mit entsprechenden Branching-Strategien sinnvoll für die SW-Entwicklung einsetzen.

Auf der anderen Seite ermöglicht Git ein sehr einfaches verteiltes Arbeiten. Auch hier ergeben sich verschiedene Workflows, wie man mit anderen Entwicklern an einem Projekt arbeiten will/kann.

Im Folgenden sollen also die Fragen betrachtet werden:

1. **Wie setze ich Branches sinnvoll ein?** Antwort: Branchingstrategien wie GitFlow oder GitHub Flow
2. **Wie gestalte ich die Zusammenarbeit?** Antwort: Workflows mit Git ...

2.4.2. Umgang mit Branches: Themen-Banches



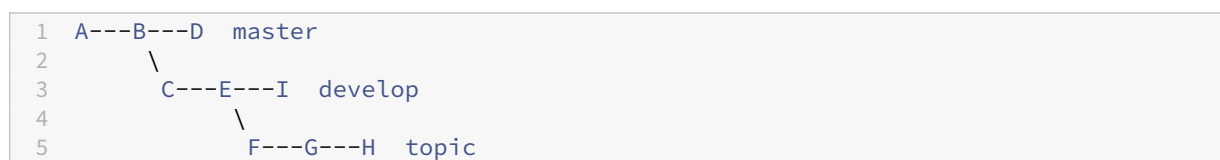
Branchen ist in Git sehr einfach und schnell. Deshalb wird (gerade auch im Vergleich mit SVN) gern und viel gebrancht.

Ein häufiges anzutreffendes Modell ist dabei die Nutzung von **Themen-Branches**: Man hat einen Hauptzweig (typischerweise `master` oder `main`). Wann immer eine neue Idee oder ein Baustein unabhängig entwickelt werden soll/kann, wird ein entsprechender Themen-Branch aufgemacht. Dabei handelt es sich normalerweise um **kleine Einheiten**!

Themenbranches haben in der Regel eine **kurze Lebensdauer**: Wenn die Entwicklung abgeschlossen ist, wird die Idee bzw. der Baustein in den Hauptzweig integriert und der Themenbranch gelöscht.

- Vorteil: Die Entwicklung im Themenbranch ist in sich gekapselt und stört nicht die Entwicklung in anderen Branches (und diese stören umgekehrt nicht die Entwicklung im Themenbranch).
- Nachteil:
 - Mangelnder Überblick durch viele Branches
 - Ursprung der Themenbranches muss überlegt gewählt werden, d.h. alle dort benötigten Features müssen zu dem Zeitpunkt im Hauptzweig vorhanden sein

2.4.3. Umgang mit Branches: Langlaufende Branches



Häufig findet man in (größeren) Projekten Branches, die über die gesamte Lebensdauer des Projekts existieren, sogenannte “langlaufende Branches”.

Normalerweise gibt es einen Branch, in dem stets der stabile Stand des Projekts enthalten ist. Dies ist häufig der `master` (oder `main`). In diesem Branch gibt es nur sehr wenige Commits: normalerweise nur Merges aus dem `develop`-Branch (etwa bei Fertigstellung einer Release-Version) und ggf. Fehlerbehebungen.

Die aktive Entwicklung findet in einem separaten Branch statt: `develop`. Hier nutzt man zusätzlich Themen-Branches für die Entwicklung einzelner Features, die nach Fertigstellung in den `develop` gemergt werden.

Kleinere Projekte kommen meist mit den zwei langlaufenden Branches in der obigen Darstellung aus. Bei größeren Projekten finden sich häufig noch etliche weitere langlaufende Branches, beispielsweise “Proposed Updates” etc. beim Linux-Kernel.

- Vorteile:

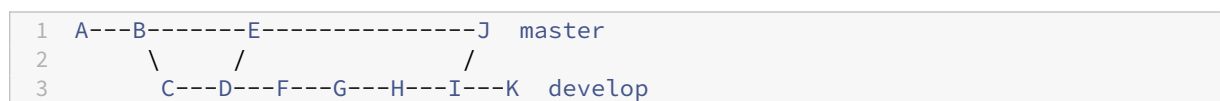
- Mehr Struktur im Projekt durch in ihrer Semantik wohldefinierte Branches
- Durch weniger Commits pro Branch lässt sich die Historie leichter verfolgen (u.a. auch aus bestimmter Rollen-Perspektive: Entwickler, Manager, ...)
- Nachteile: Bestimmte “ausgezeichnete” Branches; zusätzliche Regeln zum Umgang mit diesen beachten

2.4.4. Komplexe Branching-Strategie: Git-Flow



Das Git-Flow-Modell von Vincent Driessen (nvie.com/posts/a-successful-git-branching-model) zeigt einen in der Praxis überaus bewährten Umgang mit Branches. Lesen Sie an der angegebenen Stelle nach, besser kann man die Nutzung dieses eleganten Modells eigentlich nicht erklären :-)

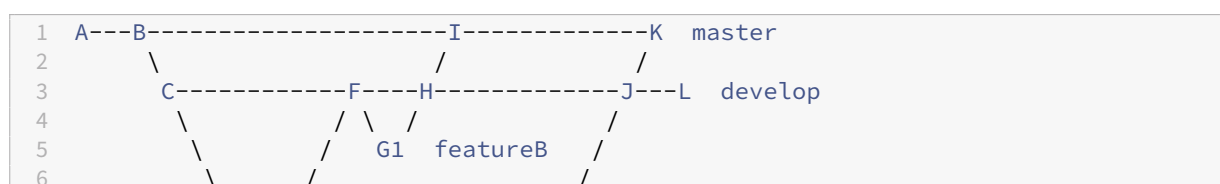
2.4.4.1. Git-Flow: Hauptzweige *master* und *develop*

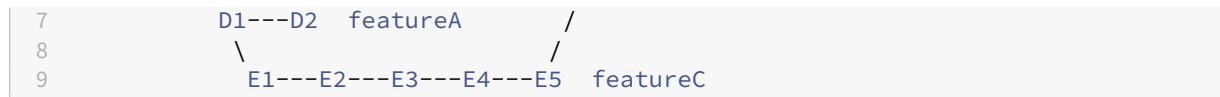


Bei Git-Flow gibt es zwei langlaufende Branches: Den *master*, der immer den stabilen Stand enthält und in den *nie* ein direkter Commit gemacht wird, sowie den *develop*, wo letztlich (ggf. über Themenbranches) die eigentliche Entwicklung stattfindet.

Änderungen werden zunächst im *develop* erstellt und getestet. Wenn die Features stabil sind, erfolgt ein Merge von *develop* in den *master*. Hier kann noch der Umweg über einen *release*-Branch genommen werden: Als “Feature-Freeze” wird vom *develop* ein *release*-Branch abgezweigt. Darin wird das Release dann aufpoliert, d.h. es erfolgen nur noch kleinere Korrekturen und Änderungen, aber keine echte Entwicklungsarbeit mehr. Nach Fertigstellung wird der *release* dann sowohl in den *master* als auch *develop* gemergt.

2.4.4.2. Git-Flow: Weitere Branches als Themen-Banches





Für die Entwicklung eigenständiger Features bietet es sich auch im Git-Flow an, vom `develop` entsprechende Themenbranches abzuzweigen und darin jeweils isoliert die Features zu entwickeln. Wenn diese Arbeiten eine gewisse Reife haben, werden die Featurebranches in den `develop` integriert.

2.4.4.3. Git-Flow: Merging-Detail



vs.



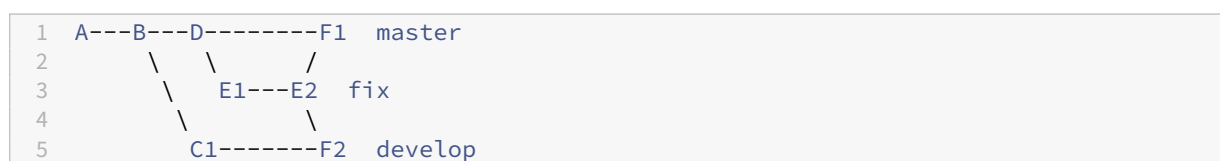
Wenn beim Mergen ein “*fast forward*” möglich ist, würde Git beim Mergen eines (Feature-) Branches in den `develop` (oder allgemein in einen anderen Branch) *keinen* separaten Commit erzeugen (Situation rechts in der Abbildung).

Damit erscheint der `develop`-Branch wie eine lineare Folge von Commits. In manchen Projekten wird dies bevorzugt, weil die Historie sehr übersichtlich aussieht.

Allerdings verliert man die Information, dass hier ein Feature entwickelt wurde und wann es in den `develop` integriert wurde (linke Seite in obiger Abbildung). Häufig wird deshalb ein extra Merge-Commit mit `git merge --no-ff <branch>` (extra Schalter “`--no-ff`”) erzwungen, obwohl ein “*fast forward*” möglich wäre.

Anmerkung: Man kann natürlich auch über Konventionen in den Commit-Kommentaren eine gewisse Übersichtlichkeit erzwingen. Beispielsweise könnte man vereinbaren, dass alle Commit-Kommentare zu einem Feature “A” mit “`feature a:`” starten müssen.

2.4.4.4. Git-Flow: Umgang mit Fehlerbehebung



Wenn im stabilen Branch (also dem `master`) ein Problem bekannt wird, darf man es nicht einfach im `master` fixen. Stattdessen wird ein extra Branch vom `master` abgezweigt, in dem der Fix entwickelt wird. Nach Fertigstellung wird dieser Branch sowohl in den `master` als auch den `develop` gemergt, damit auch im Entwicklungsbranch der Fehler behoben ist.

Dadurch entspricht jeder Commit im `master` einem Release.

2.4.5. Vereinfachte Branching-Strategie: GitHub Flow



Github verfolgt eine deutlich vereinfachte Strategie: “GitHub Flow” (vgl. [“GitHub Flow” \(S. Chacon\)](#) bzw. [“GitHub flow” \(GitHub, Inc.\)](#)).

Hier ist der stabile Stand ebenfalls immer im `master`. Features werden ebenso wie im Git-Flow-Modell in eigenen Feature-Branche entwickelt.

Allerdings zweigen Feature-Branche *immer direkt* vom `master` ab und werden nach dem Test auch immer dort wieder direkt integriert (es gibt also keine weiteren langlaufenden Branches wie `develop` oder `release`).

In der obigen Abbildung ist zu sehen, dass für die Entwicklung eines Features ein entsprechender Themenbranch vom `master` abgezweigt wird. Darin erfolgt dann die Entwicklung des Features, d.h. mehrere Commits. Das Mergen des Features in den `master` erfolgt dann aber nicht lokal, sondern mit einem “Pull-Request” auf dem Server: Sobald man im Feature-Branch einen “diskussionswürdigen” Stand hat, wird ein **Pull-Request (PR)** über die Weboberfläche aufgemacht (ein PR gehört nicht zu Git selbst, sondern zum Tooling von Github oder anderen Anbietern). In einem PR können andere Entwickler den Code kommentieren und ergänzen. Jeder weitere Commit auf dem Themenbranch wird ebenfalls Bestandteil des Pull-Requests. Parallel laufen ggf. automatisierte Tests etc. und durch das Akzeptieren des PR in der Weboberfläche erfolgt schließlich der Merge des Feature-Branche in den `master`.

2.4.6. Diskussion: Git-Flow vs. GitHub Flow

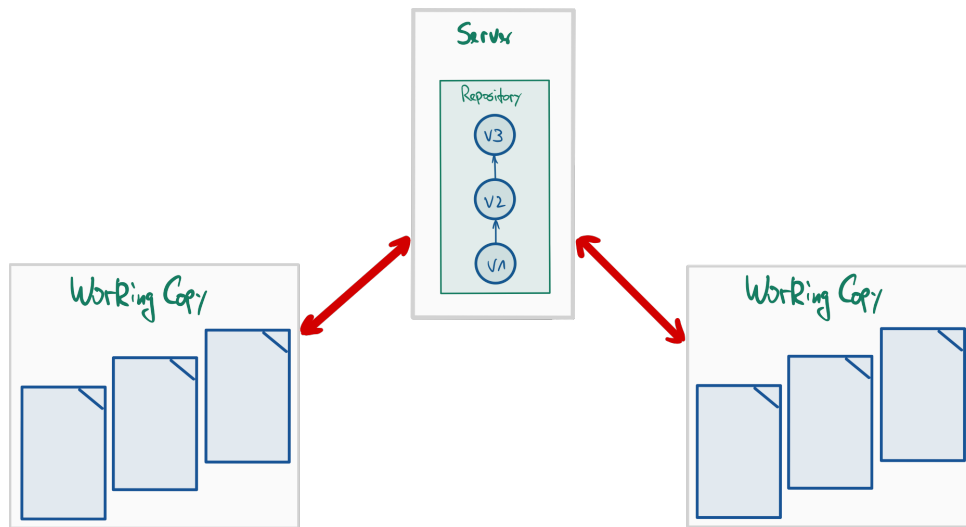
In der Praxis zeigt sich, dass das Git-Flow-Modell besonders gut geeignet ist, wenn man tatsächlich so etwas wie “Releases” hat, die zudem nicht zu häufig auftreten.

Das GitHub-Flow-Vorgehen bietet sich an, wenn man entweder keine Releases hat oder diese sehr häufig erfolgen (typisch bei agiler Vorgehensweise). Zudem vermeidet man so, dass die Feature-Branche zu lange laufen, womit normalerweise die Wahrscheinlichkeit von Merge-Konflikten stark steigt. **Achtung:** Da die Feature-Branche direkt in den `master`, also den stabilen Produktionscode gemergt werden, ist es hier besonders wichtig, vor dem Merge entsprechende Tests durchzuführen und den Merge erst zu machen, wenn alle Tests “grün” sind.

Hier ein paar Einstiegsseiten für die Diskussion, die teilweise sehr erbittert (und mit ideologischen Zügen) geführt wird (erinnert an die Diskussionen, welche Linux-Distribution die bessere sei):

- [Git-Flow-Modell von Vincent Driessen](#)
- [Kurzer Überblick über das GitHub-Flow-Modell](#)
- [Diskussion des GitHub-Flow-Modells \(Github\)](#)
- [Luca Mezzalana: “Git-Flow vs Github Flow”](#)
- [Scott Chacon, Autor des Pro-Git-Buchs](#)
- [Noch eine \(längere\) Betrachtung \(Robin Daugherty\)](#)

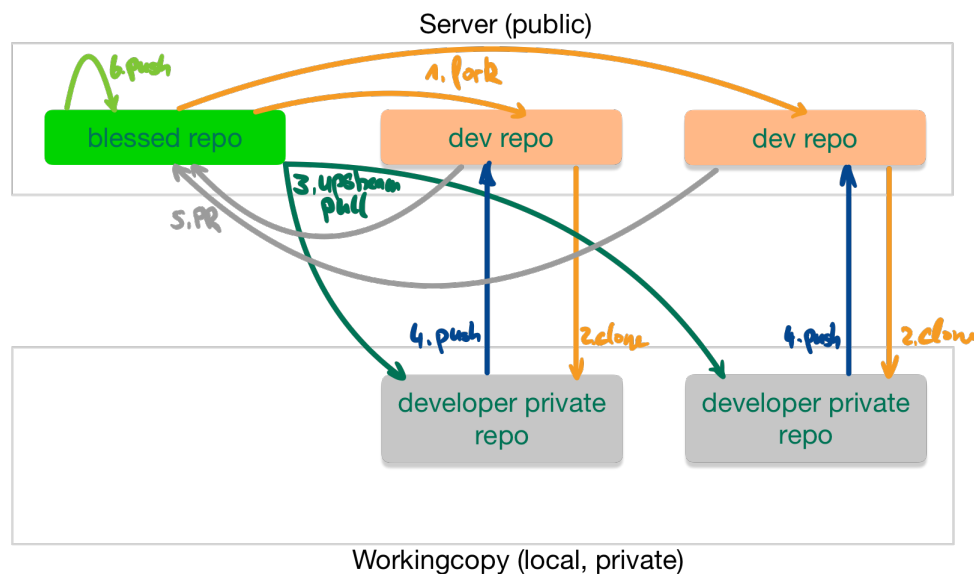
2.4.7. Zusammenarbeit: Zentraler Workflow mit Git (analog zu SVN)



In kleinen Projektgruppen wie beispielsweise Ihrer Arbeitsgruppe wird häufig ein einfacher zentralisierter Workflow bei der Versionsverwaltung genutzt. Im Mittelpunkt steht dabei ein zentrales Repository, auf dem alle Teammitglieder gleichberechtigt und direkt pushen dürfen.

- Vorteile:
 - Einfachstes denkbare Modell
 - Ein gemeinsames Repo (wie bei SVN)
 - Alle haben Schreibzugriff auf ein gemeinsames Repo
- Nachteile:
 - Definition und Umsetzung von Rollen mit bestimmten Rechten ("Manager", "Entwickler", "Gast-Entwickler", ...) schwierig bis unmöglich (das ist kein Git-Thema, sondern hängt von der Unterstützung durch den Anbieter des Servers ab)
 - Jeder darf überall pushen: Enge und direkte Abstimmung nötig
 - Modell funktioniert meist nur in sehr kleinen Teams (2..3 Personen)

2.4.8. Zusammenarbeit: Einfacher verteilter Workflow mit Git



In großen und/oder öffentlichen Projekten wird üblicherweise ein Workflow eingesetzt, der auf den Möglichkeiten von verteilten Git-Repositories basiert.

Dabei wird zwischen verschiedenen Rollen ("Integrationsmanager", "Entwickler") unterschieden.

Sie finden dieses Vorgehen beispielsweise beim Linux-Kernel und auch häufig bei Projekten auf Github.

- Es existiert ein geschütztes ("blessed") Master-Repo
 - Stellt die Referenz für das Projekt dar
 - Push-Zugriff nur für ausgewählte Personen ("Integrationsmanager")
- Entwickler
 - Forken das Master-Repo auf dem Server und klonen ihren Fork lokal
 - Arbeiten auf lokalem Klon: Unabhängige Entwicklung eines Features
 - Pushen ihren Stand in ihren Fork (ihr eigenes öffentliches Repo): Veröffentlichung des Beitrags zum Projekt (sobald fertig bzw. diskutierbar)
 - Lösen Pull- bzw. Merge-Request gegen das Master-Repo aus: Beitrag soll geprüft und ins Projekt aufgenommen werden (Merge ins Master-Repo durch den Integrationsmanager)
- Integrationsmanager
 - Prüft die Änderungen im Pull- bzw. Merge-Request und fordert ggf. Nacharbeiten an bzw. lehnt Integration ab (technische oder politische Gründe)
 - Führt Merge der Entwickler-Zweige mit den Hauptzweigen durch Akzeptieren der Pull- bzw. Merge-Requests durch: Beitrag der Entwickler ist im Projekt angekommen und ist beim nächsten Pull in deren lokalen Repos vorhanden

Den hier gezeigten Zusammenhang kann man auf weitere Ebenen verteilen, vgl. den im Linux-Kernel gelebten “Dictator and Lieutenants Workflow” (siehe Literatur).

Hinweis: Hier wird nur die Zusammenarbeit im verteilten Team dargestellt. Dazu kommt noch das Arbeiten mit verschiedenen Branches!

Anmerkung: In der Workingcopy wird das eigene (öffentliche) Repo oft als **origin** und das geschützte (“blessed”) Master-Repo als **upstream** referenziert.

2.4.8.1. Anmerkungen zum Forken

Sie könnten auch das Original-Repo direkt klonen. Allerdings würden dann die **push** dort aufschlagen, was in der Regel nicht erwünscht ist (und auch nicht erlaubt ist).

Deshalb forkt man das Original-Repo auf dem Server, d.h. auf dem Server wird eine Kopie des Original-Repos im eigenen Namespace angelegt. Auf diese Kopie hat man dann uneingeschränkten Zugriff.

Zusätzliche kurze Video-Anleitungen von GitHub: [Working with forks](#)

2.4.8.2. Anmerkungen zu den Namen für die Remotes: **origin** und **upstream**

Üblicherweise checkt man die *Kopie* lokal aus (d.h. erzeugt einen Clone). In der Workingcopy verweist dann **origin** auf die Kopie. Um Änderungen am Original-Repo zu erhalten, fügt man dieses unter dem Namen **upstream** als weiteres Remote-Repo hinzu. Dies ist eine nützliche *Konvention*.

Um Änderungen aus dem Original-Repo in den eigenen Fork (und die Workingcopy) zu bringen, führt man dann einfach folgendes aus (im Beispiel für den **master**):

```
1 $ git checkout master      # Workingcopy auf master
2 $ git pull upstream master # Aktualisiere lokalen master mit master aus
                             Original-Repo
3 $ git push origin master   # Pushe lokalen master in den Fork
```

2.4.9. Feature-Branches aktualisieren: Mergen mit **master** vs. Rebase auf **master**

Im Netz finden sich häufig Anleitungen, wonach man Änderungen im **master** mit einem Merge in den Feature-Branch holt, also sinnvoll:

```
1 $ git checkout master      # Workingcopy auf master
2 $ git pull upstream master # Aktualisiere lokalen master mit master aus
                             Vorgabe-Repo
3 $ git checkout feature     # Workingcopy auf feature
4 $ git merge master         # Aktualisiere feature: Merge master in feature
5 $ git push origin feature  # Push aktuellen feature ins Team-Repo
```

Das funktioniert rein technisch betrachtet.

Allerdings spielt in den meisten Git-Projekten der `master` (bzw. `main`) üblicherweise eine besondere Rolle und ist üblicherweise stets das **Ziel** eines Merge, aber nie die *Quelle*! D.h. per Konvention geht der Fluß von Änderungen stets **in** den `master` (und nicht heraus).

Wenn man sich nicht an diese Konvention hält, hat man später möglicherweise Probleme, die Merge-Historie zu verstehen (welche Änderung kam von woher)!

Um die Änderungen im `master` in einen Feature-Branch zu bekommen, sollte deshalb ein **Rebase** des Feature-Branches auf den aktuellen `master` bevorzugt werden.

Merk-Regel: Merge niemals nie den `master` in Feature-Branches!

Achtung: Ein Rebase bei veröffentlichten Branches ist problematisch, sobald Dritte auf diesem Branch arbeiten oder den Branch als Basis für ihre eigenen Arbeiten nutzen und dadurch entsprechend auf die Commit-IDs angewiesen sind. Nach einem Rebase stimmen diese Commit-IDs nicht mehr, was normalerweise mindestens zu Verärgerung führt ... Die Dritten müssten ihre Arbeit dann auf den neuen Feature-Branch (d.h. den Feature-Branch nach dessen Rebase) rebasen ... vgl. auch "The Perils of Rebasing" in Abschnitt "3.6 Rebasing" in (Chacon und Straub 2014).

2.4.9.1. Mögliches Szenario im Praktikum

Im Praktikum haben Sie das Vorgabe-Repo. Dieses könnten Sie als `upstream` in Ihre lokale Workingcopy einbinden.

Mit Ihrem Team leben Sie vermutlich einen zentralen Workflow, d.h. Sie binden Ihr gemeinsames Repo als `origin` in Ihre lokale Workingcopy ein.

Dann müssen Sie den lokalen `master` aus *beiden* Remotes aktualisieren. Zusätzlich wollen Sie Ihren aktuellen Themenbranch auf den aktuellen `master` rebasen.

```
1 $ git checkout master          # Workingcopy auf master
2 $ git pull upstream master     # Aktualisiere lokalen master mit master aus
  Vorgabe-Repo
3 $ git pull origin master       # Aktualisiere lokalen master mit master aus Team
  -Repo
4 $ git push origin master       # Pushe lokalen master in das Team-Repo zurück
5 $ git rebase master feature    # Rebase feature auf den aktuellen lokalen master
6 $ git push -f origin feature   # Push aktuellen feature ins Team-Repo ("-f" wg.
  geänderter IDs durch rebase)
```

Anmerkung: Dabei können in Ihrem `master` die unschönen "Rauten" entstehen. Wenn Sie das vermeiden wollen, tauschen Sie den zweiten und den dritten Schritt und führen den Pull gegen den Upstream-`master` als `pull --rebase` durch. Dann müssen Sie Ihren lokalen `master` allerdings auch force-pushen in Ihr Team-Repo und die anderen Team-Mitglieder sollten darüber informiert werden, dass sich der `master` auf inkompatible Weise geändert hat ...

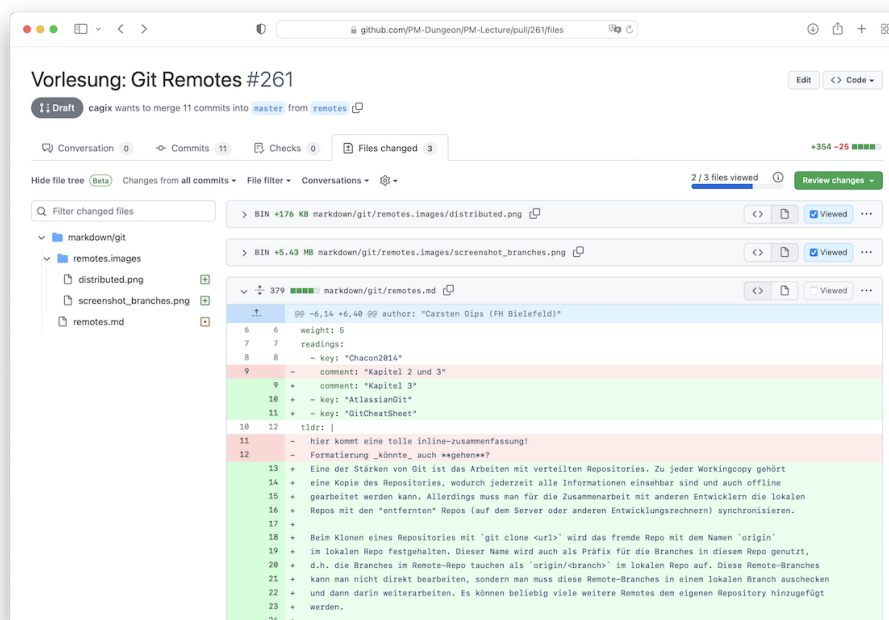
2.4.10. Kommunikation: Merge- bzw. Pull-Requests

Mergen kann man auf der Konsole (oder in der IDE) und anschließend die (neuen) Branches auf den Server pushen.

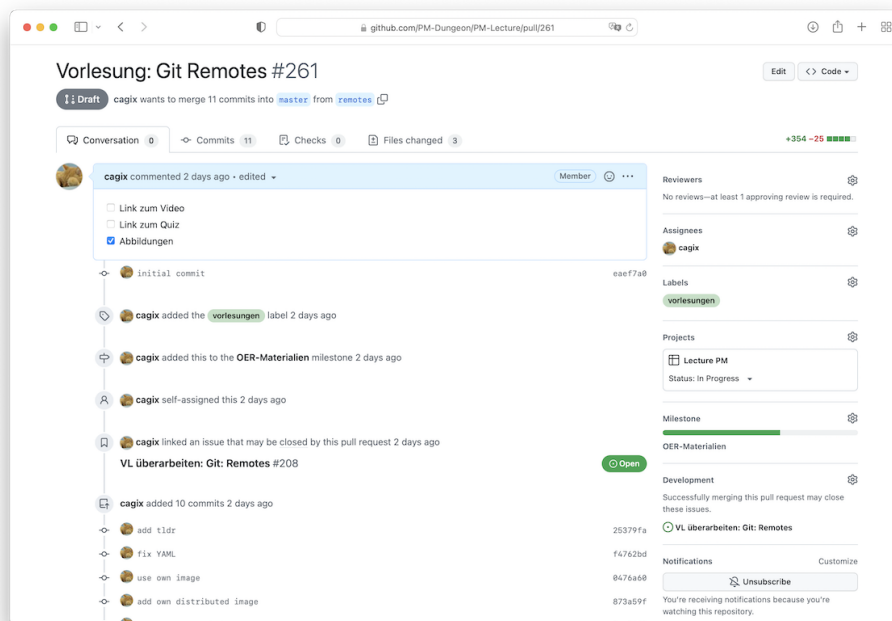
Die verschiedenen Git-Server erlauben ebenfalls ein GUI-gestütztes Mergen von Branches: "Merge-Requests" (MR, Gitlab) bzw. "Pull-Requests" (PR, Github). Das hat gegenüber dem lokalen Mergen wichtige Vorteile: Andere Entwickler sehen den beabsichtigten Merge (frühzeitig) und können direkt den Code kommentieren und die vorgeschlagenen Änderungen diskutieren, aber auch allgemeine Kommentare abgeben.

Falls möglich, sollte man einen MR/PR immer dem Entwickler zuweisen, der sich weiter um diesen MR/PR kümmern wird (also zunächst ist man das erstmal selbst). Zusätzlich kann man einen Reviewer bestimmen, d.h. wer soll sich den Code ansehen.

Hier ein Screenshot der Änderungsansicht unseres Gitlab-Servers (SW-Labor):



Nachfolgend für den selben MR aus der letzten Abbildung noch die reine Diskussionsansicht:



Zusätzliche kurze Video-Anleitungen von GitHub:

- [How to create a pull request](#)
- [How to merge a pull request](#)

2.4.11. Best Practices bei Merge-/Pull-Requests

1. MR/PR so zeitig wie möglich aufmachen

- Am besten sofort, wenn ein neuer Branch auf den Server gepusht wird!
- Ggf. mit dem Präfix “WIP” im Titel gegen unbeabsichtigtes vorzeitiges Mergen sperren ... (bei GitHub als “Draft”-PR öffnen)

2. Auswahl Start- und Ziel-Branch (und ggf. Ziel-Repo)

- Es gibt verschiedene Stellen, um einen MR/PR zu erstellen. Manchmal kann man nur noch den Ziel-Branch einstellen, manchmal beides.
- Bitte auch darauf achten, welches Ziel-Repo eingestellt ist! Bei Forks wird hier immer das Original-Repo voreingestellt!
- Den Ziel-Branch kann man ggf. auch nachträglich durch Editieren des MR/PR anpassen (Start-Branch und Ziel-Repo leider nicht, also beim Erstellen aufpassen!).

3. Titel (Summary): Das ist das, was man in der Übersicht sieht!

- Per Default wird die letzte Commit-Message eingesetzt.
- Analog zur Commit-Message: Bitte hier unbedingt einen sinnvollen Titel einsetzen: Was macht der MR/PR (kurz)?

4. Beschreibung: Was passiert, wenn man diesen MR/PR akzeptiert (ausführlicher)?

- Analog zur Commit-Message sollte hier kurz zusammengefasst werden, was der MR/PR ändert.
- Zusätzlich sollte zwingend erklärt werden,
 - **warum** die Änderung notwendig erscheint, und
 - **was** die Änderung bewirken soll.

5. Assignee: Wer soll sich drum kümmern?

- Ein MR/PR sollte immer jemanden zugewiesen sein, d.h. nicht “unassigned” sein. Ansonsten ist nicht klar, wer den Request durchführen/akzeptieren soll.
- Außerdem taucht ein nicht zugewiesener MR/PR nicht in der Übersicht “meiner” MR/PR auf, d.h. diese MR/PR haben die Tendenz, vergessen zu werden!

6. Diskussion am (und neben) dem Code

- Nur die vorgeschlagenen Code-Änderungen diskutieren!
- Weitergehende Diskussionen (etwa über Konzepte o.ä.) besser in separaten Issues erledigen, da sonst die Anzeige des MR/PR langsam wird (ist beispielsweise ein Problem bei Gitlab).

7. Weitere Commits auf dem zu mergenden Branch gehen automatisch mit in den Request

8. Weitere Entwickler kann man mit “@username” in einem Kommentar auf “CC” setzen und in die Diskussion einbinden

Anmerkung: Bei Gitlab (d.h. auch bei dem Gitlab-Server im SW-Labor) gibt es “Merge-Requests” (MR). Bei Github gibt es “Pull-Requests” (PR) ...

2.4.12. Wrap-Up

- Einsatz von Themenbranches für die Entwicklung
- Unterschiedliche Modelle:
 - Git-Flow: umfangreiches Konzept, gut für Entwicklung mit festen Releases
 - GitHub Flow: deutlich schlankeres Konzept, passend für kontinuierliche Entwicklung ohne echte Releases
- Git-Workflows für die Zusammenarbeit:
 - einfacher zentraler Ansatz für kleine Arbeitsgruppen vs.
 - einfacher verteilter Ansatz mit einem “blessed” Repo (häufig in Open-Source-Projekten zu finden)
- Aktualisieren Ihres Clones und Ihres Forks mit Änderungen aus dem “blessed” Repo
- Erstellen von Beiträgen zu einem Projekt über Pull-/Merge-Requests

Zum Nachlesen

Sie finden den Inhalt dieser Sitzung im Chacon und Straub (2014) in den Kapiteln 3 (Branching) und 5 (Zusammenarbeit) sowie 4.8 und 6 (GitLab und GitHub).

Lernziele

- k3: Branching: Ich kann mit Themenbranches in der Entwicklung arbeiten
- k2: Branching: Ich kann das 'Git-Flow'-Modell erklären
- k3: Branching: Ich kann das 'GitHub Flow'-Modell anwenden
- k2: Branching: Ich kenne verschiedene Git-Workflows für die Zusammenarbeit und kann den Ablauf erklären
- k3: Zusammenarbeit: Ich kann den zentralisierten Workflow einsetzen
- k3: Zusammenarbeit: Ich kann den einfachen verteilten Workflow mit unterschiedlichen Repos einsetzen
- k3: Zusammenarbeit: Ich kann meinen Clone und meinen Fork bei/mit Änderungen im/aus dem 'blessed Repo' aktualisieren
- k3: Zusammenarbeit: Ich kann meine Beiträge zu einem Projekt als Merge-/Pull-Requests einreichen
- k3: Zusammenarbeit: Ich kann meine Merge-/Pull-Requests aktualisieren
- k3: Zusammenarbeit: Ich kann in Merge-/Pull-Requests Anmerkungen am Code hinterlegen und an den Feedback-Diskussionen teilnehmen
- k2: PR: Ich kann den Unterschied zwischen einem Pull/Merge und einem Pull/Rebase erklären
- k2: PR: Ich verstehe, welche Commits zu einem Bestandteil eines Merge-/Pull-Requests werden (und warum)

3. Bauen von Programmen, Automatisierung, Continuous Integration

3.1. Build-Systeme: Gradle

TL;DR

Um beim Übersetzen und Testen von Software von den spezifischen Gegebenheiten auf einem Entwickler-rechner unabhängig zu werden, werden häufig sogenannte Build-Tools eingesetzt. Mit diesen konfiguriert man sein Projekt abseits einer IDE und übersetzt, testet und baut seine Applikation damit entsprechend unabhängig. In der Java-Welt sind aktuell die Build-Tools Ant, Maven und Gradle weit verbreitet.

In Gradle ist ein Java-Entwicklungsmodell quasi eingebaut. Über die Konfigurationsskripte müssen nur noch bestimmte Details wie benötigte externe Bibliotheken oder die Hauptklasse und sonstige Projekt-besonderheiten konfiguriert werden. Über “Tasks” wie `build`, `test` oder `run` können Java-Projekte übersetzt, getestet und ausgeführt werden. Dabei werden die externen Abhängigkeiten (Bibliotheken) aufgelöst (soweit konfiguriert) und auch abhängige Tasks mit erledigt, etwa muss zum Testen vorher der Source-Code übersetzt werden.

Gradle bietet eine Fülle an Plugins für bestimmte Aufgaben an, die jeweils mit neuen Tasks einher kommen. Beispiele sind das Plugin `java`, welches weitere Java-spezifische Tasks wie `classes` mitbringt, oder das Plugin `checkstyle` zum Überprüfen von Coding-Style-Richtlinien.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo [\[YT\]](#), [\[HSBI\]](#)

3.1.1. Automatisieren von Arbeitsabläufen

Works on my machine ...

Einen häufigen Ausspruch, den man bei der Zusammenarbeit in Teams zu hören bekommt, ist “Also, bei mir läuft der Code.” ...

Das Problem dabei ist, dass jeder Entwickler eine andere Maschine hat, oft ein anderes Betriebssystem oder eine andere OS-Version. Dazu kommen noch eine andere IDE und/oder andere Einstellungen und so weiter.

Wie bekommt man es hin, dass Code zuverlässig auch auf anderen Rechnern baut? Ein wichtiger Baustein dafür sind sogenannte “Build-Systeme”, also Tools, die unabhängig von der IDE (und den IDE-Einstellungen) für das

Übersetzen der Software eingesetzt werden und deren Konfiguration dann mit im Repo eingchecked wird. Damit kann die Software dann auf allen Rechnern und insbesondere dann auch auf dem Server (Stichwort “Continuous Integration”) unabhängig von der IDE o.ä. automatisiert gebaut und getestet werden.

- Build-Tools:
 - Apache Ant
 - Apache Maven
 - **Gradle**

Das sind die drei am häufigsten anzutreffenden Build-Tools in der Java-Welt.

Ant ist von den drei genannten Tools das älteste und setzt wie Maven auf XML als Beschreibungssprache. In Ant müssen dabei alle Regeln stets explizit formuliert werden, die man benutzen möchte.

In Maven wird dagegen von einem bestimmten Entwicklungsmodell ausgegangen, hier müssen nur noch die Abweichungen zu diesem Modell konfiguriert werden.

In Gradle wird eine DSL basierend auf der Skriptsprache Groovy (läuft auf der JVM) eingesetzt, und es gibt hier wie in Maven ein bestimmtes eingebautes Entwicklungsmodell. Gradle bringt zusätzlich noch einen Wrapper mit, d.h. es wird eine Art Gradle-Starter im Repo konfiguriert, der sich quasi genauso verhält wie ein fest installiertes Gradle (s.u.).

Achtung: Während Ant und Maven relativ stabil in der API sind, verändert sich Gradle teilweise deutlich zwischen den Versionen. Zusätzlich sind bestimmte Gradle-Versionen oft noch von bestimmten JDK-Versionen abhängig. In der Praxis bedeutet dies, dass man Gradle-Skripte im Laufe der Zeit relativ oft überarbeiten muss (einfach nur, damit das Skript wieder läuft - ohne dass man dabei irgendwelche neuen Features oder sonstige Vorteile erzielen würde). Ein großer Vorteil ist aber der Gradle-Wrapper (s.u.).

3.1.2. Gradle: Eine DSL in Groovy

DSL: *Domain Specific Language*

```
1 // build.gradle
2 plugins {
3     id 'java'
4     id 'application'
5 }
6
7 repositories {
8     mavenCentral()
9 }
10
11 dependencies {
12     testImplementation platform('org.junit:junit-bom:6.0.3')
13     testImplementation 'org.junit.jupiter:junit-jupiter'
14     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
15 }
16
```

```
17 application {  
18     mainClass = 'fluppie.App'  
19 }
```

Dies ist mit die einfachste Build-Datei für Gradle.

Über Plugins wird die Unterstützung für Java und das Bauen von Applikationen aktiviert, d.h. es stehen darüber entsprechende spezifische Tasks zur Verfügung.

Abhängigkeiten sollen hier aus dem Maven-Repository [MavenCentral](#) geladen werden. Zusätzlich wird hier als Abhängigkeit für den Test (`testImplementation`) die JUnit-Bibliothek in einer Maven-artigen Notation angegeben (vgl. [mvnrepository.com](#)). (Für nur zur Übersetzung der Applikation benötigte Bibliotheken verwendet man stattdessen das Schlüsselwort `implementation`.)

Bei der Initialisierung wurde als Package `fluppie` angegeben. Gradle legt darunter per Default die Klasse `App` mit einer `main()`-Methode an. Entsprechend kann man über den Eintrag `application` den Einsprungpunkt in die Applikation konfigurieren.

3.1.3. Gradle-DSL

Ein Gradle-Skript ist letztlich ein in Groovy geschriebenes Skript. [Groovy](#) ist eine auf Java basierende und auf der JVM ausgeführte Skriptsprache. Seit einigen Versionen kann man die Gradle-Build-Skripte auch in der Sprache Kotlin schreiben.

3.1.4. Konfigurationsdateien

Für das Bauen mit Gradle benötigt man drei Dateien im Projektordner:

- `build.gradle`: Die auf der Gradle-DSL beruhende Definition des Builds mit den Tasks (und ggf. Abhängigkeiten) eines Projekts.
Ein Multiprojekt hat pro Projekt eine solche Build-Datei. Dabei können die Unterprojekte Eigenschaften der Eltern-Buildskripte "erben" und so relativ kurz ausfallen.
- `settings.gradle`: Eine optionale Datei, in der man beispielsweise den Projektnamen oder bei einem Multiprojekt die relevanten Unterprojekte festlegt.
- `gradle.properties`: Eine weitere optionale Datei, in der projektspezifische Properties für den Gradle-Build spezifizieren kann.

3.1.5. Neues Gradle-Projekt mit Gradle Init anlegen

Um eine neue Gradle-Konfiguration anlegen zu lassen, geht man in einen Ordner und führt darin `gradle init` aus. Gradle fragt der Reihe nach einige Einstellungen ab:

```
1 $ gradle init  
2  
3 Select type of project to generate:
```

```
4 1: basic
5 2: application
6 3: library
7 4: Gradle plugin
8 Enter selection (default: basic) [1..4] 2
9
10 Select implementation language:
11 1: C++
12 2: Groovy
13 3: Java
14 4: Kotlin
15 5: Scala
16 6: Swift
17 Enter selection (default: Java) [1..6] 3
18
19 Split functionality across multiple subprojects?:
20 1: no - only one application project
21 2: yes - application and library projects
22 Enter selection (default: no - only one application project) [1..2] 1
23
24 Select build script DSL:
25 1: Groovy
26 2: Kotlin
27 Enter selection (default: Groovy) [1..2] 1
28
29 Select test framework:
30 1: JUnit 4
31 2: TestNG
32 3: Spock
33 4: JUnit Jupiter
34 Enter selection (default: JUnit Jupiter) [1..4] 4
35
36 Project name (default: tmp): wuppie
37 Source package (default: tmp): fluppie
```

Typischerweise möchte man eine Applikation bauen (Auswahl 2 bei der ersten Frage). Als nächstes wird nach der Sprache des Projekts gefragt sowie nach der Sprache für das Gradle-Build-Skript (Default ist Groovy) sowie nach dem Testframework, welches verwendet werden soll.

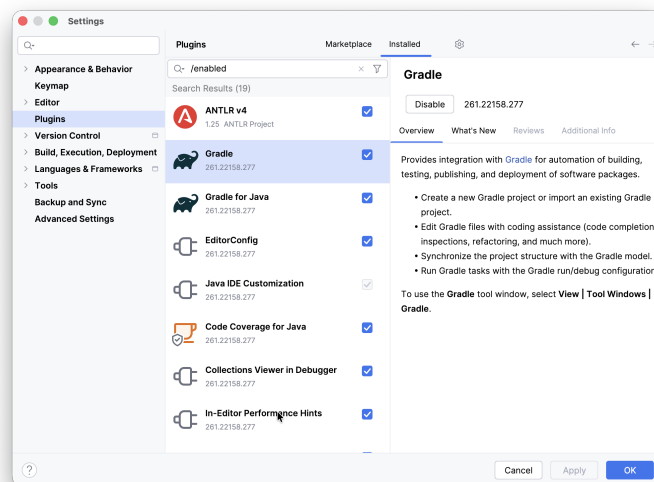
Damit wird die eingangs gezeigte Konfiguration angelegt.

Anmerkung: Die hier dargestellten Auswahloptionen und ggf. die Reihenfolge der Schritte können sich mit neueren Gradle-Versionen durchaus ändern. Das prinzipielle Vorgehen bleibt aber identisch.

Anmerkung: Man kann die schrittweise Konfiguration auch durch einen einzigen Befehl zusammenfassen, beispielsweise `gradle init --type java-application --dsl groovy --test-framework junit-jupiter` ö.ä....

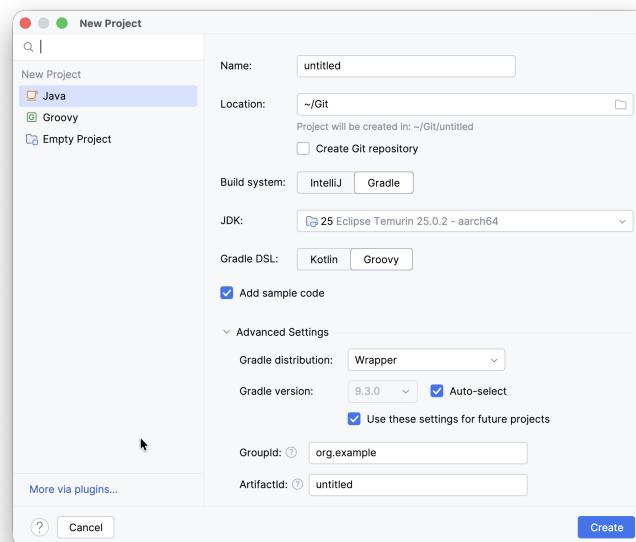
3.1.6. Gradle und IntelliJ

Installieren bzw. Aktivieren Sie in den IntelliJ-Einstellungen die Plugins für Gradle, derzeit “Gradle” und “Gradle for Java”. Ggf. haben diese Plugins weitere Abhängigkeiten, die auf Nachfrage der IDE aktiviert werden sollten.



3.1.6.1. Neues Gradle-Projekt in IntelliJ anlegen

Legen Sie ein neues Projekt an (**File** > **New** > **Project**) und wählen Sie im Einstellungsdialog als Projekttyp “Java” und bei “Build System” entsprechend “Gradle” und als “Gradle DSL” die Variante “Groovy” aus. Unter “Advanced Settings” können Sie dann noch direkt “Wrapper” auswählen, das erspart die spätere Korrektur.

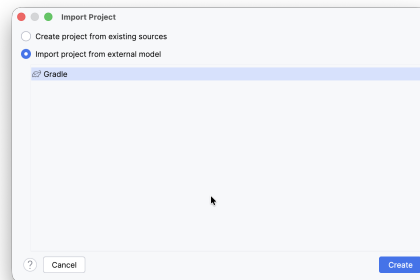


Passen Sie anschließend die Einstellungen in der `build.gradle` an.

3.1.6.2. Existierendes Gradle-Projekt in IntelliJ importieren

Importieren Sie ein existierendes Gradle-Projekt über den Dialog **File > New > Project from Existing Sources** (wenn das Projekt lokal auf Ihrem Rechner liegt) bzw. **File > New > Project from Version Control** (wenn das Projekt beispielsweise auf GitHub liegt und noch keine lokale Kopie erzeugt wurde).

Wählen Sie im nächsten Dialog “Import project from external model” und “Gradle” aus:



Passen Sie anschließend die Einstellungen in der `build.gradle` an.

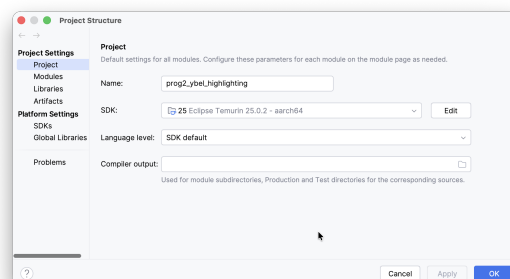
3.1.6.3. Einstellungen für IntelliJ rund um Gradle

Prinzipiell lädt IntelliJ die Gradle-Einstellungen und übernimmt diese. Damit werden dann externe Abhängigkeiten (Bibliotheken wie JUnit o.ä.) automatisch aufgelöst und heruntergeladen, Sourcecode-Pfade und sonstige Projekteinstellungen werden übernommen und der Build-Prozess wird von IntelliJ an Gradle delegiert. In der Regel klappt das zuverlässig und sehr reibungsarm.

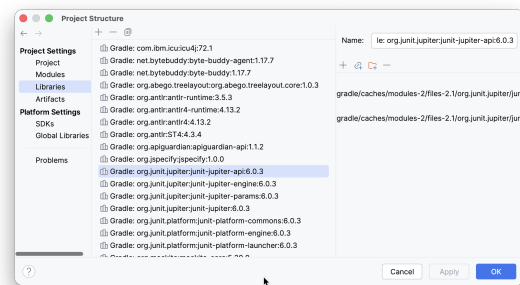
Manchmal hakt das leider aber ziemlich.

1. Check, ob die **Projekteinstellungen** in IntelliJ passen:

- i. Menü **File > Project Structure > Project Settings > Project** sollte für Ihr Projekt als SDK ein “Java 25” zeigen:

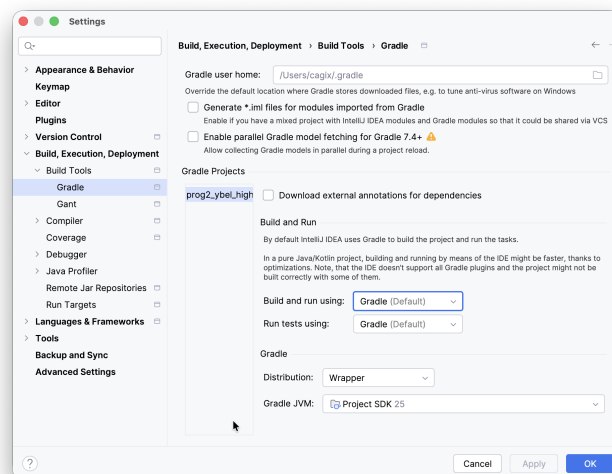


- ii. Menü **File > Project Structure > Project Settings > Libraries** sollte für Ihr Projekt die Jar-Files für die konfigurierten Abhängigkeiten (etwa JUnit) zeigen:



2. Check, ob IntelliJ mit Gradle baut:

Menü **IDEA > Settings > Build, Execution, Deployment > Build Tools > Gradle** sollte auf Gradle umgestellt sein:



Unter **“Build & Run”** sollte **“Gradle”** ausgewählt sein, die **“Distribution”** sollte auf **“Wrapper”** stehen, und als **“Gradle JVM”** sollte die für das Projekt verwendete JVM eingestellt sein, d.h. aktuell Java 25 (LTS).

3.1.7. Ordner in einem Gradle-Projekt

Durch `gradle init` wird ein neuer Ordner `wuppie/` mit folgender Ordnerstruktur angelegt:

```

1 drwxr-xr-x 4 cagix cagix 4096 Apr  8 11:43 ./
2 drwxrwxrwt 1 cagix cagix 4096 Apr  8 11:43 ../
3 -rw-r--r-- 1 cagix cagix 154 Apr  8 11:43 .gitattributes
4 -rw-r--r-- 1 cagix cagix 103 Apr  8 11:43 .gitignore
5 drwxr-xr-x 3 cagix cagix 4096 Apr  8 11:43 app/
6 drwxr-xr-x 3 cagix cagix 4096 Apr  8 11:42 gradle/
7 -rwxr-xr-x 1 cagix cagix 8070 Apr  8 11:42 gradlew*
8 -rw-r--r-- 1 cagix cagix 2763 Apr  8 11:42 gradlew.bat
9 -rw-r--r-- 1 cagix cagix 370 Apr  8 11:43 settings.gradle

```

Es werden Einstellungen für Git erzeugt (`.gitattributes` und `.gitignore`).

Im Ordner `gradle/` wird der Gradle-Wrapper abgelegt (s.u.). Dieser Ordner wird normalerweise mit ins Repo eingchecked. Die Skripte `gradlew` und `gradlew.bat` sind die Startskripte für den Gradle-Wrapper (s.u.) und werden normalerweise ebenfalls ins Repo mit eingchecked.

Der Ordner `.gradle/` (erscheint ggf. nach dem ersten Lauf von Gradle auf dem neuen Projekt) ist nur ein Hilfsordner ("Cache") von Gradle. Hier werden heruntergeladene Dateien etc. abgelegt. Dieser Ordner sollte **nicht** ins Repo eingchecked werden und ist deshalb auch per Default im generierten `.gitignore` enthalten. (Zusätzlich gibt es im User-Verzeichnis auch noch einen Ordner `.gradle/` mit einem globalen Cache.)

In `settings.gradle` finden sich weitere Einstellungen. Die eigentliche Gradle-Konfiguration befindet sich zusammen mit dem eigentlichen Projekt im Unterordner `app/`:

```
1 drwxr-xr-x 4 root root 4096 Apr 8 11:50 ./
2 drwxr-xr-x 5 root root 4096 Apr 8 11:49 ../
3 drwxr-xr-x 5 root root 4096 Apr 8 11:50 build/
4 -rw-r--r-- 1 root root 852 Apr 8 11:43 build.gradle
5 drwxr-xr-x 4 root root 4096 Apr 8 11:43 src/
```

Die Datei `build.gradle` ist die durch `gradle init` erzeugte (und eingangs gezeigte) Konfigurationsdatei, vergleichbar mit `build.xml` für Ant oder `pom.xml` für Maven. Im Unterordner `build/` werden die generierten `.class`-Dateien etc. beim Build-Prozess abgelegt.

Unter `src/` findet sich dann eine Maven-typische Ordnerstruktur für die Quellen:

```
1 $ tree src/
2 src/
3 |-- main
4 |   |-- java
5 |   |   |-- fluppie
6 |   |   |-- App.java
7 |   |-- resources
8 |-- test
9 |   |-- java
10 |   |   |-- fluppie
11 |   |   |-- AppTest.java
12 |   |-- resources
```

Unterhalb von `src/` ist ein Ordner `main/` für die Quellen der Applikation (Sourcen und Ressourcen). Für jede Sprache gibt es einen eigenen Unterordner, hier entsprechend `java/`. Unterhalb diesem folgt dann die bei der Initialisierung angelegte Package-Struktur (hier `fluppie` mit der Default-Main-Klasse `App` mit einer `main()`-Methode). Diese Strukturen wiederholen sich für die Tests unterhalb von `src/test/`.

Wer die herkömmlichen, deutlich flacheren Strukturen bevorzugt, also unterhalb von `src/` direkt die Java-Package-Strukturen für die Sourcen der Applikation und unterhalb von `test/` entsprechend die Strukturen für die JUnit-Test, der kann dies im Build-Skript einstellen:

```
1 sourceSets {
2     main {
3         java {
4             srcDirs = ['src']
5         }
6         resources {
```



```
7         srcDirs = ['res']
8     }
9 }
10 test {
11     java {
12         srcDirs = ['test']
13     }
14 }
15 }
```

3.1.8. Ablauf eines Gradle-Builds

Ein Gradle-Build hat zwei Hauptphasen: Konfiguration und Ausführung.

Während der Konfiguration wird das gesamte Skript durchlaufen (vgl. Ausführung der direkten Anweisungen eines Tasks). Dabei wird ein Graph erzeugt: welche Tasks hängen von welchen anderen ab etc.

Anschließend wird der gewünschte Task ausgeführt. Dabei werden zuerst alle Tasks ausgeführt, die im Graphen auf dem Weg zu dem gewünschten Task liegen.

Mit `gradle tasks` kann man sich die zur Verfügung stehenden Tasks ansehen. Diese sind der Übersicht halber noch nach “Themen” sortiert.

Für eine Java-Applikation sind die typischen Tasks `gradle build` zum Bauen der Applikation (inkl. Ausführen der Tests) sowie `gradle run` zum Starten der Anwendung. Wer nur die Java-Sourcen compilieren will, würde den Task `gradle compileJava` nutzen. Mit `gradle check` würde man compilieren und die Tests ausführen sowie weitere Checks durchführen (`gradle test` würde nur compilieren und die Tests ausführen), mit `gradle jar` die Anwendung in ein `.jar`-File packen und mit `gradle javadoc` die Javadoc-Dokumentation erzeugen und mit `gradle clean` die generierten Hilfsdateien aufräumen (löschen).

3.1.9. Plugin-Architektur

Für bestimmte Projekttypen gibt es immer wieder die gleichen Aufgaben. Um hier Schreibaufwand zu sparen, existieren verschiedene Plugins für verschiedene Projekttypen. In diesen Plugins sind die entsprechenden Tasks bereits mit den jeweiligen Abhängigkeiten formuliert. Diese Idee stammt aus Maven, wo dies für Java-basierte Projekte umgesetzt ist.

Beispielsweise erhält man über das Plugin `java` den Task `clean` zum Löschen aller generierten Build-Artefakte, den Task `classes`, der die Sourcen zu `.class`-Dateien kompiliert oder den Task `test`, der die JUnit-Tests ausführt...

Sie können sich Plugins und weitere Tasks relativ leicht auch selbst definieren.

3.1.10. Auflösen von Abhängigkeiten

Analog zu Maven kann man Abhängigkeiten (etwa in einer bestimmten Version benötigte Bibliotheken) im Gradle-Skript angeben. Diese werden (transparent für den User) von einer ebenfalls angegebenen Quelle, etwa einem

Maven-Repository, heruntergeladen und für den Build genutzt. Man muss also nicht mehr die benötigten `.jar`-Dateien der Bibliotheken mit ins Projekt einchecken. Analog zu Maven können erzeugte Artefakte automatisch publiziert werden, etwa in einem Maven-Repository.

Für das Projekt benötigte Abhängigkeiten kann man über den Eintrag `dependencies` spezifizieren. Dabei unterscheidet man u.a. zwischen Applikation und Tests: `implementation` und `testImplementation` für das Compilieren und Ausführen von Applikation bzw. Tests. Diese Abhängigkeiten werden durch Gradle über die im Abschnitt `repositories` konfigurierten Repositories aufgelöst und die entsprechenden `.jar`-Files geladen (in den `.gradle`/-Ordner).

Typische Repos sind das Maven-Repo selbst (`mavenCentral()`) oder das Google-Maven-Repo (`google()`).

Die Einträge in `dependencies` erfolgen dabei in einer Maven-Notation, die Sie auch im Maven-Repo mvnrepository.com finden.

3.1.11. Beispiel mit weiteren Konfigurationen (u.a. Checkstyle und Javadoc)

```
1  plugins {
2      id 'java'
3      id 'application'
4      id 'checkstyle'
5  }
6
7  repositories {
8      mavenCentral()
9  }
10
11  dependencies {
12      implementation group: 'org.apache.poi', name: 'poi', version: '5.5.1'
13  }
14
15  application {
16      mainClass = 'hangman.Main'
17  }
18
19  // use current LTS release: Java 25
20  java.toolchain.languageVersion = JavaLanguageVersion.of(25)
21  java.sourceCompatibility = JavaVersion.VERSION_25
22  java.targetCompatibility = JavaVersion.VERSION_25
23
24  tasks.withType(JavaCompile).configureEach {
25      options.encoding = 'UTF-8'
26      options.release = 25
27  }
28
29  tasks.named('run') {
30      standardInput = System.in
31  }
32
33  sourceSets {
34      main {
35          java {
36              srcDirs = ['src']
```

```
37     }
38     resources {
39         srcDirs = ['res']
40     }
41 }
42 }
43
44 checkstyle {
45     configFile = file("${rootDir}/google_checks.xml")
46     toolVersion = '13.4.2'
47 }
48
49 javadoc {
50     options.showAll()
51 }
```

Hier sehen Sie übrigens noch eine weitere mögliche Schreibweise für das Notieren von Abhängigkeiten: `implementation group: 'org.apache.poi', name: 'poi', version: '5.5.1'` und `implementation 'org.apache.poi:poi:5.5.1'` sind gleichwertig, wobei die letztere Schreibweise sowohl in den generierten Builds-Skripten und in der offiziellen Dokumentation bevorzugt wird.

3.1.12. Gradle und Ant (und Maven)

Vorhandene Ant-Buildskripte kann man nach Gradle importieren und ausführen lassen. Über die DSL kann man auch direkt Ant-Tasks aufrufen. Siehe auch [“Using Ant from Gradle”](#).

3.1.13. Gradle-Wrapper

```
1 project
2 |-- app/
3 |-- build.gradle
4 |-- gradlew
5 |-- gradlew.bat
6 `-- gradle/
7     |-- wrapper/
8         |-- gradle-wrapper.jar
9         |-- gradle-wrapper.properties
```

Zur Ausführung von Gradle-Skripten benötigt man eine lokale Gradle-Installation. Diese sollte für i.d.R. alle User, die das Projekt bauen wollen, identisch sein. Leider ist dies oft nicht gegeben bzw. nicht einfach lösbar.

Zur Vereinfachung gibt es den Gradle-Wrapper `gradlew` (bzw. `gradlew.bat` für Windows). Dies ist ein kleines Shellskript, welches zusammen mit einer kleinen `.jar`-Datei im Unterordner `gradle/` mit ins Repo eingchecked wird und welches direkt die Rolle des `gradle`-Befehls einer Gradle-Installation übernehmen kann. Man kann also in Konfigurationskripten, beispielsweise für Gitlab CI, alle Aufrufe von `gradle` durch Aufrufe von `gradlew` ersetzen.

Beim ersten Aufruf lädt `gradlew` dann die spezifizierte Gradle-Version herunter und speichert diese in einem lokalen Ordner `.gradle/`. Ab dann greift `gradlew` auf diese lokale (nicht “installierte”) `gradle`-Version zurück.

`gradle init` erzeugt den Wrapper automatisch in der verwendeten Gradle-Version mit. Alternativ kann man den Wrapper nachträglich über `gradle wrapper --gradle-version 9.5.1` in einer bestimmten (gewünschten) Version anlegen lassen.

Da der Gradle-Wrapper im Repository eingecheckt ist, benutzen alle Entwickler damit automatisch die selbe Version, ohne diese auf ihrem System zuvor installieren zu müssen. Deshalb ist der Einsatz des Wrappers einem fest installierten Gradle vorzuziehen!

Important

Oft findet sich im `.gitignore` der Eintrag `*.jar`, d.h. Jar-Files werden üblicherweise als generierte Binärdateien nicht ins Repo eingecheckt. Dadurch wird aber auch der Gradle-Wrapper oft vergessen, und beim Build in der CI-Pipeline schlägt das dann fehl. Entweder definieren Sie für `gradle/wrapper/gradle-wrapper.jar` eine Ausnahme im `.gitignore`, oder fügen Sie den Wrapper mit Hilfe des Force-Flags der Versionskontrolle hinzu: `git add -f gradle/wrapper/gradle-wrapper.jar`.

Caution

Windows-User: Der Gradle-Wrapper `gradlew` muss ausführbar sein! Wenn Sie auf einem POSIX-System arbeiten (Linux, macOS, BSD), dann wird das `x`-Bit (Unix-Executable-Bit) bereits beim Initialisieren des Projekts durch `gradle init` oder über IntelliJ automatisch korrekt gesetzt und bei einem `git add` mit ins Repo eingecheckt. Windows und Windows-Dateisysteme kennen das Unix-Executable-Bit nicht, deshalb muss man hier manuell nacharbeiten und das Shell-Skript einmalig mit dem gesetzten `x`-Bit im Git-Repo einchecken: `git add --chmod=+x gradlew`. Alternativ arbeiten Sie auf Ihrem Windows-Rechner einfach in einer WSL-Umgebung und damit in einem virtualisierten Linux.

3.1.14. Ausblick: Maven

Wie oben erwähnt, gibt es neben Gradle in der Java-Welt zwei weitere verbreitete Build-Tools: [Ant](#) und [Maven](#) (beide von Apache).

In Ant werden alle Dinge (Ziele, Regeln, ...) manuell definiert und konfiguriert (und das in XML), Dependencies müssen manuell oder über ein extra Tool (Ivy) aufgelöst werden. Dagegen ist in Maven ähnlich wie bei Gradle ein Lebenszyklus für Java-Anwendungen eingebaut und es müssen nur noch Abweichungen davon sowie die Festlegung von Versionen und Dependencies in XML formuliert werden ("*Convention over Configuration*"). In Maven nennt man die Ziele "Goals", was den Gradle-Tasks entspricht.

Im Gegensatz zu Gradle haben sich Maven-Konfigurationen als sehr stabil erwiesen. Projektkonfigurationen funktionieren oft über einen sehr langen Zeitraum hinweg, und auch bei den eher seltenen Versionssprüngen von Maven gibt es deutlich weniger Pflegeaufwand im Vergleich zu Gradle, wo teilweise im Halbjahrestakt oft deutliche Änderungen an der DSL vorgenommen werden und man früher oder später immer wieder gezwungen ist, die Projektkonfiguration entsprechend nachzuziehen. Der Nachteil von Maven ist, dass die Konfiguration in XML erfolgt und für moderne Lesegewohnheiten eher sperrig aussieht. Die Formulierung von "Extra-Wünschen" geht in Gradle über die Groovy-DSL meist relativ einfach, in Maven muss man dafür eigene Plugins schreiben.

Das obige `build.gradle` mit der Apache-POI-Abhängigkeit und der konfigurierten Java-Version und dem Checkstyle-Plugin könnte man ungefähr in folgende `pom.xml` (so nennt man die Maven-Konfigurationsdatei)

übersetzen:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
5             apache.org/xsd/maven-4.0.0.xsd">
6
7     <modelVersion>4.0.0</modelVersion>
8
9     <groupId>hangman</groupId>
10    <artifactId>hangman</artifactId>
11    <version>1.0-SNAPSHOT</version>
12
13    <properties>
14        <maven.compiler.release>25</maven.compiler.release>
15        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16        <!-- nötig für das Exec-Plugin (Main-Klasse) -->
17        <exec.mainClass>hangman.Main</exec.mainClass>
18    </properties>
19
20    <!-- Entspricht dependencies { implementation 'org.apache.poi:poi:5.5.1' } -->
21    <dependencies>
22        <dependency>
23            <groupId>org.apache.poi</groupId>
24            <artifactId>poi</artifactId>
25            <version>5.5.1</version>
26        </dependency>
27    </dependencies>
28
29    <build>
30        <!-- Entspricht sourceSets: src = Java, res = Ressourcen -->
31        <sourceDirectory>src</sourceDirectory>
32        <resources>
33            <resource>
34                <directory>res</directory>
35            </resource>
36        </resources>
37
38        <plugins>
39            <!-- Compiler-Einstellungen: Java 25, UTF-8 -->
40            <plugin>
41                <groupId>org.apache.maven.plugins</groupId>
42                <artifactId>maven-compiler-plugin</artifactId>
43                <version>3.13.0</version>
44                <configuration>
45                    <release>${maven.compiler.release}</release>
46                    <encoding>${project.build.sourceEncoding}</encoding>
47                </configuration>
48            </plugin>
49
50            <!-- Entspricht application { mainClass = 'hangman.Main' } -->
51            <plugin>
52                <groupId>org.codehaus.mojo</groupId>
53                <artifactId>exec-maven-plugin</artifactId>
54                <version>3.5.0</version>
55                <configuration>
```

```

54         <mainClass>${exec.mainClass}</mainClass>
55         <!-- StandardInput von der Konsole wird standardmäßig
durchgereicht -->
56     </configuration>
57 </plugin>
58
59
60     <!-- Entspricht checkstyle { ... } -->
61     <plugin>
62         <groupId>org.apache.maven.plugins</groupId>
63         <artifactId>maven-checkstyle-plugin</artifactId>
64         <version>3.6.0</version>
65         <configuration>
66             <configLocation>google_checks.xml</configLocation>
67         </configuration>
68     </plugin>
69
70     <!-- Grobe Entsprechung zu javadoc { options.showAll() } -->
71     <plugin>
72         <groupId>org.apache.maven.plugins</groupId>
73         <artifactId>maven-javadoc-plugin</artifactId>
74         <version>3.10.0</version>
75         <configuration>
76             <show>public</show>
77         </configuration>
78     </plugin>
79 </plugins>
80 </build>
81 </project>

```

Gradle nutzt einen `plugins`-Block zur Spezifikation der Plugins, bei Maven werden Plugins im `<build>`-Bereich eingetragen. Die Projekt-Identität (`groupId`, `artifactId`, `version`) steht bei Maven oben im `project`-Block - das sind die typischen Maven-Koordinaten `groupId:artifactId:version`.

Die Deklaration der Dependencies ist im Prinzip wie bei Gradle (nur eben in XML statt in der Groovy-DSL). Die Einträge kann man sich direkt für die jeweilige Bibliothek von [Maven Central](#) kopieren. Das Repository Maven Central ist in Maven der Default und muss (im Gegensatz zu Gradle) nicht extra angegeben werden.

Gradle bekommt mit dem `application`-Plugin einen `run`-Task. In Maven nutzten wir dafür das Plugin `exec-maven-plugin`, welches beim Befehl `mvn exec:java` die konfigurierte Main-Klasse startet. Vorsicht: Während ein `gradle run` das Projekt bei Bedarf automatisch baut und dann die konfigurierte Klasse startet, wird in Maven mit `mvn exec:java` tatsächlich nur die konfigurierte Klasse ausgeführt - für das Bauen muss man selbst sorgen. Oft wird deshalb `mvn compile exec:java` (Kompilieren und Ausführen) oder `mvn verify exec:java` (Kompilieren, Tests, Ausführen) genutzt oder alternativ eine zusätzliche `<executions>`-Konfiguration für das Plugin `exec-maven-plugin` angelegt. Die Konsoleneingabe wird in Maven automatisch ans Programm weitergereicht, in Gradle war dafür eine extra Konfiguration notwendig.

Sowohl in Gradle als auch in Maven sind die Standardpfade im Projekt `src/main/java` und `src/main/resources`, aber man kann diese Pfade bei Bedarf relativ frei anpassen.

Inzwischen gibt es auch für Maven einen sogenannten Wrapper. Beim Maven-Wrapper wird jedoch nur eine schlanke, rein textbasierte Konfiguration im Projekt mitversioniert (`mvnw`, `mvnw.cmd` und das Verzeichnis `.mvn` / mit Konfigurations-/Textdateien). Beim Gradle-Wrapper gehört hingegen immer auch eine Binärdatei (`gradle`

–`wrapper.jar`) ins Versionskontrollsystem. Da Git für Binärdateien keinen inhaltlichen Diff berechnen kann, wird bei Änderungen an diesem JAR intern jedes Mal die komplette Datei als Änderung gespeichert. Das kann sich im Laufe der Zeit nachteilig auf die Größe des Git-Repositorys auswirken.

Der [Maven Getting Started Guide](#) ist eine gute Einstiegshilfe über den hier vorgestellten Ausblick hinaus.

3.1.15. Wrap-Up

- Automatisieren von Arbeitsabläufen mit Build-Tools/-Skripten
- Einstieg in **Gradle** (DSL zur Konfiguration)
 - Typisches Java-Entwicklungsmodell eingebaut
 - Konfiguration der Abweichungen (Abhängigkeiten, Namen, ...)
 - Gradle-Wrapper: Ersetzt eine feste Installation

Zum Nachlesen

Hier einige nützliche Einsprungpunkte in die offizielle Gradle-Dokumentation:

- [“Getting Started”](#)
- [“Building Java Applications Sample”](#)
- [“Building Java Applications with libraries Sample”](#)
- [“Building Java Libraries Sample”](#)
- [“Building Java & JVM projects”](#)

Lernziele

- k3: Ich kann einfache Gradle-Skripte schreiben und verstehen

Challenges

Analyse komplexeres Build-Skript

Betrachten Sie das Buildskript `gradle.build` aus [Dungeon-CampusMinden/Dungeon](#).

Erklären Sie, in welche Abschnitte das Buildskript unterteilt ist und welche Aufgaben diese Abschnitte jeweils erfüllen. Gehen Sie dabei im *Detail* auf das Plugin `java` und die dort bereitgestellten Tasks und deren Abhängigkeiten untereinander ein.

Praktische Übungen

- Bauen Sie ein Minimalprojekt mit Gradle-Wrapper.
- Importieren Sie das Projekt in IntelliJ.
- Fügen Sie Abhängigkeiten hinzu: JUnit 6 und lassen Sie die IDE einen einfachen Test schreiben.

3.2. Java: Strukturieren mit Packages

TL;DR

Packages strukturieren Java-Projekte, indem sie Klassen in logisch zusammengehörige Bereiche aufteilen, ähnlich wie Ordner im Dateisystem. Sie schaffen eigene Namensräume, sodass Klassen mit gleichem Namen (z.B. `java.util.List` und `java.awt.List`) nebeneinander existieren können, und sie sind die Basis für Sichtbarkeit auf Package-Ebene, also für interne vs. externe APIs.

Übliche Konventionen orientieren sich am umgedrehten Domain-Namen (`de.hsbi`), ergänzt um Projekt (`prog2`) und fachliche/technische Unterteilung (`library.app`, `library.model`). Das Default-Package (keine **package**-Deklaration in der Datei) ist eine Einbahnstraße: Code in benannten Packages kann nicht auf Klassen im Default-Package zugreifen und viele Tools erwarten benannte Packages - deshalb sollten Sie es nicht in Projekten verwenden.

Videos

Vorlesung [YT], [HSBI]

3.2.1. Typisches Java-Projekt

Hier ist ein typisches Java-Projekt zu sehen:

```
1  src/
2  |___main/
3  |___java/
4  |   ___Book.java
5  |   ___LibraryService.java
6  |   ___ConsoleUI.java
7  |   ___Member.java
8  |   ___AppMain.java
```

Es gibt die Maven-Ordnerstruktur `src/main/java/`, darunter dann verschiedene Klassen.

Beobachtung: Alles liegt “wild” im selben Namespace. Man kann am Klassennamen erkennen, welche Aufgaben die Klassen jeweils vermutlich haben. Gleichzeitig wird es bereits bei den gezeigten fünf Klassen etwas unübersichtlich - in typischen Projekten ist die Anzahl der Klassen ungleich höher! Wenn das Projekt wächst, fehlt die Orientierung.

3.2.2. Listen aus dem JDK

```
1  java.util.List      vs.      java.awt.List
```

Beobachtung: Beide Listen finden sich in der Java-API - aber es sind unterschiedliche Listenimplementierungen. Der einfache Klassenname `List` ist nicht mehr eindeutig, man braucht noch einen Präfix (Spoiler: ein Package), um die beiden Listen auseinander halten zu können.

3.2.3. Was ist ein Package in Java

Packages sind eine Ordnerstruktur unterhalb des Source-Ordners im Projekt.

- Dateisystem-Sicht: `src/main/java/de/hsbi/prog2/wuppie/`
- Java-Sicht: `de.hsbi.prog2.wuppie`

Der Teil `src/main/java/` ist der Pfad zum Source-Ordner. IDEs suchen darunter nach den Klassen etc. Der hintere Teil `de/hsbi/prog2/wuppie/` der Ordnerhierarchie bildet das Package `de.hsbi.prog2.wuppie`.

D.h. man kann für Ordner unterhalb des Source-Ordners einfach die Dateisystem-Trenner durch Punkte ersetzen und kommt auf das Package.

3.2.4. Aufgaben von Packages in Java

In Java dienen Packages mehreren Zielen:

1. Strukturierung des Codes

Verwandte Klassen/Interfaces werden zusammengefasst. Erleichtert Orientierung in größeren Projekten. Analogie: Verzeichnisse/Ordner im Dateisystem.

2. Namensräume (Namespace)

Verhindert Namenskonflikte: Zwei Klassen `List` können in verschiedenen Packages existieren. Über den vollqualifizierten Namen kann man gezielt auf die gewünschte Klasse zugreifen: `java.util.List` vs. `java.awt.List`.

3. Sichtbarkeit / Kapselung Package-Ebene als "Freundeskreis" von Klassen

Es gibt verschiedene Sichtbarkeitsmodifikatoren:

- **public**: Von überall sichtbar: aus allen Klassen, in allen Packages, auch aus anderen Modulen/Projekten (sofern auf dem Classpath)
- **protected**: Sichtbar innerhalb desselben Packages und zusätzlich in Unterklassen (abgeleitete Klassen), auch wenn diese in anderen Packages liegen
- **private**: Sichtbar nur innerhalb derselben Klasse; keine andere Klasse (auch nicht im selben Package oder als Unterklasse) kann direkt darauf zugreifen
- ohne Modifikator: "package-private": Sichtbar nur innerhalb desselben Packages; Klassen, Methoden und Felder ohne Modifikator bilden eine "Package-interne" API

Damit kann man gezielt Klassenstrukturen aufbauen, die beispielsweise für bestimmte User-Gruppen gedacht sind: Developer, User, ... Man bekommt klare Grenzen und kann zwischen der internen API (auf Projekte-Ebene, aber eben auch auf Package-Ebene) und der externen API differenzieren.

3.2.5. Konventionen

```
1 de.hsbi.prog2.wuppie
```

- Umgedrehter Domain-Name: `de.hsbi` (von “hsbi.de”)
- Zusätzlich Projekt: `prog2`
- Fachliche/technische Strukturen: `wuppie`

Prinzipiell kann man Package-Namen fast beliebig wählen.

Zur Vermeidung von Namenskollisionen nutzen viele Projekte den umgedrehten Domain-Namen. Danach kommt normalerweise der Projektname.

Dies entspricht auch den Maven-Koordinaten: Die `groupId` wäre `de.hsbi`, die `artifactId` wäre in diesem Beispiel `prog2`.

Üblicherweise werden für Packages nur Kleinbuchstaben verwendet. Vermeiden Sie unbedingt Umlaute und sonstige Sonderzeichen! Normalerweise werden auch keine Versionsnummern o.ä. (`prog2_v3`, `newmodel`, `testneu`) in Packages verwendet.

3.2.6. Packages strukturieren

Es liegt komplett bei Ihnen, wie Sie Ihre Packages aufteilen...

Üblichweise nutzt man eine von zwei Strategien für die Strukturierung unterhalb des Wurzel-Packages:

1. Fachliche Strukturierung: Trennung nach Domänenthemen

- `customer`: “Alles zu Kunden”
- `order`: “Alles zu Bestellungen”
- `product`: “Alles zu Produkten”

2. Technische Strukturierung: Trennung nach “Art der Aufgabe”

- `model`: Domänenobjekte
- `service`: Geschäftslogik
- `persistence`: Datenbankzugriff
- `ui`: Benutzeroberfläche
- `util`: Hilfsklassen

Innerhalb der Packages kann man natürlich weiter unterteilen.

Wichtig: Packages sollten klar benannt und voneinander abgegrenzt sein. Packages sollten nicht in ihrer Verantwortung überlappen. Machen Sie Packages nicht zu voll, aber vermeiden Sie auch eine Flut von fast leeren Packages...

Hinweis: `util` ist ein beliebtes “Mülleimer”-Package. Hier sollten an sich nur echte generische Helfer landen, die in mehreren Packages/Klassen gebraucht werden. Wenn `util` voll läuft, ist das oft ein Zeichen für eine schlechte Aufteilung der anderen Packages.

Hinweis: Organisieren Sie Ihre Projekte frühzeitig in Packages. Fangen Sie mit wenigen Top-Level Packages an. Wenn Sie merken, dass die Struktur nicht passt, können Sie verfeinern und/oder umbauen. Lassen Sie internen Code bewusst “private” oder “package-private”.

Für das obige Beispiel könnte beispielsweise die Aufteilung nach technischen Kriterien so aussehen:

```
1  library/  
2  |____src/  
3  |____main/  
4  |____java/  
5  |____de/  
6  |____hsbi/  
7  |____prog2/  
8  |____library/  
9  |____app/  
10 |____LibraryService.java  
11 |____ConsoleUI.java  
12 |____AppMain.java  
13 |____model/  
14 |____Book.java  
15 |____Member.java
```

Der Source-Ordner ist hier der übliche `src/main/java/`. Die `groupId` ist hier `de.hsbi.prog2`, die `artifactId` wäre in diesem Beispiel `library` (als Projektordner und gleichzeitig als Package benutzt). Darunter gibt es die beiden Top-Level Packages `app` und `model`, wobei in `model` alle Typen zur Modellierung der Daten (hier Bücher und Mitglieder) landen und `app` beheimatet die Bibliothekslogik und das Userinterface sowie den Starter mit der `main()`-Methode.

Es wäre auch eine fachliche Aufteilung denkbar, etwa `loan`, `user`, `catalog` o.ä. ...

3.2.7. Praktischer Einsatz von Packages

1. Deklaration am Beginn der Java-Dateien: `package de.hsbi.prog2.library.app;`

Jede Java-Datei beginnt mit einer solchen Package-Deklaration. Davor darf höchstens ein (Javadoc-) Kommentar kommen.

Es gibt genau eine Package-Deklaration pro Datei. Sie darf nur dann fehlen, wenn die Datei im Default-Package ist (also direkt im Source-Ordner liegt).

Achten Sie auf die Schreibweise!

2. Importe von Typen und Packages

Die Importe folgen auf die Package-Deklaration. Wenn es keine gibt, dann sind die Importe der Beginn der Java-Datei.

- `import de.hsbi.prog2.library.model.Book;`

Hier wird die Klasse `Book` aus dem Package `de.hsbi.prog2.library.model` importiert. Danach können Sie die Klasse direkt mit ihrem **einfachen Namen** ansprechen, d.h. ganz normal mit `Book` arbeiten, beispielsweise `new Book()` aufrufen o.ä.

Sie können natürlich auch den **vollqualifizierten Namen** nutzen, also statt `Book` immer `de.hsbi.prog2.library.model.Book` schreiben. Dies wird i.d.R. aber als Anti-Pattern gesehen. Vermeiden Sie nach Möglichkeit vollqualifizierte Namen im Code.

- `import de.hsbi.prog2.library.model.*;`

Mit dem Wildcard importieren Sie alle Klassen, die in `de.hsbi.prog2.library.model` definiert sind.

Für Übungsprojekte ist das gegebenenfalls noch akzeptabel. In echten Projekten kann es dadurch aber schnell Namenskollisionen geben, weshalb die meisten Projekte gezielte Imports für bessere Lesbarkeit/Wartbarkeit einsetzen.

- `import static java.lang.Math.max;`

Das ist ein sogenannter **statischer Import**. Damit werden statische Member einer Klasse importiert und können direkt ohne die definierende Klasse genutzt werden.

Im Beispiel: Statt `int bigger = Math.max(3, 5);` kann man jetzt einfach `int bigger = max(3, 5);` schreiben.

Auch hier sparsam einsetzen - Gefahr von Namenskollisionen!

3.2.8. Einbahnstraße Default-Package

```
1  src/
2  |___main/
3  |   ___java/
4  |       ___ConsoleUI.java
5  |       ___de/
6  |           ___hsbi/
7  |               ___prog2/
8  |                   ___library/
9  |                       ___app/
10 |                           ___LibraryService.java
11 |                           ___AppMain.java
12 |                           ___model/
13 |                               ___Book.java
14 |                               ___Member.java
```

Die Klasse `ConsoleUI` liegt direkt im Source-Ordner, also direkt unter `src/main/java/`. Sie ist keinem speziellen Package zugeordnet, d.h. sie liegt im **Default-Package**.

Die anderen Klassen sind nach Funktionalität in benannte Packages aufgeteilt: `de.hsbi.prog2.library.model` beherbergt `Book` und `Member`, und in `de.hsbi.prog2.library.app` gibt es die Bedienlogik der Bibliothek (`LibraryService`) und den Starter (`AppMain`).

PROBLEM: Die Klassen im Default-Package können auf die Klassen in den benannten Packages zugreifen (via `import`, sofern Sichtbarkeit passt). Andersherum ist dies **nicht** möglich, d.h. `LibraryService` oder

`AppMain` können nicht auf `ConsoleUI` zugreifen! **Code in benannten Packages kann keine Klassen aus dem Default-Package verwenden.**

PROBLEM: Tools, Build-Systeme, Frameworks, Libraries, Class-Loader erwarten meist keinen Code im Default-Package. JUnit beispielsweise erwartet benannte Packages. **Code im Default-Package lässt sich schlecht als Bibliothek verwenden oder in andere Projekte integrieren.**

Das Default-Package ist eine Einbahnstraße! Nutzen Sie es nicht.

3.2.9. Wrap-Up

- Packages sind logische Container für Klassen/Interfaces und entsprechen Ordnern unterhalb des Source-Ordnern
 - Strukturierung des Codes
 - Vermeidung von Namenskollisionen (Namespaces)
 - Grenze für Sichtbarkeit (`package-private`)
- Zwei gängige Strukturierungsstrategien:
 - fachlich/domainorientiert (z.B. `customer`, `order`, `product`),
 - technisch (z.B. `model`, `service`, `persistence`, `ui`, `util`).
- Verwendung von Klassennamen ohne vollqualifizierten Namen per **`import`**
- Default-Package vermeiden

Zum Nachlesen

Lesen Sie zu Packages im [Packages Tutorial \(Oracle\)](#) nach.

Lernziele

- k2: Ich kann den Einsatz von Packages in Java erklären
- k2: Ich kann zwischen dem einfachen Klassennamen und dem vollqualifizierten Klassennamen unterscheiden
- k2: Ich kann die Probleme des Default-Packages erklären
- k3: Ich kann Packages erstellen und Klassen zuordnen
- k3: Ich kann Klassen und Packages importieren

Challenges

Nehmen Sie Ihr letztes Prog1-Projekt und überlegen Sie: Wie würden Sie es in 3..4 Packages aufteilen? Bearbeiten Sie die [Package Challenge](#).

3.3. Continuous Integration (CI)

TL;DR

In größeren Projekten mit mehreren Teams werden die Beteiligten i.d.R. nur noch “ihre” Codestellen kompilieren und testen. Dennoch ist es wichtig, das gesamte Projekt regelmäßig zu “bauen” und auch umfangreichere Testsuiten regelmäßig laufen zu lassen. Außerdem ist es wichtig, das in einer definierten Umgebung zu tun und nicht auf einem oder mehreren Entwicklerrechnern, die i.d.R. (leicht) unterschiedlich konfiguriert sind, um zuverlässige und nachvollziehbare Ergebnisse zu bekommen. Weiterhin möchte man auf bestimmte Ereignisse reagieren, wie etwa neue Commits im Git-Server, oder bei Pull-Requests möchte man vor dem Merge automatisiert sicherstellen, dass damit die vorhandenen Tests alle “grün” sind und auch die Formatierung etc. stimmt.

Dafür hat sich “Continuous Integration” etabliert. Hier werden die angesprochenen Prozesse regelmäßig auf einem dafür eingerichteten System durchgeführt. Aktivitäten wie Übersetzen, Testen, Style-Checks etc. werden in sogenannten “Pipelines” oder “Workflows” zusammengefasst und automatisiert durch Commits, Pull-Requests oder Merges auf dem Git-Server ausgelöst. Die Aktionen können dabei je nach Trigger und Branch unterschiedlich sein, d.h. man könnte etwa bei PR gegen den Master umfangreichere Tests laufen lassen als bei einem PR gegen einen Develop-Branch. In einem Workflow oder einer Pipeline können einzelne Aktionen wiederum von anderen Aktionen abhängen. Das Ergebnis kann man dann auf dem Server einsehen oder bekommt man komfortabel als Report per Mail zugeschickt.

Wir schauen uns hier exemplarisch GitHub Actions und GitLab CI/CD an. Um CI sinnvoll einsetzen zu können, benötigt man Kenntnisse über Build-Tools. “CI” tritt üblicherweise zusammen mit “CD” (Continuous Delivery) auf, also als “CI/CD”. Der “CD”-Teil ist nicht Gegenstand der Betrachtung in dieser Lehrveranstaltung.

Videos

Vorlesung CI [YT], [HSBI]

Demos:

- GitHub Actions [YT], [HSBI]
- GitLab CI/CD [YT], [HSBI]

3.3.1. Motivation: Zusammenarbeit in Teams

3.3.1.1. Szenario

- Projekt besteht aus diversen Teilprojekten
- Verschiedene Entwicklungs-Teams arbeiten (getrennt) an verschiedenen Projekten
- Tester entwickeln Testsuiten für die Teilprojekte
- Tester entwickeln Testsuiten für das Gesamtprojekt

3.3.1.2. Manuelle Ausführung der Testsuiten reicht nicht

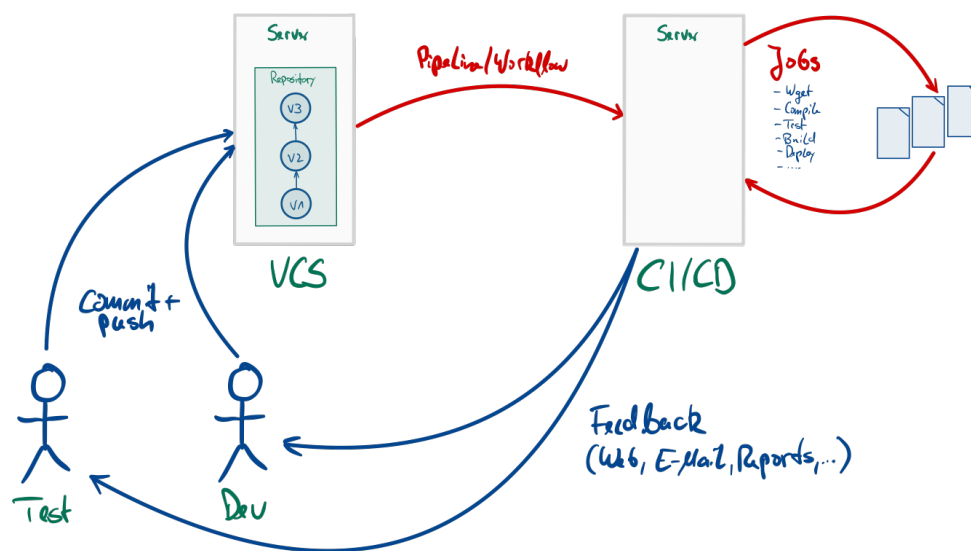
- Belastet den Entwicklungsprozess

- Keine (einheitliche) Veröffentlichung der Ergebnisse
- Keine (einheitliche) Eskalation bei Fehlern
- Keine regelmäßige Integration in Gesamtprojekt

3.3.1.3. Continuous Integration

- Regelmäßige, automatische Ausführung: Build und Tests
- Reporting
- Weiterführung der Idee: Regelmäßiges Deployment (*Continuous Deployment*)

3.3.2. Continuous Integration (CI)



3.3.2.1. Vorgehen

- Entwickler und Tester committen ihre Änderungen regelmäßig (Git, SVN, ...)
- CI-Server arbeitet Build-Skripte ab, getriggert durch Events: Push-Events, Zeit/Datum, ...
 - Typischerweise wird dabei:
 - * Das Gesamtprojekt übersetzt ("gebaut")
 - * Die Unit- und die Integrationstests abgearbeitet
 - * Zu festen Zeiten werden zusätzlich Systemtests gefahren
 - Typische weitere Builds: "Nightly Build", Release-Build, ...
 - Ergebnisse jeweils auf der Weboberfläche einsehbar (und per E-Mail)

3.3.2.2. Einige Vorteile

- Tests werden regelmäßig durchgeführt (auch wenn sie lange dauern oder die Maschine stark belasten)

- Es wird regelmäßig ein Gesamt-Build durchgeführt
- Alle Teilnehmer sind über aktuellen Projekt(-zu-)stand informiert

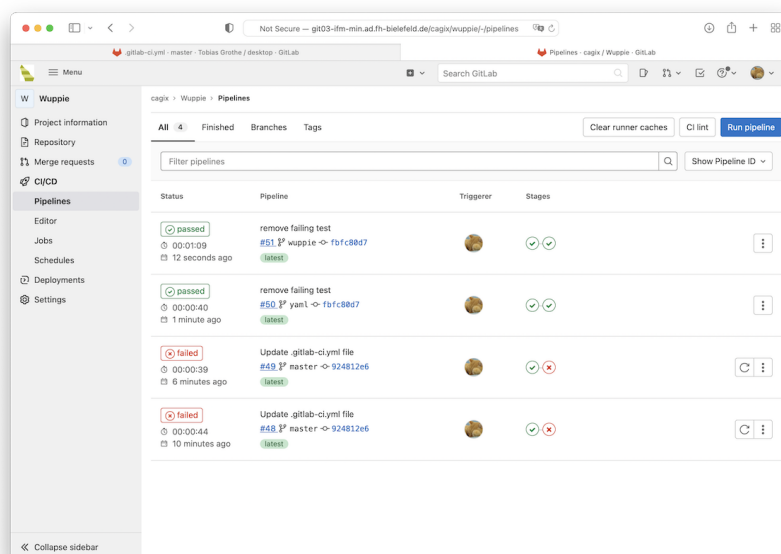
3.3.2.3. Beispiele für verbreitete CI-Umgebungen

- [Jenkins](#)
- [GitLab CI/CD](#)
- [GitHub Actions](#) und [GitHub CI/CD](#)
- [Bamboo](#)
- [Travis CI](#)

3.3.3. GitLab CI/CD

Siehe auch [“Get started with Gitlab CI/CD”](#). (Für den Zugriff wird VPN benötigt!)

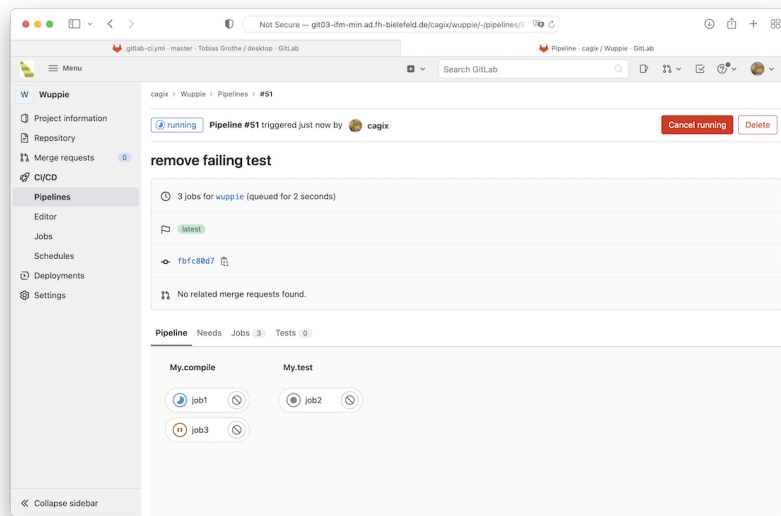
3.3.3.1. Übersicht über Pipelines



- In Spalte “Status” sieht man das Ergebnis der einzelnen Pipelines: “pending” (die Pipeline läuft gerade), “cancelled” (Pipeline wurde manuell abgebrochen), “passed” (alle Jobs der Pipeline sind sauber durchgelaufen), “failed” (ein Job ist fehlgeschlagen, Pipeline wurde deshalb abgebrochen)
- In Spalte “Pipeline” sind die Pipelines eindeutig benannt aufgeführt, inkl. Trigger (Commit und Branch)
- In Spalte “Stages” sieht man den Zustand der einzelnen Stages

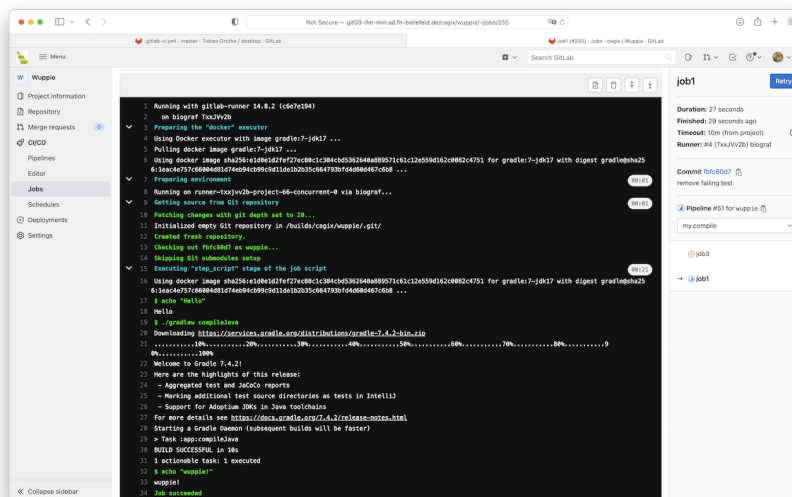
Wenn man mit der Maus auf den Status oder die Stages geht, erfährt man mehr bzw. kann auf eine Seite mit mehr Informationen kommen.

3.3.3.2. Detailansicht einer Pipeline



Wenn man in eine Pipeline in der Übersicht klickt, werden die einzelnen Stages dieser Pipeline genauer dargestellt.

3.3.3.3. Detailansicht eines Jobs



Wenn man in einen Job einer Stage klickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

3.3.3.4. GitLab CI/CD: Konfiguration mit YAML-Datei

Datei `.gitlab-ci.yml` im Projekt-Ordner:

```
1  stages:
2    - my.compile
3    - my.test
4
5  job1:
6    script:
7      - echo "Hello"
8      - ./gradlew compileJava
9      - echo "wuppie!"
10   stage: my.compile
11   only:
12     - wuppie
13
14  job2:
15    script: "./gradlew test"
16    stage: my.test
17
18  job3:
19    script:
20      - echo "Job 3"
21    stage: my.compile
```

Stages

Unter `stages` werden die einzelnen Stages einer Pipeline definiert. Diese werden in der hier spezifizierten Reihenfolge durchgeführt, d.h. zuerst würde `my.compile` ausgeführt, und erst wenn alle Jobs in `my.compile` erfolgreich ausgeführt wurden, würde anschließend `my.test` ausgeführt.

Dabei gilt: Die Jobs einer Stage werden (potentiell) parallel zueinander ausgeführt, und die Jobs der nächsten Stage werden erst dann gestartet, wenn alle Jobs der aktuellen Stage erfolgreich beendet wurden.

Wenn keine eigenen `stages` definiert werden, kann man ([lt. Doku](#)) auf die Default-Stages `build`, `test` und `deploy` zurückgreifen. **Achtung:** Sobald man eigene Stages definiert, stehen diese Default-Stages *nicht* mehr zur Verfügung!

Jobs

`job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `script`). Alternativ kann man die bei `job2` gezeigte Syntax nutzen, wenn nur ein Befehl zu bearbeiten ist.

Die Befehle werden von GitLab CI/CD in einer Shell ausgeführt.

- Die Jobs `job1` und `job2` sind der Stage `my.compile` zugeordnet (Abschnitt `stage`). Einer Stage können mehrere Jobs zugeordnet sein, die dann parallel ausgeführt werden.

Wenn ein Job nicht explizit einer Stage zugeordnet ist, wird er ([lt. Doku](#)) zur Default-Stage `test` zugewiesen. (Das geht nur, wenn es diese Stage auch gibt!)

- Mit `only` und `except` kann man u.a. Branches oder Tags angeben, für die dieser Job ausgeführt (bzw. nicht ausgeführt) werden soll.

Durch die Kombination von Jobs mit der Zuordnung zu Stages und Events lassen sich unterschiedliche Pipelines für verschiedene Zwecke definieren.

3.3.3.5. Hinweise zur Konfiguration von GitLab CI/CD

Im Browser in den Repo-Einstellungen arbeiten:

1. Unter `Settings` > `General` > `Visibility, project features, permissions` das CI/CD aktivieren
2. Prüfen unter `Settings` > `CI/CD` > `Runners`, dass unter `Available shared Runners` mind. ein shared Runner verfügbar ist (mit grün markiert ist)
3. Unter `Settings` > `CI/CD` > `General pipelines` einstellen:
 - `Git strategy: git clone`
 - `Timeout: 10m`
 - `Public pipelines: false` (nicht angehakt)
4. YAML-File (`.gitlab-ci.yml`) in Projektwurzel anlegen, Aufbau siehe oben
5. Build-Skript erstellen, **lokal** lauffähig bekommen, dann in Jobs nutzen
6. Im `.gitlab-ci.yml` die relevanten Branches einstellen (s.o.)
7. Pushen, und unter `CI/CD` > `Pipelines` das Builden beobachten
 - in Status reinklicken und schauen, ob und wo es hakt
8. `README.md` anlegen in Projektwurzel (neben `.gitlab-ci.yml`), Markdown-Schnipsel aus `Settings` > `CI/CD` > `General pipelines` > `Pipeline status` auswählen und einfügen

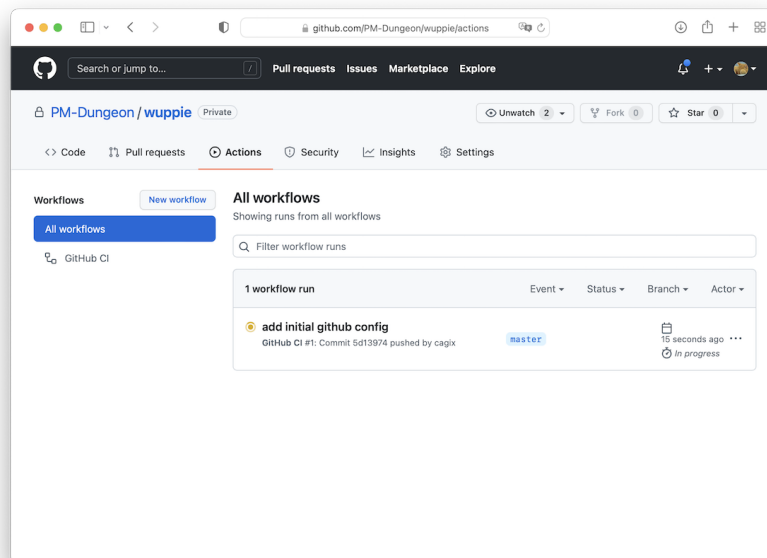
Optional:

9. Ggf. Schedules unter `CI/CD` > `Schedules` anlegen
10. Ggf. extra Mails einrichten: `Settings` > `Integrations` > `Pipeline status emails`

3.3.4. GitHub Actions

Siehe “[GitHub Actions: Automate your workflow from idea to production](#)” und auch “[GitHub: CI/CD explained](#)”.

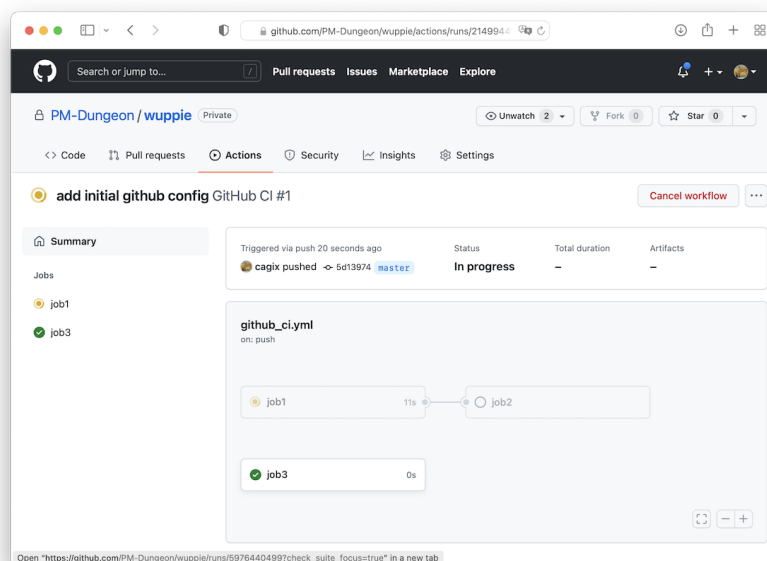
3.3.4.1. Übersicht über Workflows



Hier sieht man das Ergebnis der letzten Workflows. Dazu sieht man den Commit und den Branch, auf dem der Workflow gelaufen ist sowie wann er gelaufen ist. Über die Spalten kann man beispielsweise nach Status oder Event filtern.

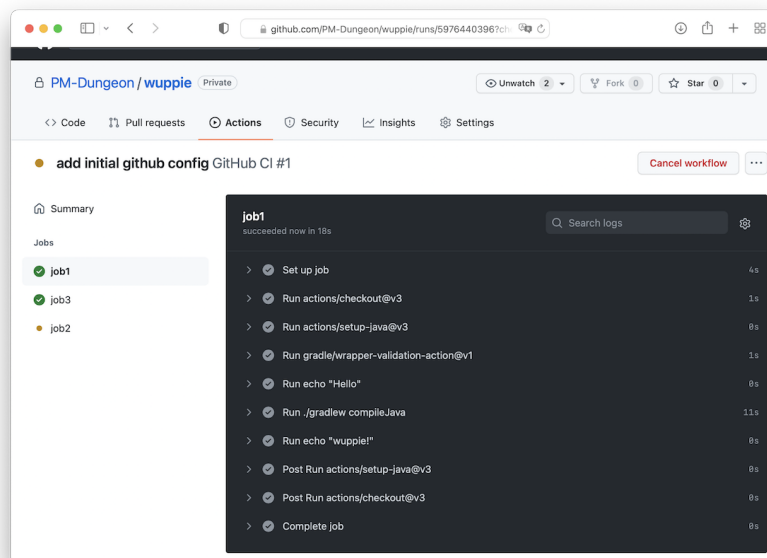
In der Abbildung ist ein Workflow mit dem Namen “GitHub CI” zu sehen, der aktuell noch läuft.

3.3.4.2. Detailansicht eines Workflows



Wenn man in einen Workflow in der Übersicht anklickt, werden die einzelnen Jobs dieses Workflows genauer dargestellt. „job3“ ist erfolgreich gelaufen, „job1“ läuft gerade, und „job2“ hängt von „job1“ ab, d.h. kann erst nach dem erfolgreichen Lauf von „job2“ starten.

3.3.4.3. Detailansicht eines Jobs



Wenn man in einen Job anklickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

3.3.4.4. GitHub Actions: Konfiguration mit YAML-Datei

Workflows werden als YAML-Dateien im Ordner `.github/workflows/` angelegt.

```

1 name: GitHub CI
2
3 on:
4   # push on master branch
5   push:
6     branches: [master]
7   # manually triggered
8   workflow_dispatch:
9
10 jobs:
11
12   job1:
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v6
16       - uses: actions/setup-java@v5

```

```
17     with:
18         java-version: '25'
19         distribution: 'temurin'
20     - uses: gradle/actions/wrapper-validation@v6
21     - run: echo "Hello"
22     - run: ./gradlew compileJava
23     - run: echo "wuppie!"
24
25 job2:
26     needs: job1
27     runs-on: ubuntu-latest
28     steps:
29     - uses: actions/checkout@v6
30     - uses: actions/setup-java@v5
31       with:
32         java-version: '25'
33         distribution: 'temurin'
34     - uses: gradle/actions/wrapper-validation@v6
35     - run: ./gradlew test
36
37 job3:
38     runs-on: ubuntu-latest
39     steps:
40     - run: echo "Job 3"
```

Workflowname und Trigger-Events

Der Name des Workflows wird mit dem Eintrag `name` spezifiziert und sollte sich im Dateinamen widerspiegeln, also im Beispiel `.github/workflows/github_ci.yml`.

Im Eintrag `on` können die Events definiert werden, die den Workflow triggern. Im Beispiel ist ein Push-Event auf dem `master`-Branch definiert sowie mit `workflow_dispatch`: das manuelle Triggern (auf einem beliebigen Branch) freigeschaltet.

Jobs

Die Jobs werden unter dem Eintrag `jobs` definiert: `job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `steps`), die auf einem aktuellen virtualisierten Ubuntu-Runner ausgeführt werden.

Es wird zunächst das Repo mit Hilfe der Checkout-Action ausgecheckt (`uses: actions/checkout@v6`), das JDK eingerichtet/installiert (`uses: actions/setup-java@v5`) und der im Repo enthaltene Gradle-Wrapper auf Unversehrtheit geprüft (`uses: gradle/actions/wrapper-validation@v6`).

Die Actions sind vordefinierte Actions und im Github unter github.com/ + Action zu finden, d.h. `actions/checkout` oder `actions/setup-java`. Actions können von jedermann definiert und bereitgestellt werden, in diesem Fall handelt es sich um von GitHub selbst im Namespace “actions” bereit gestellte direkt nutzbare Actions. Man kann Actions auch selbst im Ordner `.github/actions/` für das Repo definieren (Beispiel: pfla.github.io).

Mit `run` werden Befehle in der Shell auf dem genutzten Runner (hier Ubuntu) ausgeführt.

- Die Jobs `job2` ist von `job1` abhängig und wird erst gestartet, wenn `job1` erfolgreich abgearbeitet ist.

Ansonsten können die Jobs prinzipiell parallel ausgeführt werden.

Durch die Kombination von Workflows mit verschiedenen Jobs und Abhängigkeiten zwischen Jobs lassen sich unterschiedliche Pipelines (“Workflows”) für verschiedene Zwecke definieren.

Es lassen sich auch andere Runner benutzen, etwa ein virtualisiertes Windows oder macOS. Man kann auch über einen “Matrix-Build” den Workflow auf mehreren Betriebssystemen gleichzeitig laufen lassen.

Man kann auch einen Docker-Container benutzen. Dabei muss man beachten, dass dieser am besten aus einer Registry (etwa von Docker-Hub oder aus der GitHub-Registry) “gezogen” wird, weil das Bauen des Docker-Containers aus einem Docker-File in der Action u.U. relativ lange dauert.

Der Gradle-Wrapper muss ausführbar sein

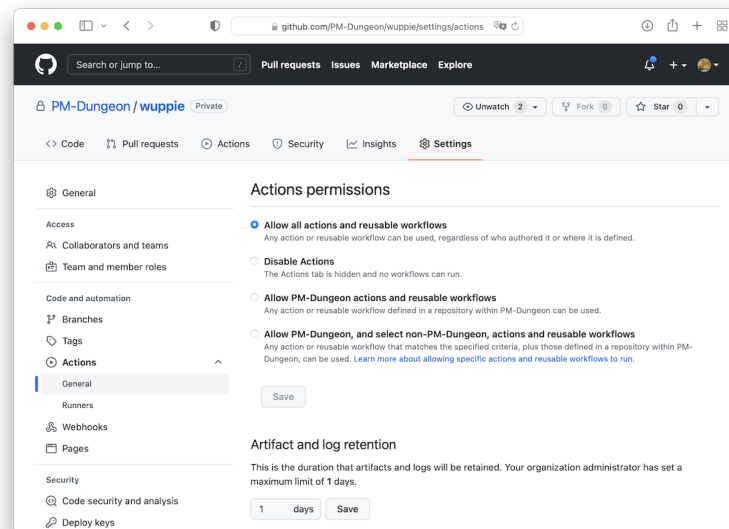
Im Step – `run: ./gradlew test` wird der Gradle-Wrapper ausgeführt, der im Repo eingchecked ist. Genauer gesagt wird das Shell-Skript `gradlew` gestartet. Dies ist unter POSIX-Systemen (u.a. Linux) nur möglich, wenn das sogenannte `x`-Bit (Unix-Executable-Bit) gesetzt ist. Entweder setzen Sie dieses im Workflow *jedes Mal als extra Step* vor einem Aufruf von `gradlew`, etwa per – `run: chmod +x gradlew`, oder Sie checken das Shell-Skript einmalig mit dem gesetzten `x`-Bit im Git-Repo ein: `git add --chmod=+x gradlew`.

Randnotiz: Wenn Sie auf einem POSIX-System arbeiten (Linux, macOS, BSD), dann wird das `x`-Bit bereits beim Initialisieren des Projekts durch `gradle init` oder über IntelliJ automatisch korrekt gesetzt und ins Repo eingchecked. Windows-Dateisysteme kennen das Unix-Executable-Bit nicht, deshalb muss man hier von Hand nacharbeiten. Alternativ arbeiten Sie auf Ihrem Windows-Rechner am besten in einer WSL-Umgebung.

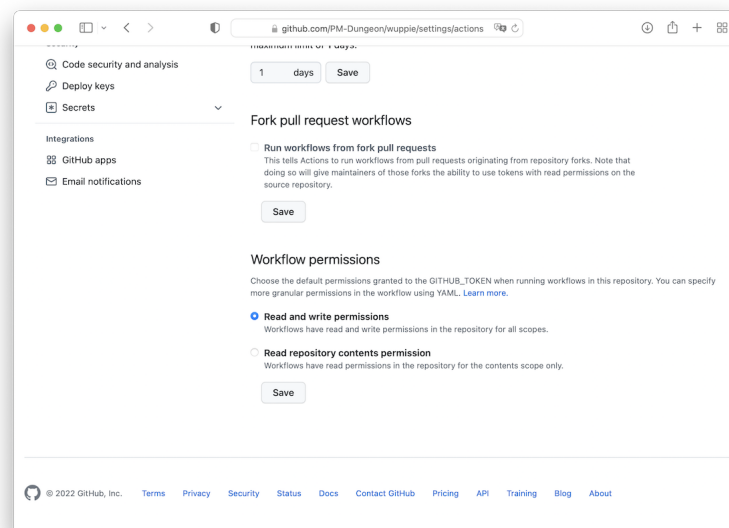
3.3.4.5. Hinweise zur Konfiguration von GitHub Actions

Im Browser in den Repo-Einstellungen arbeiten:

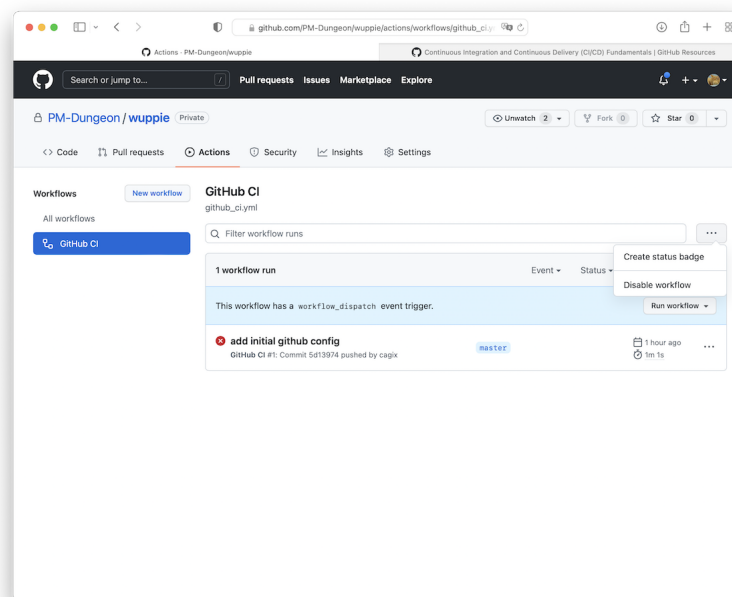
1. Unter `Settings > Actions > General > Actions permissions` die Actions aktivieren (Auswahl, welche Actions erlaubt sind)



2. Unter **Settings** > **Actions** > **General** > **Workflow permissions** ggf. bestimmen, ob die Actions das Repo nur lesen dürfen oder auch zusätzlich schreiben dürfen



3. Unter **Actions** > **<WORKFLOW>** den Workflow ggf. deaktivieren:



3.3.5. Wrap-Up

Überblick über Continuous Integration:

- Konfigurierbare Aktionen, die auf dem Gitlab-/GitHub-Server ausgeführt werden
- Unterschiedliche Trigger: Commit, Merge, ...
- Aktionen können Branch-spezifisch sein
- Aktionen können von anderen Aktionen abhängen

Zum Nachlesen

Eine gute Quelle zum Nachlesen bietet der Blog [What is CI/CD? \(GitHub\)](#).

Lernziele

- k2: Ich kann Arbeitsweise von/mit CI (GitHub, GitLab) erklären

Challenges

Betrachten Sie erneut das Projekt [Theatrical Players Refactoring Kata](#). Erstellen Sie für dieses Projekt einen GitHub-Workflow, der das Projekt kompiliert und die Testsuite ausführt (nur für den Java-Teil, den restlichen Code können Sie ignorieren).

Dabei soll das Ausführen der JUnit-Tests nur dann erfolgen, wenn das Kompilieren erfolgreich durchgeführt wurde.

Der Workflow soll automatisch für Commits in den Hauptbranch sowie für Pull-Requests loslaufen. Es soll zusätzlich auch manuell aktivierbar sein.

3.4. Einführung in ANTLR

TL;DR

In dieser Veranstaltung lernen Sie ANTLR4 als “Blackbox-Werkzeug” kennen, ohne sich bereits tief mit Compilerbau beschäftigen zu müssen. Ausgehend von einem gegebenen Grammatik-File erzeugt ANTLR automatisch Java-Klassen für Lexer, Parser, Kontextklassen und Visitors. Über Gradle binden Sie ANTLR als Plugin und als Bibliothek ein und lassen beim Build-Prozess aus den `.g4`-Dateien den passenden Code generieren.

Im Zentrum steht nicht das Schreiben eigener Grammatiken, sondern die praktische Nutzung der generierten Klassen: Diese übersetzen eingegebenen Text zunächst in Token, anschließend in einen Parse-Baum, und traversieren diesen Baum mit dem Visitor-Pattern. Dabei arbeiten Sie mit Kontextklassen wie `FooParser.ProgContext` oder `FooParser.StmtContext` und überschreiben Methoden wie `visitProg` oder `visitStmt` in einer eigenen Visitor-Klasse, die von `FooBaseVisitor<T>` erbt. Ziel ist, dass Sie den generierten Baum als Datenstruktur verstehen und verarbeiten können - etwa zum Auswerten einfacher Ausdrücke oder als Grundlage für Anwendungen wie Syntax-Highlighting. Die theoretischen Grundlagen zu Grammatiken, Lexer- und Parser-Konstruktion sowie abstrakten Syntaxbäumen folgen dann systematisch im Modul Compilerbau im 3. Semester.

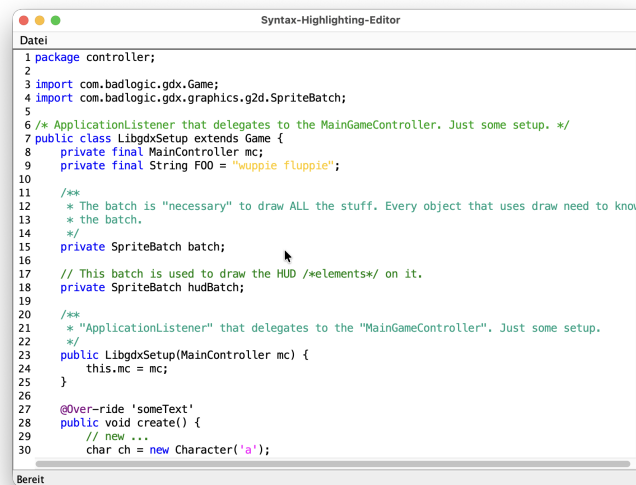
Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo:

- Beispiel zu Lexing & Parsing & Parse-Tree [\[YT\]](#), [\[HSBI\]](#)
- Live-Coding: Einbinden von ANTLR in Gradle, Arbeiten mit dem generierten Lexer und Parser und Programmieren eines Visitors [\[YT\]](#), [\[HSBI\]](#)

3.4.1. Motivation & Einordnung



Von regulären Ausdrücken zu “richtigen” Bäumen: ANTLR

Kurzer Rückblick auf das Übungsblatt mit Syntax-Highlighting via Regex: Regex erkennt Muster im Text, aber kennt keine Struktur.

Problemstellung:

- Wie komme ich von einzelnen Mustern zu einer Satzstruktur?
- Wie kann ich gültigen Java-Code von ungültiger Syntax unterscheiden?

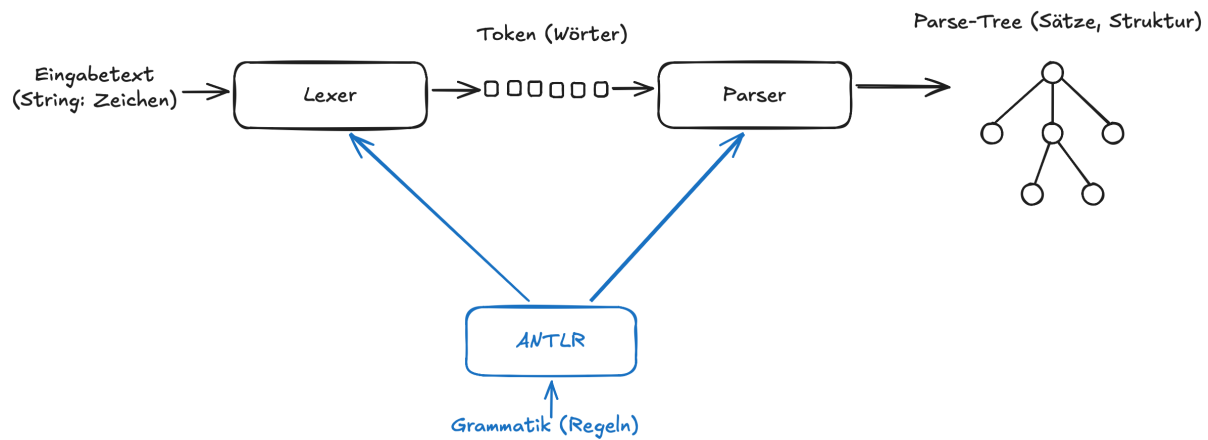
ANTLR4 als Lösung: [ANTLR](#) ist ein in Java geschriebenes Tool und eine Bibliothek, welche aus Text eine Struktur (einen Parse-Baum) erzeugt. ANTLR ist ein recht beliebtes Standardtool und kann die generierten Artefakte (Lexer, Parser, ...) nicht nur in Java, sondern in verschiedenen weiteren Sprachen erzeugen, beispielsweise Python, C++, CSharp, JavaScript, Go, ...

Important

Man kann ANTLR als eigenständiges Tool installieren und/oder über das Gradle-Plugin nutzen und/oder das IntelliJ-Plugin einsetzen. Wir gehen hier den Weg über das Gradle-Plugin, d.h. es gibt keinen (guten) Grund für eine “richtige” Installation von ANTLR.

Einordnung ins Curriculum: Wir betrachten jetzt im 2. Semester nur die rein praktische Nutzung als Bibliothek. Später im 3. Semester werden wir tiefer in die Grundlagen des Compilerbaus einsteigen und uns Grammatiken, Lexer und Parser, AST usw. näher anschauen.

3.4.2. ANTLR als Blackbox-Pipeline



Wir nutzen ANTLR als:

- Java-Bibliothek + Codegenerator, den wir über Gradle einbinden
- Eingabe: eine vorgegebene Grammatikdatei + Ihr Quelltext
- Ausgabe: automatisch erzeugte Java-Klassen, die einen zum eingegebenen Quelltext passenden Baum repräsentieren

Important

Wichtige Begriffe:

- **Lexer:** zerteilt Zeichenstrom (Eingabe) in eine Folge von Wörtern (Token)
- **Token:** Tupel (`Tokenname`, `optional: Wert`)
- **Parser:** unterteilt Tokensequenz in gültige Sätze -> Parse Tree
- **Parse Tree:** repräsentiert die Struktur der Sätze, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.
- **Grammatik:** Regeln für den Aufbau der betrachteten Sprache (formale Beschreibung der Wörter und Sätze)
- **ANTLR** generiert aus der Grammatik einen Lexer und Parser plus diverse Hilfsklassen.

Wie ein Lexer oder Parser funktioniert, was genau eine Grammatik ist usw. schauen wir uns in Compilerbau an. Für Prog2 brauchen Sie nur ein grundlegendes Bild und die Begriffe.

3.4.3. Technische Einbindung (Gradle, Projektstruktur)

ANTLR im Java/Gradle-Projekt nutzen: Einbindung als Tool (Gradle-Plugin) sowie als Bibliothek (Dependencies).

Basiskonfiguration:

```

1 plugins {
2     id 'java'
  
```

```
3     id 'antlr'
4 }
5
6 repositories {
7     mavenCentral()
8 }
9
10 dependencies {
11     antlr 'org.antlr:antlr4:4.13.2'
12     implementation 'org.antlr:antlr4-runtime:4.13.2'
13 }
```

In der Gradle-Konfiguration `build.gradle` wird das ANTLR-Plugin für Gradle aktiviert und zusätzlich werden die Dependencies für die ANTLR-Bibliothek konfiguriert. Die Grammatik-Dateien liegen dann im Source-Tree unterhalb von `src/main/antlr/`.

Explizite Angabe der Pfade für die generierten Klassen

Das ANTLR-Plugin für Gradle hat eine eigene Vorstellung, wohin die generierten Klassen geschrieben werden. Die Defaults passen in der Regel sehr gut, aber in der Praxis ist es oft hilfreich, den Ausgabeordner explizit anzugeben und auch für IntelliJ als Source-Ordner zu kennzeichnen.

Dies erreicht man beispielsweise durch einen zusätzlichen Block in `build.gradle`, der ein Verzeichnis wie `build/generated-src/antlr/main` definiert, dieses als zusätzlichen Java-Source-Ordner einträgt. Zusätzlich kann man den Task `generateGrammarSource` so konfigurieren, dass automatisch immer die Klassen für das Visitor-Pattern mit generiert werden:

```
1 def antlrGenDir = layout.buildDirectory.dir('generated-src/antlr/main')
2
3 sourceSets {
4     main {
5         java.srcDir(antlrGenDir)
6     }
7 }
8
9 tasks.named('generateGrammarSource') {
10     maxHeapSize = '64m'
11     arguments.addAll(['-visitor', '-long-messages'])
12     outputDirectory = antlrGenDir.get().asFile
13 }
```

Build-Prozess

Mit der beschriebenen Konfiguration wird ANTLR in den Gradle-Build-Prozess eingebunden. Bei jedem `./gradlew build` werden aus den Grammatiken in `src/main/antlr/` mit ANTLR die entsprechenden Java-Klassen (Lexer, Parser, *Context-Klassen, Visitor, ...) generiert und in einem Unterordner des Build-Ordners gespeichert. Das hat den Vorteil, dass die generierten Klassen von Git ignoriert werden, aber sie müssen für die IDE explizit als "Sourcen" deklariert werden (dies passiert durch das ANTLR-Plugin in Gradle normalerweise automatisch; mit der oben skizzierten zusätzlichen Konfiguration ist man auf der sicheren Seite). anschließend werden alle Java-Klassen im Projekt ganz normal kompiliert und (bei `./gradlew run`) ausgeführt.

Das bedeutet aber auch, dass nach einem `./gradlew clean` die generierten Klassen ebenfalls mit entfernt werden (sie liegen ja im Build-Ordner). Damit fehlen dann in Ihrem Source-Code zunächst die Importe für Lexer, Parser, Visitor etc. und die IDE zeigt eine Menge Fehler an. Führen Sie dann einmal `./gradlew build` oder `./gradlew generateGrammarSource` aus, um die generierten ANTLR-Dateien wieder neu zu erzeugen.

Sie arbeiten dann nur noch mit den Lexer-/Parser-Klassen wie etwa `FooParser`, den Kontext-Klassen wie `FooParser.StatementContext` und erweitern den generierten Basis-Visitor wie `FooBaseVisitor`. (Der Präfix `Foo` kommt von der betrachteten Grammatik, diese würde hier also `Foo.g4` heissen.)

Tip

Diese Gradle-Konfiguration müssen Sie sich nicht im Detail merken. Nutzen Sie sie in diesem Kurs als Template und passen Sie sie bei Bedarf an.

3.4.4. Beispielgrammatik

```
1 grammar MiniCalc;
2
3 // Programm besteht aus beliebig vielen Statements
4 prog : stmt* EOF ;
5
6 // Statement: entweder Zuweisung oder nackter Ausdruck
7 stmt : ID '=' expr ';'
8       | expr ';'
9       ;
10
11 // Ausdruck: eine oder mehrere Zahlen, addiert
12 expr : INT ('+' INT)* ;
13
14 // LEXER-Regeln (Token)
15 ID   : [a-z][a-zA-Z0-9_]* ;
16 INT  : [0-9]+ ;
17
18 WS   : [ \t\r\n]+ -> skip ;
```

Die gezeigte Grammatik besteht aus verschiedenen Teilen. Als erste Zeile findet man immer die Deklaration `grammar <NAME>;`, wobei der Name `<NAME>` auch der Dateiname ist (`NAME.g4`) und später den Präfix für die generierten Lexer-/Parser-/Visitor-Klassen bildet.

Darunter finden sich verschiedene Regeln, die die betrachtete Sprache beschreiben. Es gibt zwei Arten von Regeln:

1. **Lexer-Regeln:** Diese beginnen mit einem Grossbuchstaben und definieren einen regulären Ausdruck, der auf den Eingabetext angewendet wird.

Jede Lexer-Regel entspricht einem Token. Im Beispiel gibt es die Regel `INT`, die den regulären Ausdruck `[0-9]+` anwendet. Damit matcht das Token `INT` auf alle Zeichenfolgen, die aus einer oder mehreren Ziffern bestehen. Zusätzlich gibt es noch nicht benannte Token, das sind die in `'` eingeschlossenen Zeichenketten, beispielsweise `'='` oder `','`. Die Whitespace-Token (`WS`) werden im Beispiel zwar gematcht, aber danach verworfen (`-> skip`) - wir brauchen diese nicht für die Prüfung der Strukturen.

2. **Parser-Regeln:** Diese beginnen mit einem Kleinbuchstaben und können andere Parser-Regeln “aufrufen” und auch Lexer-Regeln beinhalten.

Die Parser-Regeln nutzen auch die aus regulären Ausdrücken bekannten Operatoren + (einmal oder öfter) und * (beliebig oft). Damit kann man die Regel

```
1 prog : stmt* EOF;
```

so lesen: Ein Programm `prog` besteht aus beliebig vielen Statements (`stmt`), gefolgt von einem `EOF` (die Eingabe ist zu Ende, “End Of File”).

Die Regel

```
1 expr : INT ('+' INT)*;
```

bedeutet: eine Zahl `INT` (Token), gefolgt von keinmal, einmal oder mehrfach der Folge + `INT`. Also sind z.B. `1`, `1 + 2` oder `1 + 2 + 3 + 4` gültige Ausdrücke.

Die erste Parser-Regel im File ist die sogenannte “Start-Regel”, die wir auf dem generierten Parser aufrufen können.

3.4.5. Minimaler Java-Code: Zerlegen der Eingabe in Token

```
1 var input = CharStreams.fromString(text);
2 var lexer = new MiniCalcLexer(input);
3 var tokens = new CommonTokenStream(lexer);
4
5 tokens.fill(); // fill stream (fetch all tokens from lexer)
6
7 for (var t : tokens.getTokens()) {
8     var tokenName = MiniCalcLexer.VOCABULARY.getSymbolicName(t.getType());
9     System.out.printf(
10         "%-10s line=%d col=%d text='%s'\n",
11         tokenName, t.getLine(), t.getCharPositionInLine(), t.getText());
12 }
```

Aus einer Grammatik `MiniCalc.g4` wurde ein Lexer generiert, der über `MiniCalcLexer` zur Verfügung steht. Der Lexer erwartet als Input einen `CharStream`, den wir aus dem Input (String) erzeugen und dem Konstruktor von `MiniCalcLexer` übergeben. Das Lexer-Objekt wird wiederum in den Konstruktor von `CommonTokenStream` übergeben und stellt den Token-Stream dar, den der aus der Grammatik `MiniCalc` generierte Lexer für den Eingabe-String erzeugt.

Normalerweise arbeiten wir nicht direkt mit dem Token-Stream, sondern übergeben diesen dem Parser. Im Beispiel wird demonstriert, wie man beispielsweise alle gefundenen Token der Reihe nach ausgeben kann. Dazu muss einmalig `tokens.fill()` aufgerufen werden (dies übernimmt sonst der Parser).

Die Eingabe `a = 1 + 2;` liefert bei der gezeigten Grammatik und dem Beispiel-Code die folgende Ausgabe:

```
1 ID      line=1 col=0 text='a'
2 null    line=1 col=2 text='='
3 INT     line=1 col=4 text='1'
4 null    line=1 col=6 text='+'
5
```

```

5 INT      line=1 col=8 text='2'
6 null     line=1 col=9 text=';';
7 EOF      line=1 col=10 text='<EOF>'

```

Man erkennt, wie die Eingabe in einzelne Wörter (Token) zerlegt wurde. Leerzeichen wurden entfernt. Für benannte Token, d.h. Lexer-Regeln wie `ID` und `INT`, wird der Tokenname mit ausgegeben (`ID`, `INT`). Für die impliziten Token, die in der Grammatik nur über Literale wie `'='` angelegt sind, gibt es keinen symbolischen Namen - `VOCABULARY.getSymbolicName()` liefert dafür `null`. Weiterhin sieht man für jedes Token, in welcher Zeile es gefunden wurde und an welcher Position es startet. `EOF` ist ein vordefiniertes Token, welches das Ende der Eingabe kennzeichnet.

Note

Der gesamte Eingabetext muss so in gültige Token überführt werden können, dass jede Zeichenposition zu einem Token gehört. Wenn der Lexer auf Zeichen stösst, die zu keiner der definierten Token-Regeln passen, meldet er einen Fehler (über seine Error-Listener) und je nach Konfiguration kann dies auch zu einer Exception führen.

3.4.6. Minimaler Java-Code: Text -> Baum

```

1 var input = CharStreams.fromString(text);
2 var lexer = new MiniCalcLexer(input);
3 var tokens = new CommonTokenStream(lexer);
4
5 var parser = new MiniCalcParser(tokens);
6 var tree = parser.prog(); // Wurzelknoten des Baums (Startregel der Grammatik)
7
8 IO.println(tree.toStringTree(parser));

```

Hier wird das übliche Vorgehen gezeigt, wenn man mit dem Parse-Tree arbeiten möchte. Aus dem Eingabetext wird ein `CharStream` erzeugt und damit ein `MiniCalcLexer`. Mit diesem wird der `CommonTokenStream` angelegt und in einen neuen `MiniCalcParser` gesteckt. Damit haben wir einen Parser, der von ANTLR beim Kompilieren über Gradle aus der Grammatik generiert wurde und der für den übergebenen Eingabetext die vom Lexer gebildeten Token in einen Baum übersetzt. Da unsere Grammatik mit der Regel `prog` anfängt ("Start-Regel"), können wir uns den Baum mit Hilfe von `parser.prog()` zurückgeben lassen und damit weiter arbeiten. Die Baumwurzel `tree` ist ein Objekt vom Typ `MiniCalcParser.ProgContext`.

Note

Wenn die Token nicht entsprechend den Regeln in der Grammatik auftauchen, meldet der Parser Syntaxfehler. Standardmässig versucht ANTLR4, sich von Fehlern zu erholen und weiterzuparsen; je nach Konfiguration (z.B. mit einer "Bail-Strategie") kann der Parser aber auch mit einer Exception abbrechen.

Die Eingabe `a = 1 + 2;` liefert bei der gezeigten Grammatik und dem Beispiel-Code die folgende Ausgabe:

```

1 (prog (stmt a = (expr 1 + 2) ;) <EOF>)

```

Dabei wird jeder Knoten in Klammern ausgegeben: Zuerst der Knotenname, danach die Kinder (die selbst wieder Knoten sind). Die Token bilden die Blätter des Baumes; hier wird nur der Wert ausgegeben (nicht der

Tokenname).

Diesen Code können Sie als Schablone verwenden. Ab hier arbeiten wir normalerweise nur noch mit `tree`...

3.4.7. Der Parse-Tree: Struktur

```

1  grammar MiniCalc;
2
3  prog  : stmt* EOF ;
4  stmt  : ID '=' expr ';'
5         | expr ';'
6         ;
7  expr  : INT ('+' INT)* ;
8
9  ID    : [a-z][a-zA-Z0-9_]* ;
10 INT   : [0-9]+ ;
11 WS    : [ \t\r\n]+ -> skip ;

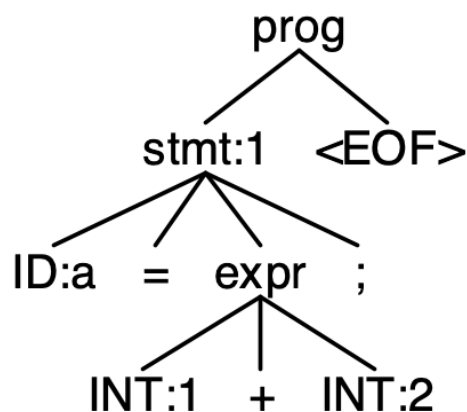
```

Eingabe `a = 1 + 2;` liefert:

```

1  (prog (stmt a = (expr 1 + 2) ;) <EOF>)

```



Der Parse-Tree spiegelt in seiner Struktur direkt die Regeln der Grammatik wider. Jede verwendete Parserregel bildet einen (inneren) Knoten im Baum, die konkreten Eingabetokens bilden die Blätter.

- Wir starten mit `prog` als Startregel und damit als Wurzelknoten. Die Regel `prog` erlaubt beliebig viele `stmt` gefolgt von `EOF`. In unserer Eingabe gibt es genau ein `stmt`, deshalb hat `prog` zwei Kinder:
 - einen `stmt`-Knoten, und
 - ein `EOF`-Token als Blattknoten.
- Die Regel `stmt` hat zwei Alternativen:
 - `ID '=' expr ';' ;`, oder
 - `expr ';' ;`.

Unsere Eingabe `a = 1 + 2`; passt zur ersten Alternative. Entsprechend hat der `stmt`-Knoten die folgenden Kinder:

- ein `ID`-Token mit dem Wert `a`,
- das (implizite) Token `=`,
- einen `expr`-Knoten, und
- das (implizite) Token `;`.

- Die Regel `expr` lautet:

```
1  expr : INT ('+' INT)* ;
```

Das bedeutet: ein `INT`, gefolgt von beliebig vielen Wiederholungen von `+` `INT`. In der Eingabe `1 + 2` ist das:

- ein erstes `INT`-Token mit dem Wert `1`,
- das (implizite) Token `+`, und
- ein zweites `INT`-Token mit dem Wert `2`.

Diese drei Tokens bilden die Blätter unterhalb des `expr`-Knotens.

Wenn der Parser am Ende beim `<EOF>`-Token angekommen ist und alle Regeln erfolgreich angewendet wurden, ist die komplette Eingabe abgedeckt und der Parse-Tree vollständig.

3.4.8. Der Parse-Tree: Klassenhierarchie

```
1  grammar MiniCalc;
2
3  prog  : stmt* EOF ;
4  stmt  : ID '=' expr ';'
5         | expr ';'
6         ;
7  expr  : INT ('+' INT)* ;
8
9  ID    : [a-z][a-zA-Z0-9_]* ;
10 INT   : [0-9]+ ;
11 WS    : [ \t\r\n]+ -> skip ;
```



Wie sieht der erzeugte Baum in Java aus?

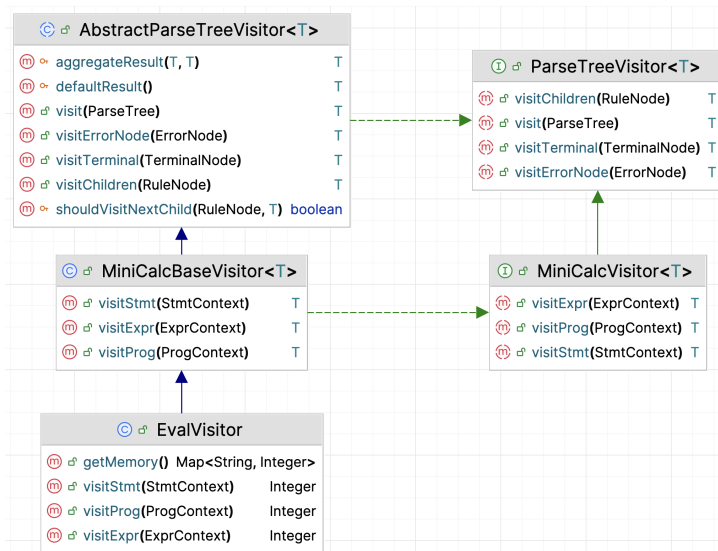
- Grundprinzip:
 - Jeder (nicht-Blatt-) Knoten im Baum ist eine Instanz einer generierten **Kontext-Klasse**
 - Jede **Kontext-Klasse** entspricht einer Grammatik-Regel (nur Parser-Regeln), z.B.:
 - * `MiniCalcParser.ProgContext` (Regel `prog`),
 - * `MiniCalcParser.StmtContext` (Regel `stmt`),
 - * `MiniCalcParser.ExprContext` (Regel `expr`),...
 - Basisklassen (nicht im UML-Diagramm oben gezeigt):
 - * Alle Baumknoten (Kontext-Klassen, Parser-Regeln) erben von einer gemeinsamen Basisklasse `ParserRuleContext` (und dieser wiederum in einer längeren Vererbungskette von `ParseTree`)
 - * Alle Blätter (Token, Lexer-Regeln) erben von einer gemeinsamen Basisklasse `TerminalNode` (und dieser wiederum von `ParseTree`)
- Baumstruktur:
 - Jeder Knoten hat Kinderknoten (andere Kontexte oder Tokens)
 - Zugriff auf Elemente: Beispiel Regel `stmt` : `ID '=' expr ';' | expr ';' ;`
 - * Kontext-Klasse `MiniCalcParser.StmtContext`
 - * Zugriff auf Token `ID`: `TerminalNode ID()`

- * Zugriff auf Kontext `expr: ExprContext expr()`
- Zusätzlich gibt es aus der Basisklasse `ParserRuleContext` für jede Kontext-Klasse noch z.B.:
 - * `int getChildCount()`: Wie viele Kinder hat dieser Knoten?
 - * `ParseTree getChild(int i)`: liefere den Kindknoten mit Index `i` zurück (Indexbereich 0 bis `getChildCount() - 1`)
 - * `String getText()`: liefere den gematchten Text für die Regel zurück (bei Token ist es der gematchte Input für das Token)

3.4.9. Traversierung mit Visitor-Pattern

Den Baum “besuchen” - Visitor-Pattern in ANTLR angewendet.

ANTLR generiert auf Anforderung die nötigen Klassen und Strukturen, um das Visitor-Pattern einfach implementieren zu können. Wir schauen uns hier nur die Anwendung des Patterns an; zur Funktionsweise des Patterns siehe Lektion Visitor-Pattern.



3.4.9.1. ANTLR-Visitor:

- ANTLR generiert ein `MiniCalcVisitor<T>`-Interface und eine `MiniCalcBaseVisitor<T>`-Basisklasse mit Standard-Implementierungen
- Jede Regel `xxx` in der Grammatik erzeugt:
 - eine Kontext-Klasse `XxxContext`, und
 - eine `visitXxx`-Methode `T visitXxx(XxxContext ctx)`
- Jeder Knotentyp hat eine eigene `visitXxx`-Methode, z.B.:
 - `T visitProg(MiniCalcParser.ProgContext ctx)`
 - `T visitStmt(MiniCalcParser.StmtContext ctx)`

- `T visitExpr(MiniCalcParser.ExprContext ctx)`
- Aus der Basisklasse `AbstractParseTreeVisitor<T>` erbt jeder Visitor u.a. die Hilfsmethoden `visit(ParseTree tree)` und `visitChildren(RuleNode node)`:
 - `visit(ParseTree tree)` ist eine praktische Hilfsmethode, die für einen Knoten `tree` an `tree.accept(this)` delegiert
 - `visitChildren(RuleNode node)` traversiert alle Kinder eines Knotens `node`
- Defaultimplementierung im `MiniCalcBaseVisitor<T>`: `T visitXxx(XxxContext ctx) { return visitChildren(ctx); }`

3.4.9.2. Vorgehen:

1. Eigene Visitor-Klasse schreiben, die von `MiniCalcBaseVisitor<T>` ableitet, z.B.

```
1 public class EvalVisitor extends MiniCalcBaseVisitor<Integer> {}
```

2. Nur die Methoden überschreiben, die relevant sind, z.B. `visitProg`, `visitStmt`, `visitExpr`, ...

- Beispielidee für `visitStmt` (informell):

```
1 /** stmt : ID '=' expr ';' | expr ';' */
2 @Override
3 public Integer visitStmt(MiniCalcParser.StmtContext ctx) {
4     if (ctx.ID() != null) { // ID '=' expr ';'
5         String name = ctx.ID().getText();
6         Integer value = visit(ctx.expr());
7         memory.put(name, value);
8         return value;
9     } else
10        return visit(ctx.expr()); // expr ';'
11 }
```

- Grammatik: `stmt : ID '=' expr ';' | expr ';' ;`
- Wenn `ctx.ID() != null`, handelt es sich um eine Zuweisung (`ID '=' expr ';' ;`):
 - * Namen mit `ctx.ID().getText()` auslesen,
 - * Wert mit `visit(ctx.expr())` berechnen,
 - * in einer Map `memory.put(name, value)` speichern (für spätere Verarbeitung).
- Sonst handelt es sich um ein einfaches `expr ';' ;`:
 - * nur den Wert `visit(ctx.expr())` zurückgeben.

3. Visitor anwenden:

```
1 var tree = parser.prog();
2 var eval = new EvalVisitor();
3 var result = eval.visit(tree);
4
5 IO.println("Umgebung: " + eval.getMemory());
```

- Mit `var tree = parser.prog();` den Wurzelknoten holen,

- Visitor-Objekt erzeugen, z.B. `var eval = new EvalVisitor();`,
- `var result = eval.visit(tree);` aufrufen,
- z.B. die Umgebung mit `eval.getMemory()` ausgeben.

3.4.9.3. Vorteile:

- Klare Trennung: Struktur (Baum) vs. Verarbeitung (Visitor)
- Erleichtert spätere Erweiterungen (weitere Visitors für andere Aufgaben, z.B. Auswertung, Pretty-Printer, Linter)

3.4.9.4. Beobachtungen:

- `visitStmt` zeigt explizit, dass `ID` hier nur ein Token ist und dass hier direkt über `ctx.ID().getText()` zugegriffen wird. Für Token gibt es keine Kontext-Objekte, d.h. es gibt kein `IDContext`.
- `visitStmt` zeigt den Umgang mit einer Alternative: Entweder gilt `ID '=' expr ';' oder expr ';'.` D.h. man fragt in diesem Fall ab, ob es eine `ID` gibt und reagiert entsprechend: `if (ctx.ID() != null) -> wenn es keine ID gibt, liefert ctx.ID() den Wert null.`
- Wenn man die Kindknoten nicht direkt per Methode aufrufen will (oder kann, wenn es mehrere Kindknoten mit nicht vorher bekannter Anzahl wie in der Regel `expr : INT ( '+' INT ) * ;` gibt), kann man die tatsächliche Anzahl der Kindknoten per Aufruf `ctx.getChildCount()` abfragen und dann mit `ctx.getChild(i)` gezielt auf ein Kind zugreifen (Indexbereich 0 bis `getChildCount() - 1`).

3.4.9.5. Hinweis: Visitor-Rückgaben und Aggregation in ANTLR vs. zustandsbehaftete Visitoren

Wir nutzen hier in Prog2 einfach einen **zustandsbehafteten Visitor**: Die `visitXYZ`-Methoden arbeiten mit einer internen **Hilfsdatenstruktur** wie `Stack`, `Map` oder `StringBuilder` (oder andere, abhängig vom Ziel) und schreiben ihre Ergebnisse dort hinein. Das ist leicht zu verstehen und man muss sich nicht darum kümmern, wie ANTLR Ergebnisse aus Kindknoten zusammenführt.

Sie überschreiben die `visitXYZ`-Methoden für Knoten, die Ausgaben erzeugen sollen (z.B. `prog`, `stmt`, `expr`). Andere Regeln können unüberschrieben bleiben, solange Sie in Ihren überschriebenen Methoden die notwendigen Kinder mit `visit(...)` (oder `visitChildren(...)`) weiter besuchen.

Beispiel: Pretty-Printing für arithmetische Ausdrücke mit `StringBuilder` (einfach und robust):

```

1  class PrettyPrinterState extends MiniCalcBaseVisitor<Void> {
2      private final StringBuilder out = new StringBuilder();
3
4      // hier fehlt noch visitProg für korrekte Zeilenumbrüche zw. den Statements
5      // hier fehlt noch visitExpr für eine "schöne" Formatierung der Expressions
6
7      @Override
8      public Void visitStmt(MiniCalcParser.StmtContext ctx) {
9          if (ctx.ID() != null) { // stmt : ID '=' expr ';'
10             out.append(ctx.ID().getText()).append(" = ");
11             visit(ctx.expr());
12             out.append(";");

```

```

13     } else {                                // stmt : expr ';'
14         visit(ctx.expr());
15         out.append(";");
16     }
17     return null;
18 }
19
20 public String result() {
21     return out.toString();                    // Zugriff auf das Ergebnis
22 }
23 }

```

Für Fortgeschrittene: Man kann auch mit Rückgabewerten arbeiten und die Standardlogik aus `AbstractParseTreeVisitor` nutzen. Dann ist wichtig zu wissen: `visitChildren()` sammelt die Ergebnisse der Kinder über eine *interne Aggregation*. Wenn man will, dass wirklich alle Kind-Ergebnisse korrekt zusammenkommen (zum Beispiel bei `String`), muss man in der eigenen Visitor-Klasse normalerweise `defaultResult()` und `aggregateResult()` geeignet überschreiben.

Vorsicht: Ein häufiges Problem beim Rückgabe-Visitor ist das "Teile verschlucken". In `AbstractParseTreeVisitor` ist die Standardaggregation so ausgelegt, dass oft nur ein Teilergebnis weitergegeben wird (typisch: das letzte Nicht-Default-Ergebnis). Deshalb muss man für `String` meist `defaultResult()` und `aggregateResult()` überschreiben, damit wirklich wie gewünscht in allen Fällen konkateniert wird.

Beispiel für Fortgeschrittene: Pretty-Printing für arithmetische Ausdrücke mit ANTLR-Aggregation mit Typ-Variablen/Rückgabety `String` (mehr "Magie", aber sauberer):

```

1  class PrettyVisitorAggregation extends MiniCalcBaseVisitor<String> {
2      @Override protected String defaultResult() {
3          return "";
4      }
5
6      @Override
7      protected String aggregateResult(String aggregate, String nextResult) {
8          return aggregate + nextResult;
9      }
10
11     // hier fehlt noch visitProg für korrekte Zeilenumbrüche zw. den Statements
12     // hier fehlt noch visitExpr für eine "schöne" Formatierung der Expressions
13
14     @Override
15     public String visitStmt(MiniCalcParser.StmtContext ctx) {
16         if (ctx.ID() != null)                // stmt : ID '=' expr ';'
17             return ctx.ID().getText() + " = " + visit(ctx.expr()) + ";\n";
18         else                                // stmt : expr ';'
19             return visit(ctx.expr()) + ";\n";
20     }
21
22     // Ergebnis über den Rückgabety - keine Hilfsmethode und interne
23     // Hilfsdatenstruktur notwendig
24 }

```

Merke: `StringBuilder` ist oft die beste Wahl für den Einstieg. Die eingebaute Aggregation ist praktisch, aber nur dann gut nutzbar, wenn man die Sammellogik (`defaultResult/aggregateResult`) passend definiert.

3.4.10. Ausblick: Pattern Matching auf Bäumen (neuere Java-Versionen)

Ausblick auf eine spätere Lesson Sealed Classes & Pattern Matching:

```
1 Object node = ...;
2 switch (node) {
3     case NumberNode n -> ...
4     case BinaryOpNode b -> ...
5     // ...
6 }
```

Statt mit dem Visitor-Pattern durch den Baum zu iterieren, kann man das in neueren Java-Versionen auch mit Pattern Matching auf Typen machen. Dies schauen wir uns in der Sitzung Sealed Classes & Pattern Matching genauer an, hier nur der Ausblick.

3.4.11. Syntaxhighlighting: Vergleich Regex-Ansatz vs. ANTLR-Ansatz

- Regex-Ansatz:
 - Arbeitet rein textbasiert (Zeile für Zeile, Zeichenketten)
 - Schwer, verschachtelte Strukturen korrekt zu behandeln
 - Kaum Information über Kontext (z.B. ob etwas eine Variable, ein Keyword oder Teil eines Strings ist)
- ANTLR-Ansatz:
 - Erstellt einen strukturierten Baum:
 - * Kennt Blöcke, Ausdrücke, Anweisungen, ...
 - Kontextabhängige Verarbeitung wird möglich:
 - * Unterschiedliche Behandlung je nach Position im Baum
- Für das Syntax-Highlighting-Beispiel:
 - Nutzung des Token-Streams vom Lexer für das reine Syntax-Highlighting (z.B. farbige Darstellung je nach Tokentyp, ohne den Parse-Baum zu betrachten)
 - Alternativ Traversierung des Parse-Tree mit einem Visitor:
 - * Highlighting abhängig von Knotentyp statt von vagen Textmustern
 - * Achtung: Token mit `->` `skip` tauchen nicht mehr im Baum auf, d.h. üblicherweise sind das Whitespaces und Kommentare; für exakte Quelltext-Rekonstruktion müssen diese ggf. separat behandelt werden

3.4.12. Ausblick auf das 3. Semester (Compilerbau)

Wie es weitergeht: Vom Parse-Baum zum Compiler

- Sie arbeiten in Prog2 nur mit `FooLexer`, `FooParser`, `XxxContext`-Klassen, `FooBaseVisitor`:
 - den Standard-Code, um `tree = parser.program()` zu bekommen
 - das Wissen: Knoten sind `XxxContext`-Objekte

- einen eigenen Visitor, der von `FooBaseVisitor<...>` erbt
- überschreiben der passenden `visitXxx`-Methoden
- Was Sie heute “unter der Haube” ignorieren durften:
 - Wie die Grammatik aufgebaut ist und warum
 - Wie Lexer und Parser intern funktionieren
 - Unterschied Parse-Baum vs. abstrakter Syntaxbaum (AST)
- Im 3. Semester (Compilerbau) vertiefen wir:
 - Definition eigener Grammatiken
 - Schreiben und Erweitern eigener Sprachen
 - Konstruktion von Lexer und Parser (inkl. ANTLR-Details)
 - Systematische AST-Konstruktion, semantische Analyse, Interpreter, Compiler, Linter, Code-Formatter

Tip

Verbindung zu heute: Alles, was Sie jetzt zu Baumstrukturen, Visitor und Pattern Matching gelernt haben, wird im nächsten Semester direkt wiederverwendet!

3.4.13. Wrap-Up

- ANTLR als Werkzeug: aus einer gegebenen Grammatik werden Lexer, Parser, Kontextklassen und Visitor-Schnittstellen generiert
- Einbettung in ein Java/Gradle-Projekt:
 - `antlr`-Plugin in `build.gradle`
 - `.g4`-Dateien unter `src/main/antlr/`
 - generierte Sourcen im Build-Ordner
- Üblicher Workflow: Eingabetext -> CharStream -> Lexer -> TokenStream -> Parser -> Parse-Tree
- Struktur des Parse-Baums:
 - jedes Knotenobjekt ist eine `XxxContext`-Klasse
 - Blätter sind Token
 - Whitespaces/Kommentare werden i.d.R. übersprungen/entfernt
- Visitor-Pattern in ANTLR:
 - eigene Klasse von generiertem `FooBaseVisitor<T>` ableiten
 - `visitXxx`-Methoden bei Bedarf überschreiben
 - Baum mit `visitor.visit(tree)` traversieren.

Tip

Die gezeigten Beispiele (Grammatik, Visitor, Main-Klasse) sowie die Einbindung von ANTLR in Ihre `build.gradle` finden Sie als **fertig eingerichtetes Gradle-Projekt** im [Haupt-Repo](#). Importieren Sie den Ordner `antlr-demo` als neues Gradle-Projekt in Ihre IDE.

Zum Nachlesen

Sie finden auf der [ANTLR-Homepage](#) Links zur Dokumentation und auch zum [ANTLR-Projekt](#) auf GitHub. Eine ältere, aber dennoch interessante Quelle vom “Kopf” hinter ANTLR (Terence Parr) ist (Parr 2014), die auch online verfügbar ist.

Die [ANTLR-Dokumentation](#) und die [ANTLR-FAQ](#) bieten ebenfalls interessante Einstiegspunkte zur Vertiefung für Fortgeschrittene.

Lernziele

- k2: Sie können erläutern, welche Rolle Grammatik, Lexer, Parser, Kontextklassen und Visitors in einem ANTLR-Projekt spielen, und wie diese in den Build-Prozess mit Gradle eingebunden werden.
- k3: Sie können eine gegebene ANTLR-Grammatik in ein Gradle-Projekt integrieren, den erzeugten Parse-Baum mit einem eigenen Visitor traversieren und daraus eine einfache Verarbeitung (z.B. Auswertung von Ausdrücken oder einfaches Syntax-Highlighting) implementieren.

Challenges

Betrachten Sie die folgende ANTLR-Grammatik:

```
1 grammar OneTwo;  
2  
3 s : a EOF ;  
4 a : '1' a '1' | '2' ;
```

1. Welche Token können Sie hier vom Lexer erwarten?
2. Welche Eingaben würde der Parser akzeptieren?
3. Wie sieht der Parse-Tree aus, welche Knoten und Blätter gibt es und wie sehen die zugehörigen (generierten) Klassen und Methoden aus?

3.5. Debugging

TL;DR

Der Debugger ist ein zentrales Werkzeug in der Softwareentwicklung. Man kann ihn einsetzen, um sowohl Exceptions als auch Logikfehler systematisch zu finden, statt mit im Code strategisch verstreuten `IO.println` den Zustand des ausgeführten Programms zu erraten.

Man kann im Debugger an den zu untersuchenden Stellen **Breakpoints** setzen und die Programmausführung dort stoppen lassen. Mit **Step Over** kann eine Anweisung ausgeführt werden, mit **Step Into** kann in die Anweisung hineingesprungen werden (etwa in einen Methodenaufruf), und mit **Step Out** führe ich die aktuelle Methode aus und kehre zum Aufrufer zurück. Mit dem **Variablenfenster** und dem **Call Stack** kann ich gezielt beobachten, was mein Programm wirklich tut und sogar Variablenwerte verändern und Ausdrücke live evaluieren. Mit bedingten Breakpoints kann man dynamisch auf Programmzustände reagieren.

Man versucht Fehler zu beobachten und zu reproduzieren, um sich eine Hypothese zu bilden. Mit Break-

points hält man den Debugger bei der Programmausführung an den “interessanten” Stellen an und kann den Code dann schrittweise ausführen, um ihn kontrolliert analysieren und verbessern zu können.

Videos

Vorlesung [YT], [HSBI]

3.5.1. Software hat (immer) Fehler

- **Testen** = Aufdecken/Provozieren von Fehlern
- **Debuggen** = Finden der Stelle im Code

3.5.2. Warum Debuggen?

Typische Fehlerarten:

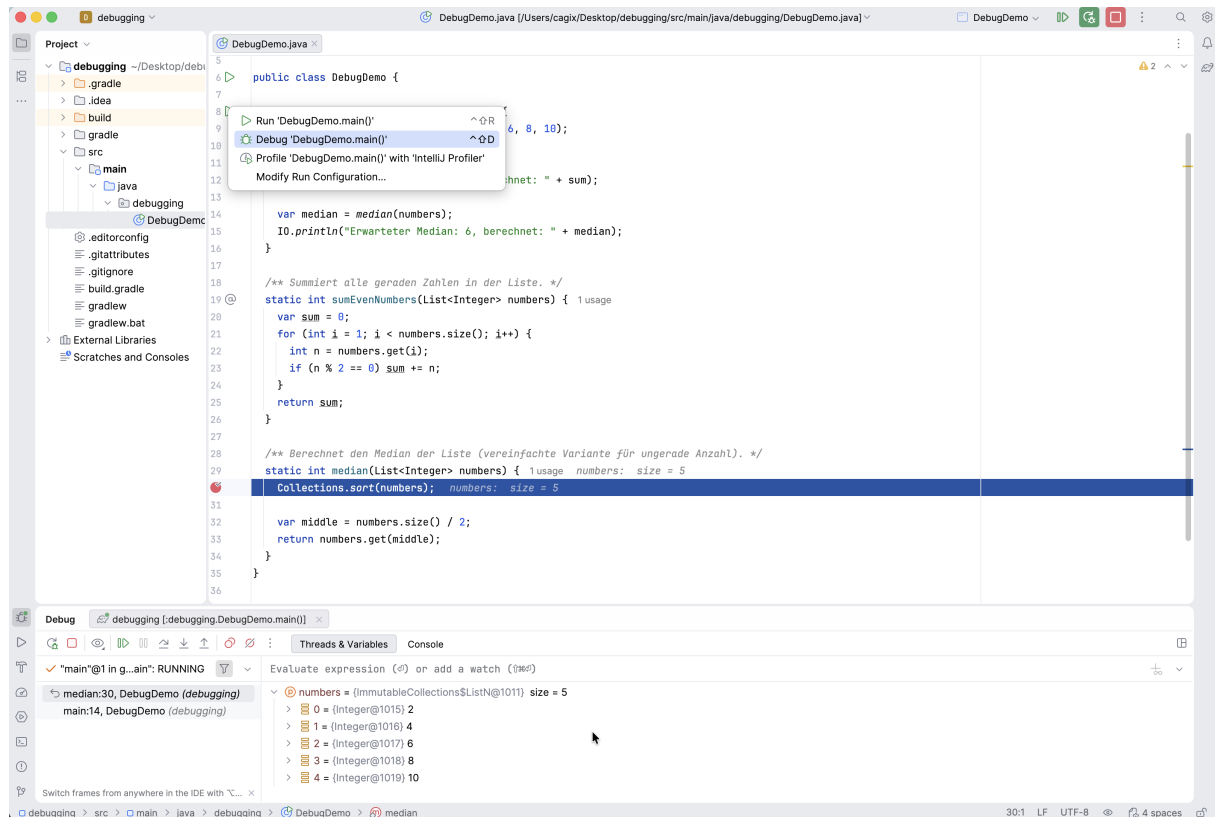
- **Exceptions:** Programm bricht ab
 - z.B. `NullPointerException`, `IndexOutOfBoundsException`, `UnsupportedOperationException`
- **Logikfehler:** Programm läuft durch, Ergebnis ist aber falsch

Warum Debugger statt `IO.println`:

- **Alle Variablenwerte** in einem Zustand sichtbar
- Programm **Schritt für Schritt** ausführbar
- **Call Stack** (Wer hat wen aufgerufen?)
- Komplexe Bedingungen (z.B. **bedingte Breakpoints**)

... und man muss auch nicht daran denken, dass man die ganzen `IO.println`, die überall im Code verstreut sind, auch alle wieder entfernen muss :)

3.5.3. Die wichtigsten Debugger-Werkzeuge



- **Breakpoint**
 - Rotes Markierungssymbol an einer Codezeile
 - Programm hält an dieser Stelle an
 - Bedingte Breakpoints prüfen einen Ausdruck und halten nur an, wenn die Bedingung erfüllt ist
- **Run / Debug**
 - Programm im **Debug-Modus** starten (statt normalem Run)
- **Step Over**
 - Nächste Zeile ausführen, Methodenaufrufe werden “übersprungen” (ausgeführt)
- **Step Into**
 - In den aufgerufenen Methoden-Body hineinspringen
- **Step Out**
 - Aktuellen Stack-Frame (laufender Aufruf einer Methode/Funktion) bis zum Ende ausführen und zurück zum Aufrufer
- **Resume**
 - Setze das Programm fort (bis zum nächsten Breakpoint oder bis zum Programmende - was von beidem als erstes auftritt)

- **Variables / Watches**

- Aktuelle Werte von Variablen und Ausdrücken ansehen und (bei Bedarf) ändern
- Watches: können Variablen und komplexere Ausdrücke halten, werden ganz oben in der Variablenliste angezeigt

- **Evaluate Expressions**

- Ausdrücke im laufenden Programm auswerten, auch mit Nebeneffekten
- Ausdrücke können auch als “Watch” angelegt werden und bleiben dauerhaft im Blick

- **Call Stack**

- Liste der aktuell verschachtelten Methodenaufrufe (Stack Frames)

Die Begriffe können je nach IDE (IntelliJ, Eclipse, VS Code, ...) leicht variieren.

3.5.4. Standard-Vorgehen beim Debuggen

1. **Fehler beobachten**

- Was genau passiert? Exception? Falsches Ergebnis?

2. **Fehler reproduzierbar machen**

- Konkrete Eingaben / Szenario festlegen

3. **Hypothese bilden**

- Was könnte im Code falsch laufen?

4. **Breakpoint setzen**

- In einer relevanten Methode / vor einer verdächtigen Stelle

5. **Im Debug-Modus starten**

- Programm läuft bis zum Breakpoint

6. **Schrittweise ausführen & Variablen beobachten**

- [Step Over](#) / [Step Into](#), Variablenfenster, ggf. Ausdrucksauswertung

7. **Fehler lokalisieren und fixen**

- Code korrigieren, dann wieder bei Schritt 1 beginnen

3.5.5. Demo 1: Exception beim Median (Crash)

3.5.5.1. Beispielmethode:

Beispiel: `debugging.DebugDemo#median`

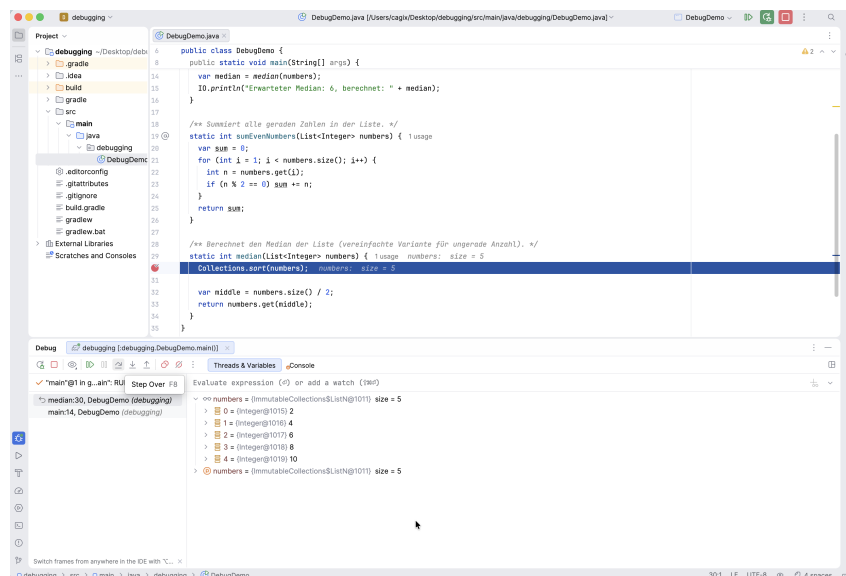
```
1 static int median(List<Integer> numbers) {
2     Collections.sort(numbers); // BUG: UnsupportedOperationException
3     var middle = numbers.size() / 2;
4     return numbers.get(middle);
5 }
```

3.5.5.2. Situation:

- `numbers` kommt aus `List.of(2, 4, 6, 8, 10)`;
- `List.of` liefert eine **unveränderliche** Liste
- `Collections.sort(numbers)` versucht, die Liste zu ändern
- -> `java.lang.UnsupportedOperationException`

3.5.5.3. Schritte in der Demo:

1. Programm normal ausführen -> Absturz, Stacktrace ansehen
2. Im Stacktrace die Zeile in `median` finden
3. **Breakpoint** in `median` auf die Zeile mit `Collections.sort(...)` setzen
4. Programm im **Debug-Modus** starten
5. Im Debugger:
 - Prüfen: Was ist der Typ von `numbers`?
 - **Step Over** auf `Collections.sort(numbers)`; -> Exception tritt auf
 - Call Stack ansehen (Wer hat `median` aufgerufen?)



3.5.5.4. Anschließender Fix (live in der Demo):

```

1 static int median(List<Integer> numbers) {
2     // Kopie in eine veränderliche Liste
3     List<Integer> copy = new ArrayList<>(numbers);
4     Collections.sort(copy);
5
6     var middle = copy.size() / 2;
7     return copy.get(middle);
8 }

```

Dann Programm erneut im Debug-Modus starten und prüfen, ob der Crash behoben ist.

3.5.6. Demo 2: Logikfehler bei der Summe (falsches Ergebnis)

3.5.6.1. Beispielmethode:

Beispiel: `debugging.DebugDemo#sumEvenNumbers`

```
1 static int sumEvenNumbers(List<Integer> numbers) {
2     var sum = 0;
3     // BUG: Schleife startet bei 1, Element mit Index 0 (Wert 2) wird nie
      besucht
4     for (int i = 1; i < numbers.size(); i++) {
5         var n = numbers.get(i); if (n % 2 == 0) sum += n;
6     }
7     return sum;
8 }
```

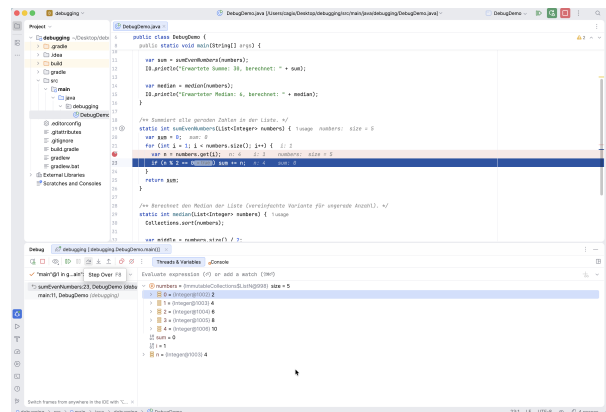
3.5.6.2. Ausgabe in main:

```
1 var sum = sumEvenNumbers(numbers);
2 IO.println("Erwartete Summe: 30, berechnet: " + sum);
```

Ergebnis: Erwartete Summe: 30, berechnet: 28

3.5.6.3. Schritte in der Demo:

1. Breakpoint in der Zeile mit `var n = numbers.get(i);` (in der Schleife) setzen
2. Programm im Debug-Modus starten
3. Bei jedem Halt:
 - `i`, `n` und `sum` im Variablenfenster beobachten
 - Mit `Step Over` zur nächsten Zeile
4. Fragen an die Studierenden:
 - Welche Werte nimmt `i` an?
 - Welche Elemente der Liste werden tatsächlich besucht?
 - Warum wird die 2 (Index 0) nicht berücksichtigt?



3.5.6.4. Anschließender Fix (live in der Demo):

```
1 for (int i = 0; i < numbers.size(); i++) { // i startet jetzt bei 0
2     var n = numbers.get(i);
3     if (n % 2 == 0) sum += n;
4 }
```

Danach erneut im Debug-Modus ausführen und die Werte von `i`, `n` und `sum` beobachten.

3.5.7. Tipps zum selbstständigen Üben

- Probieren Sie in Ihrer IDE:
 - Einen **Bedingten Breakpoint**: z.B. in der Schleife von `sumEvenNumbers` nur halten, wenn `i == 0` oder `n > 5`
 - Das **Ändern von Variablenwerten** im Debugger
 - Das **Auswerten von Ausdrücken** (Evaluate Expression)
- Ersetzen Sie in `main` die Beispiel-Liste durch andere Daten:
 - z.B. leere Liste, eine Liste mit nur einem Element, ungerade/gerade Anzahl
- Ziel: Sie sollten das Gefühl haben, dass Sie den Debugger **aktiv steuern** können, statt nur “zuzuschauen”.

3.5.8. Wrap-Up

- Debugger ist zentrales Werkzeug:
 - Breakpoints, Step Over/Into/Out, Variablen, Call Stack
- Vorgehen:
 - Fehler beobachten -> reproduzieren -> Hypothese -> Breakpoint -> schrittweise ausführen -> fixen

Zum Nachlesen

Zum Weiterlesen für Interessierte kann ich das Online-Buch meines Kollegen Andreas Zeller empfehlen: [The Debugging Book](#). Das Buch geht aber deutlich über den hier besprochenen Inhalt hinaus ...

Lernziele

- k3: Ich kann einen Fehler gezielt **reproduzieren**
- k3: Ich kann eine **Exception** mit Hilfe des **Stacktraces** und des Debuggers analysieren
- k3: Ich kann einen **Logikfehler** (falsches Ergebnis) mit Breakpoints und schrittweiser Ausführung finden
- k2: Ich kann die wichtigsten Debugger-Funktionen in meiner IDE benennen

Challenges

Nehmen Sie Ihren eigenen Code aus Übungen/Projekten und debuggen Sie bewusst ein paar Methoden, auch wenn (scheinbar) alles funktioniert.

3.6. Logging

TL;DR

Im Paket `java.util.logging` findet sich eine einfache Logging-API.

Über die Methode `getLogger()` der Klasse `Logger` (*Factory-Method-Pattern*) kann ein (neuer) Logger erzeugt werden, dabei wird über den String-Parameter eine Logger-Hierarchie aufgebaut analog zu den Java-Package-Strukturen. Der oberste Logger (der “Basis-Logger” bzw. “Root-Logger”) hat den leeren Namen.

Jeder Logger kann mit einem Log-Level (Klasse `Level`) eingestellt werden; Log-Meldungen unterhalb des eingestellten Levels werden verworfen.

Vom Logger nicht verworfene Log-Meldungen werden an den bzw. die Handler des Loggers und (per Default) an den Eltern-Logger weiter gereicht. Die Handler haben ebenfalls ein einstellbares Log-Level und werfen alle Nachrichten unterhalb der eingestellten Schwelle. Zur tatsächlichen Ausgabe gibt man einem Handler noch einen Formatter mit. Defaultmäßig hat nur der Root-Logger einen Handler.

Der Root-Logger (leerer String als Name) hat als Default-Level (wie auch sein Console-Handler) “**Info**” eingestellt.

Nachrichten, die durch Weiterleitung nach oben empfangen wurden, werden nicht am Log-Level des empfangenden Loggers gemessen, sondern akzeptiert und an die Handler des Loggers und (sofern nicht deaktiviert) an den Eltern-Logger weitergereicht.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo:

- Logging (Überblick) [\[YT\]](#), [\[HSBI\]](#)

- Log-Level [YT], [HSBI]
- Handler und Formatter [YT], [HSBI]
- Weiterleitung an den Eltern-Logger [YT], [HSBI]

3.6.1. Wie prüfen Sie die Werte von Variablen/Objekten?

1. Debugging

- Beeinflusst Code nicht
- Kann schnell komplex und umständlich werden
- Sitzung transient - nicht reproduzierbar/speicherbar

2. “Poor-man’s-debugging” (Ausgaben mit `IO.println`)

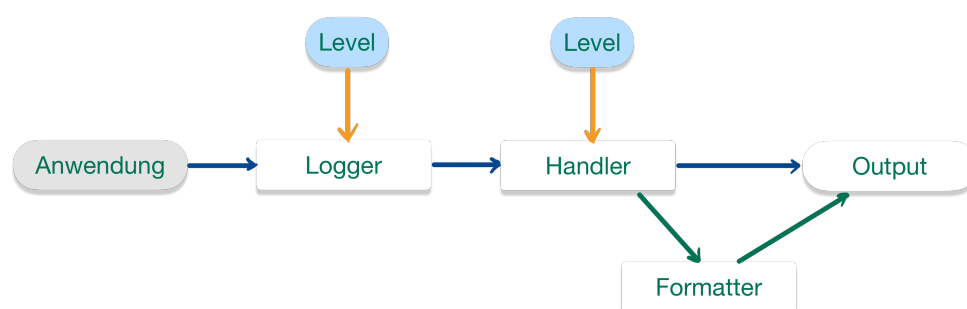
- Müssen irgendwann vor Release/Produktivbetrieb entfernt werden
- Ausgabe nur auf einem Kanal (Konsole)
- Keine Filterung nach Problemgrad - keine Unterscheidung zwischen Warnungen, einfachen Informationen, ...

3. Logging

- Verschiedene (Java-) Frameworks:
`java.util.logging` (JDK), `log4j` (Apache), `SLF4J`, `Logback`, ...

3.6.2. Java Logging API - Überblick

Paket `java.util.logging`



Eine Applikation kann verschiedene Logger instanziiieren. Die Logger bauen per Namenskonvention hierarchisch aufeinander auf. Jeder Logger kann selbst mehrere Handler haben, die eine Log-Nachricht letztlich auf eine bestimmte Art und Weise an die Außenwelt weitergeben.

Log-Meldungen werden einem Level zugeordnet. Jeder Logger und Handler hat ein Mindest-Level eingestellt, d.h. Nachrichten mit einem kleineren Level werden verworfen.

Zusätzlich gibt es noch Filter, mit denen man Nachrichten (zusätzlich zum Log-Level) nach weiteren Kriterien filtern kann.

[Konsole: logging.LoggingDemo](#)

3.6.3. Erzeugen neuer Logger

```
1 import java.util.logging.Logger;
2 Logger l = Logger.getLogger(MyClass.class.getName());
```

- **Factory-Methode** der Klasse `java.util.logging.Logger`

```
1 public static Logger getLogger(String name);
```

=> Methode liefert bereits **vorhandenen Logger** mit diesem Namen (sonst neuen Logger)

- **Best Practice:**

Nutzung des vollqualifizierten Klassennamen: `MyClass.class.getName()`

- Leicht zu implementieren
- Leicht zu erklären
- Spiegelt modulares Design
- Ausgaben enthalten automatisch Hinweis auf Herkunft (Lokalität) der Meldung
- **Alternativen:** Funktionale Namen wie "XML", "DB", "Security"

3.6.4. Ausgabe von Logmeldungen

```
1 public void log(Level level, String msg);
```

- Diverse Convenience-Methoden (Auswahl):

```
1 public void warning(String msg)
2 public void info(String msg)
3 public void entering(String srcClass, String srcMethod)
4 public void exiting(String srcClass, String srcMethod)
```

- Beispiel

```
1 import java.util.logging.Logger;
2 Logger l = Logger.getLogger(MyClass.class.getName());
3 l.info("Hello World :-)");
```

3.6.5. Wichtigkeit von Logmeldungen: Stufen

- `java.util.logging.Level` definiert 7 Stufen:
 - `SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`
(von höchster zu niedrigster Prio)
 - Zusätzlich `ALL` und `OFF`
- Nutzung der Log-Level:
 - Logger hat Log-Level: Meldungen mit kleinerem Level werden verworfen
 - Prüfung mit `public boolean isLoggable(Level)`
 - Setzen mit `public void setLevel(Level)`

Konsole: `logging.LoggingLevel`

=> Warum wird im Beispiel nach `log.setLevel(Level.ALL)` ; trotzdem nur ab `INFO` geloggt? Wer erzeugt eigentlich die Ausgaben?!

3.6.6. Jemand muss die Arbeit machen ...

- Pro Logger **mehrere** Handler möglich
 - Logger übergibt nicht verworfene Nachrichten an Handler
 - Handler haben selbst ein Log-Level (analog zum Logger)
 - Handler verarbeiten die Nachrichten, wenn Level ausreichend
- Standard-Handler: `StreamHandler`, `ConsoleHandler`, `FileHandler`
- Handler nutzen zur Formatierung der Ausgabe einen `Formatter`
- Standard-Formatter: `SimpleFormatter` und `XMLFormatter`

Konsole: `logging.LoggingHandler`

=> Warum wird im Beispiel nach dem Auskommentieren von `log.setUseParentHandlers(false)` ; immer noch eine zusätzliche Ausgabe angezeigt (ab `INFO` aufwärts)?!

3.6.7. Ich ... bin ... Dein ... Vater ...

- Logger bilden **Hierarchie** über Namen
 - Trenner für Namenshierarchie: “.” (analog zu Packages) => mit jedem “.” wird eine weitere Ebene der Hierarchie aufgemacht ...
 - Jeder Logger kennt seinen Eltern-Logger: `Logger#getParent()`
 - Basis-Logger: leerer Name (“”)

* Voreingestelltes Level des Basis-Loggers: `Level.INFO` (!)

- Weiterleiten von Nachrichten
 - Nicht verworfene Log-Aufrufe werden an Eltern-Logger weitergeleitet (Default)
 - * Abschalten mit `Logger#setUseParentHandlers(false)`;
 - Diese leiten an ihre Handler sowie an ihren Eltern-Logger weiter (unabhängig von Log-Level!)

Konsole: `logging.LoggingParent`; Tafel: Skizze Logger-Baum

3.6.8. Ausgabe von Logmeldungen - revisited

Bei genauerem Hinschauen auf die API von `java.util.logging` erkennt man, dass es die `log`-Funktion in zwei Varianten gibt: Einmal mit einem `String` (der Message) als Parameter, und einmal mit einem `Supplier` (funktionales Interface aus dem JDK):

```
1 public void log(Level level, String msg);
2 public void log(Level level, Supplier<String> msgSupplier);
```

Das gilt auch für die meisten Convenience-Log-Funktionen:

```
1 public void warning(String msg)
2 public void warning(Supplier<String> msgSupplier)
3
4 public void info(String msg)
5 public void info(Supplier<String> msgSupplier)
```

Damit kann man den Aufruf auch mit einem Lambda-Ausdruck gestalten:

```
1 import java.util.logging.Logger;
2 Logger l = Logger.getLogger(MyClass.class.getName());
3
4 l.info("Hello World");           // String
5 l.info(() -> "Hello World");    // Supplier
```

Das Logging mit `java.util.logging` ist an sich bereits sehr effizient: Wenn bei einem Aufruf einer Log-Funktion die eingestellten Level beim Logger, den Handlern und den Elternloggern nicht erreicht werden, wird nichts ausgegeben. Um noch effizienter zu werden, kann man auch das Bauen der Message selbst davon abhängig machen, ob man sie wirklich braucht: Während bei der ersten Aufruf-Variante mit dem `String`-Parameter die Message bereits vor dem Aufruf der Log-Methode berechnet werden muss, wird sie in der zweiten Variante erst dann wirklich berechnet, wenn sie gebraucht wird. Wenn die Log-Methode tatsächlich loggen kann (die verschiedenen Level werden erreicht), dann und nur dann wertet sie den übergebenen Lambda-Ausdruck aus und erzeugt dabei die auszugebende Message. (Anmerkung: Im obigen Beispiel ist das egal, da der `String` ohnehin fest ist und stets bekannt/berechnet ist. Aber stellen Sie sich statt `"Hello World"` einen Funktionsaufruf vor, der mehr oder weniger aufwändig einen `String` produziert ...)

3.6.9. Best Practices

- Pro Klasse ein **private static final** `Logger LOG = Logger.getLogger(MyClass.class.getName());`
- Keine Log-Ausgaben in Bibliotheken mit `System.out/System.err`, sondern immer ein Logging-Framework verwenden
- Log-Meldungen:
 - Klar, kurz, technisch brauchbar (“Was ist passiert? Wo? Welche Daten/IDs?”)
 - Keine sensiblen Daten (Passwörter, Tokens, personenbezogene Daten)
- Log-Level-Richtlinien:
 - **SEVERE**: Fehler, nach denen fachlich nicht sinnvoll weitergearbeitet werden kann
 - **WARNING**: Unerwartete Situationen, aber das System läuft weiter
 - **INFO**: Wichtige Statusmeldungen
 - **CONFIG**: Konfigurationsdetails
 - **FINE, FINER, FINEST**: eher für detaillierte Diagnose/Entwicklung

Wir haben in den Beispielen den Logger immer mit `setLevel(...)` und `setUseParentHandlers(false)` etc. im Code konfiguriert. Es gibt aber - analog zu größeren Frameworks wie SLF4J - auch bei `java.util.logging` die Möglichkeit, die Konfiguration von außen über eine Properties-Datei vorzunehmen (`logging.properties`).

Tip

In vielen professionellen Projekten werden heute (für flexible Konfiguration und bessere Integrationen) Logging-Frameworks wie SLF4J mit Logback genutzt. Die hier vorgestellte Logik (Logger, Level, Handler/Appender, Formatter/Layouts, Hierarchie) überträgt sich jedoch nahezu 1:1 darauf.

3.6.10. Wrap-Up

- Java Logging API im Paket `java.util.logging`
- Neuer Logger über **Factory-Methode** der Klasse `Logger`
 - Einstellbares Log-Level (Klasse `Level`)
 - Handler kümmern sich um die Ausgabe, nutzen dazu Formatter
 - Mehrere Handler je Logger registrierbar
 - Log-Level auch für Handler einstellbar (!)
 - Logger (und Handler) “interessieren” sich nur für Meldungen ab bestimmter Wichtigkeit
 - Logger reichen nicht verworfene Meldungen defaultmäßig an Eltern-Logger weiter (rekursiv)

Zum Nachlesen

Lesen Sie in der Dokumentation von Oracle zu den JDK Core Libraries nach: [Java Logging Overview](#).

Lernziele

- k3: Ich kann die Java Logging API im Paket `java.util.logging` aktiv einsetzen
- k3: Ich kann eigene Handler und Formatter schreiben

Challenges**Logger-Konfiguration**

Betrachten Sie den folgenden Java-Code:

```
1  import java.util.logging.*;
2
3  public class Logging {
4      public static void main(String... args) {
5          Logger l = Logger.getLogger("Logging");
6          l.setLevel(Level.FINE);
7
8          ConsoleHandler myHandler = new ConsoleHandler();
9          myHandler.setFormatter(new SimpleFormatter() {
10              public String format(LogRecord record) {
11                  return "WUPPIE\n";
12              }
13          });
14          l.addHandler(myHandler);
15
16          l.info("A");
17          l.fine("B");
18          l.finer("C");
19          l.finest("D");
20          l.severe("E");
21      }
22  }
```

Welche Ausgaben entstehen durch den obigen Code? Erklären Sie, welche der Logger-Aufrufe zu einer Ausgabe führen und wieso und wie diese Ausgaben zustande kommen bzw. es keine Ausgabe bei einem Logger-Aufruf gibt. Gehen Sie dabei auf jeden der fünf Aufrufe ein.

Analyse eines Live-Beispiels aus dem Dungeon

Analysieren Sie die Konfiguration des Loggings im Dungeon-Projekt: [Dungeon-CampusMinden/Dungeon: core.utils.logging.DungeonLoggerConfig](#).

3.7. Einführung in Docker

TL;DR

Container sind im Gegensatz zu herkömmlichen VMs eine schlanke Virtualisierungslösung. Dabei laufen die Prozesse direkt im Kernel des Host-Betriebssystems, aber abgeschottet von den anderen Prozessen durch Linux-Techniken wie `cgroups` und `namespaces` (unter Windows kommt dafür der WSL2 zum Einsatz, unter macOS wird eine kleine Virtualisierung genutzt).

Container sind sehr nützlich, wenn man an mehreren Stellen eine identische Arbeitsumgebung benötigt.

Man kann dabei entweder die Images (fertige Dateien) oder die Dockerfiles (Anweisungen zum Erzeugen eines Images) im Projekt verteilen. Tatsächlich ist es nicht unüblich, ein Dockerfile in das Projekt-Repo mit einzuchecken.

Im Gegensatz zu herkömmlichen VMs bieten Container keine starke zusätzliche Sicherheitsschicht, da die im Container laufende Software direkt im Kernel des Host-Betriebssystems ausgeführt wird. Container isolieren Prozesse voneinander, sind aber keine vollwertigen “Sandboxes” wie separate VMs.

Es gibt auf DockerHub fertige Images, die man sich ziehen und starten kann. Ein solches gestartetes Image nennt sich dann Container und enthält beispielsweise Dateien, die in den Container gemountet oder kopiert werden. Man kann auch eigene Images bauen, indem man eine entsprechende Konfiguration (Dockerfile) schreibt. Jeder Befehl bei der Erstellung eines Images erzeugt einen neuen Layer, die sich dadurch mehrere Images teilen können.

In der Konfiguration einer Gitlab-CI-Pipeline kann man mit `image` ein Docker-Image angeben, welches dann in der Pipeline genutzt wird.

VSCoode kann über das Remote-Plugin sich (u.a.) mit Containern verbinden und dann im Container arbeiten (editieren, compilieren, debuggen, testen, ...).

In dieser kurzen Einheit kann ich Ihnen nur einen ersten Einstieg in das Thema geben. Wir haben uns beispielsweise nicht Docker Compose oder Kubernetes angeschaut, und auch die Themen Netzwerk (zwischen Containern oder zwischen Containern und anderen Rechnern) und Volumes habe ich außen vor gelassen. Dennoch kommt man in der Praxis bereits mit den hier vermittelten Basiskenntnissen erstaunlich weit ...

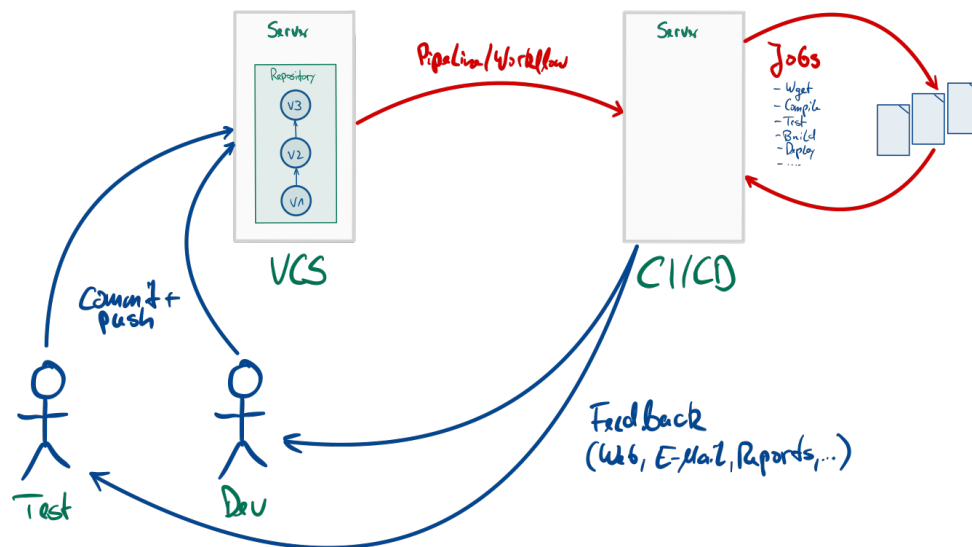
Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo:

- Container in der Konsole [\[YT\]](#), [\[HSBI\]](#)
- GitHub Actions und Docker [\[YT\]](#), [\[HSBI\]](#)
- GitLab CI/CD und Docker [\[YT\]](#), [\[HSBI\]](#)
- VSCoode und Docker [\[YT\]](#), [\[HSBI\]](#)

3.7.1. Motivation CI/CD: WFM (Works For Me)



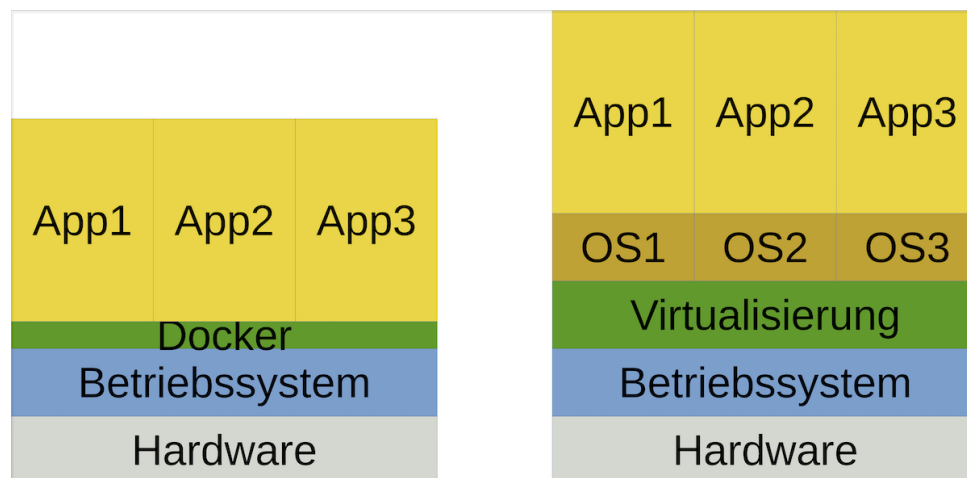
Auf dem CI-Server muss man eine Arbeitsumgebung konfigurieren und bereitstellen, für Java-basierte Projekte muss beispielsweise ein JDK existieren und man benötigt Tools wie Maven oder Gradle, um die Buildskripte auszuführen. Je nach Projekt braucht man dann noch weitere Tools und Bibliotheken. Diese Konfigurationen sind unabhängig vom CI-Server und werden üblicherweise nicht direkt installiert, sondern über eine Virtualisierung bereitgestellt.

Selbst wenn man keine CI-Pipelines einsetzt, hat man in Projekten mit mehreren beteiligten Personen häufig das Problem “WFM” (“works for me”). Jeder Entwickler hat sich auf ihrem Rechner eine Entwicklungsumgebung aufgesetzt und nutzt in der Regel seine bevorzugte IDE oder sogar unterschiedliche JDK-Versionen ... Dadurch kann es schnell passieren, dass Probleme oder Fehler auftreten, die sich nicht von allen Beteiligten immer nachvollziehen lassen. Hier wäre eine einheitliche Entwicklungsumgebung sinnvoll, die in einer “schlanken” Virtualisierung bereitgestellt wird.

Als Entwickler kann man zeitgleich in verschiedenen Projekten beteiligt sein, die unterschiedliche Anforderungen an die Entwicklungstools mit sich bringen. Es könnte beispielsweise passieren, dass man zeitgleich drei bestimmte Python-Versionen benötigt. In den meisten Fällen schafft man es (mit ein wenig Aufwand), diese Tools nebeneinander zu installieren. Oft ist das in der Praxis aber schwierig und fehleranfällig.

In diesen Fällen kann eine Virtualisierung helfen.

3.7.2. Virtualisierung: Container vs. VM



Wenn man über Virtualisierung auf dem Desktop spricht, kann man grob zwei Varianten unterscheiden. In beiden Fällen ist die Basis die Hardware (Laptop, Desktop-Rechner) und das darauf laufende (Host-) Betriebssystem (Linux, FreeBSD, macOS, Windows, ...). Darauf läuft dann wiederum die Virtualisierung.

Im rechten Bild wird eine herkömmliche Virtualisierung mit virtuellen Maschinen (VM) dargestellt. Dabei wird in der VM ein komplettes Betriebssystem (das "Gast-Betriebssystem") installiert und darin läuft dann die gewünschte Anwendung. Die Virtualisierung (VirtualBox, VMware, ...) läuft dabei als Anwendung auf dem Host-Betriebssystem und stellt dem Gast-Betriebssystem in der VM einen Rechner mit CPU, RAM, ... zur Verfügung und übersetzt die Systemaufrufe in der VM in die entsprechenden Aufrufe im Host-Betriebssystem. Dies benötigt in der Regel entsprechende Ressourcen: Durch das komplette Betriebssystem in der VM ist eine VM (die als Datei im Filesystem des Host-Betriebssystems liegt) oft mehrere 10GB groß. Für die Übersetzung werden zusätzlich Hardwareressourcen benötigt, d.h. hier gehen CPU-Zyklen und RAM "verloren" ... Das Starten einer VM dauert entsprechend lange, da hier ein komplettes Betriebssystem hochgefahren werden muss. Dafür sind die Prozesse in einer VM relativ stark vom Host-Betriebssystem abgekapselt, so dass man hier von einer "Sandbox" sprechen kann: Viren o.ä. können nicht so leicht aus einer VM "ausbrechen" und auf das Host-Betriebssystem zugreifen (quasi nur über Lücken im Gast-Betriebssystem kombiniert mit Lücken in der Virtualisierungssoftware).

Im linken Bild ist eine schlanke Virtualisierung auf Containerbasis dargestellt. Die Anwendungen laufen direkt als Prozesse im Host-Betriebssystem, ein Gast-Betriebssystem ist nicht notwendig. Durch den geschickten Einsatz von `namespaces` und `cgroups` und anderen in Linux und FreeBSD verfügbaren Techniken werden die Prozesse abgeschottet, d.h. der im Container laufende Prozess "sieht" die anderen Prozesse des Hosts nicht. Die Erstellung und Steuerung der Container übernimmt hier beispielsweise Docker. Die Container sind dabei auch wieder Dateien im Host-Filesystem. Dadurch benötigen Container wesentlich weniger Platz als herkömmliche VMs, der Start einer Anwendung geht deutlich schneller und die Hardwareressourcen (CPU, RAM, ...) werden effizient genutzt. Nachteilig ist, dass hier in der Regel ein Linux-Host benötigt wird (für Windows wird mittlerweile der Linux-Layer (WSL) genutzt; für macOS wurde bisher eine Linux-VM im Hintergrund hochgefahren, mittlerweile wird aber eine eigene schlanke Virtualisierung eingesetzt). Außerdem steht im Container üblicherweise kein graphisches Benutzerinterface zur Verfügung. Da die Prozesse direkt im Host-Betriebssystem laufen, stellen Container keine wirkliche Sicherheitsschicht ("Sandboxes") dar!

In allen Fällen muss die Hardwarearchitektur beachtet werden: Auf einer Intel-Maschine können normalerweise keine VMs/Container basierend auf ARM-Architektur ausgeführt werden und umgekehrt.

Tip

- Container bieten **Isolation**, aber keine starke **Sicherheitsgrenze** wie eine vollwertige VM.
- Praktisch heißt das:
 - Ja, ein Container-Prozess kann nicht “einfach so” andere Host-Prozesse sehen (namespaces etc.).
 - Aber: Wenn der Container-Prozess ausbricht (Kernel-Lücke, Docker-Misskonfiguration, `--privileged` etc.), ist der Host direkt betroffen.

Durch Container entsteht im Vergleich zu herkömmlichen VMs keine starke zusätzliche Sicherheits-schicht: Die Prozesse laufen im Kernel des Host-Systems. Container isolieren Prozesse zwar voneinander (namespaces, cgroups), sind aber keine harte Sandbox wie eine eigenständige VM.

3.7.3. Getting started

- DockerHub: fertige Images => hub.docker.com/search
- Image downloaden: `docker pull <IMAGE>`
- Image starten: `docker run <IMAGE>`

3.7.3.1. Begriffe

- **Docker-File:** Beschreibungsdatei, wie Docker ein Image erzeugen soll.
- **Image:** Enthält die read-only Schichten, die lt. dem Docker-File in das Image gepackt werden sollen. Liegen als Dateien auf dem Host. Kann gestartet werden und erzeugt damit einen Container.
- **Container:** Ein laufendes Images (genauer: eine laufende Instanz eines Images + Schreib-Layer + Metadaten). Kann dann auch zusätzliche Daten enthalten.

Tip

Ein Container basiert auf einem Image. Das Image liegt als Dateien vor, der Container selbst ist eine laufende Instanz + zusätzlicher Schreib-Layer.

3.7.3.2. Beispiele

```
1 docker pull debian:stable-slim
2 docker run --rm -it debian:stable-slim /bin/sh
```

`debian` ist ein fertiges Images, welches über DockerHub bereit gestellt wird. Mit dem Postfix `stable-slim` wird eine bestimmte Version angesprochen.

Mit `docker run debian:stable-slim` startet man das Image, es wird ein Container erzeugt. Dieser enthält den aktuellen Datenstand, d.h. wenn man im Image eine Datei anlegt, wäre diese dann im Container enthalten.

Mit der Option `--rm` wird der Container nach Beendigung automatisch wieder gelöscht. Da jeder Aufruf von `docker run <IMAGE>` einen neuen Container erzeugt, würden sich sonst recht schnell viele Container auf dem Dateisystem des Hosts ansammeln, die man dann manuell aufräumen müsste. Man kann aber einen beendeten Container auch erneut laufen lassen ... (vgl. Dokumentation von `docker`). Mit der Option `--rm` sind aber auch im Container angelegte Daten wieder weg! Mit der Option `-it` wird der Container interaktiv gestartet und man landet in einer Shell.

Bei der Definition eines Images kann ein "Entry Point" definiert werden, d.h. ein Programm, welches automatisch beim Start des Container ausgeführt wird. Häufig erlauben Images aber auch, beim Start ein bestimmtes auszuführendes Programm anzugeben. Im obigen Beispiel ist das `/bin/sh`, also eine Shell ...

```
1 docker pull eclipse-temurin:latest
2 docker run --rm -v "$PWD":/data -w /data eclipse-temurin:latest javac Hello
  .java
3 docker run --rm -v "$PWD":/data -w /data eclipse-temurin:latest java Hello
```

Auch für Java gibt es vordefinierte Images mit einem JDK. Das Tag "latest" zeigt dabei auf die letzte stabile Version des `eclipse-temurin`-Images. Üblicherweise wird "latest" von den Entwicklern immer wieder weiter geschoben, d.h. auch bei anderen Images gibt es ein "latest"-Tag. Gleichzeitig ist es die Default-Einstellung für die Docker-Befehle, d.h. es kann auch weggelassen werden: `docker run eclipse-temurin:latest` und `docker run eclipse-temurin` sind gleichwertig. Alternativ (und besser!) kann man hier auch hier wieder eine konkrete Version angeben.

Tip

In CI/CD-Pipelines sollte man `latest` möglichst vermeiden, weil sich das Image unbemerkt ändern kann. Besser: eine konkrete Version taggen (z.B. `eclipse-temurin:25.0.3_9-jdk-ubi10-minimal`).

Important

Früher gab es offizielle `openjdk`-Images auf Docker Hub. Diese sind inzwischen *deprecated* und werden nicht mehr gepflegt. Stattdessen nutzen wir hier die offiziellen Eclipse-Temurin-Images (`eclipse-temurin:<version>`), die aktuelle OpenJDK-Builds bereitstellen.

Über die Option `-v` wird ein Ordner auf dem Host (hier durch `"$PWD"` dynamisch ermittelt) in den Container eingebunden ("gemountet"), hier auf den Ordner `/data`. Dort sind dann die Dateien sichtbar, die im Ordner `"$PWD"` enthalten sind. Über die Option `-w` kann ein Arbeitsverzeichnis definiert werden.

Mit `javac Hello.java` wird `javac` im Container aufgerufen auf der Datei `/data/Hello.java` im Container, d.h. die Datei `Hello.java`, die im aktuellen Ordner des Hosts liegt (und in den Container gemountet wurde). Das Ergebnis (`Hello.class`) wird ebenfalls in den Ordner `/data/` im Container geschrieben und erscheint dann im Arbeitsverzeichnis auf dem Host ... Analog kann dann mit `java Hello` die Klasse ausgeführt werden.

[Demo: Container in der Konsole](#)

3.7.4. Images selbst definieren

```

1 FROM debian:stable-slim
2
3 ARG USERNAME=pandoc
4 ARG USER_UID=1000
5 ARG USER_GID=1000
6
7 RUN apt-get update && apt-get install -y --no-install-recommends \
8     apt-utils bash wget make graphviz biber \
9     texlive-base texlive-plain-generic texlive-latex-base \
10    #
11    && groupadd --gid $USER_GID $USERNAME \
12    && useradd -s /bin/bash --uid $USER_UID --gid $USER_GID -m $USERNAME \
13    #
14    && apt-get autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*
15
16 WORKDIR /pandoc
17 USER $USERNAME

```

`docker build -t <NAME> -f <DOCKERFILE> .`

`FROM` gibt die Basis an, d.h. hier ein Image von Debian in der Variante `stable-slim`, d.h. das ist der Basis-Layer für das zu bauende Docker-Image.

Über `ARG` werden hier Variablen gesetzt.

`RUN` ist der Befehl, der im Image (hier Debian) ausgeführt wird und einen neuen Layer hinzufügt. In diesen Layer werden alle Dateien eingefügt, die bei der Ausführung des Befehls erzeugt oder angelegt werden. Hier im Beispiel wird das Debian-Tool `apt-get` gestartet und weitere Debian-Pakete installiert.

Da jeder `RUN`-Befehl einen neuen Layer anlegt, werden die restlichen Konfigurationen ebenfalls in diesem Lauf durchgeführt. Insbesondere wird ein nicht-Root-User angelegt, der von der UID und GID dem Default-User in Linux entspricht. Die gemounteten Dateien haben die selben Rechte wie auf dem Host, und durch die Übereinstimmung von UID/GID sind die Dateien problemlos zugreifbar und man muss nicht mit dem Root-User arbeiten (dies wird aus offensichtlichen Gründen als Anti-Pattern angesehen). Bevor der `RUN`-Lauf abgeschlossen wird, werden alle temporären und später nicht benötigten Dateien von `apt-get` entfernt, damit diese nicht Bestandteil des Layers werden.

Mit `WORKDIR` und `USER` wird das Arbeitsverzeichnis gesetzt und auf den angegebenen User umgeschaltet. Damit muss der User nicht mehr beim Aufruf von außen gesetzt werden.

Über `docker build -t <NAME> -f <DOCKERFILE> .` wird aus dem angegebenen Dockerfile und dem Inhalt des aktuellen Ordners (".") ein neues Image erzeugt und mit dem angegebenen Namen benannt.

Hinweis zum Umgang mit Containern und Updates: Bei der Erstellung eines Images sind bestimmte Softwareversionen Teil des Images geworden. Man kann prinzipiell in einem Container die Software aktualisieren, aber dies geht in dem Moment wieder verloren, wo der Container beendet und gelöscht wird. Außerdem widerspricht dies dem Gedanken, dass mehrere Personen mit dem selben Image/Container arbeiten und damit auch die selben Versionsstände haben. In der Praxis löscht man deshalb das alte Image einfach und erstellt ein neues, welches dann die aktualisierte Software enthält.

[Beispiel: debian-latex.df](#)

3.7.5. CI-Pipeline (GitLab)

```
1 default:
2     image: eclipse-temurin:25-jdk
3
4 job1:
5     stage: build
6     script:
7         - java -version
8         - javac Hello.java
9         - java Hello
10        - ls -lags
```

In den Gitlab-CI-Pipelines (analog wie in den GitHub-Actions) kann man Docker-Container für die Ausführung der Pipeline nutzen.

Mit `image: eclipse-temurin:25-jdk` wird das Docker-Image `eclipse-temurin:25-jdk` vom DockerHub geladen und durch den Runner für die Stages als Container ausgeführt. Die Aktionen im `script`-Teil, wie beispielsweise `javac Hello.java` werden vom Runner an die Standard-Eingabe der Shell des Containers gesendet. Im Prinzip entspricht das dem Aufruf auf dem lokalen Rechner: `docker run eclipse-temurin:25-jdk javac Hello.java`.

[Demo: GitLab CI/CD und Docker](#)

3.7.6. CI-Pipeline (GitHub)

```
1 name: demo
2 on:
3     push:
4         branches: [master]
5     workflow_dispatch:
6
7 jobs:
8     job1:
9         runs-on: ubuntu-latest
10        container: eclipse-temurin:25-jdk
11        steps:
12            - uses: actions/checkout@v7
13            - run: java -version
14            - run: javac Hello.java
15            - run: java Hello
16            - run: ls -lags
```

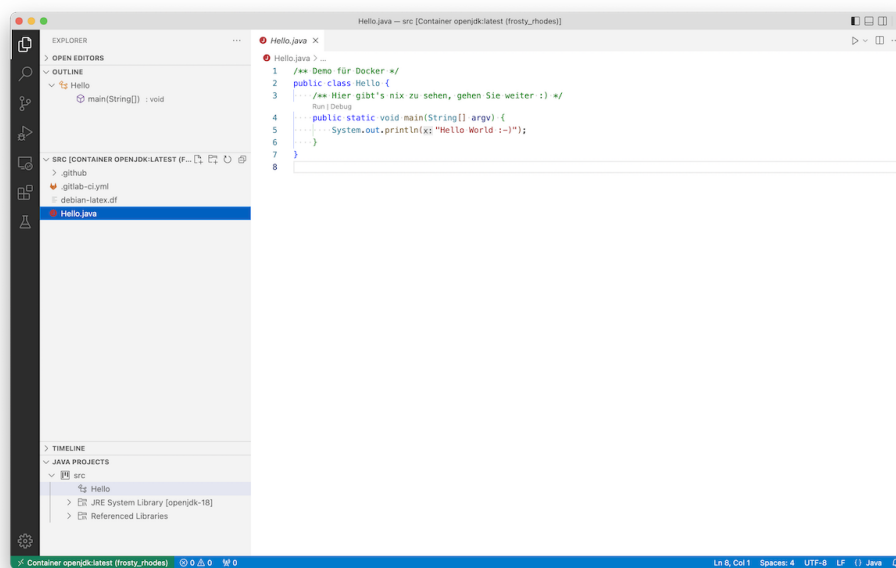
<https://stackoverflow.com/questions/71283311/run-github-workflow-on-docker-image-with-a-dockerfile>
<https://docs.github.com/en/actions/using-jobs/running-jobs-in-a-container>

In den GitHub-Actions kann man Docker-Container für die Ausführung der Pipeline nutzen.

Mit `container: eclipse-temurin:25-jdk` wird das Docker-Image `eclipse-temurin:25-jdk` vom DockerHub geladen und auf dem Ubuntu-Runner als Container ausgeführt. Die Aktionen im `steps`-Teil, wie beispielsweise `javac Hello.java` werden vom Runner an die Standard-Eingabe der Shell des Containers gesendet. Im Prinzip entspricht das dem Aufruf auf dem lokalen Rechner: `docker run eclipse-temurin:25-jdk javac Hello.java`.

Demo: GitHub Actions und Docker

3.7.7. Bonus/Ausblick: VSCode und DevContainers (früher “Remote - Containers”)



1. VSCode (Host): DevContainers Extension installieren
2. VSCode (Host): Projektordner öffnen und eine DevContainer-Konfiguration hinzufügen
3. VSCode (Host): “Reopen in Container” => neues Fenster (Container)
4. VSCode (Container): Java-Extensions (z.B. “Java Extension Pack”) installieren lassen
5. VSCode (Container): Dateien editieren, kompilieren, debuggen, testen ...

Mit Visual Studio Code (VSCode) können Sie direkt **in einem Container** entwickeln. Dazu installieren Sie sich VSCode lokal auf dem Host und dort die Extension “Dev Containers” (sie ist Teil des “Remote Development“-Pakets).

Für ein Projekt gehen Sie typischerweise so vor:

- Sie öffnen den Projektordner in VSCode.
- Über die Command Palette (z.B. `F1`) wählen Sie “**Dev Containers: Add Dev Container Configuration File ...**” und fügen eine Konfiguration hinzu (entweder aus einem Java-Template oder als eigene `devcontainer.json`).
- VSCode legt dann im Projekt den Ordner `.devcontainer/` mit einer `devcontainer.json` (und ggf. einem Dockerfile) an.

- Anschließend können Sie den Befehl **“Reopen in Container”** auswählen. VSCode baut und startet den Container automatisch und öffnet ein neues VS-Code-Fenster, das mit diesem Dev Container verbunden ist.

In diesem neuen Fenster arbeiten Sie ganz normal, aber **alle Aktionen** (Editieren, Kompilieren, Testen, Debuggen, Tools, Extensions) **laufen im Container**. Sie öffnen also Dateien im Container, übersetzen und testen dort, und nutzen die im Container installierten Tools. Über die Dev-Container-Konfiguration können sogar VSCode-Extensions direkt im Container installiert werden.

Damit benötigen Sie auf dem Host eigentlich nur noch VSCode und Docker (oder Podman), aber keine lokalen Java-Tools o.ä. mehr. Die gesamte Entwicklungsumgebung (JDK-Version, Build-Tools, Java-Extensions in VSCode, zusätzliche Pakete) ist über die DevContainer-Konfiguration im Projekt definiert und versioniert.

Tip

Beispiel: Dev Container für Java 25 mit Temurin

Ein typischer, schlanker DevContainer für ein Java-Projekt mit OpenJDK 25 (Temurin) könnte z.B. so aussehen:

```
1 // .devcontainer/devcontainer.json
2 {
3     "name": "Java 25 DevContainer",
4     "image": "eclipse-temurin:25-jdk",
5     "workspaceFolder": "/workspace",
6     "workspaceMount": "source=${localWorkspaceFolder},target=/workspace,
7         type=bind",
8     "customizations": {
9         "vscode": {
10             "extensions": [
11                 "vscjava.vscode-java-pack"
12             ]
13         }
14     }
15 }
```

Erläuterungen dazu:

- `"image": "eclipse-temurin:25-jdk"` nutzt ein Docker-Image mit einem OpenJDK-25-Build von Eclipse Temurin.
- `"workspaceFolder": "/workspace"` und `"workspaceMount": "..."` binden den lokalen Projektordner in den Container unter `/workspace` ein. Das ist der Ordner, in dem Sie im Container arbeiten.
- Unter `"customizations.vscode.extensions"` wird das Java Extension Pack automatisiert im Container installiert. Beim ersten Start des DevContainers sorgt VSCode dafür, dass diese Extensions in der Container-Instanz von VSCode zur Verfügung stehen (Sprache-Support, Debugging, Test-Integration, ...).

Anmerkung: IntelliJ kann remote nur debuggen, d.h. das Editieren, Übersetzen, Testen läuft lokal auf dem Host (und benötigt dort den entsprechenden Tool-Stack). Für das Debuggen kann Idea das übersetzte Projekt auf ein Remote (SSH, Docker) schieben und dort debuggen.

Noch einen Schritt weiter geht das Projekt [code-server](#): Dieses stellt u.a. ein Docker-Image [codercom/code-server](#) bereit, welches einen Webserver startet und über diesen kann man ein im Container laufendes (angepasstes) VSC erreichen. Man braucht also nur noch Docker und das Image und kann dann über den Webbrowser programmieren. Der Projektordner wird dabei in den Container gemountet, so dass die Dateien entsprechend zur Verfügung stehen:

```
1 docker run -it --name code-server -p 127.0.0.1:8080:8080 -v "$HOME/.config:/home/coder/.config" -v "$PWD:/home/coder/project" codercom/code-server:latest
```

Auf diesem Konzept setzt auch der kommerzielle Service [GitHub Codespaces](#) von GitHub auf.

Tip

Diese Entwicklung hat in den letzten Jahren Fahrt aufgenommen und ist mittlerweile unter dem Namen “DevContainer” (besser) bekannt:

- Die Extension heißt “Dev Containers” (bzw. ist Teil des “Remote Development”-Pakets)
- Der zentrale Begriff ist der “Dev Container” bzw. die Konfiguration über eine [devcontainer.json](#)
- Im VS-Code-UI finden Sie z.B.:
 - “Open Folder in Container ...”
 - “Add Dev Container Configuration File ...”
 - “Reopen in Container”

Dazu gibt es auch das eigenständige Projekt:

[containers.dev](#) und [code.visualstudio.com/docs/devcontainers/containers](#)

Demo: VSCode und Docker

3.7.8. Wrap-Up

- Schlanke Virtualisierung mit Containern (kein eigenes OS)
- *Kein* echter Sandbox-Effekt
- Begriffe: Docker-File vs. Image vs. Container
- Ziehen von vordefinierten Images
- Definition eines eigenen Images
- Arbeiten mit Containern: lokal, CI/CD, VSCode ...

Zum Nachlesen

Link-Sammlung

- [Wikipedia: Docker](#)
- [Wikipedia: Virtuelle Maschinen](#)
- [Docker: Überblick, Container](#)
- [Docker: HowTo](#)

- [DockerHub: Suche nach fertigen Images](#)
- [Docker und Java](#)
- [Dockerfiles: Best Practices](#)
- [Gitlab, Docker](#)
- [VSCode: Entwickeln in Docker-Containern](#)

Lernziele

- k2: Ich kann zwischen Containern und VMs unterscheiden
- k1: Ich kenne typische Einsatzgebiete für Container
- k2: Ich verstehe, dass Container als abgeschottete Prozesse auf dem Host laufen - kein echter Sandbox-Effekt
- k3: Ich kann Container von DockerHub ziehen
- k3: Ich kann Container starten
- k3: Ich kann eigene Container definieren und bauen
- k3: Ich kann Container in GitLab CI/CD und/oder GitHub Actions einsetzen

4. Testen mit JUnit und Mockito

4.1. Testen mit JUnit (JUnit-Basics)

TL;DR

Fehler schleichen sich durch Zeitdruck und hohe Komplexität schnell in ein Softwareprodukt ein. Die Folgen können von “ärgerlich” über “teuer” bis hin zu (potentiell) “tödlich” reichen. Richtiges Testen ist also ein wichtiger Aspekt bei der Softwareentwicklung!

JUnit ist ein Java-Framework, mit dem Unit-Tests (aber auch andere Teststufen) implementiert werden können. In JUnit zeichnet man eine Testmethode mit Hilfe der Annotation `@Test` an der entsprechenden Methode aus. Dadurch kann man Produktiv- und Test-Code prinzipiell mischen; Best Practice ist aber das Anlegen eines weiteren Ordners `test/` und das Spiegeln der Package-Strukturen und sowie pro Klasse eine korrespondierende Test-Klasse zu nutzen. In den Testmethoden baut man den Test auf, führt schließlich den Testschritt durch (beispielsweise konkreter Aufruf der zu testenden Methode) und prüft anschließend mit einem `assert*()`, ob das erzielte Ergebnis dem erwarteten Ergebnis entspricht. Ist alles OK, ist der Test “grün”, sonst “rot”. Da ein fehlschlagendes `assert*()` den Test abbricht, werden eventuell danach folgende Prüfungen **nicht** mehr durchgeführt und damit ggf. weitere Fehler maskiert. Deshalb ist es gute Praxis, in einer Testmethode nur einen Testfall zu implementieren und i.d.R. auch nur wenige Aufrufe von `assert*()` pro Testmethode zu haben.

Über die verschiedenen `assume*()`-Methoden kann geprüft werden, ob eventuelle Vorbedingungen für das Ausführen eines Testfalls erfüllt sind - anderenfalls wird der Testfall dann übersprungen.

Videos

Vorlesung:

- Teil 1 [YT], [HSBI]
- Teil 2 [YT], [HSBI]

Demos:

- Anlegen von Testfällen [YT], [HSBI]
- `assume()` vs. `assert()` [YT], [HSBI]
- Parametrisierte Tests [YT], [HSBI]

4.1.1. Software-Fehler und ihre Folgen

1982: Absturz eines F117 Kampffjets: in der Softwaresteuerung Höhen- und Seitenruder vertauscht
<http://de.wikipedia.org/wiki/Programmfehler>

2005: TOKIOTER BÖRSE: Chef tritt ab wg. Softwarefehler (mehrere 100 Millionen Dollar Verlust)

<http://www.manager-magazin.de/unternehmen/karriere/0,2828,druck-391434,00.html>

2006: VW ruft 3500 Passat-Modelle zurück. Ein Softwarefehler kann zum Absterben des Motors führen.

http://auto.t-online.de/volkswagen-rueckruf-fuer-den-passat-wegen-softwarefehler/id_12821896/index

2007: Defektes Computersystem für den Tod von zehn Soldaten verantwortlich

<http://www.heise.de/newsticker/meldung/Defektes-Computersystem-fuer-den-Tod-von-zehn-Soldaten-verantwortlich-186999.html?view=print>

2008: Volvo-Rückruf: XC90 mit Software-Problem (Zündung vs. Klimaanlage)

<http://www.auto-motor-und-sport.de/news/volvo-rueckruf-xc90-mit-software-problem-711983.html>

2010: Softwarefehler in EC-Karten-Sicherheitschip: Kunden bekommen kein Geld am Automaten

<http://www.tagesspiegel.de/wirtschaft/ec-karten-panne-zum-jahreswechsel/1658102.html>

2010: Rückrufe kosten Toyota mehr als eine Milliarde Euro (Softwareprobleme)

<http://www.spiegel.de/wirtschaft/unternehmen/0,1518,druck-675874,00.html>

2010: Ford: Rückrufe wg. Softwareproblemen im Bremssystem

<http://www.automobil-produktion.de/2010/02/rueckruf-jetzt-auch-ford/>

2011: Honda startet umfangreiche Rückrufaktion, weltweit etwa 2,5 Millionen Autos betroffen (Softwareprobleme können zu Schäden am Getriebe führen)

http://www.handelsblatt.com/unternehmen/industrie/softwareprobleme-honda-startet-umfangreiche-rueckrufaktion/v_detail_tab_print/4470752.html

2012: Report "Softwareentwicklung 2012" (Accso): "Softwarefehler kosten die deutsche Volkswirtschaft Milliarden"

<http://www.elektronikpraxis.voel.de/index.cfm?pid=5416&pk=361330&print=true&printtype=article>

4.1.1.1. (Einige) Ursachen für Fehler

- Zeit- und Kostendruck
- Mangelhafte Anforderungsanalyse
- Hohe Komplexität
- Mangelhafte Kommunikation
- Keine/schlechte Teststrategie
- Mangelhafte Beherrschung der Technologie
- ...

4.1.1.2. Irgendjemand muss mit Deinen Bugs leben!

Leider gibt es im Allgemeinen keinen Weg zu zeigen, dass eine Software korrekt ist. Man kann (neben formalen Beweisansätzen) eine Software oft nur unter möglichst vielen Bedingungen ausprobieren, um ihr Verhalten zu beobachten und um die versteckten Bugs zu triggern.

Mal abgesehen von der verbesserten *User-Experience* führt weniger fehlerbehaftete Software auch dazu, dass man seltener mitten in der Nacht geweckt wird, weil irgendwo wieder ein Server gecrasht ist ... Weniger fehlerbehaftete Software ist auch leichter zu ändern und zu pflegen! In realen Projekten macht Maintenance den größten Teil an der Softwareentwicklung aus ... Während Ihre Praktikumsprojekte vermutlich nach der Abgabe nie wieder angeschaut werden, können echte Projekte viele Jahre bis Jahrzehnte leben! D.h. irgendwer muss sich dann mit Ihren Bugs herumärgern - vermutlich sogar Sie selbst ;)

Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live. Code for readability.

– John F. Woods

Dieses Zitat taucht immer mal wieder auf, beispielsweise auf der [OSCON 2014](#) ... Es scheint aber tatsächlich, dass [John F. Woods](#) die ursprüngliche Quelle war (vgl. [Stackoverflow: 876089](#)).

Da wir nur wenig Zeit haben und zudem vergesslich sind und obendrein die Komplexität eines Projekts mit der Anzahl der Code-Zeilen i.d.R. nicht-linear ansteigt, müssen wir das Testen automatisieren. Und hier kommt JUnit ins Spiel.

4.1.2. Zentrale Begriffe: Was wann testen?

- **Modultest / Unit-Test**
 - Testen einer Klasse und ihrer Methoden
 - Test auf gewünschtes inneres Verhalten (Parameter, Schleifen, ...)
- **Integrationstest**
 - Test des korrekten Zusammenspiels mehrerer Komponenten
 - Fokus auf Schnittstellen, Datenaustausch, Zusammenspiel
- **Systemtest / End-to-End-Test (E2E)**
 - Test des kompletten Systems unter produktiven Bedingungen
 - Fokus: Gesamter Geschäftsprozess, UI, Use Cases
 - Funktionale und nichtfunktionale Anforderungen testen
- **Regressionstest**
 - Änderungen dürfen bestehende Funktionalität nicht brechen
 - Fokus: Wiederholung von bestehenden Tests nach Code-Änderungen

=> Verweis auf Wahlfach “Softwarequalität”

Dies sind einige ausgewählte Testarten - man kann ohne große Probleme noch viel genauer hinschauen und viele weitere Arten und Stufen unterscheiden.

4.1.3. JUnit: Test-Framework für Java

JUnit - Open Source Java Test-Framework zur Erstellung und Durchführung wiederholbarer Tests

- JUnit 3
 - Tests müssen in eigenen Testklassen stehen
 - Testklassen müssen von Klasse `TestCase` erben
 - Testmethoden müssen mit dem Präfix `“test”` beginnen
- JUnit 4
 - Annotation `@Test` für Testmethoden
 - Kein Zwang zu spezialisierten Testklassen (insbesondere kein Zwang mehr zur Ableitung von `TestCase`)
 - Freie Namenswahl für Testmethoden (benötigen nicht mehr Präfix `“test”`)

Damit können prinzipiell auch direkt im Source-Code Methoden als JUnit-Testmethoden ausgezeichnet werden ... (das empfiehlt sich in der Regel aber nicht)

- **JUnit 5 und 6 = JUnit Platform + JUnit Jupiter + JUnit Vintage**

- Erweiterung um mächtigere Annotationen
- Aufteilung in spezialisierte Teilprojekte

Das Teilprojekt “JUnit Platform” ist die Grundlage für das JUnit-Framework. Es bietet u.a. einen Console-Launcher, um Testsuiten manuell in der Konsole zu starten oder über Builder wie Maven oder Gradle.

Das Teilprojekt “JUnit Jupiter” ist das neue Programmiermodell zum Schreiben von Tests seit JUnit 5. Es beinhaltet eine TestEngine zum Ausführen der in Jupiter geschriebenen Tests.

Das Teilprojekt “JUnit Vintage” beinhaltet eine TestEngine zum Ausführen von Tests, die in JUnit 3 oder JUnit 4 geschrieben sind.

Tip

Wie der Name schon sagt, ist das Framework für Modultests (“Unit-Tests”) gedacht. Man kann damit aber auch prima auf anderen Teststufen arbeiten und beispielsweise Systemtests definieren.

Caution

In dieser Lehrveranstaltung besprechen wir JUnit am Beispiel **JUnit 6**. **Sofern nicht explizit etwas anderes vermerkt ist, meint die Angabe “JUnit” immer “JUnit (Version 6.x)”**.

Tip

Mit JUnit 3 sollte definitiv nicht mehr aktiv gearbeitet werden, d.h. insbesondere keine neuen Tests mehr erstellt werden, da diese Version nicht mehr weiterentwickelt wird. Es kann sein, dass Ihnen in Produktivumgebungen noch häufig Testsuiten in JUnit 4 begegnen - achten Sie dort auf die verwendeten Annotationen,

die sich teilweise leicht von der modernen Variante unterscheiden.

4.1.4. Einbinden von JUnit (Gradle)

Am einfachsten bindet man JUnit über das Build-Tool in das Projekt ein. Dabei werden dann automatisch alle Abhängigkeiten aufgelöst und die relevanten Jar-Files heruntergeladen und in den Classpath eingebunden.

Für Gradle muss man zwei Einträge vornehmen: (a) JUnit als Dependency deklarieren und (b) für die Test-Targets die JUnit-Plattform aktivieren. Am einfachsten geht es wie in den [JUnit-Beispielprojekten](#) gezeigt mit einer BOM-Deklaration (*Bill of Materials*), die auch die Version trägt. Dann noch ein Eintrag für Jupiter (beinhaltet die Testengine und die API) sowie zum tatsächlichen Ausführen der Tests den Launcher. Im Beispiel werden diese Dependencies so deklariert, dass sie wirklich nur für das Übersetzen der Tests ("testImplementation") bzw. zur Laufzeit beim Ausführen der Tests ("testRuntimeOnly") eingebunden werden.

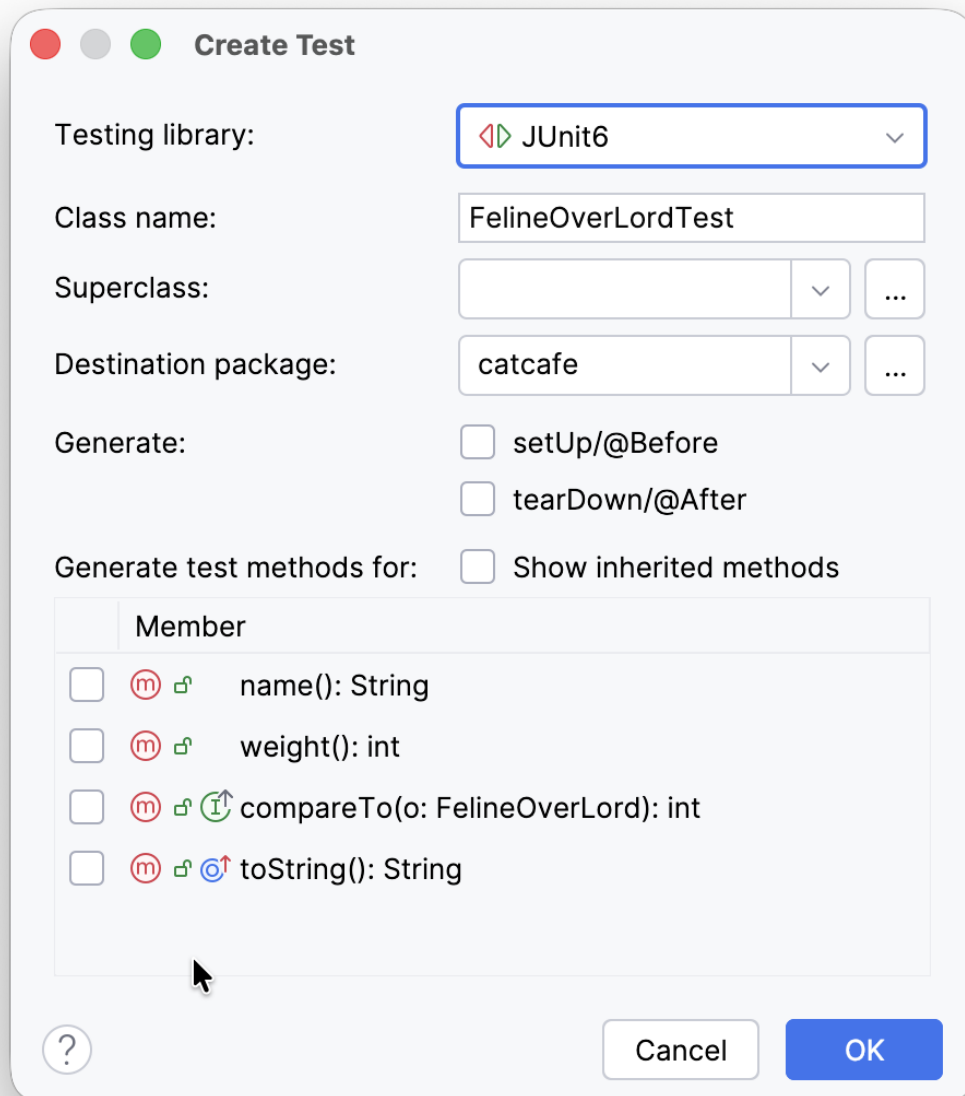
```
1  plugins {
2      id 'java'
3  }
4
5  repositories {
6      mavenCentral()
7  }
8
9  dependencies {
10     testImplementation(platform('org.junit:junit-bom:6.0.3'))
11     testImplementation('org.junit.jupiter:junit-jupiter')
12     testRuntimeOnly('org.junit.platform:junit-platform-launcher')
13 }
14
15 test {
16     useJUnitPlatform()
17 }
```

Wie üblich findet man die Einträge für die Dependencies auf [Maven Central](#).

Ein Blick in die [JUnit-Dokumentation](#) ist immer lohnenswert.

4.1.5. Anlegen und Organisation der Tests mit JUnit

- Anlegen neuer Tests: Klasse auswählen, Kontextmenü [Generate](#) > [Test](#)
- **Best Practice:** Spiegeln der Paket-Hierarchie
 - Toplevel-Ordner `src/test/java/` (statt `src/main/java/`)
 - Package-Strukturen spiegeln
 - Testklassen mit Suffix `"Test"`



Für die Klasse `foo.bar.Wuppie` im `src/main/java/-`Ordner erzeugen Sie also die Testklasse `foo.bar.WuppieTest` im `src/test/java/-`Ordner. Aus Java-Sicht werden beide Ordner als “Source-Ordner” deklariert (über Gradle).

4.1.5.1. Vorteile dieses Vorgehens

- Die Testklassen sind aus Java-Sicht im selben Package wie die Source-Klassen, d.h. Zugriff auf Package-sichtbare Methoden etc. ist gewährleistet
- Durch die Spiegelung der Packages in einem separaten Testordner erhält man eine gute getrennte Übersicht über jeweils die Tests und die Sourcen
- Die Wiederverwendung des Klassennamens mit dem Anhang "Test" erlaubt die schnelle Erkennung, welche Tests hier vorliegen

In der Paketansicht liegen dann die Source- und die Testklassen immer direkt hintereinander (da sie im selben Paket sind und mit dem selben Namen anfangen) => besserer Überblick!

4.1.5.2. Anmerkung: Die (richtige) JUnit-Bibliothek muss im Classpath liegen!

Die IDE's fragen typischerweise beim Anlegen des ersten Tests nach, ob sie die passenden Jar-Files für JUnit herunterladen und in den (lokalen) Classpath einfügen sollen.

Achtung: Das passt dann für das lokale Bauen/Testen auf Ihrem Rechner, klappt aber nicht mehr, wenn das Projekt mit verschiedenen Personen bearbeitet wird. Dann müsste jede für sich sicherstellen, dass die richtigen Bibliotheken vorhanden sind. Deshalb: Bitte nutzen Sie ein Buildtool wie Gradle und konfigurieren Sie dort JUnit als externe Dependency. Das Buildtool kümmert sich dann um das Auflösen und Herunterladen der Jar-Files, und da die Konfiguration im Projekt eingecheckt ist, klappt das auch für alle anderen am Projekt beteiligten Personen.

4.1.6. Definition von Testmethoden

Annotation `@Test` vor Testmethode schreiben

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import org.junit.jupiter.api.Test;
3
4 class DemoResults {
5     @Test
6     void passes_when_assertion_is_true() {
7         assertEquals(0, 1 - 1);
8     }
9 }
```

Hinweis Sichtbarkeit: Während in JUnit 4 die Testmethoden mit der Sichtbarkeit `public` versehen sein müssen und keine Parameter haben (dürfen), spielt die Sichtbarkeit ab JUnit 5 keine Rolle mehr (und die Testmethoden dürfen Parameter aufweisen). *Best Practice* ist aktuell, auf `package-private` zu gehen. Die Testklassen/-methoden gehören nicht in die öffentliche API, und man möchte normalerweise von Testklassen auch nicht erben.

Hinweis Namen: In Java werden Methoden üblicherweise in *camelCase* geschrieben. Bei den Tests wird häufig davon bewusst abgewichen und es werden Unterstriche eingesetzt, um den Methodennamen zu einem "Satz" zu machen - und dieser "Satz" erklärt, was der Test macht. Wie bei den normalen Methodennamen muss man hier

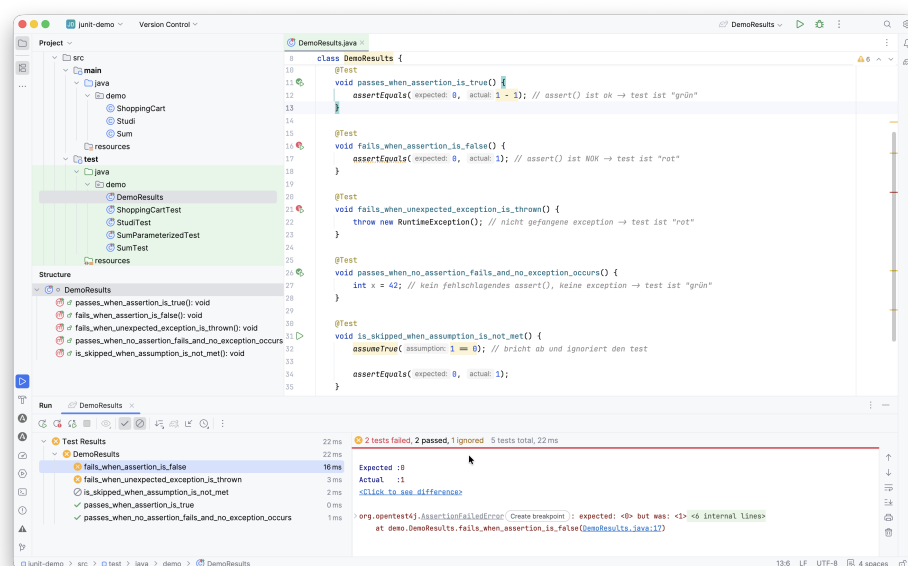
aber auch aufpassen, dass die Namen nicht zu lang und damit unübersichtlich werden (auch wenn sie etwas länger ausfallen dürfen als bei normalen Methoden).

4.1.7. Ergebnis prüfen

Klasse `org.junit.jupiter.api.Assertions` enthält diverse **statische** Methoden zum Prüfen:

```
1 // Argument muss true bzw. false sein
2 static void assertTrue(boolean condition);
3 static void assertFalse(boolean condition);
4
5 // Gleichheit im Sinne von equals()
6 static void assertEquals(Object expected, Object actual);
7 static void assertEquals(long expected, long actual);
8
9 // Test sofort fehlschlagen lassen
10 static <V> V fail(String message);
11
12 ...
```

4.1.8. Mögliche Testausgänge bei JUnit: rot/grün/ignoriert



1. **grün**: Testausgang positiv ("passed")

- Alle Assertions sind erfolgreich
- Es gibt keine unbehandelte Exception

2. **rot**: Testausgang negativ ("failed")

- Ein Assert ist fehlgeschlagen
- `Assert.fail()` wurde aufgerufen
- Unbehandelte Exception aufgetreten

3. **ignoriert:** Testausgang “ignored”

- Vorbedingung via `assume()` nicht erfüllt
- Test mit `@Disabled` annotiert

4.1.9. Anmerkungen zu den Testmethoden

4.1.9.1. Pro Testmethode nur eine Testidee => wenige Asserts

Pro Testmethode sollte nur eine konkrete Testidee umgesetzt werden, so dass die Prüfung des Ergebnisses mit möglichst **wenigen** Asserts stattfinden kann!

Hintergrund: Schlägt ein Assert fehl, wird der Test abgebrochen und der Rest nicht mehr überprüft ... Dadurch können weitere Fehler maskiert werden.

4.1.9.2. Tests sollen sich selbst erklären

Wenn ich nur die Testmethode lese, sollte ich ungefähr verstehen, was passiert.

4.1.9.3. Tests sollten unabhängig von einander sein

Jeder Test steht für sich und bekommt seine eigene “Welt” (Fixture). Es sollte keine statische “globalen” Zustände geben, die über mehrere Tests hinweg verändern werden (außer mit einem extrem gutem Grund, und dann sehr gut dokumentiert).

Wenn Testmethode A vor Testmethode B laufen muss, weil A einen Zustand des Testobjekts herstellt, den B braucht, führt das in meiner Erfahrung **immer** zu Problemen, selbst wenn die Abhängigkeit gut dokumentiert sind.

Außerdem kann man nicht davon ausgehen, dass die Ausführungsreihenfolge der Testmethoden deterministisch ist!

4.1.9.4. Tests sollten deterministisch sein (keine “flaky tests”)

Tests dürfen nicht zufällig mal grün, mal rot sein. Die Berechnung und Prüfung in Testmethoden sollten sich immer deterministisch verhalten.

- Kein `Thread.sleep(...)` ohne Not.
- Kein Abhängigsein von:
 - aktueller Uhrzeit,
 - Netzwerk,

- externen Services,
- Reihenfolge anderer Tests!

4.1.9.5. Saubere Assertions: “Act, then assert”

Keine Logik in Assertions (keine Aufrufe, keine komplizierten Berechnungen). Das macht dem Test schwer lesbar (viele Asserts, versteckte Bedingungen) und wartbar.

4.1.9.6. To “assert” or to “assume”?

- Mit `Assertions.assert*` werden Testergebnisse geprüft
 - Test wird ausgeführt
 - Ergebnis: grün/rot
- Mit `Assumptions.assume*` werden Annahmen über den Zustand geprüft
 - Test wird abgebrochen, wenn Annahme nicht erfüllt (Ergebnis: “Ignored”)
 - Prüfen von Vorbedingungen: Ist der Test hier ausführbar/anwendbar?

Im JUnit-Kontext nutzen wir `Assumptions.assume*` für das **Überprüfen von Annahmen** (im Sinne von **Vorbedingungen**): Wenn ein `Assumptions.assume*` fehlschlägt, wird der Testfall abgebrochen bzw. als “ignoriert” gewertet.

Dagegen setzen wir `Assertions.assert*` für das **Überprüfen der Testergebnisse** ein, d.h. ein fehlschlagendes `Assertions.assert*` lässt den Testfall “rot” werden.

4.1.9.7. Java: “assert”-Keyword

Neben den wohlbekannten `Assertions.assert*`-Methoden aus JUnit gibt es auch direkt von Java ein etwas verstecktes `assert`-Keyword, mit dem man Annahmen über Zustände und Werte explizit ausdrücken kann:

```
1 public void foo() {
2     String bar = wuppie();
3     assert bar != null : "result of wuppie() must not be null";
4 }
```

Das `assert` besteht aus einer zu prüfenden Bedingung und einem String. Wenn die Bedingung erfüllt ist, läuft der Code einfach normal weiter. Anderenfalls wird ein `AssertionError` geworfen mit dem angegebenen String als Message.

Allerdings sind diese JVM-Assertions per Default **deaktiviert**. Man muss sie beim Aufruf manuell über die Option `-enableassertions` bzw. `-ea` (Kurzschreibweise) aktivieren (`java -ea main`)! Dies gilt auch beim Start über die IDE oder Gradle ...

Warning

Die Assertions sind per Default deaktiviert und müssen erst manuell aktiviert werden. Außerdem wird bei Verletzung der Bedingung eine *unchecked exception* (ein Error) geworfen, der auf einen nicht korrigierbaren Programzustand hindeutet.

1. Nutzen Sie das Java-`assert` deshalb nicht als Ersatz für das normale Prüfen von Parametern von `public` Methoden (also Methoden der Schnittstelle, die Ihre Kunden aufrufen).
2. Während der Entwicklungszeit kann das Java-`assert` aber ganz nützlich sein, weil Sie so interne Annahmen sichtbar und prüfbar machen (vorausgesetzt, Sie haben `-ea` aktiviert).
Analog könnte ein Java-`assert` an Stellen eingebaut werden, die eigentlich nicht erreichbar sein sollten (etwa nach einer Dauerschleife oder in einem nicht erreichbaren `default`-Zweig in einem `switch`).
3. Bitte das Java-`assert` **nie** in einer JUnit-Testmethode statt der “richtigen” JUnit-`assert`* verwenden!
4. Das Java-`assert` ist in einer JUnit-Testmethode **kein** Ersatz für die JUnit-`assume`*-Methoden!

4.1.10. BDD: “Given - When - Then”-Mantra

Aus dem `Behavior-driven development` stammt die Strukturierung nach den Punkten “given - when - then” (oft auch als “*given - when - then*”-Mantra bezeichnet).

Betrachten Sie noch einmal die Schnittstelle der `Studi`-Klasse:

```
1 public class Studi {
2     public String getName();
3     public void setName(String name);
4     public int getCredits();
5     public void addToCredits(int credits);
6 }
```

Dafür wurde ein Test geschrieben:

```
1 // Before BDD
2 class StudiTest {
3     @Test
4     void testAddToCredits() {
5         Studi s = new Studi();
6         int cps = 2;
7         s.addToCredits(cps);
8         assertEquals(cps, s.getCredits());
9     }
10 }
```

Dieser Code ist in seiner Absicht nicht sofort verständlich. Es fällt auf, dass

1. am Anfang eine Art Setup des Tests vorgenommen wird und das Testobjekt initialisiert wird (“**given**”).
2. Dann wird die zu untersuchende Aktion ausgeführt (“**when**”), gefolgt von

3. einem Vergleich des tatsächlichen mit dem erwarteten Ergebnis ("then").

Diese gedachte Struktur kann (und sollte!) man mit entsprechenden Kommentaren auch sichtbar machen:

```
1 // With BDD: given-when-then
2 class StudiTest {
3     @Test
4     void accumulates_credits_within_upper_limit() {
5         // given
6         var studi = new Studi();
7         var startCredits = 2;
8
9         // when
10        studi.addToCredits(startCredits);
11        var endCredits = studi.getCredits();
12
13        // then
14        assertEquals(startCredits, endCredits);
15    }
16 }
```

In Testframeworks wie [Spock](#) oder [Cucumber](#) ist dies sogar bereits in die Sprache (eine DSL zum Testen) eingebaut! Einen schönen Blog zum Thema finden Sie hier: [Spock testing framework versus JUnit](#).

Weiterhin könnte (und sollte) man die implizit getroffenen Annahmen über das SUT für alle sichtbar machen:

```
1 class StudiTest {
2     @Test
3     void accumulates_credits_within_upper_limit() {
4         // given
5         var studi = new Studi();
6         var startCredits = 2;
7         assumeTrue(s.getCredits() == 0, "initial credits should be 0");
8
9         // when
10        studi.addToCredits(startCredits);
11        var endCredits = studi.getCredits();
12
13        // then
14        assertEquals(startCredits, endCredits);
15    }
16 }
```

Der Test würde ohnehin fehlschlagen, wenn ein neues `Studi`-Objekt mit einem anderen Wert für die Credits initialisiert würde. Aber so zeigt das `assume` direkt unsere (bisher) implizite Annahme sichtbar an, und bei einer Verletzung dieser Annahme würde der Testfall mit einer entsprechenden Mitteilung nicht ausgeführt.

... Und nun könnte man sich fragen, warum man das Erhöhen von Credits nur für ein *neues* `Studi`-Objekt testet und nicht auch für andere Zustände des `Studi`-Objekts? ... => Parametrisierte Tests!

4.1.11. Test-Fixtures: Testübergreifende Konfiguration

```
1 private Studi x;
2
```

```

3  @BeforeEach
4  void setUp() { x = new Studi(); }
5
6  @Test
7  void toString_formats_name_and_credits() {
8      // Studi x = new Studi();
9      assertEquals("Heinz (15cps)", x.toString());
10 }

```

- Vor/nach *jedem* einzelnen Test:
 - **@BeforeEach** : wird **vor jeder** Testmethode aufgerufen
 - **@AfterEach** : wird **nach jeder** Testmethode aufgerufen
- Einmalig (Klassenmethoden):
 - **@BeforeAll** : wird **einmalig** vor allen Tests aufgerufen (**static!**)
 - **@AfterAll** : wird **einmalig** nach allen Tests aufgerufen (**static!**)

Beispiel für den Einsatz von **@BeforeEach**

Annahme: **alle/viele** Testmethoden brauchen **neues** Objekt *x* vom Typ *Studi*

```

1  private Studi x;
2
3  @BeforeEach
4  void setUp() {
5      x = new Studi("Heinz", 15);
6  }
7
8  @Test
9  void toString_formats_name_and_credits() {
10     // Studi x = new Studi("Heinz", 15);
11     assertEquals("Name: Heinz, credits: 15", x.toString());
12 }
13
14 @Test
15 public void getName_returns_student_name() {
16     // Studi x = new Studi("Heinz", 15);
17     assertEquals("Heinz", x.getName());
18 }

```

Achtung: Übertreiben Sie es nicht mit den Test-Fixtures!

Häufig findet man die Test-Fixtures (beispielsweise die **@BeforeEach**-Methode) ganz oben im Code, und wenn man dann weiter unten an einer Testmethode arbeitet, hat man die Setup-Methode nicht mehr im Blick (“verstecktes Setup”).

Das ist problematisch, weil:

- Sie beim Lesen einer Testmethode nicht mehr direkt sehen, in welchem Zustand die Objekte am Anfang des Tests sind.
- eine Änderung im Setup **alle** Testmethoden betreffen kann, ohne dass Sie das beim Ändern im Kopf haben (starke Kopplung).

Typischerweise entsteht dann schnell ein “One size fits nobody”-Setup: Es gibt nur *eine* gemeinsame Setup-Methode, die automatisch vor *allen* Tests aufgerufen wird, und man versucht, dort eine gemeinsame Basis für sehr unterschiedliche Tests zu schaffen. Diese Methode wird immer größer und unübersichtlicher.

Deshalb gibt es Strömungen, die vom Gebrauch von Test-Fixtures komplett abraten: Lieber ein **explizites Setup in der Testmethode**, das Sie direkt sehen, als ein “magisches” gemeinsames Fixture.

Ich empfehle einen pragmatischen Mittelweg:

- Nutzen Sie Test-Fixtures (z.B. `@BeforeEach`) **sparsam**.
- Lagern Sie dort nur die **grundsätzliche, einfache Initialisierung** aus, die für alle Tests offensichtlich gleich ist (z.B. das Erzeugen eines Service-Objekts).
- Alles, was über einfache und offensichtliche Dinge hinausgeht, gehört in die **einzelnen Testmethoden**. Dort definieren Sie das “fachliche Setup” für den jeweiligen Testfall.

Das verletzt an dieser Stelle bewusst das DRY-Prinzip (es kann etwas Code-Duplikation geben), führt aber zu **besser verständlichen und wartbaren Tests** und ist beim Testen deshalb akzeptiert.

Wenn die Testmethoden dadurch zu lang werden, können Sie Teile in **Hilfsmethoden** auslagern, die Sie gezielt in einzelnen Tests verwenden. Diese Hilfsmethoden werden dann *nicht automatisch* vor jedem Test aufgerufen (anders als `@BeforeEach`), sondern nur dort, wo Sie sie explizit einsetzen.

Der Grundsatz aus dem “normalen” Code “*keine globale magische Welt*” gilt auch für Testcode – und dort eigentlich sogar noch stärker.

4.1.12. Test von Exceptions

Traditionelles Testen von Exceptions mit **try** und **catch**:

```
1 @Test
2 void divisionByZero_throwsArithmeticException() {
3     try {
4         int i = 0 / 0;
5         fail("keine ArithmeticException ausgelöst");
6     } catch (ArithmeticException aex) {
7         assertNotNull(aex.getMessage());
8     } catch (Exception e) {
9         fail("falsche Exception geworfen");
10    }
11 }
```

Ab JUnit 5 kann man hier `org.junit.jupiter.api.Assertions.assertThrows` nutzen. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
1 @Test
2 void divisionByZero_throwsArithmeticException() {
3     assertThrows(java.lang.ArithmeticException.class, () -> { int i = 0 / 0; })
4     ;
5 }
```


4.1.13. Parametrisierte Tests

Manchmal möchte man den selben Testfall mehrfach mit anderen Werten (Parametern) durchführen.

```

1  class Sum {
2      public static int sum(int i, int j) {
3          return i + j;
4      }
5  }
6
7  class SumTest {
8      @Test
9      void sum_of_one_and_one_is_two() {
10         assertEquals(2, Sum.sum(1, 1));
11     }
12     // und mit (2,2, 4), (2,2, 5), ...???
13 }

```

Prinzipiell könnte man dafür entweder in einem Testfall eine Schleife schreiben, die über die verschiedenen Parameter iteriert. In der Schleife würde dann jeweils der Aufruf der zu testenden Methode und das gewünschte Assert passieren. Alternativ könnte man den Testfall entsprechend oft duplizieren mit jeweils den gewünschten Werten.

Beide Vorgehensweisen haben Probleme: Im ersten Fall würde die Schleife bei einem Fehler oder unerwarteten Ergebnis abbrechen, ohne dass die restlichen Tests (Werte) noch durchgeführt würden. Im zweiten Fall bekommt man eine unnötig große Anzahl an Testmethoden, die bis auf die jeweiligen Werte identisch sind (Code-Duplizierung).

In JUnit werden **parametrisierte Tests** mit der Annotation `@ParameterizedTest` gekennzeichnet (statt mit `@Test`).

Mit Hilfe von "Source Annotations" wie `@ValueSource` oder `@CsvSource` und anderen können die zu verwendenden Werte spezifiziert werden. Dabei gibt es eine große Vielzahl an Möglichkeiten, im Folgenden wird die Spezifikation mit `@ValueSource` und `@CsvSource` direkt an einer Testmethode gezeigt.

In der Annotation `@ValueSource` kann man ein einfaches Array von Werten (Strings oder primitive Datentypen) angeben, mit denen der Test ausgeführt wird. Dazu bekommt die Testmethode einen entsprechenden passenden Parameter:

```

1  @ParameterizedTest
2  @ValueSource(strings = {"wuppie", "fluppie", "foo"})
3  void testWuppie(String candidate) {
4      assertTrue(candidate.equals("wuppie"));
5  }

```

Für das Testen der Summenfunktion könnte man mit `@CsvSource` die Testfälle so ausdrücken:

```

1  @ParameterizedTest
2  @CsvSource({
3      // s1,  s2,  result
4      "0,      0,      0",
5      "10,     0,      10",
6      "0,      11,     11",

```

```
7         "-2,    10,    8"
8     })
9     void sum_adds_two_integers(int s1, int s2, int erg) {
10         assertEquals(erg, Sum.sum(s1, s2));
11     }
```

Beispiel

4.1.14. Best Practices

1. Eine Testmethode behandelt exakt eine Idee/ein Szenario (einen Testfall). Das bedeutet auch, dass man in der Regel nur ein bis wenige `assert*` pro Testmethode benutzt.

Wenn man verschiedene Ideen in eine Testmethode kombiniert, wird der Testfall unübersichtlicher und auch schwerer zu warten. Außerdem können so leichter versteckte Fehler auftreten: Das erste oder zweite oder dritte `assert*` schlägt fehl - und alle dahinter kommenden `assert*` werden nicht mehr ausgewertet!

2. Wenn die selbe Testidee mehrfach wiederholt wird (mit anderen Werten), sollte man diese Tests zu einem parametrisierten Test zusammenfassen.

Das erhöht die Lesbarkeit drastisch - und man läuft auch nicht in das Problem der Benennung der Testmethoden.

3. Es wird nur das Verhalten der öffentlichen Schnittstelle getestet, nicht die inneren Strukturen einer Klasse oder Methode.

Es ist verlockend, auch private Methoden zu testen und in den Tests auch die Datenstrukturen o.ä. im Blick zu behalten und zu testen. Das führt aber zu sehr "zerbrechlichen" (*brittle*) Tests: Sobald sich etwas an der inneren Struktur ändert, ohne dass sich das von außen beobachtbare Verhalten ändert und also die Klasse/Methode immer noch ordnungsgemäß funktioniert, gehen all diese "internen" Tests kaputt. Nicht ohne Grund wird in der objektorientierten Programmierung mit Kapselung (Klassen, Methoden, ...) gearbeitet.

4. Von Setup- und Teardown-Methoden sollte eher sparsam Gebrauch gemacht werden.

Normalerweise folgen wir in der objektorientierten Programmierung dem DRY-Prinzip ([Don't repeat yourself](#)). Entsprechend liegt es nahe, häufig benötigte Elemente in einer Setup-Methode zentral zu initialisieren und ggf. in einer Teardown-Methode wieder freizugeben.

Das führt aber speziell bei Unit-Tests dazu, dass die einzelnen Testmethoden schwerer lesbar werden: Sie hängen von einer gemeinsamen, zentralen Konfiguration ab, die man üblicherweise nicht gleichzeitig mit dem Code der Testmethode sehen kann (begrenzter Platz auf der Bildschirmseite).

Wenn nun in einem oder vielleicht mehreren Testfällen der Wunsch nach einer leicht anderen Konfiguration auftaucht, muss man die gemeinsame Konfiguration entsprechend anpassen bzw. erweitern. Dabei muss man dann aber *alle* anderen Testmethoden mit bedenken, die ja ebenfalls von dieser Konfiguration abhängen! Das führt in der Praxis dann häufig dazu, dass die gemeinsame Konfiguration sehr schnell sehr groß und verschachtelt und entsprechend unübersichtlich wird.

Jede Änderung an dieser Konfiguration kann leicht einen oder mehrere Testfälle kaputt machen (man hat ja i.d.R. nie alle Testfälle gleichzeitig im Blick), weshalb man hier unbedingt mit passenden `assume*` arbeiten muss - aber dann kann man eigentlich auch stattdessen die Konfiguration direkt passend für den jeweiligen Testfall in der jeweiligen Testmethode erledigen!

5. Wie immer sollten auch die Namen der Testmethoden klar über ihren Zweck Auskunft geben.

Da Tests oft auch als “ausführbare Dokumentation” betrachtet werden, ist eine sinnvolle Benennung besonders wichtig. Oft werden hier deshalb Ausnahmen von den üblichen Java-Konventionen erlaubt. Man findet häufig das aus dem [Behavior-driven development](#) bekannte “given - when - then”-Mantra. Siehe auch [The subtle Art of Java Test Method Naming](#) und auch [Spock testing framework versus JUnit](#).

Der Präfix “test” für Testmethoden wird seit JUnit 4.x nicht mehr benötigt, aber dennoch ist es in vielen Projekten Praxis, diesen Präfix beizubehalten - damit kann man in der Package-Ansicht in der IDE leichter zwischen den “normalen” und den Testmethoden unterscheiden. Das ist analog zum Suffix “Test” für die Klassennamen der Testklassen ...

Diese Erfahrungen werden ausführlich in (Winters u. a. 2020, pp. 231-256) diskutiert.

Eine lesenswerte Diskussion von “Anti-Pattern” beim Testen finden Sie im Blog [Software Testing Anti-patterns](#).

4.1.15. Wrap-Up

JUnit als Framework für (Unit-) Tests

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbereitung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`

Zum Nachlesen

Zum Nachlesen ist die Dokumentation im [JUnit-Projekt](#) sehr empfehlenswert. Ebenfalls möchte ich Ihnen das Buch meines Kollegen Stephan Kleuker ans Herz legen: Kleuker (2026).

Das Video von Kevlin Henney [Structure and Interpretation of Test Cases \(Kevlin Henney, GOTO 2022\)](#) ist sehr interessant.

Hier noch weitere lesenswerte Blog-Beiträge zum Thema Testen:

- [Spock testing framework versus JUnit](#)
- [Software Testing Anti-patterns](#)
- [The subtle Art of Java Test Method Naming](#)

Lernziele

- k3: Ich kann Tests mit JUnit unter Nutzung der Annotation `@Test` erstellen
- k3: Ich kann Testergebnisse prüfen
- k2: Ich kenne den Unterschied zwischen `assert` und `assume`
- k3: Ich kann vor/nach jedem Test bestimmten Code ausführen

- k2: Ich habe verstanden, warum `@Before` und `@After` sparsam einzusetzen sind
- k3: Ich kann die Ausführung von Tests steuern, beispielsweise Tests ignorieren oder mit zeitlicher Begrenzung ausführen
- k3: Ich kann das Auftreten von Exceptions prüfen
- k3: Ich kann Tests zu Testsuiten zusammenfassen

Challenges

Einfache JUnit-Tests I

Betrachten Sie die folgende einfache (und nicht besonders sinnvolle) Klasse `MyList<T>`:

```
1 public class MyList<T> {
2     protected final List<T> list = new ArrayList<>();
3
4     public boolean add(T element) { return list.add(element); }
5     public int size() { return list.size(); }
6 }
```

Schreiben Sie mit Hilfe von JUnit (4.x oder 5.x) einige Unit-Tests für die beiden Methoden `MyList<T>#add` und `MyList<T>#size`.

Einfache JUnit-Tests II

Betrachten Sie die Methode `String concat(String str)` der Klasse `String` aus dem JDK.

Implementieren Sie drei verschiedenartige Unit-Testfälle (inklusive der Eingabe- und Rückgabewerte) für diese Methode mit Hilfe von JUnit (Version 4.x oder 5.x).

Setup und Teardown

Sie haben in den Challenges in "Intro SW-Test" erste JUnit-Tests für die Klasse `MyList<T>` implementiert. Wie müssten Sie Ihre JUnit-Tests anpassen, wenn Sie im obigen Szenario Setup- und Teardown-Methoden einsetzen würden?

Testmethoden

Betrachten Sie den folgenden Code. Was fällt Ihnen auf?

```
1 public class Studi {
2     public int getCredits();
3     public void addToCredits(int credits);
4
5     @Test
6     void testStudi() {
7         Studi s = new Studi();
8         s.addToCredits(2);
9         assertEquals(2, s.getCredits());
10    }
11 }
```

Parametrisierte Tests

Betrachten Sie die folgende einfache Klasse `MyMath`:

```
1 public class MyMath {
2     public static String add(String s, int c) {
3         return s.repeat(c);
4     }
5 }
```

```
5 }
```

Beim Testen der Methode `MyMath#add` fällt auf, dass man hier immer wieder den selben Testfall mit lediglich anderen Werten ausführt - ein Fall für parametrisierte Tests.

Schreiben Sie mit Hilfe von JUnit (4.x oder 5.x) einige parametrisierte Unit-Tests für die Methode `MyMath#add`.

Komplexerer Test

Betrachten Sie die Klasse `ShoppingCart`:

```
1 package junit6;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import java.util.Objects;
6
7 public class ShoppingCart {
8
9     public static class Item {
10         private final String id;
11         private final String name;
12         private final int unitPriceInCents;
13
14         public Item(String id, String name, int unitPriceInCents) {
15             this.id = Objects.requireNonNull(id);
16             this.name = Objects.requireNonNull(name);
17             if (unitPriceInCents < 0) {
18                 throw new IllegalArgumentException("Price must not be
19                     negative");
20             }
21             this.unitPriceInCents = unitPriceInCents;
22         }
23
24         public String getId() {
25             return id;
26         }
27
28         public int getUnitPriceInCents() {
29             return unitPriceInCents;
30         }
31     }
32
33     private final Map<String, Integer> quantitiesByItemId = new HashMap<>();
34     ;
35     private final Map<String, Item> itemsById = new HashMap<>();
36     private int percentageDiscount = 0;
37
38     public void addItem(Item item, int quantity) {
39         if (quantity <= 0) {
40             throw new IllegalArgumentException("Quantity must be positive");
41         }
42         ;
43         itemsById.putIfAbsent(item.getId(), item);
44         quantitiesByItemId.merge(item.getId(), quantity, Integer::sum);
45     }
46 }
```

```
43
44     public void removeItem(String itemId, int quantity) {
45         Integer current = quantitiesById.get(itemId);
46         if (current == null || quantity <= 0) {
47             return;
48         }
49         int newQuantity = current - quantity;
50         if (newQuantity <= 0) {
51             quantitiesById.remove(itemId);
52         } else {
53             quantitiesById.put(itemId, newQuantity);
54         }
55     }
56
57     public void applyPercentageDiscount(int discount) {
58         if (discount < 0 || discount > 100) {
59             throw new IllegalArgumentException("Discount must be between 0
60                 and 100");
61         }
62         this.percentageDiscount = discount;
63     }
64
65     public int getTotalPriceInCents() {
66         int sum = 0;
67         for (Map.Entry<String, Integer> entry : quantitiesById.entrySet()) {
68             Item item = itemsById.get(entry.getKey());
69             int quantity = entry.getValue();
70             sum += item.getUnitPriceInCents() * quantity;
71         }
72         int discountAmount = sum * percentageDiscount / 100;
73         return sum - discountAmount;
74     }
75
76     public boolean isEmpty() {
77         return quantitiesById.isEmpty();
78     }
79
80     public void clear() {
81         quantitiesById.clear();
82         percentageDiscount = 0;
83     }
```

Definieren Sie für diese Klasse verschiedene Testfälle. Achten Sie auf die Testidee, die Namen für die Methoden, den inneren Aufbau ("given - when - then") und setzen Sie auch gezielt parametrisierte Tests ein.

4.2. JUnit2 Testfallermittlung: Wie viel und was muss man testen?

TL;DR

Mit Hilfe der Äquivalenzklassenbildung kann man Testfälle bestimmen. Dabei wird der Eingabebereich für jeden Parameter einer Methode in Bereiche mit gleichem Verhalten der Methode eingeteilt (die sogenannten "Äquivalenzklassen"). Dabei können einige Äquivalenzklassen (ÄK) gültigen Eingabebereichen entsprechen ("gültige ÄK"), also erlaubten/erwarteten Eingaben (die zum gewünschten Verhalten führen), und die restlichen ÄK entsprechen dann ungültigen Eingabebereichen ("ungültige ÄK"), also nicht erlaubten Eingaben, die von der Methode zurückgewiesen werden sollten. Jede dieser ÄK muss in mindestens einem Testfall vorkommen, d.h. man bestimmt einen oder mehrere zufällige Werte in den ÄK. Dabei können über mehrere Parameter hinweg verschiedene gültige ÄK in einem Testfall kombiniert werden. Bei den ungültigen ÄK kann dagegen immer nur ein Parameter eine ungültige ÄK haben, für die restlichen Parameter müssen gültige ÄK genutzt werden, und diese werden dabei als durch diesen Testfall "nicht getestet" betrachtet.

Zusätzlich entstehen häufig Fehler bei den Grenzen der Bereiche, etwa in Schleifen. Deshalb führt man zusätzlich noch eine Grenzwertanalyse durch und bestimmt für jede ÄK den unteren und den oberen Grenzwert und erzeugt aus diesen Werten zusätzliche Testfälle.

Wenn in der getesteten Methode der Zustand des Objekts eine Rolle spielt, wird dieser wie ein weiterer Eingabeparameter für die Methode betrachtet und entsprechend in die ÄK-Bildung bzw. GW-Analyse einbezogen.

Wenn ein Testfall sich aus den gültigen ÄK/GW speist, spricht man auch von einem "Positiv-Test"; wenn ungültige ÄK/GW genutzt werden, spricht man auch von einem "Negativ-Test".

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

4.2.1. Hands-On (10 Minuten): Wieviel und was muss man testen?

```
1 public class Studi {
2     private int credits = 0;
3
4     public void addToCredits(int credits) {
5         if (credits < 0) {
6             throw new IllegalArgumentException("Negative Credits!");
7         }
8         if (this.credits + credits > 210) {
9             throw new IllegalArgumentException("Mehr als 210 Credits!");
10        }
11        this.credits += credits;
12    }
13 }
```

4.2.1.1. JEDE Methode mindestens testen mit/auf:

- Positive Tests: Gutfall (Normalfall) => "gültige ÄK/GW"
- Negativ-Tests (Fehlbedienung, ungültige Werte) => "ungültige ÄK/GW"
- Rand- bzw. Extremwerte => GW
- Exceptions

=> Anforderungen abgedeckt (Black-Box)?

=> Wichtige Pfade im Code abgedeckt (White-Box)?

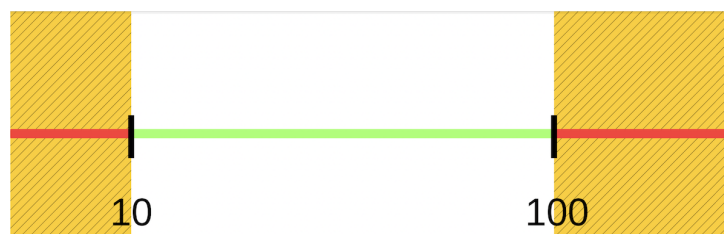
4.2.1.2. Praxis

- Je kritischer eine Klasse/Methode/Artefakt ist, um so intensiver testen!
- Suche nach Kompromissen: Testkosten vs. Kosten von Folgefehlern; beispielsweise kein Test generierter Methoden

=> "Erzeugen" der Testfälle über die Äquivalenzklassenbildung und Grenzwertanalyse (siehe nächste Folien). Mehr dann später im Wahlfach "Softwarequalität" ...

4.2.2. Äquivalenzklassenbildung

Beispiel: Zu testende Methode mit Eingabewert x , der zw. 10 und 100 liegen soll



- Zerlegung der Definitionsbereiche in Äquivalenzklassen (ÄK):
 - Disjunkte Teilmengen, wobei
 - Werte *einer* ÄK führen zu *gleichartigem* Verhalten
- Annahme: Eingabeparameter sind untereinander unabhängig
- Unterscheidung gültige und ungültige ÄK

4.2.2.1. Bemerkungen

Hintergrund: Da die Werte einer ÄK zu gleichartigem Verhalten führen, ist es egal, *welchen* Wert man aus einer ÄK für den Test nimmt.

Formal hat man *eine* ungültige ÄK (d.h. die Menge aller ungültigen Werte). In der Programmierpraxis macht es aber einen Unterschied, ob es sich um Werte unterhalb oder oberhalb des erlaubten Wertebereichs handelt (Fallunterscheidung). Beispiel: Eine Funktion soll Werte zwischen 10 und 100 verarbeiten. Dann sind alle Werte kleiner 10 oder größer 100 mathematisch gesehen in der selben ÄK “ungültig”. Praktisch macht es aber Sinn, eine ungültige ÄK für “kleiner 10” und eine weitere ungültige ÄK für “größer 100” zu betrachten ...

Traditionell betrachtet man nur die Eingabeparameter. Es kann aber Sinn machen, auch die Ausgabeseite zu berücksichtigen (ist aber u.U. nur schwierig zu realisieren).

4.2.2.2. Faustregeln bei der Bildung von ÄK

- Falls eine Beschränkung einen Wertebereich spezifiziert: Aufteilung in eine gültige und zwei ungültige ÄK

Beispiel: Eingabewert x soll zw. 10 und 100 liegen

- Gültige ÄK: $[10, 100]$
- Ungültige ÄKs: $x < 10$ und $100 < x$

- Falls eine Beschränkung eine minimale und maximale Anzahl von Werten spezifiziert: Aufteilung in eine gültige und zwei ungültige ÄK

Beispiel: Jeder Studi muss pro Semester an mindestens einer LV teilnehmen, maximal sind 5 LVs erlaubt.

- Gültige ÄK: $1 \leq x \leq 5$
- Ungültige ÄKs: $x = 0$ (keine Teilnahme) und $5 < x$ (mehr als 5 Kurse)

- Falls eine Beschränkung eine Menge von Werten spezifiziert, die möglicherweise unterschiedlich behandelt werden: Für jeden Wert dieser Menge eine eigene gültige ÄK erstellen und zusätzlich insgesamt eine ungültige ÄK

Beispiel: Das Hotel am Urlaubsort ermöglicht verschiedene Freizeitaktivitäten: Segway-fahren, Tauchen, Tennis, Golf

- Gültige ÄKs:
 - * Segway-fahren
 - * Tauchen
 - * Tennis
 - * Golf
- Ungültige ÄK: “alles andere”

- Falls eine Beschränkung eine Situation spezifiziert, die zwingend erfüllt sein muss: Aufteilung in eine gültige und eine ungültige ÄK

Hinweis: Werden Werte einer ÄK vermutlich nicht gleichwertig behandelt, dann erfolgt die Aufspaltung der ÄK in kleinere ÄKs. Das ist im Grunde die analoge Überlegung zu mehreren ungültigen ÄKs.

ÄKs sollten für die weitere Arbeit einheitlich und eindeutig benannt werden. Typisches Namensschema: “gÄKn” und “uÄKn” für gültige bzw. ungültige ÄKs mit der laufenden Nummer n .

4.2.3. ÄK: Erstellung der Testfälle

- Jede ÄK durch *mindestens einen TF* abdecken
- Dabei pro Testfall
 - mehrere *gültige* ÄKs kombinieren, oder
 - genau *eine ungültige* ÄK untersuchen (restl. Werte aus gültigen ÄK auffüllen; diese gelten dann aber nicht als getestet!)

Im Prinzip muss man zur Erstellung der Testfälle (TF) eine paarweise vollständige Kombination über die ÄK bilden, d.h. jede ÄK kommt mit jeder anderen ÄK in einem TF zur Ausführung.

Erinnerung: Annahme: Eingabeparameter sind untereinander unabhängig! => Es reicht, wenn *jede* gültige ÄK *einmal* in einem TF zur Ausführung kommt. => Kombination verschiedener gültiger ÄK in *einem* TF.

Achtung: Dies gilt nur für die **gültigen** ÄK! Bei den ungültigen ÄKs dürfen diese nicht miteinander in einem TF kombiniert werden! Bei gleichzeitiger Behandlung verschiedener ungültiger ÄK bleiben u.U. Fehler unentdeckt, da sich die Wirkungen der ungültigen ÄK überlagern!

Für jeden Testfall (TF) wird aus den zu kombinierenden ÄK ein zufälliger Repräsentant ausgewählt.

4.2.4. ÄK: Beispiel: Eingabewert x soll zw. 10 und 100 liegen

4.2.4.1. Äquivalenzklassen

Eingabe	gültige ÄK	ungültige ÄK
x	gÄK1: $[10, 100]$	uÄK2: $x < 10$ uÄK3: $100 < x$

4.2.4.2. Tests

Testnummer	1	2	3
geprüfte ÄK	gÄK1	uÄK2	uÄK3
x	42	7	120
Erwartetes Ergebnis	OK	Exception	Exception

4.2.5. Grenzwertanalyse



Beobachtung: Grenzen in Verzweigungen/Schleifen kritisch

- Grenzen der ÄK (kleinste und größte Werte) **zusätzlich** testen
 - “gültige Grenzwerte” (*gGW*): Grenzwerte von gültigen ÄK
 - “ungültige Grenzwerte” (*uGW*): Grenzwerte von ungültigen ÄK

Zusätzlich sinnvoll: Weitere grenznahe Werte, d.h. weitere Werte “rechts” und “links” der Grenze nutzen.

Bildung der Testfälle:

- Jeder GW muss in mind. einem TF vorkommen

Pro TF darf ein GW (gültig oder ungültig) verwendet werden, die restlichen Parameter werden (mit zufälligen Werten) aus gültigen ÄK aufgefüllt, um mögliche Grenzwertprobleme nicht zu überlagern.

4.2.6. GW: Beispiel: Eingabewert x soll zw. 10 und 100 liegen

4.2.6.1. Äquivalenzklassen

Eingabe	gültige ÄK	ungültige ÄK
x	$g\ddot{A}K1: [10, 100]$	$u\ddot{A}K2: x < 10$ $u\ddot{A}K3: 100 < x$

4.2.6.2. Grenzwertanalyse

Zusätzliche Testdaten: 9 ($u\ddot{A}K2o$) und 10 ($g\ddot{A}K1u$) sowie 100 ($g\ddot{A}K1o$) und 101 ($u\ddot{A}K3u$)

4.2.6.3. Tests

Testnummer	4	5	6	7
geprüfter GW	gÄK1u	gÄK1o	uÄK2o	uÄK3u
x	10	100	9	101
Erwartetes Ergebnis	OK	OK	Exception	Exception

Hinweis: Die Ergebnisse der GW-Analyse werden **zusätzlich** zu den Werten aus der ÄK-Analyse eingesetzt. Für das obige Beispiel würde man also folgende Tests aus der kombinierten ÄK- und GW-Analyse erhalten:

Testnummer	1	2	3	4	5	6	7
geprüfte(r) ÄK/GW	gÄK1	uÄK2	uÄK3	gÄK1u	gÄK1o	uÄK2o	uÄK3u
x	42	7	120	10	100	9	101
Erwartetes Ergebnis	OK	Exception	Exception	OK	OK	Exception	Exception

4.2.7. Anmerkung: Analyse abhängiger Parameter

Wenn das Ergebnis von der Kombination der Eingabewerte abhängt, dann sollte man dies bei der Äquivalenzklassenbildung berücksichtigen: Die ÄK sind in diesem Fall in Bezug auf die Kombinationen zu bilden!

Schauen Sie sich dazu das Beispiel im Kleuker (2026), Abschnitt “4.3 Analyse abhängiger Parameter” an.

Die einfache ÄK-Bildung würde in diesem Fall versagen, da die Eingabewerte nicht unabhängig sind. Leider ist die Betrachtung der möglichen Kombinationen u.U. eine sehr komplexe Aufgabe ...

Analoge Überlegungen gelten auch für die ÄK-Bildung im Zusammenhang mit objektorientierter Programmierung. Die Eingabewerte und der Objektzustand müssen dann *gemeinsam* bei der ÄK-Bildung betrachtet werden!

Vergleiche Kleuker (2026), Abschnitt “4.4 Äquivalenzklassen und Objektorientierung”.

4.2.8. Wrap-Up

- Gründliches Testen ist ebenso viel Aufwand wie Coden
- **Äquivalenzklassenbildung:**
 - Eingabebereiche (inkl. Objektzustand) in disjunkte gültige und ungültige Äquivalenzklassen zerlegen
 - Alle Eingabewerte einer ÄK werden vom Testobjekt gleich behandelt und führen zum gleichen Verhalten/Ergebnis
- **Grenzwertanalyse:** Für jede ÄK untere/obere Grenze und angrenzende ungültige Werte zusätzlich testen (gültige/ungültige Grenzwerte)

• Testfallstrategie:

- **Positiv-Tests:** Kombination mehrerer gültiger ÄK in einem Testfall erlaubt
- **Negativ-Tests:** Pro Testfall **genau eine ungültige ÄK**, alle anderen Parameter aus gültigen ÄK - diese gelten dabei noch nicht als getestet

Zum Nachlesen

Zum Nachlesen möchte ich Ihnen Kapitel 4 “Testfallerstellung mit Äquivalenzklassen” im Buch meines Kollegen Stephan Kleuker ans Herz legen: Kleuker (2026).

Lernziele

- k2: Ich kann Merkmale schlecht testbaren Codes erklären
- k2: Ich kann Merkmale guter Unit-Tests erklären
- k3: Ich kann Testfällen mittels Äquivalenzklassenbildung und Grenzwertanalyse erstellen

Challenges**ÄK/GW: RSV Flotte Speiche**

Der RSV Flotte Speiche hat in seiner Mitgliederverwaltung (**MitgliederVerwaltung**) die Methode **testBeitritt** implementiert. Mit dieser Methode wird geprüft, ob neue Mitglieder in den Radsportverein aufgenommen werden können.

```
1 public class MitgliederVerwaltung {
2
3     /**
4      * Testet, ob ein Mitglied in den Verein aufgenommen werden kann.
5      *
6      * <p>Interessierte Personen müssen mindestens 16 Jahre alt sein, um
7      * aufgenommen
8      * werden zu können. Die Motivation darf nicht zu niedrig und auch
9      * nicht zu hoch
10     * sein und muss zwischen 4 und 7 (inklusive) liegen, sonst wird der
11     * Antrag
12     * abgelehnt.
13     *
14     * <p>Der Wertebereich beim Alter umfasst die natürlichen Zahlen
15     * zwischen 0 und 99
16     * (inklusive), bei der Motivation sind die natürlichen Zahlen zwischen
17     * 0 und 10
18     * (inklusive) erlaubt.
19     *
20     * <p>Bei Verletzung der zulässigen Wertebereiche der Parameter wird
21     * eine
22     * <code>IllegalArgumentException</code> geworfen.
23     *
24     * @param alter      Alter in Lebensjahren, Bereich [0, 99]
25     * @param motivation  Motivation auf einer Scala von 0 bis 10
26     * @return <code>true</code>, wenn das Mitglied aufgenommen werden kann
27     *         ,
28     *         sonst <code>false</code>
```

```

22      * @throws <code>IllegalArgumentException</code>, wenn Parameter auß
      erhalb
23      *
      *                               der zulässigen
      Wertebereiche
24      */
25      public boolean testBeitritt(int alter, int motivation) {
26          // Implementierung versteckt
27      }
28  }

```

1. Führen Sie eine Äquivalenzklassenbildung durch und geben Sie die gefundenen Äquivalenzklassen (ÄK) an: laufende Nummer, Definition (Wertebereiche o.ä.), kurze Beschreibung (gültige/ungültige ÄK, Bedeutung).
2. Führen Sie zusätzlich eine Grenzwertanalyse durch und geben Sie die jeweiligen Grenzwerte (GW) an.
3. Erstellen Sie aus den ÄK und GW wie in der Vorlesung diskutiert Testfälle. Geben Sie pro Testfall (TF) an, welche ÄK und/oder GW abgedeckt sind, welche Eingaben Sie vorsehen und welche Ausgabe Sie erwarten.

Hinweis: Erstellen Sie separate (zusätzliche) TF für die GW, d.h. integrieren Sie diese *nicht* in die ÄK-TF.

4. Implementieren Sie die Testfälle in JUnit (JUnit 4 oder 5).
 - Fassen Sie die Testfälle der gültigen ÄK in einem parametrisierten Test zusammen.
 - Für die ungültigen ÄKs erstellen Sie jeweils eine eigene JUnit-Testmethode. Beachten Sie, dass Sie auch die Exceptions testen müssen.

ÄK/GW: LSF

Das LSF bestimmt mit der Methode `LSF#checkStudentCPS`, ob ein Studierender bereits zur Bachelorarbeit oder Praxisphase zugelassen werden kann:

```

1  class LSF {
2      public static Status checkStudentCPS(Student student) {
3          if (student.credits() >= Status.BACHELOR.credits) return Status.
              BACHELOR;
4          else if (student.credits() >= Status.PRAXIS.credits) return Status.
              PRAXIS;
5          else return Status.NONE;
6      }
7  }
8
9  record Student(String name, int credits, int semester) { }
10
11  enum Status {
12      NONE(0), PRAXIS(110), BACHELOR(190); // min: 0, max: 210
13
14      public final int credits;
15      Status(int credits) { this.credits = credits; }
16  }

```

1. Führen Sie eine Äquivalenzklassenbildung für die Methode `LSF#checkStudentCPS` durch.
2. Führen Sie zusätzlich eine Grenzwertanalyse für die Methode `LSF#checkStudentCPS` durch.
3. Erstellen Sie aus den ÄK und GW wie in der Vorlesung diskutiert Testfälle.
4. Implementieren Sie die Testfälle in JUnit (JUnit 4 oder 5).
 - Fassen Sie die Testfälle der gültigen ÄK in einem parametrisierten Test zusammen.
 - Für die ungültigen ÄKs erstellen Sie jeweils eine eigene JUnit-Testmethode. Beachten Sie, dass Sie auch die Exceptions testen müssen.

4.3. JUnit3: Mocking mit Mockito

TL;DR

Häufig hat man es in Softwaretests mit dem Problem zu tun, dass die zu testenden Klassen von anderen, noch nicht implementierten Klassen oder von zufälligen oder langsamen Operationen abhängen. In solchen Situationen kann man auf "Platzhalter" für diese Abhängigkeiten zurückgreifen. Dies können einfache Stubs sein, also Objekte, die einfach einen festen Wert bei einem Methodenaufruf zurückliefern oder Mocks, wo man auf die Argumente eines Methodenaufrufs reagieren kann und passende unterschiedliche Rückgabewerte zurückgeben kann.

Mockito ist eine Java-Bibliothek, die zusammen mit JUnit das Mocking von Klassen in Java erlaubt. Man kann hier zusätzlich auch die Interaktion mit dem gemockten Objekt überprüfen und testen, ob eine bestimmte Methode mit bestimmten Argumenten aufgerufen wurde und wie oft.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo:

- Demo Mocking: Stubs [\[YT\]](#), [\[HSBI\]](#)
- Demo Mocking: Mocks [\[YT\]](#), [\[HSBI\]](#)
- Demo Mocking: Spy [\[YT\]](#), [\[HSBI\]](#)
- Demo Mocking: `verify()` [\[YT\]](#), [\[HSBI\]](#)
- Demo Mocking: Matcher [\[YT\]](#), [\[HSBI\]](#)

4.3.1. Motivation: Entwicklung einer Studi-/Prüfungsverwaltung

4.3.1.1. Szenario

Zwei Teams entwickeln eine neue Studi-/Prüfungsverwaltung für die Hochschule. Ein Team modelliert dabei die Studierenden, ein anderes Team modelliert die Prüfungsverwaltung LSF.

- Team A:

```
1 public class Studi {
```

```
2    String name; LSF lsf;
3
4    public Studi(String name, LSF lsf) {
5        this.name = name; this.lsf = lsf;
6    }
7
8    public boolean anmelden(String modul) { return lsf.anmelden(name, modul); }
9    public boolean einsicht(String modul) { return lsf.ergebnis(name, modul) > 50; }
10 }
```

- Team B:

```
1 public class LSF {
2     public boolean anmelden(String name, String modul) { throw new
        UnsupportedOperationException(); }
3     public int ergebnis(String name, String modul) { throw new
        UnsupportedOperationException(); }
4 }
```

Team B kommt nicht so recht vorwärts, Team A ist fertig und will schon testen.

Wie kann Team A seinen Code testen?

Optionen:

- Gar nicht testen?!
- Das LSF selbst implementieren? Wer pflegt das dann? => manuell implementierte Stubs
- Das LSF durch einen Mock ersetzen => Einsatz der Bibliothek "mockito"

4.3.1.2. Motivation Mocking und Mockito

[Mockito](#) ist ein Mocking-Framework für JUnit. Es simuliert das Verhalten eines realen Objektes oder einer realen Methode.

Wofür brauchen wir denn jetzt so ein Mocking-Framework überhaupt?

Wir wollen die Funktionalität einer Klasse isoliert vom Rest testen können. Dabei stören uns aber bisher so ein paar Dinge:

- Arbeiten mit den echten Objekten ist langsam (zum Beispiel aufgrund von Datenbankzugriffen)
- Objekte beinhalten oft komplexe Abhängigkeiten, die in Tests schwer abzudecken sind
- Manchmal existiert der zu testende Teil einer Applikation auch noch gar nicht, sondern es gibt nur die Interfaces.
- Oder es gibt unschöne Seiteneffekte beim Arbeiten mit den realen Objekten. Zum Beispiel könnte es sein, dass immer eine E-Mail versendet wird, wenn wir mit einem Objekt interagieren.

In solchen Situationen wollen wir eine Möglichkeit haben, das Verhalten eines realen Objektes bzw. der Methoden zu simulieren, ohne dabei die originalen Methoden aufrufen zu müssen. (Manchmal möchte man das dennoch, aber dazu später mehr...)

Und genau hier kommt Mockito ins Spiel. Mockito hilft uns dabei, uns von den externen Abhängigkeiten zu lösen, indem es sogenannte Mocks, Stubs oder Spies anbietet, mit denen sich das Verhalten der realen Objekte simulieren/überwachen und testen lässt.

4.3.1.3. Aber was genau ist denn jetzt eigentlich Mocking?

Ein Mock-Objekt (“etwas vortäuschen”) ist im Software-Test ein Objekt, das als Platzhalter (Attrappe) für das echte Objekt verwendet wird.

Mocks sind in JUnit-Tests immer dann nützlich, wenn man externe Abhängigkeiten hat, auf die der eigene Code zugreift. Das können zum Beispiel externe APIs sein oder Datenbanken etc. ... Mocks helfen einem beim Testen nun dabei, sich von diesen externen Abhängigkeiten zu lösen und seine Softwarefunktionalität dennoch schnell und effizient testen zu können ohne evtl. auftretende Verbindungsfehler oder andere mögliche Seiteneffekte der externen Abhängigkeiten auszulösen.

Dabei simulieren Mocks die Funktionalität der externen APIs oder Datenbankzugriffe. Auf diese Weise ist es möglich Softwaretests zu schreiben, die scheinbar die gleichen Methoden aufrufen, die sie auch im regulären Softwarebetrieb nutzen würden, allerdings werden diese wie oben erwähnt allerdings für die Tests nur simuliert.

Mocking ist also eine Technik, die in Softwaretests verwendet wird, in denen die gemockten Objekte anstatt der realen Objekte zu Testzwecken genutzt werden. Die gemockten Objekte liefern dabei bei einem vom Programmierer bestimmten (Dummy-) Input, einen dazu passenden gelieferten (Dummy-) Output, der durch seine vorhersagbare Funktionalität dann in den eigentlichen Testobjekten gut für den Test nutzbar ist.

Dabei ist es von Vorteil die drei Grundbegriffe “Mock”, “Stub” oder “Spy”, auf die wir in der Vorlesung noch häufiger treffen werden, voneinander abgrenzen und unterscheiden zu können.

4.3.1.4. Dabei bezeichnet ein

- **Stub:** Ein Stub ist ein Objekt, dessen Methoden nur mit einer minimalen Logik für den Test implementiert wurden. Häufig werden dabei einfach feste (konstante) Werte zurückgeliefert, d.h. beim Aufruf einer Methode wird unabhängig von der konkreten Eingabe immer die selbe Ausgabe zurückgeliefert.
- **Mock:** Ein Mock ist ein Objekt, welches im Gegensatz zum Stub bei vorher definierten Funktionsaufrufen mit vorher definierten Argumente eine definierte Rückgabe liefert.
- **Spy:** Ein Spy ist ein Objekt, welches Aufrufe und übergebene Werte protokolliert und abfragbar macht. Es ist also eine Art Wrapper um einen Stub oder einen Mock.

4.3.1.5. Mockito Setup

- Gradle: `build.gradle`

```
1 dependencies {
2     testImplementation platform('org.junit:junit-bom:6.0.3')
3     testImplementation 'org.junit.jupiter:junit-jupiter'
4     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
```

```

5
6     testImplementation 'org.mockito:mockito-core:5.23.0'
7 }

```

- Maven: pom.xml

```

1  <dependencyManagement>
2      <dependencies>
3          <dependency>
4              <groupId>org.junit</groupId>
5              <artifactId>junit-bom</artifactId>
6              <version>6.0.3</version>
7              <type>pom</type>
8              <scope>import</scope>
9          </dependency>
10     </dependencies>
11 </dependencyManagement>
12
13 <dependencies>
14     <dependency>
15         <groupId>org.junit.jupiter</groupId>
16         <artifactId>junit-jupiter</artifactId>
17         <scope>test</scope>
18     </dependency>
19     <dependency>
20         <groupId>org.mockito</groupId>
21         <artifactId>mockito-core</artifactId>
22         <version>5.23.0</version>
23         <scope>test</scope>
24     </dependency>
25 </dependencies>

```

Seit Java 21 werden bestimmte [Einschränkungen](#) auf Seiten der JVM implementiert, die aktuell zu einer Warnung beim Ausführen von Mockito führen und die in Zukunft möglicherweise das direkte Ausführen von Mockito komplett verhindern.

In der [Dokumentation von Mockito](#) findet sich ein Hinweis, dass man `mockito-core` explizit als einen “Java Agent” konfigurieren sollte. Damit landet man bei der folgenden Ergänzung der Gradle-Konfiguration für Mockito (vgl. mit dem `dependencies`-Abschnitt oben):

```

1  configurations {
2      mockitoAgent
3  }
4
5  dependencies {
6      testImplementation platform('org.junit:junit-bom:6.0.3')
7      testImplementation 'org.junit.jupiter:junit-jupiter'
8      testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
9
10     testImplementation 'org.mockito:mockito-core:5.23.0'
11     mockitoAgent('org.mockito:mockito-core:5.23.0') { transitive = false }
12 }
13
14 test {
15     useJUnitPlatform()

```

```

16     jvmArgs "-javaagent:${configurations.mockitoAgent.asPath}"
17 }

```

4.3.2. Manuell Stubs implementieren

Team A könnte manuell das LSF rudimentär implementieren (nur für die Tests, einfach mit festen Rückgabewerten): **Stubs**

```

1  public class StudiStubTest {
2      Studi studi; LSF lsf;
3
4      @Before
5      public void setUp() { lsf = new LsfStub(); studi = new Studi("Harald", lsf); }
6
7      @Test
8      public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
9      @Test
10     public void testEinsicht() { assertTrue(studi.einsicht("PM-Dungeon")); }
11
12
13     // Stub für das noch nicht fertige LSF
14     class LsfStub extends LSF {
15         public boolean anmelden(String name, String modul) { return true; }
16         public int ergebnis(String name, String modul) { return 80; }
17     }
18 }

```

Problem: Wartung der Tests (wenn das richtige LSF fertig ist) und Wartung der Stubs (wenn sich die Schnittstelle des LSF ändert, muss auch der Stub nachgezogen werden).

Problem: Der Stub hat nur eine Art minimale Default-Logik (sonst könnte man ja das LSF gleich selbst implementieren). Wenn man im Test andere Antworten braucht, müsste man einen weiteren Stub anlegen ...

[Demo hsbi.StudiStubTest](#)

4.3.3. Mockito: Mocking von ganzen Klassen

Lösung: Mocking der Klasse `LSF` mit Mockito für den Test von `Studi`: `mock()`.

```

1  public class StudiMockTest {
2      Studi studi; LSF lsf;
3
4      @Before
5      public void setUp() { lsf = mock(LSF.class); studi = new Studi("Harald", lsf); }
6
7      @Test
8      public void testAnmelden() {
9          when(lsf.anmelden(anyString(), anyString())).thenReturn(true);
10         assertTrue(studi.anmelden("PM-Dungeon"));
11     }

```

```

12
13     @Test
14     public void testEinsichtI() {
15         when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80);
16         assertTrue(studi.einsicht("PM-Dungeon"));
17     }
18     @Test
19     public void testEinsichtII() {
20         when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(40);
21         assertFalse(studi.einsicht("PM-Dungeon"));
22     }
23 }

```

Der Aufruf `mock(LSF.class)` erzeugt einen Mock der Klasse (oder des Interfaces) `LSF`. Dabei wird ein Objekt vom Typ `LSF` erzeugt, mit dem man dann wie mit einem normalen Objekt weiter arbeiten kann. Die Methoden sind allerdings nicht implementiert...

Mit Hilfe von `when().thenReturn()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Mock passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man dann für bestimmte Argumentwerte andere Rückgabewerte definieren. `when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80)` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Mock `lsf` den Wert 80 zurück.

Dies kann man in weiten Grenzen flexibel anpassen.

Mit Hilfe der Argument-Matcher `anyString()` wird jedes String-Argument akzeptiert.

[Demo hsbi.StudiMockTest](#)

4.3.4. Mockito: Spy = Wrapper um ein Objekt

Team B hat das `LSF` nun implementiert und Team A kann es endlich für die Tests benutzen. Aber das `LSF` hat eine Zufallskomponente (`ergebnis()`). Wie kann man nun die Reaktion des Studis testen (`einsicht()`)?

Lösung: Mockito-Spy als partieller Mock einer Klasse (Wrapper um ein Objekt): `spy()`.

```

1  public class StudiSpyTest {
2      Studi studi; LSF lsf;
3
4      @Before
5      public void setUp() { lsf = spy(LSF.class); studi = new Studi("Harald",
6                          lsf); }
7
8      @Test
9      public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
10
11     @Test
12     public void testEinsichtI() {
13         doReturn(80).when(lsf).ergebnis("Harald", "PM-Dungeon");
14         assertTrue(studi.einsicht("PM-Dungeon"));
15     }
16     @Test
17     public void testEinsichtII() {

```

```
17         doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon");
18         assertFalse(studi.einsicht("PM-Dungeon"));
19     }
20 }
```

Der Aufruf `spy(LSF.class)` erzeugt einen Spy um ein Objekt der Klasse `LSF`. Dabei bleiben zunächst die Methoden in `LSF` erhalten und können aufgerufen werden, sie können aber auch mit einem (partiellen) Mock überlagert werden. Der Spy zeichnet wie der Mock die Interaktion mit dem Objekt auf.

Mit Hilfe von `doReturn().when()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Spy passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man analog zum Mock für bestimmte Argumentwerte andere Rückgabewerte definieren. `doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon")` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Spy `lsf` den Wert 40 zurück.

Wenn man die Methoden nicht mit einem partiellen Mock überschreibt, dann wird einfach die originale Methode aufgerufen (Beispiel: In `studi.anmelden("PM-Dungeon")` wird `lsf.anmelden("Harald", "PM-Dungeon")` aufgerufen.).

Auch hier können Argument-Matcher wie `anyString()` eingesetzt werden.

[Demo hsbi.StudiSpyTest](#)

4.3.5. Wurde eine Methode aufgerufen?

```
1 public class VerifyTest {
2     @Test
3     public void testAnmelden() {
4         LSF lsf = mock(LSF.class); Studi studi = new Studi("Harald", lsf);
5
6         when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);
7
8         assertTrue(studi.anmelden("PM-Dungeon"));
9
10
11         verify(lsf).anmelden("Harald", "PM-Dungeon");
12         verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");
13
14         verify(lsf, atLeast(1)).anmelden("Harald", "PM-Dungeon");
15         verify(lsf, atMost(1)).anmelden("Harald", "PM-Dungeon");
16
17         verify(lsf, never()).ergebnis("Harald", "PM-Dungeon");
18
19         verifyNoMoreInteractions(lsf);
20     }
21 }
```

Mit der Methode `verify()` kann auf einem Mock oder Spy überprüft werden, ob und wie oft und in welcher Reihenfolge Methoden aufgerufen wurden und mit welchen Argumenten. Auch hier lassen sich wieder Argument-Matcher wie `anyString()` einsetzen.

Ein einfaches `verify(mock)` prüft dabei, ob die entsprechende Methode exakt einmal vorher aufgerufen

wurde. Dies ist äquivalent zu `verify(mock, times(1))`. Analog kann man mit den Parametern `atLeast()` oder `atMost` bestimmte Unter- oder Obergrenzen für die Aufrufe angeben und mit `never()` prüfen, ob es gar keinen Aufruf vorher gab.

`verifyNoMoreInteractions(lsf)` ist interessant: Es ist genau dann **true**, wenn es außer den vorher abgefragten Interaktionen keinerlei sonstigen Interaktionen mit dem Mock oder Spy gab.

```

1 LSF lsf = mock(LSF.class);
2 Studi studi = new Studi("Harald", lsf);
3
4 when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);
5
6 InOrder inOrder = inOrder(lsf);
7
8 assertTrue(studi.anmelden("PM-Dungeon"));
9 studi.anmelden("Wuppie");
10
11 inOrder.verify(lsf).anmelden("Harald", "Wuppie");
12 inOrder.verify(lsf).anmelden("Harald", "PM-Dungeon");

```

Mit `InOrder` lassen sich Aufrufe auf einem Mock/Spy oder auch auf verschiedenen Mocks/Spies in eine zeitliche Reihenfolge bringen und so überprüfen.

[Demo hsbi.VerifyTest](#)

4.3.6. Fangen von Argumenten

```

1 public class MatcherTest {
2     @Test
3     public void testAnmelden() {
4         LSF lsf = mock(LSF.class); Studi studi = new Studi("Harald", lsf);
5
6         when(lsf.anmelden(anyString(), anyString())).thenReturn(false);
7         when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);
8
9         assertTrue(studi.anmelden("PM-Dungeon"));
10        assertFalse(studi.anmelden("Wuppie?"));
11
12        verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");
13        verify(lsf, times(1)).anmelden("Harald", "Wuppie?");
14
15        verify(lsf, times(2)).anmelden(anyString(), anyString());
16        verify(lsf, times(1)).anmelden(eq("Harald"), eq("Wuppie?"));
17        verify(lsf, times(2)).anmelden(argThat(new MyHaraldMatcher()),
18            anyString());
19    }
20
21    class MyHaraldMatcher implements ArgumentMatcher<String> {
22        public boolean matches(String s) { return s.equals("Harald"); }
23    }
24 }

```

Sie können die konkreten Argumente angeben, für die der Aufruf gelten soll. Alternativ können Sie mit vordefinierten `ArgumentMatchers` wie `anyString()` beispielsweise auf beliebige Strings reagieren oder selbst einen eigenen `ArgumentMatcher<T>` für Ihren Typ `T` erstellen und nutzen.

Wichtig: Wenn Sie für einen Parameter einen `ArgumentMatcher` einsetzen, müssen Sie für die restlichen Parameter der Methode dies ebenfalls tun. Sie können keine konkreten Argumente mit `ArgumentMatcher` mischen.

Sie finden viele weitere vordefinierte Matcher in der Klasse `ArgumentMatchers`. Mit der Klasse `ArgumentCaptor<T>` finden Sie eine alternative Möglichkeit, auf Argumente in gemockten Methoden zu reagieren. Schauen Sie sich dazu die Javadoc von `Mockito` an.

[Demo hsbi.MatcherTest](#)

4.3.7. Ausblick: PowerMock

Mockito sehr mächtig, aber unterstützt (u.a.) keine

- Konstruktoren
- private Methoden
- final Methoden
- static Methoden (ab Version 3.4.0 scheint auch [Mockito statische Methoden](#) zu unterstützen)

=> Lösung: [PowerMock](#)

4.3.8. Ausführlicheres Beispiel: WuppiWarenlager

Credits: Der Dank für die Erstellung des nachfolgenden Beispiels und Textes geht an [@jedi101](#).

[Demo: WuppiWarenlager \(wuppie.stub\)](#)

Bei dem gezeigten Beispiel unseres `WuppiStores` sieht man, dass dieser normalerweise von einem fertigen Warenlager die Wuppis beziehen möchte. Da dieses Lager aber noch nicht existiert, haben wir uns kurzerhand einfach einen Stub von unserem `IWuppiWarenlager`-Interface erstellt, in dem wir zu Testzwecken händisch ein Paar Wuppis ins Lager geräumt haben.

Das funktioniert in diesem Mini-Testbeispiel ganz gut aber, wenn unsere Stores erst einmal so richtig Fahrt aufnehmen und wir irgendwann weltweit Wuppis verkaufen, wird der Code des `IWuppiWarenlagers` wahrscheinlich sehr schnell viel komplexer werden, was unweigerlich dann zu Maintenance-Problemen unserer händisch angelegten Tests führt. Wenn wir zum Beispiel einmal eine Methode hinzufügen wollen, die es uns ermöglicht, nicht immer alle Wuppis aus dem Lager zu ordern oder vielleicht noch andere Methoden, die Fluppis orderbar machen, hinzufügen, müssen wir immer dafür sorgen, dass wir die getätigten Änderungen händisch in den Stub des Warenlagers einpflegen.

Das will eigentlich niemand...

4.3.8.1. Einsatz von Mockito

Aber es gibt da einen Ausweg. Wenn es komplexer wird, verwenden wir Mocks.

Bislang haben wir noch keinen Gebrauch von Mockito gemacht. Das ändern wir nun.

Demo: WuppiWarenlager (wuppie.mock)

Wie in diesem Beispiel gezeigt, müssen wir nun keinen Stub mehr von Hand erstellen, sondern überlassen dies Mockito.

```
1 IWuppiWarenlager lager = mock(IWuppiWarenlager.class);
```

Anschließend können wir, ohne die Methode `getAllWuppis()` implementiert zu haben, dennoch so tun als, ob die Methode eine Funktionalität hätte.

```
1 // Erstellen eines imaginären Lagerbestands.
2 List<String> wuppisImLager = Arrays.asList("GruenerWuppi", "RoterWuppi");
3 when(lager.getAlleWuppis()).thenReturn(wuppisImLager);
```

Wann immer nun die Methode `getAlleWuppis()` des gemockten Lagers aufgerufen wird, wird dieser Aufruf von Mockito abgefangen und wie oben definiert verändert. Das Ergebnis können wir abschließend einfach in unserem Test testen:

```
1 // Erzeugen des WuppiStores.
2 WuppiStore wuppiStore = new WuppiStore(lager);
3
4 // Bestelle alle Wuppis aus dem gemockten Lager List<String>
5 bestellteWuppis = wuppiStore.bestelleAlleWuppis(lager);
6
7 // Hat die Bestellung geklappt?
8 assertEquals(2, bestellteWuppis.size());
```

4.3.8.2. Mockito Spies

Manchmal möchten wir allerdings nicht immer gleich ein ganzes Objekt mocken, aber dennoch Einfluss auf die aufgerufenen Methoden eines Objekts haben, um diese testen zu können. Vielleicht gibt es dabei ja sogar eine Möglichkeit unsere JUnit-Tests, mit denen wir normalerweise nur Rückgabewerte von Methoden testen können, zusätzlich auch das Verhalten also die Interaktionen mit einem Objekt beobachtbar zu machen. Somit wären diese Interaktionen auch testbar.

Und genau dafür bietet Mockito eine Funktion: der sogenannte "Spy".

Dieser Spion erlaubt es uns nun zusätzlich das Verhalten zu testen. Das geht in die Richtung von **BDD - Behavior Driven Development**.

Demo: WuppiWarenlager (wuppie.spy)

```
1 // Spion erstellen, der unser wuppiWarenlager überwacht.
2 this.wuppiWarenlager = spy(WuppiWarenlager.class);
```

Hier hatten wir uns einen Spion erzeugt, mit dem sich anschließend das Verhalten verändern lässt:


```
1 when(wuppiWarenlager.getAlleWuppis()).thenReturn(Arrays.asList(new Wuppi("
    Wuppi007")));
```

Aber auch der Zugriff lässt sich kontrollieren/testen:

```
1 verify(wuppiWarenlager).addWuppi(normalerWuppi);
2 verifyNoMoreInteractions(wuppiWarenlager);
```

Die normalen Testmöglichkeiten von JUnit runden unseren Test zudem ab.

```
1 assertEquals(1,wuppiWarenlager.lager.size());
```

4.3.9. Mockito und Annotationen

In Mockito können Sie wie oben gezeigt mit `mock()` und `spy()` neue Mocks bzw. Spies erzeugen und mit `verify()` die Interaktion überprüfen und mit `ArgumentMatcher<T>` bzw. den vordefinierten `ArgumentMatchers` auf Argumente zuzugreifen bzw. darauf zu reagieren.

Zusätzlich/alternativ gibt es in Mockito zahlreiche Annotationen, die **ersatzweise** statt der genannten Methoden genutzt werden können. Hier ein kleiner Überblick über die wichtigsten in Mockito verwendeten Annotation:

- `@Mock` wird zum Markieren des zu mockenden Objekts verwendet.

```
1 @Mock
2 WuppiWarenlager lager;
```

- `@RunWith(MockitoJUnitRunner.class)` ist der entsprechende JUnit-Runner, wenn Sie Mocks mit `@Mock` anlegen.

```
1 @RunWith(MockitoJUnitRunner.class)
2 public class ToDoBusinessMock {...}
```

- `@Spy` erlaubt das Erstellen von partiell gemockten Objekten. Dabei wird eine Art Wrapper um das zu mockende Objekt gewickelt, der dafür sorgt, dass alle Methodenaufrufe des Objekts an den Spy delegiert werden. Diese können über den Spion dann abgefangen/verändert oder ausgewertet werden.

```
1 @Spy
2 ArrayList<Wuppi> arrayListenSpion;
```

- `@InjectMocks` erlaubt es, Parameter zu markieren, in denen Mocks und/oder Spies injiziert werden. Mockito versucht dann (in dieser Reihenfolge) per Konstruktorinjektion, Setterinjektion oder Propertyinjektion die Mocks zu injizieren. Weitere Informationen darüber findet man hier: [Mockito Dokumentation](#)

Anmerkung: Es ist aber nicht ratsam "Field- oder Setterinjection" zu nutzen, da man nur bei der Verwendung von "Constructorinjection" sicherstellen kann, dass eine Klasse nicht ohne die eigentlich notwendigen Parameter instanziiert wurde.

```
1 @InjectMocks
2 Wuppi fluppi;
```

- `@Captor` erlaubt es, die Argumente einer Methode abzufangen/auszuwerten. Im Zusammenspiel mit Mockitos `verify()`-Methode kann man somit auch die einer Methode übergebenen Argumente verifizieren.

```
1 @Captor
2 ArgumentCaptor<String> argumentCaptor;
```

- `@ExtendWith(MockitoExtension.class)` wird in JUnit5 verwendet, um die Initialisierung von Mocks zu vereinfachen. Damit entfällt zum Beispiel die noch unter JUnit4 nötige Initialisierung der Mocks durch einen Aufruf der Methode `MockitoAnnotations.openMocks()` im Setup des Tests (`@Before` bzw. `@BeforeEach`).

4.3.9.1. Prüfen der Interaktion mit `verify()`

Mit Hilfe der umfangreichen `verify()`-Methoden, die uns Mockito mitliefert, können wir unseren Code unter anderem auf unerwünschte Seiteneffekte testen. So ist es mit `verify` zum Beispiel möglich abzufragen, ob mit einem gemockten Objekt interagiert wurde, wie damit interagiert wurde, welche Argumente dabei übergeben worden sind und in welcher Reihenfolge die Interaktionen damit erfolgt sind.

Hier nur eine kurze Übersicht über das Testen des Codes mit Hilfe von Mockitos `verify()`-Methoden.

```
1 @Test
2 public void testVerify_DasKeineInteraktionMitDerListeStattgefundenHat() {
3     // Testet, ob die spezifizierte Interaktion mit der Liste nie stattgefunden
4     // hat.
5     verify(fluppisListe, never()).clear();
6 }
```

```
1 @Test
2 public void testVerify_ReihenfolgeDerInteraktionenMitDerFluppisListe() {
3     // Testet, ob die Reihenfolge der spezifizierten Interaktionen mit der
4     // Liste eingehalten wurde.
5     fluppisListe.clear();
6     InOrder reihenfolge = inOrder(fluppisListe);
7     reihenfolge.verify(fluppisListe).add("Fluppi001");
8     reihenfolge.verify(fluppisListe).clear();
9 }
```

```
1 @Test
2 public void testVerify_FlexibleArgumenteBeimZugriffAufFluppisListe() {
3     // Testet, ob schon jemals etwas zu der Liste hinzugefügt wurde.
4     // Dabei ist es egal welcher String eingegeben wurde.
5     verify(fluppisListe).add(anyString());
6 }
```

```
1 @Test
2 public void testVerify_InteraktionenMitHilfeDesArgumentCaptor() {
3     // Testet, welches Argument beim Methodenaufruf übergeben wurde.
4     fluppisListe.addAll(Arrays.asList("BobDerBaumeister"));
5     ArgumentCaptor<List> argumentMagnet = ArgumentCaptor.forClass(FluppisListe.
6     class);
7     verify(fluppisListe).addAll(argumentMagnet.capture());
8 }
```

```
7 List<String> argumente = argumentMagnet.getValue();
8 assertEquals("BobDerBaumeister", argumente.get(0));
9 }
```

Demo: WuppiWarenlager (wuppie.verify)

4.3.10. Wrap-Up

- Gründliches Testen ist ebenso viel Aufwand wie Coden!
- Mockito ergänzt JUnit:
 - Mocken ganzer Klassen (`mock()`, `when().thenReturn()`)
 - Wrappen von Objekten (`spy()`, `doReturn().when()`)
 - Auswerten, wie häufig Methoden aufgerufen wurden (`verify()`)
 - Auswerten, mit welchen Argumenten Methoden aufgerufen wurden (`anyString`)

Zum Nachlesen

Sie finden beispielsweise in den Baelding-Tutorials "[Mockito Series](#)" weitere Informationen zum Thema Mockito.

Lernziele

- k2: Ich kann die Begriffe: Mocking, Mock, Stub, Spy unterscheiden und erklären
- k3: Ich kann Mocks in Mockito anlegen und nutzen
- k3: Ich kann Spies in Mockito anlegen und nutzen
- k3: Ich kann die Interaktion mit Mocks/Spies über `verify()` prüfen
- k3: Ich kann den `ArgumentMatcher` praktisch einsetzen

Challenges

Betrachten Sie die drei Klassen `Utility.java`, `Evil.java` und `UtilityTest.java`:

```
1 public class Utility {
2     private int intResult = 0;
3     private Evil evilClass;
4
5     public Utility(Evil evilClass) {
6         this.evilClass = evilClass;
7     }
8
9     public void evilMethod() {
10         int i = 2 / 0;
11     }
12
13     public int nonEvilAdd(int a, int b) {
14         return a + b;
15     }
16 }
```

```
17     public int evilAdd(int a, int b) {
18         evilClass.evilMethod();
19         return a + b;
20     }
21
22     public void veryEvilAdd(int a, int b) {
23         evilMethod();
24         evilClass.evilMethod();
25         intResult = a + b;
26     }
27
28     public int getIntResult() {
29         return intResult;
30     }
31 }
32
33 public class Evil {
34     public void evilMethod() {
35         int i = 3 / 0;
36     }
37 }
38
39 public class UtilityTest {
40     private Utility utilityClass;
41     // Initialisieren Sie die Attribute entsprechend vor jedem Test.
42
43     @Test
44     void test_nonEvilAdd() {
45         Assertions.assertEquals(10, utilityClass.nonEvilAdd(9, 1));
46     }
47
48     @Test
49     void test_evilAdd() {
50         Assertions.assertEquals(10, utilityClass.evilAdd(9, 1));
51     }
52
53     @Test
54     void test_veryEvilAdd() {
55         utilityClass.veryEvilAdd(9, 1);
56         Assertions.assertEquals(10, utilityClass.getIntResult());
57     }
58 }
```

Testen Sie die Methoden `nonEvilAdd`, `evilAdd` und `veryEvilAdd` der Klasse `Utility.java` mit dem `JUnit`- und dem `Mockito-Framework`.

Vervollständigen Sie dazu die Klasse `UtilityTest.java` und nutzen Sie Mocking mit `Mockito`, um die Tests zum Laufen zu bringen. Die Tests dürfen Sie entsprechend verändern, aber die Aufrufe aus der Vorgabe müssen erhalten bleiben. Die Klassen `Evil.java` und `Utility.java` dürfen Sie nicht ändern. *Hinweis:* Die Klasse `Evil.java` und die Methode `evilMethod()` aus `Utility.java` lösen eine ungewollte bzw. "zufällige" Exception aus, auf deren Auftreten jedoch *nicht* getestet werden soll. Stattdessen sollen diese Klassen bzw. Methoden mit Mockito "weggemockt" werden, so dass die vorgegebenen Testmethoden (wieder) funktionieren.

4.4. JUnit4: Property based Testing & Approval Testing

TL;DR

Approval Testing hilft Ihnen, komplexe Ausgaben wie Reports, HTML oder JSON praktikabel zu testen. Statt im Test einen riesigen `expected`-String manuell zusammenzubauen und zu pflegen, frieren Sie einmal einen geprüften Output als “approved” (in einer Textdatei) ein und lassen künftige Testläufe automatisch dagegen vergleichen. Änderungen sehen Sie als Diff im Repository und können bewusst entscheiden, ob das Verhalten gewollt angepasst wurde oder ein Fehler vorliegt - ideal auch als Sicherheitsnetz vor und nach Refactorings.

Property-Based Testing (PTB) ergänzt klassische Beispieltests, indem es allgemeine Eigenschaften (“Für alle Eingaben aus diesem Bereich muss X gelten”) über viele automatisch generierte Eingaben prüft. Die Äquivalenzklassenanalyse dient dabei als zentrales Denkwerkzeug: Sie strukturieren damit den Eingaberaum und formulieren daraus Properties für jqwik.

Während die Idee von Approval Testing sehr überschaubar ist, können wir nur einen allerersten Blick in das Thema Property-Based Testing werfen.

Videos

Vorlesung [YT], [HSBI]

4.4.1. Motivation: Mehr als nur Beispieltests

- Bisher: klassische Unit-Tests mit JUnit (`@Test`, `assertEquals`, ...)
- Problem 1: Komplexe/umfangreiche Outputs sind schwer “per Hand” zu vergleichen
- Problem 2: Einzelne Beispieltests decken nur wenige Eingaben ab
- Idee: Zwei Ergänzungen
 - Approval Testing
 - Property-Based Testing (PBT)

In klassischen Tests formulieren wir für ein paar ausgewählte Beispiele “Eingabe X -> erwartete Ausgabe Y”. Das ist gut für Dokumentation und einfache Funktionen, stößt aber an Grenzen, wenn Outputs lang/komplex sind oder wenn wir viele verschiedene Eingaben testen wollen. Approval Testing hilft beim Umgang mit komplexen Ausgaben, Property-Based Testing beim automatisierten Durchprobieren vieler Eingaben anhand allgemeiner Eigenschaften.

4.4.2. Approval Testing: Idee und Einsatzgebiete

```
1 @Test
2 void report_looks_as_expected() {
3     // given
4     List<Order> orders = List.of(...); // !!!!
```

```
5     String expected = "..."; // ???!  
6  
7     // when  
8     String report = OrderReportGenerator.generateReport(orders);  
9  
10    // then  
11    assertEquals(expected, report);  
12 }
```

- Grundidee:
 - Einmal gültigen Output als “Goldstandard” abspeichern (**approved**)
 - Zukünftig generierten Output automatisch dagegen vergleichen
- Gut geeignet für:
 - Reports, Tabellen, formatierten Text
 - HTML, JSON, Logs, Konsolen-Output
- Typischer Einsatz:
 - Refactoring ohne Verhaltensänderung absichern

Beim Approval Testing wird nicht im Test ein erwarteter String **manuell** zusammengebaut. Stattdessen **prüft man einmalig einen generierten Output**, “segnet ihn ab” und speichert ihn als “approved” in einem String oder einer Datei. Künftige Testläufe erzeugen einen “received”-Output und vergleichen ihn mit der “approved”-Version. Unterschiede fallen als Diffs auf. Besonders beim Refactoring ist das hilfreich: Wenn sich der Output nicht ändert, war das Refactoring verhaltensgleich.

4.4.3. Beispiel: Order-Report mit manuellen Approval-Tests

Produktionscode: Textreport aus Bestellungen erzeugen:

```
1 public record Order(String id, String customer, List<String> items) {}  
2  
3 public class OrderReportGenerator {  
4     public static String generateReport(List<Order> orders) {  
5         var sb = new StringBuilder();  
6         sb.append("=== ORDER REPORT ===\n");  
7         orders.forEach( o -> {  
8             sb.append("Order: ").append(o.id()).append("\n");  
9             sb.append("Customer: ").append(o.customer()).append("\n");  
10            sb.append("Items:\n");  
11            o.items().forEach(item -> sb.append(" - ").append(item).append(  
12                "\n"));  
13            sb.append("\n");  
14        });  
15        sb.append("Total orders: ").append(orders.size()).append("\n");  
16        return sb.toString();  
17    }  
18 }
```

Approval-Test:

1. Test laufen lassen und Vergleich mit leerem String -> rot
 2. Ausgabe von JUnit prüfen und als Erwartung übernehmen -> grün
-

Erster Lauf mit leerem "expected"-String:

```
1  class OrderReportGeneratorTest {
2
3      @Test
4      void report_looks_as_expected() {
5          // given
6          List<Order> orders = List.of(
7              new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
8              new Order("B-002", "Bob", List.of("Laptop")),
9              new Order("C-003", "Charlie", List.of("HDMI Cable", "USB
10                 Hub")));
11          // 1. Test absichtlich fehlschlagen lassen, 2. Ausgabe übernehmen
12          String expected = "..."; // ???!
13
14          // when
15          String report = OrderReportGenerator.generateReport(orders);
16
17          // then
18          assertEquals(expected, report);
19      }
20 }
```

Nach dem ersten Lauf Ergänzen des "expected"-Strings zu:

```
1  class OrderReportGeneratorTest {
2
3      @Test
4      void report_looks_as_expected() {
5          // given
6          List<Order> orders = List.of(
7              new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
8              new Order("B-002", "Bob", List.of("Laptop")),
9              new Order("C-003", "Charlie", List.of("HDMI Cable", "USB
10                 Hub")));
11          String expected = ""
12              === ORDER REPORT ===
13              Order: A-001
14              Customer: Alice
15              Items:
16              - Keyboard
17              - Mouse
18
19              Order: B-002
20              Customer: Bob
```

```

20         Items:
21         - Laptop
22
23         Order: C-003
24         Customer: Charlie
25         Items:
26         - HDMI Cable
27         - USB Hub
28
29         Total orders: 3
30         """;
31
32         // when
33         String report = OrderReportGenerator.generateReport(orders);
34
35         // then
36         assertEquals(expected, report);
37     }
38 }

```

4.4.4. Beispiel: Order-Report mit Bibliothek “ApprovalTests”

Java-Bibliothek [ApprovalTests.Java](#) zur Unterstützung von Approval Tests

- vergleicht gesamten Report-String mit *.approved-Datei
- kein manuelles Zusammensetzen eines langen “expected”-Strings nötig

Einbindung in Gradle:

```

1 dependencies {
2     testImplementation 'com.approvaltests:approvaltests:30.1.1'
3 }

```

Assert im Test über den Aufruf `Approvals.verify(actual);`:

- Legt die Ausgabe `actual` in einer Datei im Build-Ordner an: `<TESTCLASS>.<TESTMETHOD>.received.txt`
- Inhalt einmalig manuell prüfen
- Wenn ok: Datei verschieben in Testordner (parallel zur Test-Klasse): `<TESTCLASS>.<TESTMETHOD>.approved.txt`
- Diese “*.approved.txt”-Datei mit in Git einchecken
- Folgende Testläufe vergleichen Output mit der “*.approved.txt”-Datei

```

1 class OrderReportGeneratorTest {
2
3     @Test
4     void report_looks_as_expected() {
5         // given
6         List<Order> orders = List.of(
7             new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
8             new Order("B-002", "Bob", List.of("Laptop")),

```



```
9         new Order("C-003", "Charlie", List.of("HDMI Cable", "USB
10             Hub")));
11     // when
12     String report = OrderReportGenerator.generateReport(orders);
13
14     // then
15     // ApprovalTests.Java: vergleicht gegen <TESTCLASS>.<TESTMETHOD>.
16     // approved.txt
17     Approvals.verify(report);
18 }
```

Der Approval-Test ruft die normale Produktionsmethode auf, erzeugt einen kompletten Report und übergibt diesen an `Approvals.verify(report)`. Beim ersten Lauf wird eine “received”-Datei erzeugt, das Sie manuell überprüfen und (falls ok) als “approved”-Datei übernehmen. Die Namenskonvention ist `<TESTCLASS>.<TESTMETHOD>.approved.txt` und muss strikt eingehalten werden. Die “approved”-Datei muss “neben” der Test-Klasse liegen, also im selben Ordner. Spätere Testläufe vergleichen dann den aktuellen Output mit der “approved”-Version; bei Änderungen zeigt ein Diff genau, was sich im Verhalten geändert hat. So müssen Sie keinen langen `expected`-String im Test pflegen.

Tip

Beachten Sie die Ausgabe in der Konsole bei der Ausführung des Tests - dort steht, wo Sie die zu prüfende Ausgabedatei `<TESTCLASS>.<TESTMETHOD>.received.txt` finden!

4.4.5. Approval Testing & Git

- Approved-Dateien werden im Repository versioniert
- Diffs der Approved-Files zeigen Verhaltensänderungen im Code-Review
- Workflow bei legitimer Änderung:
 - Test schlägt fehl -> Diff prüfen -> neue Version als “approved” übernehmen

Da die “approved”-Ausgabedateien im Versionskontrollsystem liegen, können Sie bei jedem Pull-Request sehen, wie genau sich der Output geändert hat. Das ist besonders wertvoll, wenn formatiertes Text- oder HTML-Output betroffen ist. Wenn sich eine Änderung als gewollt herausstellt, aktualisieren Sie einfach die Approved-Datei und dokumentieren damit die neue, korrekte Ausgabe.

4.4.6. Property-Based Testing: Grundidee

- Klassischer Test:
 - “Für Eingabe X erwarte ich Ergebnis Y”
- Property-Based Test:
 - “Für alle Eingaben mit Eigenschaft E muss Eigenschaft P gelten”
- Ein Framework (z.B. jqwik) generiert viele Eingaben automatisch

- Fokus: Allgemeine Eigenschaften / Invarianten statt einzelner Beispiele

Beim Property-Based Testing überlegen Sie nicht mehr nur konkrete Beispielwerte, sondern formulieren allgemeine Gesetze. Zum Beispiel: "Sortieren liefert immer eine aufsteigend geordnete Liste mit denselben Elementen" oder "Die Steuer steigt nie, wenn das Einkommen sinkt". jwvik erzeugt dann viele unterschiedliche Testdaten, inklusive Grenzfällen, und versucht aktiv, Gegenbeispiele zu finden. So werden mehr Teile des Eingaberaums automatisch abgedeckt.

4.4.7. Anknüpfung: Äquivalenzklassenanalyse

- Äquivalenzklassenanalyse:
 - Eingaberaum in Bereiche mit ähnlichem Verhalten aufteilen
 - Je Klasse einige repräsentative Testfälle + Grenzwerte wählen
- Property-Based Testing (PBT):
 - Nutzt dieselben Bereiche/Eigenschaften
 - Aber: Framework erzeugt viele Werte innerhalb der Bereiche
- Kein "doppelt genäht", sondern:
 - Äquivalenzklassen = Denkwerkzeug
 - PBT = automatisierte Stichproben über diese Klassen

Sie kennen bereits die Idee, den Eingaberaum in Äquivalenzklassen aufzuteilen und pro Klasse typische sowie Grenzfälle zu testen. Property-Based Testing (PBT) baut genau darauf auf: Sie formulieren Eigenschaften, die für ganze Klassen (z.B. "Einkommen zwischen 10k und 20k") gelten müssen, und überlassen dem Framework die Auswahl vieler konkreter Werte in diesen Bereichen. Das Denken in Bereichen und Grenzwerten bleibt, wird aber automatisiert "ausgerollt".

4.4.8. Durchgängiges Beispiel: Steuerfunktion - Spezifikation

- Funktion: `calculateTax(int income)`
 - Einkommen in ganzen Euro (`income >= 0`)
 - Rückgabe: Steuerbetrag (ganze Euro, abgerundet)
- Steuerregeln (vereinfacht):
 - < 10 000: 0 %
 - 10 000 - 20 000: 10 % auf den Teil über 10 000
 - 20 000 - 50 000: zusätzlich 20 % auf den Teil über 20 000
 - > 50 000: zusätzlich 30 % auf den Teil über 50 000
- Negative Einkünfte: `IllegalArgumentException`

```

1 public class TaxCalculator {
2     public static int calculateTax(int income) {
3         if (income < 0) throw new IllegalArgumentException("Income must be >= 0");
4
5         double tax = 0.0;
6
7         if (income <= 10_000) tax = 0.0;
8         else if (income <= 20_000) tax = 0.10 * (income - 10_000);
9         else if (income <= 50_000) tax = 0.10 * 10_000 + 0.20 * (income - 20_000);
10        else tax = 0.10 * 10_000 + 0.20 * 30_000 + 0.30 * (income - 50_000);
11
12        return (int) Math.floor(tax);
13    }
14 }

```

4.4.9. Steuerfunktion: Äquivalenzklassen & Grenzwerte

- Äquivalenzklassen:
 - E1: `income < 0` (ungültig)
 - E2: `0 <= income < 10000`
 - E3: `10000 <= income <= 20000`
 - E4: `20000 < income <= 50000`
 - E5: `income > 50000`
- Wichtige Grenzwerte:
 - Rund um 0: -1, 0, 1
 - Rund um 10000: 9999, 10000, 10001
 - Rund um 20000: 19999, 20000, 20001
 - Rund um 50000: 49999, 50000, 50001

Jede Äquivalenzklasse beschreibt Bereiche, in denen die Steuerfunktion dasselbe Rechenmuster nutzt.

Um diese Klassen herum identifizieren wir Grenzwerte, an denen das Verhalten wechselt (z.B. 9999 vs. 10000). Das ist die klassische Vorbereitung für systematische Tests - und gleichzeitig eine gute Basis für Properties.

4.4.10. Klassische JUnit-Tests: Beispiele aus den Klassen

- Für jede Äquivalenzklasse einige repräsentative Werte testen
- Zusätzlich Grenzfälle explizit prüfen
- Negative Eingaben: Exception erwarten

```

1 // E1: Ungültige Eingabe
2 @Test
3 void negative_income_throws_exception() {

```

```

4     assertThrows(IllegalArgumentException.class, () -> TaxCalculator.
        calculateTax(-1));
5 }
6
7 // E2: 0 <= income < 10_000
8 @Test
9 void income_below_10k_is_tax_free() {
10     assertEquals(0, TaxCalculator.calculateTax(0));
11     assertEquals(0, TaxCalculator.calculateTax(5_000));
12     assertEquals(0, TaxCalculator.calculateTax(9_999));
13 }
14
15 // ...

```

Diese klassischen Tests setzen direkt die Äquivalenzklassenanalyse um: Für jede Klasse und jeden Grenzbe-
reich wählen wir einige typische Einkünfte und erwarten einen exakt berechneten Steuerbetrag. Diese Tests
dokumentieren das Verhalten in verständlichen Beispielen.

4.4.11. jqwik: Einfache Property - Steuer nie negativ

Ziel-Eigenschaft: Für alle gültigen Einkommen gilt: Steuerbetrag ≥ 0

Nutzung von jqwik:

- Einbindung in Gradle:

```

1 dependencies {
2     testImplementation 'net.jqwik:jqwik:1.10.1'
3 }

```

- Testfall: Annotation `@Property` statt `@Test`
- Generieren von Werten: Annotation `@ForAll` an Parametern generiert viele ganzzahlige Einkommen

```

1 class TaxCalculatorProperties {
2     @Property
3     void tax_is_never_negative(@ForAll @IntRange(min = 0, max = 1_000_000) int
        income) {
4         int tax = TaxCalculator.calculateTax(income);
5         assertTrue(tax >= 0);
6     }
7 }

```

Hier formulieren wir eine sehr allgemeine Eigenschaft unserer Steuerberechnung: Bei keinem gültigen Einkom-
men darf eine negative Steuer herauskommen. Die Java-Bibliothek jqwik generiert viele Einkommen im Bereich 0
bis 1000000 und wendet die Funktion darauf an. Mit der einfachen JUnit-Assertion `assertTrue(tax >= 0)`
prüfen wir die Eigenschaft wie gewohnt.

Der Test beschreibt nicht konkrete Zahlen, sondern ein generelles Gesetz.

4.4.12. jqwik: Monotonie-Eigenschaft für die Steuer

- Intuition: Wenn Einkommen steigt, darf Steuer nicht sinken
- Formale Property: Für alle a, b mit $a \leq b$ gilt: $\text{tax}(a) \leq \text{tax}(b)$

```

1  @Property
2  void tax_is_monotone_non_decreasing(
3      @ForAll @IntRange(min = 0, max = 1_000_000) int a,
4      @ForAll @IntRange(min = 0, max = 1_000_000) int b) {
5
6      int lower = Math.min(a, b);
7      int higher = Math.max(a, b);
8
9      int taxLower = TaxCalculator.calculateTax(lower);
10     int taxHigher = TaxCalculator.calculateTax(higher);
11
12     assertTrue(taxLower <= taxHigher);
13 }

```

Diese Property fasst eine wesentliche Anforderung an ein Steuersystem zusammen: Wer mehr verdient, soll nicht weniger Steuer zahlen. jqwik erzeugt viele Zahlenpaare, die wir im Test sortieren (**lower**, **higher**) und für die wir jeweils die entsprechenden Steuerbeträge vergleichen.

Ein Verstoß gegen diese Monotonie würde auf inkonsistente Steuerregeln oder Implementierungsfehler hinweisen - und jqwik würde automatisch ein Gegenbeispiel liefern.

4.4.13. jqwik: Verbindung zu Äquivalenzklassen (Bracket-Property)

- Idee: Innerhalb einer Steuerklasse (z.B. 10000 - 20000) gilt ein linearer Zusammenhang
- Property-Beispiel: Innerhalb 10000 - 20000 steigt die Steuer mit 10 % der Einkommensdifferenz

```

1  @Property
2  void within_bracket_10k_to_20k_tax_grows_with_roughly_10_percent(
3      @ForAll @IntRange(min = 10_000, max = 19_000) int base,
4      @ForAll @IntRange(min = 0, max = 1_000) int delta) {
5      int income1 = base; int income2 = base + delta;
6      if (income2 > 20_000) income2 = 20_000;
7
8      int tax1 = TaxCalculator.calculateTax(income1);
9      int tax2 = TaxCalculator.calculateTax(income2);
10     int diffIncome = income2 - income1;
11     int diffTax = tax2 - tax1;
12     int expected = (int) Math.floor(0.10 * diffIncome);
13
14     // Wegen Rundung erlauben wir +/- 1 Euro Toleranz
15     assertTrue(diffTax >= expected - 1 && diffTax <= expected + 1);
16 }

```

In dieser Property verbinden wir explizit die Äquivalenzklasse "Einkommen zwischen 10000 und 20000" mit einer Eigenschaft (*Property*): In diesem Bereich gilt eine konstante Steuerquote von 10 %. Die Eingaben werden von jqwik auf diesen Bereich beschränkt, und wir prüfen, ob die Steuerdifferenz 10 % der Einkommensdifferenz

entspricht. Weil in der Implementierung mit `double` gearbeitet und mit `Math.floor` gerundet wird, kann es zu Abweichungen um 1 Euro kommen, was durch eine kleine Toleranz berücksichtigt wird.

4.4.14. Wrap-Up

- Klassische Unit-Tests:
 - Dokumentieren Verhalten an Beispielen
 - Nutzen Äquivalenzklassen & Grenzwerte manuell
- Approval Testing:
 - Erleichtert Tests komplexer Outputs (“Goldstandard”)
 - Passt gut zu Refactoring & Code-Review
- Property-Based Testing:
 - Formuliert allgemeine Eigenschaften über alle Eingaben
 - Nutzt Äquivalenzklassen als Denkgrundlage, generiert aber viele Fälle automatisch

Für die Praxis bedeutet das: Sie starten bei der gedanklichen Strukturierung mit Äquivalenzklassenanalyse, schreiben daraus ein paar klassische JUnit-Tests zur Dokumentation und Verständlichkeit und ergänzen diese dann durch Properties, die allgemeine Invarianten absichern. Wenn zusätzlich komplexe Text- oder Datenstrukturen zu testen sind, kann Approval Testing helfen, Outputs als referenzierte Goldstandards zu verwalten. Alle drei Techniken ergänzen sich und helfen Ihnen, robusteren und besser geprüften Code zu schreiben.

Das Thema Property-Based Testing geht sehr tief, wir schaffen hier nur einen ersten Einblick in die Grundideen. Frameworks wie jqwik suchen beispielsweise bei der Ausführung nach “kleinen Gegenbeispiele” (sogenanntes *Shrinking*), d.h. es wird hier nicht einfach nur eine Spielart von “Random Testing”.

Tip

Sie finden den Beispielcode in einem vorkonfigurierten Gradle-Projekt im Ordner [ptb](#).

Zum Nachlesen

Approval Testing:

- Schönes Video: <https://youtu.be/YAXGU2J7XjM> (ab 17:12 Approval testing ft. Emily Bache)
- Bibliothek [ApprovalTests.Java](#)
- [ApprovalTests Getting Started](#)

Property based Testing:

- Schönes Video (letzter Teil): <https://youtu.be/MWsk1h8pv2Q> (ab 37:33 FizzBuzz)
- Blog von Johannes Link (Maintainer von jqwik): [Property-based Testing in Java: Jqwik - a JUnit 5 Test Engine](#)
- Doku vom [jqwik-Projekt](#)

Lernziele

- k2: Ich kann das Grundprinzip von Approval Testing an einem selbst gewählten Beispiel erklären (Goldstandard/“approved“-Output, Vergleich mit “received“-Output, Rolle von Diff-Ansichten).
- k2: Ich kann den Unterschied zwischen klassischen Beispieltests und Property-Based Testing in eigenen Worten erläutern (Beispiele vs. allgemeine Eigenschaften über viele Eingaben).
- k2: Ich kann erklären, wie Äquivalenzklassenanalyse die Auswahl von Testfällen strukturiert und warum sie auch für Property-Based Testing wichtig ist.
- k2: Ich kann typische Einsatzszenarien für Approval Testing und Property-Based Testing nennen und begründen, warum diese Techniken dort sinnvoll sind.
- k3: Ich kann für eine gegebene Funktion sinnvolle Äquivalenzklassen formulieren und daraus konkrete JUnit-Beispieltests ableiten.
- k3: Ich kann für eine Funktion mit komplexem String-Output (z.B. Report oder formatierte Liste) einen einfachen Approval-Test schreiben und den Approved-Output im Projekt verwalten.
- k3: Ich kann für eine gegebene Funktion mindestens eine geeignete Property formulieren und diese mit jqwik als `@Property`-Test umsetzen.
- k3: Ich kann in einem bestehenden Test-Set begründet entscheiden, ob Beispieltests, Approval Tests oder Property-Based Tests (oder eine Kombination) sinnvoll sind, und dies kurz begründen.

5. Modern Java: Funktionaler Stil und Stream-API

5.1. Lambda-Ausdrücke und funktionale Interfaces

TL;DR

Mit einer anonymen inneren Klasse erstellt man gewissermaßen ein Objekt einer “Wegwerf”-Klasse: Man leitet *on-the-fly* von einem Interface ab oder erweitert eine Klasse und implementiert die benötigten Methoden und erzeugt von dieser Klasse sofort eine Instanz (Objekt). Diese neue Klasse ist im restlichen Code nicht sichtbar.

Anonyme innere Klassen sind beispielsweise in Swing recht nützlich, wenn man einer Komponente einen Listener mitgeben will: Hier erzeugt man eine anonyme innere Klasse basierend auf dem passenden Listener-Interface, implementiert die entsprechenden Methoden und übergibt das mit dieser Klasse erzeugte Objekt als neuen Listener der Swing-Komponente.

Mit Java 8 können unter gewissen Bedingungen diese anonymen inneren Klassen zu Lambda-Ausdrücken (und Methoden-Referenzen) vereinfacht werden. Dazu muss die anonyme innere Klasse ein sogenanntes **funktionales Interface** implementieren.

Funktionale Interfaces sind Interfaces mit *genau einer abstrakten Methode*. Es können beliebig viele Default-Methoden im Interface enthalten sein, und es können **public** sichtbare abstrakte Methoden von `java.lang.Object` geerbt/überschrieben werden.

Die Lambda-Ausdrücke entsprechen einer anonymen Methode: Die Parameter werden aufgelistet (in Klammern), und hinter einem Pfeil kommt entweder *ein* Ausdruck (Wert - gleichzeitig Rückgabewert des Lambda-Ausdrucks) oder beliebig viele Anweisungen (in geschweiften Klammern, mit Semikolon):

- Form 1: `(parameters) -> expression`
- Form 2: `(parameters) -> { statements; }`

Der Lambda-Ausdruck muss von der Signatur her genau der einen abstrakten Methode im unterliegenden funktionalen Interface entsprechen.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demos:

- Anonyme innere Klassen [\[YT\]](#), [\[HSBI\]](#)
- Lambda-Ausdrücke [\[YT\]](#), [\[HSBI\]](#)
- Funktionale Interfaces selbst definiert [\[YT\]](#), [\[HSBI\]](#)
- Vordefinierte funktionale Interfaces im JDK [\[YT\]](#), [\[HSBI\]](#)

5.1.1. Problem: Sortieren einer Studi-Liste

```
1 List<Studi> sl = new ArrayList<>();
2
3 // Liste sortieren?
4 sl.sort(???); // Parameter: java.util.Comparator<Studi>
```

```
1 public class MyCompare implements Comparator<Studi> {
2     @Override public int compare(Studi o1, Studi o2) {
3         return o1.getCredits() - o2.getCredits();
4     }
5 }
```

```
1 // Liste sortieren?
2 MyCompare mc = new MyCompare();
3 sl.sort(mc);
```

Da `Comparator<T>` ein Interface ist, muss man eine extra Klasse anlegen, die die abstrakte Methode aus dem Interface implementiert und ein Objekt von dieser Klasse erzeugen und dieses dann der `sort()`-Methode übergeben.

Die Klasse bekommt wie in Java üblich eine eigene Datei und ist damit in der Package-Struktur offen sichtbar und “verstopft” mir damit die Strukturen: Diese Klasse ist doch nur eine Hilfsklasse ... Noch schlimmer: Ich brauche einen Namen für diese Klasse!

Den ersten Punkt könnte man über verschachtelte Klassen lösen: Die Hilfsklasse wird innerhalb der Klasse definiert, die das Objekt benötigt. Für den zweiten Punkt brauchen wir mehr Anlauf ...

5.1.2. Erinnerung: Verschachtelte Klassen (“*Nested Classes*”)

Man kann Klassen innerhalb von Klassen definieren: Verschachtelte Klassen.

- Implizite Referenz auf Instanz der äußeren Klasse, Zugriff auf **alle** Elemente
- **Begriffe:**
 - “normale” innere Klassen: “*inner classes*”
 - statische innere Klassen: “*static nested classes*”
- Einsatzzweck:
 - Hilfsklassen: Zusätzliche Funktionalität kapseln; Nutzung **nur** in äußerer Klasse
 - Kapselung von Rückgabewerten

Sichtbarkeit: Wird u.U. von äußerer Klasse “überstimmt”

5.1.2.1. Innere Klassen (“*Inner Classes*”)

- Objekt der äußeren Klasse muss existieren
- Innere Klasse ist normales Member der äußeren Klasse

- Implizite Referenz auf Instanz äußerer Klasse
- Zugriff auf **alle** Elemente der äußeren Klasse
- Sonderfall: Definition innerhalb von Methoden ("local classes")
 - Nur innerhalb der Methode sichtbar
 - Kennt zusätzlich **final** Attribute der Methode

Beispiel:

```

1 public class Outer {
2     ...
3     private class Inner {
4         ...
5     }
6
7     Outer.Inner inner = new Outer().new Inner();
8 }

```

Beispiel mit Iterator als innere Klasse: `nested.StudiListNested`

5.1.2.2. Statische innere Klassen ("Static Nested Classes")

- Keine implizite Referenz auf Objekt
- Nur Zugriff auf Klassenmethoden und -attribute

Beispiel:

```

1 class Outer {
2     ...
3     static class StaticNested {
4         ...
5     }
6 }
7
8 Outer.StaticNested nested = new Outer.StaticNested();

```

5.1.3. Lösung: Comparator als anonyme innere Klasse

```

1 List<Studi> sl = new ArrayList<>();
2
3 // Parametrisierung mit anonymer Klasse
4 sl.sort(
5     new Comparator<Studi>() {
6         @Override
7         public int compare(Studi o1, Studi o2) {
8             return o1.getCredits() - o2.getCredits();
9         }
10    }); // Semikolon nicht vergessen!!!

```

=> Instanz einer anonymen inneren Klasse, die das Interface `Comparator<Studi>` implementiert

- Für spezielle, einmalige Aufgabe: nur eine Instanz möglich

- Kein Name, kein Konstruktor, oft nur eine Methode
- Müssen Interface implementieren oder andere Klasse erweitern
 - Achtung Schreibweise: ohne **implements** oder **extends**!
- Konstruktor kann auch Parameter aufweisen
- Zugriff auf alle Attribute der äußeren Klasse plus alle **final** lokalen Variablen
- Nutzung typischerweise bei GUIs: Event-Handler etc.

[Demo: nested.DemoAnonymousInnerClass](#)

5.1.4. Vereinfachung mit Lambda-Ausdruck

```
1 List<Studi> sl = new ArrayList<>();
2
3 // Parametrisierung mit anonymer Klasse
4 sl.sort(
5     new Comparator<Studi>() {
6         @Override
7         public int compare(Studi o1, Studi o2) {
8             return o1.getCredits() - o2.getCredits();
9         }
10    }); // Semikolon nicht vergessen!!!
11
12
13 // Parametrisierung mit Lambda-Ausdruck
14 sl.sort( (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits() );
```

Anmerkung: Damit für den Parameter alternativ auch ein Lambda-Ausdruck verwendet werden kann, muss der erwartete Parameter vom Typ her ein **“funktionales Interface”** (s.u.) sein!

[Demo: nested.DemoLambda](#)

5.1.5. Syntax für Lambdas

```
1 (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits()
```

Ein Lambda-Ausdruck ist eine Funktion ohne Namen und besteht aus drei Teilen:

1. Parameterliste (in runden Klammern),
2. Pfeil
3. Funktionskörper (rechte Seite)

Falls es *genau einen* Parameter gibt, *können* die runden Klammern um den Parameter entfallen.

Dabei kann der Funktionskörper aus *einem Ausdruck* (“*expression*”) bestehen oder einer *Menge von Anweisungen* (“*statements*”), die dann in geschweifte Klammern eingeschlossen werden müssen (Block mit Anweisungen).

Der Wert des Ausdrucks ist zugleich der Rückgabewert des Lambda-Ausdrucks.

Varianten:

- `(parameters)-> expression`
- `(parameters)-> { statements; }`

5.1.6. Quiz: Welches sind keine gültigen Lambda-Ausdrücke?

1. `() -> {}`
2. `() -> "wuppie"`
3. `() -> { return "fluppie"; }`
4. `(Integer i)-> return i + 42;`
5. `(String s)-> { "foo"; }`
6. `(String s)-> s.length()`
7. `(Studi s)-> s.getCredits() > 300`
8. `(List<Studi> sl)-> sl.isEmpty()`
9. `(int x, int y)-> { System.out.println("Erg: "); System.out.println(x+y); }`
10. `() -> new Studi()`
11. `s -> s.getCps() > 100 && s.getCps() < 300`
12. `s -> { return s.getCps() > 100 && s.getCps() < 300; }`

Auflösung: (4) und (5)

return ist eine Anweisung, d.h. bei (4) fehlen die geschweiften Klammern. **"foo"** ist ein String und als solcher ein Ausdruck, d.h. hier sind die geschweiften Klammern zu viel (oder man ergänze den String mit einem **return**, also **return "foo"; ...**).

5.1.7. Definition “Funktionales Interface” (“functional interfaces”)

```

1 @FunctionalInterface
2 public interface Wuppie<T> {
3     int wuppie(T obj);
4     boolean equals(Object obj);
5     default int fluppie() { return 42; }
6 }

```

Wuppie<T> ist ein **funktionales Interface** (“functional interface”) (seit Java 8)

- Hat **genau eine abstrakte Methode**
- Hat evtl. weitere Default-Methoden
- Hat evtl. weitere abstrakte Methoden, die **public** Methoden von `java.lang.Object` überschreiben

Die Annotation `@FunctionalInterface` selbst ist nur für den Compiler: Falls das Interface *kein* funktionales Interface ist, würde er beim Vorhandensein dieser Annotation einen Fehler werfen. Oder anders herum: Allein durch das Annotieren mit `@FunctionalInterface` wird aus einem Interface noch kein funktionales Interface! Vergleichbar mit `@Override` ...

Während man für eine anonyme Klasse lediglich ein “normales” Interface (oder eine Klasse) benötigt, braucht man für Lambda-Ausdrücke zwingend ein passendes funktionales Interface!

Anmerkung: Es scheint keine einheitliche deutsche Übersetzung für den Begriff *functional interface* zu geben. Es wird häufig mit “funktionales Interface”, manchmal aber auch mit “Funktionsinterface” übersetzt.

Das in den obigen Beispielen eingesetzte Interface `java.util.Comparator<T>` ist also ein funktionales Interface: Es hat nur *eine* eigene abstrakte Methode `int compare(T o1, T o2);`.

Im Package `java.util.function` sind einige wichtige funktionale Interfaces bereits vordefiniert, beispielsweise `Predicate` (Test, ob eine Bedingung erfüllt ist) und `Function` (verarbeite einen Wert und liefere einen passenden Ergebniswert). Diese kann man auch in eigenen Projekten nutzen!

5.1.8. Quiz: Welches ist kein funktionales Interface?

```

1 public interface Wuppie {
2     int wuppie(int a);
3 }
4
5 public interface Fluppie extends Wuppie {
6     int wuppie(double a);
7 }
8
9 public interface Foo {
10 }
11
12 public interface Bar extends Wuppie {
13     default int bar() { return 42; }
14 }
```

Auflösung:

- `Wuppie` hat *genau eine* abstrakte Methode => funktionales Interface
- `Fluppie` hat zwei abstrakte Methoden => **kein** funktionales Interface
- `Foo` hat gar keine abstrakte Methode => **kein** funktionales Interface
- `Bar` hat *genau eine* abstrakte Methode (und eine Default-Methode) => funktionales Interface

5.1.9. Lambdas und funktionale Interfaces: Typprüfung

```

1 interface java.util.Comparator<T> {
2     int compare(T o1, T o2);    // abstrakte Methode
3 }
```

```

1 // Verwendung ohne weitere Typinferenz
2 Comparator<Studi> c1 = (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits
   ();
3
4 // Verwendung mit Typinferenz
5 Comparator<Studi> c2 = (o1, o2) -> o1.getCredits() - o2.getCredits();
```

Der Compiler prüft in etwa folgende Schritte, wenn er über einen Lambda-Ausdruck stolpert:

1. In welchem Kontext habe ich den Lambda-Ausdruck gesehen?
2. OK, der Zieltyp ist hier `Comparator<Studi>`.
3. Wie lautet die **eine** abstrakte Methode im `Comparator<T>`-Interface?
4. OK, das ist `int compare(T o1, T o2)`;
5. Da `T` hier an `Studi` gebunden ist, muss der Lambda-Ausdruck der Methode `int compare(Studi o1, Studi o2)`; entsprechen: 2x `Studi` als Parameter und als Ergebnis ein `int`
6. Ergebnis:
 - a) Cool, passt zum Lambda-Ausdruck `c1`. Fertig.
 - b) D.h. in `c2` müssen `o1` und `o2` vom Typ `Studi` sein. Cool, passt zum Lambda-Ausdruck `c2`. Fertig.

5.1.10. Wrap-Up

- Anonyme Klassen: “Wegwerf”-Innere Klassen
 - Müssen Interface implementieren oder Klasse erweitern
- Java8: **Lambda-Ausdrücke** statt anonymer Klassen (**funktionales Interface nötig**)
 - Zwei mögliche Formen:
 - * Form 1: `(parameters) -> expression`
 - * Form 2: `(parameters) -> { statements; }`
 - Im jeweiligen Kontext muss ein **funktionales Interface** verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode
 - Der Lambda-Ausdruck muss von der Signatur her dieser einen abstrakten Methode entsprechen

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials [“Writing Your First Lambda Expression” \(Oracle\)](#), [“When to Use Nested Classes, Local Classes, Anonymous Classes, and Lambda Expressions”](#) und [“Using Lambdas Expressions in Your Application” \(Oracle\)](#) nach.

Lernziele

- k2: Ich kenne die Definition ‘Funktionales Interface’
- k3: Ich kann innere und anonyme Klassen praktisch einsetzen
- k3: Ich kann eigene funktionale Interfaces erstellen
- k3: Ich kann Lambda-Ausdrücke formulieren und einsetzen

Challenges

Beispiel aus einem Code-Review im [Dungeon-CampusMinden/Dungeon](#)

Erklären Sie folgenden Code:

```
1 public interface IFightAI {
2     void fight(Entity entity);
```

```

3  }
4
5  public class AIComponent extends Component {
6      private final IFightAI fightAI;
7
8      fightAI =
9          entity1 -> {
10              System.out.println("TIME TO FIGHT!");
11              // todo replace with melee skill
12          };
13  }

```

Spielen mit Lambdas

Sie finden in einem Spiel folgenden Code:

```

1  public class Main {
2      public static void main(String[] args) {
3          DoorTile door = new DoorTile();
4          Entity lever1 = new Entity(), lever2 = new Entity(), lever3 = new
            Entity();
5
6          // ganz viel Code
7
8          if (!door.isOpen() && (lever1.isOn() && (lever2.isOn() || lever3.
                isOn())) door.open();
9
10         // ganz viel Code
11     }
12 }
13
14 class DoorTile {
15     public boolean isOpen() { return false; }
16     public void open() { }
17 }
18 class Entity {
19     public boolean isOn() { return false; }
20 }

```

Dabei stört, dass die Verknüpfung der konkreten Objekte und Zustände zum Öffnen der konkreten Tür fest (und zudem mitten) im Programm hinterlegt ist.

Schreiben Sie diesen Code um: Definieren Sie eine statische Hilfsmethode, die ein Door-Tile und drei Entitäten als Argument entgegen nimmt und dafür einen Lambda-Ausdruck zurückliefert, mit dem (a) die gezeigte Bedingung überprüft werden kann, und mit dem (falls die Bedingung erfüllt ist) (b) die Aktion (`door.open()`) ausgeführt werden kann. Statt der gezeigten fest codierten **if**-Abfrage soll dieser Lambda-Ausdruck ausgewertet werden: `doorhandle.test().accept()`;

Damit haben Sie sich eine “Factory-Method” geschrieben (Entwurfsmuster), mit der diese Bedingung dynamisch erzeugt werden kann (auch für andere Objekte).

Hinweis: Der Lambda-Ausdruck wird “zweistufig” sein müssen ...

Sortieren mit Lambdas und funktionalen Interfaces

Betrachten Sie die Klasse [Student](#).

1. Definieren Sie eine Methode, die das Sortieren einer [Student](#)-Liste erlaubt. Übergeben Sie die Liste

- als Parameter.
2. Schaffen Sie es, das Sortierkriterium ebenfalls als Parameter zu übergeben (als Lambda-Ausdruck)?
 3. Definieren Sie eine weitere Methode, die wieder eine `Student`-Liste als Parameter bekommt und liefern sie das erste `Student`-Objekt zurück, welches einer als Lambda-Ausdruck übergebenen Bedingung genügt.
 4. Definieren Sie noch eine Methode, die wieder eine `Student`-Liste als Parameter bekommt sowie einen Lambda-Ausdruck, welcher aus einem `Student`-Objekt ein Objekt eines anderen Typen `T` berechnet. Wenden Sie in der Methode den Lambda-Ausdruck auf jedes Objekt der Liste an und geben sie die resultierende neue Liste als Ergebnis zurück.

Verwenden Sie in dieser Aufgabe jeweils Lambda-Ausdrücke. Rufen Sie alle drei/vier Methoden an einem kleinen Beispiel auf.

5.2. Methoden-Referenzen

TL;DR

Seit Java8 können **Referenzen auf Methoden** statt anonymer Klassen eingesetzt werden (**funktionales Interface nötig**).

Dabei gibt es drei mögliche Formen:

- Form 1: Referenz auf eine statische Methode: `ClassName::staticMethodName` (wird verwendet wie `(args) -> ClassName.staticMethodName(args)`)
- Form 2: Referenz auf eine Instanz-Methode eines Objekts: `objectref::instanceMethodName` (wird verwendet wie `(args) -> objectref.instanceMethodName(args)`)
- Form 3: Referenz auf eine Instanz-Methode eines Typs: `ClassName::instanceMethodName` (wird verwendet wie `(o1, args) -> o1.instanceMethodName(args)`)

Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode. Die Methoden-Referenz muss von der Syntax her dieser einen abstrakten Methode entsprechen (bei der dritten Form wird die Methode auf dem ersten Parameter aufgerufen).

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demos:

- Referenz auf statische Methode [\[YT\]](#), [\[HSBI\]](#)
- Referenz auf Instanz-Methode (Objekt) [\[YT\]](#), [\[HSBI\]](#)
- Referenz auf Instanz-Methode (Typ) [\[YT\]](#), [\[HSBI\]](#)

5.2.1. Beispiel: Sortierung einer Liste


```
1 List<Studi> sl = new ArrayList<Studi>();
2
3 // Anonyme innere Klasse
4 Collections.sort(sl, new Comparator<Studi>() {
5     @Override public int compare(Studi o1, Studi o2) {
6         return Studi.cmpCpsClass(o1, o2);
7     }
8 });
9
10
11 // Lambda-Ausdruck
12 Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));
13
14 // Methoden-Referenz
15 Collections.sort(sl, Studi::cmpCpsClass);
```

5.2.1.1. Anmerkung

Für das obige Beispiel wird davon ausgegangen, dass in der Klasse `Studi` eine statische Methode `cmpCpsClass()` existiert:

```
1 public static int cmpCpsClass(Studi s1, Studi s2) {
2     return s1.getCps() - s2.getCps();
3 }
```

Wenn man im Lambda-Ausdruck nur Methoden der eigenen Klasse aufruft, kann man das auch direkt per *Methoden-Referenz* abkürzen!

- Erinnerung: `Comparator<T>` ist ein funktionales Interface
- Instanzen können wie üblich durch Ableiten bzw. anonyme Klassen erzeugt werden
- Alternativ kann seit Java8 auch ein passender Lambda-Ausdruck verwendet werden
- Ab Java8: Referenzen auf passende Methoden (Signatur!) können ein funktionales Interface “implementieren”
 - Die statische Methode `static int cmpCpsClass(Studi s1, Studi s2)` hat die selbe Signatur wie `int compare(Studi s1, Studi s2)` aus `Comparator<Studi>`
 - Kann deshalb wie eine Instanz von `Comparator<Studi>` genutzt werden
 - Name der Methode spielt dabei keine Rolle

5.2.2. Überblick: Arten von Methoden-Referenzen

1. Referenz auf eine statische Methode

- Form: `ClassName::staticMethodName`
- Wirkung: Aufruf mit `(args) -> ClassName.staticMethodName(args)`

2. Referenz auf Instanz-Methode eines bestimmten Objekts

- Form: `objectref::instanceMethodName`

- Wirkung: Aufruf mit `(args) -> objectref.instanceMethodName(args)`

3. Referenz auf Instanz-Methode eines bestimmten Typs

- Form: `ClassName::instanceMethodName`
- Wirkung: Aufruf mit `(arg0, rest) -> arg0.instanceMethodName(rest)`
(`arg0` ist vom Typ `ClassName`)

Anmerkung: Analog zur Referenz auf eine statische Methode gibt es noch die Form der Referenz auf einen Konstruktor: `ClassName::new`. Für Referenzen auf Konstruktoren mit mehr als 2 Parametern muss ein eigenes passendes funktionales Interface mit entsprechend vielen Parametern definiert werden ...

5.2.3. Methoden-Referenz 1: Referenz auf statische Methode

```

1 public class Studi {
2     public static int cmpCpsClass(Studi s1, Studi s2) {
3         return s1.getCredits() - s2.getCredits();
4     }
5
6     public static void main(String... args) {
7         List<Studi> sl = new ArrayList<Studi>();
8
9         // Referenz auf statische Methode
10        Collections.sort(sl, Studi::cmpCpsClass);
11
12        // Entsprechender Lambda-Ausdruck
13        Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));
14    }
15 }

```

Demo: [methodreferences.DemoStaticMethodReference](#)

`Collections.sort()` erwartet in diesem Szenario als zweiten Parameter eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **statische Methode `cmpCpsClass` der Klasse `Studi`** hat die **selbe Signatur** und wird deshalb von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich `Studi.cmpCpsClass(o1, o2)` aufgerufen.

5.2.4. Methoden-Referenz 2: Referenz auf Instanz-Methode (Objekt)

```

1 public class Studi {
2     public int cmpCpsInstance(Studi s1, Studi s2) {
3         return s1.getCredits() - s2.getCredits();
4     }
5
6     public static void main(String... args) {
7         List<Studi> sl = new ArrayList<Studi>();
8         Studi holger = new Studi("Holger", 42);

```

```

 9
10      // Referenz auf Instanz-Methode eines Objekts
11      Collections.sort(sl, holger::cmpCpsInstance);
12
13      // Entsprechender Lambda-Ausdruck
14      Collections.sort(sl, (o1, o2) -> holger.cmpCpsInstance(o1, o2));
15  }
16 }

```

Demo: [methodreferences.DemoInstanceMethodReferenceObject](#)

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Objekts `holger`** hat die selbe Signatur und wird entsprechend von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich `holger.cmpCpsInstance(o1, o2)` aufgerufen.

5.2.5. Methoden-Referenz 3: Referenz auf Instanz-Methode (Typ)

```

1 public class Studi {
2     public int cmpCpsInstance(Studi studi) {
3         return this.getCredits() - studi.getCredits();
4     }
5
6     public static void main(String... args) {
7         List<Studi> sl = new ArrayList<Studi>();
8
9         // Referenz auf Instanz-Methode eines Typs
10        Collections.sort(sl, Studi::cmpCpsInstance);
11
12        // Entsprechender Lambda-Ausdruck
13        Collections.sort(sl, (o1, o2) -> o1.cmpCpsInstance(o2));
14    }
15 }

```

Demo: [methodreferences.DemoInstanceMethodReferenceType](#)

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Typs `Studi`** hat die Signatur `int cmpCpsInstance(Studi studi)` und wird von `Collections.sort()` so genutzt: Statt `compare(o1, o2)` wird nun für jeden Vergleich `o1.cmpCpsInstance(o2)` aufgerufen.

5.2.6. Ausblick: Threads

Erinnerung an bzw. Vorgriff auf Nebenläufige Programmierung (vgl. auch [Einführung in die nebenläufige Programmierung](#) (Rheinwerk Verlag) und [Lesson: Concurrency](#) (Oracle)):

```

1 public interface Runnable {
2     void run();
3 }

```

Damit lassen sich Threads auf verschiedene Arten erzeugen:

```

1 public class ThreadStarter {
2     public static void wuppie() { System.out.println("wuppie(): wuppie"); }
3 }
4
5
6 Thread t1 = new Thread(new Runnable() {
7     public void run() {
8         System.out.println("t1: wuppie");
9     }
10 });
11
12 Thread t2 = new Thread(() -> System.out.println("t2: wuppie"));
13
14 Thread t3 = new Thread(ThreadStarter::wuppie);

```

Beispiel: [methodreferences.ThreadStarter](#)

5.2.7. Ausblick: Datenstrukturen als Streams

Erinnerung an bzw. Vorgriff auf “Stream-API”:

```

1 class X {
2     public static boolean gtFour(int x) { return (x > 4) ? true : false; }
3 }
4
5 List<String> words = Arrays.asList("Java8", "Lambdas", "PM",
6     "Dungeon", "libGDX", "Hello", "World", "Wuppie");
7
8 List<Integer> wordLengths = words.stream()
9     .map(String::length)
10    .filter(X::gtFour)
11    .sorted()
12    .collect(toList());

```

Beispiel: [methodreferences.CollectionStreams](#)

- Collections können als Datenstrom betrachtet werden: `stream()`
 - Iteration über die Collection, analog zu externer Iteration mit `foreach`
- Daten aus dem Strom filtern: `filter`, braucht Prädikat
- Auf alle Daten eine Funktion anwenden: `map`
- Daten im Strom sortieren: `sort` (auch mit Comparator)
- Daten wieder einsammeln mit `collect`

=> Typische Elemente **funktionaler Programmierung**

=> Verweis auf Wahlfach “Spezielle Methoden der Programmierung”

5.2.8. Wrap-Up

Seit Java8: **Methoden-Referenzen** statt anonymer Klassen (**funktionales Interface nötig**)

- Drei mögliche Formen:
 - Form 1: Referenz auf statische Methode: `ClassName::staticMethodName`
(verwendet wie `(args) -> ClassName.staticMethodName(args)`)
 - Form 2: Referenz auf Instanz-Methode eines Objekts: `objectref::instanceMethodName`
(verwendet wie `(args) -> objectref.instanceMethodName(args)`)
 - Form 3: Referenz auf Instanz-Methode eines Typs: `ClassName::instanceMethodName`
(verwendet wie `(o1, args) -> o1.instanceMethodName(args)`)
- Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden (d.h. ein Interface mit **genau** einer abstrakten Methode)

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials [“Writing Lambda Expressions as Method References” \(Oracle\)](#) und [“Combining Lambda Expressions” \(Oracle\)](#) nach.

Lernziele

- k2: Ich verstehe die Definition von ‘Funktionale Interfaces’ und kann sie erklären
- k3: Ich kann Methoden-Referenzen lesen und selbst formulieren

Challenges

Betrachten Sie den folgenden Java-Code:

```
1 public class Cat {
2     int gewicht;
3     public Cat(int gewicht) { this.gewicht = gewicht; }
4
5     public static void main(String... args) {
6         List<Cat> clouder = new ArrayList<>();
7         clouder.add(new Cat(100)); clouder.add(new Cat(1)); clouder.add(
8             new Cat(10));
9         clouder.sort(...);
10    }
11 }
```

1. Ergänzen Sie den Methodenaufruf `clouder.sort(...)`; mit einer geeigneten anonymen Klasse, daß der `clouder` aufsteigend nach Gewicht sortiert wird.
2. Statt einer anonymen Klasse kann man auch Lambda-Ausdrücke einsetzen. Geben Sie eine konkrete Form an.
3. Statt einer anonymen Klasse kann man auch Methodenreferenzen einsetzen. Dafür gibt es mehrere Formen. Geben Sie für zwei Formen der Methodenreferenz sowohl den Aufruf als auch die

Implementierung der entsprechenden Methoden in der Klasse `Cat` an.

5.3. Interfaces: Default-Methoden

TL;DR

Seit Java8 können Methoden in Interfaces auch fertig implementiert sein: Sogenannte **Default-Methoden**. Dazu werden die Methoden mit dem neuen Schlüsselwort **default** gekennzeichnet. Die Implementierung wird an die das Interface implementierenden Klassen (oder Interfaces) vererbt und kann bei Bedarf überschrieben werden.

Da eine Klasse von einer anderen Klasse erben darf, aber mehrere Interfaces implementieren kann, könnte es zu einer Mehrfachvererbung einer Methode kommen: Eine Methode könnte beispielsweise in verschiedenen Interfaces als Default-Methode angeboten werden, und wenn eine Klasse diese Interfaces implementiert, steht eine Methode mit der selben Signatur auf einmal mehrfach zur Verfügung. Dies muss (u.U. manuell) aufgelöst werden.

Auflösung von Mehrfachvererbung:

- Regel 1: Klassen gewinnen
- Regel 2: Sub-Interfaces gewinnen
- Regel 3: Methode explizit auswählen

Aktuell ist der Unterschied zu abstrakten Klassen: Interfaces können **keinen Zustand** haben, d.h. keine Attribute/Felder.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo:

- Demo Regel 1 [\[YT\]](#), [\[HSBI\]](#)
- Demo Regel 2 [\[YT\]](#), [\[HSBI\]](#)
- Demo Regel 3 [\[YT\]](#), [\[HSBI\]](#)

5.3.1. Problem: Etablierte API (Interfaces) erweitern

```
1 interface Klausur {  
2     void anmelden(Studi s);  
3     void abmelden(Studi s);  
4 }
```

=> Nachträglich noch **void schreiben(Studi s);** ergänzen?

Wenn ein Interface nachträglich erweitert wird, müssen alle Kunden (also alle Klassen, die das Interface implementieren) auf die neuen Signaturen angepasst werden. Dies kann viel Aufwand verursachen und API-Änderungen damit unmöglich machen.

5.3.2. Default-Methoden: Interfaces mit Implementierung

Seit Java8 können Interfaces auch Methoden implementieren. Es gibt zwei Varianten: Default-Methoden und statische Methoden.

```
1 interface Klausur {
2     void anmelden(Studi s);
3     void abmelden(Studi s);
4
5     default void schreiben(Studi s) {
6         ... // Default-Implementierung
7     }
8
9     default void wuppie() {
10         throw new java.lang.UnsupportedOperationException();
11     }
12 }
```

Methoden können in Interfaces seit Java8 implementiert werden. Für Default-Methoden muss das Schlüsselwort **default** vor die Signatur gesetzt werden. Klassen, die das Interface implementieren, können diese Default-Implementierung erben oder selbst neu implementieren (überschreiben). Alternativ kann die Klasse eine Default-Methode neu *deklarieren* und wird damit zur abstrakten Klasse.

Dies ähnelt abstrakten Klassen. Allerdings kann in abstrakten Klassen neben dem Verhalten (implementierten Methoden) auch Zustand über die Attribute gespeichert werden.

5.3.3. Problem: Mehrfachvererbung

Drei Regeln zum Auflösen bei Konflikten:

1. **Klassen gewinnen:** Methoden aus Klasse oder Superklasse haben höhere Priorität als Default-Methoden
2. **Sub-Interfaces gewinnen:** Methode aus am meisten spezialisiertem Interface mit Default-Methode wird gewählt Beispiel: Wenn B **extends** A dann ist B spezialisierter als A
3. Sonst: Klasse muss **Methode explizit auswählen:** Methode überschreiben und gewünschte (geerbte) Variante aufrufen: X.**super**.m(. . .) (X ist das gewünschte Interface)

Auf den folgenden Folien wird dies anhand kleiner Beispiele verdeutlicht.

5.3.4. Auflösung Mehrfachvererbung: 1. Klassen gewinnen

```
1 interface A {
2     default String hello() { return "A"; }
3 }
4 class C {
5     public String hello() { return "C"; }
6 }
7 class E extends C implements A {}
8
9
10 /** Mehrfachvererbung: 1. Klassen gewinnen */
```

```
11 public class DefaultTest1 {
12     public static void main(String... args) {
13         String e = new E().hello();
14     }
15 }
```

[Demo: defaultmethods.rule1.DefaultTest1](#)

Die Klasse `E` erbt sowohl von Klasse `C` als auch vom Interface `A` die Methode `hello()` (Mehrfachvererbung). In diesem Fall “gewinnt” die Implementierung aus Klasse `C`.

1. Regel: Klassen gewinnen immer. Deklarationen einer Methode in einer Klasse oder einer Oberklasse haben Vorrang von allen Default-Methoden.

5.3.5. Auflösung Mehrfachvererbung: 2. Sub-Interfaces gewinnen

```
1 interface A {
2     default String hello() { return "A"; }
3 }
4 interface B extends A {
5     @Override default String hello() { return "B"; }
6 }
7 class D implements A, B {}
8
9
10 /** Mehrfachvererbung: 2. Sub-Interfaces gewinnen */
11 public class DefaultTest2 {
12     public static void main(String... args) {
13         String e = new D().hello();
14     }
15 }
```

[Demo: defaultmethods.rule2.DefaultTest2](#)

Die Klasse `D` erbt sowohl vom Interface `A` als auch vom Interface `B` die Methode `hello()` (Mehrfachvererbung). In diesem Fall “gewinnt” die Implementierung aus Klasse `B`: Interface `B` ist spezialisierter als `A`.

2. Regel: Falls Regel 1 nicht zutrifft, gewinnt die Default-Methode, die am meisten spezialisiert ist.

5.3.6. Auflösung Mehrfachvererbung: 3. Methode explizit auswählen

```
1 interface A {
2     default String hello() { return "A"; }
3 }
4 interface B {
5     default String hello() { return "B"; }
6 }
7 class D implements A, B {
8     @Override public String hello() { return A.super.hello(); }
9 }
10
11
```



```

12 /** Mehrfachvererbung: 3. Methode explizit auswählen */
13 public class DefaultTest3 {
14     public static void main(String... args) {
15         String e = new D().hello();
16     }
17 }

```

Demo: [defaultmethods.rule3.DefaultTest3](#)

Die Klasse `D` erbt sowohl vom Interface `A` als auch vom Interface `B` die Methode `hello()` (Mehrfachvererbung). In diesem Fall *muss* zur Auflösung die Methode in `D` neu implementiert werden und die gewünschte geerbte Methode explizit aufgerufen werden. (Wenn dies unterlassen wird, führt das selbst bei Nicht-Nutzung der Methode `hello()` zu einem Compiler-Fehler!)

Achtung: Der Aufruf der Default-Methode aus Interface `A` erfolgt mit `A.super.hello()`; (nicht einfach durch `A.hello()`;)!

3. Regel: Falls weder Regel 1 noch 2 zutreffen bzw. die Auflösung noch uneindeutig ist, muss man manuell durch die explizite Angabe der gewünschten Methode auflösen.

5.3.7. Quiz: Was kommt hier raus?

```

1 interface A {
2     default String hello() { return "A"; }
3 }
4 interface B extends A {
5     @Override default String hello() { return "B"; }
6 }
7 class C implements B {
8     @Override public String hello() { return "C"; }
9 }
10 class D extends C implements A, B {}
11
12
13 /** Quiz Mehrfachvererbung */
14 public class DefaultTest {
15     public static void main(String... args) {
16         String e = new D().hello(); // ???
17     }
18 }

```

Die Klasse `D` erbt sowohl von Klasse `C` als auch von den Interfaces `A` und `B` die Methode `hello()` (Mehrfachvererbung). In diesem Fall “gewinnt” die Implementierung aus Klasse `C`: Klassen gewinnen immer (Regel 1).

Beispiel: [defaultmethods.quiz.DefaultTest](#)

5.3.8. Statische Methoden in Interfaces

```

1 public interface Collection<E> extends Iterable<E> {
2     boolean add(E e);
3     ...

```

```

4  }
5  public class Collections {
6      private Collections() { }
7      public static <T> boolean addAll(Collection<? super T> c, T... elements)
8          {...}
9      ...
10     }

```

Typisches Pattern in Java: Interface plus Utility-Klasse (Companion-Klasse) mit statischen Hilfsmethoden zum einfacheren Umgang mit Instanzen des Interfaces (mit Objekten, deren Klasse das Interface implementiert). Beispiel: `Collections` ist eine Hilfs-Klasse zum Umgang mit `Collection`-Objekten.

Seit Java8 können in Interfaces neben Default-Methoden auch statische Methoden implementiert werden.

Die Hilfsmethoden können jetzt ins Interface wandern => Utility-Klassen werden obsolet ... Aus Kompatibilitätsgründen würde man die bisherige Companion-Klasse weiterhin anbieten, wobei die Implementierungen auf die statischen Methoden im Interface verweisen (*SKIZZE, nicht real!*):

```

1  public interface CollectionX<E> extends Iterable<E> {
2      boolean add(E e);
3      static <T> boolean addAll(CollectionX<? super T> c, T... elements) { ... }
4      ...
5  }
6  public class CollectionsX {
7      public static <T> boolean addAll(CollectionX<? super T> c, T... elements) {
8          return CollectionX.addAll(c, elements); // Verweis auf Interface
9      }
10     ...
11 }

```

5.3.9. Interfaces vs. Abstrakte Klassen

- **Abstrakte Klassen:** Schnittstelle und Verhalten und Zustand
- **Interfaces:**
 - vor Java 8 nur Schnittstelle
 - ab Java 8 Schnittstelle und Verhalten

Unterschied zu abstrakten Klassen: Kein Zustand, d.h. keine Attribute

- Design:
 - Interfaces sind beinahe wie abstrakte Klassen, nur ohne Zustand
 - Klassen können nur von **einer** (abstrakten) Klasse erben, aber **viele** Interfaces implementieren

5.3.10. Wrap-Up

Seit Java8: Interfaces mit Implementierung: **Default-Methoden**

- Methoden mit dem Schlüsselwort **default** können Implementierung im Interface haben
- Die Implementierung wird vererbt und kann bei Bedarf überschrieben werden
- Auflösung von Mehrfachvererbung:
 - Regel 1: Klassen gewinnen
 - Regel 2: Sub-Interfaces gewinnen
 - Regel 3: Methode explizit auswählen
- Unterschied zu abstrakten Klassen: **Kein Zustand**

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials [“Default Methods”](#) und [“Implementing an Interface”](#) nach.

Lernziele

- k2: Ich weiss, dass in Interfaces Default-Methoden erstellt werden können
- k2: Ich kann den Unterschied zwischen Interfaces mit Default-Methoden und abstrakten Klassen erklären
- k2: Ich verstehe das Problem der Mehrfachvererbung bei Interfaces mit Default-Methoden
- k3: Ich kann Interfaces mit Default-Methoden erstellen und einsetzen
- k3: Ich habe die Regeln zum Auflösen der Mehrfachvererbung verstanden und kann sie in der Praxis nutzen

Challenges

Erklären Sie die Code-Schnipsel in der [Vorgabe](#) und die jeweils entstehenden Ausgaben.

5.4. Record-Klassen

TL;DR

Häufig schreibt man relativ viel *Boiler Plate Code*, um einfach ein paar Daten plus den Konstruktor und die Zugriffsmethoden zu kapseln. Und selbst wenn die IDE dies zum Teil abnehmen kann - lesen muss man diesen Overhead trotzdem noch.

Für den Fall von Klassen mit **final** Attributen wurden in Java14 die **Record-Klassen** eingeführt. Statt dem Schlüsselwort **class** wird das neue Schlüsselwort **record** verwendet. Nach dem Klassennamen kommen in runden Klammern die “Komponenten” - eine Auflistung der Parameter für den Standardkonstruktor (Typ, Name). Daraus wird automatisch ein “kanonischer Konstruktor” mit exakt diesen Parametern generiert. Es werden zusätzlich **private final** Attribute generiert für jede Komponente, und diese werden durch den kanonischen Konstruktor gesetzt. Außerdem wird für jedes Attribut automatisch ein Getter mit dem Namen des Attributs generiert (also ohne den Präfix “get”).

Beispiel:

```
1 public record StudiR(String name, int credits) {}
```

Der Konstruktor und die Getter können überschrieben werden, es können auch eigene Methoden definiert werden (eigene Konstruktoren *müssen* den kanonischen Konstruktor aufrufen). Es gibt außer den über die Komponenten definierten Attribute keine weiteren Attribute. Da eine Record-Klasse intern von `java.lang.Record` ableitet, kann eine Record-Klasse nicht von weiteren Klassen ableiten (erben). Man kann aber beliebig viele Interfaces implementieren. Record-Klassen sind implizit final, d.h. man nicht von Record-Klassen erben.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demo: [\[YT\]](#), [\[HSBI\]](#)

5.4.1. Motivation; Klasse Studi

```
1 public class Studi {
2     private final String name;
3     private final int credits;
4
5     public Studi(String name, int credits) {
6         this.name = name;
7         this.credits = credits;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public int getCredits() {
15        return credits;
16    }
17 }
```

5.4.2. Klasse Studi als Record

```
1 public record StudiR(String name, int credits) {}
```

- Immutable Klasse mit Feldern `String name` und `int credits`
=> “(`String name`, `int credits`)” werden “Komponenten” des Records genannt
- Standardkonstruktor setzt diese Felder (“Kanonischer Konstruktor”)
- Getter für beide Felder:

```
1 public String name() { return this.name; }
2 public int credits() { return this.credits; }
```

Record-Klassen wurden in Java14 eingeführt und werden immer wieder in neuen Releases erweitert/ergänzt.

Der **kanonische Konstruktor** hat das Aussehen wie die Record-Deklaration, im Beispiel also `public StudiR (String name, int credits)`. Dabei werden die Komponenten über eine Kopie der Werte initialisiert.

Für die Komponenten werden automatisch private Attribute mit dem selben Namen angelegt.

Für die Komponenten werden automatisch Getter angelegt. Achtung: Die Namen entsprechen denen der Komponenten, es fehlt also der übliche "get"-Präfix!

5.4.3. Eigenschaften und Einschränkungen von Record-Klassen

- Records erweitern implizit die Klasse `java.lang.Record`:
Keine andere Klassen mehr erweiterbar! (Interfaces kein Problem)
- Record-Klassen sind implizit **final**
- Keine weiteren (Instanz-) Attribute definierbar (nur die Komponenten)
- Keine Setter definierbar für die Komponenten: Attribute sind **final**
- Statische Attribute mit Initialisierung erlaubt

5.4.4. Records: Prüfungen im Konstruktor

Der Konstruktor ist erweiterbar:

```
1 public record StudiS(String name, int credits) {  
2     public StudiS(String name, int credits) {  
3         if (name == null) { throw new IllegalArgumentException("Name cannot be  
4             null!"); }  
5         else { this.name = name; }  
6  
7         if (credits < 0) { this.credits = 0; }  
8         else { this.credits = credits; }  
9     }  
}
```

In dieser Form muss man die Attribute selbst setzen.

Alternativ kann man die "kompakte" Form nutzen:

```
1 public record StudiT(String name, int credits) {  
2     public StudiT {  
3         if (name == null) { throw new IllegalArgumentException("Name cannot be  
4             null!"); }  
5  
6         if (credits < 0) { credits = 0; }  
7     }  
}
```

In der kompakten Form kann man nur die Werte der Parameter des Konstruktors ändern. Das Setzen der Attribute ergänzt der Compiler nach dem eigenen Code.

Es sind weitere Konstruktoren definierbar, diese *müssen* den kanonischen Konstruktor aufrufen:

```
1 public StudiT() {  
2     this("", 42);  
3 }
```

5.4.5. Getter und Methoden

Getter werden vom Compiler automatisch generiert. Dabei entsprechen die Methoden-Namen den Namen der Attribute:

```
1 public record StudiR(String name, int credits) {}  
2  
3 public static void main(String... args) {  
4     StudiR r = new StudiR("Sabine", 75);  
5  
6     int x = r.credits();  
7     String y = r.name();  
8 }
```

Getter überschreibbar und man kann weitere Methoden definieren:

```
1 public record StudiT(String name, int credits) {  
2     public int credits() { return credits + 42; }  
3     public void wuppie() { System.out.println("WUPPIE"); }  
4 }
```

Die Komponenten/Attribute sind aber **final** und können nicht über Methoden geändert werden!

5.4.6. Weitere Eigenschaften von Records

Für Record-Klassen wird automatisch die Vergleichbarkeit implementiert, d.h. es wird automatisch eine passende Überschreibung für die von `Object` geerbten Methoden `equals` und `hashCode` erzeugt.

Die folgende Record-Klasse `Position`

```
1 public record Position(int x, int y) {}
```

würde sich im traditionellen Java so als **class** formulieren lassen:

```
1 public final class Position {  
2     private final int x;  
3     private final int y;  
4  
5     public Position(int x, int y) {  
6         this.x = x;  
7         this.y = y;  
8     }  
9  
10    public int x() {  
11        return x;  
12    }  
13 }
```

```

14     public int y() {
15         return y;
16     }
17
18     @Override
19     public boolean equals(Object obj) {
20         if (obj == this) return true;
21         if (obj == null || obj.getClass() != this.getClass()) return false;
22         var that = (Position) obj;
23         return this.x == that.x && this.y == that.y;
24     }
25
26     @Override
27     public int hashCode() {
28         return Objects.hash(x, y);
29     }
30 }

```

D.h. wir sparen uns durch die Deklaration als `record` ziemlich viel Boilerplate-Code und bekommen u.a. diese Dinge “geschenkt”:

- Felder `x` und `y`
- Standardkonstruktor zum Setzen beider Felder
- Getter `x()` und `y()`
- Überschreibung für `equals`, welche die Objekt-IDs, den Typ der Objekte und die Felder berücksichtigt
- Überschreibung für `hashCode`, welche die Felder der Klasse berücksichtigt

Dadurch funktioniert dann auch so etwas ohne Probleme:

```

1 List<Position> body;
2
3 ...
4
5 public boolean occupies(Position position) {
6     return body.contains(position);
7 }

```

Ohne korrekt überschriebenes `equals` würde sonst in der Liste bei `body.contains(position)` nur nach den Objekt-IDs verglichen, was in dem Fall ungünstig wäre - wir suchen ja nicht nur nach exakt dem selben Objekt, sondern allgemein nach einer identischen Position, also auch anderen Objekten mit identisch gesetzten Feldern `x` und `y`.

5.4.7. Beispiel aus den Challenges

In den Challenges zum Thema Optional gibt es die Klasse `Katze` in den [Vorgaben](#).

Die Katze wurde zunächst “klassisch” modelliert: Es gibt drei Eigenschaften `name`, `gewicht` und `lieblingsBox`. Ein Konstruktor setzt diese Felder und es gibt drei Getter für die einzelnen Eigenschaften. Das braucht 18 Zeilen Code (ohne Kommentare Leerzeilen). Zudem erzeugt der Boilerplate-Code relativ viel “visual noise”, so dass der eigentliche Kern der Klasse schwerer zu erkennen ist.

In einem Refactoring wurde diese Klasse durch eine äquivalente Record-Klasse ersetzt, die nur noch 2 Zeilen Code (je nach Code-Style auch nur 1 Zeile) benötigt. Gleichzeitig wurde die Les- und Wartbarkeit deutlich verbessert.

The screenshot shows a code editor with a file named `Katze.java` in the package `challenges.optional`. The original code (lines 3-29) defines a `Katze` class with private fields `name`, `gewicht`, and `lieblingsBox`, a constructor, and three getter methods. A diff view is overlaid on the bottom half of the code, showing the new `record` definition (lines 3-9) which is significantly shorter and uses `@param` annotations for documentation.

```

1  package challenges.optional;
2
3  - public class Katze {
4  -     private String name;
5  -     private float gewicht;
6  -     private Box LieblingsBox;
7  -
8  -     /**
9  -     * @param name Name der Katze
10 -     * @param gewicht Gewicht der Katze in kg
11 -     * @param LieblingsBox Lieblingsbox der Katze
12 -     */
13 -     public Katze(String name, float gewicht, Box LieblingsBox) {
14 -         this.name = name;
15 -         this.gewicht = gewicht;
16 -         this.LieblingsBox = LieblingsBox;
17 -     }
18 -
19 -     public String name() {
20 -         return name;
21 -     }
22 -
23 -     public float gewicht() {
24 -         return gewicht;
25 -     }
26 -
27 -     public Box LieblingsBox() {
28 -         return LieblingsBox;
29 -     }
30
31  + /**
32  + * @param name      Name der Katze
33  + * @param gewicht   Gewicht der Katze in kg
34  + * @param LieblingsBox Lieblingsbox der Katze
35  + */
36  + public record Katze(String name, float gewicht, Box LieblingsBox) {
37  +
38  + }

```

5.4.8. Wrap-Up

- Records sind immutable Klassen:
 - final** Attribute (entsprechend den Komponenten)
 - Kanonischer Konstruktor
 - Automatische Getter (Namen wie Komponenten)
- Konstruktoren und Methoden können ergänzt/überschrieben werden
- Keine Vererbung von Klassen möglich (kein **extends**)

Zum Nachlesen

Schöne Doku: [“Using Record to Model Immutable Data”](#).

Lernziele

- k2: Ich verstehe, dass Record-Klassen implizit final sind
- k2: Ich weiss, dass Record-Klassen einen kanonischen Konstruktor haben
- k2: Ich verstehe, dass die Attribute in Record-Klassen implizit final sind und automatisch angelegt und über den Konstruktor gesetzt werden
- k2: Ich weiss, dass die Getter in Record-Klassen so benannt sind wie die Namen der Komponenten, also keinen Präfix 'get' haben
- k2: Ich weiss, dass der kanonische Konstruktor ergänzt werden kann
- k2: Ich weiss, dass weitere Methoden definiert werden können
- k2: Ich verstehe, dass Record-Klassen nicht von anderen Klassen erben können, aber Interfaces implementieren können
- k3: Ich kann Record-Klassen praktisch einsetzen

Challenges

Betrachten Sie den folgenden Code:

```
1 public interface Person {
2     String getName();
3     Date getBirthday();
4 }
5
6 public class Student implements Person {
7     private static final SimpleDateFormat DATE_FORMAT = new
8         SimpleDateFormat("dd.MM.yyyy");
9
10    private final String name;
11    private final Date birthday;
12
13    public Student(String name, String birthday) throws ParseException {
14        this.name = name;
15        this.birthday = DATE_FORMAT.parse(birthday);
16    }
17
18    public String getName() { return name; }
19    public Date getBirthday() { return birthday; }
```

Schreiben Sie die Klasse `Student` in eine Record-Klasse um. Was müssen Sie zusätzlich noch tun, damit die aktuelle API erhalten bleibt?

5.5. Sealed-Typen & Pattern Matching

TL;DR

Pattern Matching in modernem Java nimmt Ihnen viel typischen Boilerplate ab: Statt erst mit **instanceof** den Typ zu prüfen und dann manuell zu casten, können Sie **Type Patterns** verwenden (**if** (**obj instanceof String s**) {...} oder **case IntLiteral lit -> ...** im **switch**). Mit **switch-Expressions** (**switch** (...) { **case ... -> ...** }) wird **switch** selbst zu einem Ausdruck, der einen Wert liefert, ohne **break** und Fall-Through, was den Code kürzer, sicherer und leichter lesbar macht.

Besonders nützlich wird das in Kombination mit **sealed Interfaces/Klassen und Records**: Sealed-Typen legen eine abgeschlossene Menge erlaubter Untertypen fest, sodass der Compiler prüfen kann, ob ein **switch** wirklich alle Fälle abdeckt (*exhaustive check*). Records modellieren reine Daten, und mit **Record-Patterns** können Sie diese Daten direkt im **switch** dekonstruieren (**case Point(int x, int y) -> ...**), ohne explizite Getter-Aufrufe.

Zusammen ermöglichen Ihnen **record + sealed + Pattern Matching** in Java einen deklarativen, gut typgesicherten Stil - ähnlich wie algebraische Datentypen in funktionalen Sprachen - mit deutlich weniger Fehlerquellen und Redundanz im Code.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

5.5.1. Was nervt Sie an diesem Code?

```

1 interface Expr {}
2 record IntLiteral(int value) implements Expr {}
3 record Add(Expr left, Expr right) implements Expr {}
4 record Negate(Expr inner) implements Expr {}
5
6 int evalOld(Expr e) {
7     if (e instanceof IntLiteral) {
8         IntLiteral lit = (IntLiteral) e;
9         return lit.value();
10    } else if (e instanceof Add) {
11        Add add = (Add) e;
12        return evalOld(add.left()) + evalOld(add.right());
13    } else if (e instanceof Negate) {
14        Negate neg = (Negate) e;
15        return -evalOld(neg.inner());
16    } else {
17        throw new IllegalArgumentException("Unknown expression: " + e);
18    }
19 }
```

Cast-Boilerplate, fehlende Compiler-Unterstützung (alle Fälle abgedeckt???), fragile **else**-Kette mit vielen **instanceof**, Redundanzen (Typprüfung plus Cast)

... die modernere Variante würde so gehen:

```

1  int evalWithInstanceofPattern(Expr e) {
2      if (e instanceof IntLiteral lit) {
3          return lit.value();
4      } else if (e instanceof Add add) {
5          return evalWithInstanceofPattern(add.left()) +
                  evalWithInstanceofPattern(add.right());
6      } else if (e instanceof Negate neg) {
7          return -evalWithInstanceofPattern(neg.inner());
8      } else {
9          throw new IllegalArgumentException("Unknown expression: " + e);
10     }
11 }

```

(Erklärung folgt in den nächsten Folien) ... und es geht noch besser!

5.5.2. Pattern Matching für instanceof

```

1  // vor Java 16:
2  if (obj instanceof String) {
3      String s = (String) obj;
4      if (s.length() > 5) {
5          IO.println(s.toUpperCase());
6      }
7  }

```

```

1  // modern:
2  if (obj instanceof String s && s.length() > 5) {
3      IO.println(s.toUpperCase());
4  }

```

- Type Pattern: `String s` ist direkt Teil der **instanceof**-Abfrage
- Pattern Variable `s` ist nur im **true**-Zweig sichtbar -> weniger Fehler, kein expliziter Cast

5.5.3. Switch Expressions

Rückblick: Das sollte Ihnen bekannt vorkommen:

```

1  switch (day) {
2      case MONDAY:
3      case FRIDAY:
4          IO.println("Almost weekend");
5          break;
6      default:
7          IO.println("Another day");
8  }

```

Moderne Variante als **switch-Expression**:

```

1 String message = switch (day) {
2     case MONDAY, FRIDAY -> "Almost weekend";
3     case SATURDAY, SUNDAY -> "Weekend!";
4     default -> "Another day";
5 };

```

Beobachtungen:

- Klassische Fall-Through-Variante: **case** xyz: ... **break**;

```

1 switch (x) {
2     case 1:
3         IO.println("eins");
4         // Gefahr: Fall-Through, wenn break fehlt
5     case 2:
6         IO.println("zwei");
7         break;
8     default:
9         IO.println("andere Zahl");
10 }

```

- Moderne Expression-Variante: **case** xyz -> ...

```

1 String result = switch (x) {
2     case 1 -> "eins";
3     case 2 -> "zwei";
4     default -> "andere Zahl";
5 };

```

- Kann **Expression** sein: **switch** (...) { ... } liefert einen **Wert**
- Kein Fall-Through: jeder **case** steht für sich; mehrere Werte können zusammengefasst werden:

```

1 String result = switch (x) {
2     case 1, 2 -> "kleine Zahl"; // mehrere Labels in einem Fall
3     case 3 -> "drei";
4     default -> "andere Zahl";
5 };

```

- Form:

- Single-Expression-Case: **case** ... -> expression;
- Block-Case: **case** ... -> { ...; returnWert; } via **yield**:

```

1 String result = switch (x) {
2     case 1 -> {
3         IO.println("Seiteneffekt");
4         yield "eins"; // 'yield' ist das 'return' für Switch-
                       Expressions
5     }
6     default -> "andere Zahl";
7 };

```

Vorteile:

- Kein **break** mehr, kein Fall-Through
- Switch liefert einen **Wert** (im Beispiel: `String message`)
- Scoping ist klarer: der Body hinter `->` ist ein eigener Block (besonders gut sichtbar bei `{ ... }`), lokale Variablen sind entsprechend begrenzt sichtbar
- Arrow-Syntax `->` fördert Expression-Style (passt gut zu Lambdas/Streams)

5.5.4. Type Patterns im switch

Unsere Klassenhierarchie wie oben, aber diesmal mit **sealed Interface**:

```
1 sealed interface Expr permits IntLiteral, Add, Negate {}
2
3 record IntLiteral(int value) implements Expr {}
4 record Add(Expr left, Expr right) implements Expr {}
5 record Negate(Expr inner) implements Expr {}
```

Bei einem **sealed Interface** kann ich angeben, **wer dieses Interface implementiert**. Andere Klassen als die in der `permits`-Klausel angegebenen Klassen dürfen das Interface nicht implementieren (und die angegebenen **müssen**).

Mit **sealed Interfaces** kann ich nun ein **switch** über den **Typ** von `Expr` machen:

```
1 int eval(Expr e) {
2     return switch (e) {
3         case IntLiteral lit -> lit.value();
4         case Add add -> eval(add.left()) + eval(add.right());
5         case Negate neg -> -eval(neg.inner());
6     };
7 }
```

5.5.4.1. Wichtige Punkte:

- **case** `IntLiteral lit` ist ein **Type Pattern**: Typprüfung + Cast + Bindung in einem Schritt
 - Type Patterns funktionieren für alle Referenztypen (nicht nur für Records)
 - Auch **case null** funktioniert wie erwartet
- Kein **default** nötig wegen **sealed**-Hierarchy: Hier kann der Compiler prüfen, ob alle erlaubten Subtypen (`IntLiteral`, `Add`, `Negate`) abgedeckt sind `->` **exhaustive switch**
- Begrenzte Vererbung: Nur die angegebenen Typen (`IntLiteral`, `Add`, `Negate`) dürfen `Expr` implementieren/erweitern

Tip

Type Pattern (z.B. `case IntLiteral lit -> ...`) kann für **alle Referenztypen** verwendet werden, auch ohne **sealed**.

Exhaustive switch bekommt man nur, wenn die Hierarchie "geschlossen" ist (**sealed/enum/record**).

Tip

Wenn Sie eine neue `record Multiply(Expr left, Expr right) implements Expr {}` hinzufügen, wird der Compiler meckern:

- wenn sie die `permits`-Klausel des sealed Interface nicht ergänzen (**begrenzte Vererbung**)
- wenn Sie das Pattern im `switch` nicht ergänzen (**exhaustive switch**)

Damit bekommen wir eine deutlich bessere Compiler-Unterstützung bei Änderungen: Wenn neue Varianten hinzugefügt werden, werden alle relevanten `switch`-Stellen gefunden und geprüft. Das resultiert in einem besseren Typsicherheits-Check im Vergleich mit einem `default`-Case, der stillschweigend die neuen Varianten subsummieren würde. (Dies ist ähnlich wie bei Algebraischen Datentypen in funktionalen Sprachen.)

5.5.4.2. Weiterer Nutzen von sealed (jenseits von switch)

- Modellierung: Sie drücken im Typ-System aus: “Diese Menge von Subtypen ist abgeschlossen.” Das ist fachlich sinnvoll (z.B. `Shape = Circle | Rectangle | Square`), nicht nur ein Compiler-Trick.
- Sicherheit/Kapselung: Bibliotheksautoren können verhindern, dass fremder Code beliebige zusätzliche Implementierungen einschmuggelt.
- Lesbarkeit / Wartbarkeit: Andere Entwickler sehen sofort, welche Untertypen es geben darf, ohne das ganze Projekt durchsuchen zu müssen.

Tip

Sealed-Typen sind die Java-Variante von “abgeschlossenen Summentypen” / ADTs (*algebraic data types*). Sie helfen dem Compiler beim Pattern Matching und helfen Ihnen beim Modellieren einer endlichen Menge von Varianten.

5.5.4.3. Hinweis:

Wenn Sie die lange `permits`-Aufzählung stört, können Sie die Klassen auch direkt im `sealed`-Interface definieren:

```
1 sealed interface Expr {
2     record IntLiteral(int value) implements Expr {}
3     record Add(Expr left, Expr right) implements Expr {}
4     record Negate(Expr inner) implements Expr {}
5 }
```

Nachteil: Statt `case Add add ->` muss es dann `case Expr.Add add ->` heißen ...

5.5.5. Guarded Patterns im switch

```
1 int sign(Expr e) {
2     return switch (e) {
3         case Expr.IntLiteral lit when lit.value() > 0 -> 1;
```

```

4     case Expr.IntLiteral lit when lit.value() < 0 -> -1;
5     case Expr.IntLiteral lit -> 0;
6     default -> throw new IllegalArgumentException("Not a literal: " + e);
7 };
8 }

```

- `case Expr.IntLiteral lit when lit.value() > 0` ist ein Pattern mit zusätzlicher Bedingung (*Guard*)
- Lesbarkeit oft besser als verschachtelte `if`-Blöcke

Caution

Achtung: Im `switch`-Pattern-Matching wird die Zusatzbedingung mit `when` geschrieben (nicht mit `&&` wie bei `if`).

5.5.6. Record-Patterns / Dekonstruktion

In einem Spiel könnte ein Punkt so modelliert werden:

```

1 record Point(int x, int y) {}

```

Wenn ich in älteren Java-Versionen die Manhattan-Distanz eines Objekts zum Ursprung bestimmen wollte, dann musste ich nach dem Typ des Objekts fragen und entweder (falscher Typ) eine Exception werfen oder (korrekter Typ, also `Point`) einen Cast machen und dann den Abstand ausrechnen und dabei die Attribute des `Point` per Getter abfragen:

```

1 static int manhattan(Object o) {
2     if (o instanceof Point) {
3         Point p = (Point) o;
4         return Math.abs(p.x()) + Math.abs(p.y());
5     }
6     throw new IllegalArgumentException("Not a point: " + o);
7 }

```

Mit Pattern Matching und Record-Pattern kann ich das alles elegant in einem `switch` machen:

```

1 static int manhattan(Object o) {
2     return switch (o) {
3         case Point(int x, int y) -> Math.abs(x) + Math.abs(y);
4         default -> throw new IllegalArgumentException("Not a point: " + o);
5     };
6 }

```

- Im `switch` wird nach dem konkreten Typ von `o` geschaut:
 - kein separates `instanceof` notwendig
 - kein separater Cast notwendig
- `case Point(int x, int y)` dekonstruiert das Record:
 - Verwendet implizit den (kanonischen) Konstruktor von `Point`

- Felder werden direkt in lokale Variablen gemappt
- Kein expliziter Getter-Aufruf mehr im Body nötig
- Das Dekonstruieren klappt für Record-Typen auch im **instanceof**: `o instanceof Point(int x, int y)`
- Derzeit klappt das leider nur mit den **kanonischen Konstruktoren** der Record-Klassen!

Tip

Das geht auch verschachtelt:

```
1 sealed interface Command permits Move, DrawLine, SetColor {}
2
3 record Move(Point to) implements Command {}
4 record DrawLine(Point from, Point to) implements Command {}
5 record SetColor(int r, int g, int b) implements Command {}
```

Nun ein Switch mit verschachtelten Record-Patterns:

```
1 void handle(Command cmd) {
2     switch (cmd) {
3         case Move(Point(int x, int y)) -> IO.println("Move cursor to " + x
4             + "," + y);
5         case DrawLine(Point(int x1, int y1), Point(int x2, int y2)) -> IO.
6             println("Draw line from " + x1 + "," + y1 + " to " + x2 + "," +
7                 y2);
8         case SetColor(int r, int g, int b) -> IO.println("Set color to RGB(
9             " + r + "," + g + "," + b + ")");
10    }
11 }
```

Hier sieht man:

- Nested Patterns: `Move(Point(int x, int y))` -> sofort Zugriff auf `x, y`
- Kombiniert mit `sealed` bekommen Sie wieder einen **exhaustive** Switch ohne **default**

Important

Record-Patterns dekonstruieren tatsächlich nur Records (also Klassen, die mit `record` deklariert wurden) Für “normale” Klassen (klassisches `class`) gibt es noch keine allgemeine Dekonstruktion per Pattern. (Stand Java 25)

5.5.7. Verbindung zu “funktionalem Stil”

- Pattern Matching + Switch-Expressions ermöglichen **ausdrucksbasierten** Stil, ähnlich wie in funktionalen Sprachen (Haskell, Scala, ...)
- Dadurch lassen sich Datenstrukturen (Records, Sealed Hierarchies) klar und knapp “entpacken”
- Die Kombination aus:
 - `record` (für Daten),
 - `sealed` (für abgeschlossene Hierarchien),

- Pattern Matching (**instanceof**, **switch**, Record-Patterns) erlaubt einen deklarativen Stil, ideal z.B. in Kombination mit Streams/Optionals

Beispiel: Liste von `Expr` filtern und auswerten (nur Literale):

```
1 List<Expr> exprs = List.of(  
2     new IntLiteral(1),  
3     new Add(new IntLiteral(2), new IntLiteral(3)),  
4     new Negate(new IntLiteral(4))  
5 );  
6  
7 int sumOfLiterals = exprs.stream()  
8     .mapToInt(e -> switch (e) {  
9         case IntLiteral lit -> lit.value();  
10        default -> 0; // neutrales Element bzgl. der Summe  
11    })  
12    .sum();
```

5.5.8. Wrap-Up

Für Datenhierarchien (z.B. `Expr`, `Shape`, `Command`) nutzen Sie heute:

- **record** für Daten,
- **sealed** für eine abgeschlossene Variantenmenge, und
- Pattern Matching im modernen **switch** (mit Arrow-Syntax) für knappen, typsicheren und gut wartbaren Code im (teilweise) funktionalen Stil.

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials “[Using Pattern Matching](#)” (Oracle) nach. Das Thema Pattern Matching ist aktuell in aktiver Entwicklung. Einige Features haben es bereits in die verschiedenen Java-Releases geschafft, andere stecken aktuell noch in der Pipeline. Halten Sie die Augen offen - es kann auch passieren, dass bereits verabschiedete Syntax nachträglich noch einmal zurückgenommen und geändert wird. Das [Project Amber](#) ist die zentrale Stelle für alle Entwicklungen rund um Pattern Matching in Java.

Lernziele

- k2: Ich kann erklären, was **sealed** Typen, Records und Pattern Matching in Java leisten.
- k3: Ich kann einfache **sealed** Hierarchien mit Records modellieren und per **switch**-Expression auswerten.

Challenges

Definieren Sie eine kleine Shape-Hierarchie mit **sealed** und berechnen Sie Fläche/Umfang per **switch** + Record-Patterns.

5.6. Stream-API

TL;DR

Mit der Collection-API existiert in Java die Möglichkeit, Daten auf verschiedenste Weisen zu speichern (`Collection<T>`). Mit der Stream-API gibt es die Möglichkeit, diese Daten in einer Art Pipeline zu verarbeiten. Ein `Stream<T>` ist eine Folge von Objekten vom Typ `T`. Die Verarbeitung der Daten ist "lazy", d.h. sie erfolgt erst auf Anforderung (durch die terminale Operation).

Ein Stream hat eine Datenquelle und kann beispielsweise über `Collection#stream()` oder `Stream.of()` angelegt werden. Streams speichern keine Daten. Die Daten werden aus der verbundenen Datenquelle geholt.

Auf einem Stream kann man eine Folge von intermediären Operationen wie `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` ... durchführen. Alle diese Operationen arbeiten auf dem Stream und erzeugen einen neuen Stream als Ergebnis. Dadurch kann die typische Pipeline-artige Verkettung der Operationen ermöglicht werden. Die intermediären Operationen werden erst ausgeführt, wenn der Stream durch eine terminale Operation geschlossen wird.

Terminale Operationen wie `count()`, `forEach()`, `allMatch()` oder `collect()`

- `collect(Collectors.toList())` (bzw. direkt mit `stream.toList()` (ab Java16))
- `collect(Collectors.toSet())`
- `collect(Collectors.toCollection(LinkedList::new))` (als `Supplier<T>`)

stoßen die Verarbeitung des Streams an und schließen den Stream damit ab.

Wir können hier nur die absoluten Grundlagen betrachten. Die Stream-API ist sehr groß und mächtig und lohnt die weitere selbstständige Auseinandersetzung :-)

Videos

Vorlesung [YT], [HSBI]

Demo:

- Demo Stream-API [YT], [HSBI]
- Vordefinierte funktionale Interfaces im JDK [YT], [HSBI]

5.6.1. Motivation

Es wurden Studis, Studiengänge und Fachbereiche modelliert (aus Gründen der Übersichtlichkeit einfach als Record-Klassen).

Nun soll pro Fachbereich die Anzahl der Studis ermittelt werden, die bereits 100 ECTS oder mehr haben. Dazu könnte man über alle Studiengänge im Fachbereich iterieren, und in der inneren Schleife über alle Studis im Studiengang. Dann filtert man alle Studis, deren ECTS größer 100 sind und erhöht jeweils den Zähler:

```
1 public record Studi(String name, int credits) {}
2 public record Studiengang(String name, List<Studi> studis) {}
3 public record Fachbereich(String name, List<Studiengang> studiengaenge) {}
4
```

```
5 private static long getCountFB(Fachbereich fb) {  
6     long count = 0;  
7     for (Studiengang sg : fb.studiengaenge()) {  
8         for (Studi s : sg.studis()) {  
9             if (s.credits() > 100) count += 1;  
10        }  
11    }  
12    return count;  
13 }
```

Dies ist ein Beispiel, welches klassisch in OO-Manier als Iteration über Klassen realisiert ist. (Inhaltlich ist es vermutlich nicht sooo sinnvoll.)

5.6.2. Innere Schleife mit Streams umgeschrieben

```
1 private static long getCountSG(Studiengang sg) {  
2     return sg.studis().stream()  
3         .map(Studi::credits)  
4         .filter(c -> c > 100)  
5         .count();  
6 }  
7  
8 private static long getCountFB2(Fachbereich fb) {  
9     long count = 0;  
10    for (Studiengang sg : fb.studiengaenge()) {  
11        count += getCountSG(sg);  
12    }  
13    return count;  
14 }
```

5.6.2.1. Erklärung des Beispiels

Im Beispiel wurde die innere Schleife in einen Stream ausgelagert.

Mit der Methode `Collection#stream()` wird aus der Collection ein neuer Stream erzeugt. Auf diesem wird für jedes Element durch die Methode `map()` die Methode `Studi#credits()` angewendet, was aus einem Strom von `Studi` einen Strom von `Integer` macht. Mit `filter()` wird auf jedes Element das Prädikat `c -> c > 100` angewendet und alle Elemente aus dem Strom entfernt, die der Bedingung nicht entsprechen. Am Ende wird mit `count()` gezählt, wie viele Elemente im Strom enthalten sind.

5.6.2.2. Was ist ein Stream?

Ein "Stream" ist ein Strom (Folge) von Daten oder Objekten. In Java wird die Collections-API für die Speicherung von Daten (Objekten) verwendet. Die Stream-API dient zur Iteration über diese Daten und entsprechend zur Verarbeitung der Daten. In Java speichert ein Stream keine Daten.

Das Konzept kommt aus der funktionalen Programmierung und wurde in Java nachträglich eingebaut (wobei dieser Prozess noch lange nicht abgeschlossen zu sein scheint).

In der funktionalen Programmierung kennt man die Konzepte “map”, “filter” und “reduce”: Die Funktion “map()” erhält als Parameter eine Funktion und wendet diese auf alle Elemente eines Streams an. Die Funktion “filter()” bekommt ein Prädikat als Parameter und prüft jedes Element im Stream, ob es dem Prädikat genügt (also ob das Prädikat mit dem jeweiligen Element zu **true** evaluiert - die anderen Objekte werden entfernt). Mit “reduce()” kann man Streams zu einem einzigen Wert zusammenfassen (denken Sie etwa an das Aufsummieren aller Elemente eines Integer-Streams). Zusätzlich kann man in der funktionalen Programmierung ohne Probleme unendliche Ströme darstellen: Die Auswertung erfolgt nur bei Bedarf und auch dann auch nur so weit wie nötig. Dies nennt man auch “*lazy evaluation*”.

Die Streams in Java versuchen, diese Konzepte aus der funktionalen Programmierung in die objektorientierte Programmierung zu übertragen. Ein Stream in Java hat eine Datenquelle, von wo die Daten gezogen werden - ein Stream speichert selbst keine Daten. Es gibt “intermediäre Operationen” auf einem Stream, die die Elemente verarbeiten und das Ergebnis als Stream zurückliefern. Daraus ergibt sich typische Pipeline-artige Verkettung der Operationen. Allerdings werden diese Operationen erst durchgeführt, wenn eine “terminale Operation” den Stream “abschließt”. Ein Stream ohne eine terminale Operation macht also tatsächlich *nichts*.

Die Operationen auf dem Stream sind üblicherweise zustandslos, können aber durchaus auch einen Zustand haben. Dies verhindert üblicherweise die parallele Verarbeitung der Streams. Operationen sollten aber nach Möglichkeit keine *Seiteneffekte* haben, d.h. keine Daten außerhalb des Streams modifizieren. Operationen dürfen auf keinen Fall die Datenquelle des Streams modifizieren!

5.6.3. Erzeugen von Streams

```
1 List<String> l1 = List.of("Hello", "World", "foo", "bar", "wuppie");
2 Stream<String> s1 = l1.stream();
3
4 Stream<String> s2 = Stream.of("Hello", "World", "foo", "bar", "wuppie");
5
6 Random random = new Random();
7 Stream<Integer> s3 = Stream.generate(random::nextInt);
8
9 Pattern pattern = Pattern.compile(" ");
10 Stream<String> s4 = pattern.splitAsStream("Hello world! foo bar wuppie!");
```

Dies sind möglicherweise die wichtigsten Möglichkeiten, in Java einen Stream zu erzeugen.

Ausgehend von einer Klasse aus der Collection-API kann man die Methode `Collection#stream()` aufrufen und bekommt einen seriellen Stream.

Alternativ bietet das Interface `Stream` verschiedene statische Methoden wie `Stream.of()` an, mit deren Hilfe Streams angelegt werden können. Dies funktioniert auch mit Arrays ...

Und schließlich kann man per `Stream.generate()` einen Stream anlegen, wobei als Argument ein “Supplier” (Interface `java.util.function.Supplier<T>`) übergeben werden muss. Dieses Argument wird dann benutzt, um die Daten für den Stream zu generieren.

Wenn man aufmerksam hinschaut, findet man an verschiedensten Stellen die Möglichkeit, die Daten per Stream zu verarbeiten, u.a. bei regulären Ausdrücken.

Man kann per `Collection#parallelStream()` auch parallele Streams erzeugen, die intern das "Fork&Join-Framework" nutzen. Allerdings sollte man nur dann parallele Streams anlegen, wenn dadurch tatsächlich Vorteile durch die Parallelisierung zu erwarten sind (Overhead!).

5.6.4. Intermediäre Operationen auf Streams

```
1 private static void dummy(Studiengang sg) {
2     sg.studis().stream()
3         .peek(s -> System.out.println("Looking at: " + s.name()))
4         .map(Studi::credits)
5         .peek(c -> System.out.println("This one has: " + c + " ECTS"))
6         .filter(c -> c > 5)
7         .peek(c -> System.out.println("Filtered: " + c))
8         .sorted()
9         .forEach(System.out::println);
10 }
```

An diesem (weitestgehend sinnfreien) Beispiel werden einige intermediäre Operationen demonstriert.

Die Methode `peek()` liefert einen Stream zurück, der aus den Elementen des Eingabestroms besteht. Auf jedes Element wird die Methode `void accept(T)` des `Consumer<T>` angewendet (Argument der Methode), was aber nicht zu einer Änderung der Daten führt. **Hinweis:** Diese Methode dient vor allem zu Debug-Zwecken! Durch den Seiteneffekt kann die Methode eine schlechtere Laufzeit zur Folge haben oder sogar eine sonst mögliche parallele Verarbeitung verhindern oder durch eine parallele Verarbeitung verwirrende Ergebnisse zeigen!

Die Methode `map()` liefert ebenfalls einen Stream zurück, der durch die Anwendung der Methode `R apply(T)` der als Argument übergebenen `Function<T, R>` auf jedes Element des Eingabestroms entsteht. Damit lassen sich die Elemente des ursprünglichen Streams verändern; für jedes Element gibt es im Ergebnis-Stream ebenfalls ein Element (der Typ ändert sich, aber nicht die Anzahl der Elemente).

Mit der Methode `filter()` wird ein Stream erzeugt, der alle Objekte des Eingabe-Streams enthält, auf denen die Anwendung der Methode `boolean test(T)` des Arguments `Predicate<T>` zu `true` evaluiert (der Typ und Inhalt der Elemente ändert sich nicht, aber die Anzahl der Elemente).

Mit `sorted()` wird ein Stream erzeugt, der die Elemente des Eingabe-Streams sortiert (existiert auch mit einem `Comparator<T>` als Parameter).

Diese Methoden sind alles **intermediäre** Operationen. Diese arbeiten auf einem Stream und erzeugen einen neuen Stream und werden erst dann ausgeführt, wenn eine terminale Operation den Stream abschließt.

Dabei sind die gezeigten intermediären Methoden bis auf `sorted()` ohne inneren Zustand. `sorted()` ist eine Operation mit innerem Zustand (wird für das Sortieren benötigt). Dies kann ordentlich in Speicher und Zeit zuschlagen und u.U. nicht/nur schlecht parallelisierbar sein. Betrachten Sie den fiktiven parallelen Stream `stream.parallel().sorted().skip(42)`: Hier müssen erst *alle* Elemente sortiert werden, bevor mit `skip(42)` die ersten 42 Elemente entfernt werden. Dies kann auch nicht mehr parallel durchgeführt werden.

Die Methode `forEach()` schließlich ist eine **terminale** Operation, die auf jedes Element des Eingabe-Streams die Methode `void accept(T)` des übergebenen `Consumer<T>` anwendet. Diese Methode ist eine **terminale Operation**, d.h. sie führt zur Auswertung der anderen *intermediären* Operationen und schließt den Stream ab.

5.6.5. Was tun, wenn eine Methode Streams zurückliefert

Wir konnten vorhin nur die innere Schleife in eine Stream-basierte Verarbeitung umbauen. Das Problem ist: Die äußere Schleife würde einen Stream liefern (Stream von Studiengängen), auf dem wir die `map`-Funktion anwenden müssten und darin dann für jeden Studiengang einen (inneren) Stream mit den Studis eines Studiengangs verarbeiten müssten.

```

1 private static long getCountSG(Studiengang sg) {
2     return sg.studis().stream().map(Studi::credits).filter(c -> c > 100).count
3     ();
4 }
5 private static long getCountFB2(Fachbereich fb) {
6     long count = 0;
7     for (Studiengang sg : fb.studiengaenge()) {
8         count += getCountSG(sg);
9     }
10    return count;
11 }

```

Dafür ist die Methode `flatMap()` die Lösung. Diese Methode bekommt als Argument ein Objekt vom Typ `Function<? super T, ? extends Stream<? extends R>>` mit einer Methode `Stream<? extends R> apply(T)`. Die Methode `flatMap()` verarbeitet den Stream in zwei Schritten:

1. Mappe über alle Elemente des Eingabe-Streams mit der Funktion. Im Beispiel würde also aus einem `Stream<Studiengang>` jeweils ein `Stream<Stream<Studi>>`, also alle `Studiengang`-Objekte werden durch je ein `Stream<Studi>`-Objekt ersetzt. Wir haben jetzt also einen Stream von `Stream<Studi>`-Objekten, also einen `Stream<Stream<Studi>>`.
2. "Klopfe den verschachtelten Stream wieder flach", d.h. nimm die einzelnen `Studi`-Objekte aus den `Stream<Studi>`-Objekten und setze diese stattdessen in den Stream. Das Ergebnis ist dann wie gewünscht ein `Stream<Studi>` (Stream mit `Studi`-Objekten).

```

1 private static long getCountFB3(Fachbereich fb) {
2     return fb.studiengaenge().stream()
3         .flatMap(sg -> sg.studis().stream())
4         .map(Studi::credits)
5         .filter(c -> c > 100)
6         .count();
7 }

```

Zum direkten Vergleich hier noch einmal der ursprüngliche Code mit zwei verschachtelten Schleifen und entsprechenden Hilfsvariablen:

```

1 private static long getCountFB(Fachbereich fb) {
2     long count = 0;
3     for (Studiengang sg : fb.studiengaenge()) {
4         for (Studi s : sg.studis()) {
5             if (s.credits() > 100) count += 1;
6         }
7     }
8     return count;
9 }

```

Während `map` also eine Funktion $f : T \mapsto R$ auf alle Elemente des Streams anwendet und so aus einem `stream<T>` einen `stream<R>` erzeugt, wendet `flatMap` eine Funktion $f : T \mapsto \text{stream}<R>$ auf alle Elemente des Streams an und packt die Ergebnis-Streams `stream<R>` wieder aus, weshalb man im Ergebnis wie bei `map` aus einem `stream<T>` einen `stream<R>` erhält.

5.6.6. Streams abschließen: Terminale Operationen

```
1 Stream<String> s = Stream.of("Hello", "World", "foo", "bar", "wuppie");
2
3 long count = s.count();
4 s.forEach(System.out::println);
5 String first = s.findFirst().get();
6 Boolean b = s.anyMatch(e -> e.length() > 3);
7
8 List<String> s1 = s.collect(Collectors.toList());
9 List<String> s2 = s.toList(); // ab Java16
10 Set<String> s3 = s.collect(Collectors.toSet());
11 List<String> s4 = s.collect(Collectors.toCollection(LinkedList::new));
```

Streams müssen mit **einer terminalen Operation** abgeschlossen werden, damit die Verarbeitung tatsächlich angestoßen wird (*lazy evaluation*).¹

Es gibt viele verschiedene terminale Operationen. Wir haben bereits `count()` und `forEach()` gesehen. In der Sitzung zu “Optionals” werden wir noch `findFirst()` näher kennenlernen.

Daneben gibt es beispielsweise noch `allMatch()`, `anyMatch()` und `noneMatch()`, die jeweils ein Prädikat testen und einen Boolean zurückliefern (matchen alle, mind. eines oder keines der Objekte im Stream).

Mit `min()` und `max()` kann man sich das kleinste und das größte Element des Streams liefern lassen. Beide Methoden benötigen dazu einen `Comparator<T>` als Parameter.

Mit der Methode `collect()` kann man eine der drei Methoden aus `Collectors` über den Stream laufen lassen und eine `Collection` erzeugen lassen:

1. `toList()` sammelt die Elemente in ein `List`-Objekt (bzw. direkt mit `stream.toList()` (ab Java16))
2. `toSet()` sammelt die Elemente in ein `Set`-Objekt
3. `toCollection()` sammelt die Elemente durch Anwendung der Methode `T get()` des übergebenen `Supplier<T>`-Objekts auf

Die ist nur die sprichwörtliche “Spitze des Eisbergs”! Es gibt viele weitere Möglichkeiten, sowohl bei den intermediären als auch den terminalen Operationen. Schauen Sie in die Dokumentation!

[Demo: streams.Demo](#)

¹Anmerkung: Das obige Beispiel dient als Überblick gebräuchlicher terminaler Operationen, es ist nicht als lauffähiges Programm gedacht! Auf einem Stream kann immer nur **eine** terminale Operation ausgeführt werden - d.h. nach der Ausführung von `s.count()` wäre der Stream `s` verarbeitet und es können keine weiteren Operationen auf diesem Stream durchgeführt werden. Dito für die anderen gezeigten terminalen Operationen.

5.6.7. Spielregeln

- Operationen dürfen nicht die Stream-Quelle modifizieren
- Operationen können die Werte im Stream ändern (`map`) oder die Anzahl (`filter`)
- Keine Streams in Attributen/Variablen speichern oder als Argumente übergeben: Sie könnten bereits “gebraucht” sein!
=> Ein Stream sollte immer sofort nach der Erzeugung benutzt werden
- Operationen auf einem Stream sollten keine Seiteneffekte (Veränderungen von Variablen/Attributen außerhalb des Streams) haben (dies verhindert u.U. die parallele Verarbeitung)

5.6.8. Wrap-Up

`Stream<T>`: Folge von Objekten vom Typ `T`, Verarbeitung “lazy” (Gegenstück zu `Collection<T>`: Dort werden Daten **gespeichert**, hier werden Daten **verarbeitet**)

Tip

Fließband-Metapher

Einen Stream kann man sich vielleicht wie ein Fließband in einer Fabrik vorstellen: Die Daten werden auf dem Fließband in eine Richtung transportiert und durchlaufen verschiedene Stationen, wo auf den Daten gearbeitet wird. In manchen Stationen werden Objekte vom Fließband geschubst (Daten herausgefiltert), in manchen Stationen werden die Objekte bearbeitet (Daten verändert), in manchen Stationen werden aus mehreren Teilen neue Objekte gebaut ...

Es ist nur eine Metapher! Sie endet spätestens damit, dass die Streams *lazy* sind und dass sämtliche Operationen erst dann ausgeführt werden, wenn eine terminale Operation den Stream abschließt.

- Neuen Stream anlegen: `Collection#stream()` oder `Stream.of()` ...
- Intermediäre Operationen: `peek()`, `map()`, `flatMap()`, `filter()`, `sorted()` ...
- Terminale Operationen: `count()`, `forEach()`, `allMatch()`, `collect()` ...
 - `collect(Collectors.toList())`
 - `collect(Collectors.toSet())`
 - `collect(Collectors.toCollection())` (mit `Supplier<T>`)
- Streams speichern keine Daten
- Intermediäre Operationen laufen erst bei Abschluss des Streams los
- Terminale Operation führt zur Verarbeitung und Abschluss des Streams

Zum Nachlesen

Schöne Doku: “[The Stream API](#)”, und auch “[Package java.util.stream](#)”.

Lernziele

- k2: Ich verstehe, dass Streams die Daten nicht sofort verarbeiten ('lazy' Verarbeitung)
- k2: Ich verstehe, dass ich mit `map()` den Typ (und Inhalt) von Objekten im Stream, aber nicht die Anzahl verändere
- k2: Ich verstehe, dass ich mit `filter()` die Anzahl der Objekte im Stream, aber nicht deren Typ (und Inhalt) verändere
- k2: Ich verstehe, warum Streams nicht in Attributen gehalten oder als Parameter herumgereicht werden sollten
- k3: Ich kann einen Stream erzeugen
- k3: Ich kann verschiedene intermediäre Operationen verketteten
- k3: Ich kann mit einer terminalen Operation einen Stream abschließen und damit die Berechnung durchführen
- k3: Ich kann `flatMap()` einsetzen

Challenges

Betrachten Sie den folgenden Java-Code:

```
1 record Cat(int weight){};
2
3 public class Main {
4     public static void main(String... args) {
5         List<Cat> clouder = new ArrayList<>();
6         clouder.add(new Cat(100)); clouder.add(new Cat(1)); clouder.add(
7             new Cat(10));
8
9         sumOverWeight(8, clouder);
10    }
11
12    private static int sumOverWeight(int threshold, List<Cat> cats) {
13        int result = 0;
14        for (Cat c : cats) {
15            int weight = c.weight();
16            if (weight > threshold) {
17                result += weight;
18            }
19        }
20        return result;
21    }
```

Schreiben Sie die Methode `sumOverWeight` unter Beibehaltung der Funktionalität so um, dass statt der `for`-Schleife und der `if`-Abfrage Streams und Stream-Operationen eingesetzt werden. Nutzen Sie passende Lambda-Ausdrücke und nach Möglichkeit Methodenreferenzen.

Betrachten Sie den folgenden Java-Code:

```
1 public class Main {
2     public static String getParameterNamesJson(String[] parameterNames) {
3         if (parameterNames.length == 0) {
4             return "[]";
5         }
6     }
7 }
```

```

5      }
6
7      StringBuilder sb = new StringBuilder();
8      sb.append("[");
9      for (int i = 0; i < parameterNames.length; i++) {
10         if (i > 0) {
11             sb.append(", ");
12         }
13         sb.append("\"").append(escapeJson(parameterNames[i])).append(
14             "\"");
15     }
16     sb.append("]");
17     return sb.toString();
18
19     private static String escapeJson(String parameterName) {
20         // does something or another ...
21     }
22 }

```

Schreiben Sie die Methode `getParameterNamesJson` unter Beibehaltung der Funktionalität so um, dass statt der **for**-Schleife und der **if**-Abfrage Streams und Stream-Operationen eingesetzt werden. Nutzen Sie passende Lambda-Ausdrücke und nach Möglichkeit auch Methodenreferenzen.

5.7. Fehlerbehandlung mit Optional und Result

TL;DR

Häufig hat man in Methoden den Fall, dass es keinen Wert gibt, und man liefert dann **null** als “kein Wert vorhanden” zurück. Dies führt dazu, dass die Aufrufer eine entsprechende **null**-Prüfung für die Rückgabewerte durchführen müssen, bevor sie das Ergebnis nutzen können.

Optional schließt elegant den Fall “kein Wert vorhanden” ein: Es kann mit der Methode **Optional.ofNullable()** das Argument in ein **Optional** verpacken (Argument **!= null**) oder ein **Optional.empty()** zurückliefern (“leeres” **Optional**, wenn Argument **== null**).

Man kann **Optionals** prüfen mit **isEmpty()** und **ifPresent()** und dann direkt mit **ifPresent()**, **orElse()** und **orElseThrow()** auf den verpackten Wert zugreifen. Besser ist aber der Zugriff über die stream-ähnlichen Methoden von **Optional**: **map()**, **filter**, **flatMap()**, ... Dabei gibt es keine terminalen Operationen - es handelt sich ja auch nicht um einen Stream, nur die Optik erinnert daran.

Optional ist vor allem für Rückgabewerte gedacht, die den Fall “kein Wert vorhanden” einschließen sollen. Attribute, Parameter und Sammlungen sollten nicht **Optional**-Referenzen speichern, sondern “richtige” (unverpackte) Werte (und eben zur Not **null**). **Optional** ist kein Ersatz für **null**-Prüfung von Methoden-Parametern (nutzen Sie hier beispielsweise passende Annotationen). **Optional** ist auch kein Ersatz für vernünftiges Exception-Handling im Fall, dass etwas Unerwartetes passiert ist. Liefern Sie **niemals null** zurück, wenn der Rückgabotyp der Methode ein **Optional** ist!

Result<E, R> kapselt entweder ein erfolgreiches Ergebnis **R** oder einen Fehler **E** und macht damit im Typ explizit sichtbar, dass und welche Fehler auftreten können. Im Unterschied zu **Optional<T>**, das nur zwischen “Wert da” und “kein Wert” unterscheidet, erlaubt **Result<E, R>** die genaue Modellierung und

Weitergabe von Fehlersituationen und lässt sich mit `map/flatMap` sehr elegant ähnlich wie ein Stream verketteten.

Videos

Vorlesung [YT], [HSBI]

Demo:

- Demo Optional [YT], [HSBI]
- Diskussion `null` vs. `Optional` am Beispiel PM-Dungeon [YT], [HSBI]

5.7.1. `Optional<T>` - “Vielleicht gibt es einen Wert”

```

1 public class LSF {
2     private Set<Studi> sl;
3
4     public Studi getBestStudi() {
5         if (sl == null) return null; // Fehler: Es gibt noch keine Sammlung
6         Studi best = null;
7         for (Studi s : sl) {
8             if (best == null) best = s;
9             if (best.credits() < s.credits()) best = s;
10        }
11        return best;
12    }
13 }
14 public static void main(String... args) {
15     LSF lsf = new LSF(); Studi best = lsf.getBestStudi();
16     if (best != null) {
17         String name = best.name();
18         if (name != null) { /* mach was mit dem Namen ... */ }
19     }
20 }

```

5.7.1.1. Problem: `null` wird an (zu) vielen Stellen genutzt

`null` wird sehr gern für diese Situationen genutzt (Aufzählung nicht vollständig):

- Es gibt keinen Wert (“not found”)
- Felder wurden (noch) nicht initialisiert
- Es ist ein Problem oder etwas Unerwartetes aufgetreten

Problem: `null` ist “unsichtbar” im Typ (Rückgabotyp, Parametertyp, ...)!

=> Parameter und Rückgabewerte müssen **stets** auf `null` geprüft werden (oder Annotationen wie `@NotNull` eingesetzt werden ...)

5.7.1.2. Lösung

`Optional<T>` bedeutet: Entweder ein Wert vom Typ `T` oder kein Wert (`empty`). Damit wird im Typ signalisiert, dass es möglicherweise keine Werte gibt und dass die Nutzenden sich mit diesem Fall aktiv auseinander setzen müssen.

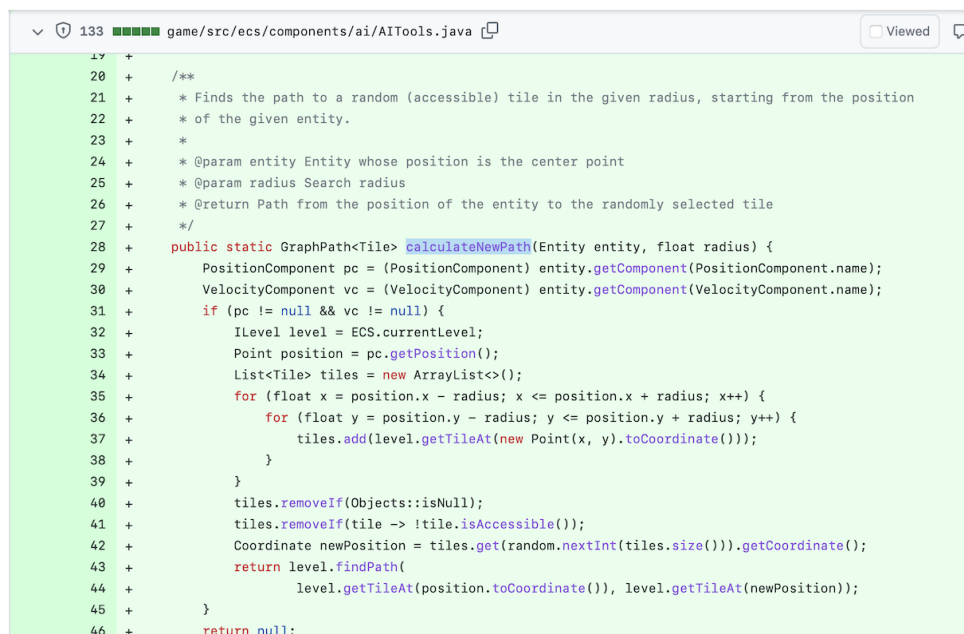
- `Optional<T>` für Rückgabewerte, die “kein Wert vorhanden” mit einschließen (statt `null` bei Abwesenheit von Werten)
- `@NotNull/@Nullable` für Parameter einsetzen (oder separate Prüfung mit `Objects.requireNonNull()`)
- Exceptions werfen in Fällen, wo ein Problem aufgetreten ist

5.7.1.3. Anmerkungen

- Verwendung von `null` auf Attribut-Ebene (Klassen-interne Verwendung) ist okay!
- `Optional<T>` ist **kein** Ersatz für `null`-Checks!
- `null` ist **kein** Ersatz für vernünftiges Error-Handling! Das häufig zu beobachtende “Irgendwas Unerwartetes ist passiert, hier ist `null`” ist ein **Anti-Pattern**!

5.7.1.4. Beispiel aus der Praxis im PM-Dungeon

Schauen Sie sich einmal das Review zu den `ecs.components.ai.AITools` in <https://github.com/Dungeon-CampusMinden/Dungeon/pull/128#pullrequestreview-1254025874> an.



```
17 +
20 + /**
21 +  * Finds the path to a random (accessible) tile in the given radius, starting from the position
22 +  * of the given entity.
23 +  *
24 +  * @param entity Entity whose position is the center point
25 +  * @param radius Search radius
26 +  * @return Path from the position of the entity to the randomly selected tile
27 +  */
28 + public static GraphPath<Tile> calculateNewPath(Entity entity, float radius) {
29 +     PositionComponent pc = (PositionComponent) entity.getComponent(PositionComponent.name);
30 +     VelocityComponent vc = (VelocityComponent) entity.getComponent(VelocityComponent.name);
31 +     if (pc != null && vc != null) {
32 +         ILevel level = ECS.currentLevel;
33 +         Point position = pc.getPosition();
34 +         List<Tile> tiles = new ArrayList<>();
35 +         for (float x = position.x - radius; x <= position.x + radius; x++) {
36 +             for (float y = position.y - radius; y <= position.y + radius; y++) {
37 +                 tiles.add(level.getTileAt(new Point(x, y).toCoordinate()));
38 +             }
39 +         }
40 +         tiles.removeIf(Objects::isNull);
41 +         tiles.removeIf(tile -> !tile.isAccessible());
42 +         Coordinate newPosition = tiles.get(random.nextInt(tiles.size())).getCoordinate();
43 +         return level.findPath(
44 +             level.getTileAt(position.toCoordinate()), level.getTileAt(newPosition));
45 +     }
46 +     return null;
```

Die Methode `AITools#calculateNewPath` soll in der Umgebung einer als Parameter übergebenen Entität nach einem Feld (`Tile`) suchen, welches für die Entität betretbar ist und einen Pfad von der Position der Entität zu diesem Feld an den Aufrufer zurückliefern.

Zunächst wird in der Entität nach einer `PositionComponent` und einer `VelocityComponent` gesucht. Wenn es (eine) diese(r) Components nicht in der Entität gibt, wird der Wert `null` an den Aufrufer von `AITools#calculateNewPath` zurückgeliefert. (Anmerkung: Interessanterweise wird in der Methode nicht mit der `VelocityComponent` gearbeitet.)

Dann wird in der `PositionComponent` die Position der Entität im aktuellen Level abgerufen. In einer Schleife werden alle Felder im gegebenen Radius in eine Liste gespeichert. (Anmerkung: Da dies über die `float`-Werte passiert und nicht über die Feld-Indizes wird ein `Tile` u.U. recht oft in der Liste abgelegt. Können Sie sich hier einfache Verbesserungen überlegen?)

Da `level.getTileAt()` offenbar als Antwort auch `null` zurückliefern kann, werden nun zunächst per `tiles.removeIf(Objects::isNull)`; all diese `null`-Werte wieder aus der Liste entfernt. Danach erfolgt die Prüfung, ob die verbleibenden Felder betretbar sind und nicht-betretbare Felder werden entfernt.

Aus den verbleibenden (betretbaren) Feldern in der Liste wird nun eines zufällig ausgewählt und per `level.findPath()` ein Pfad von der Position der Entität zu diesem Feld berechnet und zurückgeliefert. (Anmerkung: Hier wird ein zufälliges Tile in der Liste der umgebenden Felder gewählt, von diesem die Koordinaten bestimmt, und dann noch einmal aus dem Level das dazugehörige Feld geholt - dabei hatte man die Referenz auf das Feld bereits in der Liste. Können Sie sich hier eine einfache Verbesserung überlegen?)

Zusammengefasst:

- Die als Parameter `entity` übergebene Referenz darf offenbar *nicht* `null` sein. Die ersten beiden Statements in der Methode rufen auf dieser Referenz Methoden auf, was bei einer `null`-Referenz zu einer `NullPointerException`-Exception führen würde. Hier wäre `null` ein Fehlerzustand.
- `entity.getComponent()` kann offenbar `null` zurückliefern, wenn die gesuchte Component nicht vorhanden ist. Hier wird `null` als "kein Wert vorhanden" genutzt, was dann nachfolgende `null`-Checks notwendig macht.
- Wenn es die gewünschten Components nicht gibt, wird dem Aufrufer der Methode `null` zurückgeliefert. Hier ist nicht ganz klar, ob das einfach nur "kein Wert vorhanden" ist oder eigentlich ein Fehlerzustand?
- `level.getTileAt()` kann offenbar `null` zurückliefern, wenn kein Feld an der Position vorhanden ist. Hier wird `null` wieder als "kein Wert vorhanden" genutzt, was dann nachfolgende `null`-Checks notwendig macht (Entfernen aller `null`-Referenzen aus der Liste).
- `level.findPath()` kann auch wieder `null` zurückliefern, wenn kein Pfad berechnet werden konnte. Hier ist wieder nicht ganz klar, ob das einfach nur "kein Wert vorhanden" ist oder eigentlich ein Fehlerzustand? Man könnte beispielsweise in diesem Fall ein anderes Feld probieren?

Der Aufrufer bekommt also eine `NullPointerException`, wenn der übergebene Parameter `entity` nicht vorhanden ist oder den Wert `null`, wenn in der Methode etwas schief lief oder schlicht kein Pfad berechnet werden konnte oder tatsächlich einen Pfad. Damit wird der Aufrufer gezwungen, den Rückgabewert vor der Verwendung zu untersuchen.

Allein in dieser einen kurzen Methode macht `null` so viele extra Prüfungen notwendig und den Code dadurch schwerer lesbar und fehleranfälliger! `null` wird als (unvollständige) Initialisierung und als Rück-

gabewert und für den Fehlerfall genutzt, zusätzlich ist die Semantik von `null` nicht immer klar.

(Anmerkung: Der Gebrauch von `null` hat nicht wirklich etwas mit “der Natur eines ECS” zu tun. Die Methode wurde mittlerweile komplett überarbeitet und ist in der hier gezeigten Form glücklicherweise nicht mehr zu finden.)

Entsprechend hat sich in diesem [Review](#) die nachfolgende Diskussion ergeben:

133
game/src/ecs/components/ai/AITools.java
Viewed

AMatutat marked this conversation as resolved.
Hide resolved

cagix
on Jan 18 • edited
Member

Was bedeutet `null` als Ergebnis? Die Operation hat nicht geklappt, weil es keine `PositionComponent` und/oder `VelocityComponent` gab? Wäre das nicht die Stelle, eine passende Exception auszulösen (dito in `PositionComponent` und `VelocityComponent`)? Dann braucht der Aufrufer nicht die ganzen `if (wuppie != null) { ... } else { return null; }` ... (das "Pattern" taucht ganz schön häufig auf)

AMatutat
on Jan 18
Member
Author

Jop. Das pattern gibt es häufiger und die Problematik lässt sich auf die Natur eines ECS zurückführen. Ich wollte nochmal in der großen Welt gucken, wie andere damit umgehen.

Dauer Lösung ist es so nicht.
Ist ja auch nur draft

cagix
on Jan 18
Member

Schon klar. Leider hält ein Provisorium oft länger, als einem lieb ist ;)

Ich denke, die Lösung heißt `Exception` oder `Option<T>`, wobei letzteres einen etwas anderen Code-Stil nach sich zieht (oder Du endest dann analog zu den `if (null)`-Abfragen mit `.isEmpty()`).

cagix
on Jan 18 • edited
Member

Schau mal <https://github.com/Programmierungsmethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#lsf-liefert-jetzt-optional-zurück> und <https://github.com/Programmierungsmethoden/PM-Lecture/blob/master/markdown/modern-java/optional.md#zugriff-auf-optional-objekte>: das passt eigentlich exakt zu dem Problem hier, der Zugriff dann mit `best = lsf.getBestStudi().orElseThrow();` (mit `public Optional<Studi> getBestStudi()`).

AMatutat
on Jan 18 • edited
Member
Author

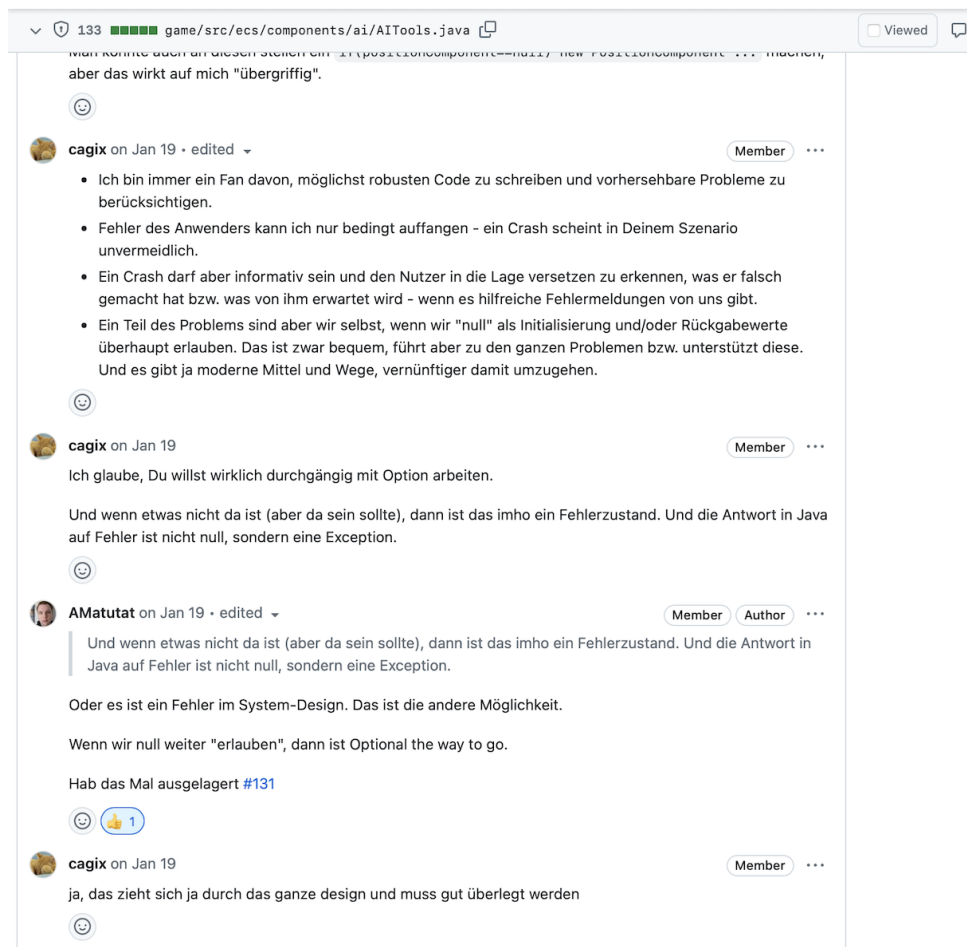
Das wäre eine Möglichkeit, welches sich am aktuellen Konzept orientiert.

Ich weiß nur nicht, ob ein null-Check (in welcher Form auch immer) überhaupt notwendig ist. Vielleicht sollte das Programm hier crashen.

Immerhin hab ich (als Anwender) in meiner Entität eine Funktion hinterlegt (über das `AComponent`), welches nunmal das `PositionComponent` und `VelocityComponent` voraussetzt, dann bin ich vielleicht auch dafür verantwortlich, dass ich die benötigten Components auch hinzufügen.

Aber so wirklich richtig klingt das auch nicht.

Man könnte auch an diesen stellen ein `if(positionComponent==null) new PositionComponent ...` machen, aber das wirkt auf mich "übergreifig".



5.7.2. Erzeugen von *Optional*-Objekten

Konstruktor ist **private** ...

- "Kein Wert": `Optional.empty()`
- Verpacken eines non-**null** Elements: `Optional.of()`
(`NullPointerException` wenn Argument **null**!)
- Verpacken eines "unsicheren"/beliebigen Elements: `Optional.ofNullable()`
 - Liefert verpacktes Element, oder
 - `Optional.empty()`, falls Element **null** war

Es sollte in der Praxis eigentlich nur wenige Fälle geben, wo ein Aufruf von `Optional.of()` sinnvoll ist. Ebenso ist `Optional.empty()` nur selten sinnvoll.

Stattdessen sollte stets `Optional.ofNullable()` verwendet werden.

Important

null kann nicht in **Optional<T>** verpackt werden! (Das wäre dann eben **Optional.empty()**.)

5.7.3. LSF liefert jetzt *Optional* zurück

```
1 public class LSF {
2     private Set<Studi> sl;
3
4     public Optional<Studi> getBestStudi() throws IllegalStateException {
5         // Fehler: Es gibt noch keine Sammlung
6         if (sl == null) throw new IllegalStateException("There ain't any
7             collection");
8
9         Studi best = null;
10        for (Studi s : sl) {
11            if (best == null) best = s;
12            if (best.credits() < s.credits()) best = s;
13        }
14        // Entweder Optional.empty() (wenn best==null) oder Optional.of(best)
15        // sonst
16        return Optional.ofNullable(best);
17    }
18 }
```

Das Beispiel soll verdeutlichen, dass man im Fehlerfall nicht einfach **null** oder **Optional.empty()** zurückliefern soll, sondern eine passende Exception werfen soll.

Wenn die Liste aber leer ist, stellt dies keinen Fehler dar! Es handelt sich um den Fall “kein Wert vorhanden”. In diesem Fall wird statt **null** nun ein **Optional.empty()** zurückgeliefert, also ein Objekt, auf dem der Aufrufer die üblichen Methoden aufrufen kann.

5.7.4. Zugriff auf *Optional*-Objekte

In der funktionalen Programmierung gibt es schon lange das Konzept von **Optional**, in Haskell ist dies beispielsweise die Monade **Maybe**. Allerdings ist die Einbettung in die Sprache von vornherein mit berücksichtigt worden, insbesondere kann man hier sehr gut mit *Pattern Matching* in der Funktionsdefinition auf den verpackten Inhalt reagieren.

In Java gibt es die Methode **Optional#isEmpty()**, die einen Boolean zurückliefert und prüft, ob es sich um ein leeres **Optional** handelt oder ob hier ein Wert “verpackt” ist. Die Methode **Optional#isPresent()** hat die invertierte Semantik.

Für den direkten Zugriff auf die Werte gibt es die Methoden **Optional#orElseThrow()** und **Optional#orElse()**. Damit kann man auf den verpackten Wert zugreifen, oder es wird eine Exception geworfen bzw. ein Ersatzwert geliefert.

Zusätzlich gibt es **Optional#ifPresent()**, die als Parameter ein **java.util.function.Consumer** erwartet, also ein funktionales Interface mit einer Methode **void accept(T)**, die das Objekt verarbeitet

(sofern vorhanden).

```

1  Studi best, anne;
2
3  // Testen und dann verwenden
4  if (!lsf.getBestStudi().isEmpty()) {
5      best = lsf.getBestStudi().get();
6      // mach was mit dem Studi ...
7  }
8
9  // Arbeite mit Consumer
10 lsf.getBestStudi().ifPresent(studi -> {
11     // mach was mit dem Studi ...
12 });
13
14 // Studi oder Alternative (wenn Optional.empty())
15 best = lsf.getBestStudi().orElse(anne);
16
17 // Studi oder NoSuchElementException (wenn Optional.empty())
18 best = lsf.getBestStudi().orElseThrow();

```

Es gibt noch eine Methode `get()`, die so verhält wie `orElseThrow()`. Vom Namen her wirkt sie wie ein “normaler Getter”, deshalb ist sie leicht zu missbrauchen (Aufruf ohne vorherige Prüfung). Verwenden Sie besser explizit `orElseThrow()` bzw. `orElse(...)`.

Tip

Da `getBestStudi()` eine `NullPointerException` werfen kann (vgl. Implementierung am Anfang), sollte der Aufruf möglicherweise in ein `try/catch` verpackt werden.

Beispiel: [optional.traditional.Demo](#)

5.7.5. Einsatz mit Stream-API

```

1  public class LSF {
2      ...
3      public Optional<Studi> getBestStudi() throws IllegalStateException {
4          if (sl == null) throw new IllegalStateException("There ain't any
5              collection");
6          return sl.stream()
7              .sorted((s1, s2) -> s2.credits() - s1.credits())
7              .findFirst(); // Optional<Studi>
8      }
9  }
10
11
12 public static void main(String... args) {
13     ...
14     String name = lsf.getBestStudi() // Optional<Studi>
15         .map(Studi::name) // Optional<String>
16         .orElseThrow();
17 }

```

Beispiel: optional.streams.Demo

Im Beispiel wird in `getBestStudi()` die Sammlung als Stream betrachtet, über die Methode `sorted()` und den Lambda-Ausdruck für den `Comparator` sortiert ("falsch" herum: absteigend in den Credits der Studis in der Sammlung), und `findFirst()` ist die terminale Operation auf dem Stream, die ein `Optional<Studi>` zurückliefert: entweder den Studi mit den meisten Credits (verpackt in `Optional<Studi>`) oder `Optional.empty()`, wenn es überhaupt keine Studis in der Sammlung gab.

In `main()` wird dieses `Optional<Studi>` mit den Stream-Methoden von `Optional<T>` bearbeitet, zunächst mit `Optional#map()`. Man braucht nicht selbst prüfen, ob das von `getBestStudi()` erhaltene Objekt leer ist oder nicht, da dies von `Optional#map()` erledigt wird: Es wendet die Methodenreferenz auf den verpackten Wert an (sofern dieser vorhanden ist) und liefert damit den Namen des Studis als `Optional<String>` verpackt zurück. Wenn es keinen Wert, also nur `Optional.empty()` von `getBestStudi()` gab, dann ist der Rückgabewert von `Optional#map()` ein `Optional.empty()`. Wenn der Name, also der Rückgabewert von `Studi : : name`, `null` war, dann wird ebenfalls ein `Optional.empty()` zurückgeliefert. Dadurch wirft `orElseThrow()` dann eine `NoSuchElementException`. Man kann also direkt mit dem String `name` weiterarbeiten ohne extra `null`-Prüfung - allerdings will man noch ein Exception-Handling einbauen (dies fehlt im obigen Beispiel aus Gründen der Übersicht) ...

5.7.6. Weitere Optionals

Für die drei primitiven Datentypen `int`, `long` und `double` gibt es passende Wrapper-Klassen von `Optional<T>`: `OptionalInt`, `OptionalLong` und `OptionalDouble`.

Diese verhalten sich analog zu `Optional<T>`, haben aber keine Methode `ofNullable()`, da dies hier keinen Sinn ergeben würde: Die drei primitiven Datentypen repräsentieren Werte - diese können nicht `null` sein.

5.7.7. Regeln für Optional

1. Nutze `Optional` nur als Rückgabe für "kein Wert vorhanden"

`Optional` ist nicht als Ersatz für eine `null`-Prüfung o.ä. gedacht, sondern als Repräsentation, um auch ein legitimes "kein Wert vorhanden" zurückliefern zu können.

Wichtig: Nicht als Ersatz für eine Exception o.ä. missbrauchen!

2. Nutze nie `null` für eine `Optional`-Variable oder einen `Optional`-Rückgabewert

Wenn man ein `Optional` als Rückgabe bekommt, sollte das niemals selbst eine `null`-Referenz sein. Das macht das gesamte Konzept kaputt!

Nutzen Sie stattdessen `Optional.empty()`.

3. Nutze `Optional.ofNullable()` zum Erzeugen eines `Optional`

Diese Methode verhält sich "freundlich" und erzeugt automatisch ein `Optional.empty()`, wenn das Argument `null` ist. Es gibt also keinen Grund, dies mit einer Fallunterscheidung selbst erledigen zu wollen.

Beim Erzeugen von Optionals aus vorhandenen Werten verwenden Sie in der Regel `Optional.ofNullable(...)` statt selbst `if (x == null)... Optional.empty()... Optional.of(x)` zu formulieren. Für Rückgabewerte, bei denen klar ist "hier gibt es gerade keinen Wert", ist `return Optional.empty();` natürlich sinnvoll.

4. Erzeuge keine `Optional` als Ersatz für die Prüfung auf `null`

Wenn Sie auf `null` prüfen müssen, müssen Sie auf `null` prüfen. Der ersatzweise Einsatz von `Optional` macht es nur komplexer - prüfen müssen Sie hinterher ja immer noch.

5. Nutze `Optional` nicht in Attributen, Methoden-Parametern und Sammlungen

Nutzen Sie `Optional` vor allem für Rückgabewerte.

Attribute sollten immer direkt einen Wert haben oder `null`, analog Parameter von Methoden o.ä. ... Hier hilft `Optional` nicht, Sie müssten ja trotzdem eine `null`-Prüfung machen, nur eben dann über den `Optional`, wodurch dies komplexer und schlechter lesbar wird.

Aus einem ähnlichen Grund sollten Sie auch in Sammlungen keine `Optional` speichern!

6. Vermeiden Sie Muster wie `if (opt.isPresent()) { T v = opt.get(); ... }`.

Häufig sieht man (leider) das folgende Muster:

```
1  if (opt.isPresent()) {  
2      T v = opt.get();  
3      // ...  
4  }
```

Der direkte Zugriff auf ein `Optional` entspricht dem Prüfen auf `null` und dann dem Auspacken. Dies ist nicht nur Overhead, sondern auch schlechter lesbar.

Vermeiden Sie den direkten Zugriff und nutzen Sie `Optional` mit den Stream-Methoden `map/flatMap/filter` und am Ende der Verarbeitungskette `orElse(...)` oder `orElseThrow(...)`. So ist dies von den Designern gedacht.

Important

`Optional<T>` für (normale) Abwesenheit eines Wertes, nicht für Fehler mit Erklärung!

5.7.8. Was ist das Problem mit Exceptions?

```
1  public static int parsePort(String input) {  
2      int port = Integer.parseInt(input);  
3      if (port < 1024 || port > 65535) {  
4          throw new IllegalArgumentException("Port out of range");  
5      }  
6      return port;  
7  }
```

Problempunkte des traditionellen Exception Handlings in Java

1. Fehler sind “unsichtbar” im Typ

Aus der Signatur `int parsePort(String)` sehen Sie nicht, dass hier ein Fehlerfall möglich ist - etwa wenn der String kein gültiger Port ist. Ob und welche Fehler auftreten können, erfahren Sie nur über Javadoc, externe Dokumentation oder durch Blick in die Implementierung.

Selbst bei checked Exceptions hat sich leider oft die Praxis etabliert, diese sehr früh zu fangen und als `RuntimeException` (oder eine andere unchecked Exception) neu zu werfen. Damit spart man sich das “lästige” `throws` an der Methode und die Aufrufer müssen keinen `try/catch`-Block schreiben.

Auswirkungen dieser Praxis:

- Der Typsignatur ist nicht mehr anzusehen, dass hier ein Fehlerfall existiert
- Die Compiler-Unterstützung beim Erinnern an Fehlerfälle geht verloren
- Fehlerpfade werden “versteckt” und auftretende Laufzeitfehler wirken plötzlich überraschend

Der scheinbare Vorteil ist, dass der Code “aufgeräumter” wirkt (weniger `throws`, weniger `try/catch`), aber erkaufte wird das mit geringerer Transparenz und schlechterer Wartbarkeit. Das ist in der Regel **keine gute Praxis**.

2. Verstreute Fehlerbehandlung

`try/catch`-Blöcke befinden sich oft weit entfernt von dem Ort, an dem der Fehler tatsächlich entstehen kann. Eine Methode wirft z.B. eine Exception, diese wird nach “oben” durchgereicht und irgendwo weiter oben in der Aufrufkette gefangen.

Probleme dabei:

- Wer eine Methode aufruft, sieht an der Stelle oft nicht, **wie** der Fehler später behandelt wird
- Es besteht die Gefahr, Exceptions zu “verschlucken” (`catch (Exception e) {}`) oder sie zu vergessen (ein `catch`-Block, der zwar da ist, aber nur ein kommentarloses Logging macht)
- Verantwortlichkeiten verschwimmen: Wo gehört die Behandlung des Fehlers wirklich hin? Zur aufrufenden Methode? Zur Geschäftslogik? Zur GUI?

Ergebnis: Die Fehlerszenarien sind schwer nachzuvollziehen und häufig erst nach mehreren Ebenen im Call-Stack erkennbar.

3. Zusätzlicher (nicht offensichtlicher) Kontrollfluss

`try/catch` stellt neben `if`, `while`, `for`, `switch` einen weiteren Kontrollflussmechanismus dar. Gerade bei verschachtelten Strukturen kann das zu komplexem und fehleranfälligem Code führen:

- Der “normale” Kontrollfluss (z.B. per `if/else`) und der Kontrollfluss im Fehlerfall (`throw` → Sprung in einen `catch`-Block) sind vermischt
- Eine Exception kann den Kontrollfluss an ganz andere Stellen springen lassen, ohne dass dies in der Signatur sichtbar ist
- `finally`-Blöcke kommen als **weiterer Spezialfall** hinzu und machen eine “mentale Simulation” des Programmablaufs noch anspruchsvoller

Gerade für Einsteiger (aber auch in großen Projekten) erschwert das Verstehen, Testen und formale Argumentieren über den Programmablauf.

4. Zusammenspiel mit Stream-API und Lambdas/Methodenreferenzen

In der Stream-API werden Operationen auf einer Folge von Daten durchgeführt (z.B. `map`, `filter`, `flatMap`). Meist kommen dabei Lambda-Ausdrücke oder Methodenreferenzen zum Einsatz.

Probleme mit Exceptions:

- Funktionsschnittstellen wie `Function<T, R>` oder `Predicate<T>` deklarieren keine checked Exceptions. Lambdas, die checked Exceptions werfen, “passen” nicht ohne Weiteres hinein.
- Das Einbauen von `try/catch` in Lambdas führt schnell zu unübersichtlichem und schwer lesbarem Code:

```
1 list.stream()
2   .map(s -> {
3       try {
4           return parsePort(s);
5       } catch (NumberFormatException e) {
6           // Fehlerbehandlung mitten im Lambda
7           return DEFAULT_PORT;
8       }
9   })
10  .forEach(IO::println);
```

- Alternativen wie “wrappe jede Exception in eine `RuntimeException`” verschieben das Problem nur: Die eigentlichen Fehlerfälle sind weiterhin nicht Typ-sichtbar und schwer gezielt behandelbar.

Moderne Alternativen wie `Optional<T>` oder Result-Typen (z.B. `Result<T, E>`) lassen sich dagegen sehr gut mit Streams kombinieren, weil sie Fehler als **normale Werte im Typ** ausdrücken und mit den üblichen Funktionsoperationen (`map`, `flatMap`, `filter`, ...) bearbeitet werden können.

5. Erschwerte Komposition und Wiederverwendung

Methoden, die Exceptions werfen, lassen sich schlechter zu größeren Funktionalitäten zusammensetzen als Methoden, die ihren Fehlerzustand explizit im Typ ausdrücken:

- Wenn jede Methode “irgendetwas” wirft, wird `throws` schnell sehr allgemein (`throws Exception`), was die Information für Aufrufer entwertet
- Wiederverwendbare Hilfsfunktionen müssen sich an den kleinsten gemeinsamen Nenner anpassen (z.B. alles in `RuntimeException` verpacken), damit sie überall einsetzbar sind
- Eine explizite Modellierung von Fehlern als Teil des Rückgabetyps (z.B. `Optional<T>`, `Result<T, E>`) zwingt zur bewussten Entscheidung:
 - Was ist ein “normaler” Fehlerrückgabewert?
 - Was ist wirklich ein außergewöhnlicher Zustand, der eine Exception rechtfertigt?

6. Vermischung von “erwartbaren” Fehlern und echten Ausnahmen

In vielen Codebasen werden Exceptions sowohl für erwartbare Situationen (z.B. “Datei nicht gefunden”, “Eingabe nicht parsbar”) als auch für wirklich außergewöhnliche Fälle (z.B. Programmierfehler, Invarianten verletzt) genutzt. Das führt zu:

- Unklarheit: Muss ich als Aufrufer diesen Fehlerfall aktiv behandeln, oder ist das ein Programmierfehler?

- Überbenutzung von Exceptions als “normaler” Kontrollfluss: Exceptions werden geworfen, obwohl das Verhalten eigentlich Teil der üblichen Logik ist (z.B. “Nutzer existiert nicht”).

Besser:

- Erwartbare Alternativen im Typ ausdrücken (`Optional`, `Result`, Domänen-Typen)
- Exceptions für wirklich unerwartete / in der aktuellen Schicht nicht sinnvoll behandelbare Situationen reservieren

Gibt es eine Möglichkeit, Fehler so zu modellieren, dass sie im Typ sichtbar sind?

Wir könnten überlegen, ein `Optional<Integer>` als Ergebnis zu liefern: Wenn es einen Fehler gab, dann haben wir nur ein `Optional.empty()`:

```
1 public static Optional<Integer> parsePort(String input) {
2     int port = Integer.parseInt(input);
3     if (port < 1024 || port > 65535) {
4         return Optional.empty();
5     }
6     return Optional.of(port);
7 }
```

Damit hätten wir hier aber nicht wirklich etwas gewonnen:

1. `Integer.parseInt(input)` kann immer noch eine `NumberFormatException` liefern, die der Aufrufer fangen müsste
2. Im Fehlerfall (Ports außerhalb des Bereichs) bekommen wir zwar nun ein `Optional.empty()`, wissen aber nicht *warum*
3. Das Handling auf der Aufrufer-Seite wird zu einem etwas umständlichen `ifPresentOrElse` (Port vorhanden oder Reaktion auf falsche Portnummer)

=> Wir haben hier **nicht** den Fall, dass wir “kein Ergebnis” (keinen Port) als normales Ergebnis haben können. Stattdessen missbrauchen wir `Optional<T>`, was uns tatsächlich nicht wirklich weiter hilft. `Optional<T>` trägt keinerlei Information mit sich, warum es zu `Optional.empty()` gekommen ist.

Tip

Merke: `Optional<T>` eignet sich schlecht für Fälle, in denen Sie die Art des Fehlers unterscheiden wollen. Dafür brauchen Sie einen **Fehlertyp** → `Result<E, R>`.

5.7.9. Result<E, R>: Fehler als Datentyp modellieren

Wir wollen das Ergebnis und den Fehler in einem gemeinsamen Typ kapseln. Dazu können wir uns einen generischen Typ `Result<E, R>` definieren, der entweder ein Ergebnis vom Typ `R` mit sich führt oder einen Fehler vom Typ `E` (samt Informationen über den Fehler).

```
1 public sealed interface Result<E, R> permits Result.Ok, Result.Err {
2
3     record Ok<E, R>(R value) implements Result<E, R> {}
4     record Err<E, R>(E error) implements Result<E, R> {}
5 }
```

```

5
6     static <E, R> Result<E, R> ok(R value) { return new Ok<>(value); }
7     static <E, R> Result<E, R> err(E error) { return new Err<>(error); }
8 }

```

Der Typ `Result<E, R>` wird als sealed Typ definiert mit exakt zwei Ausprägungen:

- `Ok<E, R>`: kapselt das normale Ergebnis, welches über `value()` abrufbar ist
- `Err<E, R>`: kapselt den Fehlerfall, welcher über `error()` abrufbar ist

Die statischen Hilfsmethoden `ok(R value)` und `err(E error)` sind lediglich Convenience-Methoden, um die Ergebnisse leichter erzeugen zu können.

5.7.10. Umbau unseres Beispiels auf `Result<E, R>`

Jetzt können wir uns einen Typ für unsere Fehler definieren und unser Beispiel umbauen:

```

1 public enum PortError {
2     NOT_A_NUMBER,
3     OUT_OF_RANGE
4 }

```

```

1 public static Result<PortError, Integer> parsePort(String input) {
2     try {
3         int port = Integer.parseInt(input);
4         if (port < 1024 || port > 65535) {
5             return Result.err(PortError.OUT_OF_RANGE);
6         }
7         return Result.ok(port);
8     } catch (NumberFormatException e) {
9         return Result.err(PortError.NOT_A_NUMBER);
10    }
11 }

```

In der Methode haben wir zwei Stellen, wo ein Fehler auftreten kann: das `Integer.parseInt(input)` kann eine `NumberFormatException` werfen, und der Port könnte außerhalb des zulässigen Bereichs liegen. Zur Modellierung unseres Fehler bauen wir uns ein Enum, welches genau diese beiden Fälle über Konstanten repräsentiert. Im Fehlerfall geben wir dann ein `Err (ENUMKONSTANTE)` zurück (über die Convenience-Methode `Result.err()`), und im Erfolgsfall müssen wir unseren Port in ein `Ok` kapseln und nutzen hier die Convenience-Methode `Result.ok()`.

5.7.11. Handling auf Aufrufer-Seite

```

1 public static void foo(String input) {
2     Result<PortError, Integer> result = PortParser.parsePort(input);
3
4     switch (result) {
5         case Result.Ok<PortError, Integer> ok -> {
6             int port = ok.value();
7             IO.println("Starte Server auf Port " + port);

```

```

8      }
9      case Result.Err<PortError, Integer> err -> handleError(err.error());
10     }
11 }
12
13 private static void handleError(PortError err) {
14     switch (err) {
15         case NOT_A_NUMBER -> System.err.println("Eingabe ist keine Zahl.");
16         case OUT_OF_RANGE -> System.err.println("Port muss zwischen 1024 und
17                               65535 liegen.");
18     }
19 }

```

Der Fehlertyp ist jetzt Teil des Methodentyps: `Result<PortError, Integer>` → klar sichtbar, welche Art Fehler vorkommen kann. Auf der Aufruferseite kann man elegant Pattern Matching einsetzen, und durch die sealed Hierarchie wird auch ein **default**-Zweig unnötig. Kein **throws** mehr nötig, kein versehentliches “Durchrutschen” von Exceptions, und die aufrufenden Methoden müssen entscheiden, was mit `Err` passiert.

Tip

Exkurs für Fortgeschrittene: Das kann man noch ein wenig ausbauen und in die Stream-Verarbeitung einbinden ...

5.7.11.1. Schritt 1: Szenario erweitern

Erweitern wir unser Szenario von oben um einige Verarbeitungsstufen:

```

1 public int parsePort(String input) throws NumberFormatException,
   IllegalArgumentExcepion {
2     int port = Integer.parseInt(input);
3     if (port < 1024 || port > 65535) {
4         throw new IllegalArgumentException("Port out of range");
5     }
6     return port;
7 }
8
9 public Socket openSocket(int port) throws IOException {
10     return new Socket("localhost", port);
11 }
12
13 public String sendHello(Socket socket) throws IOException {
14     socket.getOutputStream().write("HELLO".getBytes());
15     byte[] buffer = new byte[1024];
16     int n = socket.getInputStream().read(buffer);
17     return new String(buffer, 0, n);
18 }

```

Das würde zusammengebaut mit traditionellem Exception-Handling irgendwie so aussehen:

```

1 public void connectToServerAndPrintAnswer(String userInput) {
2     try {
3         int port = parsePort(userInput);
4         Socket socket = openSocket(port);
5         String response = sendHello(socket);

```



```

6      IO.println("Antwort: " + response);
7  } catch (NumberFormatException e) {
8      System.err.println("Port ist keine Zahl.");
9  } catch (IllegalArgumentException e) {
10     System.err.println("Ungültiger Portbereich.");
11 } catch (IOException e) {
12     System.err.println("Netzwerkfehler: " + e.getMessage());
13 }
14 }

```

5.7.11.2. Schritt 2: Umbauen auf Result<E,R>

Wir haben drei verschiedene Fehlersituationen im erweiterten Szenario. Diese können wir über ein Enum modellieren:

```

1 public enum ConnectionError {
2     INVALID_PORT_FORMAT,
3     PORT_OUT_OF_RANGE,
4     NETWORK_ERROR
5 }

```

Nun können wir hingehen und die drei Hilfsfunktionen `parsePort`, `openSocket` und `sendHello` auf passende `Result<E,R>` umbauen:

```

1 public Result<ConnectionError, Integer> parsePort(String input) {
2     try {
3         int port = Integer.parseInt(input);
4         if (port < 1024 || port > 65535) {
5             return Result.err(ConnectionError.PORT_OUT_OF_RANGE);
6         }
7         return Result.ok(port);
8     } catch (NumberFormatException e) {
9         return Result.err(ConnectionError.INVALID_PORT_FORMAT);
10    }
11 }
12
13 public Result<ConnectionError, Socket> openSocket(int port) {
14     try {
15         Socket socket = new Socket("localhost", port);
16         return Result.ok(socket);
17     } catch (IOException e) {
18         return Result.err(ConnectionError.NETWORK_ERROR);
19     }
20 }
21
22 public Result<ConnectionError, String> sendHello(Socket socket) {
23     try {
24         socket.getOutputStream().write("HELLO".getBytes());
25         byte[] buffer = new byte[1024];
26         int n = socket.getInputStream().read(buffer);
27         return Result.ok(new String(buffer, 0, n));
28     } catch (IOException e) {
29         return Result.err(ConnectionError.NETWORK_ERROR);
30 }

```

```
31 }
```

Bitte beachten Sie die unterschiedlichen Rückgabe-Typen: Wir haben immer unseren `ConnectionError` als Fehlertyp dabei, aber der Ergebnistyp unterscheidet sich von Funktion zu Funktion: `Integer`, `Socket`, `String`. Das wird jetzt "interessant" beim Zusammenbauen und Aufrufen:

```
1 public void connectToServerAndPrintAnswer(String userInput) {
2     Result<ConnectionError, Integer> port = parsePort(userInput);
3     switch (port) {
4         case Result.Ok<ConnectionError, Integer> okp -> {
5             Result<ConnectionError, Socket> socket = openSocket(okp.value());
6             switch (socket) {
7                 case Result.Ok<ConnectionError, Socket> oks -> {
8                     Result<ConnectionError, String> response = sendHello(
9                         oks.value());
10                    switch (response) {
11                        case Result.Ok<ConnectionError, String> okr -> IO.
12                            println("Antwort: " + okr.value());
13                        case Result.Err<ConnectionError, String> errr ->
14                            handleError(errr.error());
15                    }
16                }
17            case Result.Err<ConnectionError, Socket> errs ->
18                handleError(errs.error());
19        }
20    case Result.Err<ConnectionError, Integer> errp -> handleError(errp.
21        error());
22    }
23 }
24
25 private static void handleError(ConnectionError err) {
26     switch (err) {
27         case INVALID_PORT_FORMAT -> System.err.println("Port ist keine Zahl
28             .");
29         case PORT_OUT_OF_RANGE -> System.err.println("Port muss zwischen
30             1024 und 65535 liegen.");
31         case NETWORK_ERROR -> System.err.println("Netzwerkfehler (keine
32             Message)");
33     }
34 }
```

Wir bekommen bei jedem Aufruf ein anderes `Result<ConnectionError, T>`. Mit Pattern Matching und `switch/case` kann das halbwegs elegant auseinander nehmen, und da der Error-Typ überall gleich ist, reicht uns eine gemeinsame Funktion für den Fehlerfall.

Huh. Das funktioniert. Lesbar ist aber anders ...

5.7.11.3. Schritt 3: Blick in die Java Stream-API

Unser Problem ist, dass jede der drei Funktionen ein anderes `Result<ConnectionError, T>` zurückliefert. Bevor wir jeweils die nächste Funktion aufrufen können (im Erfolgsfall), müssen wir das `Ok<ConnectionError, T>` auspacken und den Wert vom Typ `T` in die nächste Funktion überge-

ben. Im Fehlerfall haben wir ein `Err<ConnectionError, T>`, packen den Fehlerwert vom Typ `ConnectionError` aus, erzeugen die Reaktion (hier nur eine simple Ausgabe) und brechen die weitere Verarbeitung ab.

Unsere drei Hilfsfunktionen `parsePort`, `openSocket` und `sendHello` haben in der ersten Version (mit traditionellen Exceptions) eine Abbildung $T \rightarrow R$ implementiert (mit passenden Typen). In der zweiten Version wurden daraus Abbildungen $T \rightarrow \text{Result}<\text{ConnectionError}, R>$.

Aus der Java Stream-API kennen wir die beiden Funktionen `map` und `flatMap`, die auf einem `Stream<T>` arbeiten:

1. `<R> Stream<R> map(Function<? super T, ? extends R> mapper)` ist eine Funktion, mit einer übergebenen Funktion `mapper: T  $\rightarrow$  R` über die Elemente des `Stream<T>` iteriert und für jedes Element die `mapper`-Funktion aufruft, d.h. aus Elementen vom Typ `T` werden neue Elemente vom Typ `R` erzeugt. Diese werden in den Stream gelegt, so dass wir im Ergebnis einen `Stream<R>` zurück erhalten.
2. `<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)` ist analog zu `map` eine Funktion, die über alle Elemente des `Stream<T>` läuft und auf jedes Element vom Typ `T` die `mapper`-Funktion anwendet. Diesmal erzeugt `mapper` aber nicht einfach ein "normales" Ergebnis vom Typ `R`, sondern gibt selbst ein `Stream<? extends R>` zurück, d.h. `mapper: T  $\rightarrow$  Stream<? extends R>`. Die `flatMap`-Funktion muss entsprechend das Ergebnis der `mapper`-Funktion "auspacken" und in das eigene Ergebnis "umverpacken", um wir im Ergebnis einen `Stream<R>` bekommen.

Wenn wir jetzt `Stream` mit `Result` ersetzen und `T` mit den jeweiligen Eingabetypen unserer Hilfsfunktionen, haben wir

- `Function<? super String, ? extends Result<ConnectionError, Integer>> parsePort`
- `Function<? super Integer, ? extends Result<ConnectionError, Socket>> openSocket`
- `Function<? super Socket, ? extends Result<ConnectionError, String>> sendHello`

=> Was wir brauchen, ist eine Art `flatMap` für unser `Result<E, R>`!

(Ja, wir bauen uns hier eine Art Monade in Java nach ...)

5.7.11.4. Schritt 4: Ergänzen von `map` und `flatMap` in `Result<E, R>`

```

1 public sealed interface Result<E, R> permits Result.Ok, Result.Err {
2
3     record Ok<E, R>(R value) implements Result<E, R> {}
4     record Err<E, R>(E error) implements Result<E, R> {}
5
6     static <E, R> Result<E, R> ok(R value) { return new Ok<>(value); }
7     static <E, R> Result<E, R> err(E error) { return new Err<>(error); }
8

```

```

 9 // ----- Functor: apply function R -> B -----
10 default <B> Result<E, B> map(Function<? super R, ? extends B> f) {
11     // return flatMap(a -> Result.ok(f.apply(a))); // für
12     // Fortgeschrittene: das würde reichen ...
13     return switch (this) {
14         case Ok<E, R> ok -> Result.ok(f.apply(ok.value));
15         case Err<E, R> e -> Result.err(e.error);
16     };
17 }
18 // ----- Monad: apply function R -> Result<E,B> -----
19 default <B> Result<E, B> flatMap(Function<? super R, ? extends Result<E
20     , B>> f) {
21     return switch (this) {
22         case Ok<E, R> ok -> f.apply(ok.value);
23         case Err<E, R> e -> Result.err(e.error);
24     };
25 }
26 // ----- unwrap like in Optional<T> -----
27 default R orElseThrow(Function<? super E, ? extends RuntimeException> f
28     ) {
29     return switch (this) {
30         case Ok<E, R> ok -> ok.value;
31         case Err<E, R> e -> throw f.apply(e.error);
32     };
33 }
34 default R orElse(R fallback) {
35     return switch (this) {
36         case Ok<E, R> ok -> ok.value;
37         case Err<E, R> _ -> fallback;
38     };
39 }
40 }

```

Die `map`-Funktion nimmt eine Funktion `f: R → B` entgegen. Im Erfolgsfall (d.h. `map` arbeitet auf einem `Ok`), packen wir den Wert mit `value()` aus, wenden die Funktion darauf an und verpacken das Ergebnis in ein `Ok` und liefern das zurück. Im Fehlerfall (`Err`) geben wir einfach unseren Fehlerwert neu verpackt zurück - d.h. es wird nichts weiter berechnet. (Anmerkung: Man könnte diese Implementierung der `map`-Funktion auf ein einfaches `return flatMap(a -> Result.ok(f.apply(a)));` zurückführen. Warum?) `flatMap` übernimmt die ganze Arbeit. Im Fehlerfall geben wir einfach unseren Fehlerwert neu verpackt zurück - d.h. es wird nichts weiter berechnet. Das Umverpacken ist ausschließlich notwendig, um den Typ-Signaturen zu genügen. Im Erfolgsfall holen wir mit `value()` den Wert vom Typ `R` heraus, wenden die übergebene Funktion `Function<? super R, ? extends Result<E, B>>` an und bekommen ein `Result<E, B>` zurück, welches wir direkt zurückgeben können.

Zusätzlich habe ich zwei Funktionen definiert, die sich analog zu den `orElse`-Methoden von `Optional<T>` verhalten.

5.7.11.5. Schritt 5: Vereinfachen der Aufrufkette

Was wir gesucht haben, war `flatMap`: Jede der drei Hilfsfunktionen `parsePort`, `openSocket` und `sendHello` ist eine Abbildung $T \rightarrow \text{Result}<\text{ConnectionError}, R>$. D.h. wir müssen nach dem Aufruf einer solchen Funktion die weitere Berechnung “kurzschließen”, wenn es ein `Err` ist. Im `Ok`-Fall packen wir den Wert aus und übergeben ihn an die nächste Funktion.

```

1 public void runWithResult(String userInput) {
2     Result<ConnectionError, String> result =
3         parsePort(userInput)           // Result<ConnectionError, Integer>
4         .flatMap(this::openSocket)     // Result<ConnectionError, Socket>
5         .flatMap(this::sendHello);    // Result<ConnectionError, String>
6
7     switch (result) {
8         case Result.Ok<ConnectionError, String> ok -> IO.println("Antwort:
9             " + ok.value());
10        case Result.Err<ConnectionError, String> err -> handleError(err.
11            error());
12    }
13 }

```

In einem weiteren Schritt könnte man auch den verwendeten Fehler-Typ ergänzen, so dass man die Message aus dem `try/catch` mit hochreichen kann ...

Das Konzept nennt sich “Monade” und ist in der funktionalen Programmierung seit langer Zeit bekannt und wird erfolgreich angewendet. Man sieht an den Typen, dass es hier entweder einen Wert oder einen Fehler geben kann. Die Fehlerbehandlung mit `try/catch` erfolgt an der Stelle, wo der Fehler auftritt. Die Aufrufer können ein `try/catch` nicht “vergessen”, sondern müssen dank der Signatur reagieren. Man braucht als Aufrufer kein umständliches `try/catch` mehr, welches den Kontrollfluss ändert; das Bearbeiten von “guten Werten” und “Fehlern” ist vereinheitlicht. Dank `map` und `flatMap` und `orElse` lassen sich die Aufrufe elegant verketteten, mit identischer Funktionalität wie in der ersten Variante.

5.7.12. Diskussion: Exceptions vs. Result/Optional

5.7.12.1. Wann sind Exceptions sinnvoll?

- Unerwartete, außergewöhnliche oder technische Fehler:
 - I/O-Fehler (Dateisystem, Netzwerk, Datenbank nicht erreichbar)
 - Ressourcenprobleme (`OutOfMemoryError`, zu wenig Speicher, kaputte Umgebung)
 - Programmierfehler / Invarianten verletzt (z.B. `IllegalStateException`)
- Fehler, die in der aktuellen Schicht **nicht sinnvoll behandelbar** sind:
 - Konfigurationsfehler beim Start
 - interne Konsistenzverletzungen

“Ich habe nicht erwartet, dass das passiert - und kann hier auch nicht sinnvoll darauf reagieren.”

Richtlinie:

- Exceptions nur für **Ausnahmesituationen** nutzen, die *nicht* zum normalen Verhalten gehören

- Exceptions dürfen das Programm abbrechen oder in eine übergeordnete Fehlerbehandlung führen (z.B. globale Fehlerseite, Log + Exit)

5.7.12.2. Wann Result / Optional?

- **Optional:**
 - “Wert fehlt, aber das ist ein normaler Fall (kein Fehler)”:
 - * Suche liefert eventuell nichts (`findUserByEmail` → `Optional`)
 - * Konfigurationseintrag ist optional
- **Result:**
 - Erwartbare, domänenspezifische Fehlerfälle **mit Bedeutung**:
 - * Validierungsfehler (Passwort zu kurz, E-Mail ungültig)
 - * Suchergebnis nicht eindeutig (`UserNichtEindeutig`, `UserNichtGefunden`)
 - * Domänenregeln verletzt (z.B. “Kontostand reicht nicht aus”)
 - Aufrufer *sollen* diese Fälle explizit behandeln (z.B. Fehlermeldung anzeigen, Eingabe erneut abfragen)

“Das gehört zum normalen Verhalten meiner Funktion - ich möchte explizit damit umgehen.”

Richtlinie:

- Domänenlogik bevorzugt mit `Result/Optional` modellieren
- Exceptions eher an den “Rändern” des Systems (I/O, Frameworks, technische Schicht)

5.7.12.3. Anmerkungen

In funktionalen Sprachen wie Haskell, Scala oder auch Rust sind solche Typen (`Maybe`, `Option`, `Result`) Standard. In Java müssen wir sie uns bewusst bauen bzw. einsetzen.

Typische **Schichtenaufteilung**:

- **Ganz außen (I/O, Framework):** Exceptions (z.B. Spring Controller, Datenbanktreiber)
- **In der Domänenlogik:** `Result / Optional` für erwartbare Fälle
- **Bei schwerwiegenden Invariantenbrüchen:** gezielte Runtime-Exceptions

Dadurch:

- bleibt der Kontrollfluss in der Domänenlogik “normal” (keine versteckten Sprünge),
- sind Fehlerfälle im Typ sichtbar,
- lassen sich Streams, Lambdas und Methodenreferenzen gut nutzen.

Important

Exceptions führen einen **zweiten Kontrollfluss** ein. Das kann schnell zu unübersichtlichen Strukturen und Abläufen führen.

Exceptions passen zudem schlecht zur modernen Verarbeitung mit der Stream-API, Lambda-Ausdrücken

und Methodenreferenzen.

Nutzen Sie - **wo immer es fachlich passt** - `Optional` und `Result`. Reservieren Sie Exceptions für wirklich außergewöhnliche oder unvorhersehbare Fälle.

5.7.13. Wrap-Up

1. `Optional<T>`:

- Vermeidet `null` und `NullPointerException`
- Macht normale “kein Wert”-Fälle im Typ sichtbar
- Nicht als universeller Ersatz für jedes `null`, sondern vor allem als Rückgabewert
- Bietet mit `map`, `flatMap`, `orElse*`, `ifPresent` bequeme Werkzeuge, in die Stream-API integriert

2. `Result<E, R>` (oder ähnliche Typen):

- Modelliert Erfolg **oder** Fehler explizit
- Typ zeigt: Funktion kann ein Ergebnis oder einen Fehler liefern
- Vereinfacht Testing: Kein `try/catch` nötig, Fehler sind nur Daten
- Mit `map` und `flatMap` lassen sich Pipelines elegant schreiben

3. Designentscheidungen: Fragen Sie sich: Ist dies ein normaler Fehlerfall?

- Ja → `Optional/Result`
- Nein → Exception

Zum Nachlesen

Lesen Sie zu diesem Thema im Dev.java-Tutorial von Oracle [“Using Optionals”](#) nach.

Weitere interessante Links:

- [“What You Might Not Know About Optional”](#)
- [“Experienced Developers Use These 7 Java Optional Tips to Remove Code Clutter”](#)
- [“Code Smells: Null”](#)
- [“Class Optional”](#)

Lernziele

- k2: Ich kann erklären, warum Optionals vor allem für Rückgabewerte gedacht sind
- k2: Ich kann erklären, warum kein `null` zurückgeliefert werden darf, wenn der Rückgabebetyp ein `Optional<T>` ist
- k3: Ich kann (ggf. leere) Optionals mit `Optional.ofNullable()` erzeugen
- k3: Ich kann auf Optionals klassisch über die direkten Hilfsmethoden der Klasse zugreifen
- k3: Ich kann auf Optionals elegant per Stream-API zugreifen
- k3: Ich kann Fehlerzustände über das Typsystem mit `Result<E, R>` modellieren

Challenges

Optional und Stream-API

1. Erklären Sie den folgenden Code-Schnipsel aus dem [Dungeon](#):

```

1 Skill fireball =
2     new Skill(
3         new FireballSkill(
4             () ->
5                 hero.fetch(CollideComponent.class)
6                     .map(cc -> cc
7                         .center(hero)
8                         .add(viewDirection.toPoint()))
9                     .orElseThrow(
10                        () -> MissingComponentException.build(
11                            hero,
12                            CollideComponent.class)),
13                 FIREBALL_RANGE,
14                 FIREBALL_SPEED,
15                 FIREBALL_DMG),
16         1);

```

Hinweise:

- `Entity#fetch: <T extends Component> Optional<T> fetch(final Class <T> klass)`
 - `CollideComponent#center: Point center(final Entity entity)`
 - `Point#add: Point add(final Point other)`
2. Was würde sich ändern, wenn statt `map` ein `flatMap` verwendet würde? Wie ist das bei richtigen Streams?
 3. Was passiert im folgenden Beispiel? Warum funktioniert das auch ohne terminale Stream-Operation?

```

1 Game.hero()
2     .flatMap(e -> e.fetch(AmmunitionComponent.class))
3     .map(AmmunitionComponent::resetCurrentAmmunition);

```

Hinweis: `Game.hero(): static Optional<Entity> hero()`.

4. Können Sie die beiden obigen Beispiele in "klassischer" Schreibweise umformulieren?

String-Handling

Können Sie den folgenden Code so umschreiben, dass Sie statt der `if`-Abfragen und der einzelnen direkten Methodenaufrufe die Stream-API und `Optional<T>` nutzen?

```

1 String format(final String text, String replacement) {
2     if (text.isEmpty()) {
3         return "";
4     }
5
6     final String trimmed = text.trim();
7     final String withSpacesReplaced = trimmed.replaceAll(" +", replacement)
8     ;

```



```
8  
9     return replacement + withSpacesReplaced + replacement;  
10 }
```

Ein Aufruf `format("Hello World ... ", "_");` liefert den String `"_Hello_World_..._"`.

6. Entwurfsmuster

6.1. Observer-Pattern

TL;DR

Eine Reihe von Objekten möchte über Änderungen in einem anderen ("zentralen") Objekt informiert werden. Dazu könnte das "zentrale" Objekt eine Zugriffsmethode anbieten, die die anderen Objekte regelmäßig aufrufen ("pollen").

Mit dem Observer-Pattern kann man das aktive Polling vermeiden und in ein Push-Modell umbauen. Die interessierten Objekte "registrieren" sich beim "zentralen" Objekt. Sobald dieses eine Änderung erfährt oder Informationen bereitstehen o.ä., wird das "zentrale" Objekt alle registrierten Objekte durch den Aufruf einer Methode benachrichtigen. Dafür müssen die registrierten Objekte eine gemeinsame Schnittstelle implementieren.

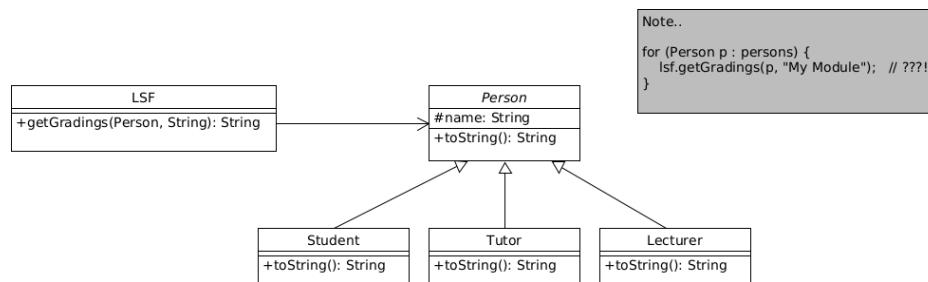
Das "zentrale" Objekt, welches abgefragt wird, nennt man "*Observable*" (oder manchmal auch "*Subject*"). Die Objekte, die die Information abfragen möchten, nennt man "*Observer*" (oder gelegentlich auch "*Client*").

Aspekt	Polling (Pull)	Observer (Push)
Wer startet den Kontakt?	Client (z.B. Student) fragt aktiv nach	Subject (z.B. LSF) meldet sich von sich aus
Häufigkeit	Regelmäßig, auch wenn sich nichts ändert	Nur, wenn sich der Zustand wirklich ändert
Kosten	Viele unnötige Anfragen	Nur relevante Benachrichtigungen
Kopplung	Alle kennen die konkrete(n) Abfragemethode(n)	Alle kennen nur das gemeinsame Observer -Interface

Videos

Vorlesung [[YT](#)], [[HSBI](#)]

6.1.1. Gedankenexperiment: Verteilung der Prüfungsergebnisse



Die Studierenden möchten nach einer Prüfung wissen, ob für einen bestimmten Kurs die/ihre Prüfungsergebnisse im LSF bereit stehen.

Dazu modelliert man eine Klasse `LSF` und implementiert eine Abfragemethode, die dann alle interessierten Objekte regelmäßig aufrufen können. Dies sieht dann praktisch etwa so aus:

```

1 List<Person> persons = List.of(new Lecturer("Frau Holle"),
2                               new Student("Heinz"),
3                               new Student("Karla"),
4                               new Tutor("Kolja"),
5                               new Student("Wuppie"));
6 LSF lsf = new LSF();
7
8 for (Person p : persons) {
9     lsf.getGradings(p, "My Module"); // polling
10 }
  
```

Eigentlich müsste man diese Abfrage natürlich **regelmäßig** machen, z.B. jede Minute:

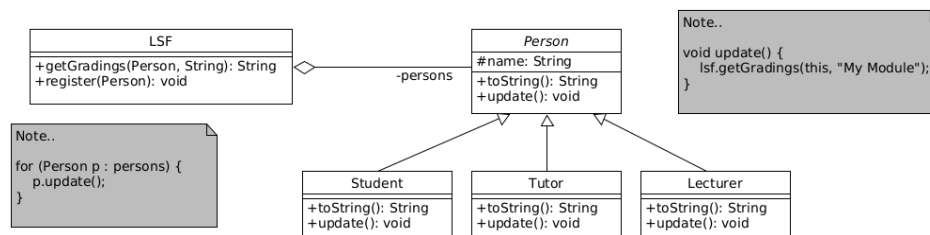
```

1 while (true) {
2     for (Person p : persons) {
3         lsf.getGradings(p, "My Module");
4     }
5
6     Thread.sleep(60_000); // jede Minute nachschauen
7 }
  
```

Problem:

- Alle fragen ständig aktiv nach, ob sich etwas geändert hat ("Polling").
- Meistens gibt es **nichts Neues**, trotzdem werden immer wieder Methoden aufgerufen.
- Das kostet Zeit/Ressourcen!
- Das gezeigte Vorgehen koppelt zudem alle Objekte (hier Personen) direkt an die konkrete Abfragemethode von `LSF`.

6.1.2. Elegantere Lösung: Observer-Entwurfsmuster



Sie erstellen im **LSF** eine Methode `register()`, mit der sich interessierte Personen beim **LSF** registrieren können.

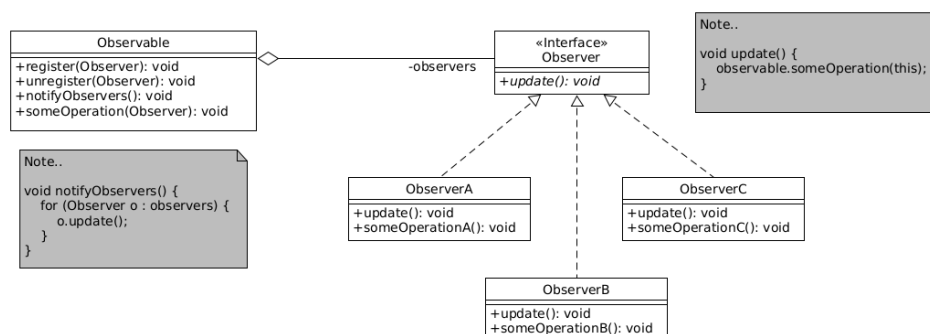
Zur Benachrichtigung der Personen (registrierte Objekte) brauchen diese eine geeignete Methode, die traditionell `update()` benannt wird.

Der typische Ablauf sieht dann so aus:

1. Interessierte Objekte (z.B. **Student**, **Lecturer**) implementieren eine Methode `update()`.
2. Diese Objekte rufen einmalig `lsf.register(this)` auf, um sich anzumelden.
3. Wenn sich im **LSF** etwas ändert (z.B. neue Noten verfügbar sind), ruft **LSF** von sich aus seine interne Methode `notifyObservers()` auf.
4. `notifyObservers()` ruft dann auf allen registrierten Objekten deren `update()`-Methode auf.

[Demo: observer](#)

6.1.3. Observer-Pattern verallgemeinert



Im vorigen Beispiel wurde die Methode `update()` einfach der gemeinsamen Basisklasse **Person** hinzugefügt. Normalerweise möchte man die Aspekte **Person** und **Observer** aber sauber trennen und definiert sich dazu ein separates Interface **Observer** mit der Methode `update()`, die dann alle "interessierten" Klassen (zusätzlich zur bestehenden Vererbungshierarchie) implementieren.

Ein mögliches, sehr einfaches Java-Skelett für unser LSF-/Student-Beispiel sähe entsprechend folgendermaßen aus:

```
1 public interface Observer {
2     void update();
3 }
4
5 public class LSF {
6     private List<Observer> observers = new ArrayList<>();
7
8     public void register(Observer o) {
9         observers.add(o);
10    }
11
12    public void unregister(Observer o) {
13        observers.remove(o);
14    }
15
16    void notifyObservers() {
17        for (Observer o : observers) {
18            o.update();
19        }
20    }
21
22    public void neueNotenSindDa() {
23        // ... interne Logik ...
24        notifyObservers(); // alle angemeldeten Observer benachrichtigen
25    }
26 }
27
28 public class Student extends Person implements Observer {
29     @Override
30     public void update() {
31         System.out.println("Student: Ich schaue mir die neuen Noten an.");
32     }
33 }
```

Die Klasse für das zu beobachtende Objekt benötigt dann eine Methode `register()`, mit der sich Observer registrieren können. Die Objektreferenzen werden dabei einfach einer geeigneten internen Sammlung hinzugefügt.

Häufig findet sich dann noch eine Methode `unregister()`, mit der sich bereits registrierte Beobachter wieder abmelden können. Weiterhin findet man häufig eine Methode `notifyObservers()`, die man von außen auf dem beobachteten Objekt aufrufen kann und die dann auf allen registrierten Beobachtern deren Methoden `update()` aufruft. (Dieser Vorgang kann aber auch durch eine sonstige Zustandsänderung im beobachteten Objekt durchgeführt werden.)

Der typische Lebenszyklus eines Observers sieht also so aus:

1. Observer-Objekt wird erzeugt (z.B. `new Student()`).
2. Das Observer-Objekt registriert sich einmalig beim Observable (z.B. beim `LSF lsf` per `lsf.register(student)`).
3. Das Observable ändert irgendwann seinen Zustand (z.B. neue Noten).
4. Das Observable ruft intern `notifyObservers()` auf.
5. `notifyObservers()` ruft bei allen registrierten Observern `update(...)` auf.

Tip

Im obigen Beispiel wurde für die Observer eine Liste verwendet: `List<Observer> observers`. Je nach Anwendungsfall kann das aber auch eine andere Datenstruktur sein - beispielsweise könnte man mit einer Menge (`Set`) vermeiden, dass sich Observer mehrfach registrieren können. Der verwendete Datentyp hängt nicht am Observer-Pattern!

Important

In der Standarddefinition des Observer-Patterns nach (Gamma u. a. 2011) werden beim Aufruf der Methode `update()` **keine Werte** an die Observer mitgegeben. Jeder Observer muss sich entsprechend eine eigene Referenz auf das beobachtete Objekt halten, um von dort dann weitere Informationen erhalten zu können. Wir nutzen im Rahmen dieses Moduls in der Regel eine **erweiterte Variante**, bei der `update()` einen oder mehrere Parameter mitbekommt, z.B.

- eine Referenz auf das `Observable`, oder (meist besser)
- direkt die relevanten Daten.

Ein angepasstes Interface für das obige Beispiel könnte beispielsweise so aussehen, wenn wir eine Referenz auf das `Observable` übergeben:

```
1 public interface Observer {  
2     void update(LSF source);  
3 }
```

Oder noch besser, wir geben direkt die relevanten Daten mit:

```
1 public interface Observer {  
2     void update(Gradings neueNoten);  
3 }
```

Mit der zweiten Variante werden die relevanten Daten direkt an den Observer mitgegeben und diese sparen sich dadurch die entsprechende Nachfrage beim `Observable`.

Dies muss dann natürlich im `Observer`-Interface nachgezogen werden.

Caution

Die typische Implementierung von `notifyObservers` sieht ungefähr so aus:

```
1 void notifyObservers() {  
2     for (Observer o : observers) {  
3         o.update();  
4     }  
5 }
```

Für jeden Observer wird die Methode `update()` aufgerufen (ob nun mit oder ohne Argumente). Damit geht der Kontrollfluss an den jeweiligen Observer und dessen Implementation von `update()` über und kehrt erst hier in die Schleife zurück, wenn `update()` im aktuellen Observer fertig ist. Das setzt voraus, dass sich alle Observer vernünftig verhalten!

Tip

Es gibt in der Java-Standardbibliothek (`java.util`) bereits die Klassen `Observer` und `Observable`, die aber als “deprecated” gekennzeichnet sind. Sinnvollerweise nutzen Sie nicht diese vorgegebene Variante, sondern implementieren Ihre eigenen Interfaces/Klassen, wenn Sie das Observer-Pattern einsetzen wollen!

6.1.4. Wrap-Up

Observer-Pattern: Benachrichtige registrierte Objekte über Statusänderungen

- Interface `Observer` mit Methode `update()`
- Interessierte Objekte als “Beobachter”:
 1. implementieren das Interface `Observer`
 2. registrieren sich beim beobachteten Objekt (`Observable` / `Subject`)
- Beobachtetes Objekt ruft auf allen registrierten Objekten `update()` auf
- `update()` kann auch Parameter haben (z.B. Quelle oder neue Daten)

Zum Nachlesen

Auch wenn es für C++ geschrieben ist, lässt sich zum Thema Observer-Pattern das Kapitel 4 “Observer” im Nystrom (2014) sehr gut lesen. Der Verweis auf Gamma u. a. (2011) der “Gang of Four” darf natürlich nicht fehlen.

Lernziele

- k2: Ich kenne den Aufbau des Observer-Patterns und kann dies an einem Beispiel erklären
- k3: Ich kann das Observer-Pattern auf konkrete Beispiele (etwa den PM-Dungeon) anwenden

Challenges**Observer: Restaurant**

Stellen Sie sich ein Restaurant vor, in welchem man nicht eine komplette Mahlzeit bestellt, sondern aus einzelnen Komponenten auswählen kann. Die Kunden bestellen also die gewünschten Komponenten, suchen sich einen Tisch und warten auf die Fertigstellung ihrer Bestellung. Da die Küche leider nur sehr klein ist, werden immer alle Bestellungen einer bestimmten Komponente zusammen bearbeitet - also beispielsweise werden alle bestellten Salate angerichtet oder die alle bestellten Pommes-Portionen zubereitet. Sobald eine solche Komponente fertig ist, werden alle Kunden aufgerufen, die diese Komponente bestellt haben ...

Modellieren Sie dies in Java. Nutzen Sie dazu das Observer-Pattern, welches Sie ggf. leicht anpassen müssen.

Tipp: Überlegen Sie, ob das Restaurant für **jede Komponente** eine eigene Liste von Observer (Kunden) führen sollte oder ob eine gemeinsame Liste mit zusätzlicher Information zur Komponente genügt.

Observer: Einzel- und Großhandel

In den **Vorgaben** finden Sie ein Modell für eine Lieferkette zwischen Großhandel und Einzelhandel. Wenn beim Einzelhändler eine Bestellung von einem Kunden eingeht (**Einzelhandel#bestellen**), speichert dieser den **Auftrag** zunächst in einer Liste ab. In regelmäßigen Abständen (**Einzelhandel#loop**) sendet der Einzelhändler die offenen Bestellungen an seinen Großhändler (**Grosshandel#bestellen**). Hat der Großhändler die benötigte Ware vorrätig, sendet er diese an den Einzelhändler (**Einzelhandel#empfangen**). Dieser kann dann den Auftrag gegenüber seinem Kunden erfüllen (keine Methode vorgesehen).

Anders als der Einzelhandel speichert der Großhandel keine Aufträge ab. Ist die benötigte Ware bei einer Bestellung also nicht oder nicht in ausreichender Zahl auf Lager, wird diese nicht geliefert und der Einzelhandel muss (später) eine neue Bestellung aufgeben.

Der Großhandel bekommt regelmäßig (**Grosshandel#loop**) neue Ware für die am wenigsten vorrätigen Positionen.

Im aktuellen Modell wird der Einzelhandel nicht über den neuen Lagerbestand des Großhändlers informiert und kann daher nur "zufällig" neue Bestellanfragen an den Großhändler senden.

Verbessern Sie das Modell, indem Sie das Observer-Pattern integrieren.

- Wer ist Observer?
- Wer ist Observable?
- Welche Informationen werden bei einem **update** mitgeliefert?

Tip: Überlegen Sie zuerst, **wo** der "interessante" Zustand liegt (z.B. Lagerbestand) und **wer** an Änderungen dieses Zustands interessiert ist.

Bauen Sie in alle Aktionen vom Einzelhändler und vom Großhändler passendes Logging ein.

Anmerkung: Sie dürfen nur die Vorgaben-Klassen **Einzelhandel** und **Grosshandel** verändern, die anderen Vorgaben-Klassen dürfen Sie nicht bearbeiten. Sie können zusätzlich benötigte eigene Klassen/Interfaces implementieren.

6.2. Template-Method-Pattern

TL;DR

Das Template-Method-Pattern ist ein Entwurfsmuster, bei dem ein gewisses Verhalten in einer Methode implementiert wird, die wie eine Schablone agiert, der sogenannten "Template-Methode". Darin werden dann u.a. Hilfsmethoden aufgerufen, die in der Basisklasse entweder als **abstract** markiert sind oder mit einem leeren Body implementiert sind ("Hook-Methoden"). Über diese Template-Methode legt also die Basisklasse ein gewisses Verhaltensschema fest ("Template") - daher auch der Name.

In den ableitenden Klassen werden dann die abstrakten Methoden und/oder die Hook-Methoden implementiert bzw. überschrieben und damit das Verhalten verfeinert.

Zur Laufzeit ruft man auf den Objekten die Template-Methode auf. Dabei wird von der Laufzeitumgebung der konkrete Typ der Objekte bestimmt (auch wenn man sie unter dem Typ der Oberklasse führt) und die am tiefsten in der Vererbungshierarchie implementierten Methoden aufgerufen. D.h. die Aufrufe der Hilfsmethoden in der Template-Methode führen zu den in der jeweiligen ableitenden Klasse implementierten Varianten.

Videos

Vorlesung [YT], [HSBI]

6.2.1. Motivation: Syntax-Highlighting im Tokenizer

In einem Compiler ist meist der erste Arbeitsschritt, den Eingabestrom in einzelne Token aufzubrechen. Dies sind oft die verschiedenen Schlüsselwörter, Operationen, Namen von Variablen, Methoden, Klassen etc. ... Aus der Folge von Zeichen (also dem eingelesenen Programmcode) wird ein Strom von Token, mit dem die nächste Stufe im Compiler dann weiter arbeiten kann.

```
1 public class Lexer {
2     private final List<Token> allToken; // alle verfügbaren Token-Klassen
3
4     public List<Token> tokenize(String string) {
5         List<Token> result = new ArrayList<>();
6
7         while (string.length() > 0) {
8             for (Token t : allToken) {
9                 Token token = t.match(string);
10                if (token != null) {
11                    result.add(token);
12                    string = string.substring(
13                        token.getContent().length(),
14                        string.length());
15                }
16            }
17        }
18
19        return result;
20    }
21 }
```

Dazu prüft man jedes Token, ob es auf den aktuellen Anfang des Eingabestroms passt. Wenn ein Token passt, erzeugt man eine Instanz dieser Token-Klasse und speichert darin den gematchten Eingabeteil, den man dann vom Eingabestrom entfernt. Danach geht man in die Schleife und prüft wieder alle Token ... bis irgendwann der Eingabestrom leer ist und man den gesamten eingelesenen Programmcode in eine dazu passende Folge von Token umgewandelt hat.

Anmerkung: Abgesehen von fehlenden Javadoc etc. hat das obige Code-Beispiel mehrere Probleme: Man würde im realen Leben nicht mit `String`, sondern mit einem Zeichenstrom arbeiten. Außerdem fehlt noch eine Fehlerbehandlung, wenn nämlich keines der Token in der Liste `allToken` auf den aktuellen Anfang des Eingabestroms passt.

6.2.2. Token-Klassen mit formatiertem Inhalt

Um den eigenen Tokenizer besser testen zu können, wurde beschlossen, dass jedes Token seinen Inhalt als formatiertes HTML-Schnipsel zurückliefern soll. Damit kann man dann alle erkannten Token formatiert ausgeben und erhält eine Art Syntax-Highlighting für den eingelesenen Programmcode.

```

1  public abstract class Token {
2      protected String content;
3
4      abstract protected String getHtml();
5  }
6  public class Keyword extends Token {
7      @Override
8      protected String getHtml() {
9          return "<font color=\"red\"><b>" + this.content + "</b></font>";
10     }
11 }
12 public class StringContent extends Token {
13     @Override
14     protected String getHtml() {
15         return "<font color=\"green\">" + this.content + "</font>";
16     }
17 }
18
19
20 Token t = new Keyword();
21 LOG.info(t.getHtml());

```

In der ersten Umsetzung erhält die Basisklasse `Token` eine weitere abstrakte Methode, die jede Token-Klasse implementieren muss und in der die Token-Klassen einen String mit dem Token-Inhalt und einer Formatierung für HTML zurückgeben.

Dabei fällt auf, dass der Aufbau immer gleich ist: Es werden ein oder mehrere Tags zum Start der Format-Sequenz mit dem Token-Inhalt verbunden, gefolgt mit einem zum verwendeten startenden HTML-Format-Tag passenden End-Tag.

Auch wenn die Inhalte unterschiedlich sind, sieht das stark nach einer Verletzung von **DRY** aus ...

6.2.3. Don't call us, we'll call you

```

1  public abstract class Token {
2      protected String content;
3
4      public final String getHtml() {
5          return htmlStart() + this.content + htmlEnd();
6      }
7
8      abstract protected String htmlStart();
9      abstract protected String htmlEnd();
10 }
11 public class Keyword extends Token {
12     @Override protected String htmlStart() { return "<font color=\"red\"><b>";
13     }
14     @Override protected String htmlEnd() { return "</b></font>"; }
15 }
16 public class StringContent extends Token {
17     @Override protected String htmlStart() { return "<font color=\"green\">"; }
18     @Override protected String htmlEnd() { return "</font>"; }
19 }

```

```

19
20
21 Token t = new Keyword();
22 LOG.info(t.getHtml());

```

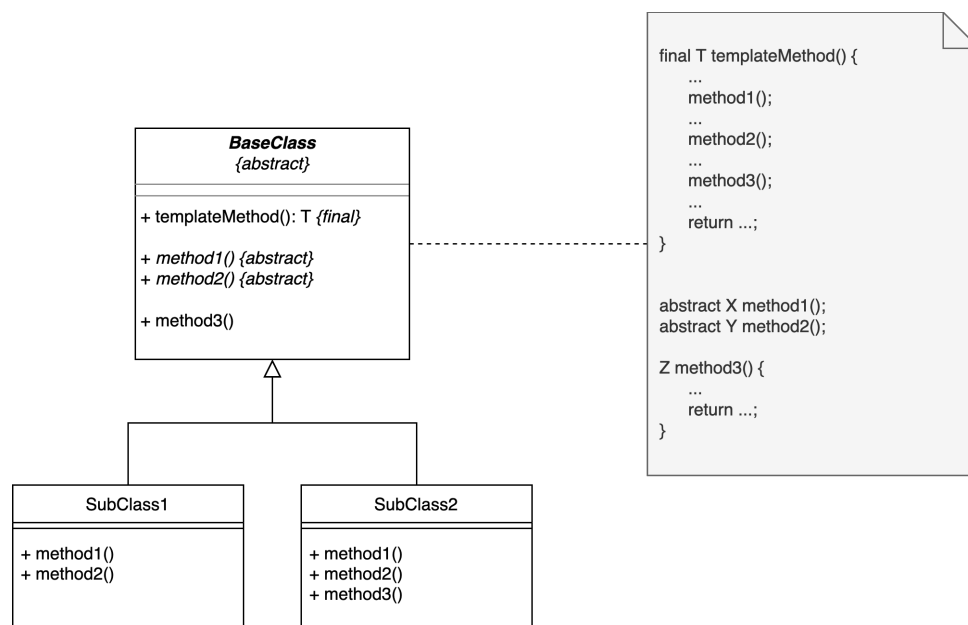
Wir können den Spaß einfach umdrehen ([“inversion of control”](#)) und die Methode zum Zusammenbasteln des HTML-Strings bereits in der Basisklasse implementieren. Dazu “rufen” wir dort drei Hilfsmethoden auf, die die jeweiligen Bestandteile des Strings (Format-Start, Inhalt, Format-Ende) erzeugen und deren konkrete Implementierung wir in der Basisklasse nicht kennen. Dies ist dann Sache der ableitenden konkreten Token-Klassen.

Objekte vom Typ `Keyword` sind dank der Vererbungsbeziehung auch `Token` (Vererbung: *is-a-Beziehung*). Wenn man nun auf einem `Token t` die Methode `getHtml()` aufruft, wird zur Laufzeit geprüft, welchen Typ `t` tatsächlich hat (im Beispiel `Keyword`). Methodenaufrufe werden dann mit den am tiefsten in der vorliegenden Vererbungshierarchie implementierten Methoden durchgeführt: Hier wird also die von `Token` geerbte Methode `getHtml()` in `Keyword` aufgerufen, die ihrerseits die Methoden `htmlStart()` und `htmlEnd()` aufruft. Diese sind in `Keyword` implementiert und liefern nun die passenden Ergebnisse.

Die Methode `getHtml()` wird auch als *“Template-Methode”* bezeichnet. Die beiden darin aufgerufenen Methoden `htmlStart()` und `htmlEnd()` in `Token` werden auch als *“Hilfsmethoden”* (oder *“Helper Methods”*) bezeichnet.

Dies ist ein Beispiel für das [Template-Method-Pattern](#).

6.2.4. Template-Method-Pattern



6.2.4.1. Aufbau Template-Method-Pattern

In der Basisklasse implementiert man eine Template-Methode (in der Skizze `templateMethod`), die sich auf anderen in der Basisklasse deklarierten (Hilfs-) Methoden “abstützt” (diese also aufruft; in der Skizze `method1`, `method2`, `method3`). Diese Hilfsmethoden können als **abstract** markiert werden und *müssen* dann von den ableitenden Klassen implementiert werden (in der Skizze `method1` und `method2`). Man kann aber auch einige/alle dieser aufgerufenen Hilfsmethoden in der Basisklasse implementieren (beispielsweise mit einem leeren Body - sogenannte “Hook”-Methoden) und die ableitenden Klassen *können* dann diese Methoden überschreiben und das Verhalten so neu formulieren (in der Skizze `method3`).

Damit werden Teile des Verhaltens an die ableitenden Klassen ausgelagert.

6.2.4.2. Verwandtschaft zum Strategy-Pattern

Das Template-Method-Pattern hat eine starke Verwandtschaft zum Strategy-Pattern.

Im Strategy-Pattern haben wir Verhalten komplett an andere Objekte *delegiert*, indem wir in einer Methode einfach die passende Methode auf dem übergebenen Strategie-Objekt aufgerufen haben.

Im Template-Method-Pattern nutzen wir statt Delegation die Mechanismen Vererbung und dynamische Polymorphie und definieren in der Basis-Klasse abstrakte oder Hook-Methoden, die wir bereits in der Template-Methode der Basis-Klasse aufrufen. Damit ist das grobe Verhalten in der Basis-Klasse festgelegt, wird aber in den ableitenden Klassen durch das dortige Definieren oder Überschreiben der Hilfsmethoden verfeinert. Zur Laufzeit werden dann durch die dynamische Polymorphie die tatsächlich implementierten Hilfsmethoden in den ableitenden Klassen aufgerufen. Damit lagert man im Template-Method-Pattern gewissermaßen nur Teile des Verhaltens an die ableitenden Klassen aus.

6.2.5. Wrap-Up

Template-Method-Pattern: Verhaltensänderung durch Vererbungsbeziehungen

- Basis-Klasse:
 - Template-Methode, die Verhalten definiert und Hilfsmethoden aufruft
 - Hilfsmethoden: Abstrakte Methoden (oder “Hook”: Basis-Implementierung)
- Ableitende Klassen: Verfeinern Verhalten durch Implementieren der Hilfsmethoden
- Zur Laufzeit: Dynamische Polymorphie: Aufruf der Template-Methode nutzt die im tatsächlichen Typ des Objekts implementierten Hilfsmethoden

Zum Nachlesen

Der [Refactoring.Guru](#) hat eine schöne Zusammenfassung des Template-Method-Patterns. Der Verweis auf Gamma u. a. (2011) der “[Gang of Four](#)” darf natürlich nicht fehlen.

Lernziele

- k3: Ich kann das Template-Method-Entwurfsmuster praktisch anwenden

Challenges

Schreiben Sie eine abstrakte Klasse Drucker. Implementieren Sie die Funktion `kopieren`, bei der zuerst die Funktion `scannen` und dann die Funktion `drucken` aufgerufen wird. Der Kopiervorgang ist für alle Druckertypen identisch, das Scannen und Drucken ist abhängig vom Druckertyp.

Implementieren Sie zusätzlich zwei unterschiedliche Druckertypen:

- `Tintendrucker` **extends** `Drucker`
 - `Tintendrucker#drucken` loggt den Text "Drucke das Dokument auf dem Tintendrucker."
 - `Tintendrucker#scannen` loggt den Text "Scanne das Dokument mit dem Tintendrucker."
- `Laserdrucker` **extends** `Drucker`
 - `Laserdrucker#drucken` loggt den Text "Drucke das Dokument auf dem Laserdrucker."
 - `Laserdrucker#scannen` loggt den Text "Scanne das Dokument mit dem Laserdrucker."

Nutzen Sie das Template-Method-Pattern.

6.3. Visitor-Pattern

TL;DR

Häufig bietet es sich bei Datenstrukturen an, die Traversierung nicht direkt in den Klassen der Datenstrukturen zu implementieren, sondern in Hilfsklassen zu verlagern. Dies gilt vor allem dann, wenn die Datenstruktur aus mehreren Klassen besteht (etwa ein Baum mit verschiedenen Knotentypen) und/oder wenn man nicht nur eine Traversierungsart ermöglichen will oder/und wenn man immer wieder neue Arten der Traversierung ergänzen will. Das würde nämlich bedeuten, dass man für jede weitere Form der Traversierung in *allen* Klassen eine entsprechende neue Methode implementieren müsste.

Das Visitor-Pattern lagert die Traversierung in eigene Klassenstruktur aus.

Die Klassen der Datenstruktur bekommen nur noch eine `accept()`-Methode, in der ein Visitor übergeben wird und rufen auf diesem Visitor einfach dessen `visit()`-Methode auf (mit einer Referenz auf sich selbst als Argument).

Der Visitor hat für jede Klasse der Datenstruktur eine Überladung der `visit()`-Methode. In diesen kann er je nach Klasse die gewünschte Verarbeitung vornehmen. Üblicherweise gibt es ein Interface oder eine abstrakte Klasse für die Visitor, von denen dann konkrete Visitor ableiten.

Bei Elementen mit "Kindern" muss man sich entscheiden, wie die Traversierung implementiert werden soll. Man könnte in der `accept()`-Methode den Visitor an die Kinder weiter reichen (also auf den Kindern `accept()` mit dem Visitor aufrufen), bevor man die `visit()`-Methode des Visitors mit sich selbst als Referenz aufruft. Damit ist die Form der Traversierung in den Klassen der Datenstruktur fest verankert und über den Visitor findet "nur" noch eine unterschiedliche Form der Verarbeitung statt. Alternativ überlässt man es dem Visitor, die Traversierung durchzuführen: Hier muss in den `visit()`-Methoden für

die einzelnen Elemente entsprechend auf mögliche Kinder reagiert werden.

In diesem Pattern spricht man von “Double-Dispatch”:

- Zur Compile-Zeit ist bei einem Aufruf wie `e.accept(v)` nur der Obertyp `Expr` bekannt. Zur **Laufzeit** entscheidet der tatsächliche Typ von `e` (z.B. `AddExpr`), welche konkrete `accept`-Methode ausgeführt wird. -> *erster Dispatch*: dynamischer Dispatch auf das **Element**.
- In der jeweiligen `accept`-Methode steht dann z.B. `v.visit(this)`. Der **statische** Typ von `this` ist dort z.B. `AddExpr`, dadurch ist zur Compile-Zeit klar, welche `visit`-Überladung gemeint ist (`visit(AddExpr)`). Zur **Laufzeit** entscheidet der tatsächliche Typ von `v` (z.B. `EvalVisitor` vs. `PrintVisitor`), welche Implementierung dieser `visit(AddExpr)`-Methode aufgerufen wird. -> *zweiter Dispatch*: dynamischer Dispatch auf den **Visitor**.

Zusammen: Die endgültige Methode ergibt sich aus dem Zusammenspiel des Laufzeittyps des Elements (welche `accept`-Methode) und des Laufzeittyps des Visitors (welche `visit`-Implementierung für den konkreten Elementtyp).

Das Pattern wird traditionell gern für die Traversierung von Datenstrukturen eingesetzt. Es hilft aber auch, wenn man einer gewissen Anzahl von Klassen je eine neue Hilfsmethode hinzufügen möchte - normalerweise müsste man jetzt jede Klasse einzeln ergänzen. Mit dem Visitor-Pattern muss lediglich ein neuer Visitor mit den Hilfsmethoden implementiert werden.

Videos

Vorlesung [YT], [HSBI]

Demo:

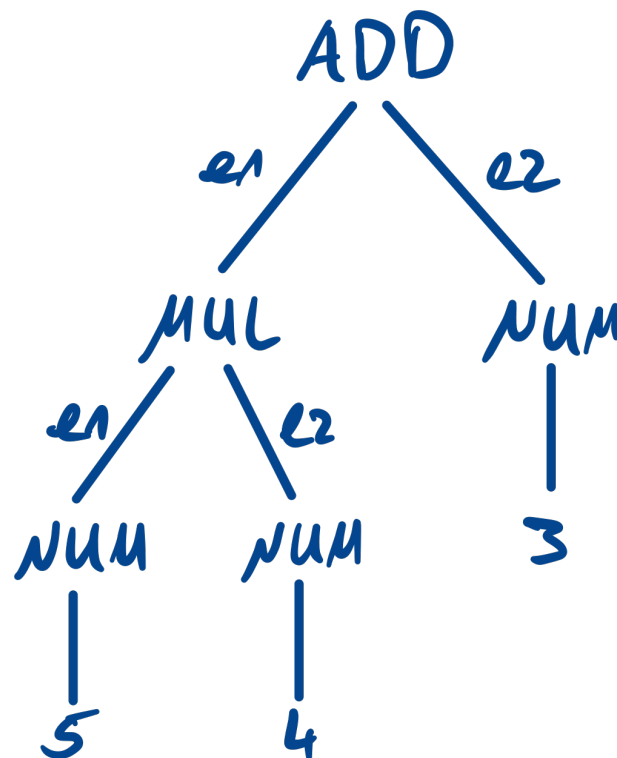
- Bäume und direkte Traversierung (1/3) [YT], [HSBI]
- Traversierung mit dem Visitor-Pattern (2/3) [YT], [HSBI]
- Double Dispatch (3/3) [YT], [HSBI]

6.3.1. Motivation: Parsen von “5*4+3”

Zum Parsen von Ausdrücken (*Expressions*) könnte man diese einfache Grammatik einsetzen. Ein Ausdruck ist dabei entweder ein einfacher Integer oder eine Addition oder Multiplikation zweier Ausdrücke.

```
1 add : mul ('+' mul)* ;
2 mul : num ('*' num)* ;
3 num : INT ;
```

Beim Parsen von “5*4+3” würde dabei der folgende Parse-Tree entstehen:

**Tip**

Wer genau hinschaut und sich schon die ANTLR-Session angeschaut hat, wird merken, dass dieser Baum tatsächlich *kein* Parse-Tree von ANTLR basierend auf der Grammatik ist. Hier habe ich schon etwas vorgearbeitet und den Parse-Tree vereinfacht - das nennt man dann auch oft "Abstract Syntax Tree" (AST). Wir werden so eine Transformation von einem vollständigen Parse-Tree hin zu einem AST auch noch als Übungsaufgabe programmieren und werden das Visitor-Pattern nutzen, um den Parse-Tree zu traversieren.

6.3.2. Erinnerung: Baumstrukturen**Begriffe**

- Ein Baum ist eine **hierarchische Datenstruktur**
 - besteht aus **Knoten** (Nodes) und **Kanten** (Edges)
 - es gibt genau **eine Wurzel** (Root)
 - jeder Knoten (außer der Wurzel) hat **genau einen Elternknoten**
- Knoten mit Kindern: **innere Knoten**, ohne Kinder: **Blätter**
- Wichtige Begriffe:
 - **Tiefe** eines Knotens = Länge des Pfads von der Wurzel bis zu diesem Knoten
 - **Höhe** des Baums = maximale Tiefe eines Knotens
 - **Pfad**: Folge von Knoten von der Wurzel zu einem Knoten

Beispiel



- Root: **Root**
- Innere Knoten: **Root, A, B**
- Blätter: **C, D, E**

Typische Einsatzgebiete

- Dateisystem
- GUI-Komponentenbäume
- **Parsebäume** für Programme/Sprachen

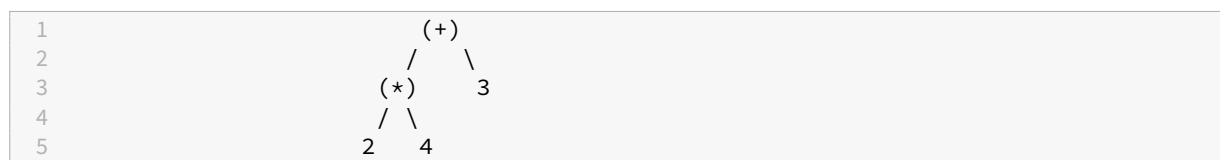
Tip

Die Parse-Trees in ANTLR sind genau solche Bäume:

- Wurzel = kompletter Input
- innere Knoten = Grammatikregeln / Sprachkonstrukte
- Blätter = Token

6.3.3. Erinnerung: Varianten der Datenhaltung

Variante A: **Daten nur in den Blättern**



Variante B: **Daten in allen Knoten**

6.3.4. Erinnerung: Implementierungsvarianten

Binäre Bäume (explizite Felder)

- zwei Kinder: **left, right**
- typisch für: Binärbäume, Binärsuchbäume, Heaps, ...

```

1 public class BinaryTreeNode {
2     public BinaryTreeNode left;
3     public BinaryTreeNode right;
4 }
  
```



```

5     public Object value; // Payload
6 }

```

Allgemeine (n-äre) Bäume (Kinderliste)

- beliebig viele Kinder: Speicherung in einer Liste
- typisch für: **Parsebäume / ASTs**, XML/HTML-DOM, GUI-Hierarchien, ...

```

1 public class TreeNode {
2     public List<TreeNode> children;
3
4     public Object value; // Payload
5 }

```

Oder auch mit **getrennten Typen für Knoten und Blätter** (hier im Beispiel: Daten nur in den Blättern):

```

1 public interface Tree {}
2 public class Node implements Tree {
3     Tree leftChild;
4     Tree rightChild;
5 }
6 public class Leaf implements Tree {
7     Object data;
8 }

```

Tip

Die ANTLR-Parsebäume sind im Wesentlichen **allgemeine (n-äre) Bäume mit Kinderliste**. Die Knoten enthalten typischerweise:

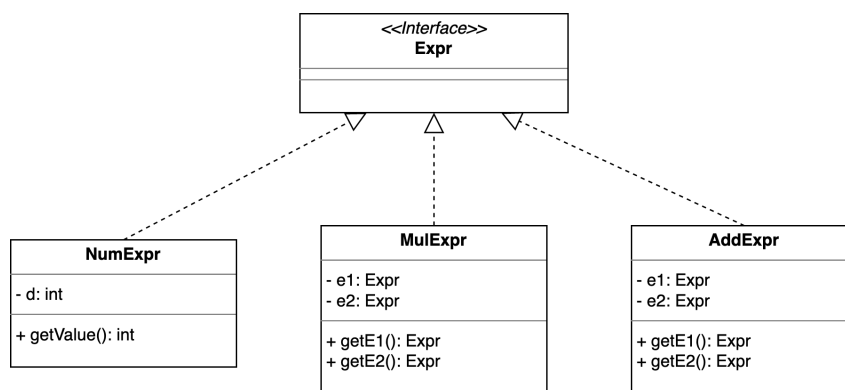
- Kontext für eine Grammatikregel (für innere Knoten), oder
- ein Token (für Blätter)

6.3.5. Erinnerung: Algorithmische Varianten bei Bäumen

- **Suchbäume**
 - Binäre Suchbäume (BST):
 - * Invariante: links < Wert < rechts
 - Selbstbalancierende Bäume:
 - * **AVL-Bäume**
 - * **Rot-Schwarz-Bäume**
- **Mehrweg-Suchbäume**
 - **B-Bäume, B+-Bäume**
 - * mehrere Schlüssel und mehrere Kinder pro Knoten
- **Heaps**

- Min-Heap, Max-Heap
- Grundlage für Prioritätswarteschlangen
- Einordnung:
 - Diese Varianten optimieren v.a. **Laufzeiten** (Suchen, Einfügen, Löschen)
 - Für das **Visitor-Pattern** heute wichtig:
 - * Wir brauchen “nur” eine Baumstruktur, über die wir systematisch laufen
 - * Ob der Baum balanciert ist oder nicht, ist für das Traversieren zweitrangig

6.3.6. Strukturen für den Parse-Tree



Der Parse-Tree für diese einfache Grammatik ist ein Binärbaum. Die Regeln werden auf Knoten im Baum zurückgeführt. Es gibt Knoten mit zwei Kindknoten, und es gibt Knoten ohne Kindknoten (“Blätter”).

Entsprechend kann man sich einfache Klassen definieren, die die verschiedenen Knoten in diesem Parse-Tree repräsentieren. Als Obertyp könnte es ein (noch leeres) Interface **Expr** geben.

```

1  public interface Expr {}
2
3  public class NumExpr implements Expr {
4      private final int d;
5
6      public NumExpr(int d) { this.d = d; }
7  }
8
9  public class MulExpr implements Expr {
10     private final Expr e1;
11     private final Expr e2;
12
13     public MulExpr(Expr e1, Expr e2) {
14         this.e1 = e1; this.e2 = e2;
15     }
16 }
17
18 public class AddExpr implements Expr {
19     private final Expr e1;
20     private final Expr e2;
21 }
  
```

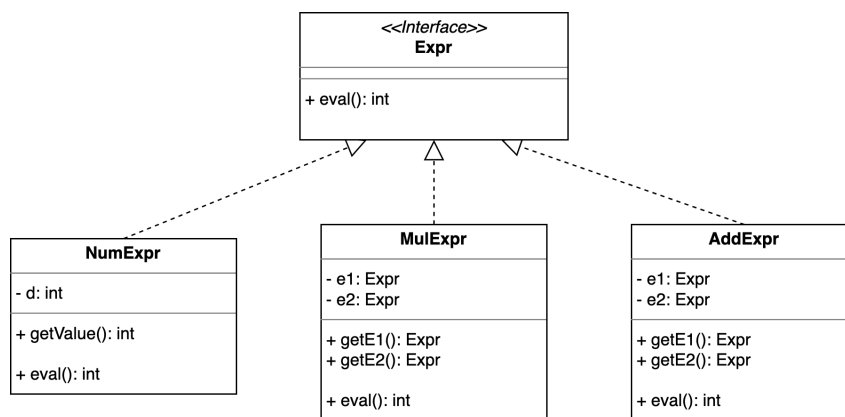
```

22     public AddExpr(Expr e1, Expr e2) {
23         this.e1 = e1; this.e2 = e2;
24     }
25 }
26
27
28 public class DemoExpr {
29     public static void main(final String... args) {
30         // 5*4+3
31         Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
            NumExpr(3));
32     }
33 }

```

6.3.7. Ergänzung I: Ausrechnen des Ausdrucks

Es wäre nun schön, wenn man mit dem Parse-Tree etwas anfangen könnte. Vielleicht möchte man den Ausdruck ausrechnen?



Zum Ausrechnen des Ausdrucks könnte man dem Interface eine `eval()`-Methode spendieren. Jeder Knoten kann für sich entscheiden, wie die entsprechende Operation ausgewertet werden soll: Bei einer `NumExpr` ist dies einfach der gespeicherte Wert, bei Addition oder Multiplikation entsprechend die Addition oder Multiplikation der Auswertungsergebnisse der beiden Kindknoten.

```

1  public interface Expr {
2      int eval();
3  }
4
5  public class NumExpr implements Expr {
6      private final int d;
7
8      public NumExpr(int d) { this.d = d; }
9      public int eval() { return d; }
10 }
11
12 public class MulExpr implements Expr {
13     private final Expr e1;

```

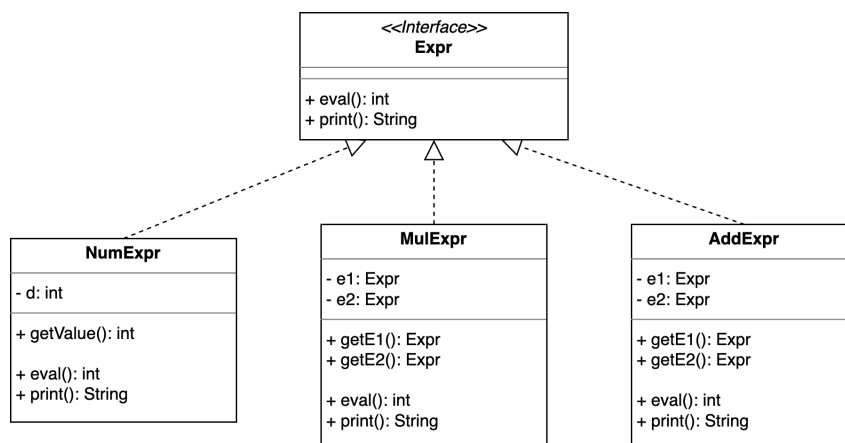
```

14     private final Expr e2;
15
16     public MulExpr(Expr e1, Expr e2) {
17         this.e1 = e1; this.e2 = e2;
18     }
19     public int eval() { return e1.eval() * e2.eval(); }
20 }
21
22 public class AddExpr implements Expr {
23     private final Expr e1;
24     private final Expr e2;
25
26     public AddExpr(Expr e1, Expr e2) {
27         this.e1 = e1; this.e2 = e2;
28     }
29     public int eval() { return e1.eval() + e2.eval(); }
30 }
31
32
33 public class DemoExpr {
34     public static void main(final String... args) {
35         // 5*4+3
36         Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
            NumExpr(3));
37
38         int erg = e.eval();
39     }
40 }

```

6.3.8. Ergänzung II: Pretty-Print des Ausdrucks

Nachdem das Ausrechnen so gut geklappt hat, will der Chef nun noch flink eine Funktion, mit der man den Ausdruck hübsch ausgeben kann:

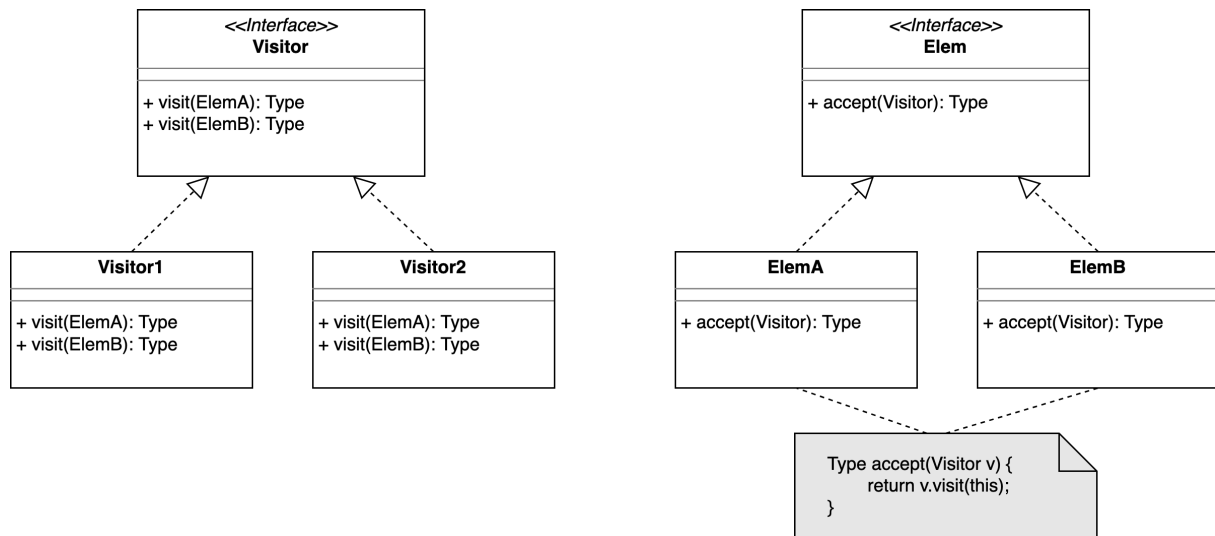


Beispiel: `direct.DemoExpr`

Das fängt an, sich zu wiederholen. Wir implementieren immer wieder ähnliche Strukturen, mit denen wir diesen Parse-Tree traversieren ... Und wir müssen für *jede* Erweiterung immer *alle* Expression-Klassen anpassen!

Das geht besser!

6.3.9. Visitor-Pattern (Besucher-Entwurfsmuster)



Das Entwurfsmuster “Besucher” (*Visitor Pattern*) lagert die Aktion beim Besuchen eines Knotens in eine separate Klasse aus.

Dazu bekommt jeder Knoten im Baum eine neue Methode, die einen Besucher akzeptiert. Dieser Besucher kümmert sich dann um die entsprechende Verarbeitung des Knotens, also um das Auswerten oder Ausgeben im obigen Beispiel.

Die Besucher haben eine Methode, die für jeden zu bearbeitenden Knoten überladen wird. In dieser Methode findet dann die eigentliche Verarbeitung statt: Auswerten des Knotens oder Ausgeben des Knotens ...

```

1 public interface Expr {
2     void accept(ExprVisitor v);
3 }
4
5 public class NumExpr implements Expr {
6     private final int d;
7
8     public NumExpr(int d) { this.d = d; }
9     public int getValue() { return d; }
10
11     public void accept(ExprVisitor v) { v.visit(this); }
12 }
13
14 public class MulExpr implements Expr {
15     private final Expr e1;
  
```

```
16     private final Expr e2;
17
18     public MulExpr(Expr e1, Expr e2) {
19         this.e1 = e1; this.e2 = e2;
20     }
21     public Expr getE1() { return e1; }
22     public Expr getE2() { return e2; }
23
24     public void accept(ExprVisitor v) { v.visit(this); }
25 }
26
27 public class AddExpr implements Expr {
28     private final Expr e1;
29     private final Expr e2;
30
31     public AddExpr(Expr e1, Expr e2) {
32         this.e1 = e1; this.e2 = e2;
33     }
34     public Expr getE1() { return e1; }
35     public Expr getE2() { return e2; }
36
37     public void accept(ExprVisitor v) { v.visit(this); }
38 }
39
40
41 public interface ExprVisitor {
42     void visit(NumExpr e);
43     void visit(MulExpr e);
44     void visit(AddExpr e);
45 }
46
47 public class EvalVisitor implements ExprVisitor {
48     private final Stack<Integer> erg = new Stack<>();
49
50     public void visit(NumExpr e) { erg.push(e.getValue()); }
51     public void visit(MulExpr e) {
52         // rechte Seite zuerst in den Stack, damit beim pop() als zweites
53         // Argument in die Operation
54         // (hier nicht relevant, aber bei nicht-kommutativen Operationen
55         // wichtig)
56         e.getE2().accept(this); e.getE1().accept(this);
57         erg.push(erg.pop() * erg.pop());
58     }
59     public void visit(AddExpr e) {
60         e.getE2().accept(this); e.getE1().accept(this);
61         erg.push(erg.pop() + erg.pop());
62     }
63     public int getResult() { return erg.peek(); }
64 }
65
66 public class PrintVisitor implements ExprVisitor {
67     private final Stack<String> erg = new Stack<>();
68
69     public void visit(NumExpr e) { erg.push("NumExpr(" + e.getValue() + ")"); }
70     public void visit(MulExpr e) {
71         e.getE2().accept(this); e.getE1().accept(this);
72         erg.push("MulExpr(" + erg.pop() + ", " + erg.pop() + ")");
73     }
74 }
```

```

71     }
72     public void visit(AddExpr e) {
73         e.getE2().accept(this); e.getE1().accept(this);
74         erg.push("AddExpr(" + erg.pop() + ", " + erg.pop() + ")");
75     }
76     public String getResult() { return erg.peek(); }
77 }
78
79
80 public class DemoExpr {
81     public static void main(final String... args) {
82         // 5*4+3
83         Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new
            NumExpr(3));
84
85         EvalVisitor v1 = new EvalVisitor();
86         e.accept(v1);
87         int erg = v1.getResult();
88
89         PrintVisitor v2 = new PrintVisitor();
90         e.accept(v2);
91         String s = v2.getResult();
92     }
93 }

```

6.3.9.1. Implementierungsdetail 1: externe vs. interne Traversierung

In den beiden Klasse `AddExpr` und `MulExpr` müssen auch die beiden Kindknoten besucht werden, d.h. hier muss der Baum weiter traversiert werden.

Man kann sich überlegen, diese Traversierung in den Klassen `AddExpr` und `MulExpr` selbst anzustoßen.

Alternativ könnte auch der Visitor die Traversierung vornehmen. Gerade bei der Traversierung von Datenstrukturen ist diese Variante oft von Vorteil, da man hier unterschiedliche Traversierungsarten haben möchte (Breitensuche vs. Tiefensuche, Pre-Order vs. Inorder vs. Post-Order, ...) und diese elegant in den Visitor verlagern kann.

Beispiel: Externe Traversierung

Bei der externen Traversierung liegt die Verantwortung für das Ablaufen der Datenstruktur beim Visitor. Im nachfolgenden Beispiel muss der Visitor zunächst die Kinder des `AddExpr` auswerten, bevor die Auswertung des Knotens selbst erfolgen kann (nur relevante Ausschnitte gezeigt):

```

1  public class AddExpr implements Expr {
2      private final Expr e1;
3      private final Expr e2;
4
5      @Override
6      public void accept(ExprVisitor v) {
7          v.visit(this);                // akzeptiere Visitor zur Verarbeitung
              des Knotens
8      }

```

```

9  }
10
11 public class EvalVisitor implements ExprVisitor {
12     private final Stack<Integer> erg = new Stack<>();
13
14     @Override
15     public void visit(AddExpr e) {
16         e.getE2().accept(this);           // Traversierung im Visitor ("extern")
17         e.getE1().accept(this);
18         erg.push(erg.pop() + erg.pop()); // Verarbeitung des Knotens selbst
19     }
20 }

```

Beispiel Traversierung extern (im Visitor): `visitor.visit.extrav.DemoExpr`

Beispiel: Interne Traversierung

Bei der internen Traversierung liegt die Verantwortung für das Ablaufen der Datenstruktur bei der Datenstruktur selbst und nicht beim Visitor (nur relevante Ausschnitte gezeigt):

```

1  public class AddExpr implements Expr {
2      private final Expr e1;
3      private final Expr e2;
4
5      @Override
6      public void accept(ExprVisitor v) {
7          e2.accept(v);           // Traversierung in Datenstruktur ("intern")
8          e1.accept(v);
9          v.visit(this);         // akzeptiere Visitor zur Verarbeitung
                                // des Knotens
10     }
11 }
12
13 public class EvalVisitor implements ExprVisitor {
14     private final Stack<Integer> erg = new Stack<>();
15
16     @Override
17     public void visit(AddExpr e) {
18         erg.push(erg.pop() + erg.pop()); // Verarbeitung des Knotens selbst
19     }
20 }

```

Beispiel Traversierung intern (in den Knotenklassen): `visitor.visit.intrav.DemoExpr`

6.3.9.2. Implementierungsdetail 2: Überladene vs. unterschiedlich benannte `visit`-Methoden

Bisher haben wir das Visitor-Interface so definiert:

```

1  public interface ExprVisitor {
2      void visit(NumExpr e);
3      void visit(MulExpr e);
4      void visit(AddExpr e);

```



```
5 }
```

Hier wird **eine Methode** `visit(...)` mehrfach mit unterschiedlichen Parameter-Typen überladen. Der Methodenname transportiert die *Aktion* ("besuche dieses Element"), der Parametertyp transportiert die *Variante* (welche konkrete Unterklasse von `Expr`).

In vielen Bibliotheken und Frameworks (und auch bei ANTLR) findet man aber häufig eine andere Schreibweise:

```
1 public interface ExprVisitor {
2     void visitNumExpr(NumExpr e);
3     void visitMulExpr(MulExpr e);
4     void visitAddExpr(AddExpr e);
5 }
```

Hier haben die `visit`-Methoden **unterschiedliche Namen**. Technisch ist das aber immer noch das gleiche Visitor-Pattern:

- Auf den besuchten Objekten gibt es weiterhin eine `accept(...)`-Methode.
- Der Visitor hat pro konkretem Typ eine eigene `visit`-Methode (nur nicht mehr überladen).
- Über `e.accept(v)` wird zuerst die passende `accept`-Implementierung und darin dann die passende Visitormethode aufgerufen (Double-Dispatch-Prinzip bleibt gleich).

Vor- und Nachteile der beiden Varianten

Variante A: Überladene `visit(...)`-Methoden

```
1 void visit(NumExpr e);
2 void visit(MulExpr e);
3 void visit(AddExpr e);
```

- Vorteile:
 - kompakte, einheitliche Benennung (`visit` überall)
 - der Typ des Arguments sagt, was besucht wird
- Nachteile:
 - lange Liste von Überladungen kann unübersichtlich werden
 - im Code muss man genauer auf den Parametertyp achten, um zu sehen, welche Variante gemeint ist

Variante B: Unterschiedlich benannte Methoden

```
1 void visitNumExpr(NumExpr e);
2 void visitMulExpr(MulExpr e);
3 void visitAddExpr(AddExpr e);
```

- Vorteile:
 - schon am Methodennamen erkennbar, welcher Typ behandelt wird (`visitMulExpr` vs. `visitAddExpr`)
 - gut geeignet, wenn man viele verschiedene Knotentypen hat (z.B. in großen Grammatiken)
- Nachteile:

- etwas mehr Schreibarbeit
- die Verbindung zum abstrakten Pattern (“eine `visit`-Operation, spezialisiert für jede Unterklasse”) ist weniger direkt sichtbar

Tip

ANTLR generiert typischerweise Visitor-Interfaces mit **eigenen Namen pro Grammatikregel**, z.B.:

```
1 public interface ExprVisitor<T> extends ParseTreeVisitor<T> {
2     T visitAdd(ExprParser.AddContext ctx);
3     T visitMul(ExprParser.MulContext ctx);
4     T visitNum(ExprParser.NumContext ctx);
5 }
```

Das entspricht genau der zweiten Variante oben. Wichtig ist für Sie:

- Beides sind gültige Implementierungen des **gleichen** Visitor-Patterns.
- Das **Double-Dispatch-Prinzip** und die Trennung von Datenstruktur (`Expr`) und Operationen (`Visitor`) bleiben unverändert.
- Die Wahl der Namenskonvention ist eine Design-Entscheidung bzw. durch das verwendete Tool (wie ANTLR) vorgegeben.

6.3.9.3. Double-Dispatch**Tip**

Im Visitor-Pattern spricht man oft von “**Double-Dispatch**”. Gemeint ist damit, dass bei Aufrufen wie

```
1 e.accept(v);
```

und

```
1 v.visit(this);
```

zur Laufzeit zwei verschiedene Typinformationen benutzt werden, um zur passenden Methode zu gelangen:

1. der tatsächliche (dynamische) Typ des besuchten Objekts (also von `e`), und
2. der tatsächliche (dynamische) Typ des Visitors (also von `v`).

Schauen wir uns das Schritt für Schritt an.

1. Erster Dispatch: Auswahl der richtigen accept-Methode

Im Code steht z.B.:

```
1 Expr e = new AddExpr(...);
2 ExprVisitor v = new EvalVisitor();
3
4 e.accept(v);
```

- Der **statische Typ** von `e` ist `Expr`.

- Der **dynamische Typ** von `e` ist hier `AddExpr`.

Zur **Compile-Zeit** weiß der Compiler nur: “`e` ist irgendetwas vom Typ `Expr`”. Zur **Laufzeit** sieht die JVM dann, dass `e` tatsächlich ein `AddExpr` ist und ruft deshalb die `accept`-Methode von `AddExpr` auf:

```
1 public class AddExpr implements Expr {
2     public void accept(ExprVisitor v) {
3         v.visit(this);
4     }
5 }
```

Das ist der **erste Dispatch**: Die konkrete `accept`-Implementierung wird anhand des **Laufzeittyps des Elements** (hier `AddExpr`) bestimmt.

2. Zweiter Dispatch: Auswahl der passenden `visit`-Methode

Innerhalb von `AddExpr` steht:

```
1 public void accept(ExprVisitor v) {
2     v.visit(this);
3 }
```

- Der **statische Typ** von `v` ist `ExprVisitor` (Interface).
- Der **dynamische Typ** von `v` ist hier `EvalVisitor` (konkrete Implementierung).
- Der **statische Typ** von `this` in dieser Methode ist `AddExpr`.

Damit passiert Folgendes:

1. Über den **dynamischen Typ des Visitors** (`EvalVisitor`) wird entschieden, welche Implementierung von `visit(AddExpr)` in der Visitor-Hierarchie aufgerufen wird (falls `EvalVisitor` diese Methode überschreibt, dann diese, sonst evtl. eine Implementierung in einer Oberklasse).
2. Welche **Überladung** von `visit(...)` gemeint ist (`visit(NumExpr)`, `visit(AddExpr)`, `visit(MulExpr)`, ...) hängt am **statischen Typ** des Arguments `this`. In der Methode `AddExpr.accept` ist das statisch der Typ `AddExpr`, deshalb wird dort immer die Überladung `visit(AddExpr)` ausgewählt.

Important

In Java wird die Auswahl der passenden *Überladung* (`visit(AddExpr)` vs. `visit(MulExpr) ...`) **zur Compile-Zeit** anhand des statischen Typs des Arguments getroffen - hier also anhand des Typs `AddExpr` in der jeweiligen `accept`-Methode. Zur Laufzeit wird dann über den Typ des Visitors entschieden, welche konkrete Implementierung dieser Methode ausgeführt wird.

3. Warum braucht das Pattern diesen Mechanismus?

In den `accept()`-Methoden der besuchten Klassen ist nur der gemeinsame Obertyp der Visitors bekannt (`ExprVisitor`). Das ist wichtig, weil Sie so später beliebig viele verschiedene konkrete Visitor-Klassen (z.B. `EvalVisitor`, `PrintVisitor`, `TypeCheckVisitor`, ...) ergänzen/nutzen können, ohne die Klassen der Datenstruktur noch einmal anpassen zu müssen.

Das Visitor-Pattern nutzt also zwei Stufen:

1. **Dynamischer Dispatch** auf das besuchte Objekt -> "e ist zur Laufzeit ein `AddExpr`, also nutze `AddExpr.accept()`."
2. **Auswahl der passenden `visit`-Methode** anhand
 - des **dynamischen Typs** des Visitors (`EvalVisitor`, `PrintVisitor`, ...), und (zusätzlich wegen der überladenen `visit()`-Methoden)
 - des **statischen Typs** des Elements in `accept(visit(AddExpr e))` (streng genommen kein Bestandteil des Double Dispatch).

Diese Kombination der Auflösung zur Laufzeit bezeichnet man im Kontext des Visitor-Patterns als **Double-Dispatch**: Die endgültige Methode entsteht aus dem Zusammenwirken beider Typen - des Typs des Elements und des Typs des Visitors.

Java: Overloading vs. Overriding

In der Basisklasse für die Visitoren im obigen Beispiel haben wir drei **überladene** `visit(...)`-Methoden:

```
1 public interface ExprVisitor {
2     void visit(NumExpr e);
3     void visit(MulExpr e);
4     void visit(AddExpr e);
5 }
```

Bei `v.visit(this)` werden deshalb *zwei verschiedene Mechanismen* aktiv - einer zur *Compilezeit*, einer zur *Laufzeit*.

1. **Überladen (Overloading)**: Auswahl der Signatur zur **Compilezeit**

Welche `visit(...)`-**Überladung** gemeint ist (also `visit(NumExpr)`, `visit(MulExpr)`, `visit(AddExpr)`), entscheidet Java zur **Compilezeit anhand der statischen Typen der Argumente**.

In der Methode der Klasse `AddExpr`:

```
1 @Override
2 public void accept(ExprVisitor v) {
3     e2.accept(v);
4     e1.accept(v);
5     v.visit(this);
6 }
```

ist `this` statisch vom Typ `AddExpr`, weil wir uns im Code der Klasse `AddExpr` befinden. Daher wird bereits zur **Compilezeit** aufgelöst: gemeint ist `visit(AddExpr e)`.

Wichtig: Hier wird *nicht* zur Laufzeit "nochmal geguckt", welcher konkrete Typ `this` ist. Das ist beim Overloading nicht dynamisch.

2. **Überschreiben (Overriding)**: Auswahl der Implementierung zur **Laufzeit**

Welche Methode (aus welcher konkreten Implementierung) dann tatsächlich ausgeführt wird, entscheidet sich zur Laufzeit über **dynamisches Binden** anhand des dynamischen Typs von `v`.

Beispiel: Wenn `v` zur Laufzeit ein `PrettyPrintVisitor` ist, dann wird zur Laufzeit dessen Implementierung von `PrettyPrintVisitor.visit(AddExpr)` ausgeführt.

Zusammen mit der Auflösung von `e.accept(v)` (`AddExpr.accept`, `MulExpr.accept`, ...) und `v.visit(this)` (`PrettyPrintVisitor.visit`, `EvalVisitor.visit`, ...) zur Laufzeit anhand der dynamischen Typen spricht man vom “Double Dispatch”.

Important

Merksatz (in Java sehr wichtig)

- Overloading (überladen): Auswahl der passenden Signatur zur Compilezeit
- Overriding (überschreiben): Auswahl der konkreten Implementierung zur Laufzeit

Tip

Wenn wir haben:

```
1 Expr x = new AddExpr(...);
2 v.visit(x);
```

dann würde **nicht** `visit(AddExpr)` gewählt! `x` ist statisch nur `Expr` ... Das würde sogar gar nicht kompilieren, da `ExprVisitor` kein `visit(Expr)` hat. Genau deshalb macht man den “Umweg” `x.accept(v)`.

6.3.9.4. Hinweis I

Man könnte nun versucht sein, eine dieser zwei Stufen zu überspringen - man könnte ja die `visit`-Methode des `EvalVisitors` direkt aufrufen und dabei die Wurzel des Baums (das Objekt `e`) übergeben.

```
1 // Beispiel von oben (Ausschnitt)
2 Expr e = new AddExpr(new MulExpr(new NumExpr(5), new NumExpr(4)), new NumExpr(3));
3 EvalVisitor v = new EvalVisitor();
4 e.accept(v);
5
6 // Direkter Aufruf - Autsch?!
7 v.visit(e);
```

Fragen Sie sich selbst: Kann das funktionieren? Was ist die Begründung?

6.3.9.5. Hinweis II

Man könnte versucht sein, die `accept()`-Methode aus den Knotenklassen in die gemeinsame Basisklasse zu verlagern: Statt

```
1 public void accept(ExprVisitor v) {
2     v.visit(this);
3 }
```

in jeder Knotenklasse einzeln zu definieren, könnte man das doch *einmalig* in der Basisklasse definieren:

```
1 public abstract class Expr {
```

```

2    /** Akzeptiere einen Visitor für die Verarbeitung */
3    public void accept(ExprVisitor v) {
4        v.visit(this);
5    }
6 }

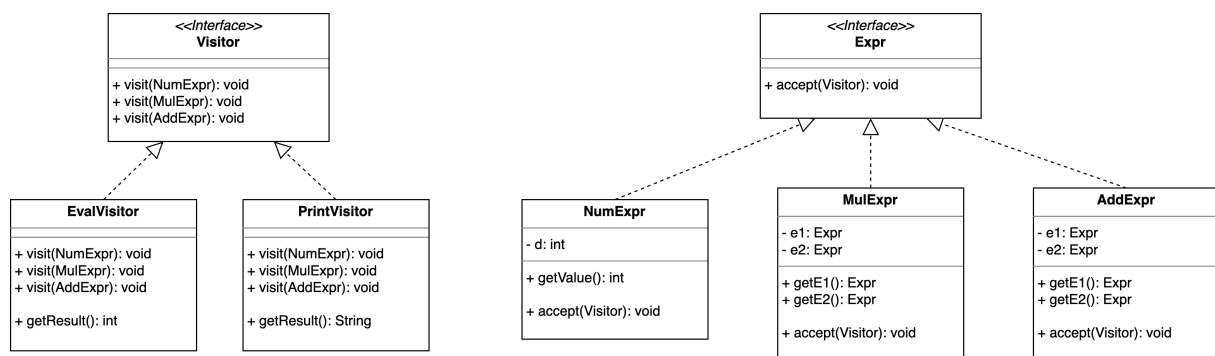
```

Dies wäre tatsächlich schön, weil man so Code-Duplizierung vermeiden könnte. Aber es funktioniert in Java leider nicht. (Warum?)

6.3.9.6. Hinweis III

Während die `accept()`-Methode nicht in die Basisklasse der besuchten Typen (im UML-Diagramm zum Visitor-Pattern oben das Interface `Elem` bzw. im Beispiel oben das Interface `Expr`) verlagert werden kann, kann man die `visit()`-Methoden im Interface `Visitor` durchaus als Default-Methoden im Interface implementieren.

6.3.10. Ausrechnen des Ausdrucks mit einem Visitor



Demo: [visitor.visit.extrav.DemoExpr](#)

6.3.11. Diskussion

In der typischen OO-Denkweise geht man davon aus, dass man eher neue Klassen über Vererbung hinzufügt als dass man in einer bestehenden Vererbungshierarchie in jeder der beteiligten Klassen neue Methoden einbaut. Man leitet einfach von der gewünschten Klasse ab und definiert mittels Überschreiben von Methoden o.ä. das geänderte Verhalten und erbt den Rest - es wird also nur eine neue Klasse hinzugefügt samt den überschriebenen Teilen.

Wenn man allerdings in einer solchen Hierarchie in allen Klassen eine neue Methode einbauen muss, die dann auch noch in den einzelnen Klassen individuell implementiert werden muss, dann kommt das Visitor-Pattern zur Hilfe und erspart Arbeit. Es muss nämlich in der Klassenhierarchie nur einmal die Schnittstelle für den Visitor einbaut werden (pro Klasse eine `accept`-Methode). Danach kann man von außen sehr einfach neue Methoden (also neue Visitoren) erstellen und nutzen, ohne die Klassenhierarchie noch einmal ändern zu müssen.

Siehe auch [When should I use the Visitor Design Pattern?](#).

Ein anderer Blick ist auf die Rolle der jeweiligen Klassen: Es gibt Objekte für/in Datenstrukturen, und es gibt Algorithmen, die auf diesen Objekten bzw. Datenstrukturen arbeiten. Im Sinne des sauberen OO-Designs würde man diese Strukturen trennen: “Trenne Algorithmen von den Objekten, auf denen die Algorithmen arbeiten.”

Vergleiche auch die Darstellung des Visitor-Patterns in [Visitor \(Refactoring Guru\)](#).

6.3.12. Ausblick: Pattern Matching auf Bäumen (neuere Java-Versionen)

Ausblick auf eine spätere Lesson Sealed Classes & Pattern Matching:

```
1 Object node = ...;
2 switch (node) {
3     case NumberNode n -> ...
4     case BinaryOpNode b -> ...
5     // ...
6 }
```

Statt mit dem Visitor-Pattern durch den Baum zu iterieren, kann man das in neueren Java-Versionen auch mit Pattern Matching auf Typen machen. Dies schauen wir uns in der Sitzung Sealed Classes & Pattern Matching genauer an, hier nur der Ausblick.

6.3.13. Wrap-Up

Visitor-Pattern: Auslagern der Traversierung in eigene Klassenstruktur

Tip

Wir haben das Visitor-Pattern am Beispiel von Bäumen kennengelernt. Das Pattern beschränkt sich aber nicht auf Bäume, sondern kann allgemein für Traversierung von Datenstrukturen eingesetzt werden.

- Klassen der traversierten Datenstruktur
 - bekommen eine `accept()`-Methode für einen Visitor
 - rufen darin den Visitor mit sich selbst als Argument auf
- Visitor
 - hat für jede Klasse der Datenstruktur eine `visit()`-Methode
 - Rückgabewerte schwierig: Intern halten oder per `return` (dann aber unterschiedliche `visit()`-Methoden für die verschiedenen Rückgabetypen!)
- (Double-) Dispatch: Kombination aus
 - dynamischem Dispatch auf das **Element** (`accept`) und
 - dynamischem Dispatch auf den **Visitor** (`visit` für den konkreten Elementtyp)

Zum Nachlesen

Der [Refactoring.Guru](#) hat eine schöne Zusammenfassung des Visitor-Patterns. Der Verweis auf Gamma u. a. (2011) der *“Gang of Four”* darf natürlich nicht fehlen.

Lernziele

- k2: Ich verstehe den Aufbau des Visitor-Patterns und kann den Double-Dispatch erklären
- k3: Ich kann das Visitor-Pattern auf konkrete Beispiele anwenden

Challenges**Visitor-Pattern praktisch (und einfach)**

Betrachten Sie den folgenden Code und erklären Sie das Ergebnis:

```

1 interface Fruit { }
2 class Apple implements Fruit { }
3 class Orange implements Fruit { }
4 class Banana implements Fruit { }
5 class Foo extends Apple { }
6
7 public class FruitBasketDirect {
8     public static void main(String... args) {
9         List<Fruit> basket = List.of(new Apple(), new Apple(), new Banana()
10                                     , new Foo());
11
12         int oranges = 0; int apples = 0; int bananas = 0; int foo = 0;
13
14         for (Fruit f : basket) {
15             if (f instanceof Apple) apples++;
16             if (f instanceof Orange) oranges++;
17             if (f instanceof Banana) bananas++;
18             if (f instanceof Foo) foo++;
19         }
20     }

```

Das Verwenden von **instanceof** ist unschön und fehleranfällig. Schreiben Sie den Code unter Einsatz des Visitor-Patterns um.

Diskutieren Sie Vor- und Nachteile des Visitor-Patterns.

6.4. Dependency Injection

TL;DR

Dependency Injection (DI) ist ein Prinzip, mit dem Klassen ihre Abhängigkeiten nicht selbst mit **new** erzeugen (oder hart kodiert verwenden), sondern sie von außen übergeben bekommen.

Statt direkt eine konkrete Klasse wie **Lsf** zu verwenden, programmieren wir gegen eine Abstraktion, zum Beispiel ein Interface wie **GradeService**. Die dazugehörigen Objekte werden dann nicht mehr in der Klasse selbst erzeugt, sondern von außen übergeben, etwa per Konstruktor-Injektion. Auf diese Weise ist

klar sichtbar, welche Abhängigkeiten eine Klasse benötigt, und die konkreten Implementierungen werden an einer zentralen Stelle (zum Beispiel in `main` oder durch ein Framework) zusammengesteckt. Dadurch sinkt die Kopplung, weil die Fachlogik nicht mehr an ein bestimmtes System gebunden ist, sondern nur noch an dessen Schnittstelle.

Für Tests ist das besonders wichtig: Nur wenn Abhängigkeiten injiziert werden, können wir sie durch Stubs oder Mocks ersetzen und echte Unit Tests ohne I/O-Abhängigkeiten schreiben. So werden Tests schneller, deterministischer und besser in CI/CD-Pipelines integrierbar. Gleichzeitig verbessert DI die Wartbarkeit und Erweiterbarkeit des Codes, weil Implementierungen leicht ausgetauscht oder ergänzt werden können. Frameworks wie Spring automatisieren DI im Hintergrund, setzen aber genau dieses einfache Grundprinzip um, das wir hier manuell angewendet haben.

Die Einordnung ist schwierig: Manche betrachten DI als "Pattern", andere sehen es als Technik für testbaren Code ...

Videos

Vorlesung [YT], [HSBI]

6.4.1. Motivation aus Tests: Ich kann nicht mocken

Wir wollen das Verhalten von `Student#printGradeFor` testen:

- ohne das echte `Lsf` anzusprechen
- stattdessen mit einem Fake/Stub für die Noten (eigenes `Lsf`-Testdouble injizieren)

Problem: Der Produktivcode sieht bisher so aus:

```
1 public class Lsf {
2     public double getGrade(String matrikelnummer, String modulName) {
3         return 1.7; // Dummy-Wert (und Dummy-Typ!)
4     }
5 }
```

```
1 public class Student {
2     private final String matrikelnummer;
3     private final String name;
4
5     public Student(String matrikelnummer, String name) {
6         this.matrikelnummer = matrikelnummer; this.name = name;
7     }
8     public String printGradeFor(String modulName) {
9         Lsf lsf = new Lsf(); // Harte Kopplung!
10        double grade = lsf.getGrade(matrikelnummer, modulName);
11        return String.format("%s hat im Modul %s die Note %s erreicht", name,
12                               modulName, grade);
12    }
13 }
```

Wichtige Beobachtung:

- `Student` ruft selbst `new Lsf()` auf

- Als Tester habe ich keine Chance, ein alternatives Objekt einzuschleusen
- Alle Test-Doubles, die wir in den vorherigen Sitzungen kennengelernt haben, bringen nichts, solange der Code sich seine Abhängigkeiten selbst baut.

“Wir können so viel über Mocks wissen, wie wir wollen: Wenn der Code überall **new** ruft, kommen wir mit sauberen Unit Tests nicht weit.”

6.4.2. Was ist eine “Dependency”?

Begriffsklärung:

Eine **Dependency** (Abhängigkeit) ist ein Objekt, das eine Klasse **benötigt**, um ihre Aufgabe zu erfüllen.

- Beispiele:
 - **Student** braucht etwas, das Noten liefert (das **Lsf**)
 - Ein **Dozent** braucht vielleicht einen **GradeSubmissionService**
 - Ein **Controller** braucht ein **Repository**
- Typischerweise:
 - Eine Abhängigkeit ist ein Feld in der Klasse
 - häufig mit **private final** => “Ohne dieses Objekt kann ich meine Arbeit nicht machen.”

Es geht nicht nur um externe Systeme. Jede Klasse, von der Ihre Klasse konkret abhängt, ist eine Dependency.

Je weniger Ihre Klasse über die konkrete Implementierung weiß, desto besser:

- reduziert Kopplung
- erhöht Wiederverwendbarkeit
- macht Tests einfacher

Unser aktuelles Problem mit **Student** und **Lsf** ist ein Beispiel für eine ungünstig gehandhabte Dependency.

6.4.3. Warum ist Kopplung schlecht? (speziell für Tests)

- Schlechte Testbarkeit
 - Unit Tests müssen mit dem “echten” **Lsf** arbeiten
 - Externe Systeme (Datenbank, Netzwerk, ...) machen Tests langsam/fragil
 - Test Doubles lassen sich nicht (oder nur sehr schwer) aufbauen
- Verletzung von Verantwortlichkeiten
 - **Student** kümmert sich um:
 - * die Fachlogik (Noten abrufen), und um
 - * die Erzeugung der Abhängigkeit

- Abhängigkeit von Änderungen
 - Änderung am `Lsf` kann Änderung an allen Klassen erzwingen, die `new Lsf()` aufrufen

In größeren Projekten führt dieses Muster oft schnell zu:

- schwer wartbarem Code (“Spaghetti-Code”),
- Klassen, die “alles wissen und alles können”,
- Problemen beim Refactoring.

Aus Sicht von Unit Tests ist der zentrale Punkt:

Harte Kopplung verhindert echte Isolation der zu testenden Klasse. Wir testen dann immer halb das `Lsf` mit, obwohl wir nur `Student` testen wollen.

6.4.4. Idee von Dependency Injection

Grundprinzip:

- Statt: Klasse erstellt ihre Abhängigkeiten selbst mit `new`
- Besser: Klasse bekommt ihre Abhängigkeiten von außen “injiziert”

Typische Formen:

- Konstruktor-Injektion
- Setter-Injektion
- Interface-basierte Injektion (z.B. über Frameworks)

Tip

Don't call `new` all over the place - ask for what you need.

In unserem Beispiel:

- `Student` soll nicht mehr selbst `new Lsf()` aufrufen.
- Stattdessen soll er ein Objekt bekommen, das Noten liefern kann.

“Injection” bedeutet hier nur: Jemand anderes (z.B. `main`, ein Framework, ein DI-Container) steckt die passende Abhängigkeit in die Klasse hinein, z.B. über den Konstruktor.

Wir betrachten nachfolgend die einfachste Variante: Konstruktor-Injektion.

6.4.5. Schritt 1: Abstraktion der Abhängigkeit

Wir führen zunächst ein neues Interface für Notenabfragen ein:

```
1 public interface GradeService {  
2     double getGrade(String matrikelnummer, String modulName);  
3 }
```

Tip

Ein **double** zur Repräsentation einer Note zu nehmen ist ... nicht sooo toll. Hier sollte man sich besser was anderes überlegen, beispielsweise ein Enum oder etwas anderes. Ein **double** kann zwar technisch auch die typischen Noten-Werte wie "1.3" halten, aber eben auch noch viele andere Werte, die wir hier nicht haben wollen. Um also später unnötige Checks und Operationen zu vermeiden, sollte man den Datentyp **Grade** lieber vernünftig modellieren (Enum o.ä.).

Das **double** habe ich hier im Beispielszenario nur genommen, um nicht unnötig von den wesentlichen Details abzulenken!

Das alte **Lsf** verpacken wir jetzt in einen **GradeService**:

```
1 public class LsfGradeService implements GradeService {
2     private final Lsf lsf;
3
4     public LsfGradeService(Lsf lsf) {
5         this.lsf = lsf;
6     }
7
8     @Override
9     public double getGrade(String matrikelnummer, String modulName) {
10        return lsf.getGrade(matrikelnummer, modulName);
11    }
12 }
```

Wichtig:

- **GradeService** ist eine Abstraktion (Interface) für "irgendetwas, das Noten liefern kann"
- **LsfGradeService** ist eine konkrete Implementierung davon
- Später könnten wir weitere Implementierungen hinzufügen:
 - **NeuesSystemGradeService**
 - **OfflineGradeService**
 - **DummyGradeService** für Demos oder Tests

Student soll nur noch **GradeService** kennen, nicht mehr direkt das **Lsf**.

6.4.6. Schritt 2: Student mit Dependency Injection

Nun erweitern wir **Student** und eliminieren das feste **new Lsf()**. Stattdessen übergeben wir ein Objekt vom Typ **GradeService** im Konstruktor ("Konstruktor-Injektion"):

```
1 public class Student {
2     private final String matrikelnummer;
3     private final String name;
4     private final GradeService gradeService; // Dependency
5
6     // Dependency Injection
7     public Student(String matrikelnummer, String name, GradeService gradeService)
8     {
9         this.matrikelnummer = matrikelnummer;
10        this.name = name;
```

```

10     this.gradeService = gradeService;
11 }
12
13 public String printGradeFor(String modulName) {
14     double grade = gradeService.getGrade(matrikelnummer, modulName);
15     return String.format("%s hat im Modul %s die Note %s erreicht", name,
16                           modulName, grade);
17 }

```

Hier passiert **Dependency Injection**:

- **Student erzeugt** kein **Lsf** mehr
- **Student** verlangt beim Erzeugen: "Gib mir ein **GradeService**-Objekt."
- Die konkrete Implementierung wird von außen bestimmt

Vorteile:

- **Student** ist nur noch an die Abstraktion **GradeService** gekoppelt
- Wir können in verschiedenen Kontexten unterschiedliche Implementierungen injizieren:
 - echte LSF-Anbindung
 - Dummy für eine Demo
 - Fake für Tests

Tip

Die Übergabe eines konkreten **GradeService**-Objekts im Konstruktor funktioniert technisch und löst unser Kopplungsproblem ausreichend gut. Man darf sich aber schon die Frage stellen, ob ein **Student** wirklich ein festes Feld für einen **GradeService** haben muss, oder ob man das nicht einfach jeweils dynamisch beim Aufruf der Methode **Student#printGradeFor** mitgeben könnte ... Auch das wäre "Dependency Injection" (nur eben keine "Konstruktor-Injektion").

6.4.7. Schritt 3: Verdrahtung im "Composition Root"

Wo kommen die Objekte her?

```

1 public class Demo {
2     public static void main(String[] args) throws InterruptedException {
3         // Echtes LSF
4         Lsf lsf = new Lsf();
5         GradeService gradeService = new LsfGradeService(lsf);
6
7         Student alex = new Student("123456", "Alex Muster", gradeService);
8         alex.printGradeFor("Programmieren 2");
9     }
10 }

```

Alternative Implementierung (z.B. Demo/Test):

```

1 public class DummyGradeService implements GradeService {
2     @Override

```

```
3 public double getGrade(String matrikelnummer, String modulName) {
4     return 1.0; // immer sehr gut :)
5 }
6 }
```

```
1 public class Demo {
2     public static void main(String[] args) throws InterruptedException {
3         // Dummy-LSF
4         GradeService dummy = new DummyGradeService();
5         Student chris = new Student("654321", "Chris Beispiel", dummy);
6         chris.printGradeFor("Programmieren 2");
7     }
8 }
```

Der Ort, an dem Sie alle Objekte “verdrahten” (also erstellen und zusammenstecken), wird oft **Composition Root** genannt.

- In kleinen Programmen ist das die `main`-Methode
- In größeren Anwendungen übernehmen das Frameworks/DI-Container (z.B. Spring)

Wichtig:

- Die Fachklassen (`Student`, Services, ...) kümmern sich nicht mehr um `new` ihrer Abhängigkeiten
- Sie bleiben “dumm” in Bezug auf Erzeugung und Konfiguration

Für uns als Tester ist entscheidend: Im Composition Root können wir entscheiden, welche Implementierung wir verwenden, ohne den Produktivcode von `Student` ändern zu müssen.

6.4.8. DI und Testbarkeit (JUnit)

JUnit-Test mit Fake-Implementierung (hier als Lambda-Ausdruck):

```
1 class StudentTest {
2     @Test
3     void createGradeMessage_usesInjectedGradeService() {
4         // given
5         GradeService fake = (_, modulName) -> modulName.equals("Programmieren 2") ?
6             1.3 : 4.0;
7         Student student = new Student("123456", "Alex Muster", fake);
8
9         // when
10        String message = student.printGradeFor("Programmieren 2");
11
12        // then
13        assertEquals("Alex Muster hat im Modul Programmieren 2 die Note 1.3
14            erreicht", message);
15    }
16 }
```

Durch DI:

- können wir im Test gezielt kontrollierte Implementierungen verwenden (z.B. als Lambda-Ausdruck `fake`),
- sind wir nicht vom echten `Lsf` abhängig,

- werden Tests schneller, stabiler und einfacher in CI/CD-Pipelines integrierbar.

Demo Dependency Injection

6.4.9. Ausblick: Verbindung zu Frameworks

Was machen Spring & Co.?

- In größeren Java-Projekten nutzen viele von Ihnen später:
 - Spring / Spring Boot
 - Jakarta CDI, Guice, ...
- Dort gibt es:
 - DI-Container, die Objekte verwalten,
 - Annotations wie `@Service`, `@Repository`, `@Controller`,
 - Konstruktor-Injektion oder Annotations wie `@Autowired`.

Grundprinzip bleibt aber genau das gleiche wie im Beispiel:

- Klassen deklarieren, welche Abhängigkeiten sie brauchen
- Ein anderer Teil des Systems (Container/Framework) liefert die konkreten Implementierungen

Wichtig fürs Verständnis:

- Was Sie hier “per Hand” gebaut haben (Interfaces, Konstruktor-Injektion, Composition Root), ist das Kernprinzip von Dependency Injection.
- Frameworks nehmen Ihnen nur die “Fleißarbeit” ab:
 - Instanzen erzeugen
 - Abhängigkeiten auflösen
 - Lebenszyklen verwalten

Wenn Sie das einfache Beispiel verstanden haben, fällt Ihnen der Einstieg in Spring & Co. wesentlich leichter.

6.4.10. Wrap-Up

Kerngedanken von Dependency Injection:

- Eine Klasse soll **nicht selbst** ihre Abhängigkeiten mit **new** erzeugen
- Stattdessen soll sie **deklarieren**, was sie braucht (z.B. im Konstruktor)
- Abhängigkeiten werden über **Abstraktionen** (Interfaces) modelliert
- Konkrete Implementierungen werden von außen “injiziert”:
 - in kleinen Programmen durch Sie selbst (z.B. in `main`)
 - in größeren durch DI-Container/Frameworks

Vorteile (insbesondere aus Testsicht):

- Geringere Kopplung
- Bessere Testbarkeit (einfache Verwendung von Test Doubles)
- Einfacherer Austausch von Implementierungen
- Bessere Wartbarkeit, gerade in größeren Projekten und CI/CD-Umgebungen

Tip

Merksätze zum Mitnehmen:

1. “Don’t call new all over the place - ask for what you need.”
2. “Programmiere gegen Interfaces, nicht gegen konkrete Klassen.”
3. “Dependency Injection ist kein Magie-Werkzeug der Frameworks, sondern ein simples Prinzip, das Sie auch ohne Framework anwenden können.”
4. “Guter Testcode braucht guten Produktivcode: DI ist ein Schlüssel dazu.”

Diese Prinzipien werden Ihnen in vielen weiteren Themen begegnen: Design Patterns, Architektur, Testing, Microservices etc.

Zum Nachlesen

Der Blog [Dependency Injection Pattern in Java: Boosting Maintainability with Loose Coupling](#) ist ganz nett und hat noch eine schöne Visualisierung.

Auch interessant:

- [Java Dependency Injection: Design Pattern Tutorial \(DigitalOcean\)](#)
- [Dependency Injection Made Simple with Java Examples \(Geekific\)](#)

Lernziele

- k2: Ich kann den Begriff “Dependency Injection” in eigenen Worten erklären.
- k2: Ich kann anhand eines einfachen Beispiels erläutern, was eine Dependency ist.
- k2: Ich kann erklären, warum harte Kopplung (z.B. `new Lsf()` im Code) Unit Tests erschwert, und warum DI Unit Tests erleichtert.
- k2: Ich kann den Zusammenhang zwischen Dependency Injection und Testbarkeit (Mocks/Fakes) beschreiben.
- k3: Ich kann eine bestehende Java-Klasse so umgestalten, dass ihre Abhängigkeiten per Konstruktor-Injektion übergeben werden.
- k3: Ich kann für eine konkrete Abhängigkeit ein Interface entwerfen und eine passende Implementierung schreiben.
- k3: Ich kann eine Klasse so umbauen, dass sie statt einer konkreten Implementierung nur noch ein Interface verwendet.
- k3: Ich kann mithilfe von Dependency Injection einen Unit Test schreiben, der ein Testdouble (z.B. `FakeGradeService`) verwendet.

Challenges

CurrencyConverter + ExchangeRateProvider

- Der `CurrencyConverter` soll Beträge von einer Währung in eine andere umrechnen
- Er braucht dafür einen `ExchangeRateProvider`, der die Wechselkurse liefert
- In "echt" würde dieser Provider vielleicht eine API im Internet aufrufen
- Für Tests wollen wir aber keinen echten HTTP-Aufruf und keine Zufallswerte

Ausgangsversion ohne DI: `CurrencyConverter` erzeugt sich den Provider selbst:

```
1 public class OnlineExchangeRateProvider {
2     public double getRate(String from, String to) {
3         // Hier könnte z.B. ein HTTP-Call stehen
4         IO.println("Frage Online-Dienst nach Wechselkurs " + from + " -> " + to
5         );
6         return 1.1; // Dummy-Wert
7     }
8 }
```

```
1 public class CurrencyConverter {
2     public static double convert(String from, String to, double amount) {
3         OnlineExchangeRateProvider provider = new OnlineExchangeRateProvider();
4         double rate = provider.getRate(from, to);
5         return amount * rate;
6     }
7 }
```

Fragen:

- Wie testen wir `convert`, ohne den "Online-Dienst" zu benutzen?
- Wie testen wir unterschiedliche Kurse, Fehlerszenarien etc.?

Aufgabe:

- Bauen Sie den Code mit Hilfe von Dependency Injection um, so dass er testbar wird.

6.5. Command-Pattern

TL;DR

Das **Command-Pattern** ist die objektorientierte Antwort auf Callback-Funktionen: Man kapselt Befehle in einem Objekt.

Ziel: Eingaben/Aktionen entkoppeln und konfigurierbar machen (und optional "Undo").

1. Die `Command`-Objekte haben eine Methode `execute()` und führen dabei eine Aktion auf einem bzw. "ihrem" Receiver aus.
2. `Receiver` sind Objekte, auf denen Aktionen ausgeführt werden, im Dungeon könnten dies etwa Hero, Monster, ... sein. Receiver müssen keine der anderen Akteure in diesem Pattern kennen.

3. Damit die **Command**-Objekte aufgerufen werden, gibt es einen **Invoker**, der **Command**-Objekte hat und zu gegebener Zeit auf diesen die Methode `execute()` aufruft. Der Invoker muss dabei die konkreten Kommandos und die Receiver nicht kennen (nur die **Command**-Schnittstelle).
4. Zusätzlich gibt es einen **Client**, der die anderen Akteure kennt und alles zusammenbaut.

Videos

Vorlesung [YT], [HSBI]

6.5.1. Motivation

Irgendwo im Dungeon wird es ein Objekt einer Klasse ähnlich wie **InputHandler** geben mit einer Methode ähnlich zu `handleInput()`:

```

1 public class InputHandler {
2     public void handleInput() {
3         switch (keyPressed()) {
4             case BUTTON_W -> hero.jump();
5             case BUTTON_A -> hero.moveX();
6             case ...
7             default -> { ... }
8         }
9     }
10 }
```

Diese Methode wird je Frame einmal aufgerufen, um auf eventuelle Benutzereingaben reagieren zu können. Je nach gedrücktem Button wird auf dem Hero eine bestimmte Aktion ausgeführt...

Das funktioniert, ist aber recht unflexibel. Die Aktionen sind den Buttons fest zugeordnet und erlauben keinerlei Konfiguration.

6.5.2. Auflösen der starren Zuordnung über Zwischenobjekte

```

1 public interface Command { void execute(); }
2
3 public class Jump implements Command {
4     private Entity e;
5     public void execute() { e.jump(); }
6 }
7
8 public class InputHandler {
9     private final Command wbutton = new Jump(hero); // Über Ctor/Methoden
        setzen!
10    private final Command abutton = new Move(hero); // Über Ctor/Methoden
        setzen!
11
12    public void handleInput() {
13        switch (keyPressed()) {
14            case BUTTON_W -> wbutton.execute();
```

```

15         case BUTTON_A -> abutton.execute();
16         case ...
17         default -> { ... }
18     }
19 }
20 }

```

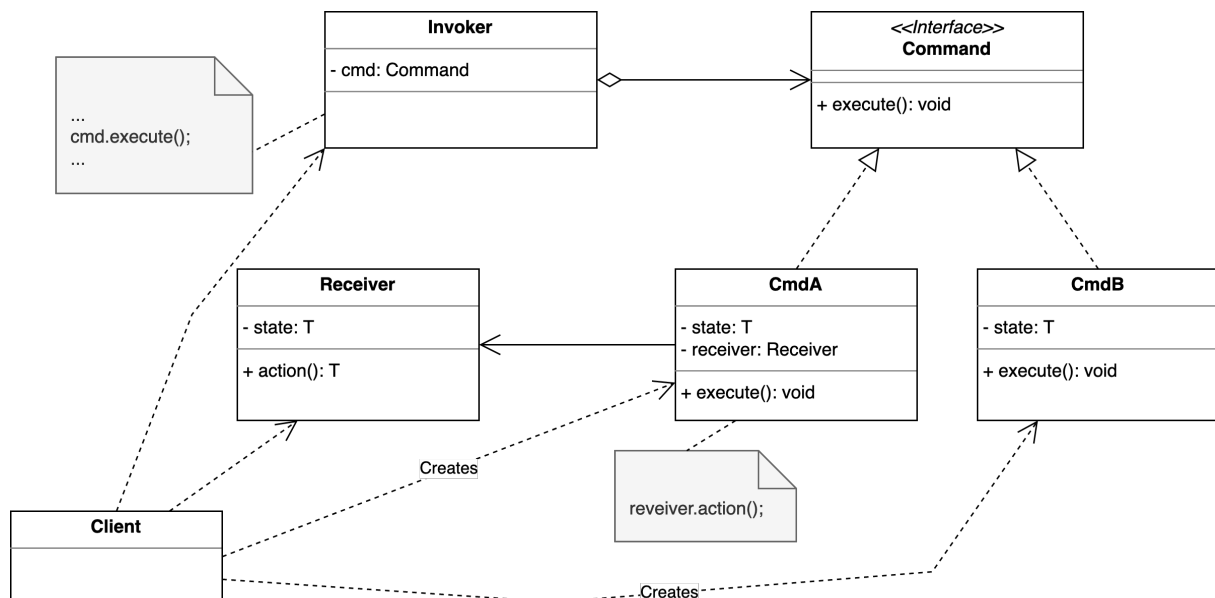
Die starre Zuordnung “Button : Aktion” wird durch das Command-Pattern aufgelöst und über Zwischenobjekte konfigurierbar gemacht.

Für die Zwischenobjekte wird ein Typ `Command` eingeführt, der nur eine `execute()`-Methode hat. Für jede gewünschte Aktion wird eine Klasse davon abgeleitet, diese Klassen können auch einen Zustand pflegen.

Den Buttons wird nun an geeigneter Stelle (Konstruktor, Methoden, ...) je ein Objekt der jeweiligen Command-Unterklassen zugeordnet. Wenn ein Button betätigt wird, wird auf dem Objekt die Methode `execute()` aufgerufen.

Damit die Kommandos nicht nur auf den Helden wirken können, kann man den Kommando-Objekten beispielsweise noch eine Entität mitgeben, auf der das Kommando ausgeführt werden soll. Im Beispiel oben wurde dafür der `hero` genutzt.

6.5.3. Command: Objektorientierte Antwort auf Callback-Funktionen



Im Command-Pattern gibt es vier beteiligte Parteien: Client, Receiver, Command und Invoker.

Ein Command ist die objektorientierte Abstraktion eines Befehls. Es hat möglicherweise einen Zustand, und kennt “seinen” Receiver und kann beim Aufruf der `execute()`-Methode eine vorher verabredete Methode auf diesem Receiver-Objekt ausführen.

Ein Receiver ist eine Klasse, die Aktionen durchführen kann. Sie kennt die anderen Akteure nicht.

Der Invoker (manchmal auch “Caller” genannt) ist eine Klasse, die Commands aggregiert und die die Kommandos “ausführt”, indem hier die `execute()`-Methode aufgerufen wird. Diese Klasse kennt nur das `Command`-Interface und keine spezifischen Kommandos (also keine der Sub-Klassen). Es kann zusätzlich eine gewisse Buchführung übernehmen, etwa um eine Undo-Funktionalität zu realisieren.

Der Client ist ein Programmteil, der die Command-Objekte aufbaut und dabei einen passenden Receiver übergibt und der die Command-Objekte dann zum Aufruf an den Invoker weiterreicht.

In unserem Beispiel lassen sich die einzelnen Teile so sortieren:

- Client: Klasse `InputHandler` (erzeugt neue `Command`-Objekte im obigen Code) bzw. `main()`, wenn man die `Command`-Objekte dort erstellt und an den Konstruktor von `InputHandler` weiterreicht
- Receiver: Objekt `hero` der Klasse `Hero` (auf diesem wird eine Aktion ausgeführt)
- Command: `Jump` und `Move`
- Invoker: `InputHandler` (in der Methode `handleInput()`)

Tip

Client = die Stelle, wo verkabelt wird (z.B. `main`), **Invoker** = Ausführen der Commands.

Die Rollen dürfen in der Praxis zusammenfallen: Eine Klasse kann mehrere Rollen übernehmen; hier in unserem Beispiel ist der `InputHandler` zugleich **Client** und **Invoker**.

6.5.4. Undo

Wir könnten das `Command`-Interface um ein paar Methoden erweitern:

```
1 public interface Command {
2     void execute();
3     void undo();
4     Command newCmd(Entity e);
5 }
```

Jetzt kann jedes Command-Objekt eine neue Instanz erzeugen mit der Entity, die dann dieses Kommando empfangen soll:

```
1 public class Move implements Command {
2     private Entity e;
3     private int x, y, oldX, oldY;
4
5     public void execute() { oldX = e.getX(); oldY = e.getY(); x = oldX + 42;
6         y = oldY; e.moveTo(x, y); }
7     public void undo() { e.moveTo(oldX, oldY); }
8     public Command newCmd(Entity e) { return new Move(e); }
9 }
10 public class InputHandler {
11     private final Command wbutton;
12     private final Command abutton;
13     private final Deque<Command> s = new ArrayDeque<>();
14
15     public void handleInput() {
```

```

16     Entity e = getSelectedEntity();
17     switch (keyPressed()) {
18         case BUTTON_W -> { s.push(wbutton.newCmd(e)); s.peek().execute(); }
19         case BUTTON_A -> { s.push(abutton.newCmd(e)); s.peek().execute(); }
20         case BUTTON_U -> s.pop().undo();
21         case ...
22         default -> { ... }
23     }
24 }
25 }

```

Über den Konstruktor von `InputHandler` (im Beispiel nicht gezeigt) würde man wie vorher die `Command`-Objekte für die Buttons setzen. Es würde aber in jedem Aufruf von `handleInput()` abgefragt, was gerade die selektierte Entität ist und für diese eine neue Instanz des zur Tastatureingabe passenden `Command`-Objekts erzeugt. Dieses wird nun in einem Stack gespeichert und danach ausgeführt.

Wenn der Button “U” gedrückt wird, wird das letzte `Command`-Objekt aus dem Stack genommen (Achtung: Im echten Leben müsste man erst einmal schauen, ob hier noch was drin ist!) und auf diesem die Methode `undo()` aufgerufen. Für das Kommando `Move` ist hier skizziert, wie ein Undo aussehen könnte: Man muss einfach bei jedem `execute()` die alte Position der Entität speichern, dann kann man sie bei einem `undo()` wieder auf diese Position verschieben. Da für jeden Move ein neues Objekt angelegt wird und dieses nur einmal benutzt wird, braucht man keine weitere Buchhaltung ...

6.5.5. Abgrenzung zum Strategy-Pattern

Beide Entwurfsmuster (Strategy-Pattern und Command-Pattern) kann man schnell verwechseln. Beide nutzen “irgendwie” Interfaces mit “irgendwelchen” Methoden ...

Strategy-Pattern: Wir tauschen den Algorithmus aus, mit dem der Empfänger seine Aktion durchführt.

Command-Pattern: Wir kapseln eine Aktion als Objekt. Dieses kann dann herumgereicht, gespeichert oder in eine Queue gesteckt werden. Es kann auch über eine undo-Fähigkeit verfügen, d.h. man kann die Aktion über das Objekt rückgängig machen.

Das Command-Pattern ist besonders nützlich, wenn Sie Befehle speichern, loggen, replayen, undoen oder asynchron ausführen wollen. Im Zusammenhang mit JUnit lassen sich Commands einzeln testen, indem man beispielsweise den Receiver mockt.

6.5.6. Wrap-Up

Command-Pattern: Kapselt Befehle als Objekte, um

- Aktionen entkoppelt und konfigurierbar zu machen,
- Undo/Redo-Funktionalität zu ermöglichen,
- Befehle zu speichern, zu loggen oder asynchron auszuführen.

Aufbau:

- **Command**-Objekte haben eine Methode `execute()`, führen darin die Aktion auf dem Receiver aus
- **Receiver** sind Objekte, auf denen Aktionen ausgeführt werden (Hero, Monster, ...)
- **Invoker** hat **Command**-Objekte und ruft darauf `execute()` auf
- **Client** kennt alle Klassen und baut alles zusammen

Objektorientierte Antwort auf Callback-Funktionen

Zum Nachlesen

Auch wenn es für C++ geschrieben ist, lässt sich zum Thema Command-Pattern das Kapitel 2 “Command” im Nystrom (2014) sehr gut lesen. Der Verweis auf Gamma u. a. (2011) der “[Gang of Four](#)” darf natürlich nicht fehlen. Der [refactoring.guru](#) hat ebenfalls ein schönes Tutorial zum Command-Pattern.

Lernziele

- k2: Ich kann den Aufbau des Command-Patterns erklären
- k3: Ich kann das Command-Pattern auf konkrete Beispiele anwenden

Challenges

Command-Pattern im Restaurant

Problem: In einem Restaurant gibt es folgende Rollen:

- Gast (bestellt Essen/Getränke)
- Kellner (nimmt Bestellungen entgegen und leitet sie weiter)
- Koch (bereitet Essen zu)
- Barkeeper (mischt Getränke)

Ziel: Implementieren Sie das Command-Pattern für dieses Szenario. Der Gast soll Bestellungen beim Kellner aufgeben, die dieser an den Koch bzw. den Barkeeper weitergibt. Bestellungen sollen auch storniert werden können.

Aufgabe: Diskutieren Sie in 10 Minuten:

- Wie modellieren Sie die Rollen (Gast, Kellner, Koch, Barkeeper)?
- Wie sieht das Command-Interface aus? Soll es `execute()` oder `execute(Receiver)` heißen?
- Wie übergibt der Kellner den **Receiver** an das Command?
- Wie implementieren Sie Undo/Redo?
- Wie ist die Klassenstruktur (Wer kennt wen)?

7. Graphische Oberflächen mit Swing und Java2D

7.1. Swing 4: Events

TL;DR

In Swing-Komponenten werden Events ausgelöst, wenn der User mit den Komponenten interagiert. Zur Bearbeitung der Events kann man Listener bei den Komponenten registrieren, die bei Auftreten eines Events benachrichtigt werden (Observer-Pattern: Die Observer werden in Swing "Listener" genannt). Es gibt für alle möglichen Formen von Interaktion mit Komponenten vordefinierte Interfaces für die Event-Listener. Da man hier wie üblich immer alle Methoden implementieren muss, selbst wenn man nur auf wenige Events reagieren möchte, gibt es zusätzlich sogenannte "Adapter": Dies sind Klassen, die das jeweilige Event-Listener-Interface mit leeren Methodenrumpfen implementieren. Bei Nutzung der Adapter-Klassen müssen dann nur noch die benötigten Methoden überschrieben werden.

Videos

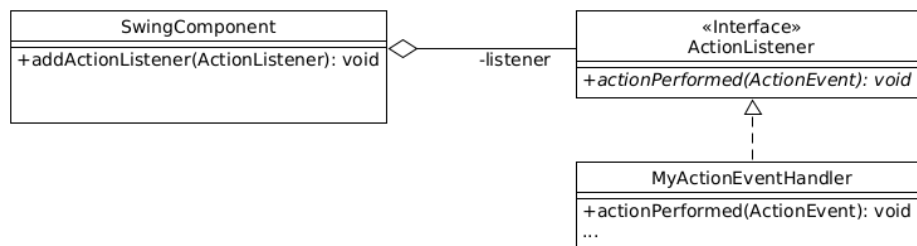
Vorlesung [[YT](#)], [[HSBI](#)]

Demos:

- Swing Events und Listener [[YT](#)], [[HSBI](#)]
- MouseListener vs. MouseAdapter [[YT](#)], [[HSBI](#)]

7.1.1. Reaktion auf Events: Anwendung Observer-Pattern

- Swing-GUI läuft in Dauerschleife
- Komponenten registrieren Ereignisse (Events):
 - Mausklick
 - Tastatureingaben
 - Mauszeiger über Komponente
 - ...
- Reaktion mit passendem Listener: Observer Pattern!



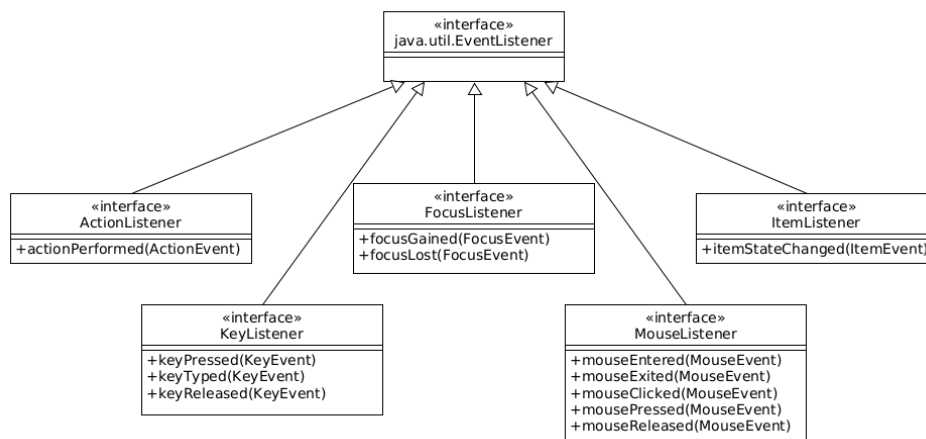
=> Observer aus dem Observer-Pattern!

In Swing werden die “Observer” als “Listener” bezeichnet.

```

1 component.addActionListener(ActionListener);
2 component.addMouseListener(MouseListener);
  
```

7.1.2. Arten von Events



Es gibt für alle möglichen Input-Arten eine Ableitung von `java.util.EventListener`, beispielsweise für Maus- oder Tastaturereignisse oder wenn ein Element den Fokus bekommt und viele weitere.

7.1.3. Details zu Listnern

- Ein Listener kann bei mehreren Observables registriert sein:

```

1 Handler single = new Handler();
2 singleButton.addActionListener(single);
3 multiButton.addActionListener(single);
  
```

- Ein Observable kann mehrere Listener bedienen:


```

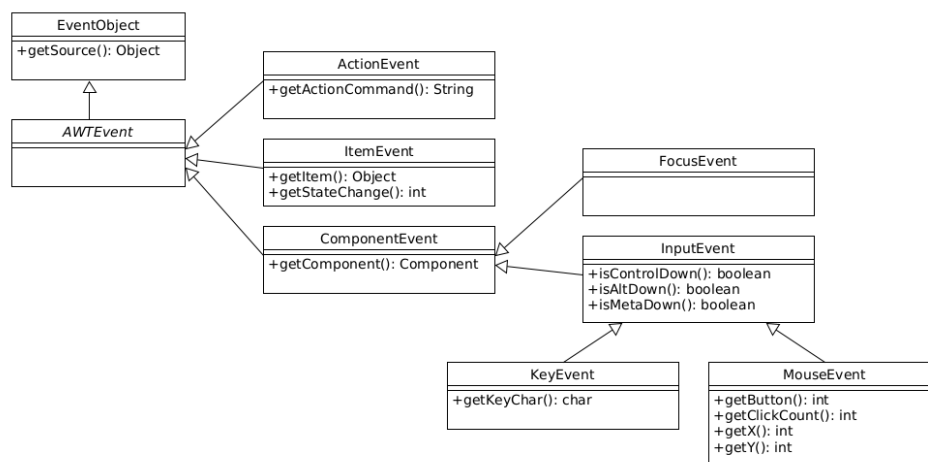
1 multiButton.addActionListener(new Handler());
2 multiButton.addActionListener(new Handler());

```

- Sequentielles Abarbeiten der Events bzw. Benachrichtigung der Observer

[Demo: events.ListenerDemo](#)

7.1.4. Wie komme ich an die Daten eines Events?



Event-Objekte: Quelle des Events plus aufgetretene Daten

[Demo: events.MouseListenerDemo](#)

7.1.5. Listener vs. Adapter

- Vielzahl möglicher Events
- Jeweils passendes Event-Objekt u. Event-Listener-Interface
- Oft nur wenige Methoden, u.U. aber viele Methoden

=> Bei Nutzung eines Event-Listeners müssen immer **alle** Methoden implementiert werden (auch nicht benötigte)!

Abhilfe: **Adapter**-Klassen:

- Für viele Event-Listener-Interfaces existieren Adapter-Klassen
- Implementieren jeweils ein Interface
- Alle Methoden mit **leerem** Body vorhanden

=> Nur benötigte Listener-Methoden überschreiben.

[Demo: events.MouseAdapterDemo](#)

7.1.6. Exkurs: Nützliches zu Swing

Über Swing wurde prinzipiell bereits im ersten Semester gesprochen, deshalb wird hier in Prog2 nur der Zusammenhang mit dem Observer-Pattern hergestellt.

Wenn Sie Ihr Wissen zu Swing auffrischen wollen, hier vier nützliche Sessions zu Swing:

- Swing Basics
- Nützliche Widgets
- Layouts und -Manager
- Java2D

7.1.7. Wrap-Up

Observer-Pattern in Swing-Komponenten:

- Events: Enthalten Source-Objekt und Informationen
- Event-Listener: Interfaces mit Methoden zur Reaktion
- Adapter: Listener mit leeren Methodenrümpfen

Zum Nachlesen

Zum Thema Swing Events können Sie im Tutorial [“Lesson: Writing Event Listeners” \(Oracle\)](#) nachlesen.

Lernziele

- k2: Unterschied zwischen den Listnern und den entsprechenden Adaptern
- k3: Anwendung des Observer-Pattern, beispielsweise als Listener in Swing, aber auch in eigenen Programmen
- k3: Nutzung von ActionListener, MouseListener, KeyListener, FocusListener

7.2. Swing 1: Basics

TL;DR

Swing baut auf AWT auf und ersetzt dieses. Der designierte Nachfolger JavaFX wurde nie wirklich als Ersatz angenommen und ist mittlerweile sogar wieder aus dem JDK bzw. der Java SE herausgenommen worden. Swing-Fenster bestehen zunächst aus Top-Level-Komponenten wie einem Frame oder einem Dialog. Darin können “atomare Komponenten” wie Buttons, Label, Textfelder, ... eingefügt werden. Zur Gruppierung können Komponenten wie Panels genutzt werden.

Fenster werden über die Methode `pack()` berechnet (Größe, Anordnung) und müssen explizit sichtbar gemacht werden (`setVisible(true)`). Per Default läuft die Anwendung weiter, nachdem das Hauptfenster geschlossen wurde (konfigurierbar).

Swing ist nicht Thread-safe, man darf also nicht mit mehreren Threads parallel die Komponenten bearbeiten. Events werden in Swing durch einen speziellen Thread verarbeitet, dem sogenannten *Event Dispatch*

Thread (EDT). Dieser wird über den Aufruf von `pack()` automatisch gestartet. Da das Hauptprogramm `main()` in einem eigenen Thread läuft und ja die ganzen Komponenten quasi schrittweise erzeugt, kann es hier zu Konflikten kommen. Deshalb sollte das Erzeugen der Swing-GUI als neuer Thread ("Runnable") dem EDT übergeben werden.

Videos

Vorlesung [YT], [HSBI]

Demo Einfaches Fenster [YT], [HSBI]

7.2.1. Wiederholung GUI in Java

- **AWT: `abstract window toolkit`**
 - Älteres Framework ("Legacy")
 - "Schwergewichtig": plattformangepasst
 - Paket `java.awt`
- **Swing**
 - Nutzt AWT
 - "Leichtgewichtig": rein in Java implementiert
 - Paket `javax.swing`
- **JavaFX**
 - Soll als Ersatz für Swing dienen
 - * Community eher verhalten
 - * Weiterentwicklung immer wieder unklar
 - * Nicht mehr im JDK/Java SE Plattform enthalten
 - Vergleichsweise komplexes Framework, auch ohne Java programmierbar (Skriptsprache FXML)

Anmerkung: In Swing reimplementierte Klassen aus AWT: Präfix "J": `java.awt.Button` (AWT) => `javax.swing.JButton` (Swing)

7.2.2. Graphische Komponenten einer GUI

- Top-Level Komponenten
 - Darstellung direkt auf Benutzeroberfläche des Betriebssystems
 - Beispiele: Fenster, Dialoge
- Atomare Komponenten
 - Enthalten i.d.R. keine weiteren Komponenten

- Beispiele: Label, Buttons, Bilder
- Gruppierende Komponenten
 - Bündeln und gruppieren andere Komponenten
 - Beispiele: JPanel

Achtung: Unterteilung nicht im API ausgedrückt: Alle Swing-Bausteine leiten von Klasse `javax.swing.JComponent` ab!

=> Nutzung "falscher" Methoden führt zu Laufzeitfehlern.

7.2.3. Ein einfaches Fenster

```
1 public class FirstWindow {
2     public static void main(String[] args) {
3         JFrame frame = new JFrame("Hello World :)");
4
5         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6
7         frame.pack();
8         frame.setVisible(true);
9     }
10 }
```

7.2.3.1. Elemente

Es wird ein neuer Frame angelegt als Top-Level-Komponente. Der Fenstertitel wird auf "Hello World :)" gesetzt.

Zusätzlich wird spezifiziert, dass sich das Programm durch Schließen des Fensters beenden soll. Anderenfalls würde man zwar das sichtbare Fenster schließen, aber das Programm würde weiter laufen.

Mit der Swing-Methode `pack()` werden alle Komponenten berechnet und die Fenstergröße bestimmt, so dass alle Komponenten Platz haben. Bis dahin ist das Fenster aber unsichtbar und wird erst über den Aufruf von `setVisible(true)` auch dargestellt.

7.2.3.2. Swing und Multithreading: Event Dispatch Thread

Leider ist die Welt nicht ganz so einfach. In Swing werden Events wie das Drücken eines Buttons durch den *Event Dispatch Thread* (EDT) abgearbeitet. (Zum Thema Events in Swing siehe Einheit "Swing Events".) Der EDT wird mit dem Erzeugen der visuellen Komponenten für die Swing-Objekte durch den Aufruf der Swing-Methoden `show()`, `setVisible()` und `pack()` erstellt. Bereits beim Realisieren der Komponenten könnten diese Events auslösen, die dann durch den EDT verarbeitet werden und an mögliche Listener verteilt werden. Dummerweise wird das `main()` von der JVM aber in einem eigenen Thread abgearbeitet - es könnten also zwei Threads parallel durch die hier erzeugte Swing-GUI laufen, und Swing ist **nicht Thread-safe**! Komponenten dürfen nicht durch verschiedene Threads manipuliert werden.

Die Lösung ist, die Realisierung der Komponenten als Job für den EDT zu "verpacken":

```
1 SwingUtilities.invokeLater(  
2     new Runnable() {  
3         public void run() {  
4             JFrame frame = new JFrame("Hello World :");  
5  
6             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
7  
8             frame.pack();  
9             frame.setVisible(true);  
10        }  
11    });
```

Mit `new Runnable()` wird ein neues Objekt vom Typ `Runnable` angelegt - im Prinzip ein neuer, noch nicht gestarteter Thread mit der Hauptmethode `run()`. Dieses `Runnable` wird mit `SwingUtilities.invokeLater()` dem EDT zu Ausführung übergeben.

Zum Thema "Nebenläufige Programmierung" auch [Einführung in die nebenläufige Programmierung](#) (Rheinwerk Verlag) und [Lesson: Concurrency \(Oracle\)](#) sowie speziell in Bezug auf Swing "Concurrency in Swing".

Beispiel: `basics.FirstWindow`

Demo: `basics.SecondWindow`

7.2.4. Wrap-Up

- Swing baut auf AWT auf und nutzt dieses
- JavaFX ist moderner, aber kein Swing-Ersatz geworden
- Basics:
 - Swing-Fenster haben Top-Level-Komponenten: `JFrame`, ...
 - Atomare Komponenten wie Buttons, Label, ... können gruppiert werden
 - Fenster müssen explizit sichtbar gemacht werden
 - Nach Schließen des Fensters läuft die Applikation weiter (Default)
 - Swing-Events werden durch den *Event Dispatch Thread* (EDT) verarbeitet
=> Aufpassen mit Multithreading!

Zum Nachlesen

Zum Thema Swing Basics können Sie im Tutorial "[Using Top-Level Containers](#)" (Oracle) nachlesen.

Lernziele

- k2: Unterschied und Zusammenhang zwischen Swing und AWT

7.3. Swing 2: Nützliche Widgets

TL;DR

Neben den Standardkomponenten `JFrame` für ein Fenster, `JPanel` für ein Panel (auch zum Gruppieren anderer Komponenten), `JButton` (Button) und `JTextArea` (Texteingabe) gibt es eine Reihe weiterer nützlicher Swing-Komponenten:

- `JRadioButton` für Radio-Buttons und `JCheckBox` für Checkbox-Buttons sowie `ButtonGroup` für die logische Verbindung von diesen Buttons (es kann nur ein Button einer `ButtonGroup` aktiv sein - wenn ein anderer Button aktiviert wird, wird der zuletzt aktive Button automatisch deaktiviert)
- Dateiauswahldialoge mit `JFileChooser` und `FileFilter` zum Vorfiltern der Anzeige
- Einfache (modale) Dialoge mit `JOptionPane`
- `JTabbedPane` als Panel mit Tabs
- `JScrollPane`, um Eingabefelder bei Bedarf scrollbar zu machen
- Anlegen einer Menüleiste mit `JMenuBar`, dabei sind die Menüs `JMenu` und die Einträge `JMenuItem`
- Kontextmenüs mit `JPopupMenu`

Videos

Vorlesung [YT], [HSBI]

Demos:

- `JRadioButton` [YT], [HSBI]
- `JFileChooser` [YT], [HSBI]
- `JOptionPane` [YT], [HSBI]
- `JTabbedPane` und `JScrollPane` [YT], [HSBI]
- `JMenuBar` [YT], [HSBI]
- `JPopupMenu` [YT], [HSBI]

7.3.1. Radiobuttons: *JRadioButton*



- Erzeugen einen neuen “Knopf” (rund)
 - vergleiche `JCheckBox` => eckiger “Knopf”
- Parameter: Beschriftung und Aktivierung
- Reagieren mit `ItemListener`
- **Logische Gruppierung der Buttons:** `ButtonGroup`

- `JRadioButton` sind **unabhängige** Objekte
- Normalerweise nur ein Button aktiviert
- Aktivierung eines Buttons => vormals aktivierter Button deaktiviert

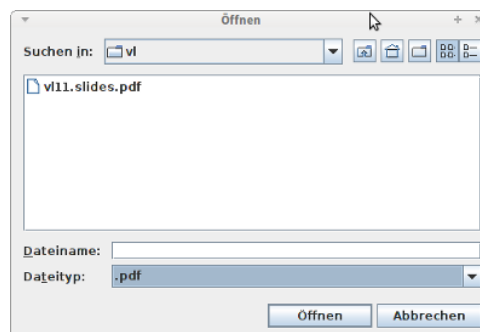
```

1  JRadioButton b1 = new JRadioButton("Button 1", true);
2  JRadioButton b2 = new JRadioButton("Button 2", false);
3
4  ButtonGroup radioGroup = new ButtonGroup();
5  radioGroup.add(b1);    radioGroup.add(b2);

```

Demo: [widgets.RadioButtonDemo](#)

7.3.2. Dateien oder Verzeichnisse auswählen: *JFileChooser*



```

1  JFileChooser fc = new JFileChooser("Startverzeichnis");
2  fc.setFileSelectionMode(JFileChooser.FILES_AND_DIRECTORIES);
3  if (fc.showOpenDialog() == JFileChooser.APPROVE_OPTION)
4      fc.getSelectedFile()

```

- `fc.setFileSelectionMode()`: Dateien, Ordner oder beides auswählbar
- Anzeigen mit `fc.showOpenDialog()`
- Rückgabewert vergleichen mit `JFileChooser.APPROVE_OPTION`: Datei/Ordner wurde ausgewählt => Prüfen!
- Selektierte Datei als `File` bekommen: `fc.getSelectedFile()`

Filtern der Anzeige: *FileFilter*

- Setzen mit `JFileChooser.setFileFilter()`
- Überschreiben von
 - `boolean accept(File f)`
 - `String getDescription()`

Demo: [widgets.FileChooserDemo](#)

7.3.3. TabbedPane und Scroll-Bars

- **TabbedPane:** `JTabbedPane`

- Container für weitere Komponenten
- Methode zum Hinzufügen anderer Swing-Komponenten:

```
1 public void addTab(String title, Icon icon, Component component, String  
   tip)
```

- **Scroll-Bars:** `JScrollPane`

- Container für weitere Komponenten
- Scroll-Bars werden bei Bedarf sichtbar
- Hinzufügen einer Komponente:

```
1 JPanel panel = new JPanel();  
2 JTextArea text = new JTextArea(5, 10);  
3  
4 JScrollPane scrollText = new JScrollPane(text);  
5 panel.add(scrollText);
```

[Demo: widgets.TabbedPaneDemo](#)

7.3.4. Dialoge mit `JOptionPane`



```
1 JOptionPane.showMessageDialog(  
2     this,  
3     "Krasse Warnung!",  
4     "Das ist mein Titel",  
5     JOptionPane.WARNING_MESSAGE)
```


Ein Dialog ist ein eigenes Top-Level-Fenster, welches zumindest eine Message zeigt. Zusätzlich kann man den Fenster-Titel einstellen und ein kleines Icon anzeigen lassen, was verdeutlichen soll, ob es sich um eine Bestätigung oder Frage oder Warnung etc. handelt.

Damit der Dialog auch wirklich bedient werden muss, ist er “modal”, d.h. er liegt “vor” der Elternkomponente. Diese wird als Referenz übergeben und bekommt erst wieder den Fokus, wenn der Dialog geschlossen wurde.

[Demo: widgets.DialogDemo](#)

7.3.5. Menüs mit *JMenuBar*, *JMenu* und *JMenuItem*

```
1 JMenuBar menuBar = new JMenuBar();
2 JMenu menu1 = new JMenu("(M)ein Menü");
3 JMenuItem m1 = new JMenuItem("Text: A");
4 JMenuItem m2 = new JMenuItem("Text: B");
5
6 menu1.add(m1);
7 menu1.add(m2);
8
9 menuBar.add(menu1);
10
11 frame.setJMenuBar(menuBar);
```

Eine Menüleiste wird über das Objekt *JMenuBar* realisiert. Diese ist eine Eigenschaft des Frames und kann nur dort hinzugefügt werden.

In der Menüleiste kann es mehrere Menüs geben, diese werden mit Objekten vom Typ *JMenu* erstellt.

Wenn man mit der Maus ein Menü ausklappt, wird eine Liste der Menüeinträge angezeigt. Diese sind vom Typ *JMenuItem* und verhalten sich wie Buttons.

[Demo: widgets.MenuDemo](#)

7.3.6. Kontextmenü mit *JPopupMenu*

- Menü kann über anderen Komponenten angezeigt werden
- Einträge vom Typ *JMenuItem* hinzufügen (beispielsweise *JRadioButtonMenuItem*)

```
1 public JMenuItem add(JMenuItem menuItem)
```

- Menü über der aufrufenden Komponente “invoker” anzeigen

```
1 public void show(Component invoker, int x, int y)
```

7.3.6.1. Details zu *JMenuItem*

- Erweitert *AbstractButton*
- Reagiert auf *ActionEvent* => *ActionListener* implementieren für Reaktion auf Menüauswahl

7.3.6.2. Details zum Kontextmenü

Triggern der Anzeige eines `JPopupMenu`

- Beispielsweise über `MouseListener` einer (anderen!) Komponente
- Darin Reaktion auf `MouseEvent.isPopupTrigger()` => `JPopupMenu.show()` aufrufen

```
1 JFrame myFrame = new JFrame();
2 JPopupMenu kontextMenu = new JPopupMenu();
3
4 myFrame.addMouseListener(new MouseAdapter() {
5     public void mousePressed(MouseEvent e) {
6         if (e.isPopupTrigger()) {
7             kontextMenu.show(e.getComponent(), e.getX(), e.getY());
8         }
9     }
10 });
```

Demo: [widgets.PopupDemo](#)

7.3.7. Wrap-Up

Nützliche Swing-Komponenten:

- Scroll-Bars
- Panel mit Tabs
- Dialogfenster und Dateiauswahl-Dialoge
- Menüleisten und Kontextmenü
- Radiobuttons und Checkboxes, logische Gruppierung

Zum Nachlesen

Zum Thema Swing Widgets können Sie im Tutorial [“Lesson: Using Swing Components” \(Oracle\)](#) nachlesen.

Lernziele

- k3: Umgang mit komplexeren Swing-Komponenten: `JRadioButton`, `JFileChooser`, `JOptionPane`, `JTabbedPane`, `JScrollPane`, `JMenuBar`, `JPopupMenu`
- k3: Nutzung von `ActionListener`, `MouseListener`, `KeyListener`, `FocusListener`

7.4. Swing 3: Layout-Manager

TL;DR

Zur Anordnung von Komponenten greift Swing auf sogenannte Layout-Manager zurück. Hier gibt es viele verschiedene Ausprägungen und Spielarten. Es gibt beispielsweise:

- das `BorderLayout`, eine gitterartige Struktur mit fünf Elementen,

- das `FlowLayout`, eine einzeilige Anordnung mit automatischem Umbruch bei Platzmangel,
- das `GridLayout`, eine tabellenartige Struktur, in der alle Elemente gleich groß dargestellt werden, und
- das `GridBagLayout`, welches sich prinzipiell wie das `GridLayout` verhält und mehr Möglichkeiten bietet. Zur Anordnung der Komponenten greift man hier auf `GridBagConstraints` zurück und kann sehr genau und sehr flexibel definieren, wo und wie die Komponenten angeordnet sein sollen und sich bei Größenänderungen des Containers verhalten sollen.

Videos

Vorlesung [YT], [HSBI]

Demos:

- `BorderLayout` [YT], [HSBI]
- `FlowLayout` [YT], [HSBI]
- `GridLayout` [YT], [HSBI]
- `GridBagLayout` [YT], [HSBI]

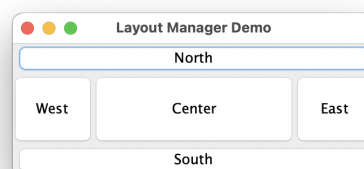
7.4.1. Überblick

Anordnung der Komponenten in einem Container ist abhängig vom **Layout**

Verschiedene beliebte Layout-Manager:

- `BorderLayout`
- `FlowLayout`
- `GridLayout`
- `GridBagLayout`
- ...

7.4.2. BorderLayout



```
1 JPanel contentPane = new JPanel();  
2
```

```

3  contentPane.setLayout(new BorderLayout());
4
5  contentPane.add(new JButton("North"), BorderLayout.NORTH); // also: PAGE_START
6  contentPane.add(new JButton("West"), BorderLayout.WEST); // also: LINE_START
7  contentPane.add(new JButton("Center"), BorderLayout.CENTER);
8  contentPane.add(new JButton("East"), BorderLayout.EAST); // also: LINE_END
9  contentPane.add(new JButton("South"), BorderLayout.SOUTH); // also: PAGE_END

```

Es gibt fünf verschiedene Bereiche, in denen die Komponenten bei einem Border-Layout angeordnet werden können. Für die "historischen" Konstanten `NORTH`, `SOUTH`, `WEST` und `EAST` gibt es mittlerweile neue Namen, die eher am Aufbau einer Seite orientiert werden können.

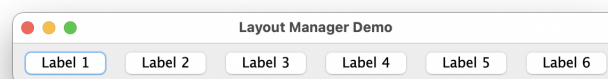
Man kann auch nur einige Teile nutzen, bei der Tabellen-Demo beispielsweise wurde nur der `NORTH`-Bereich für den Tabellenkopf und der `CENTER`-Bereich für die eigentliche Tabelle genutzt.

Wenn das Fenster vergrößert wird, bekommt zunächst der Mittelteil den neuen zur Verfügung stehenden Platz. Die anderen Bereiche werden dabei auf vergrößert, aber nur so weit, dass der neue verfügbare Platz ggf. ausgefüllt wird.

Mit den Methoden `setHgap()` und `setVgap()` kann der Abstand zwischen den Komponenten eingestellt werden (horizontal und vertikal, Abstände in Pixel).

[Demo: layout.Border](#)

7.4.3. FlowLayout



```

1  JPanel contentPane = new JPanel();
2
3  contentPane.setLayout(new FlowLayout());
4
5  contentPane.add(new JButton("Label 1"));
6  contentPane.add(new JButton("Label 2"));
7  contentPane.add(new JButton("Label 3"));

```

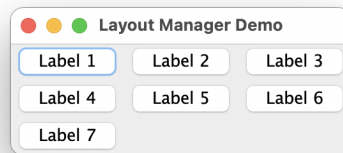
Das `FlowLayout` ist ein sehr einfaches Layout, welches per Default in `JPanel` genutzt wird.

Die Komponenten werden der Reihe nach in einer Zeile angeordnet. Wenn der Platz nicht ausreicht, bricht diese Zeile um in mehrere Zeilen.

Per Default werden die Komponenten zentriert angeordnet. Über den Konstruktor oder die Methoden `setAlignment()` und `setHgap()` bzw. `setVgap()` kann aber eine andere Ausrichtung definiert werden, ebenso wie ein vertikales und horizontales Padding zwischen den Komponenten.

[Demo: layout.Flow](#)

7.4.4. GridLayout



```

1  JPanel contentPane = new JPanel();
2
3  contentPane.setLayout(new GridLayout(0, 3));
4
5  contentPane.add(new JButton("Label 1"));
6  contentPane.add(new JButton("Label 2"));
7  contentPane.add(new JButton("Label 3"));

```

Das `GridLayout` ist ein sehr einfaches Layout mit einer tabellenartigen Struktur. Dabei werden die Komponenten nacheinander auf die “Zellen” verteilt, beginnend mit der ersten Zeile. Alle Komponenten werden dabei gleich groß dargestellt.

Über den Konstruktor wird die Anzahl der gewünschten Zeilen und Spalten angegeben. Es darf auch für einen der beiden Parameter der Wert 0 verwendet werden, in diesem Fall werden so viele Zeilen oder Spalten angelegt, wie für die hinzugefügten Komponenten benötigt.

Auch in diesem Layout kann das Padding über die Methoden `setHgap()` bzw. `setVgap()` eingestellt werden.

[Demo: layout.Grid](#)

7.4.5. Komplexer Layout-Manager: GridBagLayout

- Layout-Manager ähnlich zu `GridLayout`
- Zusätzlich `GridBagConstraints`: Verhalten bei Größenveränderungen

Constraint	Bedeutung
<code>gridx</code>	Spalte für Komponente (linke obere Ecke)
<code>gridy</code>	Zeile für Komponente (linke obere Ecke)
<code>gridwidth</code>	Anzahl der Spalten für Komponente
<code>gridheight</code>	Anzahl der Zeilen für Komponente

Constraint	Bedeutung
<code>fill</code>	Vergrößert Komponente in Richtung: <code>NONE</code> , <code>HORIZONTAL</code> , <code>VERTICAL</code> , <code>BOTH</code>
<code>weightx</code>	Platz in x-Richtung unter Grid-Slots nach "Gewicht" aufteilen
<code>weighty</code>	Platz in y-Richtung unter Grid-Slots nach "Gewicht" aufteilen

Beim Hinzufügen einer Komponente wird eine Instanz der Klasse `GridBagConstraints` mitgegeben. Diese definiert, wie die Komponente in der gitterartigen Struktur konkret angeordnet werden soll: Startposition im Gitter (x, y) bzw (Spalte, Zeile), wie viele Spalten oder Zeilen soll die Komponente überstreichen und wie soll auf Größenänderungen des Containers reagiert werden.

Beispiel:

```

1  JPanel contentPane = new JPanel();
2  contentPane.setLayout(new GridBagLayout());
3
4  GridBagConstraints c2 = new GridBagConstraints();
5  c2.gridx = 1;
6  c2.gridy = 0;
7  c2.gridheight = 2;
8  c2.fill = GridBagConstraints.VERTICAL;
9  c2.weightx = 0.5;
10 c2.weighty = 0.5;
11
12 contentPane.add(new JButton("Label 2"), c2);

```

Der Button wird dem Panel mit dem `GridBagLayout` hinzugefügt und soll in Spalte 1 und Zeile 0 angeordnet werden. Er soll sich dabei über 2 Zeilen erstrecken (und 1 Spalte). Der Button soll sich in vertikaler Richtung vergrößern, sofern Platz zur Verfügung steht.

Dem Grid-Slot wird ein Gewicht in x- und in y-Richtung von je 0.5 mitgegeben. Bei einer Änderung des Containers in der jeweiligen Richtung wird der neue Platz unter den Slots gemäß ihren Gewichten aufgeteilt.

[Demo: layout.GridBag](#)

7.4.6. Wrap-Up

- Anordnung von Komponenten lässt sich mit Layout-Manager steuern
- Auswahl von beliebigen Layout-Managern:
 - `BorderLayout`: Gitterartige Struktur mit fünf Elementen
 - `FlowLayout`: Zeilenweise Anordnung (Umbruch bei Platzmangel)
 - `GridLayout`: Tabellenartige Struktur, Elemente gleich groß
 - `GridBagLayout`: Wie `GridLayout`, mit mehr Möglichkeiten: `GridBagConstraints`

Zum Nachlesen

Zum Thema Swing Layouts und -manager können Sie im Tutorial [“Lesson: Laying Out Components Within a Container” \(Oracle\)](#) nachlesen.

Lernziele

- k3: Anwenden der verschiedenen Layout-Manager: BorderLayout, FlowLayout, GridLayout, GridBagLayout

Challenges

In den [Vorgaben](#) eine Implementierung für ein TicTacToe-Spiel. Ihre Aufgabe ist es, eine grafische Benutzeroberfläche für das Spiel zu entwickeln.

Ihr Fenster soll sich immer in der Mitte des Bildschirms starten und einen Titel besitzen.

Beim Start des Spiels sollen beide Spieler ihre Namen eingeben können, nutzen Sie dafür `JTextField`. Stellen Sie sicher, dass nur gültige Eingaben getätigt werden. Sind die beiden Namen gültig, erstellen Sie die jeweiligen `Player` und starten Sie ein Spiel `TicTacToe`.

Im Zentrum Ihres Spielfensters soll das Spielfeld angezeigt werden. Nutzen Sie dafür das `GridLayout` und `JButton`. Die Buttons repräsentieren dabei die Felder des Spiels. Beim Drücken eines Buttons soll die Methode `TicTacToe#makeMove` aufgerufen werden.

Mit `TicTacToe#getGameField` können Sie sich das aktuelle Spielfeld übergeben lassen. Sorgen Sie dafür, dass Ihre Oberfläche immer den aktuellen Zustand des Spielfeldes anzeigt.

Prüfen Sie nach jedem Spielzug den Status des Spiels mit `TicTacToe#getCurrentGameState`:

- Wenn ein Spieler gewonnen hat, soll ein `JOptionPane` angezeigt werden und dem Gewinner gratulieren.
- Wenn ein Unentschieden gespielt wurde, soll ein `JOptionPane` angezeigt werden und das Unentschieden angezeigt werden.
- In beiden Fällen soll danach eine neue Runde gestartet werden.

Im unteren Bereich des Fensters soll der Spieler angezeigt werden, der aktuell am Zug ist. Im unteren Bereich des Fensters soll auch der aktuelle Punktestand angezeigt werden.

Das Fenster soll eine Menüleiste mit folgenden Punkten haben:

- Exit: Beendet das Programm.
- New Game: Startet das Spiel neu und erlaubt die neue Eingabe der Spielernamen.
- Clear: Setzt das aktuelle Spielfeld zurück.

Denken Sie bei der Umsetzung daran, dass der Benutzer nur die Oberfläche sieht und bedienen kann. Stellen Sie sicher, dass alle Bedienelemente verständlich sind. Nutzen Sie ggf. `JLabel`, um Texte auf der UI anzuzeigen.

7.5. Swing 6: Einführung in Graphics und Java 2D

TL;DR

Swing-Komponenten zeichnen mit `paintComponent()` auf einem `Graphics`-Objekt. Die Methode wird von Swing selbst aufgerufen; man kann sie durch den Aufruf von `repaint()` auf einer Swing-Komponente aber manuell triggern.

Die Klasse `Graphics` bietet verschiedene einfache Methoden zum Zeichnen von Linien, Rechtecken, Ovalen und Texten ... Die davon ableitende Klasse `Graphics2D` bietet deutlich mehr Möglichkeiten, und das Argument beim Aufruf von `paintComponent()` ist zwar formal vom Typ `Graphics`, in der Praxis aber oft vom Typ `Graphics2D` (Typprüfung und anschließender Cast nötig).

Das Koordinatensystem in Java2D hat den Ursprung in der linken oberen Ecke.

Geometrische Primitive und Text werden in der aktuell ausgewählten Zeichenfarbe gerendert. Die Rechtecke, Ovale und Polygone existieren auch als "gefüllte" Variante.

Da bei einem Aufruf von `paintComponent()` stets das komplette Objekt neu gezeichnet wird, kann man dies in einer Game-Loop nutzen: Pro Schritt berechnet man für alle Objekte die neue Position, lässt ggf. weitere Interaktion o.ä. berechnen und zeichnet anschließend die Objekte über den Aufruf von `repaint()` neu. In der Game-Loop werden also keine Threads benötigt.

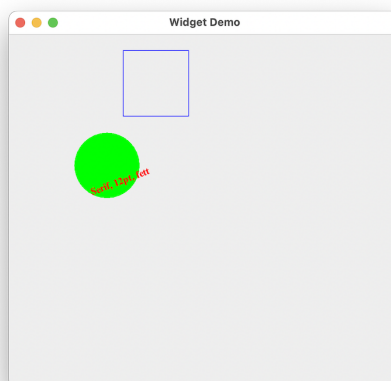
Videos

Vorlesung [YT], [HSBI]

Demos:

- Geometrische Objekte [YT], [HSBI]
- Fonts [YT], [HSBI]
- Polygone [YT], [HSBI]
- Bewegung und 2D-Spiel [YT], [HSBI]

7.5.1. GUIs mit Java



7.5.2. Einführung in die Java 2D API

- Bisher: Anordnung von Widgets als GUI
- Jetzt: Wie kann man mit Java zeichnen etc.?

Swing-Komponenten erben von `javax.swing.JComponent`:

```
1 public void paintComponent(Graphics g)
```

- Wird durch Events aufgerufen
- Oder "von Hand" mit `void repaint()`

Methode `repaint()` der Swing-Komponente aufrufen => dadurch wird dann intern die Methode `paintComponent()` der Komponente aufgerufen zum Neuzeichnen auf dem Graphics-Objekt.

Objekt vom Typ `Graphics` stellt graphischen Kontext dar

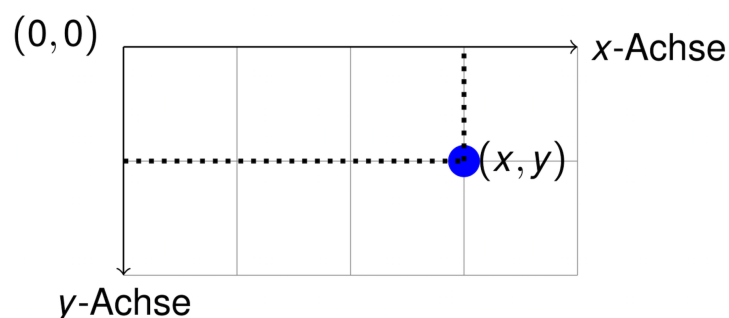
- Geom. Primitive zeichnen mit `draw*` und `fill*`
- Rendern mit `drawString` und `drawImage`
- ...

`Graphics2D` beherrscht zusätzliche Methoden zum Beeinflussen des Renderings

=> **Methode überschreiben und auf der GUI malen**

- Basis: `java.awt.Graphics`; davon abgeleitet `java.awt.Graphics2D`
- Methode zum Zeichnen: `paintComponent()`
- Umgang mit Farben: `java.awt.Color`
- Umgang mit Zeichen und Fonts: `java.awt.Font`
- Geom. Primitive: `java.awt.Polygon`, `java.awt.geom.{Line2D, Rectangle2D, Ellipse2D}`,...

7.5.3. Java2D Koordinatensystem



- Koordinatensystem lokal zum Graphics-Objekt
- Einheiten in Pixel(!)

7.5.4. Einfache Objekte zeichnen

Methoden von `java.awt.Graphics` (Auswahl):

```
1 public void drawLine(int x1, int y1, int x2, int y2)
2 public void drawRect(int x, int y, int width, int height)
3 public void fillRect(int x, int y, int width, int height)
4 public void drawOval(int x, int y, int width, int height)
5 public void fillOval(int x, int y, int width, int height)
```

Vorher Strichfarbe setzen: `Graphics.setColor(Color color)`:

- Farb-Konstanten in `java.awt.Color`: `RED`, `GREEN`, `WHITE`, ...
- Ansonsten über Konstruktor, beispielsweise als RGB:

```
1 public Color(int r, int g, int b) // Rot/Grün/Blau, Werte zw. 0 und 255
```

[Demo: java2d.SimpleDrawings](#)

7.5.5. Fonts und Strings

Fonts über Font-Klasse einstellen: `Graphics.setFont(Font font)`;

```
1 public Font(String name, int style, int size)
```

`Graphics` kann Strings "zeichnen":

```
1 public void drawString(String str, int x, int y);
```

Vorher Font und Farbe setzen!

[Demo: java2d.SimpleFonts](#)

7.5.6. Einfache Polygone definieren

Polygone zeichnen: `Graphics.drawPolygon(Polygon p)`:

```
1 public Polygon()
2 public Polygon(int[] xPoints, int[] yPoints, int points)
3 public void addPoint(int x, int y)
```

=> weitere Methoden von `Graphics` (Auswahl):

```
1 public void drawPolyline(int[] xp, int[] yp, int np)
2
3 public void drawPolygon(int[] xp, int[] yp, int np)
4 public void drawPolygon(Polygon p)
```

Polygone mit Farbe füllen: `Graphics.fillPolygon(Polygon p)`

```
1 public void fillPolygon(int[] xp, int[] yp, int np)
2 public void fillPolygon(Polygon p)
```

Statt `drawPolygon()`

Vorher Farbe setzen!

[Demo: java2d.SimplePoly](#)

7.5.7. Ausblick I: Umgang mit Bildern

```
1 BufferedImage img = ImageIO.read(new File("DukeWave.gif"));
2
3 boolean Graphics.drawImage(Image img, int x, int y, ImageObserver observer);
```

7.5.8. Ausblick II: *Graphics2D* kann noch mehr ...

```
1 Graphics g;
2 Graphics2D g2 = (Graphics2D) g;
```

=> `Line2D`, `Rectangle2D`, ...

- Strichstärken, Strichmuster
- Clippings
- Transformationen: rotieren, ...
- Zeichnen in Bildern, Rendern von Ausschnitten
- ...

7.5.9. Spiele mit Bewegung

Beobachtung: `paintComponent()` schreibt `Graphics`-Objekt komplett neu!

- Kein Löschen von Objekten nötig
- Es müssen alle im nächsten Schritt sichtbaren Objekte stets neu gezeichnet werden

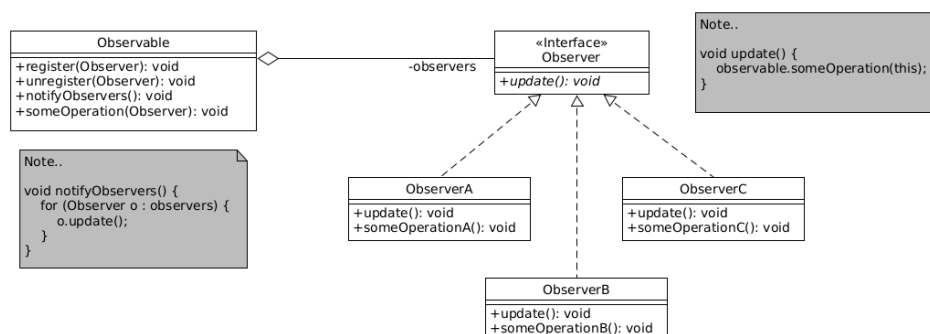
Idee: Je Zeitschritt:

1. Position der Objekte neu berechnen
2. Weitere Berechnungen: Kollision, Interaktion, Angriff, ...

3. Objekte mit `paintComponent()` neu in GUI zeichnen

- Möglichkeit 1: Alle Objekte in zentraler Datenstruktur halten und die Bewegung im Hauptprogramm berechnen
 - Unschön: Das Hauptprogramm muss Hintergrundwissen über die Objekte und deren Bewegung haben
 - Möglichkeit 2: Die Objekte wissen selbst, wie sie sich bewegen und haben eine Methode, deren Aufruf die Bewegung durchführt
 - Objekte als Listener im Hauptprogramm registrieren
 - Hauptprogramm gibt Zeittakt vor und ruft je Schritt für alle Listener die Bewege-Methode auf => Listener berechnen ihre neue Position
 - Hauptprogramm kann weitere Prüfungen (Kollision etc) auslösen
 - Hauptprogramm ruft für alle Listener eine Paint-Methode auf => Listener stellen sich auf GUI dar ...
- => Observer-Pattern nutzen

7.5.10. Erinnerung: Observer Pattern



- Anzahl der Observer muss nicht bekannt sein - zur Laufzeit erweiterbar!
- Verschiedene Update-Methoden für unterschiedliche Observer denkbar
- **Push-Modell:** Benötigte Daten werden der Update-Methode mitgegeben
- **Pull-Modell:** Update-Methode nur als Trigger, Observer holen sich die Daten selbst
- Referenz auf Observable mitgeben - Observer braucht dann keine Referenz auf das Observable halten und kann sich bei verschiedenen Observables registrieren
- **Observer** werden (vor allem im Swing-Umfeld) manchmal auch **Listener** genannt

7.5.11. Spielobjekte als Observer (Listener)

Objekte können sich auf **Graphics** darstellen:

- Ursprung, Breite, Höhe
- Schrittweite pro Bewegungsschritt

```

1  abstract class GameObject {
2      abstract void move();
3      abstract void paintTo(Graphics g); // entspricht Observer#update()
4  }
5
6  class GameRect extends GameObject {
7      int x, y, deltaX;
8      void move() { x += deltaX; }
9      void paintTo(Graphics g) {
10         g.drawRect(x, y, 80, 80);
11     }
12 }

```

Weitere evtl. nützliche Methoden:

- Check auf Kollision
- Methode zum Umdrehen der Bewegungsrichtung

7.5.12. Oberfläche zusammenbauen

1. Spielfeld von `JPanel` ableiten: `Observable`
2. Observer registrieren: Liste mit Spiel-Objekten anlegen
3. `paintComponent()` vom Spielfeld überschreiben
 - für alle Observer (Spiel-Objekte) `paintTo()` aufrufen
4. Hauptschleife für Spiel:
 - Taktgeber (Zeit, Interaktion)
 - Je Schritt `move()` für alle Observer aufrufen
 - Weitere Berechnungen (Kollisionen, Interaktionen, ...)
 - `Spielfeld.repaint()` aufrufen => Neuzeichnen mit `paintComponent()`

Pro Schritt:

1. `move()` für alle Objekte aufrufen: Objekte setzen ihren Ursprung weiter (**ohne Aktualisierung des Bildes!**)
2. Prüfungen: Kollision/Berührung, aus dem Bild wandern ...
 - Beispiele für Verhalten bei Berührung:
 - beide kehren ihre Bewegungsrichtung um
 - das kleinere Objekt verschwindet
 - Beispiele für Umgang mit Objekten, die aus dem Bild wandern:
 - auf der anderen Seite einblenden
 - Bewegungsrichtung umkehren
 - Objekt aus dem Spiel nehmen
3. Interaktionen
 - Greifen sich Monster und Held an?

- Öffnet der Held eine Truhe?
 - Sammelt der Held etwas auf?
 - ...
4. `repaint()` im Spielfeld aufrufen => damit wird `paintComponent()` aufgerufen, in `paintComponent()` wird für alle Spielobjekte deren `paintTo()` aufgerufen und damit ein Neuzeichnen aller Objekte ausgelöst

Demo: [java2d.simplegame.J2DTeaser](#)

7.5.13. Wrap-Up

- Java2D: Swing-Komponenten zeichnen mit `paintComponent()` auf `Graphics`
- `Graphics`: Methoden zum Zeichnen von Linien, Rechtecken, Ovalen, Text ...
 - Koordinatensystem: Ursprung links oben!
 - Geom. Primitive und Text werden in ausgewählter Zeichenfarbe gerendert
 - * Rechtecke, Ovale, Polygone auch als "gefüllte" Variante
 - Mehr Möglichkeiten: `Graphics2D`
- Spiel: Game-Loop
 - Bewege Objekte: Rechne neue Position aus
 - Interagiere: Angriffe, Sammeln, ...
 - Zeichne Objekte neu

Zum Nachlesen

Zum Thema Java2D können Sie im Tutorial "[Trail: 2D Graphics](#)" (Oracle) nachlesen.

Lernziele

- k2: Unterschied und Zusammenhang zwischen Swing und AWT
- k2: Swing-Komponenten erben `paintComponent(Graphics)`
- k2: `paintComponent(Graphics)` wird durch Events oder durch `repaint()` aufgerufen
- k3: Auf `Graphics`-Objekt zeichnen mit geometrischen Primitiven: Nutzung von `draw()`, `fill()`, `drawString()`
- k3: Einstellung von Farbe und Font
- k3: Erzeugen von Bewegung ohne Nutzung von Threads

8. Fortgeschrittene Java-Themen und Umgang mit JVM

8.1. Reguläre Ausdrücke

TL;DR

Mit Hilfe von regulären Ausdrücken kann man den Aufbau von Zeichenketten formal beschreiben. Dabei lassen sich direkt die gewünschten Zeichen einsetzen, oder man nutzt Zeichenklassen oder vordefinierte Ausdrücke. Teilausdrücke lassen sich gruppieren und über *Quantifier* kann definiert werden, wie oft ein Teilausdruck vorkommen soll. Die Quantifier sind per Default **greedy** und versuchen so viel wie möglich zu matchen.

Auf der Java-Seite stellt man reguläre Ausdrücke zunächst als `String` dar. Dabei muss darauf geachtet werden, dass ein Backslash im regulären Ausdruck im Java-String geschützt (*escaped*) werden muss, indem jeweils ein weiterer Backslash voran gestellt wird. Mit Hilfe der Klasse `java.util.regex.Pattern` lässt sich daraus ein Objekt mit dem kompilierten regulären Ausdruck erzeugen, was insbesondere bei mehrfacher Verwendung günstiger in der Laufzeit ist. Dem Pattern-Objekt kann man dann den Suchstring übergeben und bekommt ein Objekt der Klasse `java.util.regex.Matcher` (dort sind regulärer Ausdruck/Pattern und der Suchstring kombiniert). Mit den Methoden `Matcher#find` und `Matcher#matches` kann dann geprüft werden, ob das Pattern auf den Suchstring passt: `find` sucht dabei nach dem ersten Vorkommen des Patterns im Suchstring, `match` prüft, ob der gesamte String zum Pattern passt.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

Demos:

- `StringSplit` [\[YT\]](#), [\[HSBI\]](#)
- `MatchFind` [\[YT\]](#), [\[HSBI\]](#)
- `Quantifier` [\[YT\]](#), [\[HSBI\]](#)
- `Groups` [\[YT\]](#), [\[HSBI\]](#)
- `Backref` [\[YT\]](#), [\[HSBI\]](#)

8.1.1. Suchen in Strings

Gesucht ist ein Programm zum Extrahieren von Telefonnummern aus E-Mails.

=> **Wie geht das?**

Leider gibt es unzählige Varianten, wie man eine Telefonnummer (samt Vorwahl und ggf. Ländervorwahl) aufschreiben kann:

```
1 030 - 123 456 789, 030-123456789, 030/123456789,  
2 +49(30)123456-789, +49 (30) 123 456 - 789, ...
```

8.1.2. Definition Regulärer Ausdruck

Ein **regulärer Ausdruck** ist eine Zeichenkette, die zur Beschreibung von Zeichenketten dient.

8.1.2.1. Anwendungen

- Finden von Bestandteilen in Zeichenketten
- Aufteilen von Strings in Tokens
- Validierung von textuellen Eingaben => "Eine Postleitzahl besteht aus 5 Ziffern"
- Compilerbau: Erkennen von Schlüsselwörtern und Strukturen und Syntaxfehlern

8.1.3. Einfachste reguläre Ausdrücke

Zeichenkette	Beschreibt
<code>x</code>	"x"
<code>.</code>	ein beliebiges Zeichen
<code>\t</code>	Tabulator
<code>\n</code>	Newline
<code>\r</code>	Carriage-return
<code>\\</code>	Backslash

8.1.3.1. Beispiel

- `abc` => "abc"
- `A.B` => "AAB" oder "A2B" oder ...
- `a\\bc` => "a\bc"

8.1.3.2. Anmerkung

In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. geschützt ("escaped") werden. => Statt "`\n`" müssen Sie im Java-Code "`\\n`" schreiben!

8.1.4. Zeichenklassen

Zeichenkette	Beschreibt
[<i>abc</i>]	“a” oder “b” oder “c”
[^ <i>abc</i>]	alles außer “a”, “b” oder “c” (Negation)
[<i>a-zA-Z</i>]	alle Zeichen von “a” bis “z” und “A” bis “Z” (Range)
[<i>a-z&&[def]</i>]	“d”, “e” oder “f” (Schnitt)
[<i>a-z&&[^bc]</i>]	“a” bis “z”, außer “b” und “c”: [<i>ad-z</i>] (Subtraktion)
[<i>a-z&&[^m-p]</i>]	“a” bis “z”, außer “m” bis “p”: [<i>a-lq-z</i>] (Subtraktion)

Zeichenklassen werden über eine Zeichenkette formuliert, die in [und] eingeschlossen wird. Dabei werden alle Zeichen aufgezählt, die in dieser Zeichenklasse enthalten sein sollen. Die Zeichenklasse verhält sich von außen betrachtet wie ein beliebiges Zeichen aus der Menge der aufgezählten Zeichen.

Beispiel: [*abc*] meint ein “a” oder “b” oder “c” ...

Wenn dem ersten Zeichen der so geformten Zeichenklasse ein ^ vorangestellt wird, sind alle Zeichen *außer* den in der Zeichenklasse bezeichneten Zeichen gemeint (Negation). In der Tabelle oben (erste Zeile) könnte man dem *abc* noch ein ^ voranstellen und hätte dann *alle* Zeichen *außer* “a”, “b” und “c”.

Für den Schnitt kann als zweite Zeichenklasse eine Negation verwendet werden, damit würde eine Subtraktion erreicht werden: Alle Zeichen in der vorderen Zeichenklasse abzüglich der Zeichen in der zweiten Zeichenklasse. In der Tabelle oben (vierte Zeile) würde man dem *def* noch ein ^ voranstellen und hätte dann die Zeichen “a” bis “z” *ohne* “d”, “e” und “f”.

Anmerkung: Das Minus-Zeichen hat in der Zeichenklasse eine besondere Bedeutung (es bildet einen Range). Deshalb muss es escaped werden, wenn es sich selbst darstellen soll.

8.1.4.1. Beispiel

- [*abc*] => “a” oder “b” oder “c”
- [*a-c*] => “a” oder “b” oder “c”
- [*a-c*][*a-c*] => “aa”, “ab”, “ac”, “ba”, “bb”, “bc”, “ca”, “cb” oder “cc”
- A[*a-c*] => “Aa”, “Ab” oder “Ac”

8.1.5. Vordefinierte Ausdrücke

Zeichenkette	Beschreibt
^	Zeilenanfang
\$	Zeilenende

Zeichenkette	Beschreibt
<code>\d</code>	eine Ziffer: <code>[0-9]</code>
<code>\w</code>	beliebiges Wortzeichen: <code>[a-zA-Z_0-9]</code>
<code>\s</code>	Whitespace (Leerzeichen, Tabulator, Newline)
<code>\D</code>	jedes Zeichen außer Ziffern: <code>[^0-9]</code>
<code>\W</code>	jedes Zeichen außer Wortzeichen: <code>[^\w]</code>
<code>\S</code>	jedes Zeichen außer Whitespaces: <code>[^\s]</code>

8.1.5.1. Beispiel

- `\d\d\d\d\d => "12345"`
- `\w\wA => "aaA", "a0A", "a_A", ...`

8.1.6. Nutzung in Java

- `java.lang.String`:

```
1 public String[] split(String regex)
2 public boolean matches(String regex)
```

[Demo: regexp.StringSplit](#)

- `java.util.regex.Pattern`:

```
1 public static Pattern compile(String regex)
2 public Matcher matcher(CharSequence input)
```

- Schritt 1: Ein Pattern compilieren (erzeugen) mit `Pattern#compile` => liefert ein Pattern-Objekt für den regulären Ausdruck zurück
- Schritt 2: Dem Pattern-Objekt den zu untersuchenden Zeichenstrom übergeben mit `Pattern#matcher` => liefert ein Matcher-Objekt zurück, darin gebunden: Pattern (regulärer Ausdruck) und die zu untersuchende Zeichenkette

- `java.util.regex.Matcher`:

```
1 public boolean find()
2 public boolean matches()
3 public int groupCount()
4 public String group(int group)
```

- Schritt 3: Mit dem Matcher-Objekt kann man die Ergebnisse der Anwendung des regulären Ausdrucks auf eine Zeichenkette auswerten

Bedeutung der unterschiedlichen Methoden siehe folgende Folien

`Matcher#group`: Liefert die Sub-Sequenz des Suchstrings zurück, die erfolgreich gematcht wurde (siehe unten "Fangende Gruppierungen")

Hinweis:

In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. extra geschützt ("escaped") werden.

=> Statt "`\n`" (regulärer Ausdruck) müssen Sie im Java-String "`\\n`" schreiben!

=> Statt "`a\\bc`" (regulärer Ausdruck, passt auf die Zeichenkette "`a\\bc`") müssen Sie im Java-String "`a\\\\bc`" schreiben!

[Demo: `regex.MatchFind`](#)

8.1.7. Unterschied zw. Finden und Matchen

- `Matcher#find`:

Regulärer Ausdruck muss im Suchstring **enthalten** sein.

=> Suche nach **erstem Vorkommen**

- `Matcher#matches`:

Regulärer Ausdruck muss auf **kompletten** Suchstring passen.

8.1.7.1. Beispiel

- Regulärer Ausdruck: `abc`, Suchstring: "blah blah abc blub"
 - `Matcher#find`: erfolgreich
 - `Matcher#matches`: kein Match - Suchstring entspricht nicht dem Muster

8.1.8. Quantifizierung

Zeichenkette	Beschreibt
<code>X?</code>	ein oder kein "X"
<code>X*</code>	beliebig viele "X" (inkl. kein "X")
<code>X+</code>	mindestens ein "X", ansonsten beliebig viele "X"
<code>X{n}</code>	exakt <i>n</i> Vorkommen von "X"
<code>X{n,}</code>	mindestens <i>n</i> Vorkommen von "X"

Zeichenkette	Beschreibt
<code>X{n,m}</code>	zwischen <i>n</i> und <i>m</i> Vorkommen von "X"

8.1.8.1. Beispiel

- `\d{5}` => "12345"
- `-?\d+\.\d*` => ???

8.1.9. Interessante Effekte

```

1 Pattern p = Pattern.compile("A.*A");
2 Matcher m = p.matcher("A 12 A 45 A");
3
4 if (m.matches())
5     String result = m.group(); // ???

```

Demo: [regex.Quantifier](#)

`Matcher#group` liefert die Inputsequenz, auf die der Matcher angesprochen hat. Mit `Matcher#start` und `Matcher#end` kann man sich die Indizes des ersten und letzten Zeichens des Matches im Eingabezeichenstrom geben lassen. D.h. für einen Matcher *m* und eine Eingabezeichenkette *s* ist `m.group()` und `s.substring(m.start(), m.end())` äquivalent.

Da bei `Matcher#matches` das Pattern immer auf den gesamten Suchstring passen muss, verwundert das Ergebnis für `Matcher#group` nicht. Bei `Matcher#find` wird im Beispiel allerdings ebenfalls der gesamte Suchstring "gefunden" ... Dies liegt am "greedy" Verhalten der Quantifizierer.

8.1.10. Nicht gierige Quantifizierung mit "?"

Zeichenkette	Beschreibt
<code>X*?</code>	non-greedy Variante von <code>X*</code>
<code>X+?</code>	non-greedy Variante von <code>X+</code>

8.1.10.1. Beispiel

- Suchstring "A 12 A 45 A":
 - `A.*A` findet/passt auf "A 12 A 45 A"
 - normale **greedy** Variante

- `A.*?A`

- * findet "A 12 A"

- * passt auf "A 12 A 45 A" (!)

non-greedy Variante der Quantifizierung; `Matcher#matches` muss trotzdem auf den gesamten Suchstring passen!

8.1.11. (Fangende) Gruppierungen

`Studi{2}` passt nicht auf "StudiStudi" (!)

Quantifizierung bezieht sich auf das direkt davor stehende Zeichen. Ggf. Gruppierungen durch Klammern verwenden!

Zeichenkette	Beschreibt
<code>X\ Y</code>	X oder Y
<code>(C)</code>	Gruppierung

8.1.11.1. Beispiel

- `(A)(B(C))`

- Gruppe 0: `ABC`

- Gruppe 1: `A`

- Gruppe 2: `BC`

- Gruppe 3: `C`

Die Gruppen heißen auch "fangende" Gruppen (engl.: "*capturing groups*").

Damit erreicht man eine Segmentierung des gesamten regulären Ausdrucks, der in seiner Wirkung aber nicht durch die Gruppierungen geändert wird. Durch die Gruppierungen von Teilen des regulären Ausdrucks erhält man die Möglichkeit, auf die entsprechenden Teil-Matches (der Unterausdrücke der einzelnen Gruppen) zuzugreifen:

- `Matcher#groupCount`: Anzahl der "fangenden" Gruppen im regulären Ausdruck
- `Matcher#group(i)`: Liefert die Subsequenz der Eingabezeichenkette zurück, auf die die jeweilige Gruppe gepasst hat. Dabei wird von links nach rechts durchgezählt, beginnend bei 1(!).

Konvention: Gruppe 0 ist das gesamte Pattern, d.h. `m.group(0) == m.group()`; ...

Hinweis: Damit der Zugriff auf die Gruppen klappt, muss auch erst ein Match gemacht werden, d.h. das Erzeugen des `Matcher`-Objekts reicht noch nicht, sondern es muss auch noch ein `matcher.find()` oder `matcher.matches()` ausgeführt werden. Danach kann man bei Vorliegen eines Matches auf die Gruppen zugreifen.

`(Studi){2} => "StudiStudi"`

[Demo: regexp.Groups](#)

8.1.12. Gruppen und Backreferences

Matche zwei Ziffern, gefolgt von den selben zwei Ziffern

`(\d\d)\1`

- Verweis auf bereits gematchte Gruppen: `\num`

`num` Nummer der Gruppe (1 ... 9)

=> Verweist nicht auf regulären Ausdruck, sondern auf jeweiligen Match!

Anmerkung: Laut Literatur/Doku nur 1 ... 9, in Praxis geht auch mehr per Backreference ...

- Benennung der Gruppe: `(?<name>X)`

`X` ist regulärer Ausdruck für Gruppe, spitze Klammern wichtig

=> Backreference: `\k<name>`

[Demo: regexp.Backref](#)

8.1.13. Beispiel Gruppen und Backreferences

Regulärer Ausdruck: Namen einer Person matchen, wenn Vor- und Nachname identisch sind.

Lösung: `([A-Z][a-zA-Z]*)\s\1`

8.1.14. Umlaute und reguläre Ausdrücke

- Keine vordefinierte Abkürzung für Umlaute (wie etwa `\d`)
- Umlaute nicht in `[a-z]` enthalten, aber in `[a-ü]`

```
1 "helloüA".matches(".*?[ü]A");
2 "azäöüß".matches("[a-ä]");
3 "azäöüß".matches("[a-ö]");
4 "azäöüß".matches("[a-ü]");
5 "azäöüß".matches("[a-ß]");
```

- Strings sind Unicode-Zeichenketten

=> Nutzung der passenden Unicode Escape Sequence `\uFFFF`

```
1 System.out.println("\u0041 :: A");
2 System.out.println("helloüA".matches(".*?A"));
3 System.out.println("helloüA".matches(".*?\u0041"));
4 System.out.println("helloü\u0041".matches(".*?A"));
```

- RegEx vordefinieren und mit Variablen zusammenbauen ala Perl nicht möglich => Umweg String-Repräsentation

8.1.15. Wrap-Up

- RegExp: Zeichenketten, die andere Zeichenketten beschreiben
- `java.util.regex.Pattern` und `java.util.regex.Matcher`
- Unterschied zwischen `Matcher#find` und `Matcher#matches`!
- Quantifizierung ist möglich, aber **greedy** (Default)

Zum Nachlesen

Zum Thema Regular Expressions können Sie in den Tutorials [“Lesson: Regular Expressions” \(Oracle\)](#) und [“Regular Expressions” \(Oracle\)](#) nachlesen.

Lernziele

- k1: Ich kenne die wichtigsten Methoden von `java.util.regex.Pattern` und `java.util.regex.Matcher`
- k2: Ich kann den Unterschied zwischen `Matcher#find` und `Matcher#matches` erklären
- k2: Ich kann zwischen greedy und non-greedy Verhalten bei regulären Ausdrücken unterscheiden
- k3: Ich kann einfache reguläre Ausdrücke bilden
- k3: Ich kann Zeichenklassen und deren Negation einsetzen
- k3: Ich kann die vordefinierten regulären Ausdrücke einsetzen
- k3: Ich kann Quantifizierer gezielt einsetzen
- k3: Ich kann komplexe Ausdrücke (u.a. mit Gruppen) konstruieren

Challenges

Schreiben Sie eine Methode, die mit Hilfe von regulären Ausdrücken überprüft, ob der eingegebene String eine nach dem folgenden Schema gebildete EMail-Adresse ist:

`name@firma.domain`

Dabei sollen folgende Regeln gelten:

- Die Bestandteile `name` und `firma` können aus Buchstaben, Ziffern, Unter- und Bindestrichen bestehen.
- Der Bestandteil `name` muss mindestens ein Zeichen lang sein.
- Der Bestandteil `firma` kann entfallen, dann entfällt auch der nachfolgende Punkt (.) und der Teil `domain` folgt direkt auf das @-Zeichen.
- Der Bestandteil `domain` besteht aus 2 oder 3 Kleinbuchstaben.

Hinweis: Sie dürfen keinen Oder-Operator verwenden.

8.2. Generics1: Generische Klassen & Methoden

TL;DR

Generische Klassen und Methoden sind ein wichtiger Baustein in der Programmierung mit Java. Dabei werden Typ-Variablen eingeführt, die dann bei der Instantiierung der generischen Klassen oder beim Aufruf von generischen Methoden mit existierenden Typen konkretisiert werden ("Typ-Parameter").

Syntaktisch definiert man die Typ-Variablen in spitzen Klammern hinter dem Klassennamen bzw. vor dem Rückgabetyt einer Methode: `public class Stack<E> { }` und `public <T> T foo(T m){ }`.

Generics = Compile-Time-Typsicherheit

Videos

Vorlesung [YT], [HSBI]

Demo Generische Methoden [YT], [HSBI]

8.2.1. Generische Strukturen

8.2.1.1. Naive Verwendung des Vectors als "raw type"

```
1 Vector speicher = new Vector();
2 speicher.add(1); speicher.add(2); speicher.add(3);
3 speicher.add("huhu"); // Laufzeitfehler
4
5 int summe = 0;
6 for (Object i : speicher) { summe += (Integer)i; }
```

Hier nutzen wir den sogenannten **Raw Type Vector**: Die Klasse ist eigentlich generisch (`Vector<E>`), wir geben aber kein Typargument an. Dadurch gehen die Informationen über den Elementtyp verloren. Effektiv werden Elemente beim Auslesen als `Object` behandelt (und es entstehen Compiler-Warnungen).

Important

Problem: Nutzung des "raw" Typs `Vector` ist nicht typsicher!

- Mögliche Fehler fallen erst zur Laufzeit und damit u.U. erst sehr spät auf: Offenbar werden im obigen Beispiel `int`-Werte erwartet, d.h. das Hinzufügen von `"huhu"` ist vermutlich ein Versehen (wird vom Compiler aber nicht bemerkt)
- Die Iteration über `speicher` kann nur allgemein als `Object` erfolgen, d.h. in der Schleife muss auf den vermuteten/gewünschten Typ gecastet werden: Hier würde dann der String `"huhu"` Probleme zur Laufzeit machen

Tip

`Vector` ist streng genommen historisch ("Legacy"). Die Klasse ist für Multithreading/Nebenläufigkeit ausgelegt (Methoden sind synchronisiert) und verursacht dadurch in klassischen nicht nebenläufigen

Programmen einen unnötigen Synchronisations-Overhead. Nutzen Sie für neuen, nicht nebenläufigen Code meist lieber `ArrayList`.

8.2.1.2. Korrekte Nutzung des Vectors als generischen Typ

```
1 Vector<Integer> speicher = new Vector<Integer>();
2 speicher.add(1); speicher.add(2); speicher.add(3);
3 speicher.add("huhu"); // Compilerfehler
4
5 int summe = 0;
6 for (Integer i : speicher) { summe += i; }
```

Vorteile beim Einsatz von Generics:

- Datenstrukturen/Algorithmen nur einmal implementieren, aber für unterschiedliche Typen nutzen
- Keine Vererbungshierarchie nötig
- Nutzung ist typsicher, Casting unnötig
- Geht nur für Referenztypen
 - Für primitive Typen gibt es Wrapper (`Integer`, `Double`, ...) und Auto-Boxing/-Unboxing
- Beispiel: Collections-API

8.2.2. Generische Klassen/Interfaces definieren

- **Definition:** “<Typ>” hinter Klassennamen

```
1 public class Stack<E> {
2     public E push(E item) {
3         ...
4         return item;
5     }
6 }
```

- `Stack<E>` => Generische (parametrisierte) Klasse (auch: “*generischer Typ*”)
- `E` => Formaler Typ-Parameter (auch: “*Typ-Variable*”) => Platzhalter in der Definition (hier `E`)

- **Einsatz:**

```
1 Stack<Integer> stack = new Stack<Integer>();
```

- `Integer` => Typ-Parameter => Typ-Argument/konkreter Typ beim Nutzen (hier `String`)
- `Stack<Integer>` => Parametrisierter Typ

8.2.3. Generische Klassen instantiieren

- Typ-Parameter in spitzen Klammern hinter Klasse bzw. Interface

```
1 ArrayList<Integer> iL = new ArrayList<Integer>();
2 ArrayList<Double> dL = new ArrayList<Double>();
```

8.2.4. Beispiel I: Einfache generische Klassen

```
1 class Tutor<T> {
2     // T kann in Tutor *fast* wie Klassenname verwendet werden
3     private T x;
4     public T foo(T t) { ... }
5 }
```

```
1 Tutor<String> a = new Tutor<String>();
2 Tutor<Integer> b = new Tutor<>(); // ab Java7: "Diamond Operator"
3
4 a.foo("wuppie");
5 b.foo(1);
6 b.foo("huhu"); // Fehlermeldung vom Compiler
```

Beispiel: [classes.GenericClasses](#)

8.2.4.1. Typ-Inferenz

Typ-Parameter kann bei `new()` auf der rechten Seite oft weggelassen werden => **Typ-Inferenz**

```
1 Tutor<String> x = new Tutor<>(); // <>: "Diamantoperator"
```

(gilt seit Java 1.7)

8.2.5. Beispiel II: Vererbung mit Typparametern

```
1 interface Fach<T1, T2> {
2     public void machWas(T1 a, T2 b);
3 }
4
5 class SHK<T> extends Tutor<T> { ... }
6
7 class PM<X, Y, Z> implements Fach<X, Z> {
8     public void machWas(X a, Z b) { ... }
9     public Y getBla() { ... }
10 }
11
12 class Studi<A,B> extends Person { ... }
13 class MyProperties extends Hashtable<Object, Object> { ... }
```

Auch Interfaces und abstrakte Klassen können parametrisierbar sein.

Bei der Vererbung sind alle Varianten bzgl. der Typ-Variablen denkbar. Zu beachten ist dabei vor allem, dass die Typ-Variablen der Oberklasse (gilt analog für Interfaces) entweder durch Typ-Variablen der Unterklasse oder durch konkrete Typen spezifiziert sind. Die Typ-Variablen der Oberklasse dürfen nicht “in der Luft hängen” (siehe auch nächste Folie)!

8.2.6. Beispiel III: Überschreiben/Überladen von Methoden

```
1 class Mensch { ... }
2
3 class Studi<T extends Mensch> {
4     public void f(T t) { ... }
5 }
6
7 class Tutor extends Studi<Mensch> {
8     public void f(Mensch t) { ... }    // Ueberschreiben
9     public void f(Tutor t) { ... }    // Ueberladen
10 }
11
12 class Prof<T> extends Mensch { ... }
```

Überschreiben bedeutet, dass eine geerbte Methode in der Unterklasse neu definiert wird. In `Studi` gibt es die Methode `public void f(T t)`, die `Tutor` erbt. Da `Tutor extends Studi<Mensch>` gilt, wird `T` zu `Mensch` konkretisiert: `Tutor` erbt damit eine Methode mit der Signatur `public void f(Mensch t)` und definiert diese anschließend neu (überschreiben).

Überladen bedeutet, eine Methode mit gleichem Namen zusätzlich mit anderer Parameterliste zu definieren (hier: `public void f(Tutor t)`).

Überschreiben erfordert gleiche Parameterliste (nach Substitution) und kompatiblen Rückgabetyt; Überladen unterscheidet sich in der Parameterliste.

8.2.7. Vorsicht: So geht es nicht!

```
1 class Foo<T> extends T { ... }
2
3 class Fluppie<T> extends Wuppie<S> { ... }
```

- Generische Klasse `Foo<T>` kann nicht selbst vom Typ-Parameter `T` ableiten (warum?)
- Bei Ableiten von generischer Klasse `Wuppie<S>` muss deren Typ-Parameter `S` bestimmt sein: etwa durch den Typ-Parameter der ableitenden Klasse, beispielsweise `Fluppie<S>` (statt `Fluppie<T>`)

8.2.8. Generische Methoden definieren

- “<Typ>” vor Rückgabetyt

```

1 public class Mensch {
2     public <T> T myst(T m, T n) {
3         return Math.random() > 0.5 ? m : n;
4     }
5 }

```

- “Mischen possible”:

```

1 public class Mensch<E> {
2     public <T> T myst(T m, T n) { ... }
3     public String myst(String m, String n) { ... }
4 }

```

8.2.9. Aufruf generischer Methoden

8.2.9.1. Aufruf

- Aufruf mit Typ-Parameter vor Methodennamen, oder
- Inferenz durch Compiler

8.2.9.2. Finden der richtigen Methode durch den Compiler

1. Zuerst Suche nach exakt passender Methode,
2. danach passend mit Konvertierungen => Compiler sucht gemeinsame Oberklasse (“least upper bound”) in Typhierarchie

8.2.9.3. Beispiel

```

1 class Mensch {
2     <T> T myst(T m, T n) { ... }
3 }
4 Mensch m = new Mensch();
5
6
7 m.<String>myst("Essen", "lecker"); // Angabe Typ-Parameter
8
9
10 m.myst("Essen", 1); // String, Integer => T: Object
11 m.myst("Essen", "lecker"); // String, String => T: String
12 m.myst(1.0, 1); // Double, Integer => T: Number

```

[Beispiel methods.GenericMethods](#)

Reihenfolge der Suche nach passender Methode gilt auch für nicht-generisch überladene Methoden

```

1 class Mensch {
2     public <T> T myst(T m, T n) {

```

```

3      System.out.println("X#myst: T");
4      return m;
5  }
6
7      // NICHT gleichzeitig erlaubt wg. Typ-Löschung (siehe spätere Sitzung):
8  /*
9      public <T1, T2> T1 myst(T1 m, T2 n) {
10         System.out.println("X#myst: T");
11         return m;
12     }
13  */
14
15     public String myst(String m, String n) {
16         System.out.println("X#myst: String");
17         return m;
18     }
19
20     public int myst(int m, int n) {
21         System.out.println("X#myst: int");
22         return m;
23     }
24 }
25
26
27 public class GenericMethods {
28     public static void main(String[] args) {
29         Mensch m = new Mensch();
30
31         m.myst("Hello World", "m");
32         m.myst("Hello World", 1);
33         m.myst(3, 4);
34         m.myst(m, m);
35         m.<Mensch>myst(m, m);
36         m.myst(m, 1);
37         m.myst(3.0, 4);
38         m.<Double>myst(3, 4);
39     }
40 }

```

8.2.10. Wrap-Up

Generics = Compile-Time-Typsicherheit

- Begriffe:
 - Generischer Typ: `Stack<T>`
 - Formaler Typ-Parameter: `T`
 - Parametrisierter Typ: `Stack<Long>`
 - Typ-Parameter: `Long`
 - Raw Type: `Stack`
- Generische Klassen: `public class Stack<E> { }`

- “<Typ>” hinter Klassennamen
- Generische Methoden: `public <T> T foo(T m) { }`
 - “<Typ>” vor Rückgabewert

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials “[Defining Simple Generics](#)” und “[Generic Methods](#)” sowie in den dev.java-Tutorials “[Introducing Generics](#)” und “[Type Inference](#)” nach.

Lernziele

- k1: Ich kenne die Begriffe ‘generischer Typ’, ‘parametrisierter Typ’, ‘formaler Typ-Parameter’, ‘Typ-Parameter’
- k3: Ich kann generische Klassen und Interfaces definieren und praktisch einsetzen
- k3: Ich kann generische Methoden definieren und praktisch einsetzen

8.3. Generics2: Bounds & Wildcards

TL;DR

Typ-Parameter können durch **Bounds** eingeschränkt werden: `<T extends ...>` bedeutet, dass der Typ-Parameter `T` nach oben eingeschränkt wird (“upper bound”). Durch **extends**-Bounds des Typ-Parameters können in der generischen Klasse/Methode dann alle Methoden der Schranke (Bound) verwendet werden. Ein **Wildcard** (?) steht für einen unbestimmten Typ. Ein Wildcard-Typ hat keinen Namen / ist nicht benennbar und ist innerhalb der Klasse/Methode nicht direkt zugreifbar. Wildcards können mit `? extends ...` nach oben (“upper bound”) oder `? super ...` nach unten (“lower bound”) eingeschränkt werden. Bei der Nutzung mit `? extends Bound` muss der konkrete Typ die Schranke selbst oder ein Subtyp davon sein. Analog muss bei der Nutzung mit `? super Bound` der konkrete Typ die Schranke selbst oder ein Supertyp (Obertyp) der angegebenen Schranke sein.

Videos

Vorlesung [[YT](#)], [[HSBI](#)]

Demo Wildcards [[YT](#)], [[HSBI](#)]

8.3.1. Bounds: Einschränken der generischen Typen

```
1 public class Cps<E extends Number> {
2     // Obere Schranke: E muss Number oder Subklasse sein
3     // => Zugriff auf Methoden aus Number möglich
4 }
5 Cps<Double> a;
6 Cps<Number> b;
7 Cps<String> c; // Fehler!!!
```

- Schlüsselwort **extends** gilt hier auch für Interfaces
- Mehrere Interfaces: nach **extends** eine Klasse oder ein Interface, danach mit "&" getrennt die restlichen Interfaces:

```
1 class Cps<E extends KlasseOderInterface & I1 & I2 & I3> {}
```

Falls eine Klasse mehreren Obertypen folgen soll, können mehrere Bound-Typen durch & verbunden werden. Der erste Bound kann eine Klasse (z.B. **Number**) sein; alle weiteren Bound-Typen müssen Interfaces sein. Wenn kein Klassen-Bound existiert, können alle Bound-Typen Interfaces sein.

Tip

Der Typ-Parameter ist **nur** mit **extends** nach oben beschränkbar. Für die Wildcards gibt es zusätzlich auch noch die Beschränkung mit **super** nach unten.

[Beispiel bounds.Cps](#)

8.3.2. Wildcards: Dieser Typ ist mir nicht so wichtig

Wildcard mit "?" => steht für unbestimmten Typ

```
1 public class Wuppie {
2     public void m1(List<?> a) { ... }
3     public void m2(List<? extends Number> b) { ... }
4 }
```

- **m1**: **List** beliebig parametrisierbar
=> In **m1** für Objekte in Liste **a** nur Methoden von **Object** nutzbar!
- **m2**: **List** muss mit **Number** oder Subklasse parametrisiert werden.
=> Dadurch für Objekte in Liste **b** alle Methoden von **Number** nutzbar ...

Die Wildcard ? steht für einen unbekannten Typ. **List<?>** erlaubt **List**-Objekte jedes Typs, aber innerhalb der Methode kann man nicht sicher auf spezifische Eigenschaften des konkreten Typs zugreifen. **Wichtig**: **List<?>** ist also **nicht** eine "Liste von **Object**", sondern eine Liste von "unbekanntem Typ".

- Typvariable: "ich benenne den Typ und kann ihn mehrfach verwenden"
- Wildcard: "ich akzeptiere etwas Unbekanntes, kann es aber nicht benennen"

Mit **List<? extends A>** erlaubt man Listen von Elementen, die **A** oder eine Unterklasse von **A** sind (*kovariant*, siehe auch Diskussion in "Generics3: Generics und Polymorphie"); man kann Elemente als **A** lesen/nutzen, aber nicht sicher als **A** der Liste hinzufügen (weil der echte Typ wg. des Wildcards unbekannt ist - es könnte ein beliebiger Untertyp von **A** sein).

Weitere Eigenschaften:

- Durch Wildcard kein Zugriff auf den Typ

- Wildcard kann durch upper bound oder lower bound eingeschränkt werden
- Geht nicht bei Klassen-/Interface-Definitionen, hier wird eine Typ-Variable benötigt

Weitere Beispiele:

- `List<?>` - Typ der Listenelemente unbekannt, nur Methoden von `Object` nutzbar
- `List<? extends T>` - Typ der Listenelemente ist `T` oder eine Unterklasse von `T`; Zugriff lesend mit den Methoden von `T` (außer `null`), Schreiben nur eingeschränkt möglich (konkreter Typ unklar wg. `?` - es könnte eine Unterklasse von `T` sein, und Schreiben von Elementen vom Typ `T` würde (wenn es erlaubt wäre) dann zur Laufzeit schief gehen - Java fängt das aber zur Compilezeit ab)
- `List<? super T>` - Typ der Listenelemente ist `T` oder eine Oberklasse von `T`; Zugriff schreibend möglich mit Werten vom Typ `T` und Untertypen, Lesen nur mit `Object` (der konkrete Typ samt Schnittstelle ist unklar: es könnte eine beliebige Oberklasse von `T` sein, die eine völlig unterschiedliche Schnittstelle als `T` hat)

=> Das soll uns als erste Einführung von Bounds und Wildcards reichen. Wir werden überwiegend die **extends**-Bounds verwenden. Für eine genauere Diskussion von “Type Erasure” (TE) und “Producer extends, Consumer super” (PECS-Regel) sowie die Varianz-Diskussion siehe Lektion “Generics3: Generics und Polymorphie”.

Bloch (2018): Wildcards **meist** für Parameter; Rückgabewerte **möglichst** konkret typisieren.

Generische Typen in Rückgabewerten sollten möglichst konkrete Typen oder Bounds verwenden, um Typ-Sicherheit zu wahren.

8.3.3. Hands-On: Ausgabe für generische Listen

Ausgabe für Listen gesucht, die sowohl Elemente der Klasse `A` als auch Elemente der Klasse `B` enthalten können

```

1  class A { void printInfo() { System.out.println("A"); } }
2  class B extends A { void printInfo() { System.out.println("B"); } }
3
4  public class X {
5      public static void main(String[] args) {
6          List<A> x = new ArrayList<A>();
7          x.add(new A()); x.add(new B());
8          printInfo(x);    // Klassenmethode in X, gesucht
9          List<B> y = new ArrayList<B>();
10         y.add(new B()); y.add(new B());
11         printInfo(y);    // Klassenmethode in X, gesucht
12     }
13 }
```

Hinweis: Dieses Beispiel berührt auch Polymorphie bei/mit generischen Datentypen, vgl. dritter Teil “Generics und Polymorphie” anschauen.

8.3.3.1. Erster Versuch (A und B und main() wie oben)

```
1 public class X {  
2     public static void printInfo(List<A> list) {  
3         for (A a : list) { a.printInfo(); }  
4     }  
5 }
```

=> **So geht's nicht!** Eine `List<B>` ist **keine** `List<A>` (auch wenn ein `B` ein `A` ist, vgl. spätere Sitzung zu Generics und Vererbung ...)!

[Beispiel wildcards.v1.X](#)

8.3.3.2. Zweiter Versuch mit Wildcards (A und B und main() wie oben)

```
1 public class X {  
2     public static void printInfo(List<?> list) {  
3         for (Object a : list) { a.printInfo(); }  
4     }  
5 }
```

=> **So gehts auch nicht!** Im Prinzip passt das jetzt für `List<A>` und `List<B>`. Dummerweise hat man durch das Wildcard keinen Zugriff mehr auf den Typ-Parameter und muss für den Typ der Laufvariablen in der **for**-Schleife dann `Object` nehmen. Aber `Object` kennt unser `printInfo` nicht ... Außerdem könnte man die Methode `X#printInfo` dank des Wildcards auch mit allen anderen Typen aufrufen ...

[Beispiel wildcards.v2.X](#)

8.3.3.3. Dritter Versuch (Lösung) mit Wildcards und Bounds (A und B und main() wie oben)

```
1 public class X {  
2     public static void printInfo(List<? extends A> list) {  
3         for (A a : list) { a.printInfo(); }  
4     }  
5 }
```

Das ist die Lösung. Man erlaubt als Argument nur `List`-Objekte und fordert, dass sie mit `A` oder einer Unterklasse von `A` parametrisiert sind. D.h. in der Schleife kann man sich auf den gemeinsamen Obertyp `A` abstützen und hat dann auch wieder die `printInfo`-Methode (von `A`) zur Verfügung ...

[Konsole wildcards.v3.X](#)

8.3.4. Wrap-Up

- Ein Wildcard (?) als Typ-Parameter steht für einen beliebigen Typ
 - Ist in Klasse oder Methode dann aber nicht mehr zugreifbar

- Mit Bounds kann man Typ-Parameter/Wildcards nach oben oder nach unten einschränken (im Sinne einer Vererbungshierarchie)
 - **extends**: Der Typ-Parameter bzw. die Wildcard muss eine Unterklasse eines bestimmten Typen sein
 - **super**: Die Wildcard muss eine Oberklasse eines bestimmten Typen sein

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den Oracle-Tutorials [“Wildcards”](#) und [“More Fun with Wildcards”](#) sowie im dev.java-Tutorial [“Wildcards”](#) nach.

Lernziele

- k3: Ich kann Wildcards und Bounds bei generischen Klassen/Methoden einsetzen

Challenges

Spieler, Mannschaften und Ligen Modellieren Sie in Java verschiedene Spielertypen sowie generische Mannschaften und Ligen, die jeweils bestimmte Spieler (-typen) bzw. Mannschaften aufnehmen können.

1. Implementieren Sie die Klasse `Spieler`, die das Interface `ISpieler` erfüllt.

```
1 public interface ISpieler {  
2     String getName();  
3 }
```

2. Implementieren Sie die beiden Klassen `FussballSpieler` und `BasketballSpieler` und sorgen Sie dafür, dass beide Klassen vom Compiler als Spieler betrachtet werden (geeignete Vererbungshierarchie).
3. Betrachten Sie das nicht-generische Interface `IMannschaft`. Erstellen Sie daraus ein generisches Interface `IMannschaft` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Mannschaften mit von `ISpieler` abgeleiteten Spielern gebildet werden können.

```
1 public interface IMannschaft {  
2     boolean aufnehmen(ISpieler spieler);  
3     boolean rauswerfen(ISpieler spieler);  
4 }
```

4. Betrachten Sie das nicht-generische Interface `ILiga`. Erstellen Sie daraus ein generisches Interface `ILiga` mit einer Typvariablen. Stellen Sie durch geeignete Beschränkung der Typvariablen sicher, dass nur Ligen mit von `IMannschaft` abgeleiteten Mannschaften angelegt werden können.

```
1 public interface ILiga {  
2     boolean aufnehmen(IMannschaft mannschaft);  
3     boolean rauswerfen(IMannschaft mannschaft);  
4 }
```

- Leiten Sie von `ILiga` das **generische** Interface `IBundesLiga` ab. Stellen Sie durch geeignete Formulierung der Typvariablen sicher, dass nur Ligen mit Mannschaften angelegt werden können, deren Spieler vom Typ `FussballSpieler` (oder abgeleitet) sind.

Realisieren Sie nun noch die Funktionalität von `IBundesLiga` als **nicht-generisches** Interface `IBundesLiga2`.

8.4. Generics3: Generics und Polymorphie

TL;DR

Auch mit generischen Klassen stehen die Mechanismen Vererbung und Überladen zur Verfügung. Dabei muss aber beachtet werden, dass generische Klassen sich **“invariant”** verhalten: Der Typ selbst folgt der Vererbungsbeziehung, eine Vererbung des Typ-Parameters begründet *keine* Vererbungsbeziehung! D.h. aus `U <: O` folgt **nicht** `A<U> <: A<O>`.

Bei Arrays ist es genau anders herum: Wenn `U <: O` dann gilt auch `U[] <: O[]` ... (Dies nennt man “kovariantes” Verhalten.)

Generics existieren eigentlich nur auf Quellcode-Ebene. Nach der Typ-Prüfung etc. entfernt der Compiler alle generischen Typ-Parameter und alle `<...>` ($\Rightarrow$ “Type-Erasure”), d.h. im Byte-Code stehen nur noch Raw-Typen bzw. die oberen Typ-Schranken der Typ-Parameter, in der Regel `Object`. Zusätzlich baut der Compiler die nötigen Casts ein. Als Anwender merkt man davon nichts, muss das “Type-Erasure” wegen der Auswirkungen aber auf dem Radar haben!

Videos

Vorlesung [YT], [HSBI]

8.4.1. Motivation: Wie verhalten sich generische Typen zueinander

Wir haben uns schon angeschaut:

- Definition generischer Klassen/Methoden: `class Box<T> { ... }, static <T> void m(T x) { ... }`
- Bounds: `T extends Number`
- Wildcards: `List<? extends Number>`

Die Frage für heute: **Wie verhalten sich generische Typen zueinander?**

```
1 interface Animal {
2     default void eat() {}
3 }
4 record Dog() implements Animal {}
5 record Cat() implements Animal {}
```

Da `Cat <: Animal` gilt: Ist dann auch `List<Cat> <: List<Animal>?`

Tip

Den `<:`-Operator lesen Sie bitte als "Subtyp-Relation", d.h. `A <: B` heißt: "A ist ein Untertyp von B" bzw. "A ist Subtyp von B".

```
1 List<Animal> animals = new ArrayList<Cat>(); // ???
```

8.4.2. Invarianz generischer Klassen in Java

Gedankenexperiment: Angenommen, `List<Animal> animals = new ArrayList<Cat>();` wäre erlaubt. Dann würde das Folgende auch erlaubt sein:

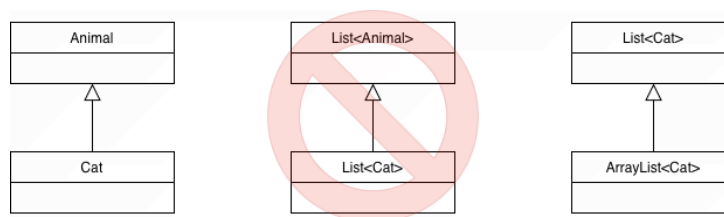
```
1 List<Cat> cats = new ArrayList<>();
2 List<Animal> animals = cats; // hypothetisch erlaubt
3
4 animals.add(new Dog());      // erlaubt, weil animals: List<Animal>
5 Cat c = cats.getFirst();    // aber jetzt steckt ein Dog in cats!
```

Widerspruch zur Typsicherheit**Important**

Java-Generics sind **invariant**

Wenn `A <: B` (A Untertyp von B), dann folgt **nicht** `List<A> <: List<B>`.

=> Polymorphie bei Generics bezieht sich auf den **Klassen-/Interface-Typ** selbst, nicht auf den Typ-Parameter. Bei der Vererbung von generischen Typen muss der Typ-Parameter identisch sein.



- Ungültige Beispiele:

- Es gilt zwar: `Cat <: Animal`, aber **nicht** `List<Cat> <: List<Animal>` (und auch **nicht** `List<Animal> <: List<Cat>`).

- Gültige Beispiele:

- `ArrayList<Cat> <: List<Cat>`
- `HashSet<String> <: Set<String>`
- `B<Double> <: A<Double>` und `B<String> <: A<String>`

```

1 class A<E> { ... }
2 class B<E> extends A<E> { ... }
3
4 A<Double> ad = new B<Double>();
5 A<String> as = new B<String>();

```

- Stack<Double> <: Vector<Double> und Stack<String> <: Vector<String>

```

1 class Vector<E> { ... }
2 class Stack<E> extends Vector<E> { ... }
3
4 Vector<Double> vd = new Stack<Double>();
5 Vector<String> vs = new Stack<String>();

```

8.4.3. Kovarianz mit ? extends (Producer: nur lesen)

Problem:

Wir wollen aus einer (übergebenen) generischen Datenstruktur lesen können: Eine Methode soll alle "Listen von Tieren" akzeptieren, egal ob List<Dog>, List<Cat> oder List<Animal>.

Lösung: ? extends Animal:

```

1 static void feedAll(List<? extends Animal> animals) {
2     for (Animal a : animals) { a.eat(); }
3
4     // animals.add(new Cat()); // Compiler-Fehler
5 }
6
7 List<Cat> cats = ... ; feedAll(cats);

```

Erklärung:

- `animals` ist eine Liste von **irgendetwas**, das **mindestens** ein `Animal` ist: Oberklasse der Listenelemente ist `Animal`
- Wir wissen deshalb: Jedes Element der Liste hat die Schnittstelle von `Animal` → `a.eat()` ist also sicher

=> Diese Liste ist ein **"Producer"** von `Animal`-Objekten, d.h. sie gibt uns `Animal`-Objekte (wir holen nur raus, wir lesen nur).

Darf man auch Elemente der Liste `animals` hinzufügen? NEIN!

- Zur Laufzeit könnte es z.B. eine `List<Dog>` sein
- Dann wäre `animals.add(new Cat())` nicht typsicher

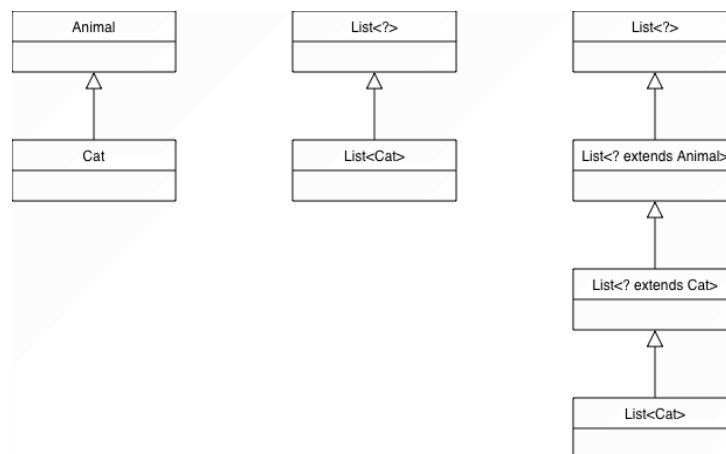
=> Konsequenz: Bei `List<? extends Animal>` ist **lesen sicher, schreiben (add) verboten**.

Important**Kovarianz** durch ? **extends** T:

- “Ich akzeptiere Listen von Untertypen von T”
- Nur sicher als **Producer** von T (lesen erlaubt)
- `List<Cat>` ist **kein** Untertyp von `List<Animal>`
- Aber: `List<Cat> <: List<? extends Animal>`

Damit bildet sich die sogenannte “Kovarianz-Leiter”:

- Unser Tiere-Beispiel: `List<Cat> <: List<? extends Cat> <: List<? extends Animal> <: List<?>`
- `Integer` und `Number` aus dem JDK: `List<Integer> <: List<? extends Integer> <: List<? extends Number> <: List<?>`

**8.4.4. Kontravarianz mit ? super (Consumer: nur schreiben)**

Jetzt die andere Richtung:

Wir wollen eine Methode, die in eine (übergebene) generische Datenstruktur hinzufügen kann: z.B. alle `Cat` in irgendeine Liste stecken, die “`Cat` oder allgemeiner” ist.

```

1  static void addCats(List<? super Cat> cats) {
2      cats.add(new Cat());           // erlaubt
3      // Cat c = cats.getFirst();   // Compiler-Fehler: Rückgabetyp ist Object
4  }
5
6  List<Cat> cats = ... ; addCats(cats);

```

Erklärung:

- `cats` ist eine Liste von `Cat` oder einem Supertyp (`Cat`, `Animal`, `Object`)

- Es ist immer sicher, eine `Cat` hinzuzufügen, denn:
 - Eine Liste von `Cat` darf Cats enthalten
 - Eine Liste von `Animal` darf Cats (als Untertyp) enthalten
 - Eine Liste von `Object` sowieso

Aber:

- Beim Lesen weiß der Compiler nicht, ob wirklich eine `List<Cat>` übergeben wurde - er kennt nur ? **super** `Cat`
- Sicherer gemeinsamer Nenner: `Object`

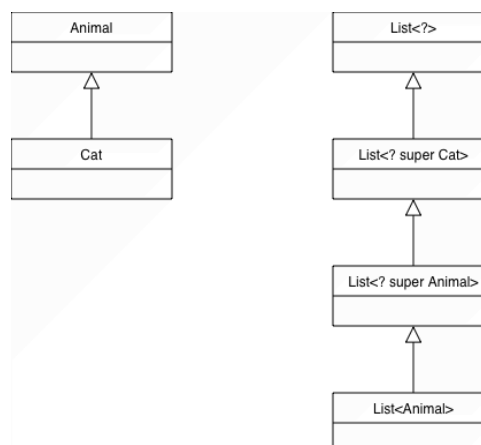
Important

Kontravarianz durch ? **super** `T`:

- “Ich akzeptiere Listen von Supertypen von `T`”
- Nur sicher als **Consumer** von `T` (schreiben/hinzufügen erlaubt)
- ? **super** `Cat` bedeutet: Typ-Argument ist `Cat`, `Animal` oder `Object`
- `List<Animal> <: List<? super Cat>`

Damit bildet sich die sogenannte “Kontravarianz-Leiter”:

- Unser Tiere-Beispiel: `List<Animal> <: List<? super Animal> <: List<? super Cat> <: List<?>`
- `Integer` und `Number` aus dem JDK: `List<Number> <: List<? super Number> <: List<? super Integer> <: List<?>`



8.4.5. PECS: Producer Extends, Consumer Super

Beide Aspekte zusammen betrachtet:

- Wenn ein Parameter nur gelesen werden soll → ? **extends** T
- Wenn ein Parameter nur geschrieben (konsumiert) werden soll → ? **super** T
- Wenn beides passieren soll → konkrete Typvariable, etwa `List<T>`

PECS: Producer Extends, Consumer Super

- **Producer** (liefert Elemente, **kovariant**): `List<? extends Animal>`
- **Consumer** (nimmt Elemente auf, **kontravariant**): `List<? super Cat>`
- Ohne Wildcard `List<Cat>`: Typ ist **invariant**; Lesen und Schreiben für genau diesen Typ

Tip

Die Begriffe können verwirrend sein - wenn man die Begriffe Producer/Consumer als Ergebnis der Methode begreift.

PECS benennt die Rolle des Parameters (der Collection) und nicht die Aktion, die Ihre Methode ausführt.

Hier eine bildliche Eselsbrücke:

```
1 void foo(List<? extends T> xs) { ... } // "extends"-Fall
2 void bar(List<? super T> ys) { ... }   // "super"-Fall
```

Die PECS-Merksätze sagen: "Was kann ich mit diesem *Parameter* **sicher** machen?"

- `List<? extends T> xs`: **Producer** von T
 - Die Liste `xs` "produziert" Objekte vom Typ T
 - "Ich kann aus `xs` Dinge vom Typ T herausbekommen"
 - "Die Liste `xs` gibt Objekte vom Typ T heraus"
- `List<? super T> ys`: **Consumer** von T
 - Die Liste `ys` "konsumiert" Objekte vom Typ T
 - "Ich kann in `ys` Dinge vom Typ T hineinstecken"
 - "Die Liste `ys` nimmt Objekte vom Typ T an"

"Producer/Consumer" meint hier die Liste (bzw. den Parametertyp), nicht die Methode!

8.4.6. Bezug zu funktionalen Interfaces

Beispiele aus der Java-Standard-Bibliothek: (JDK):

- `Function<? super T, ? extends R>`
 - Abbildung von T nach R: `R apply(T)`
 - Input (konsumiert T): ? **super** T
 - Output (produziert R): ? **extends** R

- `Comparator<? super T>`
 - `int compare(T, T)`
 - Ein `Comparator` vergleicht `T`-Objekte, "konsumiert" also `T: ? super T`

8.4.7. Typ-Löschung (*Type Erasure*)

Generics sind ein **Compile-Time-Feature**. Vor der Laufzeit werden sämtliche Typ-Parameter **gelöscht** ("erased").
Zur Laufzeit gibt es keine Generics mehr.

Der Compiler ersetzt nach Prüfung der Typen und ihrer Verwendung alle Typ-Parameter durch

1. deren obere (Typ-)Schranke und
2. passende explizite Cast-Operationen (im Byte-Code).

Die obere Typ-Schranke ist in der Regel der Typ der ersten Bounds-Klausel oder `Object`, wenn keine Einschränkungen formuliert sind.

Bei parametrisierten Typen wie `List<T>` wird der Typ-Parameter entfernt, es entsteht ein sogenannter *Raw-Typ* (`List`, quasi implizit mit `Object` parametrisiert).

=> Ergebnis: Nur **eine** (untypisierte) Klasse! **Zur Laufzeit gibt es keine Generics mehr!**

Tip

In C++ ist man den anderen möglichen Weg gegangen und erzeugt für jede Instantiierung die passende Klasse.

Beispiel: Aus dem folgenden harmlosen Code-Fragment:

```
1 class Studi<T> {
2     T myst(T m, T n) { return n; }
3
4     public static void main(String[] args) {
5         Studi<Integer> a = new Studi<>();
6         int i = a.myst(1, 3);
7     }
8 }
```

wird nach der Typ-Löschung durch Compiler (das steht dann quasi im Byte-Code):

```
1 class Studi {
2     Object myst(Object m, Object n) { return n; }
3
4     public static void main(String[] args) {
5         Studi a = new Studi();
6         int i = (Integer) a.myst(1, 3);
7     }
8 }
```

Die obere Schranke meist `Object` → `new T()` verboten/sinnfrei (s.u.)!

8.4.8. Type-Erasure bei Nutzung von Bounds

vor der Typ-Löschung durch Compiler:

```

1  class Cps<T extends Number> {
2      T myst(T m, T n) {
3          return n;
4      }
5
6      public static void main(String[] args) {
7          Cps<Integer> a = new Cps<>();
8          int i = a.myst(1, 3);
9      }
10 }
```

nach der Typ-Löschung durch Compiler:

```

1  class Cps {
2      Number myst(Number m, Number n) {
3          return n;
4      }
5
6      public static void main(String[] args) {
7          Cps a = new Cps();
8          int i = (Integer) a.myst(1, 3);
9      }
10 }
```

8.4.9. Folgen der Typ-Löschung: *new*

new mit parametrisierten Klassen ist nicht erlaubt!

```

1  class Fach<T> {
2      public T foo() {
3          return new T(); // nicht erlaubt!!!
4      }
5  }
```

Grund: Zur Laufzeit keine Klasseninformationen über **T** mehr

Im Code steht **return (CAST) new Object()**; . Das neue Object kann man anlegen, aber ein Cast nach irgendeinem anderen Typ ist sinnfrei: Jede Klasse ist ein Untertyp von **Object**, aber eben nicht andersherum. Außerdem fehlt dem Objekt vom Typ **Object** auch sämtliche Information und Verhalten, die der Cast-Typ eigentlich mitbringt ...

8.4.10. Folgen der Typ-Löschung: *static*

static mit generischen Typen ist nicht erlaubt!

```

1  class Fach<T> {
2      static T t;                // nicht erlaubt!!!
3      static Fach<T> c;         // nicht erlaubt!!!
4      static void foo(T t) { ... }; // nicht erlaubt!!!
5  }
6
7  Fach<String> a;
8  Fach<Integer> b;

```

Grund: Compiler generiert nur eine Klasse! Beide Objekte würden sich die statischen Attribute teilen (Typ zur Laufzeit unklar!).

Hinweis: Generische (statische) Methoden sind erlaubt. (Warum?)

```

1  class Fach<T> {
2      static <U> Fach<U> createEmpty() { ... } // generische statische Methode
        ist ok
3  }

```

8.4.11. Folgen der Typ-Löschung: *instanceof*

instanceof mit parametrisierten Klassen ist nicht erlaubt!

vor der Typ-Löschung durch Compiler:

```

1  class Fach<T> {
2      void printType(Fach<?> p) {
3          if (p instanceof Fach<Number>)
4              ...
5          else if (p instanceof Fach<String>)
6              ...
7      }
8  }

```

unsinniger Code nach Typ-Löschung:

```

1  class Fach {
2      void printType(Fach p) {
3          if (p instanceof Fach)
4              ...
5          else if (p instanceof Fach)
6              ...
7      }
8  }

```

8.4.12. Folgen der Typ-Löschung: *.class*

.class mit parametrisierten Klassen ist nicht erlaubt!

```

1  boolean x;
2  List<String> a = new ArrayList<String>();
3  List<Integer> b = new ArrayList<Integer>();
4
5  x = (List<String>.class == List<Integer>.class); // Compiler-Fehler
6  x = (a.getClass() == b.getClass());             // true

```

Grund: Es gibt nur `List.class` (und kein `List<String>.class` bzw. `List<Integer>.class`)!

8.4.13. Raw-Types: Ich mag meine Generics “well done” :-)

Raw-Types: Instanziierung ohne Typ-Parameter => `Object`

```

1  Stack s = new Stack(); // Stack von Object-Objekten

```

- Wegen Abwärtskompatibilität zu früheren Java-Versionen noch erlaubt.
- Nutzung wird nicht empfohlen! (Warum?)

Tip

Raw-Types darf man zwar selbst im Quellcode verwenden (so wie im Beispiel hier), **sollte** die Verwendung aber vermeiden wegen der Typ-Unsicherheit: Der Compiler sieht im Beispiel nur noch einen Stack für `Object`, d.h. dort dürfen Objekte aller Typen abgelegt werden - es kann keine Typprüfung durch den Compiler stattfinden. Auf einem `Stack<String>` kann der Compiler prüfen, ob dort wirklich nur `String`-Objekte abgelegt werden und ggf. entsprechend Fehler melden.

Etwas anderes ist es, dass der Compiler im Zuge von Type-Erasure selbst Raw-Types in den Byte-Code schreibt. Da hat er vorher bereits die Typsicherheit geprüft und er baut auch die passenden Casts ein.

Das Thema ist eigentlich nur noch aus Kompatibilität zu Java5 oder früher da, weil es dort noch keine Generics gab (wurden erst mit Java5 eingeführt).

8.4.14. Abgrenzung: Arrays vs. Generics: Reifizierung

Important

Arrays sind **reifiziert** (engl. *reified*): Sie “kennen” ihren Elementtyp **zur Laufzeit**.

```

1  String[] sa = new String[10];
2  Object[] oa = sa;           // erlaubt: Array-Kovarianz
3
4  oa[0] = "Hallo";           // ok
5  oa[0] = new Double(2.0);   // Laufzeitfehler

```

- Arrays besitzen Typinformationen über gespeicherte Elemente
- Prüfung auf Typ-Kompatibilität zur **Laufzeit** (nicht Kompilierzeit!)
- Arrays sind **kovariant**: `String[] <: Object[]` wg. `String <: Object`

Im Vergleich sind generische Typen **nicht reifiziert** (engl. *not reified*) - die Typ-Parameter existieren nur zur **Compile-Time** und werden nach der Prüfung durch den Compiler entfernt. Zur Laufzeit wird aus `List<String>` und `List<Integer>` einfach nur `List` (plus notwendige Casts, vom Compiler nach dem Type-Check automatisch eingefügt).

Tip

Arrays gab es bereits sehr früh, Generics wurden erst viel später nachträglich hinzugefügt (in Java 5). Bei Arrays fand man das Verhalten damals natürlich und pragmatisch (trotz der Laufzeit-Überprüfung). Bei der Einführung von Generics musste man Kompatibilität sicherstellen (alter Code soll auch mit neuen Compilern übersetzt werden können - obwohl im alten Code Raw-Types verwendet werden). Außerdem wollte man von Laufzeit-Prüfung hin zu Compiler-Prüfung. Da würde das von Arrays bekannte Verhalten Probleme machen ...

[Beispiel arrays.X](#)

8.4.15. Generics + Arrays: Warum passt das nicht gut?

Important

=> Keine Arrays mit parametrisierten Klassen!

Arrays sind reifiziert & kovariant

- Arrays kennen ihren Elementtyp zur **Laufzeit**
- Sie sind außerdem kovariant: `Object[] oa = new String[10];` ist erlaubt

Generics sind gelöscht & invariant

- `List<T>` ist zur Laufzeit nur `List` - ohne `<T>`, die Information über `T` ist weg
- Generische Typen sind **invariant** (kein automatischer Upcast)

Kombination führt zu Problemen und ist nicht erlaubt:

```
1 class Box<T> {
2     // T[] arr = new T[10];           // Compiler-Fehler
3 }
4
5 Foo<String>[] x = new Foo<String>[2]; // Compilerfehler
6
7
8 Foo<String[]> y = new Foo<String[]>(); // OK :-)
```

Arrays mit parametrisierten Klassen sind nicht erlaubt! Arrays brauchen zur Laufzeit Typinformationen über den Elementtyp, die aber durch die Typ-Löschung bei generischen Typen entfernt werden.

8.4.16. Diskussion Vererbung vs. Generics

Vererbung:

- *IS-A-Beziehung*: Die abgeleitete Klasse ist wie die Basisklasse und kann überall dort verwendet werden
- Anwendung: Vererbungsbeziehung vorliegend, Eigenschaften weitergeben und verfeinern
- Beispiel: Ein Student *ist eine* Person

Generics:

- Schablone (Template) für viele (unterschiedliche, aber passende) Datentypen
- Anwendung: Identischer Code für unterschiedliche Typen
- Beispiel: Datenstrukturen, Algorithmen generisch realisieren

8.4.17. Wrap-Up

Generics gibt es nur im Compiler, Arrays gibt es auch in der JVM mit vollem Typwissen.

- **Invarianz**: Generische Typen wie `List<T>` sind **invariant**
 - `List<Cat>` ist **kein** Untertyp von `List<Animal>`
 - `List<Animal>` ist **kein** Untertyp von `List<Cat>`
- **Kovarianz** mit ? **extends** `T` → Producer (lesen)
 - Mit `S <: T: List<S> <: List<? extends S> <: List<? extends T> <: List<?>`
- **Kontravarianz** mit ? **super** `T` → Consumer (schreiben)
 - Mit `S <: T: List<T> <: List<? super T> <: List<? super S> <: List<?>`
- **PECS**: *Producer Extends, Consumer Super*
- **Type Erasure**:
 - Generics gelten nur zur Compile-Time; Typ-Parameter werden zur Laufzeit gelöscht
 - Folgen: keine generischen Arrays, eingeschränktes `instanceof`, Bounds als Laufzeit-Ersatz
- Arrays sind **reifiziert** und **kovariant** → Laufzeit-Checks + `ArrayStoreException`

Zum Nachlesen

Lesen Sie zu diesem Thema im dev.java-Tutorial "[Wildcards and Subtyping](#)" nach.

Lernziele

- k2: Ich verstehe, warum `List<Dog>` kein Untertyp von `List<Animal>` ist (Invarianz)
- k2: Ich verstehe, was Type Erasure bedeutet und kann erklären, welche praktischen Folgen das hat
- k3: Ich kann Vererbungsbeziehungen mit generischen Klassen bilden
- k3: Ich kann erklären, wann ? **extends** und wann ? **super** passt (PECS)

- k3: Ich kann mit Arrays und generischen Typen umgehen

Challenges

Aufgabe: Tierlisten “füttern” und “einsammeln”

Basis wie oben:

```

1 interface Animal {
2     default void eat() {}
3 }
4 record Dog() implements Animal {
5     @Override public void eat() { IO.println("Dog eats"); }
6 }
7 record Cat() implements Animal {
8     @Override public void eat() { IO.println("Cat eats"); }
9 }

```

Betrachten Sie nun dazu:

```

1 // (1) Füttert alle Tiere in der übergebenen Liste
2 static void feedAll(List<Animal> animals) { /* ... */ }
3
4 // (2) Fügt einen Dog in die übergebene Liste ein
5 static void addDog(List<Animal> animals) { /* ... */ }
6
7 // (3) Kopiert alle Elemente von source in dest
8 static void copy(List<Animal> source, List<Animal> dest) { /* ... */ }

```

Arbeitsauftrag

1. Für welche konkreten Listentypen sollten diese Methoden aufrufbar sein? `List<Animal>`, `List<Dog>`, `List<Cat>`, `List<Object>`, ...?

Welche der drei Methoden funktionieren wie gewünscht, welche sind zu einschränkend?

2. Passen Sie die Signaturen mit ? **extends** / ? **super** so an, dass sie zu Ihrer Intuition passen: Wo brauchen Sie ? **extends**, wo ? **super**, wo eine konkrete Typvariable?
3. Versuchen Sie jeweils **kurz zu begründen**, warum Ihre Variante typsicher ist.

8.5. Exception-Handling

TL;DR

Man unterscheidet in Java zwischen **Exceptions** und **Errors**. Ein Error ist ein Fehler im System (OS, JVM), von dem man sich nicht wieder erholen kann. Eine Exception ist ein Fehlerfall innerhalb des Programmes, auf den man innerhalb des Programms reagieren kann.

Mit Hilfe von Exceptions lassen sich Fehlerfälle im Programmablauf deklarieren und behandeln. Methoden können/müssen mit dem Keyword **throws** gefolgt vom Namen der Exception deklarieren, dass sie im Fehlerfall diese spezifische (checked) Exception werfen (und nicht selbst behandeln).

Zum Exception-Handling werden die Keywords **try**, **catch** und **finally** verwendet. Dabei wird im **try**-Block der Code geschrieben, der einen potenziellen Fehler wirft. Im **catch**-Block wird das Verhalten implementiert, dass im Fehlerfall ausgeführt werden soll, und im **finally**-Block kann optional Code geschrieben werden, der sowohl im Erfolgs- als auch Fehlerfall ausgeführt wird.

Es wird zwischen **checked** Exceptions und **unchecked** Exceptions unterschieden. Checked Exceptions sind für erwartbare Fehlerfälle gedacht, die nicht vom Programm ausgeschlossen werden können, wie das Fehlen einer Datei, die eingelesen werden soll. Checked Exceptions müssen deklariert oder behandelt werden. Dies wird vom Compiler überprüft.

Unchecked Exceptions werden für Fehler in der Programmlogik verwendet, etwa das Teilen durch 0 oder Index-Fehler. Sie deuten auf fehlerhafte Programmierung, fehlerhafte Logik oder mangelhafte Eingabeprüfung hin. Unchecked Exceptions müssen nicht deklariert oder behandelt werden. Unchecked Exceptions leiten von `RuntimeException` ab.

Als Faustregel gilt: Wenn der Aufrufer sich von einer Exception-Situation erholen kann, sollte man eine checked Exception nutzen. Wenn der Aufrufer vermutlich nichts tun kann, um sich von dem Problem zu erholen, dann sollte man eine unchecked Exception einsetzen.

Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

8.5.1. Was kann schiefgehen?

```
1 public class ReciprocalCalculator {
2     public static double reciprocalFromFile(String fileName) throws IOException
3     {
4         String content = Files.readString(Path.of(fileName));
5         int value = Integer.parseInt(content);
6         return 1.0 / value;
7     }
8     public static void main(String[] args) throws IOException {
9         double result = reciprocalFromFile("zahl.txt");
10        IO.println("Kehrwert: " + result);
11    }
```

Fragen Sie sich: Was passiert, wenn:

- die Datei nicht vorhanden ist?
- wenn die nötigen Leserechte auf der Datei nicht vorhanden sind?
- wenn in der Datei ein Text statt einer Zahl steht?
- wenn in der Datei die Zahl 0 steht?
- `Files.readString(Path.of(fileName))` → `IOException`
- `Integer.parseInt(content)` → `NumberFormatException`
- `return 1.0 / value` → `ArithmeticException`

`IOException` ist eine **checked** Exception, die in `Files.readString` deklariert wird. Folge: Diese Exception

muss entweder gefangen werden oder in der **throws**-Klausel der Methode deklariert werden (würde dann beim Auftreten an den Aufrufer hochgereicht).

`NumberFormatException` und `ArithmeticException` sind **unchecked** Exceptions. Diese können gefangen und behandelt werden, aber das Auftreten beruht i.d.R. auf Logik- oder Programmierfehlern, d.h. man weiss nicht so genau, ob und wo diese auftreten.

8.5.2. Begriffe

• Checked Exceptions

- Müssen deklariert (**throws**) oder gefangen (**try/catch**) werden
- Vom Compiler überprüft
- Beispiele: `IOException`, `FileNotFoundException`

• Unchecked Exceptions

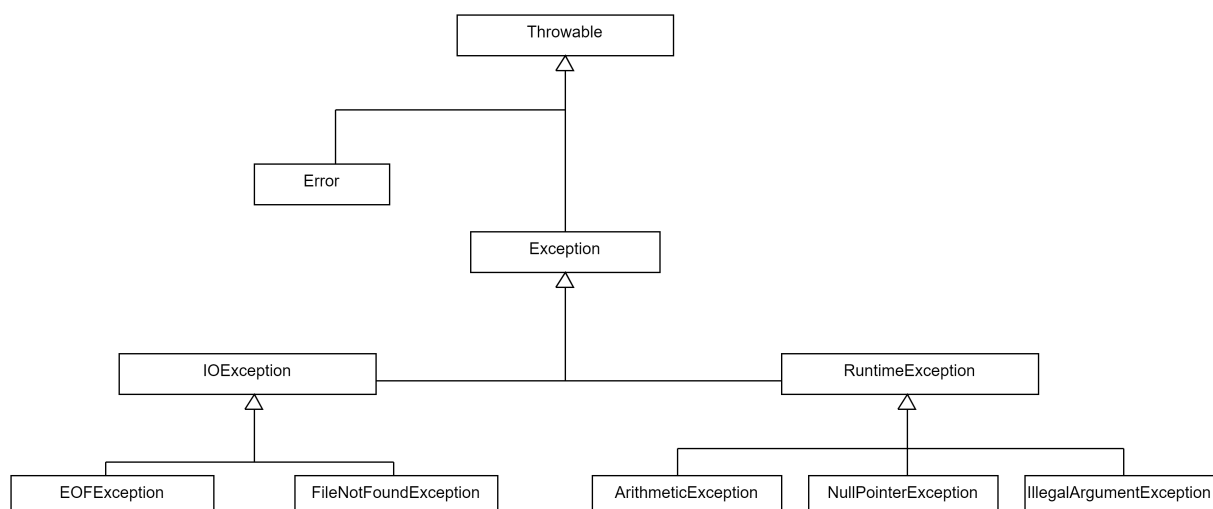
- Unterklassen von `RuntimeException`
- Keine Pflicht zur Deklaration oder zum Fangen
- Beispiele: `NullPointerException`, `IllegalArgumentException`, `ArithmeticException`

Important

Merksatz:

- Checked = *erwartbare* Fehlerquellen (z.B. I/O)
- Unchecked = meist *Programmierfehler* / Logikfehler

8.5.2.1. Throwable und Hierarchie von Exceptions und Errors (Ausschnitt)



8.5.2.2. *Exception vs. Error*

- **Error:**
 - Wird für Systemfehler verwendet (Betriebssystem, JVM, ...)
 - * `StackOverflowError`
 - * `OutOfMemoryError`
 - Von einem Error kann man sich nicht erholen
 - Sollten nicht behandelt werden
- **Exception:**
 - Ausnahmesituationen bei der Abarbeitung eines Programms
 - Können “checked” oder “unchecked” sein
 - Von Exceptions kann man sich erholen

8.5.2.3. *Unchecked vs. Checked Exceptions*

- “Checked” Exceptions:
 - Für erwartbare Fehlerfälle, deren Ursprung nicht im Programm selbst liegt
 - * `FileNotFoundException`
 - * `IOException`
 - Alle nicht von `RuntimeException` ableitende Exceptions
 - Müssen entweder behandelt (**try/catch**) oder deklariert (**throws**) werden: Dies wird vom Compiler überprüft!
- “Unchecked” Exceptions:
 - Logische Programmierfehler (“Versagen” des Programmcodes)
 - * `IndexOutOfBoundsException`
 - * `NullPointerException`
 - * `ArithmeticException`
 - * `IllegalArgumentException`
 - Leiten von `RuntimeException` oder Unterklassen ab
 - Müssen nicht deklariert oder behandelt werden

Beispiele checked Exception:

- Es soll eine Abfrage an eine externe API geschickt werden. Diese ist aber aktuell nicht zu erreichen. “Erholung”: Anfrage noch einmal schicken.
- Es soll eine Datei geöffnet werden. Diese ist aber nicht unter dem angegebenen Pfad zu finden oder die Berechtigungen stimmen nicht. “Erholung”: Aufrufer öffnet neuen File-Picker, um es noch einmal mit einer anderen Datei zu versuchen.

Beispiele unchecked Exception:

- Eine **for**-Loop über ein Array ist falsch programmiert und will auf einen Index im Array zugreifen, der nicht existiert. Hier kann der Aufrufer nicht Sinnvolles tun, um sich von dieser Situation zu erholen.
- Argumente oder Rückgabewerte einer Methode können **null** sein. Wenn man das nicht prüft, sondern einfach Methoden auf dem vermeintlichen Objekt aufruft, wird eine **NullPointerException** ausgelöst, die eine Unterklasse von **RuntimeException** ist und damit eine unchecked Exception. Auch hier handelt es sich um einen Fehler in der Programmlogik, von dem sich der Aufrufer nicht sinnvoll erholen kann.

8.5.3. Fangen und Behandeln von Exceptions mit *Try-Catch* oder *Throws*

8.5.3.1. Variante A: Exception behandeln

```
1 public static void main(String[] args) {
2     try {
3         double result = reciprocalFromFile("zahl.txt");
4         IO.println("Kehrwert: " + result);
5     } catch (IOException e) {
6         System.err.println("Fehler beim Lesen der Datei: " + e.getMessage());
7     }
8 }
```

8.5.3.2. Variante B: Exception weiterreichen

```
1 public static void main(String[] args) throws IOException {
2     double result = reciprocalFromFile("zahl.txt");
3     IO.println("Kehrwert: " + result);
4 }
```

- Im **try** Block wird der Code ausgeführt, der einen Fehler werfen könnte.
- Mit **catch** kann eine Exception gefangen und im **catch** Block behandelt werden.
- Wenn man die *checked* **IOException**-Exception nicht fangen/behandeln möchte, muss die Methode entsprechend gekennzeichnet werden.

Important

Das bloße Ausgeben des Stacktrace via `e.printStackTrace()` ist noch **kein sinnvolles Exception-Handling**! Hier sollte auf die jeweilige Situation eingegangen werden und versucht werden, den Fehler zu beheben oder dem Aufrufer geeignet zu melden!

8.5.4. Try und mehrstufiges Catch

```
1 public static void main(String[] args) {
2     try {
3         double result = reciprocalFromFile("zahl.txt");
4         IO.println("Kehrwert: " + result);
5     } catch (FileNotFoundException e) {
```

```
6      System.err.println("Dateifehler: " + e.getMessage());
7  } catch (NumberFormatException e) {
8      System.err.println("Die Datei enthält keine gültige ganze Zahl.");
9  } catch (ArithmeticException e) {
10     System.err.println("Division durch 0 ist nicht erlaubt.");
11 }
12 }
```

Eine im **try**-Block auftretende Exception wird der Reihe nach mit den **catch**-Blöcken gematcht (vergleichbar mit **switch case**).

Important

Dabei muss die Vererbungshierarchie beachtet werden. Die spezialisierteste Klasse muss ganz oben stehen, die allgemeinste Klasse als letztes. Sonst wird eine Exception u.U. zu früh in einem nicht dafür gedachten **catch**-Zweig aufgefangen.

Important

Wenn eine Exception nicht durch die **catch**-Zweige aufgefangen wird, dann wird sie an den Aufrufer weiter geleitet. Im Beispiel würde eine **IOException** nicht durch die **catch**-Zweige gefangen (**NumberFormatException** und **ArithmeticException** sind im falschen Vererbungsweig, und **FileNotFoundException** ist spezieller als **IOException**) und entsprechend an den Aufrufer weiter gereicht. Da es sich obendrein um eine checked Exception handelt, müsste man diese per **throws IOException** an der Methode deklarieren.

Tip

Nur **IOException** ist *checked*. **NumberFormatException** und **ArithmeticException** sind *unchecked* - Fangen ist optional, aber oft sinnvoll.

8.5.5. Finally und Aufräumen

```
1  BufferedReader reader = null;
2  try {
3      reader = new BufferedReader(new FileReader(fileName));
4      return reader.readLine();
5  } finally {
6      if (reader != null) reader.close();
7  }
```

Der **finally** Block wird sowohl im Fehlerfall als auch im Normalfall aufgerufen. Dies wird beispielsweise für Aufräumarbeiten genutzt, etwa zum Schließen von Verbindungen oder Input-Streams.

Tip

finally wird auch dann ausgeführt, wenn im **try** ein **return** steht.

8.5.6. Freigeben von Ressourcen: Try-with-Resources

8.5.6.1. Problem bisher

Mit **try-finally** müssen wir Ressourcen **manuell** schließen:

```
1  BufferedReader reader = null;
2  try {
3      reader = new BufferedReader(new FileReader(fileName));
4      return reader.readLine();
5  } finally {
6      if (reader != null) reader.close();
7  }
```

8.5.6.2. Lösung: try-with-resources

```
1  try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
2      return reader.readLine();
3  } // reader wird hier automatisch geschlossen
```

Im **try**-Statement können ein oder mehrere Ressourcen deklariert werden, die am Ende sicher geschlossen werden. Diese Ressourcen müssen `java.io.Closeable` oder `java.lang.AutoCloseable` implementieren.

Java schließt die Ressourcen **automatisch** am Ende des Blocks, egal ob eine Exception geworfen wurde oder nicht. Vorteil: Weniger Boilerplate, weniger Fehlerquellen (z.B. vergessenes `close()` o.ä.).

Tip

Seit Java 9 kann man auch *bereits deklarierte* (und geöffnete) Ressourcen im **try**-Kopf referenzieren und damit automatisch schließen lassen.

8.5.7. Werfen von Exceptions mit Throws

Man auch selbst aktiv Exceptions “werfen”:

```
1  int div(int a, int b) throws IllegalArgumentException {
2      if (b == 0) throw new IllegalArgumentException("Can't divide by zero");
3      return a / b;
4  }
```

Wenn man aktiv eine neue Exception werfen will, erzeugt man mit **new** ein neues Objekt der gewünschten Exception und “wirft” diese mit **throw**. Auch diese selbst geworfene Exception kann man dann entweder selbst fangen und bearbeiten oder an den Aufrufer weiterleiten und dies dann entsprechend über die **throws**-Klausel deklarieren: nicht gefangene checked Exceptions *müssen* deklariert werden, nicht gefangene unchecked Exceptions *können* deklariert werden.

Wenn mehrere Exceptions an den Aufrufer weitergeleitet werden, werden sie in der **throws**-Klausel mit Komma getrennt: **throws** Exception1, Exception2, Exception3.

8.5.8. Anti-Beispiele und Best Practices

8.5.8.1. Anti-Beispiel 1: Leerer catch-Block

```
1 try {
2     double result = reciprocalFromFile("zahl.txt");
3 } catch (IOException e) {
4     // nichts... oder e.printStackTrace()
5 }
```

Problem: Fehler wird “verschluckt” → Debugging-Hölle. Der Stacktrace (wird gern von IDEs automatisch eingefügt) ist auch nur bedingt hilfreich.

8.5.8.2. Anti-Beispiel 2: Zu allgemein fangen

```
1 try {
2     double result = reciprocalFromFile("zahl.txt");
3 } catch (Exception e) {
4     System.err.println("Irgendetwas ist schiefgegangen.");
5 }
```

Problem: Fängt auch Programmierfehler (z.B. `NullPointerException`), die Sie gar nicht “heilen” sollten. Besser: So spezifisch wie möglich fangen.

8.5.8.3. Best Practices: Fehlerbilder bewusst aufgreifen

Wenn Sie im Code `catch (Exception e)` sehen, sollte bei Ihnen ein Alarm losgehen. Damit fangen Sie zwar “alles”, aber Sie können nicht mehr gezielt und spezifisch auf einen bestimmten Fehler reagieren. Außerdem verdecken Sie Fehler, an die Sie vielleicht gar nicht gedacht haben.

1. Fangen Sie immer möglichst präzise die Fehler, die Sie erwarten.
2. Behandeln Sie die Fehler möglichst früh (sofern das möglich ist). Wenn eine Behandlung nicht möglich/sinnvoll ist, reichen Sie die Exception “hoch” an den Aufrufer.
3. Wenn Sie Exceptions nicht fangen oder selbst welche werfen, deklarieren Sie das möglichst (auch für unchecked Exceptions) an Ihrer Methode.

8.5.9. Stilfrage A: Wann checked, wann unchecked

8.5.9.1. “Checked” Exceptions

- Für erwartbare Fehlerfälle, deren Ursprung nicht im Programm selbst liegt
- Aufrufer kann sich von der Exception erholen

8.5.9.2. “Unchecked” Exceptions

- Logische Programmierfehler (“Versagen” des Programmcodes)
- Aufrufer kann sich von der Exception vermutlich nicht erholen

Vergleiche [“Unchecked Exceptions — The Controversy”](#).

8.5.10. Vertiefung: Stilfrage B: Wie viel Code im Try?

```
1 int getFirstLineAsInt(String pathToFile) {
2     FileReader fileReader = new FileReader(pathToFile);
3     BufferedReader bufferedReader = new BufferedReader(fileReader);
4     String firstLine = bufferedReader.readLine();
5
6     return Integer.parseInt(firstLine);
7 }
```

[Demo: exceptions.HowMuchTry](#)

Hier lassen sich verschiedene “Ausbaustufen” unterscheiden.

8.5.10.1. 1. Handling an den Aufrufer übergeben

```
1 int getFirstLineAsIntV1(String pathToFile) throws FileNotFoundException,
2     IOException {
3     FileReader fileReader = new FileReader(pathToFile);
4     BufferedReader bufferedReader = new BufferedReader(fileReader);
5     String firstLine = bufferedReader.readLine();
6
7     return Integer.parseInt(firstLine);
8 }
```

Der Aufrufer hat den Pfad als String übergeben und ist vermutlich in der Lage, auf Probleme mit dem Pfad sinnvoll zu reagieren. Also könnte man in der Methode selbst auf ein **try/catch** verzichten und stattdessen die `FileNotFoundException` (vom `FileReader`) und die `IOException` (vom `bufferedReader.readLine()`) per **throws** deklarieren.

Anmerkung: Da `FileNotFoundException` eine Spezialisierung von `IOException` ist, reicht es aus, lediglich die `IOException` zu deklarieren.

8.5.10.2. 2. Jede Exception einzeln fangen und bearbeiten

```
1 int getFirstLineAsIntV2(String pathToFile) {
2     FileReader fileReader = null;
3     try {
4         fileReader = new FileReader(pathToFile);
5     } catch (FileNotFoundException fnfe) {
6         fnfe.printStackTrace(); // Datei nicht gefunden
7     }
8 }
```

```
8
9     BufferedReader bufferedReader = new BufferedReader(fileReader);
10
11     String firstLine = null;
12     try {
13         firstLine = bufferedReader.readLine();
14     } catch (IOException ioe) {
15         ioe.printStackTrace(); // Datei kann nicht gelesen werden
16     }
17
18     try {
19         return Integer.parseInt(firstLine);
20     } catch (NumberFormatException nfe) {
21         nfe.printStackTrace(); // Das war wohl kein Integer
22     }
23
24     return 0;
25 }
```

In dieser Variante wird jede Operation, die eine Exception werfen kann, separat in ein **try/catch** verpackt und jeweils separat auf den möglichen Fehler reagiert.

Dadurch kann man die Fehler sehr einfach dem jeweiligen Statement zuordnen.

Allerdings muss man nun mit Behelfsinitialisierungen arbeiten und der Code wird sehr in die Länge gezogen und man erkennt die eigentlichen funktionalen Zusammenhänge nur noch schwer.

Anmerkung: Das “Behandeln” der Exceptions ist im obigen Beispiel kein gutes Beispiel für das Behandeln von Exceptions. Einfach nur einen Stacktrace zu printen und weiter zu machen, als ob nichts passiert wäre, ist **kein sinnvolles Exception-Handling**. Wenn Sie solchen Code schreiben oder sehen, ist das ein Anzeichen, dass auf dieser Ebene nicht sinnvoll mit dem Fehler umgegangen werden kann und dass man ihn besser an den Aufrufer weiter reichen sollte (siehe nächste Folie).

8.5.10.3. 3. Funktionaler Teil in gemeinsames Try und mehrstufiges Catch

```
1  int getFirstLineAsIntV3(String pathToFile) {
2      try {
3          FileReader fileReader = new FileReader(pathToFile);
4          BufferedReader bufferedReader = new BufferedReader(fileReader);
5          String firstLine = bufferedReader.readLine();
6          return Integer.parseInt(firstLine);
7      } catch (FileNotFoundException fnfe) {
8          fnfe.printStackTrace(); // Datei nicht gefunden
9      } catch (IOException ioe) {
10         ioe.printStackTrace(); // Datei kann nicht gelesen werden
11     } catch (NumberFormatException nfe) {
12         nfe.printStackTrace(); // Das war wohl kein Integer
13     }
14
15     return 0;
16 }
```

Hier wurde der eigentliche funktionale Kern der Methode in ein gemeinsames **try/catch** verpackt und mit

einem mehrstufigen **catch** auf die einzelnen Fehler reagiert. Durch die Art der Exceptions sieht man immer noch, wo der Fehler herkommt. Zusätzlich wird die eigentliche Funktionalität so leichter erkennbar.

Anmerkung: Auch hier ist das gezeigte Exception-Handling kein gutes Beispiel. Entweder man macht hier sinnvollere Dinge, oder man überlässt dem Aufrufer die Reaktion auf den Fehler.

8.5.11. Vertiefung: Stilfrage C: Wo fange ich die Exception?

```
1 private static void methode1(int x) throws IOException {
2     JFileChooser fc = new JFileChooser();
3     fc.showDialog(null, "ok");
4     methode2(fc.getSelectedFile().toString(), x, x * 2);
5 }
6
7 private static void methode2(String path, int x, int y) throws IOException {
8     FileWriter fw = new FileWriter(path);
9     BufferedWriter bw = new BufferedWriter(fw);
10    bw.write("X:" + x + " Y: " + y);
11 }
12
13 public static void main(String... args) {
14     try {
15         methode1(42);
16     } catch (IOException ioe) {
17         ioe.printStackTrace();
18     }
19 }
```

Prinzipiell steht es einem frei, wo man eine Exception fängt und behandelt. Wenn im `main()` eine nicht behandelte Exception auftritt (weiter nach oben geleitet wird), wird das Programm mit einem Fehler beendet.

Letztlich scheint es eine gute Idee zu sein, eine Exception so nah wie möglich am Ursprung der Fehlerursache zu behandeln. Man sollte sich dabei die Frage stellen: Wo kann ich sinnvoll auf den Fehler reagieren?

8.5.12. Vertiefung: Eigene Exceptions definieren

```
1 // Checked Exception
2 public class MyCheckedException extends Exception {
3     public MyCheckedException(String errorMessage) {
4         super(errorMessage);
5     }
6 }
```

```
1 // Unchecked Exception
2 public class MyUncheckedException extends RuntimeException {
3     public MyUncheckedException(String errorMessage) {
4         super(errorMessage);
5     }
6 }
```

Eigene Exceptions können durch Spezialisierung anderer Exception-Klassen realisiert werden. Dabei kann man direkt von `Exception` oder `RuntimeException` ableiten oder bei Bedarf von spezialisierteren Exception-Klassen.

Wenn die eigene Exception in der Vererbungshierarchie unter `RuntimeException` steht, handelt es sich um eine *unchecked Exception*, sonst um eine *checked Exception*.

In der Benutzung (werfen, fangen, deklarieren) verhalten sich eigene Exception-Klassen wie die Exceptions aus dem JDK.

8.5.13. Wrap-Up

- Exceptions trennen **Normalfall** und **Fehlerfall** im *Kontrollfluss*
- Checked vs. Unchecked:
 - Checked = behandeln oder deklarieren
 - Unchecked = keine Pflicht, aber bewusst einsetzen
- **try-catch-finally**:
 - **try**: “gefährlicher” Code
 - **catch**: definierte Reaktion
 - **finally**: Aufräumarbeiten

Important

Exceptions führen einen zweiten Kontrollfluss ein. Das kann schnell zu unübersichtlichen Strukturen und Abläufen führen.

Exceptions passen nicht zu moderner Verarbeitung mit der Stream-API und Lambda-Ausdrücken oder Methodenreferenzen.

Zum Nachlesen

Lesen Sie zu diesem Thema auch in den dev.java-Tutorials von Oracle “[Exceptions](#)” nach.

Lernziele

- k2: Ich kann erklären, warum wir in Java Exceptions brauchen
- k2: Ich kann typische Fehler beim Exception-Handling erkennen
- k2: Ich kann zwischen checked und unchecked Exceptions unterscheiden
- k3: Ich kann mit **try/catch/finally** und **throws** gezielt einsetzen
- k3: Ich kann eigene Exceptions definieren

Challenges

Diskussion

1. Wo im System (UI, Service, Datenzugriff) sollten Sie typischerweise Exceptions fangen?
2. Wo lassen Sie sie bewusst "nach oben durchlaufen"?

Mini-Aufgabe

Implementieren Sie

```
1 public static double safeReciprocalFromFile(String fileName) {  
2     // ...  
3 }
```

Anforderungen:

- Bei Dateifehlern: Rückgabewert `Double.NaN`
- Bei ungültiger Zahl: Rückgabewert `0.0`
- Bei Division durch 0: Rückgabewert `Double.POSITIVE_INFINITY`

Frage: Welche Exceptions fangen Sie? Welche lassen Sie durchlaufen?

Mini-Quizz:

1. Erläutern Sie in 1-2 Sätzen den Unterschied zwischen **checked** und **unchecked** Exceptions in Java.
2. Nennen Sie je **zwei Beispiele** für checked bzw. unchecked Exceptions.
3. Was macht der Compiler?

```
1 public static String readFileContent(String fileName) {  
2     return Files.readString(Path.of(fileName));  
3 }
```

- kompiliert ohne Warnungen/Fehler
 - Compilerfehler (falls ja, welche?)
 - Laufzeitfehler (aber kein Compilerfehler)
 - Compiler fügt automatisch **try-catch** hinzu
4. catch-Struktur beurteilen
- ```
1 try {
2 double result = reciprocalFromFile("zahl.txt");
3 System.out.println(result);
4 } catch (IOException e) {
5 System.err.println("I/O-Fehler");
6 } catch (Exception e) {
7 System.err.println("Allgemeiner Fehler");
8 }
```
- Welche Exceptions werden vom zweiten **catch**-Block noch erfasst?
  - Diskutieren Sie kurz: Ist das aus Ihrer Sicht eine gute Idee? Warum / warum nicht?

### 5. Designentscheidung

Sie haben eine Methode in einer Bibliothek:

```
1 public static double divide(int a, int b) {
2 return a / b;
3 }
```

Diskutieren Sie:

- Würden Sie hier eine eigene checked Exception (z.B. `DivisionByZeroException`) einführen?
- Oder vertrauen Sie auf die bestehende `ArithmeticException` (unchecked)?

## 9. Clean Code - Smells, Rules, Refactoring

### 9.1. Code Smells

#### TL;DR

Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code. Dabei helfen “Coding Conventions”, die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, ...). Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit zu verbessern.

Code, der diese Konventionen und Regeln verletzt, zeigt sogenannte “**Code Smells**” oder “Bad Smells”. Das sind Probleme im Code, die noch nicht direkt zu einem Fehler führen, die aber im Laufe der Zeit die Chance für echte Probleme deutlich erhöht.

#### Videos

Vorlesung [\[YT\]](#), [\[HSBI\]](#)

#### 9.1.1. Code Smells: Ist das Code oder kann das weg?

In dieser Sitzung geht es nicht um Fehler im Sinne von “Programm stürzt ab”, sondern um strukturelle Probleme, die die Wartbarkeit verschlechtern.

```
1 class checker {
2 static public void CheckANDDO(DATA1 inp, int c, FH.Studi
3 CustD, int x, int y, int in, int out, int c1, int c2, int c3 = 4)
4 {
5 public int i; // neues i
6 for(i=0;i<10;i++) // fuer alle i
7 {
8 inp.kurs[0] = 10; inp.kurs[i] = CustD.cred[i]/c;
9 }
10 SetDataToPlan(CustD);
11 public double myI = in*2.5; // myI=in*2.5
12 if (c1)
13 out = myI; //OK
14 else if(c3 == 4)
15 {
16 myI = c2 * myI;
17 if (c3 != 4 || true) { // unbedingt beachten!
18 //System.out.println("x:"+(x++));
```

```
19 System.out.println("x:"+(x++)); // x++
20 System.out.println("out: "+out);
21 } } }
```

Der Code im obigen Beispiel ist bewusst überzogen und nicht vollständig korrekt. Er lässt sich möglicherweise kompilieren. Und möglicherweise tut er sogar das, was er tun soll.

Dennoch: **Der Code “stinkt”** (zeigt **Code Smells**):

- Oberflächliche Smells: Formatierung, schlechte Kommentare, schlechte Namen, auskommentierter Code
- Struktur-Smells: duplizierter Code, lange Methoden/Klassen, lange Parameterlisten, tiefe Verschachtelung
- OO-Smells: fehlende Kapselung, Feature Neid, globale Variablen, Magic Numbers

Diese Liste enthält die häufigsten “Smells” und ließe sich noch beliebig fortsetzen. Schauen Sie mal in die unten angegebene Literatur :-)

### **Stinkender Code führt zu möglichen (späteren) Problemen.**

Diese Smells (z.B. “duplizierter Code”) werden wir in der Refactoring-Sitzung mit Techniken wie “Extract Method” etc. angehen.

## **9.1.2. Was ist guter (“sauberer”) Code (“Clean Code”)?**

Im Grunde bezeichnet “sauberer Code” (“Clean Code”) die Abwesenheit von Smells. D.h. man könnte Code als “sauberen” Code bezeichnen, wenn die folgenden Eigenschaften erfüllt sind (keine vollständige Aufzählung!):

- Gut (“angenehm”) lesbar
- Schnell verständlich: Geeignete Abstraktionen
- Konzentriert sich auf **eine** Aufgabe
- So einfach und direkt wie möglich
- Ist gut getestet

In (Martin 2009) lässt der Autor Robert Martin verschiedene Ikonen der SW-Entwicklung zu diesem Thema zu Wort kommen - eine sehr lesenswerte Lektüre!

=> Jemand kümmert sich um den Code; solides Handwerk

## **9.1.3. Warum ist guter (“sauberer”) Code so wichtig?**

Any fool can write code that a computer can understand. Good programmers write code that humans can understand.

Quelle: “Any fool...”: (Fowler 2011, p. 15)

Auch wenn das zunächst seltsam klingt, aber Code muss auch von Menschen gelesen und verstanden werden können. Klar, der Code muss inhaltlich korrekt sein und die jeweilige Aufgabe erfüllen, er muss kompilieren

etc. ... aber er muss auch von anderen Personen weiter entwickelt werden und dazu gelesen und verstanden werden. Guter Code ist nicht einfach nur inhaltlich korrekt, sondern kann auch einfach verstanden werden.

Code, der nicht einfach lesbar ist oder nur schwer verständlich ist, wird oft in der Praxis später nicht gut gepflegt: Andere Entwickler haben (die berechnigte) Angst, etwas kaputt zu machen und arbeiten "um den Code herum". Nur leider wird das Konstrukt dann nur noch schwerer verständlich ...

#### 9.1.3.1. Code Smells

Verstöße gegen die Prinzipien von *Clean Code* nennt man auch *Code Smells*: Der Code "stinkt" gewissermaßen. Dies bedeutet nicht unbedingt, dass der Code nicht funktioniert (d.h. er kann dennoch compilieren und die Anforderungen erfüllen). Er ist nur nicht sauber formuliert, schwer verständlich, enthält Doppelungen etc., was im Laufe der Zeit die Chance für tatsächliche Probleme deutlich erhöht.

Und weil es so wichtig ist, hier gleich noch einmal:

**Stinkender Code führt zu möglichen (späteren) Problemen.**

#### 9.1.3.2. "Broken Windows" Phänomen

Wenn ein Gebäude leer steht, wird es eine gewisse Zeit lang nur relativ langsam verfallen: Die Fenster werden nicht mehr geputzt, es sammelt sich Graffiti, Gras wächst in der Dachrinne, Putz blättert ab ...

Irgendwann wird dann eine Scheibe eingeworfen. Wenn dieser Punkt überschritten ist, beschleunigt sich der Verfall rasant: Über Nacht werden alle erreichbaren Scheiben eingeworfen, Türen werden zerstört, es werden sogar Brände gelegt ...

Das passiert auch bei Software! Wenn man als Entwickler das Gefühl bekommt, die Software ist nicht gepflegt, wird man selbst auch nur relativ schlechte Arbeit abliefern. Sei es, weil man nicht versteht, was der Code macht und sich nicht an die Überarbeitung der richtigen Stellen traut und stattdessen die Änderungen als weiteren "Erker" einfach dran pappt. Seit es, weil man keine Lust hat, Zeit in ordentliche Arbeit zu investieren, weil der Code ja eh schon schlecht ist ... Das wird mit der Zeit nicht besser ...

##### Tip

In Code-Basen sind "eingeworfene Fenster" z.B. auskommentierter Code, wilde Formatierung, ungenutzte Variablen.

"Broken Windows" Phänomen

#### 9.1.3.3. Maßeinheit für Code-Qualität ;-)

Es gibt eine "praxisnahe" (und nicht ganz ernst gemeinte) Maßeinheit für Code-Qualität: Die "WTF/m" (*What the Fuck per minute*): Thom Holwerda: [www.osnews.com/story/19266/WTFs](http://www.osnews.com/story/19266/WTFs).

Wenn beim Code-Review durch Kollegen viele "WTF" kommen, ist der Code offenbar nicht in Ordnung ...

#### 9.1.4. Code Smells: Nichtbeachtung von Coding Conventions

- Richtlinien für einheitliches Aussehen => Andere Programmierer sollen Code schnell lesen können
  - Namen, Schreibweisen
  - Kommentare (Ort, Form, Inhalt)
  - Einrückungen und Spaces vs. Tabs
  - Zeilenlängen, Leerzeilen
  - Klammern
- Beispiele: [Sun Code Conventions](#), [Google Java Style](#)
- *Hinweis*: Betrifft vor allem die (äußere) Form!

#### 9.1.5. Code Smells: Schlechte Kommentare I

- Ratlose Kommentare

```
1 /* k.A. was das bedeutet, aber wenn man es raus nimmt, geht's nicht mehr */
2 /* TODO: was passiert hier, und warum? */
```

Der Programmierer hat selbst nicht verstanden (und macht sich auch nicht die Mühe zu verstehen), was er da tut! Fehler sind vorprogrammiert!

- Redundante Kommentare:

```
1 public int i; // neues i
2 for(i=0;i<10;i++)
3 // fuer alle i
```

Was würden Sie Ihrem Kollegen erklären (müssen), wenn Sie ihm/ihr den Code vorstellen?

Wiederholen Sie nicht, was der Code tut (das kann ich ja selbst lesen), sondern **beschreiben Sie, was der Code tun sollte und warum**.

Beschreiben Sie dabei auch das Konzept hinter einem Codebaustein und gerne auch die typische Anwendung der Methode oder Klasse.

#### 9.1.6. Code Smells: Schlechte Kommentare II

- Veraltete Kommentare

Hinweis auf unsauberes Arbeiten: Oft wird im Zuge der Überarbeitung von Code-Stellen vergessen, auch den Kommentar anzupassen! Sollte beim Lesen extrem misstrauisch machen.



- Auskommentierter Code

Da ist jemand seiner Sache unsicher bzw. hat eine Überarbeitung nicht abgeschlossen. Die Chance, dass sich der restliche Code im Laufe der Zeit so verändert, dass der auskommentierte Code nicht mehr (richtig) läuft, ist groß! Auskommentierter Code ist gefährlich und dank Versionskontrolle absolut überflüssig!

- Kommentare erscheinen zwingend nötig

Häufig ein Hinweis auf ungeeignete Wahl der Namen (Klassen, Methoden, Attribute) und/oder auf ein ungeeignetes Abstraktionsniveau (beispielsweise Nichtbeachtung des Prinzips der “*Single Responsibility*”)!

Der Code soll im **Normalfall** für sich selbst sprechen: **WAS** wird gemacht. Der Kommentar erklärt im Normalfall, **WARUM** der Code das machen soll und die Konzepte und Design-Entscheidungen dahinter.

- Unangemessene Information, z.B. Änderungshistorien

Hinweise wie “wer hat wann was geändert” gehören in das Versionskontroll- oder ins Issue-Tracking-System. Die Änderung ist im Code sowieso nicht mehr sichtbar/nachvollziehbar!

### 9.1.7. Code Smells: Schlechte Namen und fehlende Kapselung

```
1 public class Studi extends Person {
2 public String n;
3 public int c;
4
5 public void prtIf() { ... }
6 }
```

Nach drei Wochen fragen Sie sich, was `n` oder `c` oder `Studi#prtIf()` wohl sein könnte! (Ein anderer Programmierer fragt sich das schon beim **ersten** Lesen.) Klassen und Methoden sollten sich erwartungsgemäß verhalten.

Wenn Dinge öffentlich angeboten werden, muss man damit rechnen, dass andere darauf zugreifen. D.h. man kann nicht mehr so einfach Dinge wie die interne Repräsentation oder die Art der Berechnung austauschen! Öffentliche Dinge gehören zur Schnittstelle und damit Teil des “Vertrags” mit den Nutzern!

- Programmierprinzip “**Prinzip der minimalen Verwunderung**”
  - Klassen und Methoden sollten sich erwartungsgemäß verhalten
  - Gute Namen ersparen das Lesen der Dokumentation
- Programmierprinzip “**Kapselung/Information Hiding**”
  - Möglichst schlanke öffentliche Schnittstelle
  - => “Vertrag” mit Nutzern der Klasse!

### 9.1.8. Code Smells: Duplizierter Code

```
1 public class Studi {
2 public String getName() { return name; }
3 public String getAddress() {
4 return strasse+", "+plz+" "+stadt;
5 }
6
7 public String getStudiAsString() {
8 return name+" ("+strasse+", "+plz+" "+stadt+");"
9 }
10 }
```

- Programmierprinzip “**DRY**” => “Don’t repeat yourself!”

Im Beispiel wird das Formatieren der Adresse mehrfach identisch implementiert, d.h. duplizierter Code. Auslagern in eigene Methode und aufrufen!

Kopierter/duplizierter Code ist problematisch:

- Spätere Änderungen müssen an mehreren Stellen vorgenommen werden
- Lesbarkeit/Orientierung im Code wird erschwert (Analogie: Reihenhaussiedlung)
- Verpasste Gelegenheit für sinnvolle Abstraktion!

### 9.1.9. Code Smells: Langer Code

- Lange Klassen
  - Faustregel: 5 Bildschirmseiten sind viel
- Lange Methoden
  - Faustregel: 1 Bildschirmseite
  - (Martin 2009): deutlich weniger als 20 Zeilen
- Lange Parameterlisten
  - Faustregel: max. 3 ... 5 Parameter
  - (Martin 2009): 0 Parameter ideal, ab 3 Parameter gute Begründung nötig
- Tief verschachtelte **if/then**-Bedingungen
  - Faustregel: 2 ... 3 Einrückungsebenen sind viel
- Programmierprinzip “**Single Responsibility**”
  - Jede Klasse ist für genau **einen Aspekt** des Gesamtsystems verantwortlich

### 9.1.9.1. Lesbarkeit und Übersichtlichkeit leiden

- Der Mensch kann sich nur begrenzt viele Dinge im Kurzzeitgedächtnis merken
- Klassen, die länger als 5 Bildschirmseiten sind, erfordern viel Hin- und Her-Scrollen, dito für lange Methoden
- Lange Methoden sind schwer verständlich (erledigen viele Dinge?)
- Mehr als 3 Parameter kann sich kaum jemand merken, vor allem beim Aufruf von Methoden
- Die Testbarkeit wird bei zu komplexen Methoden/Klassen und vielen Parametern sehr erschwert
- Große Dateien verleiten (auch mangels Übersichtlichkeit) dazu, neuen Code ebenfalls schluderig zu gliedern

### 9.1.9.2. Langer Code deutet auch auf eine Verletzung des Prinzips der Single Responsibility hin (engl. Single Responsibility Principle (SRP))

- Klassen fassen evtl. nicht zusammengehörende Dinge zusammen

```

1 public class Student {
2 private String name;
3 private String phoneAreaCode;
4 private String phoneNumber;
5
6 public void printStudentInfo() {
7 System.out.println("name: " + name);
8 System.out.println("contact: " + phoneAreaCode + "/" + phoneNumber)
9 ;
10 }
11 }
```

Warum sollte sich die Klasse `Student` um die Einzelheiten des Aufbaus einer Telefonnummer kümmern? Das Prinzip der “*Single Responsibility*” wird hier verletzt!

- Methoden erledigen vermutlich mehr als nur eine Aufgabe

```

1 public void credits() {
2 for (Student s : students) {
3 if (s.hasSemesterFinished()) {
4 ECTS c = calculateEcts(s);
5 s.setEctsSum(c);
6 }
7 }
8 }
9
10 // Diese Methode erledigt 4 Dinge: Iteration, Abfrage, Berechnung, Setzen
 ...
```

=> Erklären Sie die Methode jemandem. Wenn dabei das Wort “und” vorkommt, macht die Methode höchstwahrscheinlich zu viel!

- Viele Parameter bedeuten oft fehlende Datenabstraktion

```

1 Circle makeCircle(int x, int y, int radius);
2 Circle makeCircle(Point center, int radius); // besser!
```

Damit verwandt sind “Primitive Obsession”: überall `int`, `String` statt eigener und passenderer Typen (`PhoneNumber`, `Ects`, `Address`, o.ä.), und “Data Clumps”: immer wieder dieselben Parametergruppen (`x`, `y`, `radius` statt `Point center`).

### 9.1.9.3. Problem: Passende Abstraktionsebene finden

Smell: Schlecht gewählte Abstraktionsebene

Wenn eine Methode 20 Zeilen Implementation mit vielen Details enthält (also passend zur Faustregel der Länge einer Methode ist), aber aufrufende Methoden kaum lesbar sind, weil sie 6-8 dieser Low-Level-Methoden minutengenau orchestrieren.

### 9.1.10. Code Smells: Fehlender Umgang mit Fehlerfällen

- Ignorieren von Fehlersituationen (z.B. `null`-Rückgaben, leere Listen)
- Mischen von Geschäftslogik und Fehlerbehandlung so, dass nichts mehr lesbar ist

### 9.1.11. Code Smells: Feature Neid (engl. Feature Envy)

```
1 public class CreditsCalculator {
2 public ECTS calculateEcts(Student s) {
3 int semester = s.getSemester();
4 int workload = s.getCurrentWorkload();
5 int nrModules = s.getNumberOfModules();
6 int total = Math.min(30, workload);
7 int extra = Math.max(0, total - 30);
8 if (semester < 5) {
9 extra = extra * nrModules;
10 }
11 return new ECTS(total + extra);
12 }
13 }
```

- Zugriff auf (viele) Interna der anderen Klasse! => Hohe Kopplung der Klassen!
- Methode `CreditsCalculator#calculateEcts()` “möchte” eigentlich in `Student` sein ...

### 9.1.12. Wrap-Up

- Code entsteht nicht zum Selbstzweck => Lesbarkeit ist wichtig
- Code Smells: Code führt zu möglichen (späteren) Problemen
  - Richtiges Kommentieren und Dokumentieren

In dieser Sitzung haben wir vor allem auf Kommentare geschaut. Zum Thema Dokumentieren siehe die Einheit zu “Javadoc”.

- Einhalten von Coding Conventions
  - \* Regeln zu Schreibweisen und Layout
  - \* Leerzeichen, Einrückung, Klammern
  - \* Zeilenlänge, Umbrüche
  - \* Kommentare
- Einhalten von Prinzipien des objektorientierten Programmierens
  - \* Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
  - \* Keine Code-Duplizierung! (*DRY* - Don't repeat yourself)
  - \* Klassen und Methoden sollten sich erwartungsgemäß verhalten
  - \* Kapselung: Möglichst wenig öffentlich zugänglich machen

### Zum Nachlesen

Gute Werke zum Nachlesen sind Martin (2009) (ein Klassiker) und Passig und Jander (2013). Im Odin-Project hat man sich viele Gedanken gemacht: [“Foundations: Clean Code” \(The Odin Project\)](#), und Google hat einen vielbeachteten Style-Guide für Java geschrieben: [“Documentation Best Practices” \(Google Styleguide\)](#)

### Lernziele

- k2: Ich kann typische Programmierprinzipien wie ‘Information Hiding’, ‘DRY’, ‘Single Responsibility’ erklären
- k3: Ich kann typische Code Smells erkennen und vermeiden
- k3: Ich kann leicht lesbaren von schwer lesbarem Code unterscheiden
- k34 Ich kann Programmierprinzipien anwenden, um den Code sauberer zu gestalten
- k3: Ich kann sinnvolle Kommentare schreiben

## 9.2. Coding Conventions und Metriken

### TL;DR

Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code.

Dabei helfen “Coding Conventions”, die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, ...). Im Java-Umfeld ist der “Google Java Style” bzw. der recht ähnliche “AOSP Java Code Style for Contributors” häufig anzutreffen. Coding Conventions beinhalten typischerweise Regeln zu

- Schreibweisen und Layout
- Leerzeichen, Einrückung, Klammern
- Zeilenlänge, Umbrüche
- Kommentare

Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit

zu verbessern.

Metriken sind Kennzahlen, die aus dem Code berechnet werden, und können zur Überwachung der Einhaltung von Coding Conventions und anderen Regeln genutzt werden. Nützliche Metriken sind dabei NCSS (*Non Commenting Source Statements*), McCabe (*Cyclomatic Complexity*), BEC (*Boolean Expression Complexity*) und DAC (*Class Data Abstraction Coupling*).

Für die Formatierung des Codes kann man die IDE nutzen, muss dort dann aber die Regeln detailliert manuell einstellen. Das Tool **Spotless** lässt sich dagegen in den Build-Prozess einbinden und kann die Konfiguration über ein vordefiniertes Regelset passend zum Google Java Style/AOSP automatisiert vornehmen.

Die Prüfung der Coding Conventions und Metriken kann durch das Tool **Checkstyle** erfolgen. Dieses kann beispielsweise als Plugin in der IDE oder direkt in den Build-Prozess eingebunden werden und wird mit Hilfe einer XML-Datei konfiguriert.

Um typische Anti-Pattern zu vermeiden, kann man den Code mit sogenannten *Linter* prüfen. Ein Beispiel für die Java-Entwicklung ist **SpotBugs**, welches sich in den Build-Prozess einbinden lässt und über 400 typische problematische Muster im Code erkennt.

#### Videos

Vorlesung [YT], [HSBI]

Demo:

- Formatter und Spotless [YT], [HSBI]
- Checkstyle [YT], [HSBI]
- Checkstyle-Konfiguration mit Eclipse-CS erzeugen [YT], [HSBI]
- SpotBugs [YT], [HSBI]

### 9.2.1. Coding Conventions: Richtlinien für einheitliches Aussehen von Code

=> Ziel: Andere Programmierer sollen Code schnell lesen können

- **Namen, Schreibweisen:** UpperCamelCase vs. lowerCamelCase vs. UPPER\_SNAKE\_CASE
- **Kommentare** (Ort, Form, Inhalt): Javadoc an allen **public** und **protected** Elementen
- **Einrückungen und Spaces vs. Tabs:** 4 Spaces
- **Zeilenlängen:** 100 Zeichen
- **Leerzeilen:** Leerzeilen für Gliederung
- **Klammern:** Auf selber Zeile wie Code

Beispiele: [Sun Code Conventions](#), [Google Java Style](#), [AOSP Java Code Style for Contributors](#)

Das Thema Tabs vs. Spaces ist in Java nicht so extrem wichtig wie in Python, dennoch sollte in einem Projekt ein einheitlicher Standard verfolgt werden. Hilfreich ist hier Toolunterstützung über eine im Projekt mit versionierte `.editorconfig`, welche über ein Plugin in der IDE gelesen wird und beim Speichern von Dateien dann angewendet wird. Das Projekt [EditorConfig](#) bietet eine Erklärung für das Dateiformat und Links zu zahlreichen Plugins für IDEs und Editoren.

### 9.2.2. Beispiel nach Google Java Style/AOSP formatiert

```
1 package wuppie.deeplearning.strategy;
2
3 /**
4 * Demonstriert den Einsatz von AOSP/Google Java Style
5 * Umbruch nach 100 Zeichen |
6 */
7 public class MyWuppieStudi implements Comparable<MyWuppieStudi> {
8 private static String lastName;
9 private static MyWuppieStudi studi;
10
11 private MyWuppieStudi() {}
12
13 /** Erzeugt ein neues Exemplar der MyWuppieStudi-Spezies (max. 40 Zeilen)
14 */
15 public static MyWuppieStudi getMyWuppieStudi(String name) {
16 if (studi == null) {
17 studi = new MyWuppieStudi();
18 }
19 if (lastName == null) lastName = name;
20 return studi;
21 }
22
23 @Override
24 public int compareTo(MyWuppieStudi o) {
25 return lastName.compareTo(o.lastName);
26 }
27 }
```

Dieses Beispiel wurde nach Google Java Style/AOSP formatiert.

Die Zeilenlänge beträgt max. 100 Zeichen. Pro Methode werden max. 40 Zeilen genutzt. Zwischen Attributen, Methoden und Importen wird jeweils eine Leerzeile eingesetzt (zwischen den einzelnen Attributen *muss* aber keine Leerzeile genutzt werden). Zur logischen Gliederung können innerhalb von Methoden weitere Leerzeilen eingesetzt werden, aber immer nur eine.

Klassennamen sind UpperCamelCase, Attribute und Methoden und Parameter lowerCamelCase, Konstanten (im Beispiel nicht vorhanden) UPPER\_SNAKE\_CASE. Klassen sind Substantive, Methoden Verben.

Alle **public** und **protected** Elemente werden mit einem Javadoc-Kommentar versehen. Überschriebene Methoden müssen nicht mit Javadoc kommentiert werden, müssen aber mit **@Override** markiert werden.

Geschweifte Klammern starten immer auf der selben Codezeile. Wenn bei einem **if** nur ein Statement vorhanden ist und dieses auf die selbe Zeile passt, kann auf die umschließenden geschweiften Klammern ausnahmsweise verzichtet werden.

Es wird mit Leerzeichen eingerückt. [Google Java Style](#) arbeitet mit 2 Leerzeichen, während [AOSP](#) hier 4 Leerzeichen vorschreibt. Im Beispiel wurde nach AOSP eingerückt.

Darüber hinaus gibt es vielfältige weitere Regeln für das Aussehen des Codes. Lesen Sie dazu entsprechend auf [Google Java Style](#) und auch auf [AOSP](#) nach.

### 9.2.3. Formatieren Sie Ihren Code (mit der IDE)

Sie können den Code manuell formatieren, oder aber (sinnvollerweise) über Tools formatieren lassen. Hier einige Möglichkeiten:

- IDE: Code-Style einstellen und zum Formatieren nutzen
- `google-java-format: java -jar google-java-format.jar --replace *.java` (auch als IDE-Plugin)
- **Spotless** in Gradle:

```
1 plugins {
2 id "java"
3 id "com.diffplug.spotless" version "8.7.0"
4 }
5
6 spotless {
7 java {
8 // googleJavaFormat()
9 googleJavaFormat().aosp() // indent w/ 4 spaces
10 }
11 }
```

Prüfen mit `./gradlew spotlessCheck` (Teil von `./gradlew check`) und Formatieren mit `./gradlew spotlessApply`

#### Important

Die Versionsnummern in den Beispielen sind Momentaufnahmen. In eigenen Projekten bitte immer die aktuelle empfohlene Version aus der offiziellen Doku nachschlagen.

#### 9.2.3.1. Einstellungen der IDE's

- Eclipse:
  - `Project > Properties > Java Code Style > Formatter: Coding-Style` einstellen/einrichten
  - Code markieren, `Source > Format`
  - Komplettes Aufräumen: `Source > Clean Up` (Formatierung, Importe, Annotationen, ...) Kann auch so eingestellt werden, dass ein "Clean Up" immer beim Speichern ausgeführt wird!
- IntelliJ verfügt über ähnliche Fähigkeiten:
  - Einstellen über `Preferences > Editor > Code Style > Java`
  - Formatieren mit `Code > Reformat Code` oder `Code > Reformat File`

Die Details kann/muss man einzeln einstellen. Für die "bekannten" Styles (Google Java Style) bringen die IDE's oft aber schon eine Gesamtkonfiguration mit.

**Achtung:** Zumindest in Eclipse gibt es mehrere Stellen, wo ein Code-Style eingestellt werden kann ("Clean Up", "Formatter", ...). Diese sollten dann jeweils auf den selben Style eingestellt werden, sonst gibt es unter



Umständen lustige Effekte, da beim Speichern ein anderer Style angewendet wird als beim “Clean Up” oder beim “Format Source” ...

Analog sollte man bei der Verwendung von Checkstyle auch in der IDE im Formatter die entsprechenden Checkstyle-Regeln (s.u.) passend einstellen, sonst bekommt man durch Checkstyle Warnungen angezeigt, die man durch ein automatisches Formatieren *nicht* beheben kann.

### 9.2.3.2. Google Java Style und google-java-format

Wer direkt den [Google Java Style](#) nutzt, kann auch den dazu passenden Formatter von Google einsetzen: [google-java-format](#). Diesen kann man entweder als Plugin für IntelliJ/Eclipse einsetzen oder als Stand-alone-Tool (Kommandozeile oder Build-Skripte) aufrufen. Wenn man sich noch einen entsprechenden Git-Hook definiert, wird vor jedem Commit der Code entsprechend den Richtlinien formatiert :)

### 9.2.3.3. Spotless und google-java-format in Gradle

*Hinweis:* Bei Spotless in Gradle müssen je nach den Versionen von Spotless/google-java-format bzw. des JDK noch Optionen in der Datei `gradle.properties` eingestellt werden (siehe [Demo](#) und [Spotless > google-java-format \(Web\)](#)).

**Tipp:** Die Formatierung über die IDE ist angenehm, aber in der Praxis leider oft etwas hakelig: Man muss alle Regeln selbst einstellen (und es gibt *einige* dieser Einstellungen), und gerade IntelliJ “greift” manchmal nicht alle Code-Stellen beim Formatieren. Nutzen Sie Spotless und bauen Sie die Konfiguration in Ihr Build-Skript ein und konfigurieren Sie über den Build-Prozess.

[Demo: Konfiguration Formatter \(IDE\), Spotless/Gradle](#)

## 9.2.4. Metriken: Kennzahlen für verschiedene Aspekte zum Code

Metriken messen verschiedene Aspekte zum Code und liefern eine Zahl zurück. Mit Metriken kann man beispielsweise die Einhaltung der Coding Rules (Formate, ...) prüfen, aber auch die Einhaltung verschiedener Regeln des objektorientierten Programmierens.

### 9.2.4.1. Beispiele für wichtige Metriken (jeweils Max-Werte für PR2)

Die folgenden Metriken und deren Maximal-Werte sind gute **Erfahrungswerte** aus der Praxis und helfen, den Code Smell “Langer Code” (vgl. “Code Smells”) zu erkennen und damit zu vermeiden. Über die Metriken *BEC*, *McCabe* und *DAC* wird auch die Einhaltung elementarer Programmierregeln gemessen.

- **NCSS** (*Non Commenting Source Statements*)
  - Zeilen pro Methode: 40; pro Klasse: 250; pro Datei: 300  
*Annahme:* Eine Anweisung je Zeile ...
- **Anzahl der Methoden** pro Klasse: 10

- **Parameter** pro Methode: 3
- **BEC** (*Boolean Expression Complexity*)  
Anzahl boolescher Ausdrücke in **if** etc.: 3 (Anzahl von Verknüpfungen mit && bzw. ||: **if** (a && b && c) hat eine BEC von 3)
- **McCabe** (*Cyclomatic Complexity*)
  - Anzahl der möglichen Verzweigungen (Pfade) pro Methode + 1
  - 1-4 gut, 5-7 noch OK
- **DAC** (*Class Data Abstraction Coupling*)
  - Anzahl verschiedener Fremdtypen, die eine Klasse verwendet (z.B. als Attribute, Parameter, lokale Variablen) - Hohe Werte bedeuten: Die Klasse "kennt" sehr viele andere Klassen → starke Kopplung.
  - Werte kleiner 7 werden i.A. als normal betrachtet

### Important

Die obigen Grenzwerte sind typische **Standardwerte**, die sich in der Praxis allgemein bewährt haben (vergleiche u.a. (Martin 2009) oder auch in [AOSP: Write short methods](#) und [AOSP: Limit line length](#)). Dennoch sind das keine absoluten Werte an sich. Ein Übertreten der Grenzen ist ein **Hinweis** darauf, dass **höchstwahrscheinlich** etwas nicht stimmt, muss aber im konkreten Fall hinterfragt und diskutiert und begründet werden!

### 9.2.4.2. Metriken im Beispiel von oben

```

1 private static String lastName;
2 private static MyWuppieStudi studi;
3
4 public static MyWuppieStudi getMyWuppieStudi(String name) {
5 if (studi == null) {
6 studi = new MyWuppieStudi();
7 }
8 if (lastName == null) lastName = name;
9
10 return studi;
11 }

```

- BEC: 1 (nur ein boolescher Ausdruck im **if**)
- McCabe: 3 (es gibt zwei mögliche Verzweigungen in der Methode plus die Methode selbst)
- DAC: 1 (eine "Fremdklasse": `String`)

*Anmerkung:* In Checkstyle werden für einige häufig verwendete Standard-Klassen Ausnahmen definiert, d.h. `String` würde im obigen Beispiel *nicht* bei DAC mitgezählt/angezeigt.

=> Verweis auf LV Softwareengineering

### 9.2.5. Stil & Metriken: Checkstyle

Metriken und die Einhaltung von Coding-Conventions werden sinnvollerweise nicht manuell, sondern durch diverse Tools erfasst, etwa im Java-Bereich mit Hilfe von **Checkstyle**.

Das Tool lässt sich **Standalone über CLI** nutzen oder als Plugin für IDE's (**Eclipse** oder **IntelliJ**) einsetzen. Gradle bringt ein eigenes **Plugin** mit.

- IDE: diverse **Plugins**: **Eclipse-CS**, **CheckStyle-IDEA**
- **CLI**: `java -jar checkstyle-10.2-all.jar -c google_checks.xml *.java`
- **Plugin "checkstyle"** in Gradle:

```
1 plugins {
2 id "java"
3 id "checkstyle"
4 }
5
6 checkstyle {
7 configFile file('checkstyle.xml')
8 toolVersion '13.7.0'
9 }
```

- Aufruf: Prüfen mit `./gradlew checkstyleMain` (Teil von `./gradlew check`)
- Konfiguration: `<projectDir>/config/checkstyle/checkstyle.xml` (Default) bzw. mit der obigen Konfiguration direkt im Projektordner
- Report: `<projectDir>/build/reports/checkstyle/main.html`

#### Important

Die Versionsnummern in den Beispielen sind Momentaufnahmen. In eigenen Projekten bitte immer die aktuelle empfohlene Version aus der offiziellen Doku nachschlagen.

[Demo: IntelliJ, Checkstyle/Gradle](#)

### 9.2.6. Checkstyle: Konfiguration

Die auszuführenden Checks lassen sich über eine **XML-Datei** konfigurieren. In **Eclipse-CS** kann man die Konfiguration auch in einer GUI bearbeiten.

Das Checkstyle-Projekt stellt eine passende Konfiguration für den **Google Java Style** bereit. Diese ist auch in den entsprechenden Plugins oft bereits enthalten und kann direkt ausgewählt oder als Startpunkt für eigene Konfigurationen genutzt werden.

Der Startpunkt für die Konfigurationsdatei ist immer das Modul "Checker". Darin können sich "FileSetChecks" (Module, die auf einer Menge von Dateien Checks ausführen), "Filters" (Module, die Events bei der Prüfung von Regeln filtern) und "AuditListeners" (Module, die akzeptierte Events in einen Report überführen) befinden. Der "TreeWalker" ist mit der wichtigste Vertreter der FileSetChecks-Module und transformiert die zu prüfenden Java-Sourcen in einen *Abstract Syntax Tree*, also eine Baumstruktur, die dem jeweiligen Code unter der Java-Grammatik entspricht. Darauf können dann wiederum die meisten Low-Level-Module arbeiten.

Eine Reihe von [Standard-Checks](#) sind bereits in Checkstyle implementiert und benötigen keine weitere externe Abhängigkeiten. Man kann aber zusätzliche Regeln aus anderen Projekten beziehen (etwa via Gradle/Maven) oder sich eigene zusätzliche Regeln in Java schreiben. Die einzelnen Checks werden in der Regel als “Modul” dem “TreeWalker” hinzugefügt und über die jeweiligen Properties näher konfiguriert.

Sie finden in der [Doku](#) zu jedem Check das entsprechende Modul, das Eltern-Modul (also wo müssen Sie das Modul im XML-Baum einfügen) und auch die möglichen Properties und deren Default-Einstellungen.

```

1 <module name="Checker">
2 <module name="LineLength">
3 <property name="max" value="100"/>
4 </module>
5
6 <module name="TreeWalker">
7 <module name="AvoidStarImport"/>
8 <module name="MethodCount">
9 <property name="maxPublic" value="10"/>
10 <property name="maxTotal" value="40"/>
11 </module>
12 </module>
13 </module>

```

Alternativen/Ergänzungen: beispielsweise [MetricsReloaded](#).

[Demo: Konfiguration mit Eclipse-CS, Hinweis auf Formatter](#)

### 9.2.7. SpotBugs: Finde Anti-Pattern und potentielle Bugs (Linter)

- **SpotBugs** sucht nach über 400 potentiellen Bugs im Code
  - Anti-Pattern (schlechte Praxis, “dodgy” Code)
  - Sicherheitsprobleme
  - Korrektheit
- CLI: `java -jar spotbugs.jar options ...`
- IDE: [IntelliJ SpotBugs plugin](#), [SpotBugs Eclipse plugin](#)
- Gradle: [SpotBugs Gradle Plugin](#)

```

1 plugins {
2 id "java"
3 id "com.github.spotbugs" version "6.5.8"
4 }
5 spotbugs {
6 ignoreFailures = true
7 showStackTraces = false
8 }

```

Prüfen mit `./gradlew spotbugsMain` (in `./gradlew check`)

**Important**

Die Versionsnummern in den Beispielen sind Momentaufnahmen. In eigenen Projekten bitte immer die aktuelle empfohlene Version aus der offiziellen Doku nachschlagen.

Demo: [SpotBugs/Gradle](#)

## 9.2.8. Konfiguration für das PR2-Praktikum (Format, Metriken, Checkstyle, SpotBugs)

Im PR2-Praktikum beachten wir die obigen Coding Conventions und Metriken mit den dort definierten Grenzwerten. Diese sind bereits in der bereit gestellten Minimal-Konfiguration für Checkstyle (s.u.) konfiguriert.

### 9.2.8.1. Formatierung

- Google Java Style/AOSP: **Spotless**

Zusätzlich wenden wir den [Google Java Style](#) an. Statt der dort vorgeschriebenen Einrückung mit 2 Leerzeichen (und 4+ Leerzeichen bei Zeilenumbruch in einem Statement) können Sie auch mit 4 Leerzeichen einrücken (8 Leerzeichen bei Zeilenumbruch) (AOSP). Halten Sie sich in Ihrem Team an eine einheitliche Einrückung (Google Java Style oder AOSP).

Formatieren Sie Ihren Code vor den Commits mit **Spotless** (über Gradle) oder stellen Sie den Formatter Ihrer IDE entsprechend ein.

### 9.2.8.2. Checkstyle

- Minimal-Konfiguration für **Checkstyle** (Coding Conventions, Metriken)

Nutzen Sie die folgende **Minimal-Konfiguration** für **Checkstyle** für Ihre Praktikumsaufgaben. Diese beinhaltet die Prüfung der wichtigsten Formate nach Google Java Style/AOSP sowie der obigen Metriken. Halten Sie diese Regeln ein.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE module PUBLIC "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN" "
 https://checkstyle.org/dtds/configuration_1_3.dtd">
3
4 <module name="Checker">
5 <property name="severity" value="warning"/>
6
7 <module name="TreeWalker">
8 <module name="JavaNCSS">
9 <property name="methodMaximum" value="40"/>
10 <property name="classMaximum" value="250"/>
11 <property name="fileMaximum" value="300"/>
12 </module>
13 <module name="BooleanExpressionComplexity"/>
14 <module name="CyclomaticComplexity">
15 <property name="max" value="7"/>
16 </module>
17 </module>
18 </module>
```

```

16 </module>
17 <module name="ClassDataAbstractionCoupling">
18 <property name="max" value="6"/>
19 </module>
20 <module name="MethodCount">
21 <property name="maxTotal" value="10"/>
22 <property name="maxPrivate" value="10"/>
23 <property name="maxPackage" value="10"/>
24 <property name="maxProtected" value="10"/>
25 <property name="maxPublic" value="10"/>
26 </module>
27 <module name="ParameterNumber">
28 <property name="max" value="3"/>
29 </module>
30 <module name="MethodLength">
31 <property name="max" value="40"/>
32 </module>
33 <module name="Indentation">
34 <property name="basicOffset" value="4"/>
35 <property name="lineWrappingIndentation" value="8"/>
36 <property name="caseIndent" value="4"/>
37 <property name="throwsIndent" value="4"/>
38 <property name="arrayInitIndent" value="4"/>
39 </module>
40 <module name="TypeName"/>
41 <module name="MethodName"/>
42 <module name="MemberName"/>
43 <module name="ParameterName"/>
44 <module name="ConstantName"/>
45 <module name="OneStatementPerLine"/>
46 <module name="MultipleVariableDeclarations"/>
47 <module name="MissingOverride"/>
48 <module name="MissingJavadocMethod"/>
49 <module name="AvoidStarImport"/>
50 </module>
51
52 <module name="LineLength">
53 <property name="max" value="100"/>
54 </module>
55 <module name="FileTabCharacter">
56 <property name="eachLine" value="true"/>
57 </module>
58 <module name="NewlineAtEndOfFile"/>
59 </module>

```

Sie können diese Basis-Einstellungen auch aus dem Programmiermethoden-CampusMinden/Prog2-Lecture-Repo direkt herunterladen: [checkstyle.xml](#).

Sie können zusätzlich gern noch die weiteren (und strengeren) Regeln aus der vom Checkstyle-Projekt bereitgestellten Konfigurationsdatei für den [Google Java Style](#) nutzen. *Hinweis:* Einige der dort konfigurierten Checkstyle-Regeln gehen allerdings über den Google Java Style hinaus.

### 9.2.8.3. Linter: SpotBugs

- Vermeiden von Anti-Pattern mit **SpotBugs**

Setzen Sie zusätzlich **SpotBugs** mit ein. Ihre Lösungen dürfen keine Warnungen oder Fehler beinhalten, die SpotBugs melden würde.

## 9.2.9. Coding Conventions: Einige zentrale Grundsätze

### 9.2.9.1. Kommentieren vs. sauberen Code schreiben

- Kommentare ersetzen keinen guten Code
- Kommentare erklären **Warum**, nicht *Was* (Das “Was” steht im Code selbst)
- Keine “Selbstbeschreibungen” wie `x++; //erhöht x um 1`

### 9.2.9.2. Typische Regeln für den Einstieg

- Keine “magischen Zahlen” → Konstanten (UPPER\_SNAKE\_CASE)
- Methoden sollten eine klar erkennbare Aufgabe haben (Verweis auf Single Responsibility)
- Möglichst wenige **public**-Member, Kapselung betonen

## 9.2.10. Wrap-Up

- Code entsteht nicht zum Selbstzweck => Regeln nötig!
  - Coding Conventions
    - \* Regeln zu Schreibweisen und Layout
    - \* Leerzeichen, Einrückung, Klammern
    - \* Zeilenlänge, Umbrüche
    - \* Kommentare
  - Prinzipien des objektorientierten Programmierens (vgl. “Code Smells”)
    - \* Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
    - \* Keine Code-Duplizierung! (*DRY* - Don’t repeat yourself)
    - \* Klassen und Methoden sollten sich erwartungsgemäß verhalten
    - \* Kapselung: Möglichst wenig öffentlich zugänglich machen
- Formatieren mit **Spotless**
- Stil & Metriken: Prüfung manuell durch Code Reviews oder durch Tools wie **Checkstyle**
- Bugs & Anti-Pattern: Prüfung manuell durch Code Reviews oder durch Tools wie **SpotBugs**

**Zum Nachlesen**

Der Klassiker zu diesem Thema ist sicher Martin (2009).

**Wichtig:** Bitte nicht als “Gesetz” betrachten, sondern als starke *Empfehlung*. Die formulierten Grenzwerte sollten zumindest zum Nachdenken über das eigene Tun anregen.

Der [Google Java Style](#) ist ebenfalls ein guter Startpunkt, da viele Java-Projekte diesen Standard anwenden und es eine relativ gute Tooling-Unterstützung gibt.

**Lernziele**

- k2: Ich kann verschiedene Coding Conventions erklären
- k2: Ich kann die Metriken NCSS, McCabe, BEC, DAC erklären
- k3: Ich kann das Tool Spotless zur Formatierung des Codes nutzen
- k3: Ich kann das Tool Checkstyle zum Überprüfen von Coding Conventions und Metriken nutzen
- k2: Ich kenne das Tool SpotBugs zum Vermeiden von Anti-Pattern

## 9.3. Refactoring

**TL;DR**

Refactoring bedeutet Änderung der inneren Struktur des Codes ohne Beeinflussung äußeren Verhaltens. Mit Hilfe von Refactoring kann man Code Smells beheben, und Lesbarkeit, Verständlichkeit und Wartbarkeit von Software verbessern.

Es ist wichtig, immer nur einzelne Schritte zu machen und anschließend die Testsuite laufen zu lassen, damit nicht versehentlich Fehler oder Verhaltensänderungen beim Refactoring eingebaut werden.

Prinzipiell kann man Refactoring manuell mit Search&Replace durchführen, aber es bietet sich an, hier die IDE-Unterstützung zu nutzen. Es stehen verschiedene Methoden zur Verfügung, die nicht unbedingt einheitlich benannt sein müssen oder in jeder IDE vorkommen. Zu den häufig genutzten Methoden zählen *Rename*, *Extract*, *Move* und *Pull Up/Push Down*.

**Videos**

Vorlesung [[YT](#)], [[HSBI](#)]

Demo:

- Rename [[YT](#)], [[HSBI](#)]
- Encapsulate [[YT](#)], [[HSBI](#)]
- Extract Method [[YT](#)], [[HSBI](#)]
- Move Method [[YT](#)], [[HSBI](#)]
- Pull up [[YT](#)], [[HSBI](#)]

### 9.3.1. Was ist Refactoring?



Refactoring ist, wenn einem auffällt, daß der Funktionsname `foobar` ziemlich bescheuert ist, und man die Funktion in `sinus` umbenennt.

Quelle: "356: Refactoring" by Andreas Bogk on Lutz Donnerhacke: "Fachbegriffe der Informatik"

Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behaviour.

Quelle: "Refactoring": (Fowler 2011, p. 53)

**Refactoring:** Änderungen an der **inneren Struktur** einer Software

- Beobachtbares (äußeres) Verhalten ändert sich dabei **nicht**
  - Keine neuen Features einführen
  - Keine Bugs fixen
  - Keine öffentliche Schnittstelle ändern (*Anmerkung:* Bis auf Umbenennungen oder Verschiebungen von Elementen innerhalb der Software)
- Ziel: Verbesserung von Verständlichkeit und Änderbarkeit

**Tip**

*Anmerkung:* In der Praxis können natürlich Bugfix + Refactoring in einem Pull-Request zusammen auftreten, wenn das inhaltlich sinnvoll ist, aber begrifflich sollten wir es trennen.

### 9.3.2. Anzeichen, dass Refactoring jetzt eine gute Idee wäre

- Code “stinkt” (zeigt/enthält *Code Smells*)

Code Smells sind strukturelle Probleme, die im Laufe der Zeit zu Problemen führen können. Refactoring ändert die innere Struktur des Codes und kann entsprechend genutzt werden, um die Smells zu beheben.

- Schwer erklärbarer Code

Könnten Sie Ihren Code ohne Vorbereitung in der Abgabe erklären? In einer Minute? In fünf Minuten? In zehn? Gar nicht?

In den letzten beiden Fällen sollten Sie definitiv über eine Vereinfachung der Strukturen nachdenken.

- Verständnisprobleme, Erweiterungen

Sie grübeln in der Abgabe, was Ihr Code machen sollte?

Sie überlegen, was Ihr Code bedeutet, um herauszufinden, wo Sie die neue Funktionalität anbauen können?

Sie suchen nach Codeteilen, finden diese aber nicht, da die sich in anderen (falschen?) Stellen/Klassen befinden?

Nutzen Sie die (neuen) Erkenntnisse, um den Code leichter verständlich zu gestalten.

“Three strikes and you refactor.”

Quelle: “Three strikes...”: (Fowler 2011, p. 58)

Wenn Sie sich zum dritten Mal über eine suboptimale Lösung ärgern, dann werden Sie sich vermutlich noch öfter darüber ärgern. Jetzt ist der Zeitpunkt für eine Verbesserung.

Schauen Sie sich die entsprechenden Kapitel in (Passig und Jander 2013) und (Fowler 2011) an, dort finden Sie noch viele weitere Anhaltspunkte, ob und wann Refactoring sinnvoll ist.

### 9.3.3. Bevor Sie loslegen ...

#### 1. **Unit Tests** schreiben

- Normale und ungültige Eingaben
- Rand- und Spezialfälle

#### 2. **Coding Conventions** einhalten

- Sourcecode formatieren (lassen)

#### 3. **Haben Sie die fragliche Codestelle auch wirklich verstanden?!**

### 9.3.4. Vorgehen beim Refactoring

#### 9.3.4.1. Überblick über die Methoden des Refactorings

Die Refactoring-Methoden sind nicht einheitlich definiert, es existiert ein großer und uneinheitlicher “Katalog” an möglichen Schritten. Teilweise benennt jede IDE die Schritte etwas anders, teilweise werden unterschiedliche Möglichkeiten angeboten.

Zu den am häufigsten genutzten Methoden zählen

- Rename Method/Class/Field
- Encapsulate Field
- Extract Method/Class
- Move Method
- Pull Up, Push Down (Field, Method)

#### 9.3.4.2. Best Practice

Eine Best Practice (oder nennen Sie es einfach eine wichtige Erfahrung) ist, beim Refactoring langsam und gründlich vorzugehen. Sie ändern die Struktur der Software und können dabei leicht Fehler oder echte Probleme einbauen. Gehen Sie also langsam und sorgsam vor, machen Sie einen Schritt nach dem anderen und sichern Sie sich durch eine gute Testsuite ab, die Sie nach jedem Schritt erneut ausführen: Das Verhalten der Software soll sich ja nicht ändern, d.h. die Tests müssen nach jedem einzelnen Refactoring-Schritt immer grün sein (oder Sie haben einen Fehler gemacht).

- Kleine Schritte: immer nur **eine** Änderung zu einer Zeit
- Nach **jedem** Refactoring-Schritt **Testsuite** laufen lassen  
=> Nächster Refactoring-Schritt erst, wenn alle Tests wieder “grün”
- Versionskontrolle nutzen: **Jeden** Schritt **einzel**n committen

#### Tip

Refactoring ist nicht: Alles neu schreiben.

Refactoring ist: Viele kleine, zielgerichtete, durch Tests abgesicherte Umbauten an der Struktur (nicht Funktion).

### 9.3.5. Refactoring-Methode: Rename Method/Class/Field

#### 9.3.5.1. Motivation

Name einer Methode/Klasse/Attributs erklärt nicht ihren Zweck.

#### 9.3.5.2. Durchführung

Name selektieren, “**Refactor** > **Rename**”

### 9.3.5.3. Anschließend ggf. prüfen

Aufrufer? Superklassen?

### 9.3.5.4. Beispiel

**Vorher**

```
1 public String getTeN() {}
```

**Nachher**

```
1 public String getTelefonNummer() {}
```

## 9.3.6. Refactoring-Methode: Encapsulate Field

### 9.3.6.1. Motivation

Sichtbarkeit von Attributen reduzieren.

### 9.3.6.2. Durchführung

Attribut selektieren, “[Refactor](#) > [Encapsulate Field](#)”

### 9.3.6.3. Anschließend ggf. prüfen

Superklassen? Referenzen? (Neue) JUnit-Tests?

### 9.3.6.4. Beispiel

**Vorher**

```
1 int cps;
2
3 public void printDetails() {
4 System.out.println("Credits: " + cps);
5 }
```

**Nachher**

```
1 private int cps;
2
3 int getCps() { return cps; }
4 void setCps(int cps) { this.cps = cps; }
5
6 public void printDetails() {
```

```
7 System.out.println("Credits: " + getCps());
8 }
```

### 9.3.7. Refactoring-Methode: Extract Method/Class

#### 9.3.7.1. Motivation

- Codefragment stellt eigenständige Methode dar
- “Überschriften-Code”
- Code-Duplizierung
- Code ist zu “groß”
- Klasse oder Methode erfüllt unterschiedliche Aufgaben

#### 9.3.7.2. Durchführung

Codefragment selektieren, “**Refactor** > **Extract Method**” bzw. “**Refactor** > **Extract Class**”

#### 9.3.7.3. Anschließend ggf. prüfen

- Aufruf der neuen Methode? Nutzung der neuen Klasse?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Speziell bei Methoden:
  - Nutzung lokaler Variablen: Übergabe als Parameter!
  - Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

#### 9.3.7.4. Beispiel

##### Vorher

```
1 public void printInfos() {
2 printHeader();
3 // Details ausgeben
4 System.out.println("name: " + name);
5 System.out.println("credits: " + cps);
6 }
```

##### Nachher

```
1 public void printInfos() {
2 printHeader();
3 printDetails();
4 }
5 private void printDetails() {
6 System.out.println("name: " + name);
7 }
```

```
7 System.out.println("credits: " + cps);
8 }
```

### 9.3.8. Refactoring-Methode: Move Method

#### 9.3.8.1. Motivation

Methode nutzt (oder wird genutzt von) mehr Eigenschaften einer fremden Klasse als der eigenen Klasse.

#### 9.3.8.2. Durchführung

Methode selektieren, “**Refactor** > **Move**” (ggf. “Keep original method as delegate to moved method” aktivieren)

#### 9.3.8.3. Anschließend ggf. prüfen

- Aufruf der neuen Methode (Delegation)?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Nutzung lokaler Variablen: Übergabe als Parameter!
- Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

#### 9.3.8.4. Beispiel

##### Vorher

```
1 public class Kurs {
2 int cps;
3 String descr;
4 }
5
6 public class Studi extends Person {
7 String name;
8 int cps;
9 Kurs kurs;
10
11 public void printKursInfos() {
12 System.out.println("Kurs: " + kurs.descr);
13 System.out.println("Credits: " + kurs.cps);
14 }
15 }
```

##### Nachher

```
1 public class Kurs {
2 int cps;
3 String descr;
```

```
4
5 public void printKursInfos() {
6 System.out.println("Kurs: " + descr);
7 System.out.println("Credits: " + cps);
8 }
9 }
10
11 public class Studi extends Person {
12 String name;
13 int cps;
14 Kurs kurs;
15
16 public void printKursInfos() { kurs.printKursInfos(); }
17 }
```

### 9.3.9. Refactoring-Methode: Pull Up, Push Down (Field, Method)

#### 9.3.9.1. Motivation

- Attribut/Methode nur für die Oberklasse relevant: **Pull Up**
- Subklassen haben identische Attribute/Methoden: **Pull Up**
- Attribut/Methode nur für eine Subklasse relevant: **Push Down**

#### 9.3.9.2. Durchführung

Name selektieren, “Refactor > Pull Up” oder “Refactor > Push Down”

#### 9.3.9.3. Anschließend ggf. prüfen

Referenzen/Aufrufer? JUnit-Tests?

#### 9.3.9.4. Beispiel

##### Vorher

```
1 public class Person { }
2
3 public class Studi extends Person {
4 String name;
5 public void printDetails() { System.out.println("name: " + name); }
6 }
```

##### Nachher

```
1 public class Person { protected String name; }
2
3 public class Studi extends Person {
4 public void printDetails() { System.out.println("name: " + name); }
```

```
5 }
```

### 9.3.10. Wrap-Up

Behebung von **Code Smells** durch **Refactoring**

=> Änderung der inneren Struktur ohne Beeinflussung des äußeren Verhaltens

- Verbessert Lesbarkeit, Verständlichkeit, Wartbarkeit
- Immer nur kleine Schritte machen
- Nach jedem Schritt Testsuite laufen lassen
- Katalog von Maßnahmen, beispielsweise *Rename*, *Extract*, *Move*, *Pull Up/Push Down*, ...
- Unterstützung durch IDEs wie Eclipse, Idea, ...

#### Zum Nachlesen

Zum Thema Refactoring gibt es den Klassiker Fowler (2011), aber auch gute Tutorial-Seiten wie [Refactoring Guru](#).

#### Lernziele

- k2: Ich kann den Begriff sowie die Notwendigkeit und das Vorgehen des/beim Refactoring erklären
- k2: Ich kann die Bedeutung kleiner Schritte beim Refactoring erklären
- k2: Ich kann die Bedeutung einer sinnvollen Testsuite beim Refactoring erklären
- k2: Ich habe verstanden, dass 'Refactoring' bedeutet: Nur die innere Struktur ändern, nicht das von außen sichtbare Verhalten!
- k3: Ich kann die wichtigsten Refactoring-Methoden anwenden: Rename, Extract, Move, Pull Up/Push Down

#### Challenges

Betrachten Sie das [Theatrical Players Refactoring Kata](#). Dort finden Sie im Unterordner `java/` einige Klassen mit unübersichtlichem und schlecht strukturierten Code.

Welche *Code Smells* können Sie hier identifizieren?

Beheben Sie die Smells durch die *schrittweise Anwendung* von den aus der Vorlesung bekannten Refactoring-Methoden. Denken Sie auch daran, dass Refactoring immer durch eine entsprechende Testsuite abgesichert sein muss - ergänzen Sie ggf. die Testfälle.

#### Real-Life Refactoring

Betrachten Sie das folgende Code-Beispiel, welches aus dem [Dungeon-Projekt](#) stammt:

```
1 public void shootFireBall() {
2 AmmunitionComponent heroAC =
3 hero.fetch(AmmunitionComponent.class)
4 .orElseThrow(() -> MissingComponentException.build(hero,
5 AmmunitionComponent.class));
6 if (!heroAC.checkAmmunition()) return;
```



```

6 utils.Direction viewDirection =
7 convertPosCompDirectionToUtilsDirection(EntityUtils.
 getViewDirection(hero));
8 Skill fireball =
9 new Skill(
10 new FireballSkill(
11 () -> {
12 CollideComponent collider =
13 hero.fetch(CollideComponent.class)
14 .orElseThrow(
15 () -> MissingComponentException.build(hero,
16 CollideComponent.class));
16 Point start = collider.center(hero);
17 return start.add(new Point(viewDirection.x(), viewDirection
18 .y()));
19 },
20 1);
21 fireball.execute(hero);
22 heroAC.spendAmmo();
23 waitDelta();
24 }

```

1. Analysieren Sie den Code und versuchen Sie zu verstehen, was hier passiert. Welche Typen müssen die unterschiedlichen Operationen haben (Parametertypen, Rückgabetypen)?
2. Welche Funktionseinheiten können Sie identifizieren? Entwickeln Sie Ideen, wie dieser Code zu mittels Refactoring lesbarer und verständlicher gemacht werden kann. Nutzen Sie dabei möglichst geschickt u.a. die Java-Stream-API und Optionals und überlegen Sie, welche der Exceptions wirklich relevant sind ...
3. Kopieren Sie den Code in einen Editor (mit Syntaxunterstützung - die IDE geht auch, wird aber wegen der fehlenden Klassen und Variablen keine wirkliche Hilfe sein). Führen Sie das Refactoring "zu Fuß" durch. Vergleichen Sie Ihr Ergebnis und den ursprünglichen Code.

(Sie brauchen im Dungeon-Projekt nicht zu suchen, diese Code-Stelle ist längst bereinigt ...)

## 9.4. Javadoc

### TL;DR

Mit Javadoc kann aus speziell markierten Block-Kommentaren eine externe Dokumentation im HTML-Format erzeugt werden. Die Block-Kommentare, auf die das im JDK enthaltene Programm `javadoc` reagiert, beginnen mit `/**` (also einem zusätzlichen Stern, der für den Java-Compiler nur das erste Kommentarzeichen ist).

Die erste Zeile eines Javadoc-Kommentars ist eine "Zusammenfassung" und an fast allen Stellen der generierten Doku sichtbar. Diese Summary sollte kurz gehalten werden und eine Idee vermitteln, was die Klasse oder die Methode oder das Attribut macht.

Für die Dokumentation von Parametern, Rückgabetypen, Exceptions und veralteten Elementen existieren spezielle Annotationen: `@param`, `@return`, `@throws` und `@deprecated`.

Als Faustregel gilt: Es werden **alle** **public** und **protected** Elemente (Klassen, Methoden, Attribute) mit Javadoc kommentiert. Alle nicht-öffentlichen Elemente bekommen normale Java-Kommentare (Zeilen- oder Blockkommentare).

#### Videos

- [VL Javadoc](#)

### 9.4.1. Dokumentation mit Javadoc

```
1 /**
2 * Beschreibung Beschreibung (Summary).
3 *
4 * <p>Hier kommt dann ein laengerer Text, der die Dinge
5 * bei Bedarf etwas ausfuehrlicher erklart.
6 */
7 public void wuppie() {}
```

Javadoc-Kommentare sind (aus Java-Sicht) normale Block-Kommentare, wobei der Beginn mit `/**` eingeleitet wird. Dieser Beginn ist für das Tool `javadoc` (Bestandteil des JDK, genau wie `java` und `javac`) das Signal, dass hier ein Kommentar anfängt, den das Tool in eine HTML-Dokumentation übersetzen soll.

Typischerweise wird am Anfang jeder Kommentarzeile ein `*` eingefügt; dieser wird von Javadoc ignoriert.

Sie können neben normalem Text und speziellen Annotationen auch HTML-Elemente wie `<p>` und `<code>` oder `<ul>` nutzen.

Mit `javadoc *.java` können Sie in der Konsole aus den Java-Dateien die Dokumentation generieren lassen. Oder Sie geben das in Ihrer IDE in Auftrag ... (die dann diesen Aufruf gern für Sie tätigt).

### 9.4.2. Standard-Aufbau

```
1 /**
2 * Beschreibung Beschreibung (Summary).
3 *
4 * <p> Hier kommt dann ein laengerer Text, der die Dinge
5 * bei Bedarf etwas ausfuehrlicher erklart.
6 *
7 * @param date Tag, Wert zw. 1 .. 31
8 * @return Anzahl der Sekunden seit 1.1.1970
9 * @throws NumberFormatException
10 * @deprecated As of JDK version 1.1
11 */
12 public int setDate(int date) {
13 setField(Calendar.DATE, date);
14 }
```

- Erste Zeile bei Methoden/Attributen geht in die generierte “Summary” in der Übersicht, der Rest in die “Details”

- Die “Summary” sollte kein kompletter Satz sein, wird aber wie ein Satz geschrieben (Groß beginnen, mit Punkt beenden). Es sollte nicht beginnen mit “Diese Methode macht ...” oder “Diese Klasse ist ...”. Ein gutes Beispiel wäre “Berechnet die Steuerrückerstattung.”
- Danach kommen die Details, die in der generierten Dokumentation erst durch Aufklappen der Elemente sichtbar sind. Erklären Sie, wieso der Code was machen soll und welche Designentscheidungen getroffen wurden (und warum).
- Leerzeilen gliedern den Text in Absätze. Neue Absätze werden mit einem `<p>` eingeleitet. (Ausnahmen: Wenn der Text mit `<ul>` o.ä. beginnt oder der Absatz mit den Block-Tags.)
- Die “Block-Tags” `@param`, `@return`, `@throws`, `@deprecated` werden durch einen Absatz von der restlichen Beschreibung getrennt und tauchen in exakt dieser Reihenfolge auf. Die Beschreibung dieser Tags ist nicht leer - anderenfalls lässt man das Tag weg. Falls die Zeile für die Beschreibung nicht reicht, wird umgebrochen und die Folgezeile mit vier Leerzeichen (beginnend mit dem `@`) eingerückt.
  - Mit `@param` erklären Sie die Bedeutung eines Parameters (von links nach rechts) einer Methode. Beispiel: `@param date Tag, Wert zw. 1 .. 31.` Wiederholen Sie dies für jeden Parameter.
  - Mit `@return` beschreiben Sie den Rückgabetypp/-wert. Beispiel: `@return Anzahl der Sekunden seit 1.1.1970.` Bei Rückgabe von `void` wird diese Beschreibung weggelassen (die Beschreibung wäre dann ja leer).
  - Mit `@throws` geben Sie an, welche “checked” Exceptions die Methode wirft.
  - Mit `@deprecated` können Sie im Kommentar sagen, dass ein Element veraltet ist und möglicherweise mit der nächsten Version o.ä. entfernt wird. (siehe nächste Folie)

=> Dies sind die Basis-Regeln aus dem populären [Google-Java-Style](#).

### 9.4.3. Veraltete Elemente

```

1 /**
2 * Beschreibung Beschreibung Beschreibung.
3 *
4 * @deprecated As of v102, replaced by <code>Foo.fluppie()</code>.
5 */
6 @Deprecated
7 public void wuppie() {}

```

- Annotation zum Markieren als “veraltet” (in der generierten Dokumentation): `@deprecated`
- Für Sichtbarkeit zur Laufzeit bzw. im Tooling/IDE: normale Code-Annotation `@Deprecated`

Dies ist ein guter Weg, um Elemente einer öffentlichen API als “veraltet” zu kennzeichnen. Üblicherweise wird diese Kennzeichnung für einige wenige Releases beibehalten und danach das veraltete Element aus der API entfernt.

### 9.4.4. Autoren, Versionen, ...

```

1 /**
2 * Beschreibung Beschreibung Beschreibung.

```

```
3 *
4 * @author Dagobert Duck
5 * @version V1
6 * @since schon immer
7 */
```

- Annotationen für Autoren und Version: `@author`, `@version`, `@since`

Diese Annotationen finden Sie vor allem in Kommentaren zu Packages oder Klassen.

### 9.4.5. Was muss kommentiert werden?

- Alle **public** Klassen
- Alle **public** und **protected** Elemente der Klassen
- Ausnahme: `@Override` (An diesen Methoden *kann*, aber *muss* nicht kommentiert werden.)

Alle anderen Elemente bei Bedarf mit *normalen* Kommentaren versehen.

#### 9.4.5.1. Beispiel aus dem JDK: ArrayList

Schauen Sie sich gern mal Klassen aus der Java-API an, beispielsweise eine `java.util.ArrayList`:

- Generierte Dokumentation: zu “ArrayList” [runterscrollen](#) bzw. [direkt](#)
- Quellcode: [ArrayList.java](#)

#### 9.4.5.2. Best Practices: Was beschreibe ich eigentlich?

Unter [Documentation Best Practices](#) finden Sie eine sehr gute Beschreibung, was das Ziel der Dokumentation sein sollte. Versuchen Sie, dieses zu erreichen!

Etwas technisch, aber ebenfalls sehr lesenswert ist der Style-Guide für Java-Software [How to Write Doc Comments for the Javadoc Tool](#) von Oracle.

### 9.4.6. Wrap-Up

- Javadoc-Kommentare sind normale Block-Kommentare beginnend mit `/**`
- Generierung der HTML-Dokumentation mit `javadoc *.java`
- Erste Zeile ist eine Zusammenfassung (fast immer sichtbar)
- Längerer Text danach als “Description” einer Methode/Klasse
- Annotationen für besondere Elemente: `@param`, `@return`, `@throws`, `@deprecated`
- Faustregel: Alle **public** und **protected** Elemente mit Javadoc kommentieren!

**Zum Nachlesen**

Schauen Sie sich den [Google-Java-Style](#) an. Dieser ist sehr ausführlich dokumentiert.

**Lernziele**

- k2: Ich verstehe den Sinn von Javadoc-Kommentaren
- k2: Ich kenne den typischen Aufbau von Javadoc-Kommentaren
- k3: Ich kann sämtliche öffentlich sichtbaren Elemente mit Javadoc dokumentieren
- k3: Ich kann eine sinnvolle Summary schreiben
- k3: Ich kann verschiedene Annotationen zur Dokumentation von Parametern, Rückgabetypen, Exceptions, veralteten Elementen einsetzen
- k3: Ich kann die Dokumentation generieren

**Challenges**

Betrachten Sie die Javadoc einiger Klassen im Dungeon-Projekt: [dojo.rooms.LevelRoom](#), [dojo.rooms.MonsterRoom](#), und [contrib.components.HealthComponent](#).

Stellen Sie sich vor, Sie müssten diese Klassen in einer Übungsaufgabe nutzen (das könnte tatsächlich passieren!) ...

Können Sie anhand der Javadoc verstehen, wozu die drei Klassen dienen und wie Sie diese Klassen benutzen sollten? Vergleichen Sie die Qualität der Dokumentation. Was würden Sie gern in der Dokumentation finden? Was würden Sie ändern?

# **Teil III.**

# **Praktikum**

Hier finden Sie die Übungsblätter.

# 10. Blatt 01: Git Basics, Gradle

## 10.1. Zusammenfassung

Auf diesem Blatt üben Sie den Umgang mit Git (Repo und Commits - zunächst auf der Konsole) und das Schreiben von Gradle-Build-Skripten.

## 10.2. Aufgaben

### 10.2.1. Git

#### 10.2.1.1. Aufgabe 1.1: Git Status erklären

Betrachten Sie die folgende Ausgabe von `git status` in einer lokalen Workingcopy (*Arbeitskopie*):

```
1 pm-lecture % git status
2 On branch b03
3
4 Changes not staged for commit:
5 (use "git add <file>..." to update what will be committed)
6 (use "git restore <file>..." to discard changes in working directory)
7 modified: CONTRIBUTING.md
8 modified: homework/b03.md
9
10 Untracked files:
11 (use "git add <file>..." to include in what will be committed)
12 foo.java
13
14 no changes added to commit (use "git add" and/or "git commit -a")
```

Erklären Sie die Ausgabe.

Geben Sie eine Befehlssequenz an, mit der Sie nur die Änderungen in `foo.java` committen können.

#### 10.2.1.2. Aufgabe 1.2: Git-Spiel

Klonen Sie die [Vorgaben "Git-Quest"](#). Sie finden die Geschichte des Helden Markus im Dungeon.<sup>1</sup>

---

<sup>1</sup>Für alle, die schon mit Branches umgehen können: Betrachten Sie auf diesem Blatt bitte nur den Branch `master`.



1. Öffnen Sie eine Konsole und beantworten Sie mit Hilfe der Befehle `git checkout`, `git log` und `git show` sowie `git diff` folgende Fragen:
  - Was passierte an `tag 01`?
  - Wann hat der Held zum ersten Mal 4 `experience` Punkte?
  - Wann hat der Held zum ersten Mal 10 `hunger` Punkte?
  - Wie viele Heiltränke hat der Held insgesamt in seinem Rucksack gehabt?
  - Was hat der Held im Shop gekauft? Und wie viel Gold hat er dafür bezahlt?
  - Was passierte zwischen `tag 03` und `tag 04`, d.h. was änderte sich zwischen diesen Commits?
  - Hat der Held etwas gegessen? Falls ja, was und wann?
2. Beim letzten Commit (`tag 04.5`) ist etwas schief gelaufen, es wurden versehentlich zu wenig `experience` Punkte eingestellt. Ändern Sie diesen letzten Commit und passen Sie die `experience` Punkte auf 42 an.
3. Schreiben Sie die Geschichte in der Datei `questlog.md` fort und erzeugen Sie einen neuen Commit für `tag 04.6`. Ändern Sie bitte hierzu nur die eine Datei `questlog.md`.
4. Schreiben Sie die Geschichte noch weiter fort (`tag 04.7`), aber ändern Sie diesmal mehrere Dateien, die an diesem Tag (neuer Commit) gemeinsam eingchecked werden sollen.
5. Fälschlicherweise wurden die Statuspunkte und die Ausrüstung bisher gemeinsam in der Datei `stats.md` geführt. Korrigieren Sie das und verschieben Sie die Ausrüstungsgegenstände aus der Datei `stats.md` in eine neue Datei `gear.md`. Checken Sie Ihre Änderungen als `tag 04.8` (neuer Commit) gemeinsam ein. (*Hinweis:* Es reicht, wenn diese Änderung als letzter Commit auf der Spitze des `master`-Branches existiert. Sie brauchen/sollen die Trennung von Statuspunkten und Ausrüstung **nicht rückwirkend** in die Historie einbauen!)

Demonstrieren Sie Ihr Vorgehen im Praktikum jeweils live.

### 10.2.1.3. Aufgabe 1.3: Commit-Meldungen

Gute Commit-Meldungen schreiben erfordert Übung. Schauen Sie sich die beiden Commits [Dungeon-CampusMinden/Dungeon/commit/46530b6](#) und [Dungeon-CampusMinden/Dungeon/commit/3e37472](#) an.

Diskutieren Sie jeweils, was Ihnen an den Commits auffällt: Was gefällt Ihnen, was stört Sie? Schlagen Sie Verbesserungen vor.

## 10.2.2. Installation der Tools für Prog2

### 10.2.2.1. Aufgabe 2.1: Installation JDK

Sie benötigen für die Bearbeitung der Übungsaufgaben ein *Java Development Kit* (JDK). Wir verwenden in der Lehrveranstaltung "Programmieren 2" aus verschiedenen Gründen die aktuelle *Long-Term Support* (LTS)-Variante, d.h. derzeit das "Java SE Development Kit 25 (LTS)" (JDK 25).

Sofern noch nicht geschehen, installieren Sie bitte auf Ihrem Rechner **Java SE 25 (LTS)** in einer *64-bit Version*. Wenn Sie mehrere JDKs installiert haben sollten, stellen Sie bitte sicher, dass Sie für “Programmieren 2” tatsächlich das Java SE 25 (LTS) verwenden. Der Anbieter des JDKs sollte keine Rolle spielen.

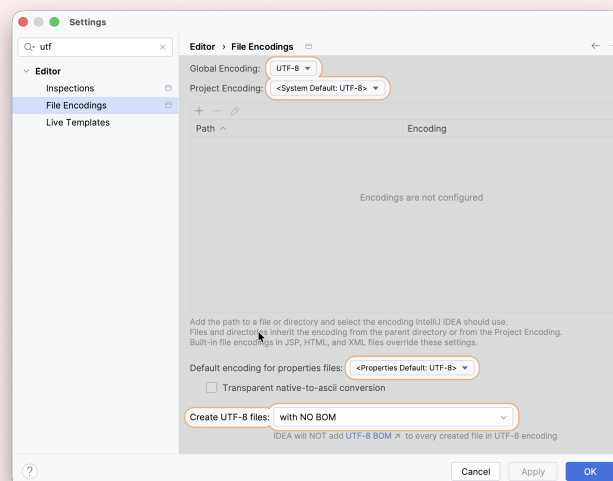
#### 10.2.2.2. Aufgabe 2.2: Installation IDE

Installieren Sie auf Ihrem Rechner eine IDE für Java Ihrer Wahl, empfohlen sind derzeit [IntelliJ IDEA \(Community Edition\)](#) und [Eclipse IDE for Java Developers](#).

Machen Sie sich mit der Arbeitsweise Ihrer IDE vertraut. Wenn Sie im Verlauf des Praktikums feststellen, dass die gewählte IDE nicht für Sie gemacht ist, können Sie jederzeit auf eine andere IDE wechseln.

##### Caution

**Windows-User:** Im Programmierumfeld, insbesondere für Source- und Konfigurations-Dateien, ist das Encoding **UTF-8** de facto Standard. D.h. wenn Sie beispielsweise ein Repo klonen, werden die Dateien im UTF-8-Encoding heruntergeladen. Sorgen Sie in den Einstellungen Ihrer IDE dafür, dass diese als Projekt-Encoding (und Datei-Encoding) UTF-8 nutzt:



Der Rest der Welt ist überwiegend auf UTF-8 umgestiegen, nur Windows nutzt scheinbar immer noch Windows-1252 oder Cp1252.

#### 10.2.2.3. Aufgabe 2.3: Deaktivierung GenAI-Support

Da Sie das Programmierhandwerk erlernen und vertiefen sollen, sollten Sie im Rahmen dieser Lehrveranstaltung keine GenAI-gestützten Assistenten benutzen. Das Durchschauen einer generierten Lösung ist nicht annähernd dasselbe wie das aktive Arbeiten an einem Problem und das Ringen um eine Lösung. Sie tun sich keinen Gefallen, wenn Sie hier abkürzen.

Bitte schalten deshalb Sie sämtliche GenAI-Unterstützung wie beispielsweise Claude, Copilot, JetBrains AI Assistant, Cursor, CodeGPT, Codeium, Tabnine, Windsurf, ... (Liste nicht vollständig) für die Bearbeitung der Übungsaufgaben in dieser Lehrveranstaltung ab.

#### 10.2.2.4. Aufgabe 2.4: Gradle

Folgen Sie der Anleitung auf [gradle.org](https://gradle.org) und installieren Sie Gradle auf Ihrem Rechner. Legen Sie in der Konsole ein neues Gradle-Projekt für eine Java-Applikation an (ohne IDE!). Das Build-Script soll in Groovy erzeugt und als Test-API soll JUnit (Version 6) verwendet werden.

Wie finden Sie auf der Konsole heraus, welche Tasks es gibt? Erklären Sie das Projektlayout, d.h. wo kommen beispielsweise die Java-Dateien hin?

Erklären Sie, in welche Abschnitte das generierte Buildskript unterteilt ist und welche Aufgaben diese Abschnitte jeweils erfüllen. Gehen Sie dabei im *Detail* auf das Plugin `application` und die dort bereitgestellten Tasks und deren Abhängigkeiten untereinander ein.

Öffnen Sie das Projekt in Ihrer IDE. Wie können Sie hier die verschiedenen Tasks ansteuern?

Machen Sie sich Notizen, welche Sie im Praktikum nutzen dürfen, um dort das Buildskript zu erklären.

#### 10.2.2.5. Aufgabe 2.5: SSH-Keys zur Authentifikation bei GitHub/GitLab

Wenn Sie Repos klonen und in Zukunft Änderungen auch wieder auf den Server zurück pushen wollen, nutzen Sie am besten das SSH-Protokoll (also die “`git@`”-URLs). Dieses arbeitet mit **SSH-Keys**, die man sich beispielsweise gezielt nur für die Authentifikation bei GitHub anlegen kann und den öffentlichen Schlüssel (*public key*) in den User-Einstellungen von GitHub registrieren kann: “[Generating a new SSH key and adding it to the ssh-agent](#)” und “[Adding a new SSH key to your GitHub account](#)” (bitte darauf achten, dass das richtige Betriebssystem ausgewählt ist).

Folgen Sie der Anleitung und erzeugen Sie sich ein Schlüsselpaar (privaten und öffentlichen Key) für die Authentifikation bei GitHub. Registrieren Sie den öffentlichen Schlüssel (*public key*) in den User-Einstellungen von GitHub (oder GitLab oder den von Ihnen präferierten Anbieter).

### 10.2.3. Aufgabe 3: Anlegen eines Java-Projektes als Basisprojekt für das Semester

Legen Sie in Ihrer IDE ein neues Java-Projekt mit Gradle-Support an.

Achten Sie bitte darauf, dass im Projektpfad **keine Leerzeichen** und **keine Sonderzeichen** (Umlaute o.ä.) vorkommen! Dies kann zu teilweise seltsamen Fehler führen.

Testen Sie bitte die genutzte Java-Version:

1. Konsole: Geben Sie den Befehl `java -version` auf der Konsole ein. Die Ausgabe sollte `openjdk version "25.0.2" 2026-01-20 LTS` (oder ähnlich) ergeben. Wichtig sind die “25” und “LTS”.

2. IDE: Erstellen Sie ein Programm, welches die Anweisung `IO.println(System.getProperty("java.version"))` ; ausführt. Beim Start über die IDE sollte dabei die Ausgabe `25.0.2` (oder ähnlich) herauskommen. Wichtig ist die "25".

Korrigieren Sie Ihr Setup, wenn Sie andere Ausgaben erhalten.

Achten Sie darauf, dass Ihre IDE tatsächlich auch mit Gradle baut und das Programm startet. Dies können Sie leicht überprüfen: Drücken Sie auf den "run"-Button (oft ein grüner Pfeil) und lassen Sie Ihr `main()` laufen. Erscheinen dabei zunächst die üblichen Gradle-Ausgaben auf der Konsole? Wenn nicht, passen Sie bitte die Einstellungen der IDE an.

Erweitern Sie nun Ihr Gradle-Setup und setzen Sie das Gradle-Plugin `Spotless` zum einheitlichen Formatieren Ihres Sources-Codes ein. Schreiben Sie ein kleine Demo mit einer `main()`-Methode und demonstrieren Sie die Wirkung von Spotless.

Nutzen Sie dieses (fast leere) Projekt als Startpunkt für die kommenden Aufgaben.

### 10.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem [im ILIAS](#): bis **04. Mai, 08:00 Uhr**
- Vorstellung im Praktikum: 04./06. Mai

# 11. Blatt 02: Git Branches, JUnit Basics; CI-Pipeline

## 11.1. Zusammenfassung

Auf diesem Blatt üben Sie den Umgang mit Git (Branches und Mergen - zunächst auf der Konsole) sowie das Erstellen erster JUnit-Tests. Weiterhin definieren Sie eine erste einfache CI-Pipeline für Ihr Repo.

## 11.2. Aufgaben

### 11.2.1. Aufgabe 1: Git-Spiel

Betrachten Sie erneut die [Vorgaben zur "Git-Quest"](#). Die Geschichte des Helden Markus findet im `master`-Branch kein Ende, sondern erst im Hilfsbranch `end`.

Machen Sie nun verschiedene Experimente mit Branches in Git, und starten Sie dabei jeweils mit einem frischen Klon der Vorgaben.

1. Ändern Sie eine Datei, die im Branch `end` nicht verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch.
2. Ändern Sie nun eine Datei, die auch im Branch `end` verändert wurde. Achten Sie dabei darauf, die Änderung an einer anderen Stelle in der Datei vorzunehmen. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch.
3. Wie (2), aber ändern Sie nun eine Stelle, die auch im Branch `end` verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem `master` einen neuen Commit. Mergen Sie danach den Branch `end` in den `master`-Branch. Was passiert, wenn die Änderung im `master` identisch zu der in `end` ist? Was passiert, wenn die Änderung im `master` anders ist als in `end`?
4. Wie (2), aber setzen Sie bitte den Branch `end` auf die Spitze von `master`, bevor Sie `end` in `master` mergen.

Was beobachten Sie jeweils? Erklären Sie Ihre Beobachtungen. Wenn es Konflikte gibt: Wie lösen Sie diese auf? Demonstrieren Sie das Vorgehen im Praktikum live.

### 11.2.2. Katzen-Café

Forken Sie das ["Cat-Cafe"](#)-Repo und erzeugen Sie sich eine lokale Arbeitskopie von Ihrem Fork.

### 11.2.2.1. Aufgabe 2.1: Gradle

Erstellen Sie eine Gradle-Konfiguration für das Java-Projekt und fügen Sie als externe Abhängigkeiten **JUnit** (Version 6.x) sowie das Gradle-Plugin **Spotless** hinzu. Passen Sie die Konfiguration so an, dass Sie

- `main()` in `catcafe.Main` über Gradle ausführen können, und
- JUnit-Tests mit Gradle ausführen können, und
- den Code mit `google-java-format` formatieren können. Können Sie für den Google-Java-Formatter die Einrückung auf 4 Leerzeichen (statt der Default 2 Leerzeichen) anpassen? Lange Strings sollen ebenfalls umgebrochen werden (falls nötig).

### 11.2.2.2. Aufgabe 2.2: JUnit

Erstellen Sie mit JUnit mindestens 10 unterschiedliche Testfälle für die Klasse `CatCafe`. Achten Sie auf den Aufbau der Testfälle und nutzen Sie das Muster “given - when - then”. Achten Sie auch auf passende Methodennamen.

Diskutieren Sie folgende Fragen:

- Warum sind die von Ihnen formulierten Testfälle relevant?
- Warum halten Sie die formulierten Testfälle für unterschiedlich?

### 11.2.3. Aufgabe 3: Remotes und CI-Pipeline

Erstellen Sie auf GitHub (oder einem anderen kostenlosen Git-Anbieter) ein öffentlich einsehbares Repo.

Verbinden Sie Ihr Repo auf dem Server mit Ihrem lokalen Repo und pushen Sie ihre Branches aus der Bearbeitung der vorigen Aufgabe “Katzen-Café” auf den Server.

Erstellen Sie eine einfache CI-Pipeline, die bei jedem Commit/Push auf dem Hauptbranch das Projekt

1. baut (kompiliert), und in einem zweiten Schritt
2. die JUnit-Tests aus der vorigen Aufgabe “Katzen-Café” ausführt, und in einem dritten Schritt
3. die korrekte Formatierung überprüft.

## 11.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem [im ILIAS](#): bis **11. Mai, 08:00 Uhr**
- Vorstellung im Praktikum: 11./13. Mai

## 12. Blatt 03: Methodenrefs, Lambdas, Observer

### 12.1. Zusammenfassung

Auf diesem Blatt üben Sie den Einsatz von Lambda-Ausdrücken und Methodenreferenzen. Sie modellieren das Observer-Pattern in einem kleinen Spiel.

### 12.2. Aufgaben

#### 12.2.1. Aufgabe 1: Calculator: Anonyme Klassen und Lambda-Ausdrücke

Klonen/forken Sie die [Vorgaben "Calculator"](#) und laden Sie das Projekt in Ihre IDE. Konfigurieren Sie Gradle für dieses Projekt.

Im Package `calculator` finden Sie einige Interfaces und Klassen, mit denen man einen einfachen Taschenrechner modellieren kann: Dieser kann einfache mathematische Operationen auf zwei Integern ausführen.

In der Klasse `calculator.Calculator` finden Sie vier mit `TODO` markierte Stellen in der Methode `setupOperationSelector`:

1. Erstellen Sie eine neue **Java-Klasse** `Sub`, die das Interface `Operation` implementiert und eine Subtraktion bereitstellt. Erweitern Sie den `Calculator` und binden Sie eine **Instanz dieser Klasse** ein. Nutzen Sie hier keine anonymen Klassen oder Lambda-Ausdrücke.
2. Erstellen Sie eine weitere Operation "Mul" (Multiplikation von zwei Integern). Nutzen Sie dazu eine passende **anonyme Klasse**.
3. Erstellen Sie eine weitere Operation "Div" (Integerdivision). Erstellen Sie einen passenden **Lambda-Ausdruck**.
4. Für die `JComboBox operationSelector` wird ein `ActionListener` mit Hilfe einer *anonymen Klasse* definiert. Konvertieren Sie dies in einen entsprechenden **Lambda-Ausdruck**.

#### 12.2.2. LockSnake: Lambda-Ausdrücke, Methodenreferenzen und Observer-Pattern

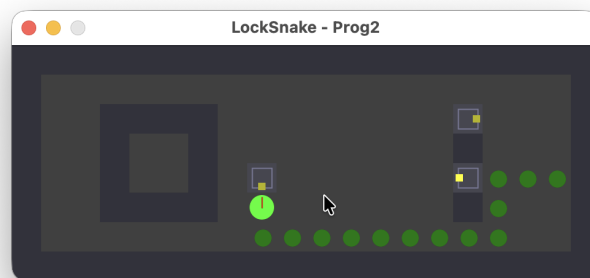
Klonen/forken Sie die [Vorgaben "LockSnake"](#) und laden Sie das Projekt in Ihre IDE. Konfigurieren Sie Gradle für dieses Projekt.

Ziel des Projekts ist es, ein kleines Swing-basiertes Spiel nach dem Vorbild von "[Mouse: P.I. For Hire Lockpicking Gameplay](#)" (16 s) zu implementieren.

Sie finden im Projekt ein lauffähiges Projektgerüst mit:

- einem lauffähigen Swing-Fenster (`Main`, `GamePanel`) inkl. Timer-Loop,
- einem fertigen Renderer (`Java2DRenderer`) sowie fertiger Level-Logik (`Level`, `LevelLoader`, Beispiellevel `src/main/resources/levels/level1.txt`),
- den Datentypen `CellType`, `Position`, `Direction`, `Pin` und `Snake`,
- einer leeren Modellierung für den Spielzustand `GameState` sowie einem leeren Spielmodell `GameEngine`.

Das fertige Spiel mit dem Beispiellevel könnte wie im folgenden Screenshot gezeigt aussehen:



(hier noch ein Link zu einem kurzen Video [\[YT\]](#)/[\[HSBI\]](#)).

#### 12.2.2.1. Aufgabe 2.1: Code-Analyse

Analysieren Sie die Vorgaben und erstellen Sie ein UML-Klassendiagramm, welches die Beziehungen zwischen den Klassen zeigt.

#### 12.2.2.2. Aufgabe 2.2: Spielmodell `GameEngine` und Spielzustand `GameState`

Ergänzen Sie das Spielmodell `GameEngine` und die Modellierung des Spielzustands `GameState` so, dass das Spiel spielbar ist und die Pin-Mechanik korrekt funktioniert. (Die mit `TODO` markierten Stellen sind ein guter Ausgangspunkt ...)

#### 12.2.2.3. Aufgabe 2.3: Observer-Pattern

1. Nutzen Sie das Observer-Pattern, so dass die GUI `GamePanel` automatisch aktualisiert wird, wenn sich der Spielzustand ändert.
2. Nutzen Sie das Observer-Pattern, so dass die `GameEngine` automatisch benachrichtigt wird und den Spielzustand aktualisieren kann, wenn eine konfigurierte Taste gedrückt wird.



#### 12.2.2.4. Aufgabe 2.4: JUnit

Schreiben Sie Unit-Tests mit JUnit 6 für Ihre Implementierung der Klasse `GameState`, um die Spielzustands-Logik zuverlässig zu validieren. Decken Sie dabei mindestens diese Kernfälle ab: Bewegungen, Pin-Interaktionen, Kollisionen/Blockaden, Gewinn-/Verlustzustände, Initialzustand.

Checkliste:

- Mindestens 10 gut dokumentierte Tests für `GameState`, wie oben beschrieben.
- Tests decken alle genannten Kernfälle ab (Wand, Pin blockiert, Pin aktivierbar, Selbstkollision, Gewinnbedingung, etc.).
- Tests verwenden klare Assertions, keine Off-by-One-Fehler, eindeutige Erwartungshaltung, sprechende Testnamen, "given-when-then"-Mantra.
- Tests bauen auf Level/Pin-Setup auf, sodass sie reproduzierbar sind (Test-Fixtures).

#### 12.2.2.5. Checkliste Abgabe

- Korrekte Behandlung von Wand / Pin blockiert / Pin aktivierbar
- Korrekte Behandlung von Selbstkollision
- Gewinnbedingung "alle Pins gesetzt"
- Mind. 10 JUnit-Tests für `GameState`
- Observer-Pattern sichtbar im Code:
  - UI (`GamePanel`) ist Observer für `GameState` in der `GameEngine`
  - `GameEngine` ist Observer für `Direction` (Tastatur-Events) im `GamePanel`
- Lösung enthält mindestens 3 Lambda-Ausdrücke
- Lösung enthält mindestens 2 Methodenreferenzen

#### 12.2.2.6. Bonus

Sie finden in den Vorgaben Skizzen für `TextureRenderer` und `MusicPlayer`. Implementieren Sie davon ausgehend einen Renderer, der mit Texturen arbeitet. Lassen Sie im Hintergrund passend zum Spielzustand Soundeffekte abspielen (weiterer Observer in `GameEngine`). Erstellen Sie weitere Levels und ermöglichen Sie eine Level-Auswahl oder ein Fortschreiten der Level, sobald die User das aktuelle Level gelöst haben.

#### 12.2.2.7. Spielregeln

##### Spielfeld/Level

Das Spielfeld besteht aus Zellen (Grid). Es gibt:

- **Wände** # (blockieren Bewegung),
- **leere Felder** . ,
- **Pin-Slots** (im Level durch Pfeile markiert, z.B. ^ v < >),

- eine **Startposition** **S** für die Snake.

(Symbole wie in der Level-Datei verwendet)

### Bewegung

- Die Spielschlange ("Snake") bewegt sich automatisch in Ticks.
- Pro Tick gilt: Es wird höchstens **ein** Schritt in Blickrichtung ausgeführt.
- Die Schlange erhält eine "Blickrichtung", wenn eine der Tasten "W A S D" oder "H J K L" oder die Pfeiltasten gedrückt werden.
- Die Blickrichtung wird als kleine rote "Nase" angezeigt.
- Die Blickrichtung bleibt erhalten, bis eine Tasteneingabe eine neue Blickrichtung setzt oder die Blickrichtung durch eine Blockade aufgelöst wird.
- Ohne gesetzte Blickrichtung passiert nichts.
- Die Schlange wächst bei jedem erfolgreich ausgeführten Schritt (der alte Kopf wird Teil des Körpers).

### Wände

- Eine Bewegung in eine Wand wird **verworfen** (Schlange bleibt stehen).

### Pins

Jeder Pin hat:

- eine Position,
- einen Zustand **LOW** (nicht gesetzt) oder **HIGH** (gesetzt/aktiviert),
- eine Aktivierungsrichtung (z.B. Pin < kann nur von links angestoßen werden).

Regeln:

- Wenn die Schlange **von der richtigen Richtung** an einen **LOW**-Pin "anstößt", wird der Pin auf **HIGH** gesetzt.
- In diesem Fall wird das Feld **nicht betreten** (Schlange bleibt auf der alten Position stehen).
- Ist der Pin bereits **HIGH** oder die Richtung falsch, blockiert er wie eine Wand.

### Selbstkollision und Ende

- Wenn die Schlange in ihren eigenen Körper laufen würde: **Game Over**.
- Wenn alle Pins **HIGH** sind: **Gewonnen** (oder nächstes Level).

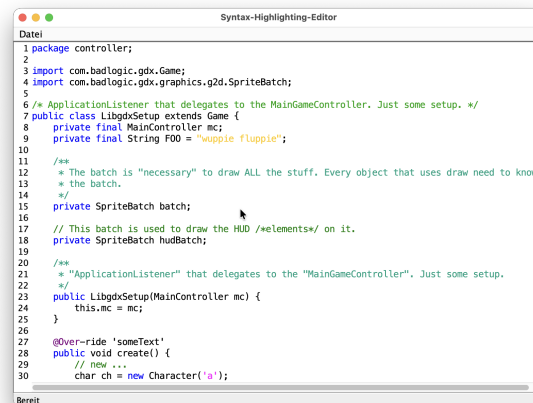
## 12.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem **im ILIAS**: bis **18. Mai, 08:00 Uhr**
- Vorstellung im Praktikum: 18./20. Mai

## 13. Blatt 04: RegEx, Template-Method; JUnit; PR

### 13.1. Zusammenfassung

Sie erhalten als Vorgabe ein kleines (fertig konfiguriertes) Java-Projekt zum [Syntax-Highlighting](#), das einen Editor auf Basis von [JTextPane](#) bereitstellt. Der Editor kann Text anzeigen, bearbeiten und farbige Hervorhebungen basierend auf sogenannten "Token-Treffern" ([HighlightRegion](#)) darstellen.



Die Hervorhebung wird über die abstrakte Klasse [SyntaxHighlighter](#) mit dem Template-Method-Pattern realisiert:

```
1 public abstract class SyntaxHighlighter {
2 public final List<HighlightRegion> computeRegions(String text) {
3 var candidates = collectMatches(text);
4 var normalized = normalize(candidates);
5 var resolved = resolveConflicts(normalized);
6 return resolved;
7 }
8
9 public abstract List<HighlightRegion> collectMatches(String text);
10
11 public List<HighlightRegion> normalize(List<HighlightRegion> candidates) {
12 return candidates.stream()
13 .filter(r -> r.start() < r.end())
14 .sorted(
15 (a, b) -> {
16 int c = Integer.compare(a.start(), b.start());
17 return (c != 0) ? c : Integer.compare(b.end(), a.end());
18 }
19);
20 }
21 }
```

```
18 })
19 .toList();
20 }
21
22 public List<HighlightRegion> resolveConflicts(List<HighlightRegion>
23 normalized) {
24 return normalized;
25 }
26 }
```

### 13.1.1. Aufgabenüberblick

Auf diesem Blatt werden Sie

1. reguläre Ausdrücke für ein einfaches Java-Syntaxhighlighting in `MiniJavaTokens` definieren,
2. einen Regex-basierten Lexer `RegexHighlighter` implementieren,
3. einen Scanning-Lexer `ScanningHighlighter` implementieren,
4. dafür jeweils JUnit-Tests schreiben,
5. und Pull-Requests mit gegenseitigem Review sowie CI nutzen.

**Bitte erst das GESAMTE Blatt durchlesen, bevor Sie mit der Bearbeitung beginnen!**

### 13.1.2. Orientierung (Vorgaben)

Für dieses Blatt sind vor allem folgende Pakete/Klassen in den Vorgaben relevant:

- `highlighting.Main`: Startklasse. Hier können Sie konfigurieren, welche Highlighter im Editor verwendet werden
- `highlighting.ui`: fertige Swing-GUI (bitte nicht ändern)
- `highlighting.core`:
  - `SyntaxHighlighter`: abstrakte Basisklasse mit Template-Methode `computeRegions`
  - `HighlightRegion`: beschreibt eine hervorgehobene Region (`start`, `end`, `color`)
- `highlighting.regex`:
  - `Token`: kapselt `Pattern` + `Color` (+ optional `matchingGroup`), liefert `List<HighlightRegion>`
  - `RegexHighlighter`: naiver Regex-Highlighter (alle Token unabhängig anwenden, nachträgliche Konfliktauflösung) - **hier ergänzen Sie die Implementierung**
  - `ScanningHighlighter`: Scanner-Variante (von links nach rechts, längstes Match gewinnt) - **hier ergänzen Sie die Implementierung**
- `highlighting.presets`:
  - `MiniJavaColours`: Farben für unterschiedliche Tokenarten
  - `MiniJavaTokens`: Liste der zu verwendenden Token - **hier ergänzen Sie die regulären Ausdrücke**
  - `Texts`: Beispieltexte

Bitte ignorieren Sie für dieses Blatt das Package `highlighting.antlr` und die ANTLR-Konfiguration im `build.gradle`.

### 13.1.3. Pflicht- und Bonusaufgaben

Auf diesem Blatt müssen **insgesamt 3 Aufgaben** bearbeitet werden. Zu *jeder* gewählten Aufgabe sind **alle zugehörigen Teilaufgaben** zu bearbeiten.

Pflichtaufgaben (müssen immer bearbeitet werden):

- **1. Reguläre Ausdrücke für das Syntaxhighlighting (MiniJavaTokens)**
- **4. Git: Pull-Requests und CI**

Zusätzlich wählen Sie bitte **genau eine** der folgenden Aufgaben:

- **2. Syntaxhighlighting mit dem `RegexHighlighter`**  
*oder*
- **3. Syntaxhighlighting mit dem `ScanningHighlighter`**

#### Important

- Bearbeiten Sie Aufgabe 1 **und** Aufgabe 4 (mit allen Teilaufgaben).
- Bearbeiten Sie **entweder** Aufgabe 2 **oder** Aufgabe 3 (mit allen Teilaufgaben).
- Fortgeschrittene sollten **zusätzlich** auch die jeweils andere Aufgabe (2 oder 3) bearbeiten.

### 13.1.4. Begriffe

- **Lexer/Scanner:** Ein Lexer (auch Scanner genannt) liest den Eingabetext von links nach rechts und zerlegt ihn in eine Folge von Token ("Wörter") wie Keywords, Bezeichner, Literale oder Kommentare o.ä., jeweils mit ihrer Position im Text.
- **Token:** Objekt aus regulärem Ausdruck (`Pattern`), Farbe (`Color`) und optional einer `matchingGroup`
- **Match/Treffer:** Konkrete Fundstelle im Eingabetext, die zu einem Token passt
- **HighlightRegion:** Ergebnis für die GUI: (`start`, `end`, `color`) für einen Treffer
- **Regionen als Intervalle:** Halb-offene Intervalle [`start`, `end`), wie bei `String.substring(start, end)`. Zwei Regionen `[0, 5)` und `[5, 8)` überlappen **nicht**.

## 13.2. Aufgaben

### 13.2.1. 1. Reguläre Ausdrücke für das Syntaxhighlighting (MiniJavaTokens)

Die Klasse `highlighting.regex.Token`:

- speichert ein vorkompiliertes `java.util.regex.Pattern` und eine `java.awt.Color`,

- kann mit `test(String s)` auf einen Text angewendet werden,
- liefert eine `List<HighlightRegion>` (jede Region: Anfang, Ende, Farbe) für alle Matches.

Optional können Sie bei Tokens eine `matchingGroup` angeben:

- `0` (Default): komplette Match-Region wird hervorgehoben
- `> 0`: nur die angegebene Capture-Group im Pattern wird für Start/Ende verwendet

### 13.2.1.1. Aufgabe 1.1: MiniJavaTokens vervollständigen

Vervollständigen Sie in der Klasse `highlighting.presets.MinijavaTokens` die Token-Liste in der Methode

```
1 public static List<Token> defaultTokens()
```

Fügen Sie der Liste weitere Token-Objekten mit passenden regulären Ausdrücken und Farben aus `MiniJavaColours` hinzu. Überlegen Sie, welche Teile im Java-Quellcode hervorgehoben werden sollten, mindestens aber:

- **Strings:** alles zwischen `"` und dem nächsten `"`
- **Characters:** genau ein Zeichen zwischen `'` und `'`
- **Keywords** (als ganze Wörter, nicht als Teil anderer Bezeichner oder Kommentare o.ä.): `package, import, class, public, private, final, return, null, new`
- **Annotationen:** beginnen mit `@`, gefolgt von Buchstaben oder Minuszeichen, z.B. `@Override`
- **Einzeilige Kommentare:** `//` bis zum Zeilenende
- **Mehrzeilige Kommentare:** `/* ... */`
- **Javadoc-Kommentare:** `/** ... */` (ggf. anders einfärben als normale Blockkommentare)
- ggf. weitere sinnvolle Token Ihrer Wahl (z.B. Zahlen, Typnamen, Interfaces, ...)

#### Hinweise:

- Vollständige Java-Lexer-Korrektheit ist **nicht** erforderlich. Vereinfachungen bei Strings/Chars sind erlaubt. Ihre Token sollten mit dem Beispieltext `highlighting.presets.Texts.START_TEXT` funktionieren.
- Die **Token-Reihenfolge** in `MiniJavaTokens` **kann wichtig** sein: z.B. Javadoc-Token vor normalem Block-Kommentar, damit `/** ... */` nicht als normales `/* ... */` behandelt wird.

### 13.2.1.2. Aufgabe 1.2: JUnit-Tests für Ihre Token

Erstellen Sie aussagekräftige JUnit-Tests für die von Ihnen definierten Token (am besten pro Token eine eigene Testmethode oder eigene Testklasse).

Testen Sie insbesondere:

- Treffer **am Anfang / in der Mitte / am Ende** eines Textes
- **Mehrere Treffer** im selben Text
- **Kein Treffer**

- **Grenzfälle**, z.B.
  - Kommentar enthält “keyword-ähnlichen” Text
  - Annotation direkt am Zeilenanfang / mit Leerzeichen
  - Strings mit `//` oder `/*` im Inhalt

Ziel ist, Vertrauen in die Korrektheit Ihrer regulären Ausdrücke zu gewinnen.

### 13.2.2. 2. Syntaxhighlighting mit dem `RegexHighlighter`

Der `highlighting.regex.RegexHighlighter` realisiert ein **naives** Syntaxhighlighting:

1. Alle Token aus `highlighting.presets.MinijavaTokens` werden unabhängig voneinander auf den Text angewendet.
2. Danach werden die Funde sortiert und Konflikte (Überlappungen) aufgelöst.

Die Template-Methode `computeRegions` in der Basisklasse `highlighting.core.SyntaxHighlighter` ist bereits gegeben und darf nicht verändert werden. Sie ruft in dieser Reihenfolge auf:

1. `collectMatches(text)`
2. `normalize(candidates)`
3. `resolveConflicts(normalized)`

`normalize` ist bereits sinnvoll implementiert (Filterung ungültiger Regionen, Sortierung). `resolveConflicts` ist aktuell eine Identitätsfunktion und muss angepasst werden.

#### 13.2.2.1. Aufgabe 2.1: `RegexHighlighter.collectMatches` implementieren

Implementieren Sie in `highlighting.regex.RegexHighlighter` die Methode

```
1 public List<HighlightRegion> collectMatches(String text)
```

so, dass

- alle Token aus `MinijavaTokens` auf den Eingabetext angewendet werden,
- alle gefundenen Regionen in einer gemeinsamen Liste gesammelt werden,
- die Liste zurückgegeben wird (Sortierung übernimmt `normalize`).

Achten Sie auf die TODO-Kommentare in der Klasse.

#### 13.2.2.2. Aufgabe 2.2: `RegexHighlighter.resolveConflicts` implementieren

Implementieren Sie in `highlighting.regex.RegexHighlighter` die Methode

```
1 public List<HighlightRegion> resolveConflicts(List<HighlightRegion> normalized)
```

mit folgender **Konfliktregel**:

- Gehen Sie die `normalized`-Liste von vorne nach hinten durch.
- Fügen Sie eine Region nur dann in die Ergebnisliste ein, wenn sie **nicht** mit einer bereits übernommenen Region überlappt:
  - Region `R` überlappt mit bereits gewählter Region `S`, falls `[R.start, R.end)` und `[S.start, S.end)` sich schneiden.
  - Wenn eine Überlappung existiert, wird `R` verworfen (da `S` früher in der Liste stand).
- Denken Sie daran, dass Intervalle halb-offen sind: `[start, end)` -> `[0, 5)` und `[5, 8)` überlappen **nicht**.

Die übergebene normalisierte Liste (`normalize`) ist bereits

- nach Startposition sortiert,
- bei gleichem Start: längere Regionen zuerst.

### 13.2.2.3. Aufgabe 2.3: JUnit-Tests für den RegexHighlighter

Erstellen Sie JUnit-Tests für den `RegexHighlighter`. Untersuchen Sie u.a.:

- einfache Fälle ohne Überlappungen,
- überlappende Regionen, z.B.
  - Keyword innerhalb eines Kommentars,
  - Javadoc-Kommentar, der auch vom normalen Blockkommentar-Token matchbar wäre,
- Fälle mit aufeinanderfolgenden Regionen, z.B. `[0, 5)` und `[5, 10)` (diese dürfen **beide** bleiben),
- Leerstring bzw. Text ohne Matches.

### 13.2.3. 3. Syntaxhighlighting mit dem ScanningHighlighter

Der `highlighting.regex.ScanningHighlighter` soll das typische Lexer-Verhalten nachbilden:

- Der Text wird **zeichenweise** von links nach rechts gelesen.
- An jeder Position wählen Sie das **längste** passende Token, das **genau an dieser Position beginnt**.
- Bei Gleichstand (mehrere Token mit gleich langer Match-Region) gewinnt das Token, das in `MiniJavaTokens` **früher in der Liste** steht.

**Wichtig, wenn kein Token passt:**

- Wenn an der aktuellen Position **kein** Token beginnt: Index um **ein Zeichen** erhöhen (`i++`).
- Wenn ein Token passt: Index direkt **hinter** das gefundene Match setzen (`i = end`).
- So vermeiden Sie Endlosschleifen und stellen sicher, dass der gesamte Text gescannt wird.

Die Klasse soll wieder von `SyntaxHighlighter` erben und nur `collectMatches` und `normalize` überschreiben.



### 13.2.3.1. Aufgabe 3.1: ScanningHighlighter.collectMatches implementieren

Implementieren Sie in `highlighting.regex.ScanningHighlighter` die Methode

```
1 public List<HighlightRegion> collectMatches(String text)
```

mit der oben beschriebenen Semantik.

Die von Ihnen erzeugte Liste der `HighlightRegion`-Objekte soll bereits

- sortiert sein,
- sich nicht überlappen,
- nur gültige Regionen enthalten.

### 13.2.3.2. Aufgabe 3.2: ScanningHighlighter.normalize überschreiben

Überschreiben Sie in `highlighting.regex.ScanningHighlighter` die Methode

```
1 public List<HighlightRegion> normalize(List<HighlightRegion> candidates)
```

so, dass sie einfach die Eingabe unverändert zurückgibt (Identität). Eine weitere Normalisierung ist hier nicht nötig, da `collectMatches` bereits eine geeignete Liste erzeugt.

`resolveConflicts` können Sie in dieser Klasse unverändert von `SyntaxHighlighter` erben (Identität).

### 13.2.3.3. Aufgabe 3.3: JUnit-Tests für den ScanningHighlighter

Erstellen Sie JUnit-Tests für den `ScanningHighlighter`. Testen Sie insbesondere:

- **“Längstes Match gewinnt”** z.B. ein Text, in dem sowohl ein kürzeres als auch ein längeres Token an derselben Position matchen könnten.
- **Gleichstand** Zwei Token matchen gleich lang - prüfen Sie, dass das frühere Token in `MiniJavaTokens` ausgewählt wird.
- **Textstellen ohne Match** Stellen ohne Token dürfen den Scanner nicht blockieren; prüfen Sie, dass der Index korrekt weiterläuft.
- Kombinationen aus Treffern und Nicht-Treffern im selben Text.

### 13.2.3.4. Aufgabe 3.4: Manueller Vergleich mit RegexHighlighter

Starten Sie die Anwendung so, dass beide Highlighter-Strategien parallel laufen (z.B. zwei Fenster nebeneinander). Vergleichen Sie die Hervorhebungen visuell:

- Gibt es Unterschiede zwischen Regex-Variante und Scanner-Variante?
- Können Sie erklären, **warum** die Unterschiede auftreten?
  - z.B. wegen “längstes Match gewinnt” vs. “erstes Token gewinnt”,
  - oder wegen der Konfliktauflösung im `RegexHighlighter`.

### 13.2.4. 4. Git: Pull-Requests und CI

Richten Sie für Ihr Projekt eine einfache CI-Pipeline ein und nutzen Sie Pull-Requests beim Erstellen Ihrer Implementierung.

#### 13.2.4.1. Aufgabe 4.1: CI-Pipeline anlegen

Nutzen Sie z.B. GitHub Actions und definieren Sie einen Workflow, der drei Jobs enthält:

- `build`: kompiliert nur das Projekt (nur die Klassen erstellen)
- `test`: führt die JUnit-Tests aus
- `format`: führt Spotless im “testenden” Modus aus

#### Trigger:

- Die Pipeline soll **nur** laufen bei
  - Pull-Requests (`pull_request`) und
  - manueller Auslösung (`workflow_dispatch`).
- Sie soll **nicht** bei Pushs in den Hauptbranch (`master` oder `main`) automatisch starten.

#### 13.2.4.2. Aufgabe 4.2: Branches und Pull-Requests

Bearbeiten Sie die drei Implementierungsaufgaben in separaten Feature-Branches:

- ein Branch für `MiniJavaTokens`,
- ein Branch für `RegexHighlighter`,
- ein Branch für `ScanningHighlighter`.

Für jeden Branch:

1. Legen Sie frühzeitig einen Pull-Request in Ihrem eigenen Repo an.
2. Passen Sie **Summary** und **Description** im PR sinnvoll an:
  - Was wird geändert und warum?
  - Was ist der aktuelle Stand?
  - Gibt es offene Punkte?

#### Empfohlene Reihenfolge:

1. Zuerst `MiniJavaTokens` fertigstellen und PR mergen.
2. Danach `RegexHighlighter` und `ScanningHighlighter` (können dann parallel entwickelt werden).

#### 13.2.4.3. Aufgabe 4.3: Gegenseitige Reviews der PR

- Lassen Sie Kommiliton:innen jeden Ihrer PR vor dem Mergen reviewen.
- Führen Sie im Gegenzug selbst Reviews für andere durch.

Achten Sie dabei auf:

- **Kommentieren an Codestellen**
  - konkrete Hinweise, Fragen, Verbesserungsvorschläge.
- **Reaktion auf Kommentare**
  - Antworten Sie auf Kommentare,
  - schließen Sie Kommentare bewusst:
    - \* manuell, wenn nichts weiter passieren soll,
    - \* durch Codeänderungen und erneuten Push, wenn Sie dem Kommentar folgen/zustimmen.
- Nutzen Sie auch die GitHub-Funktion “Code vorschlagen” in der Weboberfläche.

Mergen Sie die Pull-Requests in der **passenden Reihenfolge**, sodass Folge-Banches auf der jeweils aktuellen Basis aufsetzen.

### 13.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung mit gegenseitigem Review
- Abgabe Post Mortem [im ILIAS](#): bis **01. Juni, 08:00 Uhr**
- Vorstellung im Praktikum: 01./03. Juni

## 14. Blatt 05: ANTLR, Visitor; Äquivalenzklassen & Grenzwerte, Mocking

### 14.1. Zusammenfassung

Auf diesem Blatt arbeiten Sie mit ANTLR und nutzen den generierten Lexer für das Syntaxhighlighting. Sie traversieren den Parse-Tree mit Hilfe des Visitor-Patterns und der generierten ANTLR-Basisklasse und geben eingelesenen Java-Sourcecode korrekt eingerückt wieder aus ("Pretty Printing"). Sie üben die Erstellung von Testfällen mit der Äquivalenzklassenbildung und Grenzwertanalyse. Sie üben den Einsatz mit Mockito.

### 14.2. Aufgaben

#### 14.2.1. 1. ANTLR

Betrachten Sie noch einmal das Java-Projekt zum [Syntax-Highlighting](#).

##### Important

**Bitte aktualisieren Sie Ihren Fork und Ihre Workingcopy!**

Sie finden im Package `highlighting.antlr` die Klasse `AntlrTokenCollector`, die die abstrakte Basisklasse zum Syntaxhighlighting (`SyntaxHighlighter`) erweitert. Außerdem finden Sie hier die Klasse `PrettyPrinterVisitor`, die die von ANTLR generierte Visitor-Klasse `MiniJavaBaseVisitor<Void>` erweitert.

Im Projekt finden Sie ebenfalls eine fertige ANTLR-Grammatik, die zu Java-Code passt, und im `build.gradle` die *ready-to-use* Einbindung von ANTLR als Bibliothek. Bei jedem `./gradlew classes` oder `./gradlew run` o.ä. sollten im Build-Ordner `build/generated-src/antlr/main/` die generierten Klassen mit dem Package `highlighting/antlr/` erscheinen und in der IDE automatisch eingebunden werden.

##### 14.2.1.1. Aufgabe 1.1: Syntaxhighlighting mit dem `AntlrTokenCollector`

Implementieren Sie in `AntlrTokenCollector` die Methode `List<HighlightRegion> collectMatches(String text)`. Nutzen Sie diesmal den generierten `highlighting.antlr.MinijavaLexer`, um aus dem Eingabetext einen Token-Stream zu erzeugen, und wandeln Sie die Tokens anschließend in `HighlightRegion`-Objekte um.

Überlegen Sie, ob und wie in diesem Fall noch Konflikte zwischen Regionen entstehen können und wie die beiden Hook-Methoden `normalize` und `resolveConflicts` aus der Oberklasse `AntlrTokenCollector` in diesem Fall aussehen müssen. Überschreiben Sie die Methoden entsprechend in `AntlrTokenCollector` (falls notwendig).

Vergleichen Sie das so erzeugte Syntaxhighlighting mit den Varianten von Blatt 04. Wo gibt es Unterschiede, was sind die Gründe dafür? Welche der Varianten ist aufwändiger in der Implementierung?

#### 14.2.1.2. Aufgabe 1.2: Visitoren mit ANTLR (Pretty Printing)

Nutzen Sie die von ANTLR generierten Klassen zum einfachen “Pretty Printing” von MiniJava-Code: Sie sollen Programmcode so neu ausgeben, dass Einrückungen und Zeilenumbrüche konsistent sind.

##### 1. Schritt 1: Parse-Tree erzeugen

Nutzen Sie die von ANTLR generierten Klassen für Lexer `MiniJavaLexer` und Parser `MiniJavaParser`, um aus einem eingegebenen MiniJava-Programm einen Parse-Tree vom Typ `MiniJavaParser.CompilationUnitContext` zu erzeugen.

##### 2. Schritt 2: Visitor für das Pretty Printing

Implementieren Sie die vier mit “TODO” markierten `visitXxx`-Methoden in der Klasse `highlighting.antlr.PrettyPrinterVisitor`:

- `visitCompilationUnit`,
- `visitClassBody`,
- `visitBlock` und
- `visitStatement`.

Dieser zustandsbehaftete Visitor soll den Parse-Tree traversieren und das Ergebnis über die Hilfsmethode `result()` als `String` mit dem neu formatierten MiniJava-Programm liefern.

Konzentrieren Sie sich auf die **Struktur** des Programms und nutzen Sie zum Traversieren der Kinder bei Bedarf `visit(child)`. Eigene `visit...()`-Methoden für weitere Regeln sind nicht erforderlich.

In der Klasse `PrettyPrinterVisitor` stehen Ihnen bereits Hilfsmethoden zur Ausgabe zur Verfügung, insbesondere

- `write(...)`, `writeln(...)`, `nl()` und `indent()` für die strukturierte Ausgabe sowie
- die Standard-Visitor-Mechanismen `visit(...)`, `visitChildren(...)` und `visitTerminal(...)` (für die Token-Ausgabe mit einer einfachen Leerzeichen-Heuristik).

Nutzen Sie diese vorhandenen Methoden; Sie müssen **keine** eigenen Low-Level-Ausgabefunktionen schreiben.

Beachten Sie dabei folgende **Formatierungsregeln**:

##### 1. Statements und Deklarationen

Jedes Statement bzw. jede Deklaration, die mit `;` endet, soll auf **einer eigenen Zeile** stehen. Dazu gehören insbesondere:

- lokale Variablendeklarationen,
- Felddeklarationen,
- **return**-Statements,
- Ausdrucks-Statements.

## 2. Blöcke

- Nach einer öffnenden geschweiften Klammer { (z.B. Klassendefinition, Methodenrumpf, Block in **if/else** oder **while**) folgt ein Zeilenumbruch.
- Der Inhalt eines Blocks wird gegenüber der umgebenden Struktur **um eine Stufe eingerückt**.
- Die schließende Klammer } steht auf einer eigenen Zeile und ist so eingerückt wie die zugehörige öffnende Struktur.

## 3. Leerzeichen und Einrückung

- Einrückung erfolgt nur mit Leerzeichen (keine Tabs).
- Eine Begrenzung der Zeilenlänge ist nicht erforderlich.
- Innerhalb einer Zeile (z.B. in Ausdrücken) können Sie den Code weitgehend in der Originalreihenfolge ohne aufwändige Optimierung der Leerzeichen ausgeben. Für viele Unterregeln genügt `visit(child)`.

Es genügt also, Blöcke und Statements strukturiert einzurücken und vernünftig zu trennen. Eine “perfekte” Formatierung von Ausdrücken ist **nicht** erforderlich.

## 3. Schritt 3: Benutzung und Demonstration

- Fragen Sie auf der Konsole nach der gewünschten Anzahl von Leerzeichen pro Einrückstufe (z.B. 2, 4 oder 8).
- Übergeben Sie diese Zahl an den Konstruktor `PrettyPrinterVisitor(int indentWidth)` und verwenden Sie sie zum Einrücken.
- Geben Sie den von Ihrem Pretty-Printer erzeugten `String` z.B. auf der Konsole aus.
- Demonstrieren und dokumentieren Sie Ihre Lösung an mehreren Beispielen mit unterschiedlicher Komplexität, z.B.:
  - einfache Klasse mit einem Feld und einer Methode,
  - Methoden mit **if/else** und **while**,
  - verschachtelte Blöcke.

### Tip

Was beobachten Sie bei der Ausgabe Ihres Pretty-Printers? Wieso fehlen bestimmte Teile aus dem Input?

## Optionale Erweiterungen

Wenn Sie schon mit dem Pflichtteil gut zurechtkommen, können Sie Ihren Pretty-Printer **schrittweise erweitern**. Mögliche Ideen:

- Fügen Sie **eine Leerzeile** zwischen zwei Methodendefinitionen ein.
- Platzieren Sie **else** in **if/else**-Konstrukten direkt nach der schließenden Klammer des **if**-Blocks, z.B.:

```
1 if (condition) {
2 ...
3 } else {
4 ...
5 }
```

- Sorgen Sie dafür, dass einfachere Ausdrücke etwas “schöner” formatiert werden, z.B. genau ein Leerzeichen um Binäroperatoren (+, -, \*, /, ==, && usw.).

Diese Erweiterungen sind **optional** und beeinflussen nicht die Bewertung des Pflichtteils, zeigen aber, dass Sie mit dem Parse-Tree und dem Visitor sicher umgehen können.

### Bonus für Fortgeschrittene

Für alle, die sich weiter austoben möchten:

#### 1. Integration in den Editor

Schaffen Sie es, den Pretty-Printer in den Editor ([highlighting.ui.EditorUI](#)) zu integrieren, so dass der aktuell gezeigte Code per Knopfdruck (oder Menüeintrag oder sogar bei Änderung) neu formatiert wird?

#### 2. Mehr Sprachkonstrukte

Erweitern Sie Ihren Pretty-Printer schrittweise so, dass er weitere Teile der MiniJava-Grammatik behandelt, z.B.:

- Annotationen ([@Annotation](#)),
- **extends**- und **implements**-Listen in Klassenköpfen,
- Formatierung von Parameter- und Argumentlisten.

### Important

Für alle Schritte in dieser Aufgabe:

Es geht nur um die Ausgabe syntaktisch korrekter Java-Programme. Sie brauchen sich um die Semantik (z.B. passende Typen wie etwa keine Multiplikation von Strings mit Integern o.ä.) keine Gedanken machen! Achten Sie auf die korrekten Einrücktiefen. Die Zeilenlänge spielt hier keine Rolle, es wird einfach direkt nach jedem Statement umgebrochen (bzw. wie bei den Kontrollstrukturen gezeigt).

### Tip

Das Thema Pretty Printing ist interessant und kann recht schnell ziemlich aufwändig werden. Sie finden im Paper [“A prettier printer”](#) von Philip Wadler und im Blog [“The Hardest Program I’ve Ever Written”](#) von Bob Nystrom gut geschriebene Beiträge für eine tiefere Beschäftigung mit der Materie.

## 14.2.2. 2. Cycle Chronicles

Forken Sie das [“Cycle Chronicles”](#)-Repo und erzeugen Sie sich eine lokale Arbeitskopie von Ihrem Fork.

#### 14.2.2.1. Aufgabe 2.1: Analyse der Äquivalenzklassen & Grenzwerte

Die Methode `Shop#accept` dient zur Annahme eines neuen Auftrags eines Kunden.

Neue Aufträge sollen nur unter bestimmten Bedingungen angenommen werden:

- Es darf sich nicht um ein E-Bike handeln.
- Es darf sich nicht um ein Gravel-Bike handeln.
- Der Kunde darf nicht noch andere offene Aufträge beim Shop haben (es kann pro Kunden immer nur maximal einen offenen Auftrag in der Warteschlange geben).
- Es sind aktuell höchstens vier andere offene Aufträge vorhanden (es dürfen zu jeder Zeit maximal fünf offene Aufträge in der Warteschlange sein).

Der Rückgabewert der Methode signalisiert, ob der Auftrag angenommen wurde und in die Warteschlange der offenen Aufträge eingereiht wurde (**true**) oder ob er abgelehnt wurde (**false**).

##### Aufgaben:

1. Erstellen Sie eine Äquivalenzklassenbildung und eine Grenzwert-Analyse.
2. Erstellen Sie aus den ermittelten Äquivalenzklassen und Grenzwerten konkrete Testfälle. (**Noch keine Implementierung!**)

#### 14.2.2.2. Aufgabe 2.2: Mocking I

Implementieren Sie nun die in der vorigen Aufgabe ermittelten Testfälle für die Methode `Shop#accept` mit Hilfe von JUnit.

Leider gibt es beim Ausführen vieler Ihrer JUnit-Testmethoden eine `UnsupportedOperationException`-Exception, da die Klasse `Order` bisher nur unvollständig implementiert ist: Es existieren praktisch nur die Methodensignaturen, der Aufruf der Methoden liefert nur eine `UnsupportedOperationException`.

Setzen Sie aktiv Mocking mit Mockito ein, um Objekte der Klasse `Order` (und ggf. weiterer abhängiger Klassen) zu mocken, sodass Ihre JUnit-Tests für `Shop#accept` ausführbar werden. Begründen Sie die Anwendung von Mockito.

##### Important

Die zu testende Methode `Shop#accept` soll in der vorliegenden Implementierung im Test genutzt werden, d.h. sie darf nicht "weg-gemockt" werden!

##### Tip

In der Gradle-Konfiguration der Vorgabe ist bereits JUnit und Mockito vorkonfiguriert, d.h. die entsprechenden Abhängigkeiten werden durch Gradle aufgelöst. Wenn Sie die Vorgaben als Gradle-Projekt in Ihrer IDE öffnen, dann steht Ihnen dort auch die JUnit-Bibliothek automatisch zur Verfügung. Mit `./gradlew test` können Sie Ihre Testfälle ausführen.



### 14.2.2.3. Bonus: Aufgabe 2.3: Mocking II

Für Fortgeschrittene:

Die Methoden `Shop#repair` und `Shop#deliver` sind auch noch nicht implementiert. Überlegen Sie sich, wie sich `Shop#repair` und `Shop#deliver` verhalten sollten und schreiben Sie auf Basis dieser Annahmen JUnit-Tests mit Mockito. Begründen Sie die Anwendung von Mockito.

#### Tip

Sie *müssen* hier keine Äquivalenzklassenbildung und Grenzwertanalyse machen, können das aber natürlich gern tun.

## 14.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem [im ILIAS](#): bis **15. Juni, 08:00 Uhr**
- Vorstellung im Praktikum: 15./17. Juni

## 15. Blatt 06: Visitor vs. Pattern Matching, Records

### 15.1. Zusammenfassung

Wir wollen eine kleine Songliste erstellen und nach unseren Lieblingssongs filtern.

Betrachten Sie dazu das Repo [“Filter-DSL”](#).

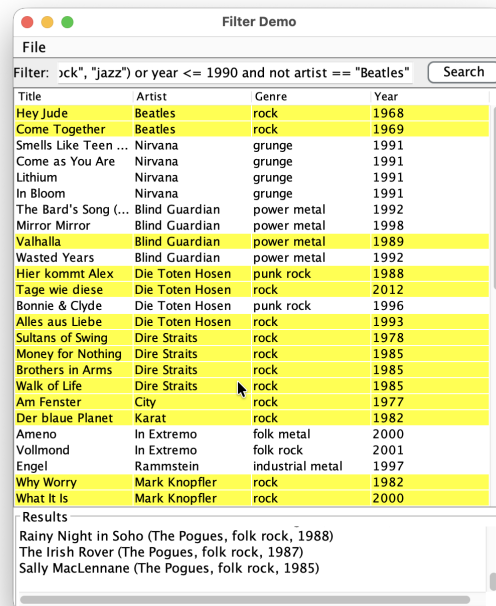
#### 15.1.1. Basismodellierung

Die Songs werden über die Record-Klasse `filter.model.MediaItem` repräsentiert, die auch gleich noch Methoden zum Einlesen von Songs aus einer CSV-Datei bereitstellt:

```
1 public record MediaItem(String title, String artist, Genre genre, int year) {
2
3 /** file system (e.g. JFileChooser) */
4 public static List<MediaItem> loadFromPath(Path path);
5
6 /** classpath resource (src/main/resources) */
7 public static List<MediaItem> loadFromResource(String resourcePath);
8 }
```

Im Ressourcen-Verzeichnis `src/main/resources/` liegt die Datei `songlist.txt` im CSV-Format, die bereits einige nette Songs enthält.

Zum Anzeigen der Liste gibt es eine kleine GUI: `filter.app.FilterApp`:



### 15.1.2. Filter-DSL

Die Suche erfolgt über eine **kleine Filtersprache**. Die Grammatik dazu finden Sie in `Filter.g4`. Darüber lassen sich verschiedene, unterschiedlich komplexe Suchausdrücke ("Queries") formulieren, beispielsweise:

- `artist == "Beatles"`
- `year == 1965`
- `artist == "Beatles" and year == 1965`
- `genre in ("rock", "jazz") or year <= 1990 and not artist == "Beatles"`

```

1 primary : comparison | '(' expr ')';
2 comparison
3 : IDENTIFIER op=COMPOP value=literal
4 | IDENTIFIER IN '(' literal (',' literal)* ')'
5 ;
6 literal : STRING | NUMBER;
7 COMPOP : '==' | '!=' | '<' | '<=' | '>' | '>=';

```

Zum Bilden von Ausdrücken wird ein Feldname (z.B. `artist`, `year`) mit einem Wert verglichen: `artist == "Beatles"` oder `year < 1965` oder ... Hierzu stehen die üblichen Vergleichsoperatoren zur Verfügung. Alternativ kann mit einem `in`-Ausdruck eine Variable mit den Werten in einer geklammerten Liste verglichen werden: `genre in ("rock", "jazz", "metal", "grunge")`. Der Ausdruck wird wahr, wenn die Variable `genre` einen der in der Liste aufgezählten Werte hat.

Ausdrücke können auch geklammert werden, d.h. `(artist == "Beatles")` ist ein gültiger Ausdruck.

```

1 expr : orExpr;

```

```

2
3 orExpr : andExpr (OR andExpr)* ;
4 andExpr : notExpr (AND notExpr)* ;
5 notExpr : NOT notExpr | primary ;

```

Zum Bilden komplexerer Ausdrücke stehen zusätzlich noch die Operatoren `or`, `and` und `not` zur Verfügung:

- `or`: Mit `or` lassen sich zwei oder mehr Ausdrücke **ODER**-verknüpfen, d.h. mindestens einer der verknüpften Teilausdrücke muss wahr sein, damit der gesamte Ausdruck wahr wird. Beispiel: `artist == "Beatles" or year <= 1965`
- `and`: Mit `and` lassen sich zwei oder mehr Ausdrücke **UND**-verknüpfen, d.h. alle verknüpften Teilausdrücke müssen wahr sein, damit der gesamte Ausdruck wahr wird. Beispiel: `year <= 1990 and artist == "Beatles"`
- `not`: Mit `not` wird ein Ausdruck negiert, d.h. der Ausdruck unter dem `not` muss falsch sein, damit der gesamte Ausdruck wahr wird. Beispiel: `not artist == "Beatles"`

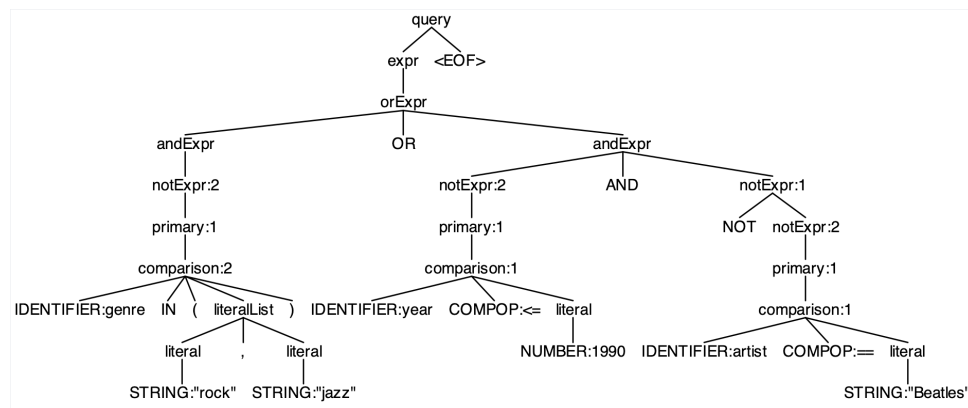
Über die Grammatik `Filter.g4` sind Vorrangregeln (Präzedenzregeln) für die Ausdrücke definiert. Am stärksten binden die Vergleichsoperatoren und die Klammern um Ausdrücke, gefolgt von `not`, `and` und `or` (in dieser Reihenfolge). Die scheinbar mehrdeutige Query `genre in ("rock", "jazz") or year <= 1990 and not artist == "Beatles"` entspricht also dieser geklammerten Variante (`genre in ("rock", "jazz") or ((year <= 1990) and (not (artist == "Beatles")))` (für eine visuelle Darstellung siehe den AST im nächsten Abschnitt).

Für diese Grammatik erzeugt uns ANTLR wieder einen Lexer und Parser sowie die Basisklassen für das Visitor-Pattern; die im Projekt vorbereitete `build.gradle` enthält bereits alle für dieses Blatt nötigen Dependencies und Plugins. Das Zusammenstecken der ANTLR-Klassen brauchen Sie nicht mehr erledigen: Es gibt in der Klasse `filter.ast.builder.AstBuilders` bereits die Methode `FilterParser.QueryContext.parse(String query)`, wo der Aufbau der verschiedenen generierten ANTLR-Klassen bereits erledigt ist, und die Sie nutzen können, um eine Query wie `artist == "Beatles"` zu parsen und in einen Parse-Tree zu verwandeln. Die Klasse bietet zusätzlich noch die Methode `Expr.fromQuery(String query, Function<FilterParser.QueryContext, Expr> translator)` an, um in einem Schritt das Parsing zu erledigen und über den injizierten `AstBuilderPattern` oder `AstBuilderVisitor` einen deutlich reduzierten Baum (den sogenannten *Abstract Syntax Tree*, AST) zu erzeugen. Die Modellierung für den AST selbst ist ebenfalls bereits vorgegeben (Klassen im Package `filter.ast.nodes`).

### 15.1.3. AST

Der von ANTLR erzeugte Parse-Tree ist oft sehr groß und enthält in den einzelnen Zweigen jeweils alle dort durchlaufenen Regeln der Grammatik. Bei der späteren Verarbeitung stört das oft, so dass man aus dem Parse-Tree eine deutlich komprimierte Version für die weiteren Schritte erzeugt, den sogenannten "Abstract Syntax Tree" (AST).

Beispielsweise würde für den Filterausdruck `genre in ("rock", "jazz") or year <= 1990 and not artist == "Beatles"` und der Grammatik `Filter.g4` der folgende Parse-Tree aufgebaut werden:



Bei genauerer Betrachtung stellt man fest, dass nach erfolgreichem Abschluss der Parsing-Phase viele der Zwischenknoten redundant oder überflüssig sind und man den Baum in eine deutlich kompaktere Form transformieren kann:

```

1 Or
2 InList genre
3 - "rock"
4 - "jazz"
5 And
6 Comparison year <= 1990
7 Not
8 Comparison artist == "Beatles"

```

Die Modellierung des AST ist nicht trivial und hängt stark vom Verwendungszweck ab. Für uns in Prog2 sind die Klassen für den AST bereits vorgegeben im Package `filter.ast.nodes`:

```

1 package filter.ast.nodes;
2
3 public sealed interface Expr {
4 record And(Expr left, Expr right) implements Expr {}
5 record Or(Expr left, Expr right) implements Expr {}
6 record Not(Expr inner) implements Expr {}
7 record Comparison(String field, CompOp op, Value value) implements Expr {}
8 record InList(String field, List<Value> values) implements Expr {}
9 }
10
11 public sealed interface Value {
12 record Str(String text) implements Value {}
13 record Num(int value) implements Value {}
14 }
15
16 public enum CompOp {
17 EQ { public String toString() { return "==" ; } },
18 NE { public String toString() { return "!=" ; } },
19 LT { public String toString() { return "<" ; } },
20 LE { public String toString() { return "<=" ; } },
21 GT { public String toString() { return ">" ; } },
22 GE { public String toString() { return ">=" ; } },
23 }

```

Der gemeinsame Obertyp für alle Knoten im Baum ist `Expr` (Ausdruck). Es gibt die einzelnen Klassen zur

Repräsentation der verschiedenen Ausdrücke in `Expr`. Die Kindknoten fast aller Baumknoten sind ihrerseits wieder vom Typ `Expr`, nur beim Knoten `Comparison` ist das rechte Kind ein `Value` (Wert). Es gibt zwei Arten von Werten in `Value`: `Str` und `Num`.

Damit lassen sich die Parse-Trees zu `Filter.g4` in deutlich kompaktere Strukturen übersetzen mit gleichwertiger Bedeutung. Zwei Punkte verdienen besondere Beachtung:

1. In der Grammatik können die `and`- und `or`-Ausdrücke mit beliebig vielen Vergleichen gebildet werden, d.h. `year <= 1990, year <= 1990 and artist == "Beatles"` und `year <= 1990 and artist == "Beatles" and year > 1960` etc. wären gültige Ausdrücke. In der AST-Struktur haben die `And`- und `Or`-Knoten jeweils genau zwei Kindknoten. Dies muss passend übertragen werden:
  - Ein Kindknoten im `and`- oder `or`-Ausdruck: `Comparison`
  - Zwei Kindknoten im `and`- bzw. `or`-Ausdruck: `And` bzw. `Or`
  - Sonst: Geeignet klammern und dann in die `And`- bzw. `Or`-Knoten übersetzen, d.h. aus `year <= 1990 and artist == "Beatles" and year > 1960` wird im AST `(year <= 1990 and artist == "Beatles") and year > 1960`
2. Klammern werden nicht explizit im AST repräsentiert. Die Klammerung wird über die Tiefe im Baum ausgedrückt: Je weiter innen die Klammerung, um so tiefer im Baum taucht der Knoten auf. Beispiel:

`(year <= 1990 or artist == "Beatles") and year > 1960`

```

1 And
2 Or
3 Comparison year <= 1990
4 Comparison artist == "Beatles"
5 Comparison year > 1960

```

Den AST erhält man, indem man den **Parse-Tree traversiert** (Visitor-Pattern oder manuell über Pattern Matching) und dabei die kompaktere Baum-Version erzeugt. Dazu sind die Klassen `filter.ast.builder.AstBuilderVisitor` und `filter.ast.builder.AstBuilderPattern` angelegt. Diese fertig zu programmieren ist Ihre Aufgabe.

#### 15.1.4. Pretty-Printing, Evaluierung und GUI

Im Package `filter.ast.printer` finden Sie einen vordefinierten Pretty-Printer `AstPrinter`, der einen AST mit der Methode `String toString(Expr expr)` in einen geklammerten String konvertiert. Diese Klasse können Sie gut bei den Approval Tests einsetzen.

Zusätzlich gibt es in `filter.ast.eval` einen `Evaluator`. Mit der Methode `boolean matches(MediaItem item, Expr expr)` wird geprüft, ob ein konkreter Song (`MediaItem item`) zu einer Query gegeben als AST (`Expr expr`) passt. Intern setzt dieser Evaluator zum Übersetzen der Query den `filter.ast.builder.AstBuilderPattern` ein.

Im Package `filter.app` gibt es eine fertige `FilterApp`. Dies ist eine Swing-GUI mit einem Menü zum Laden einer Textdatei mit Medien-Einträgen im CSV-Format und der Anzeige der Songs in einer editier- und sortierbaren Tabelle. Es gibt ein Eingabefeld für die Filter-Query. Die Treffer werden im Ergebnisbereich ausgegeben sowie in der Tabelle farblich hervorgehoben.

**Important**

Die Filterung in der Filter-App wird über den `filter.ast.eval.Evaluator` realisiert und funktioniert deshalb erst, wenn Sie den AST-BUILDER `filter.ast.builder.AstBuilderPattern` implementiert haben.

### 15.1.5. Wie wird aus Text ein Filter-Ausdruck?

Der Weg von der Texteingabe zum Evaluator ist:

1. **Lexer & Parser (ANTLR)** aus `String` → `FilterParser.QueryContext` (Parse-Tree-Wurzel)
2. **AST-BUILDER** aus `FilterParser.QueryContext` → `Expr` (Ihr eigener AST)
3. **Evaluator** aus (`MediaItem`, `Expr`) → `boolean` (passt / passt nicht)

In dieser Übung geht es primär um Schritt 2: Wie übertrage ich die Parse-Tree-Struktur in die vorgegebenen AST-Strukturen?

## 15.2. Aufgaben

### 15.2.0.1. Aufgabe 1: JUnit-Testfälle

Bevor wir an die AST-BUILDER gehen, definieren Sie sich eine Reihe von relevanten Testfällen (JUnit). Nutzen Sie dazu die leere Klasse `filter.ast.AstTest`.

### 15.2.0.2. Aufgabe 2: AST-BUILDER mit dem Visitor-Pattern

Sie finden im Package `filter.ast.builder` die vorbereitete Klasse `AstBuilderVisitor`, die vom durch ANTLR generierten Basisvisitor `FilterBaseVisitor<Void>` ableitet und den AST durch den **Einsatz des Visitor-Patterns** erzeugt.

Die Startmethode `Expr translate(FilterParser.QueryContext ctx)` wird mit dem Parse-Tree als Argument aufgerufen und soll dann die rekursive Verarbeitung über das Visitor-Pattern anstoßen.

Die relevanten Methoden sind bereits vorgegeben - Füllen Sie diese nun noch mit Leben.

**Tip**

Implementieren Sie wie in der Vorlesung gezeigt einen zustandsbehafteten Visitor. Sie arbeiten entsprechend aktiv mit mehreren Stacks, beispielsweise `Deque<Expr>` und `Deque<Value>`. Prinzipiell ist der Ablauf pro Knoten ungefähr dieser: Sie besuchen rekursiv die Kindknoten eines Knotens, nehmen sich danach Objekte von den Stacks und verarbeiten diese Objekte und pushen das Ergebnis wieder auf den richtigen Stack.

### 15.2.0.3. Aufgabe 3: AST-Builder mit manueller Traversierung und typbasiertem Pattern Matching

Sie finden im Package `filter.ast.builder` die vorbereitete Klasse `AstBuilderPattern`, die eine **manuelle Traversierung** des Parse-Trees durchführen soll und dabei **Pattern Matching auf Typen** einsetzen soll und so den AST aufbaut. Die relevanten Methoden sind auch hier bereits vorgegeben, die Startmethode ist wieder `Expr translate(FilterParser.QueryContext ctx)`. Implementieren Sie die vorgegebenen Methoden und nutzen Sie dabei nach Möglichkeit aktiv **switch/case** auf Klassen/Records/Enums.

#### Tip

Sie brauchen für den `AstBuilderPattern` keinen internen Zustand - es wird alles über die Argumente und Rückgabewerte der Methoden über- bzw. zurückgegeben. In den Methoden arbeiten Sie mit direkten Zugriffen auf die jeweiligen Kontext-Methoden.

### 15.2.0.4. Aufgabe 4: Vergleich Visitor-Pattern vs. Pattern Matching

Vergleichen Sie die beiden AST-Builder miteinander. Was sind aus Ihrer Sicht jeweils die Vor- und Nachteile der beiden Varianten?

### 15.2.0.5. Aufgabe 5: AST-Normalisierung mit Pattern Matching

Leider gibt die Form unserer Grammatik nicht so viel für Pattern Matching her, wie man sich für das Üben von **Expression Switch/Case** und **Pattern Matching** wünschen würde. Für eine geeignetere Grammatik bräuchten Sie tiefere Kenntnisse, die wir erst im dritten Semester in Compilerbau besprechen.

Damit Sie trotzdem noch ein wenig Pattern Matching üben können: Vervollständigen Sie bitte die Methode `Expr simplify(Expr e)` in der Klasse `filter.ast.builder.AstBuilders`. Schreiben Sie eine Abbildung  $AST \rightarrow AST$ , die beispielsweise doppelte `Not` eliminiert. Nutzen Sie hier aktiv Pattern Matching und Expression Switch/Case. Denken Sie daran, dass unsere AST-Klassen eine *sealed* Klassenhierarchie sind und dass für die Modellierung Record-Klassen genutzt wurden - Zeigen Sie, wie Sie aktiv Dekonstruktion einsetzen!

### 15.2.0.6. Aufgabe 6: Approval Testing

Für das Überprüfen der durch die Builder generierten ASTs mit der Zielvorstellung ("Orakel") muss man das Orakel mühsam von Hand durch das verschachtelte Erzeugen von Objekten aufbauen. Beispielsweise wäre für die einfache Query `artist == "Beatles" and year == 1965` der erwartete AST bereits recht umständlich formuliert:

```
1 var expr =
2 new Expr.And(
3 new Expr.Comparison("artist", CompOp.EQ, new Value.Str("Beatles")),
4 new Expr.Comparison("year", CompOp.EQ, new Value.Num(1965)));
```

Hier bietet sich Approval Testing an - konvertieren Sie die von Ihren Buildern erzeugten Bäume mit Hilfe des vorimplementierten Printers `filter.ast.printer.AstPrinter` in einen String (Methode `String`



`toString(Expr expr))` und nutzen Sie **Approval Testing** für die Definition des Orakels. Setzen Sie dazu die Bibliothek `ApprovalTests.Java` ein.

Schreiben Sie sich verschiedene Approval Tests mit repräsentativen und unterschiedlich komplexen Query-Ausdrücken, und vergleichen Sie auch die beiden Builder-Varianten miteinander. Nutzen Sie die leere Klasse `filter.ast.ApprovalTest`.

#### 15.2.0.7. Aufgabe 7: Property-based Testing

Formulieren Sie verschiedene Eigenschaften für die AST-Builder.

Beispielsweise sollte ein "Roundtrip" für jeweils beide Builder (individuell, aber auch gegenseitig verschränkt) möglich sein: `Query -> Parsing -> AST -> Pretty-Print -> Parsing -> AST`. Sie könnten mit einer festen, selbstdefinierten Query starten oder als Parameter für einen Property-Test mit `jqwik` die in der Klasse `filter.ast.RoundtripPropertiesTest` vordefinierten Arbitraries nutzen: `@Property boolean foo(@ForAll("simpleQueries")String query){...}`.

Treffen Sie zusätzlich Annahmen über Expressions und Values, beispielsweise über die logischen Eigenschaften der `AND`-Verknüpfung.

Implementieren Sie diese Property Tests in der Klasse `filter.ast.RoundtripPropertiesTest`.

### 15.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem im ILIAS: bis **22. Juni, 08:00 Uhr**
- Vorstellung im Praktikum: 22./24. Juni

# 16. Blatt 07: Generics, Sealed Types, Stream-API, Logging

## 16.1. Zusammenfassung

Sie modellieren auf diesem Blatt einen kleinen Zoo in Java.

Zunächst steht das Erstellen einer **Typhierarchie** mit *sealed types / records* und der Einsatz von **Generics** im Vordergrund. Anschließend erweitern Sie dieses Modell um:

- sinnvolle **Zustandsänderungen** (Tiere aufnehmen, umsetzen, abgeben),
- **Logging** der ausgeführten Aktionen,
- sowie den Einsatz der **Stream-API**.

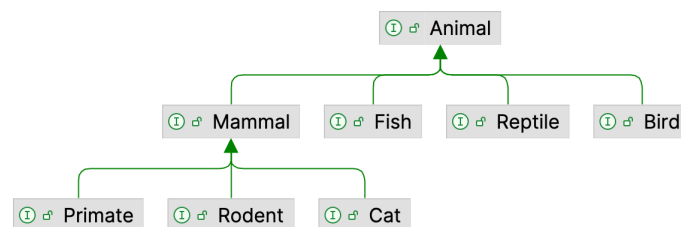
Ziel ist es, dass Sie die im Kurs behandelten Konzepte **eigenständig auf ein kleines, aber zusammenhängendes Beispiel anwenden**.

## 16.2. Aufgaben

### 16.2.1. Aufgabe 1: Zoo

#### 16.2.1.1. Aufgabe 1.1: Tier-Hierarchie modellieren mit nicht-generischen Klassen

Modellieren Sie die im Diagramm gezeigte stark vereinfachte Hierarchie von Tierarten:



Nutzen Sie dabei *sealed interfaces*.

Erstellen Sie für jedes dieser Interfaces jeweils zwei Record-Klassen, die konkrete Mitglieder der jeweiligen Tier-Klasse bzw. -Familie<sup>1</sup> repräsentieren und die jeweils das Interface implementieren. Beispielsweise könnte eine Forelle eine Art in der Klasse der Fische sein, d.h. die Java-Klasse `Trout` könnte das Interface `Fish` implementieren.

Geben Sie den Tieren jeweils mindestens einen Namen (z.B. `String name`) als Attribut.

#### Important

Nutzen Sie `permits` in Ihren `sealed`-Deklarationen so, dass nur die von Ihnen gewünschten Implementierungen erlaubt sind.

#### Tip

Da wir hier keine Java-Module verwenden, müssen alle Klassen und Interfaces, die in einer `sealed`-Hierarchie über `permits` verbunden sind, im gleichen Package liegen, z.B. `zoo.animal`. Unterpakete wie `zoo.animal.birds` sind deshalb für `sealed` hier nicht möglich.

#### 16.2.1.2. Aufgabe 1.2: Generische Gehege definieren als generische Klassen

Definieren Sie zur Repräsentation eines Geheges eine generische Klasse `Enclosure` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Gehege mit von `Animal` abgeleiteten Typen gebildet werden können.

Anforderungen:

- Jedes Gehege hat:
  - einen eindeutigen Namen (`String name`), sowie
  - eine geeignete Sammlung seiner Bewohner.
- Kein Tier kann in einem Gehege mehrfach vorkommen. Lösen Sie dies durch die Auswahl einer geeigneten inneren Datenstruktur und begründen Sie Ihre Wahl kurz.
- Implementieren Sie Methoden wie:
  - `boolean add(T animal)`
  - `boolean remove(T animal)`
  - `List<T> getInhabitants()`

#### 16.2.1.3. Aufgabe 1.3: Spezialisierte Gehege

Definieren Sie spezialisierte Gehege, die die Typhierarchie widerspiegeln:

- ein `Aquarium`, welches von `Fish` abgeleitete Tiere beherbergen kann,

<sup>1</sup>Ok, wir machen hier Informatik. Vermutlich ist die Biologie nicht ganz korrekt ;-)

- ein `Terrarium` für von `Reptile` abgeleitete Tiere,
- ein `MammalHouse` für Säugetiere (`Mammal`), sowie
- ein `CatHouse`, welches nur Objekte einer konkreten vom Interface `Cat` abgeleiteten Klasse zulässt.

#### 16.2.1.4. Aufgabe 1.4: Zoo-Verwaltung

Implementieren Sie eine Klasse `Zoo`, welche eine Liste mit Tier-Gehegen verwaltet.

Implementieren Sie folgende Methoden:

1. `addEnclosure` zum Hinzufügen eines neuen Tier-Geheges
2. `getEnclosures` liefert eine Liste aller Gehege im Zoo zurück
3. `findEnclosureByName` gibt das erste gefundene Gehege mit diesem Namen aus dem Zoo zurück (oder `null`, wenn nicht gefunden)
4. `getAllAnimals` gibt eine Liste aller Tiere in allen Gehegen zurück
5. `getAllMammals` gibt eine Liste aller Tiere in allen Gehegen zurück, deren Typ eine Subclass-Beziehung zu `Mammal` hat
6. `getAnimalsByPredicate` zum Zurückgeben aller Tiere in den Gehegen, die dem übergebenen Prädikat `Predicate<Animal>` genügen
7. `countAnimalsByType`, die für jeden Tiertyp zählt, wieviele Objekte dieser Art im Zoo sind
8. `getOvercrowdedEnclosures` gibt eine Liste aller Gehege zurück, die mehr Tiere enthalten, als der Parameter angibt
9. `String summary()` erstellt eine kurze textuelle Zusammenfassung des aktuellen Zoo-Zustands erzeugt, z.B.:

Zoo mit 3 Gehegen und 12 Tieren: 5 Mammals, 4 Birds, 3 Fish

Nutzen Sie dabei die **Stream-API**, z.B. zum Durchlaufen aller Gehege und Tiere. Nutzen Sie hier geeignet `filter`, `map`, `flatMap`, `toList` und/oder `collect` mit `Collectors.groupingBy`, `Collectors.toList`, `Collectors.joining`, `Collectors.counting` o.ä.

#### Important

Wählen Sie die Parameter und Rückgabetypen geeignet und begründen Sie Ihre Wahl kurz.

#### Tip

Wenn Sie bereits `Optional<T>` kennen, können Sie bei `findEnclosureByName` gern damit arbeiten. Ansonsten geben Sie beim Nichtfinden des gesuchten Geheges zunächst klassisch `null` zurück.

**Tip**

Zu jeder Klasse können Sie sich das passende `Class<T>`-Objekt geben lassen: `MyClass.class`. Darauf stehen Ihnen dann Operationen wie `boolean instanceof(Object obj)` oder `T cast(Object obj)` zur Verfügung.

### 16.2.2. Aufgabe 2: Logging

Fügen Sie nun **Logging** mit `java.util.logging` hinzu, um alle Zoo-Aktionen von außen nachvollziehbar zu machen.

Legen Sie dazu in `Zoo` einen Logger an und loggen Sie in jeder **public Zoo**-Methode:

- auf Level **INFO** den Start/Aufruf der Methode mit den relevanten Parametern,
- auf Level **FINE** eine kurze Zustandszusammenfassung nach erfolgreicher Ausführung einer Methoden (z.B. Anzahl Tiere im Zoo),
- auf Level **WARNING**, wenn ein angefordertes Tier oder Gehege nicht gefunden wird, und
- auf Level **SEVERE** bei schwerwiegenden Inkonsistenzen.

Demonstrieren Sie in einer Demo-Main, wie Sie das Log-Level für Ihren Logger gezielt umschalten können.

### 16.2.3. Aufgabe 3: Reflektion

Beantworten Sie die folgenden Fragen kurz und prägnant (Stichpunkte sind ausreichend). Geben Sie Ihre Antworten zusammen mit Ihrem Code in Ihrem Repo ab (z.B. im `README.md`).

#### 1. Generics

- Wo helfen Ihnen die Generics im Zoo-Szenario, Fehler bereits zur Compile-Zeit zu vermeiden?
- Nennen Sie ein Beispiel aus Ihrer Implementierung, bei dem falsche Tier-Gehege-Kombinationen durch den Typchecker verhindert werden.

#### 2. Logging

- Warum ist systematisches Logging mit einem Logger und Log-Leveln für ein Zoo-Management-System sinnvoller als Ausgaben mit `IO.println`?
- In welchen Situationen würden Sie in Ihrem System die Log-Level **INFO**, **WARNING** und ggf. **SEVERE** verwenden?

#### 3. Streams

- Wo haben Ihnen Streams im Vergleich zu klassischen Schleifen beim Formulieren Ihrer Abfragen geholfen? Wo wurde es eher unübersichtlich?

## 16.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung

- Abgabe Post Mortem [im ILIAS](#): bis **29. Juni, 08:00 Uhr**
- Vorstellung im Praktikum: 29. Juni / 01. Juli

## 17. Blatt 08: Command, Optional, Fehlermodell

### 17.1. Zusammenfassung

Betrachten Sie Ihre Zoo-Modellierung vom letzten Blatt: Sie haben dort eine Typhierarchie für Tiere modelliert mit der Basisklasse `Animal`, verschiedene generische Klassen zur Repräsentation von Gehegen erstellt und einen `Zoo`, der letztlich eine Liste von Gehegen unterschiedlicher Arten enthält und der Hilfsmethoden anbot, um Informationen abzurufen.

Auf diesem Blatt ergänzen wir dieses Zoo-Modell. Wir nutzen gezielt `Optional<T>` für Suchanfragen (wo es möglicherweise keinen Treffer gibt), und wir bauen das Command-Pattern ein, um auf Gehegen operieren zu können und die Aktionen wieder rückgängig machen zu können. Hier nutzen wir zusätzlich `Result<E, R>`, um potentielle Fehler über den Rückgabetypp weitergeben zu können und die Zuständigkeiten in der SW-Architektur zur stärken.

**PECS: Producer Extends, Consumer Super**

### 17.2. Aufgaben

#### 17.2.1. Aufgabe 1: Einführen von `Optional<T>`

Implementieren Sie eine Methode `findAnimalByName(String animalName)` in `Enclosure<T>`. Diese Methode soll im Gehege nach einem Tier mit dem übergebenen Namen suchen und das erste gefundene Tier (oder keines, wenn es kein Tier mit dem Namen gibt) zurückliefern. Nutzen Sie hier gezielt `Optional<...>` für den Rückgabetypp. Was sollten Sie hier als Typargument nutzen und warum? (Hinweis: In `Enclosure<T>` ist `T` bereits der Typ der gehaltenen Tiere.)

Implementieren Sie nun eine Methode `findAnimalByName(String animalName)` in `Zoo`. Diese Methode soll in allen Gehegen im Zoo nach dem Tier suchen und das erste gefundene Tier mit diesem Namen zurückliefern (oder keines, wenn es kein Tier mit dem Namen gibt). Setzen Sie hier die Stream-API ein und nutzen Sie dabei die `findAnimalByName` von `Enclosure<T>`. Auch hier soll der Rückgabetypp wieder `Optional<...>` sein. Begründen Sie Ihre Wahl des Typarguments für diesen konkreten Fall.

#### 17.2.2. 2. Command-Pattern für Gehege-Aktionen

In dieser Aufgabe führen Sie das **Command-Pattern** für typische Gehege-Aktionen ein.

Wir wollen in der Lage sein, das Hinzufügen und Entnehmen von Tieren aus ihren Gehegen modellieren zu können. Dabei wollen wir die Aktion jeweils rückgängig machen können und auch wiederholen können (Undo/Redo). Ein Kommando-Objekt kapselt ein Tier und die auszuführende Aktion; das Ziel-Gehege wird beim Ausführen des Kommandos als Argument übergeben. Über die generischen Typ-Parameter soll eine sichere Verwendung über den Compiler geprüft werden - beispielsweise sollten keine Fische über ein Kommando in ein Säugetiergehege gesetzt werden können.

Die Verwendung könnte ungefähr so aussehen (je nach Ihren Tier-Typen):

```

1 // Gehege
2 Enclosure<Mammal> mammalHouse = new MammalHouse<>("Säugetier-Haus");
3 CatHouse catHouse = new CatHouse("Katzen-Haus");
4
5 // Kommando-Manager für die Gehege
6 CommandManager<Enclosure<Mammal>> mammalManager = new CommandManager<>();
7 CommandManager<Enclosure<Lion>> catManager = new CommandManager<>();
8
9 // Typ-sichere Commands: hier nur Mammal bzw. Lion erlaubt
10 AddAnimalCommand<Lion> mieze = new AddAnimalCommand<>(new Lion("Mieze"));
11 AddAnimalCommand<Gibbon> kiki = new AddAnimalCommand<>(new Gibbon("Kiki"));
12 RemoveAnimalCommand<Lion> leon = new RemoveAnimalCommand<>(new Lion("Leon"));
13
14 mammalManager.executeCommand(mieze, mammalHouse);
15 mammalManager.executeCommand(kiki, mammalHouse);
16 mammalManager.executeCommand(leon, mammalHouse);
17
18 // Spezielles Katzengelände
19 AddAnimalCommand<Lion> felix = new AddAnimalCommand<>(new Lion("Felix"));
20 catManager.executeCommand(felix, catHouse);
21
22 // Undo/Redo
23 mammalManager.undo(mammalHouse);
24 mammalManager.redo(mammalHouse);
25
26 // Das folgende executeCommand() sollte vom Compiler zurückgewiesen werden!
27 AddAnimalCommand<Shark> nemo = new AddAnimalCommand<>(new Shark("Nemo"));
28 mammalManager.executeCommand(nemo, mammalHouse); // NOPE!!!

```

*Hinweis:* Bitte beachten Sie in diesem Beispiel gilt: `Lion <: Cat <: Mammal <: Animal` und `Shark <: Fish <: Animal`. Außerdem ist `CatHouse` eine für `Lion` spezialisierte `Enclosure`.

### 17.2.2.1. Aufgabe 2.1: Command-Interface

Definieren Sie ein generisches Interface `Command` wie in der Vorlesung gezeigt. Sie brauchen die beiden Aktionen `execute` und `undo`, die (abweichend von der Modellierung in der Vorlesung) als Parameter das Ziel der Aktion übergeben bekommen: Das wird später ein Tier-Gehege sein, aber lassen Sie dies hier noch abstrakt. Zusätzlich sollte ein Kommando eine textuelle Beschreibung haben, die über die Hilfsmethode `description` erreichbar ist.

Der Typ-Parameter für das Interface spielt also die Rolle des Typs des Tier-Geheges, welches für ein konkretes Kommando genutzt werden soll. Lassen Sie den Typ-Parameter hier noch allgemein ("`T`"), später in den ableitenden Klassen für die konkreten Kommandos wird das dann eingeschränkt.



### 17.2.2.2. Aufgabe 2.2: Konkrete Commands auf den Gehegen

Leiten Sie nun das generische Interface `Command` ab und definieren Sie zwei konkrete Commands, die auf einem `Enclosure<T>` arbeiten:

#### 1. `AddAnimalCommand`

- Ziel: Ein neues Tier in ein bestimmtes Gehege aufnehmen
- Attribute (Vorschlag):
  - ein konkretes Tier-Objekt
- `execute(Enclosure)`:
  - Fügt das Tier der Bewohner-Liste des übergebenen Geheges hinzu
  - Log-Message, wenn das Tier nicht hinzugefügt werden konnte
- `undo(Enclosure)`:
  - Entfernt das hinzugefügte Tier wieder aus dem Gehege
  - Log-Message, wenn noch gar kein Undo möglich ist, d.h. wenn das Kommando noch gar nicht ausgeführt wurde
  - Log-Message, wenn das Tier nicht entfernt werden konnte

#### 2. `RemoveAnimalCommand`

- Ziel: Ein Tier aus einem bestimmten Gehege entfernen
- Attribute (Vorschlag):
  - ein konkretes Tier-Objekt
- `execute(Enclosure)`:
  - Entfernt das hinzugefügte Tier aus dem Gehege
  - Log-Message, wenn das Tier nicht entfernt werden konnte
- `undo(Enclosure)`:
  - Fügt das Tier wieder der Bewohner-Liste des übergebenen Geheges hinzu
  - Log-Message, wenn noch gar kein Undo möglich ist, d.h. wenn das Kommando noch gar nicht ausgeführt wurde
  - Log-Message, wenn das Tier nicht hinzugefügt werden konnte

Beachten Sie, dass beide Kommandos einen Typ-Parameter brauchen, der den Typ des zu bearbeitenden Tieres beschreibt. Sie müssen hier geeignet mit Wildcards und Bounds arbeiten, damit die Kommandos nur für die Tiere der Tier-Hierarchie erlaubt sind. Beachten Sie auch, dass Ihre Commands von `Command` ableiten, und dass dort der Typ-Parameter ein *Gehege* beschreibt. Erinnern Sie sich an die *PECS*-Diskussion ...

#### Tip

Überlegen Sie sich:

1. Welchen Bound brauchen Sie auf dem Typ-Parameter `T` der Kommandos: Welche Tiere sind erlaubt?
2. Wie sieht der dazu passende Typ des Ziel-Geheges aus, damit Sie dort Tiere vom Typ `T` sicher hinzufügen können?

### 17.2.2.3. Aufgabe 2.3: Command-Manager mit Undo/Redo

Implementieren Sie nun die generische Klasse `CommandManager`. Der Typ-Parameter beschreibt einen Gehege-Typ. Nutzen Sie hier keine Wildcards - diese stecken bereits in den `Commands`.

Der `CommandManager` soll folgende Methoden haben:

- `executeCommand`:
  - Bekommt ein konkretes Kommando `command` und ein Ziel-Gehege `target` übergeben als Parameter
  - Führt `command.execute(target)` aus
  - Bei Erfolg:
    - \* Legt das Kommando auf den `undoStack`
    - \* Leert den `redoStack`
- `undo`:
  - Bekommt ein Ziel-Gehege `target` als Parameter übergeben
  - Wenn der `undoStack` nicht leer ist:
    - \* Nimmt das oberste Kommando vom `undoStack`
    - \* Ruft `undo(target)` darauf auf
    - \* Legt das Kommando auf den `redoStack`
- `redo`:
  - Analog zu `undo`, aber in umgekehrter Richtung (vom `redoStack` auf den `undoStack`)

Loggen Sie jeweils den Start der Methode, das Nichtausführen oder Scheitern einer Operation und am Ende der Methode den Zustand des Geheges `target`. Nutzen Sie jeweils geeignete Log-Level passend zur Bedeutung der Meldung und begründen Sie Ihre Wahl.

Demonstrieren Sie die Verwendung des `CommandManager` (beispielsweise in einem JUnit-Test), um eine Sequenz von Zoo-Aktionen auszuführen und mit Undo/Redo zu experimentieren.

### 17.2.3. Aufgabe 3: Result<E,R>

Definieren Sie sich analog zur Vorlesung einen Typ `Result<E,R>`<sup>1</sup> und legen Sie sich ein Enum an für typische Fehlersituation im Zoo-Modell.

Passen Sie `Command` und die beiden abgeleiteten Klassen `AddAnimalCommand` und `RemoveAnimalCommand` an: Die Methoden `execute` und `undo` sollten nun ein `Result<E,R>` zurückliefern (passen Sie die Typ-Variablen entsprechend an).

Entfernen Sie das Logging in `AddAnimalCommand` und `RemoveAnimalCommand` und passen Sie `CommandManager` an: Beim Aufruf der Methoden `executeCommand`, `undo` und `redo` soll das vom Kommando zurückgelieferte `Result` per Pattern Matching ausgewertet werden und passend reagiert werden. Fügen Sie das Logging hier entsprechend ein und beachten Sie dabei den Typ der zurückgelieferten Fehler.

<sup>1</sup>Sie können die in der Vorlesung erarbeitete Fassung direkt übernehmen

Ziel ist, dass die Commands nur noch fachlich über Erfolg oder Fehlerzustand entscheiden (`Result<E, R>`), und dass das Logging zentral im `CommandManager` erfolgt.

### 17.3. Bearbeitung und Abgabe

- Bearbeitung: Einzelbearbeitung
- Abgabe Post Mortem im ILIAS: bis **06. Juli, 08:00 Uhr**
- Vorstellung im Praktikum: 06./08. Juli

## **Teil IV.**

# **Organisatorisches**

## 18. Prüfungsvorbereitung

### TL;DR

#### Durchführung: Präsenz

Die Klausur wird dieses Semester elektronisch stattfinden. Dazu werden wir den E-Assessment-ILIAS der HSBI nutzen.

Die Klausur wird in den Räumen der HSBI am Campus Minden unter Aufsicht durchgeführt. Hier werden Ihnen Rechner für den Zugang zum Prüfungs-ILIAS zur Verfügung gestellt, Sie benötigen nur Ihre HSBI-Zugangsdaten (Username, Passwort), einen Studierendenausweis und Personalausweis sowie Ihren DIN-A4-Spickzettel.

#### Ablauf der Klausur

Die Prüfung (das ILIAS-Objekt) selbst schalte ich zum Start der Prüfung online. Sie loggen sich vor Ort in den ILIAS mit Ihren HSBI-Zugangsdaten ein und können die Prüfung nach dem gemeinsamen Start beginnen. Nach Ablauf der Bearbeitungszeit beenden Sie die Prüfung und loggen sich wieder aus. Parallel zur Prüfung werde ich in einer Zoom-Sitzung zur Verfügung stehen, in der Sie Fragen stellen können.

#### Hilfsmittel und Themen

Als Hilfsmittel ist ein Spickzettel (DIN A4, beidseitig beschrieben) zugelassen.

Es wird keines der behandelten Themen ausgeschlossen, allerdings eignen sich manche Themen etwas besser für Klausurfragen als andere ;-)

### Videos

#### 1. Fragetypen-Demo

- **Video** zur Fragetypen-Demo: [\[YT\]](#), [\[HSBI\]](#)
- [Fragetypen-Demo im ILIAS](#)

## 18.1. Elektronische Klausur: Termin, Materialien

### 18.1.1. Termin

Die schriftliche Prüfung erfolgt durch eine Klausur, die als digitale Prüfung auf einem Prüfungs-ILIAS durchgeführt wird.

Die Klausur wird in Präsenz in den Rechnerpools am Campus Minden durchgeführt.

Es wird in beiden Prüfungszeiträumen ein Termin angeboten. Die Termine werden vom Prüfungsamt bekannt gegeben.

Dauer: siehe Aushang bzw. Mail.

### 18.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig)

Sie dürfen **einen** Spickzettel im **DIN-A4**-Format benutzen, der beidseitig beschrieben sein kann.

Ich möchte Sie hier noch einmal ermuntern, diesen Zettel tatsächlich manuell zu erstellen (also ganz traditionell zu **schreiben**), da bereits der Schreibvorgang einen gewissen Lerneffekt bewirkt! (Wenn Sie sich bewusst dafür entscheiden, können Sie den Spickzettel natürlich auch ausdrucken.)

### 18.1.3. Einsicht

Die Prüfungseinsicht findet möglichst zeitnah und online statt. Details werden per Mail bekanntgegeben.

## 18.2. Technische Vorbereitungen

Sie benötigen am Prüfungstag:

1. **HSBI-Zugangsdaten:** Username, Passwort

Bei der Durchführung der Prüfung am Campus Minden wird Ihnen ein Rechner zur Verfügung gestellt. Dort läuft voraussichtlich ein Browser im Kiosk-Mode, wo Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie Ihre HSBI-Zugangsdaten, mit denen Sie sich auch im "normalen" ILIAS anmelden.

2. **Studierendenausweis** und Personalausweis

An der Prüfung dürfen nur Personen teilnehmen, die dafür im LSF/CAT angemeldet sind. Es findet eine entsprechende Kontrolle statt. Halten Sie dazu Ihren Studierendenausweis und Personalausweis bereit.

## 18.3. Bearbeitung des E-Assessment

1. Lesen Sie sich die Hinweise auf der Startseite durch
2. Bearbeiten Sie die Aufgaben in **einem einzigen** Browser-Tab

**Öffnen Sie die Aufgaben NICHT in parallelen Tabs!** Es kann sonst zu Fehlfunktionen von ILIAS kommen.

Bewegen Sie sich **nicht per Browser-Navigation** ("vor", "zurück" im Browser) durch die Aufgaben, sondern nutzen Sie dafür die **Buttons "nächste Frage", "Weiter" oder "Zurück" in ILIAS!**

3. Hinweis zur Anzeige der restlichen Bearbeitungsdauer

Wenn Sie den Browser bzw. das Tab mit der Prüfung im Laufe der Prüfung verlassen, wird Ihnen bei der Rückkehr unter Umständen eine falsche restliche Bearbeitungsdauer angezeigt. Sie können die Anzeige korrigieren/aktualisieren, indem Sie einfach zu einer vorigen oder nächsten Aufgabe navigieren.

Hinweis: Die restliche Bearbeitungsdauer wird im Test nur dann angezeigt, wenn diese Funktion von den Prüfenden aktiviert wurde.

4. Parallel zum E-Assessment läuft eine Zoom-Session, dort können Sie Fragen stellen

## 18.4. Fragetypen-Demo

In Ihrem ILIAS-Kurs finden Sie eine [Fragetypen-Demo](#) mit den wichtigsten Fragetypen. Machen Sie sich mit der Mechanik der Fragetypen vertraut und schauen Sie sich die Kommentare bei den einzelnen Aufgaben an. Sie können die Demo bei Bedarf beliebig oft wiederholen.

## 18.5. Hinweise zu den Inhalten

- Klausurrelevant: Vorlesung und Praktikum
- Für Verständnis u.U. hilfreich: Studium der vertiefenden Literaturangaben
- **Fragen:**
  - Schauen Sie sich die Challenges an ...
  - Schauen Sie sich die Praktikumsaufgaben an ...
  - Überlegen Sie sich, was zu einem Themengebiet im Rahmen einer Prüfung möglich ist und (wie) gefragt werden könnte ...

### Können vor Kennen

## 18.6. Beispiele für mögliche Fragen

### 18.6.1. Vererbung und Polymorphie

Betrachten Sie den folgenden Java-Code:

```
1 public class Person {
2 public String getInfo(Person p) { return "Person"; }
3 }
4
5 public class Studi extends Person {
6 public String getInfo(Studi s) { return "Studi"; }
7
8 public static void main(String[] args) {
9 Studi s = new Studi(); Person p = s;
10 System.out.println(s.getInfo(p));
11 System.out.println(s.getInfo(s));
12 }
13 }
```

Geben Sie alle Ausgaben, die das obige Programm produziert, an.

Begründen Sie Ihre Antwort kurz und stichhaltig (für *jede* Ausgabe!). Was geschieht, bzw. wieso kommt es zu der jeweiligen Ausgabe?

### 18.6.2. Reguläre Ausdrücke

Auf welche Strings passt (im Sinne von “match”) der folgende reguläre Ausdruck: `\s*([a-zA-Z0-9_.\-]+)\s*=\s*(-?\d+\.?\d*)\s;?\s*`

### 18.6.3. Versionieren mit Git

- Erklären Sie, wie man mit Git die Unterschiede zwischen zwei bestimmten Versionsständen einer Datei herausfindet.
- Was ist der Unterschied zwischen einer Workingcopy und einem Repository?
- Worin liegt der Unterschied zwischen folgenden Arbeitsschritten:
  - a. Editieren von Datei `A.txt`
  - b. `git add A.txt`
  - c. Editieren von Datei `A.txt`
  - d. `git commit`

versus

- a. Editieren von Datei `A.txt`
  - b. Editieren von Datei `A.txt`
  - c. `git add A.txt`
  - d. `git commit`
- Was würde `git diff` jeweils nach Schritt 2 anzeigen?

### 18.6.4. Generics

Was kommt hier raus? Und warum?

```

1 public class X {
2 void methode(int a) {
3 System.out.println("non-generic");
4 }
5 <T> void methode(T a) {
6 System.out.println("generisch");
7 }
8 public static void main(String[] args) {
9 X x = new X();
10 x.methode(3);
11 x.methode(new Integer(4));
12 x.methode("huhu");

```



```
13 }
14 }
```

### 18.6.5. Logging

Erklären Sie den Code. Was passiert?

```
1 class MyFormatter extends SimpleFormatter {
2 public String format(LogRecord record) {
3 return super.format(record) + "---- FAKE ----\n";
4 }
5 }
6 public class MoreLogging {
7 public static void main(String[] argv) {
8 Logger l = Logger.getLogger("MoreLogging");
9 l.setLevel(Level.FINE);
10
11 ConsoleHandler myHandler = new ConsoleHandler();
12 myHandler.setFormatter(new MyFormatter());
13 myHandler.setLevel(Level.FINER);
14 l.addHandler(myHandler);
15
16 l.info("Hello World :-)");
17 l.fine("fine");
18 l.finer("finer");
19 l.finest("finest");
20 }
21 }
```

### 18.6.6. Methodenreferenzen

Was bedeutet der folgende Code?

```
1 List<String> str = Arrays.asList("a", "b", "A", "B");
2 str.sort(String::compareToIgnoreCase);
```

#### Quellen

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.

Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. <https://git-scm.com/book/en/v2>.

Fowler, M. 2011. *Refactoring*. Addison-Wesley.

Gamma, E., R. Helm, R. E. Johnson, und J. Vlissides. 2011. *Design Patterns*. Addison-Wesley.

Kleuker, S. 2026. *Qualitätssicherung durch Softwaretests*. Springer Vieweg Wiesbaden. <https://doi.org/10.1>

007/978-3-658-50232-4.

Martin, R. 2009. *Clean Code*. Mitp.

Nystrom, R. 2014. *Game Programming Patterns*. Genever Benning. <https://github.com/munificent/game-programming-patterns>.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. <https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/>.

Passig, K., und J. Jander. 2013. *Weniger schlecht programmieren*. O'Reilly.

Winters, T., T. Manshreck, und H. Wright. 2020. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

**Exceptions:**

- [“356: Refactoring”](#) by Andreas Bogk on Lutz Donnerhacke: “Fachbegriffe der Informatik”
- *“Refactoring”*: (Fowler 2011, p. 53)
- [“A Note About Git Commit Messages”](#) by Tim Pope on tbagger.com
- *“Any fool...”*: (Fowler 2011, p. 15)
- *“Three strikes...”*: (Fowler 2011, p. 58)

Last modified: 9c868b2 2026-07-09 tdd: rework screencasts